University of Liège

Faculty of Applied Sciences
Department of Electrical Engineering & Computer Science

PhD dissertation

UNDERSTANDING RANDOM FORESTS

FROM THEORY TO PRACTICE

by Gilles Louppe



Advisor: Prof. Pierre Geurts
July 2014

arXiv:1407.7502v3 [stat.ML] 3 Jun 2015

Universidade de Liège

Faculdade de Ciências Aplicadas Departamento de
Engenharia Elétrica e Ciência da Computação

Dissertação de
doutorado

UNDERSTANDINGRANDOMFORESTS

da teoria à prática

por Gilles Louppe



Orientador: Prof. Pierre Geurts
julho de 2014

arXiv:1407.7502v3 [stat.ML] 3 Jun 2015

JURY MEMBERS

Louis Wehenkel, Professor at the Université de Liège (President);

Pierre Geurts, Professor at the Université de Liège (Advisor);

Bernard Boigelot, Professor at the Université de Liège;

Renaud Detry, Postdoctoral Researcher at the Université de Liège;

Gianluca Bontempi, Professor at the Université Libre de Bruxelles;

Gérard Biau, Professor at the Université Pierre et Marie Curie (France);

MEMBROS DO JÚRI

Louis Wehenkel, Professor da Universidade de Liège (Presidente); Pierre Geurts, Professor da Universidade de Liège (Orientador); Bernard Boigelot, professor da Universidade de Liège; Renaud Detry, pesquisador de pós-doutorado na Universidade de Liège; Gianluca Bontempi, professor da Universidade Livre de Bruxelas; GérardBiau, professor da Université Pierre et Marie Curie (França);

ABSTRACT

Data analysis and machine learning have become an integrative part of the modern scientific methodology, offering automated procedures for the prediction of a phenomenon based on past observations, unraveling underlying patterns in data and providing insights about the problem. Yet, caution should avoid using machine learning as a black-box tool, but rather consider it as a methodology, with a rational thought process that is entirely dependent on the problem under study. In particular, the use of algorithms should ideally require a reasonable understanding of their mechanisms, properties and limitations, in order to better apprehend and interpret their results.

Accordingly, the goal of this thesis is to provide an in-depth analysis of random forests, consistently calling into question each and every part of the algorithm, in order to shed new light on its learning capabilities, inner workings and interpretability. The first part of this work studies the induction of decision trees and the construction of ensembles of randomized trees, motivating their design and purpose whenever possible. Our contributions follow with an original complexity analysis of random forests, showing their good computational performance and scalability, along with an in-depth discussion of their implementation details, as contributed within Scikit-Learn.

In the second part of this work, we analyze and discuss the interpretability of random forests in the eyes of variable importance measures. The core of our contributions rests in the theoretical characterization of the Mean Decrease of Impurity variable importance measure, from which we prove and derive some of its properties in the case of multiway totally randomized trees and in asymptotic conditions. In consequence of this work, our analysis demonstrates that variable importances as computed from non-totally randomized trees (e.g., standard Random Forest) suffer from a combination of defects, due to masking effects, misestimations of node impurity or due to the binary structure of decision trees.

Finally, the last part of this dissertation addresses limitations of random forests in the context of large datasets. Through extensive experiments, we show that subsampling both samples and features simultaneously provides on par performance while lowering at the same time the memory requirements. Overall this paradigm highlights an intriguing practical fact: there is often no need to build single models over immensely large datasets. Good performance can often be achieved by building models on (very) small random parts of the data and then combining them all in an ensemble, thereby avoiding all practical burdens of making large data fit into memory.

ABSTRACT

A análise de dados e a aprendizagem automática tornaram-se parte integrante da metodologia científica moderna, oferecendo procedimentos automatizados para a previsão de um fenómeno com base em observações passadas, desvendando padrões subjacentes nos dados e fornecendo insights sobre o problema. No entanto, deve-se ter cautela em evitar o uso do aprendizado de máquina como uma ferramenta de caixa preta, mas si m considerá-lo como uma metodologia, com um processo de pensamento racional que é inteiramente dependente do problema em estudo. Em particular, o uso de algoritmos deveria idealmente exigir uma compreensão razoável dos seus mecanismos, propriedades e limitações, a fim de melhor apreender e interpretar os seus resultados.

Assim, o objetivo desta tese é fornecer uma análise aprofundada de florestas aleatórias, questionando consistentemente cada parte do algoritmo, a fim de lançar nova luz sobre suas capacidades de aprendizagem, funcionamento interno e interpretabilidade. A primeira parte deste trabalho estuda a indução de árvores de decisão e a construção de conjuntos de árvores aleatórias, motivando o seu desenho e finalidade sempre que possível. Nossas contribuições seguem com uma análise original da complexidade de florestas aleatórias, mostrando seu bom desempenho computacional e escalabilidade, juntamente com uma discussão aprofundada dos detalhes de sua implementação, conforme contribuição do Scikit-Learn.

Na segunda parte deste trabalho, analisamos e discutimos a interpretabilidade de florestas aleatórias à luz de medidas de importância variável. O núcleo de nossas contribuições repousa na caracterização teórica da medida de importância variável de Decréscimo Médio de Impureza, a partir da qual comprovamos e derivamos algumas de suas propriedades no caso de árvores multiway totalmente randomizadas e e m condições assintóticas. Como consequência deste trabalho, nossa análise demonstra que as importâncias das variáveis calculadas a partir de árvores não totalmente aleatórias (por exemplo, Floresta Aleatória padrão) sofrem de uma combinação de defeitos, devido a efeitos de mascaramento, estimativas incorretas de impureza dos nós ou devido à estrutura binária das árvores de decisão.

Finalmente, a última parte desta dissertação aborda as limitações das florestas aleatórias no contexto de grandes conjuntos de dados. Através de extensos experimentos, mostramos que a subamostragem de amostras e recursos fornece simultaneamente desempenho equivalente e, ao mesmo tempo, reduz os requisitos de memória. No geral, este paradigma destaca um facto prático intrigante: muitas vezes não há necessidade de construir modelos únicos sobre conjuntos de dados imensamente grandes. Muitas vezes, um bom desempenho pode ser alcançado através da construção de modelos em partes aleatórias (muito) pequenas dos dados e, em seguida, combinando-os todos em um conjunto, evitando assim todos os encargos práticos de fazer com que grandes dados caibam na memória.

ACKNOWLEDGMENTS

As the saying goes, good premises do not entail good stories. Yet, this dissertation would certainly not have come to its successful conclusion without the help, support and trust of colleagues, friends and family.

First and foremost, I would like to sincerely thank my advisor Pierre Geurts for his help, guidance and for the freedom I was granted throughout these years.

I am grateful to all members of the jury for their interest in this work and for taking the time to evaluate this dissertation.

In alphabetical order, I would also like to thank my colleagues who all contributed to create and maintain a pleasant and stimulating working environment: Antonio, Arnaud, Benjamin, Damien, Fabien, Julien, Loïc, Louis, Marie, Olivier, Raphaël, Van Anh, Vincent. Special thanks go to Antonio, Arnaud and Vincent who accepted to proofread parts of this manuscript.

I want take this opportunity to thank the Scikit-Learn team and all its contributors. This experience within the open source world really contributed to shape my vision of science and software development towards a model of rigor, pragmatism and openness. Thanks go to Gaël, Olivier, Lars, Mathieu, Andreas, Alexandre and Peter.

Special thanks go to the rowing team of the RCAE, for their friendship and good mood in all circumstances. Guys, I thank you all.

Even if I never succeeded to fully explain my research topics, I would finally like to warmly thank my dear friend Jérôme and my family for their help in moments of doubt.

Last but not least, Laura, I am forever grateful for your unconditional support and love.

ACKNOWLEDGMENTS

Como diz o ditado, boas premissas não implicam boas histórias. No entanto, esta dissertação certamente não teria chegado à sua conclusão bem-sucedida sem a ajuda, o apoio e a confiança de colegas, amigos e familiares.

Em primeiro lugar, gostaria de agradecer sinceramente ao meu orientador PierreGeurts pela ajuda, orientação e pela liberdade que me f oi concedida ao longo destes anos.

Estou grato a todos os membros do júri pelo seu interesse neste trabalho e por terem dispensado o seu tempo para avaliar esta dissertação.

Em ordem alfabética, gostaria também de agradecer aos meus colegas que contribuíram para criar e manter um ambiente de trabalho agradável e estimulante: Antonio, Arnaud, Benjamin, Damien, Fabien, Julien, Loïc, Louis, Marie, Olivier, Raphaël, Van Anh, Vincent. Agradecimentos es peciais vão para Antonio, Arnaud e Vincent que aceitaram revisar partes deste manuscrito.

Quero aproveitar esta oportunidade para agradecer à equipe do Scikit-Learn e a todos os seus colaboradores. Esta experiência no mundo do código aberto realmente contribuiu para moldar minha visão da ciência e do desenvolvimento de software em direção a um modelo de rigor, pragmatismo e abertura. Os agradecimentos vão para Gaël, Olivier, Lars, Mathieu, Andreas, Alexandre e Peter.

Um agradecimento especial à equipa de remo da RCAE, pela amizade e boa disposição em todas as circunstâncias. Pessoal, agradeço a todos.

Mesmo que eu nunca tenha conseguido explicar completamente os meus temas de pesquisa, gostaria finalmente de agradecer calorosamente ao meu querido amigo Jérôme e à minha família pela ajuda nos momentos de dúvida.

Por último, mas não menos importante, Laura, sou eternamente grato pelo seu apoio e amor incondicional.

CONTENTS

1 INTRODUCTION 1
1.1 Outline and contributions . . . 3
1.2 Publications . . . 4

I GROWING DECISION TREES 7

2 BACKGROUND 9
2.1 Learning from data . . . 9
2.2 Performance evaluation . . . 11
2.2.1 Estimating $\mathrm{Err}(\varphi_{\mathcal{L}})$ . . . 13
2.2.2 Bayes model and residual error . . . 14
2.3 Model selection . . . 16
2.3.1 Selecting the (approximately) best model . . . 16
2.3.2 Selecting and evaluating simultaneously . . . 18
2.4 Classes of learning algorithms . . . 20
2.4.1 Linear methods . . . 20
2.4.2 Support vector machines . . . 21
2.4.3 Neural networks . . . 22
2.4.4 Nearest neighbor methods . . . 24

3 CLASSIFICATION AND REGRESSION TREES 25
3.1 Introduction . . . 25
3.2 Tree structured models . . . 26
3.3 Induction of decision trees . . . 29
3.4 Assignment rules . . . 32
3.4.1 Classification . . . 32
3.4.2 Regression . . . 34
3.5 Stopping criteria . . . 35
3.6 Splitting rules . . . 37
3.6.1 Families Ω of splitting rules . . . 37
3.6.2 Goodness of split . . . 40
3.6.2.1 Classification . . . 40
3.6.2.2 Regression . . . 46
3.6.3 Finding the best binary split . . . 47
3.6.3.1 On an ordered variable . . . 48
3.6.3.2 On a categorical variable . . . 51
3.7 Multi-output decision trees . . . 52

4 RANDOM FORESTS 55
4.1 Bias-variance decomposition . . . 55
4.1.1 Regression . . . 55
4.1.2 Classification . . . 58
4.2 Ensemble methods based on randomization . . . 61
4.2.1 Randomized models . . . 61
4.2.2 Combining randomized models . . . 62

CONTENTS

1 introdução 1 1.1 Esboço e contribuições.3 1.2 Publicações.4

estou cultivando árvores de decisão 7

2 fundo 9

2.1 Aprendendo com os dados.9 2.2 Avaliação de desempenho.11 2.2.1 Estimativa $\mathrm{Err}(\varphi_{\mathcal{L}})$.13 2.2.2 Modelo Bayesiano e erro residual.14 2.3 Seleção de modelo.16 2.3.1 Selecionando o (aproximadamente) melhor modelo . . . . 16 2.3.2 Selecionando e avaliando simultaneamente . . . . 18 2.4 Classes de algoritmos de a prendizagem.20 2.4.1 Métodos lineares.20 2.4.2 Máquinas de vetores de suporte.21 2.4.3 Redes Neurais.22 2.4.4 Métodos do vizinho mais próximo.24

3 árvores de classificação e regressão 25

3.1 Introdução.25 3.2 Modelos estruturados em árvore.26 3.3 Indução de árvores de decisão.29 3.4 Regras de atribuição.32 3.4.1 Classificação.32 3.4.2 Regressão.34 3.5 Critérios de parada.35 3.6 Regras de divisão.37

3.6.1 Famílias Ω de regras de divisão.37 3.6.2 Qualidade da divisão.40 3.6.2.1 Classificação.40 3.6.2.2 Regressão.46 3.6.3 Encontrando a melhor divisão binária.47 3.6.3.1 Em uma variável ordenada . . . . . 48 3.6.3.2 Em uma variável categórica 51

3.7 Árvores de decisão com múltiplas saídas.52 4 florestas aleatórias 55 4.1 Decomposição de viés-variância.55 4.1.1 Regressão.55 4.1.2 Classificação.58 4.2 Métodos de conjunto baseados em randomização. . . . . 61 4.2.1 Modelos randomizados.61 4.2.2 Combinando modelos aleatórios 62

4.2.3 Bias-variance decomposition of an ensemble . . 64
4.2.3.1 Regression 64
4.2.3.2 Classification 68
4.3 Random Forests 69
4.3.1 Randomized induction algorithms 69
4.3.2 Illustration 74
4.4 Properties and features 76
4.4.1 Out-of-bag estimates 76
4.4.2 Variable importances 77
4.4.3 Proximity measures 77
4.4.4 Missing data 80
4.5 Consistency 81
5 COMPLEXITY ANALYSIS AND IMPLEMENTATION 87
5.1 Complexity of the induction procedure 87
5.2 Complexity of the prediction procedure 96
5.3 Implementation 98
5.3.1 Scikit-Learn 98
5.3.1.1 Data 100
5.3.1.2 Estimators 101
5.3.1.3 Predictors 102
5.3.1.4 API for random forests 103
5.3.2 Internal data structures 103
5.3.3 Builders 106
5.3.4 Splitters 107
5.3.5 Criteria 110
5.3.6 Parallelization 111
5.4 Benchmarks 112
5.4.1 Complexity on artificial data 112
5.4.2 Comparison with other implementations 115

II INTERPRETING RANDOM FORESTS 121
6 UNDERSTANDING VARIABLE IMPORTANCES 123
6.1 Variable importances 124
6.1.1 Importances in single decision trees 124
6.1.2 Importances in forests 124
6.2 Theoretical study 126
6.2.1 Background 126
6.2.2 Asymptotic analysis 127
6.3 Relevance of variables 130
6.4 Variable importances in random forest variants 133
6.4.1 Generalization to other impurity measures . . . 133
6.4.2 Pruning and random subspaces 136
6.4.3 Non-totally randomized trees 138
6.5 Illustration 139
6.6 Conclusions 143
7 FURTHER INSIGHTS FROM IMPORTANCES 145

4.2.3 Decomposição de polarização-variância de um conjunto. . 64
4.2.3.1 Regressão.64 4.2.3.2 Classificação.68 4.3 Florestas Aleatórias.69
4.3.1 Algoritmos de indução randomizados 69 4.3.2
Ilustração.74 4.4 Propriedades e recursos.76

4.4.1 Estimativas imediatas.76 4.4.2 Importâncias variáveis.77
4.4.3 Medidas de proximidade.77 4.4.4 Dados faltantes.80

4.5 Consistência.81

5 análise de complexidade e implementação 87

5.1 Complexidade do procedimento de indução. 87 5.2
Complexidade do procedimento de predição. 96 5.3
Implementação.98 5.3.1 Scikit-Learn.98 5.3.1.1 Dados.100 5.3.1.2
Estimadores.101 5.3.1.3 Preditores.102 5.3.1.4 API para florestas
aleatórias . . . . . 103 5.3.2 Estruturas de dados internas.103 5.3.3
Construtores.106 5.3.4 Divisores.107 5.3.5 Critérios.110 5.3.6
Paralelização.111 5.4 Referências.112 5.4.1 Complexidade em dados
artificiais.112 5.4.2 Comparação com outras implementações 115

ii interpretando florestas aleatórias 121

6 entendendo as importâncias variáveis 123

6.1 Importâncias das variáveis.124 6.1.1 Importâncias em árvores de
decisão únicas . . . . . 124 6.1.2 Importâncias nas florestas.124 6.2 Estudo
teórico.126 6.2.1 Antecedentes.126 6.2.2 Análise assintótica.127 6.3
Relevância das variáveis.130 6.4 Importâncias variáveis em variantes
florestais aleatórias . . . . 133 6.4.1 Generalização para outras medidas de
impureza . . . 133 6.4.2 Poda e subespaços aleatórios 136 6.4.3
Árvores não totalmente randomizadas.138 6.5 Ilustração.139 6.6
Conclusões.143 7 mais insights sobre importâncias 145

7.1 Redundant variables . . . 145
7.2 Bias in variable importances . . . 152
7.2.1 Bias due to masking effects . . . 152
7.2.2 Bias due to empirical impurity estimations . . . 153
7.2.3 Bias due to binary trees and threshold selection 159
7.3 Applications . . . 163
7.3.1 Feature selection . . . 163
7.3.2 Biomarker identification . . . 164
7.3.3 Network inference . . . 165

III SUBSAMPLING DATA 167
8 ENSEMBLES ON RANDOM PATCHES 169
8.1 Random Patches . . . 170
8.2 On Accuracy . . . 171
8.2.1 Protocol . . . 172
8.2.2 Small datasets . . . 172
8.2.3 Larger datasets . . . 176
8.2.4 Conclusions . . . 179
8.3 On Memory . . . 179
8.3.1 Sensitivity to α_s and α_f . . . 180
8.3.2 Memory reduction, without significant loss . . . 182
8.3.3 Memory reduction, with loss . . . 184
8.3.4 Conclusions . . . 186
8.4 Overall conclusions . . . 186
9 CONCLUSIONS 189

IV APPENDIX 193
A NOTATIONS 195
B REFERENCES 199

7.1 Variáveis redundantes.145 7.2 Viés em importâncias variáveis.152 7.2.1 Viés devido a efeitos de mascaramento.152 7.2.2 Viés devido a estimativas empíricas de impurezas . . . 153 7.2.3 Viés devido a árvores binárias e seleção de limites 159 7.3 Aplicações.163 7.3.1 Seleção de recursos.163 7.3.2 Identificação de biomarcadores.164 7.3.3 Inferência de rede.165

iii dados de subamostragem 167

8 conjuntos em patches aleatórios 169

8.1 Patches Aleatórios.170 8.2 Sobre Precisão.171 8.2.1 Protocolo.172 8.2.2 Pequenos conjuntos de dados.172 8.2.3 Conjuntos de dados maiores.176 8.2.4 Conclusões.179 8.3 Na memória.179 8.3.1 Sensibilidade a α_s e α_f .180 8.3.2 Redução de memória, sem perda significativa . . 182 8.3.3 Redução de memória, com perda.184 8.3.4 Conclusões.186 8.4 Conclusões gerais.186

9 conclusões 189

iv apêndice 193

anotações 195 b referências 199

1

INTRODUCTION

In various fields of science, technology and humanities, as in biology, meteorology, medicine or finance to cite a few, experts aim at predicting a phenomenon based on past observations or measurements. For instance, meteorologists try to forecast the weather for the next days from the climatic conditions of the previous days. In medicine, practitioners collect measurements and information such as blood pressure, age or history for diagnosing the condition of incoming patients. Similarly, in chemistry, compounds are analyzed using mass spectrometry measurements in order to determine whether they contain a given type of molecules or atoms. In all of these cases, the goal is the prediction of a response variable based on a set of observed predictor variables.

For centuries, scientists have addressed such problems by deriving theoretical frameworks from first principles or have accumulated knowledge in order to model, analyze and understand the phenomenon under study. For example, practitioners know from past experience that elderly heart attack patients with low blood pressure are generally high risk. Similarly, meteorologists know from elementary climate models that one hot, high pollution day is likely to be followed by another. For an increasing number of problems however, standard approaches start showing their limits. For example, identifying the genetic risk factors for heart disease, where knowledge is still very sparse, is nearly impractical for the cognitive abilities of humans given the high complexity and intricacy of interactions that exist between genes. Likewise, for very fine-grained meteorological forecasts, a large number of variables need to be taken into account, which quickly goes beyond the capabilities of experts to put them all into a system of equations. To break this cognitive barrier and further advance science, machines of increasing speed and capacity have been built and designed since the mid-twentieth century to assist humans in their calculations. Amazingly however, alongside this progress in terms of hardware, developments in theoretical computer science, artificial intelligence and statistics have made machines to become more than calculators. Recent advances have made them experts of their own kind, capable to learn from data and to uncover by themselves the predictive structure of problems. Techniques and algorithms that have stemmed from the field of *machine learning* have indeed now become a powerful tool for the analysis of complex and large data, successfully assisting scientists in numerous breakthroughs of various fields of science and technology. Public and famous examples

1

INTRODUCTION

Em vários domínios da ciência, da tecnologia e das humanidades, como na biologia, na meteorologia, na medicina ou nas finanças, para citar alguns, os especialistas visam prever um fenómeno com base em observações ou medições passadas. Por exemplo, os meteorologistas tentam prever o tempo para os próximos dias a partir das condições climáticas dos dias anteriores. Na medicina, os profissionais coletam medidas e informações como pressão arterial, idade ou histórico para diagnosticar a condição dos pacientes que chegam. Da mesma forma, em química, os compostos são analisados utilizando medidas de espectrometria de massa para determinar se contêm um determinado tipo de moléculas ou átomos. Em todos esses casos, o objetivo é a previsão de uma variável resposta com base em um conjunto de variáveis preditoras observadas.

Durante séculos, os cientistas abordaram estes problemas derivando quadros teóricos dos primeiros princípios ou acumulando conhecimento para modelar, analisar e compreender o fenómeno em estudo. Por exemplo, os profissionais sabem, por experiência passada, que os pacientes idosos com ataques cardíacos e pressão arterial baixa são geralmente de alto risco. Da mesma forma, os meteorologistas sabem, a partir de modelos climáticos elementares, que é provável que um dia quente e de elevada poluição seja seguido de outro. No entanto, para um número crescente de problemas, as abordagens padrão começam a mostrar os seus limites. Por exemplo, identificar os factores de risco genéticos para doenças cardíacas, onde o conhecimento ainda é muito escasso, é quase impraticável para as capacidades cognitivas dos seres humanos, dada a elevada complexidade e complexidade das interacções que existem entre os genes. Da mesma forma, para previsões meteorológicas muito refinadas, é necessário ter em conta um grande número de variáveis, o que ultrapassa rapidamente as capacidades dos especialistas para colocá-las todas num sistema de equações. Para quebrar esta barreira cognitiva e fazer avançar ainda mais a ciência, máquinas de velocidade e capacidade crescentes foram construídas e concebidas desde meados do século XX para ajudar os humanos nos seus cálculos. Surpreendentemente, contudo, juntamente com este progresso em termos de hardware, os desenvolvimentos na ciência teórica da computação, na inteligência artificial e na estatística fizeram com que as máquinas se tornassem mais do que calculadoras. Avanços recentes tornaram-nos especialistas da sua própria espécie, capazes de aprender com os dados e de descobrir por si próprios a estrutura preditiva dos problemas. As técnicas e algoritmos que surgiram no campo da aprendizagem automática tornaram-se, de facto, uma ferramenta poderosa para a análise de dados complexos e grandes, ajudando com sucesso os cientistas em numerosos avanços em vários campos da ciência e da tecnologia. Exemplos públicos e famosos

include the use of boosted decision trees in the statistical analysis that led to the detection of the Higgs boson at CERN [Chatrchyan et al., 2012], the use of random forests for human pose detection in the Microsoft Kinect [Criminisi and Shotton, 2013] or the implementation of various machine learning techniques for building the IBM Watson system [Ferrucci et al., 2010], capable to compete at the human champion level on the American TV quiz show Jeopardy.

Formally, machine learning can be defined as the study of systems that can learn from data without being explicitly programmed. According to Mitchell [1997], a computer program is said to learn from data, with respect to some class of tasks and performance measure if its performance at those tasks improves with data. In particular, machine learning provides algorithms that are able to solve classification or regression tasks, hence bringing now automated procedures for the prediction of a phenomenon based on past observations. However, the goal of machine learning is not only to produce algorithms making accurate predictions, it is also to provide insights on the predictive structure of the data [Breiman et al., 1984]. If we are aiming at the latter, then our goal is to understand what variables or interactions of variables drive the phenomenon. For practitioners, which are not experts in machine learning, interpretability is indeed often as important as prediction accuracy. It allows for a better understanding of the phenomenon under study, a finer exploration of the data and an easier self-appropriation of the results. By contrast, when an algorithm is used as a black box, yielding results seemingly out of nowhere, it may indeed be difficult to trust or accept if it cannot be understood how and why the procedure came to them. Unfortunately, the current state-of-the-art in machine learning often makes it difficult for non-experts to understand and interpret the results of an algorithm. While considerable efforts have been put to improve their prediction accuracy, it is still not clearly understood what makes machine learning algorithms truly work, and under what assumptions. Likewise, few of them actually provide clear and insightful explanations about the results they generate.

In this context, the goal of this thesis is to provide a comprehensive and self-contained analysis of a class of algorithms known as decision trees [Breiman et al., 1984] and random forests [Breiman, 2001]. While these methods have proven to be a robust, accurate and successful tool for solving countless of machine learning tasks, including classification, regression, density estimation, manifold learning or semi-supervised learning [Criminisi and Shotton, 2013], there remain many gray areas in their understanding:

(a) First, the theoretical properties and statistical mechanisms that drive the algorithm are still not clearly and entirely understood. Random forests indeed evolved from empirical successes rather than from a sound theory. As such, various parts of the algo-

incluem o uso de árvores de decisão otimizadas na análise estatística que levou à detecção do bóson de Higgs no CERN [Chatrchyan et al., 2012], o uso de florestas aleatórias para detecção de pose humana no Microsoft Kinect [Criminisi e Shotton, 2013] ou a implementação de várias técnicas de aprendizado de máquina para construir o sistema IBM Watson [Ferrucci et al., 2010], capaz de competir no mercado hu- homem de nível campeão no programa de perguntas da TV americana Jeopardy.

Formalmente, o aprendizado de máquina pode ser definido como o estudo de sistemas que podem aprender a partir de dados sem serem explicitamente programados. De acordo com Mitchell, diz-se que um programa de computador aprende com os dados, no que diz respeito a alguma classe de tarefas e medidas de desempenho, se o seu desempenho nessas tarefas melhorar com os dados. Em particular, o aprendizado de máquina fornece algoritmos capazes de resolver tarefas de classificação ou regressão, trazendo agora procedimentos automatizados para a previsão de um fenômeno com base em observações passadas. No entanto, o objetivo do aprendizado de máquina não é apenas produzir algoritmos que façam previsões precisas, mas também fornecer insights sobre a estrutura preditiva dos dados [Breiman et al., 1984]. Se pretendemos o último, então o nosso objectivo é compreender que variáveis ou interacções de variáveis impulsionam o fenómeno. Para os profissionais, que não são especialistas em aprendizado de máquina, a i nterpretabilidade é muitas vezes tão importante quanto a precisão da previsão. Permite uma melhor compreensão do fenómeno em estudo, u ma exploração mais apurada dos dados e uma mais fácil autoapropriação dos resultados. Por outro lado, quando um algoritmo é usado como uma caixa preta, produzindo resultados aparentemente surgidos do nada, pode ser realmente difícil confiar ou aceitar se não for possível compreender como e por que o procedimento chegou até eles. Infelizmente, o estado da arte atual em aprendizado de máquina muitas vezes torna difícil para leigos compreenderem e interpretarem os resultados de um algoritmo. Embora esforços consideráveis tenham sido feitos para melhorar a precisão de suas previsões, ainda não está claro o que faz os algoritmos de aprendizado de máquina realmente funcionarem e sob quais pressupostos. Da mesma forma, poucos deles realmente fornecem explicações claras e perspicazes sobre os resultados que geram.

Neste contexto, o objetivo desta tese é fornecer uma análise abrangente e independente de uma classe de algoritmos conhecidos como árvores de decisão [Breiman et al., 1984] e florestas aleatórias [Breiman, 2001]. Embora esses métodos tenham provado ser uma ferramenta r obusta, precisa e bem-sucedida para resolver inúmeras tarefas de aprendizado de máquina, incluindo classificação, regressão, estimativa de densidade, aprendizado múltiplo ou aprendizado semissupervisionado [Criminisi e Shotton, 2013], ainda existem muitas áreas cinzentas em seu entendimento:

(a) Primeiro, as propriedades teóricas e os mecanismos estatísticos que orientam o algoritmo ainda não são clara e totalmente compreendidos. Na verdade, as florestas aleatórias evoluíram a partir de sucessos empíricos e não de uma teoria sólida. Como tal, várias partes do algoritmo

rithm remain heuristic rather than theoretically motivated. For example, preliminary results have proven the consistency of simplified to very close variants of random forests, but consistency of the original algorithm remains unproven in a general setting.

(b) Second, while the construction process of a single decision tree can easily be described within half a page, implementing this algorithm properly and efficiently remains a challenging task involving issues that are easily overlooked. Unfortunately, implementation details are often omitted in the scientific literature and can often only be found by diving into (unequally documented) existing software implementations. As far as we know, there is indeed no comprehensive survey covering the implementation details of random forests, nor with their respective effects in terms of runtime and space complexity or learning ability.

(c) Third, interpreting the resulting model remains a difficult task, for which even machine learning experts still fail at finely analyzing and uncovering the precise predictive structure learned by the procedure. In particular, despite their extensive use in a wide range of applications, little is still known regarding variable importance measures computed by random forests. Empirical evidence suggests that they are appropriate for identifying relevant variables, but their statistical mechanisms and properties are still far from being understood.

All throughout this dissertation, our objective is therefore to call into question each and every part of the random forests methodology, both from a theoretical and practical point of view. Accordingly, this work aims at revisiting decision trees and random forests to hopefully shed new light on their learning capabilities, inner workings and interpretability.

1.1 OUTLINE AND CONTRIBUTIONS

Part I of this manuscript is first dedicated to a thorough treatment of decision trees and forests of randomized trees. We begin in Chapter 2 by outlining fundamental concepts of machine learning, and then proceed in Chapters 3 and 4 with a comprehensive review of the algorithms at the core of decision trees and random forests. We discuss the learning capabilities of these models and carefully study all parts of the algorithm and their complementary effects. In particular, Chapter 4 includes original contributions on the bias-variance analysis of ensemble methods, highlighting how randomization can help improve performance. Chapter 5 concludes this first part with an original space and time complexity analysis of random forests (and

O ritmo permanece heurístico em vez de motivado teoricamente. Por exemplo, resultados preliminares provaram a consistência de variantes simplificadas a muito próximas de florestas aleatórias, mas a consistência do algoritmo original permanece não comprovada num cenário geral.

(b) Em segundo lugar, embora o processo de construção de uma única árvore de decisão possa ser facilmente descrito em meia página, a implementação deste algoritmo de forma adequada e eficiente continua a ser uma tarefa desafiadora que envolve questões que são facilmente ignoradas. Infelizmente, os detalhes da implementação são muitas vezes omitidos na literatura científica e muitas vezes só podem ser encontrados mergulhando nas implementações de software existentes (documentadas de forma desigual). Tanto quanto sabemos, não existe de facto nenhum inquérito abrangente que cubra os detalhes de implementação de florestas aleatórias, nem os seus respectivos efeitos em termos de complexidade de tempo e espaço de execução ou capacidade de aprendizagem.

(c) Terceiro, interpretar o modelo resultante continua a ser uma tarefa difícil, para a qual mesmo os especialistas em aprendizagem auto mática ainda falham na análise detalhada e na descoberta da estrutura preditiva precisa aprendida pelo procedimento. Em particular, apesar do seu uso extensivo numa ampla gama de aplicações, pouco ainda se sabe sobre medidas de importância variável calculadas por florestas aleatórias. A evidência empírica sugere que são apropriadas para identificar variáveis relevantes, mas os seus mecanismos e propriedades estatísticas ainda estão longe de ser compreendidos.

Ao longo desta dissertação, o nosso objectivo é, portanto, pôr em causa toda e qualquer parte da metodologia das florestas aleatórias, tanto do ponto de vista teórico como prático. Assim, este trabalho visa revisitar árvores de decisão e florestas aleatórias para lançar uma nova luz sobre as suas capacidades de aprendizagem, funcionamento interno e interpretabilidade.

1.1 esboço e contribuições

A Parte I deste manuscrito é primeiramente dedicada a um tratamento completo de árvores de decisão e florestas de árvores aleatórias. Começamos no Capítulo 2 delineando conceitos fundamentais de aprendizado de máquina e depois prosseguimos nos Capítulos 3 e 4 com uma revisão abrangente dos algoritmos no núcleo das árvores de decisão e das florestas aleatórias. Discutimos as capacidades de aprendizagem desses modelos e estudamos cuidadosamente todas as partes do algoritmo e seus efeitos complementares. Em particular, o Capítulo 4 inclui contribuições originais sobre a análise de viés-variância de métodos de conjunto, destacando como a randomização pode ajudar a melhorar o desempenho. O Capítulo 5 conclui esta primeira parte com uma análise original da complexidade espacial e temporal de florestas aleatórias (e

their variants), along with an in-depth discussion of implementation details, as contributed within the open source Scikit-Learn library. Overall, Part I therefore presents a comprehensive review of previous work on random forests, including some original contributions both from a theoretical and practical point of view.

Part II analyzes and discusses the interpretability of random forests. In Chapter 6, we study variable importances as computed with a forest of randomized trees and study how these scores can be interpreted in order to reveal the underlying predictive structure learned from the data. In particular, we derive a theoretical framework from which we prove theoretical and practical properties of variable importances. In Chapter 7, we then exploit this framework to further study variable importances as derived from actual random forests and present successful applications of variable importance measures. Part II constitutes the main contributions of this dissertation.

Finally, Part III addresses limitations of random forests in the context of large datasets. Through extensive experiments, we show in Chapter 8 that subsampling strategies provides on par performance while simultaneously lowering the memory requirements. This chapter presents original work.

1.2 PUBLICATIONS

This dissertation summarizes several contributions to random forests algorithms. Publications that have directly stemmed from this work include:

- [Geurts and Louppe, 2011] *Learning to rank with extremely randomized trees*, Geurts Pierre and Louppe Gilles. In JMLR: Workshop and Conference Proceedings, volume 14, 2011.
- [Louppe and Geurts, 2012] *Ensembles on random patches*, Louppe Gilles and Geurts Pierre. In Machine Learning and Knowledge Discovery in Databases, pages 346–361. Springer, 2012.
- [Louppe et al., 2013] *Understanding variable importances in forests of randomized trees*, Louppe Gilles, Wehenkel Louis, Sutera Antonio and Geurts Pierre. In Advances in Neural Information Processing Systems, pages 431–439, 2013.
- [Buitinck et al., 2013] *API design for machine learning software: experiences from the scikit-learn project*, Buitinck Lars, Louppe Gilles, Blondel Mathieu et al.. In ECML-PKDD 2013 Workshop: Languages for Data Mining and Machine Learning, 2013.
- [Botta et al., 2014] *Exploiting SNP Correlations within Random Forest for Genome-Wide Association Studies*, Botta Vincent, Louppe Gilles, Geurts Pierre and Wehenkel Louis. PloS one, 9(4):e93379, 2014.

suas variantes), juntamente com uma discussão aprofundada dos detalhes de implementação, conforme contribuído na biblioteca de código aberto Scikit-Learn. No geral, a Parte I apresenta, portanto, uma revisão abrangente de trabalhos anteriores sobre florestas aleatórias, incluindo algumas contribuições originais, tanto do ponto de vista teórico quanto prático.

Partii analisa e discute a interpretabilidade de florestas aleatórias. No Capítulo 6, estudamos as importâncias das variáveis calculadas com uma floresta de árvores aleatórias e estudamos como essas pontuações podem ser interpretadas para revelar a estrutura preditiva subjacente aprendida com os dados. Em particular, derivamos um quadro teórico a partir do qual provamos propriedades teóricas e práticas de importâncias variáveis. No Capítulo 7, exploramos então esta estrutura para estudar mais detalhadamente as importâncias variáveis derivadas de florestas aleatórias reais e apresentar aplicações bem-sucedidas de medidas de importância variável. A parte II constitui as principais contribuições desta dissertação.

Finalmente, a Parte III aborda as limitações das florestas aleatórias no contexto de grandes conjuntos de dados. Por meio de experimentos extensivos, mostramos no Capítulo 8 que as estratégias de suba mostragem fornecem desempenho equivalente e, ao mesmo tempo, reduzem os requisitos de memória. Este capítulo apresenta trabalho original.

1.2 publicações

Esta dissertação resume diversas contribuições para algoritmos de florestas aleatórias. As publicações que resultaram diretamente deste trabalho incluem:

- [Geurts e Louppe, 2011] Aprendendo a classificar com árvores extremamente aleatórias, Geurts Pierre e Louppe Gilles. Em JMLR: Workshop and Conference Proceedings, volume 14, 2011. - [Louppe e Geurts, 2012] Ensembles on random patches, Louppe Gilles e Geurts Pierre. Em Aprendizado de Máquina e Descoberta de Conhecimento em Bancos de Dados, páginas 346–361. Springer, 2012. - [Louppe et al., 2013] Compreendendo as importâncias variáveis em florestas de árvores aleatórias, Louppe Gilles, Wehenkel Louis, Sutera Antonio e Geurts Pierre. Em Advances in Neural Information Processing Systems, páginas 431–439, 2013. - [Buitinck et al., 2013] Design de API para software de aprendizado de máquina: experiências do projeto scikit-learn, Buitinck Lars, Louppe Gilles, Blondel Mathieu et al.. Em ECML-PKDD 2013 Workshop: Languages for Data Mining and Machine Learning, 2013. - [Botta et al., 2014] Exploiting SNP Correlations inside Random Forest for Genome-Wide Association Studies, Botta Vincent, Louppe Gilles, Geurts Pierre e Wehenkel Louis. PloS um, 9(4):e93379, 2014.

During the course of this thesis, several fruitful collaborations have also led to the following publications. These are not discussed within this dissertation.

- [Louppe and Geurts, 2010] *A zealous parallel gradient descent algorithm*, Louppe Gilles and Geurts Pierre. In Learning on Cores, Clusters and Clouds workshop, NIPS, 2010

- [Marée et al., 2014] *A hybrid human-computer approach for large-scale image-based measurements using web services and machine learning*, Marée Raphaël, Rollus Loic, Stevens Benjamin et al. Proceedings IEEE International Symposium on Biomedical Imaging, 2014.

- [McGovern et al., 2014] *Solar Energy Prediction: An International Contest to Initiate Interdisciplinary Research on Compelling Meteorological Problems*, Amy McGovern, David John Gagne II, Lucas Eustaquio et al., 2014. *Submitted.*

- [Sutera et al., 2014] *Simple connectome inference from partial correlation statistics in calcium imaging*, Antonio Sutera, Arnaud Joly, Vincent Francois-Lavet et al., 2014. *Submitted.*

Durante o curso desta tese, diversas colaborações frutíferas também levaram às seguintes publicações. Estes não são discutidos nesta dissertação.

- [Louppe e Geurts, 2010] Um zeloso algoritmo de descida de gradiente paralelo, Louppe Gilles e GeurtsPierre. No workshop Aprendendo sobre Núcleos, Clusters e Nuvens, NIPS, 2010

- [Marée et al., 2014] Uma abordagem híbrida humano-computador para medições baseadas em imagens em grande escala usando s erviços web e aprendizado de máquina, Marée Raphaël, Rollus Loic, Stevens Benjamin et al. Proceedings IEEE International Symposium on Biomedical Imaging, 2014.

- [*McGovern et al.*, 2014] *Predição de energia solar: um concurso internacional para iniciar pesquisas interdisciplinares sobre problemas meteorológicos convincentes, Amy McGovern, David John Gagne II, Lucas Eustaquio et al.,2014. Enviado.*

- [*Sutera et al.*, 2014] *Inferência de conectoma simples a partir de estatísticas de correlação parcial em imagens de cálcio, Antonio Sutera, Arnaud Joly, VincentFrancois-Lavetetal., 2014. Enviado.*

Part I

GROWING DECISION TREES

Parte I

CRESCENDO ÁRVORES DE DECISÃO

2 BACKGROUND

Outline

In this chapter, we introduce supervised learning and fundamental concepts on which this work builds upon. In Section 2.1, we first describe classification and regression learning tasks and then formally define the notion of model. In Section 2.2, we proceed with a discussion on performance evaluation and then describe, in Section 2.3, procedures for selecting the best possible model. Finally, we conclude in Section 2.4 with a brief overview of some of the main methods in supervised learning.

2.1 LEARNING FROM DATA

In the examples introduced in Chapter 1, the objective which is sought is to find a systematic way of predicting a phenomenon given a set of measurements. In machine learning terms, this goal is formulated as the *supervised learning* task of inferring from collected data a model that predicts the value of an output variable based on the observed values of input variables. As such, finding an appropriate model is based on the assumption that the output variable does not take its value at random and that there exists a relation between the inputs and the output. In medicine for instance, the goal is to find a decision rule (i.e., a model) from a set of past cases (i.e., the collected data) for predicting the condition of an incoming patient (i.e., the output value) given a set of measurements such as age, sex, blood pressure or history (i.e., the input values).

To give a more precise formulation, let us assume as set of *cases* or *objects* taken from a universe Ω. Let us arrange the set of measurements on a case in a pre-assigned order, i.e., take the input values to be $x_1, x_2, ..., x_p$, where $x_j \in \mathcal{X}_j$ (for $j = 1, ..., p$) corresponds to the value of the input variable X_j. Together, the input values $(x_1, x_2, ..., x_p)$ form a p-dimensional input vector $\mathbf{x}$ taking its values in $\mathcal{X}_1 \times ... \times \mathcal{X}_p = \mathcal{X}$, where $\mathcal{X}$ is defined as the input space. Similarly, let us define as $y \in \mathcal{Y}$ the value of the output variable Y, where $\mathcal{Y}$ is defined as the output space[1]. By definition, both the input and the output spaces are assumed to respectively contain all possible input vectors and all possible output values. Note that input variables are some-

1 Unless stated otherwise, supervised learning is reduced to the prediction of a *single* output variable. More generally however, this framework can be defined as the prediction of one or several output variables.

2 FUNDO

Contorno

Neste capítulo, apresentamos a aprendizagem supervisionada e os conceitos fundamentais nos quais este trabalho se baseia. Na Seção 2.1, primeiro descrevemos tarefas de aprendizagem de classificação e regressão e depois definimos formalmente a noção de modelo. Na Seção 2.2, prosseguimos com uma discussão sobre avaliação de desempenho e depois descrevemos, na Seção 2.3, procedimentos para seleção do melhor modelo possível. Finalmente, concluímos na Seção 2.4 com uma breve visão geral de alguns dos principais métodos de aprendizagem supervisionada.

2.1 aprendendo com os dados

Nos exemplos introduzidos no Capítulo 1, o objectivo que se procura é encontrar uma forma sistemática de prever um fenómeno dado um conjunto de medições. Em termos de aprendizado de máquina, esse objetivo é formulado como a tarefa de aprendizado supervisionado de inferir a partir dos dados coletados um modelo que prevê o valor de uma variável de saída com base nos valores observados das variáveis de entrada. Como tal, encontrar um modelo apropriado baseia-se no pressuposto de que a variável de saída não assume o seu valor aleatoriamente e que existe uma relação entre os inputs e o output. Na medicina, por exemplo, o objetivo é encontrar uma regra de decisão (ou seja, um modelo) a partir de um conjunto de casos passados (ou seja, os dados coletados) para prever a condição de um paciente que chega (ou seja, o valor de saída) dado um conjunto de medidas como idade, sexo, pressão arterial ou histórico (ou seja, os valores de entrada).

Para dar uma formulação mais precisa, vamos supor como conjunto de casos ou objetos retirados de um universo Ω. Vamos organizar o conjunto de medições em um caso em uma ordem pré-atribuída, ou seja, considerar os valores de entrada como $x_1, x_2, ..., x_p$, onde $x_j \in \mathcal{X}_j$ (para $j = 1, ..., p$) corresponde ao valor da variável de entrada X_j. Juntos, os valores de entrada $(x_1, x_2, ..., x_p)$ formam um vetor de entrada $\mathbf{x}$ p-dimensional tomando seus valores em $\mathcal{X}_1 \times ... \times \mathcal{X}_p = \mathcal{X}$, onde $\mathcal{X}$ é definido como o espaço de entrada. Da mesma forma, vamos definir como $y \in \mathcal{Y}$ o valor da variável de saída Y, onde $\mathcal{Y}$ é definido como o espaço de saída[1]. Por definição, assume-se que tanto o espaço de entrada quanto o de saída contêm, respectivamente, todos os vetores de entrada possíveis e todos os valores de saída possíveis. Observe que as variáveis de entrada são algumas

1 Salvo indicação em contrário, a aprendizagem supervisionada é reduzida à previsão de uma única variável de saída. Mais genericamente, contudo, este quadro pode ser definido como a previsão de uma ou várias variáveis de saída.

times known as *features*, input vectors as *instances* or *samples* and the output variable as *target*.

Among variables that define the problem, we distinguish between two general types. The former correspond to quantitative variables whose values are integer or real numbers, such as age or blood pressure. The latter correspond to qualitative variables whose values are symbolic, such as gender or condition. Formally, we define them as follows:

Definition 2.1. *A variable* X_j *is* ordered *if* $\mathcal{X}_j$ *is a totally ordered set. In particular,* X_j *is said to be* numerical *if* $\mathcal{X}_j = \mathbb{R}$.

Definition 2.2. *A variable* X_j *is* categorical *if* $\mathcal{X}_j$ *is a finite set of values, without any natural order.*

In a typical supervised learning task, past observations are summarized by a dataset called *learning set*. It consists in a set of observed input vectors together with their actual output value and formally defined as follows:

Definition 2.3. *A* learning set $\mathcal{L}$ *is a set of* N *pairs of input vectors and output values* $(\mathbf{x}_1, y_1), ..., (\mathbf{x}_N, y_N)$*, where* $\mathbf{x}_i \in \mathcal{X}$ *and* $y_i \in \mathcal{Y}$.

Equivalently, a set of p-input vectors $\mathbf{x}_i$ (for $i = 1, ..., N$) can be denoted by a $N \times p$ matrix $\mathbf{X}$, whose rows $i = 1, ..., N$ correspond to input vectors $\mathbf{x}_i$ and columns $j = 1, ..., p$ to input variables X_j. Similarly, the corresponding output values can be written as a vector $\mathbf{y} = (y_1, ..., y_N)$.

Data representation

For optimal implementations of machine learning algorithms, data needs to be represented using structures which allow for high-performance numerical computation. In this work, code snippets are described under the assumption that data is represented using a data structure similar to a NumPy array [Van der Walt et al., 2011].

A NumPy array is basically a multidimensional uniform collection of values, all of the same type and organized in a given shape. For instance, a matrix $\mathbf{X}$ can be represented as a 2-dimensional NumPy array of shape $N \times p$ that contains numbers (e.g., floating point values or integers). This structure allows for random access in constant time, vectorized high-level operations and efficient memory usage.

Additionally, using a data representation which is close to the matrix formulation, like NumPy arrays, makes it possible to write implementations that are close to their original textbook formulation, thereby making them easier to code, understand and maintain.

In this framework, the supervised learning task can be stated as learning a function $\varphi : \mathcal{X} \mapsto \mathcal{Y}$ from a learning set $\mathcal{L} = (\mathbf{X}, \mathbf{y})$. The objective is to find a model such that its predictions $\varphi(\mathbf{x})$, also denoted

tempos conhecidos como recursos, vetores de entrada como instâncias ou amostras e a variável de saída como destino.

Entre as variáveis que definem o problema, distinguimos dois tipos gerais. As primeiras correspondem a variáveis quantitativas cujos valores são números inteiros ou reais, como idade ou pressão arterial. Estas últimas correspondem a variáveis qualitativas cujos valores são simbólicos, como género ou condição. Formalmente, nós os definimos da seguinte forma:

Definição 2.1. Uma variável X_j *é ordenada se* $\mathcal{X}_j$ *é um conjunto totalmente ordenado. Em particular, diz-se que* X_j *é numérico se* $\mathcal{X}_j = \mathbb{R}$.

Definição 2.2. Uma variável X_j *é categórica se* $\mathcal{X}_j$ *é um conjunto finito de valores, sem qualquer ordem natural.*

Numa tarefa típica de aprendizagem supervisionada, as observações passadas são resumidas por um conjunto de dados denominado conjunto de aprendizagem. Consiste em um conjunto de vetores de entrada observados juntamente com seu valor real de saída e formalmente definidos da seguinte forma:

Definição 2.3. Um conjunto de aprendizagem $\mathcal{L}$ *é um conjunto de* N *pares de vetores de entrada e valores de saída* $(x1, y_1), ..., (x_N, y_N)$*, onde* $x_i \in \mathcal{X}$ *e* $y_i \in \mathcal{Y}$.

Equivalentemente, um conjunto de $p$vetores de entrada x_i (para $i = 1, ..., N$) pode ser denotado por uma $N \times p$ matriz X, cujas linhas $i = 1, ..., N$ correspondem aos vetores de entrada x_i e colunas $j = 1, ..., p$ às variáveis de entrada X_j. Da mesma forma, os valores de saída correspondentes podem ser escritos como um vetor $y = (y_1, ..., y_N)$.

Representação de dados

Para implementações ideais de algoritmos de aprendizado de máquina, os dados precisam ser representados usando estruturas que permitam computação numérica de alto desempenho. Neste trabalho, trechos de código são descritos sob a suposição de que os dados são representados usando uma estrutura de dados semelhante a um array NumPy [Van der Walt et al., 2011].

Um array NumPy é basicamente uma coleção multidimensional uniforme de valores, todos do mesmo tipo e organizados em uma determinada forma. Por exemplo, uma matriz X pode ser representada como uma matriz NumPy bidimensional de formato $N \times p$ que contém números (por exemplo, valores de ponto flutuante ou inteiros). Essa estrutura permite acesso aleatório em tempo constante, operações vetorizadas de alto nível e uso eficiente de memória.

Além disso, o uso de uma representação de dados próxima à formulação da matriz, como os arrays NumPy, torna possível escrever implementações próximas à formulação original do livro didático, tornando assim mais fácil codificar, compreender e manter.

Nesta estrutura, a tarefa de aprendizagem supervisionada pode ser declarada como a aprendizagem de uma função $\varphi : \mathcal{X} \mapsto \mathcal{Y}$ a partir de um conjunto de aprendizagem $\mathcal{L} = (X,y)$. O objetivo é encontrar um modelo tal que suas previsões $\varphi($ x), também denotadas

by the variable $\hat{Y}$, are as good as possible. If Y is a categorical variable then the learning task is a classification problem. If Y is numerical variable, then learning task is a regression problem. Without loss of generality, the resulting models can be defined as follows:

Definition 2.4. *A* classifier *or* classification rule *is a function* $\varphi : \mathcal{X} \mapsto \mathcal{Y}$, *where* $\mathcal{Y}$ *is a finite set of classes (or labels) denoted* $\{c_1, c_2, ..., c_J\}$.

Definition 2.5. *A* regressor *is a function* $\varphi : \mathcal{X} \mapsto \mathcal{Y}$, *where* $\mathcal{Y} = \mathbb{R}$.

ESTIMATOR INTERFACE

We follow in this work the API conventions proposed by Buitinck et al. [2013]. Learning algorithms are described as *estimator* objects implementing the following interface:

- Hyper-parameters of an algorithm are all passed to the constructor of the estimator. The constructor does not see any actual data. All it does is to attach hyper-parameters values as public attributes to the estimator object.
- Learning is performed in a `fit` method. This method is called with a learning set (e.g., supplied as two arrays `X_train` and `y_train`). Its task is to run a learning algorithm and to determine model-specific parameters from the data. The `fit` method always returns the estimator object it was called on, which now serves as a model and can be used to make predictions.
- Predictions are performed through a `predict` method, taking as input an array `X_test` and producing as output the predictions for `X_test` based on the learned parameters. In the case of classification, this method returns labels from $\mathcal{Y} = \{c_1, c_2, ..., c_J\}$. In the case of regression, it returns numerical values from $\mathcal{Y} = \mathbb{R}$.

With this API, a typical supervised learning task is performed as follows:

```
# Instantiate and set hyper-parameters
clf = DecisionTreeClassifier(max_depth=5)
# Learn a model from data
clf.fit(X_train, y_train)
# Make predictions on new data
y_pred = clf.predict(X_test)
```

2.2 PERFORMANCE EVALUATION

In the statistical sense, input and output variables $X_1, ..., X_p$ and Y are *random variables* taking jointly their values from $\mathcal{X} \times \mathcal{Y}$ with respect to the joint probability distribution $P(X, Y)$, where X denotes the random

pela variável ˆY, são os melhores possíveis. Se Y for uma variável categórica então a tarefa de aprendizagem é um problema de classificação. Se Y for u ma variável numérica, então a tarefa de aprendizagem é um problema de regressão. Sem perda de generalidade, os modelos resultantes podem ser definidos da seguinte forma:

Definição 2.4. Um classificador ou regra de classificação é uma função $\varphi : \mathcal{X} \mapsto \mathcal{Y}$, *onde* $\mathcal{Y}$ *é um conjunto finito de classes (ou rótulos) denotado* $\{c_1, c_2, ..., c_J\}$.

Definição 2.5. Um regressor é uma função $\varphi : \mathcal{X} \mapsto \mathcal{Y}$, *onde* $\mathcal{Y} = \mathbb{R}$.

Interface do estimador

Seguimos neste trabalho as convenções de API propostas por Buitinck et al. [2013]. Algoritmos de aprendizagem são descritos como objetos estimadores que implementam a seguinte interface:

- Os hiperparâmetros do algoritmo são todos passados ao construtor do estimador. O construtor não vê nenhum dado real. Tudo o que ele faz é anexar valores de hiperparâmetros como atributos públicos ao objeto estimador.

- A aprendizagem é realizada de forma adequada. Este método é chamado com um conjunto de aprendizagem (por exemplo, fornecido como duas matrizes X_train e y_train). Sua tarefa é executar um algoritmo de aprendizado e determinar parâmetros específicos do modelo a partir dos dados. O método fit sempre retorna o objeto estimador ao qual foi chamado, que agora serve como modelo e pode ser usado para fazer previsões.

- As previsões são realizadas através de um método de previsão, tomando como entrada um array X_test e produzindo como saída as previsões para X_test com base nos parâmetros aprendidos. No caso de classificação, este método retorna rótulos de $\mathcal{Y} = \{c_1, c_2, ..., c_J\}$. No caso de regressão, retorna valores numéricos de $\mathcal{Y} = \mathbb{R}$.

Com esta API, uma tarefa típica de aprendizagem supervisionada é executada da seguinte forma:

```
# Instancie e defina hiperparâmetros clf =
DecisionTreeClassifier(max_profundidade=5) #
Aprenda um modelo a partir de dados
clf.fit(X_train, y_train) # Faça previsões sobre
novos dados y_pred =clf.predict(X_test)
```

2.2 avaliação de desempenho

No sentido estatístico, as variáveis de entrada e saída $X_1, ..., X_p$ e Y são variáveis aleatórias que tomam conjuntamente seus valores de $\mathcal{X} \times \mathcal{Y}$ em relação à distribuição de probabilidade conjunta $P(X, Y)$, onde X denota a distribuição aleatória

vector $(X_1, ..., X_p)$. That is, $P(X = \mathbf{x}, Y = y)$ is the probability that random variables X and Y take values $\mathbf{x}$ and y from $\mathcal{X}$ and $\mathcal{Y}$ when drawing an object uniformly at random from the universe Ω.

Accordingly, using an algorithm $\mathcal{A}$ for learning a model[2] $\varphi_{\mathcal{L}}$ whose predictions are as good as possible can be stated as finding a model which minimizes its expected prediction error, defined as follows:

Definition 2.6. *The* expected prediction error*, also known as* generalization error *or* test error*, of the model* $\varphi_{\mathcal{L}}$ *is*[3]

$$Err(\varphi_{\mathcal{L}}) = \mathbb{E}_{X,Y}\{L(Y, \varphi_{\mathcal{L}}(X))\}, \tag{2.1}$$

where $\mathcal{L}$ *is the learning set used to build* $\varphi_{\mathcal{L}}$ *and* L *is a loss function measuring the discrepancy between its two arguments [Geurts, 2002].*

Equation 2.1 basically measures the prediction error of $\varphi_{\mathcal{L}}$ over all possible objects in Ω (each represented by a couple $(\mathbf{x}, y) \in \mathcal{X} \times \mathcal{Y}$), including the observed couples from the learning set $\mathcal{L}$ but also all the *unseen* ones from $\mathcal{X} \times \mathcal{Y} \setminus \mathcal{L}$. Indeed, the goal is not in fact to make the very-most accurate predictions over the subset $\mathcal{L}$ of known data, but rather to learn a model which is correct and reliable on all possible data.

For classification, the most common loss function is the *zero-one* loss function[4] $L(Y, \varphi_{\mathcal{L}}(X)) = 1(Y \neq \varphi_{\mathcal{L}}(X))$, where all misclassifications are equally penalized. In this case, the generalization error of $\varphi_{\mathcal{L}}$ becomes the probability of misclassification of the model:

$$Err(\varphi_{\mathcal{L}}) = \mathbb{E}_{X,Y}\{1(Y \neq \varphi_{\mathcal{L}}(X))\} = P(Y \neq \varphi_{\mathcal{L}}(X)) \tag{2.2}$$

Similarly, for regression, the most used loss function is the *squared error* loss $L(Y, \varphi_{\mathcal{L}}(X)) = (Y - \varphi_{\mathcal{L}}(X))^2$, where large differences between the true values and the predicted values are penalized more heavily than small ones. With this loss, the generalization error of the model becomes:

$$Err(\varphi_{\mathcal{L}}) = \mathbb{E}_{X,Y}\{(Y - \varphi_{\mathcal{L}}(X))^2\} \tag{2.3}$$

2 Unless it is clear from the context, models are now denoted $\varphi_{\mathcal{L}}$ to emphasize that they are built from the learning set $\mathcal{L}$.

3 $\mathbb{E}_X\{f(X)\}$ denotes the expected value of $f(x)$ (for $x \in \mathcal{X}$) with respect to the probability distribution of the random variable X. It is defined as

$$\mathbb{E}_X\{f(X)\} = \sum_{x \in \mathcal{X}} P(X = x) f(x).$$

4 $1(condition)$ denotes the unit function. It is defined as

$$1(condition) = \begin{cases} 1 & \text{if } condition \text{ is true} \\ 0 & \text{if } condition \text{ is false} \end{cases}.$$

vetor $(X_1, ..., X_p)$. Ou seja, $P(X = x, Y = y)$ é a probabilidade de que variáveis aleatórias X e Y obtenham valores x e y de $\mathcal{X}$ e $\mathcal{Y}$ ao desenhar um objeto uniformemente e aleatoriamente do universo Ω.

Conseqüentemente, usar um algoritmo $\mathcal{A}$ para aprender um modelo[2] $\varphi_{\mathcal{L}}$ cujas previsões sejam as melhores possíveis pode ser considerado como encontrar um modelo que minimize seu erro de previsão esperado, definido como segue:

Definição 2.6. O erro de previsão esperado, também conhecido como erro de generalização ou erro de teste, do modelo $\varphi_{\mathcal{L}}$ é[3]

$$Err(\varphi_{\mathcal{L}}) = \mathbb{E}_{X,Y}\{L(Y, \varphi_{\mathcal{L}}(X))\}, \tag{2.1}$$

onde $\mathcal{L}$ *é o conjunto de aprendizagem usado para construir* $\varphi_{\mathcal{L}}$ *e* L *é uma função de perda que mede a discrepância entre seus dois argumentos [Geurts, 2002].*

A Equação 2.1 basicamente mede o erro de predição de $\varphi_{\mathcal{L}}$ sobre todos os objetos possíveis em Ω (cada um representado por um par $(x, y) \in \mathcal{X} \times \mathcal{Y}$), incluindo os pares observados do conjunto de aprendizagem $\mathcal{L}$, mas também todos os invisíveis de $\mathcal{X} \times \mathcal{Y} \setminus \mathcal{L}$. Na verdade, o objetivo não é fazer previsões mais precisas sobre o subconjunto $\mathcal{L}$ de dados conhecidos, mas sim aprender um modelo que seja correto e confiável em todos os dados possíveis.

Para classificação, a função de perda mais comum é a função de perda zero-um[4] $L(Y, \varphi_{\mathcal{L}}(X)) = 1(Y \neq \varphi_{\mathcal{L}}(X))$, onde todas as classificações incorretas são igualmente penalizadas. Neste caso, o erro de generalização de $\varphi_{\mathcal{L}}$ t orna-se a probabilidade de classificação incorreta do modelo:

$$Err(\varphi_{\mathcal{L}}) = \mathbb{E}_{X,Y}\{1(Y \neq \varphi_{\mathcal{L}}(X))\} = P(Y \neq \varphi_{\mathcal{L}}(X)) \tag{2.2}$$

Da mesma forma, para regressão, a função de perda mais utilizada é a perda de erro quadrático $L(Y, \varphi_{\mathcal{L}}(X)) = (Y - \varphi_{\mathcal{L}}(X))^2$, onde grandes diferenças entre os valores verdadeiros e os valores previstos são penalizadas mais fortemente do que pequenas. Com esta perda, o erro de generalização do modelo passa a ser:

$$Err(\varphi_{\mathcal{L}}) = \mathbb{E}_{X,Y}\{(Y - \varphi_{\mathcal{L}}(X))^2\} \tag{2.3}$$

2 A menos que fique claro no contexto, os modelos agora são denotados como $\varphi_{\mathcal{L}}$ para enfatizar que eles são construídos a partir do conjunto de aprendizagem $\mathcal{L}$.3 $\mathbb{E}_X\{f(X)\}$ denota o valor esperado de $f(x)$ (para $x \in \mathcal{X}$) em relação à distribuição de probabilidade da variável aleatória X. É definido como

$$\mathbb{E}_X\{f(X)\} = \sum_{x \in \mathcal{X}} P(X = x) f(x).$$

4 $1(condição)$ denota a função da unidade. É definido como

$$1(condition) = \begin{cases} 1 & \text{if } condition \text{ is true} \\ 0 & \text{if } condition \text{ is false} \end{cases}.$$

2.2.1 *Estimating* $\mathrm{Err}(\varphi_{\mathcal{L}})$

In practice, the probability distribution $P(X,Y)$ is usually unknown, making the direct evaluation of $\mathrm{Err}(\varphi_{\mathcal{L}})$ infeasible. Equivalently, it is often not possible to draw additional data, thereby making infeasible the empirical estimation of $\mathrm{Err}(\varphi_{\mathcal{L}})$ on a (virtually infinite) set $\mathcal{L}'$ drawn independently from $\mathcal{L}$. In most problems, $\mathcal{L}$ constitutes the only data available, on which both the model needs to be learned and its generalization error estimated. As reviewed and described on multiple occasions by several authors [Toussaint, 1974; Stone, 1978; Breiman et al., 1984; Kohavi et al., 1995; Nadeau and Bengio, 2003; Hastie et al., 2005; Arlot and Celisse, 2010], the generalization error in Equation 2.1 can however be estimated in several ways.

To make notations clearer, let us first define $\overline{E}(\varphi_{\mathcal{L}}, \mathcal{L}')$ as the average prediction error of the model $\varphi_{\mathcal{L}}$ over the set $\mathcal{L}'$ (possibly different from the learning set $\mathcal{L}$ used to produce $\varphi_{\mathcal{L}}$), that is:

$$\overline{E}(\varphi_{\mathcal{L}}, \mathcal{L}') = \frac{1}{N'} \sum_{(\mathbf{x_i}, y_i) \in \mathcal{L}'} L(y_i, \varphi_{\mathcal{L}}(\mathbf{x_i})) \tag{2.4}$$

where N' is the size of the set $\mathcal{L}'$.

The first and simplest estimate of the generalization error is the *resubstitution estimate* or *training sample estimate*. It consists in empirically estimating $\mathrm{Err}(\varphi_{\mathcal{L}})$ on the same data as the learning set $\mathcal{L}$ used to build $\varphi_{\mathcal{L}}$, that is:

$$\widehat{\mathrm{Err}}^{\mathrm{train}}(\varphi_{\mathcal{L}}) = \overline{E}(\varphi_{\mathcal{L}}, \mathcal{L}) \tag{2.5}$$

In general, the resubstitution error is a poor estimate of $\mathrm{Err}(\varphi_{\mathcal{L}})$. In particular, since most machine learning algorithms aim at precisely minimizing Equation 2.5 (either directly or indirectly), it typically results in an overly optimistic estimate of the generalization error, which accounts for all couples $(\mathbf{x}, y)$, i.e., not only those from $\mathcal{L}$.

The second approach is the *test sample estimate*. It consists in dividing the learning set $\mathcal{L}$ into two disjoint sets $\mathcal{L}_{\mathrm{train}}$ and $\mathcal{L}_{\mathrm{test}}$, called *training set* and *test set*, and then to use each part respectively for learning a model and estimating its generalization error. The test sample estimate of the generalization error of the model $\varphi_{\mathcal{L}}$ that would be obtained from $\mathcal{L}$ is then given as the average prediction error over $\mathcal{L}_{\mathrm{test}}$ of the model $\varphi_{\mathcal{L}_{\mathrm{train}}}$ built on $\mathcal{L}_{\mathrm{train}}$:

$$\widehat{\mathrm{Err}}^{\mathrm{test}}(\varphi_{\mathcal{L}}) = \overline{E}(\varphi_{\mathcal{L}_{\mathrm{train}}}, \mathcal{L}_{\mathrm{test}}) \tag{2.6}$$

As a rule-of-thumb, $\mathcal{L}_{\mathrm{train}}$ is usually taken as 70% of the samples in $\mathcal{L}$ and $\mathcal{L}_{\mathrm{test}}$ as the remaining 30%, though theoretical work [Guyon, 1997] suggests to progressively reduce the size of test set as the size of $\mathcal{L}$ increases. In any case, care must be taken when splitting $\mathcal{L}$ into two subsets, so that samples from $\mathcal{L}_{\mathrm{train}}$ can be considered independent

2.2.1 Estimativa $\mathrm{Err}(\varphi_{\mathcal{L}})$

Na prática, a distribuição de probabilidade $P(X,Y)$ geralmente é desconhecida, tornando a avaliação direta de $\mathrm{Err}(\varphi_{\mathcal{L}})$ inviável. De forma equivalente, muitas vezes não é possível extrair dados adicionais, tornando inviável a estimativa empírica de $\mathrm{Err}(\varphi_{\mathcal{L}})$ em um conjunto (virtualmente infinito) $\mathcal{L}'$desenhado independentemente de $\mathcal{L}$. Na maioria dos problemas, $\mathcal{L}$ constitui o único dado disponível, sobre o qual tanto o modelo precisa ser aprendido quanto seu erro de generalização estimado. Conforme revisado e descrito em diversas ocasiões por vários autores [Toussaint, 1974; Pedra, 1978; Breiman et al., 1984; Kohavi et al., 1995; Nadeau e Bengio, 2003; Hastie et al., 2005; Arlot e Celisse, 2010], o erro de generalização na Equação 2.1 pode, no entanto, ser estimado de várias maneiras.

Para tornar as notações mais claras, vamos primeiro definir $\overline{E}(\varphi_{\mathcal{L}}, \mathcal{L}')$ como o erro médio de predição do modelo $\varphi_{\mathcal{L}}$ sobre o conjunto $\mathcal{L}'$ (possivelmente diferente do conjunto de aprendizagem $\mathcal{L}$usado para produzir $\varphi_{\mathcal{L}}$), ou seja:

$$\overline{E}(\varphi_{\mathcal{L}}, \mathcal{L}') = \frac{1}{N'} \sum_{(\mathbf{x_i}, y_i) \in \mathcal{L}'} L(y_i, \varphi_{\mathcal{L}}(\mathbf{x_i})) \tag{2.4}$$

onde N' é o tamanho do conjunto $\mathcal{L}'$.

A primeira e mais simples estimativa do erro de generalização é a estimativa de ressubstituição ou estimativa da amostra de treinamento. Consiste em estimar empiricamente $\mathrm{Err}(\varphi_{\mathcal{L}})$ nos mesmos dados que o conjunto de aprendizagem $\mathcal{L}$ usado para construir $\varphi_{\mathcal{L}}$, ou seja:

$$\widehat{\mathrm{Err}}^{\mathrm{train}}(\varphi_{\mathcal{L}}) = \overline{E}(\varphi_{\mathcal{L}}, \mathcal{L}) \tag{2.5}$$

Em geral, o erro de substituição é uma estimativa pobre de $\mathrm{Err}(\varphi_{\mathcal{L}})$. Em particular, como a maioria dos algoritmos de aprendizado de máquina visa minimizar com precisão a Equação 2.5 (direta ou indiretamente), isso normalmente resulta em uma estimativa excessivamente otimista do erro de generalização, que é responsável por todos os pares (x, y), ou seja, não apenas aqueles de $\mathcal{L}$.

A segunda abordagem é a estimativa da amostra de teste. Consiste em dividir o conjunto de aprendizagem $\mathcal{L}$ em dois conjuntos disjuntos $\mathcal{L}_{\mathrm{train}}$ e $\mathcal{L}_{\mathrm{test}}$, chamados conjunto de treinamento e conjunto de teste, e então usar cada parte respectivamente para aprender um modelo e estimar seu erro de generalização. A estimativa da amostra de teste do erro de generalização do modelo $\varphi_{\mathcal{L}}$ que seria obtido de $\mathcal{L}$ é então dada como o erro de predição médio sobre$\mathcal{L}_{\mathrm{test}}$ do modelo$\varphi_{\mathcal{L}_{\mathrm{train}}}$ construído em $\mathcal{L}_{\mathrm{train}}$:

$$\widehat{\mathrm{Err}}^{\mathrm{test}}(\varphi_{\mathcal{L}}) = \overline{E}(\varphi_{\mathcal{L}_{\mathrm{train}}}, \mathcal{L}_{\mathrm{test}}) \tag{2.6}$$

Como regra geral, $\mathcal{L}_{\mathrm{train}}$ geralmente é considerado 70% das amostras em$\mathcal{L}$e $\mathcal{L}_{\mathrm{test}}$ como os 30% restantes, embora o trabalho teórico [Guyon,1997] sugira reduzir progressivamente o tamanho do conjunto de testes à medida que o tamanho de$\mathcal{L}$ aumenta. Em qualquer caso, deve-se tomar cuidado ao dividir $\mathcal{L}$em dois subconjuntos, para que as amostras de $\mathcal{L}_{\mathrm{train}}$ possam ser consideradas independentes

from those in $\mathcal{L}_{\text{test}}$ and drawn from the same distribution. This is however usually guaranteed by drawing $\mathcal{L}_{\text{train}}$ and $\mathcal{L}_{\text{test}}$ simply at random from $\mathcal{L}$. While being an unbiased estimate of $\text{Err}(\varphi_{\mathcal{L}})$, the test sample estimate has the drawback that it reduces the effective sample size on which the model $\varphi_{\mathcal{L}_{\text{train}}}$ is learned. If $\mathcal{L}$ is large, then this is usually not an issue, but if $\mathcal{L}$ only contains a few dozens of samples, then this strategy might not correctly approximate the true generalization error of the model that would have been learned on the entire learning set.

When $\mathcal{L}$ is small, the K-*fold cross-validation estimate* is usually preferred over the test sample estimate. It consists in randomly dividing the learning set $\mathcal{L}$ into K disjoint subsets, $\mathcal{L}_1, ..., \mathcal{L}_K$, and then to estimate the generalization error as the average prediction error over the folds $\mathcal{L}_k$ of the models $\varphi_{\mathcal{L}\setminus\mathcal{L}_k}$ learned on the remaining data:

$$\widehat{\text{Err}}^{\text{CV}}(\varphi_{\mathcal{L}}) = \frac{1}{K}\sum_{k=1}^{K}\overline{E}(\varphi_{\mathcal{L}\setminus\mathcal{L}_k}, \mathcal{L}_k) \tag{2.7}$$

The assumption behind this approach is that since each model $\varphi_{\mathcal{L}\setminus\mathcal{L}_k}$ is built using almost all $\mathcal{L}$, they should all be close to the model $\varphi_{\mathcal{L}}$ learned on the entire set. As a result the unbiased estimates $\overline{E}(\varphi_{\mathcal{L}\setminus\mathcal{L}_k}, \mathcal{L}_k)$ should also all be close to $\text{Err}(\varphi_{\mathcal{L}})$. While more computationally intensive, the K-fold cross-validation estimate has the advantage that every couple $(\mathbf{x}, y) \in \mathcal{L}$ is used for estimating the generalization error of $\varphi_{\mathcal{L}}$. In a typical setting, K is usually fixed to 10, a value often yielding stable and reliable estimates [Kohavi et al., 1995].

Expected generalization error

As we have shown, our goal is to estimate the generalization error $\text{Err}(\varphi_{\mathcal{L}})$ conditional on the learning set $\mathcal{L}$. A related quantity is the *expected* generalization error

$$\mathbb{E}_{\mathcal{L}}\{\text{Err}(\varphi_{\mathcal{L}})\}, \tag{2.8}$$

averaging over everything which is random, including the randomness in the learning set $\mathcal{L}$ used to produce $\varphi_{\mathcal{L}}$. As discussed by Hastie et al. [2005], this quantity is close, yet different from $\text{Err}(\varphi_{\mathcal{L}})$. The authors point out that most estimates, including K-fold cross-validation, effectively estimate Equation 2.8 rather than Equation 2.1.

2.2.2 *Bayes model and residual error*

In theory, when the probability distribution $P(X, Y)$ is known, the best possible model, i.e., the model φ_B which minimizes the generalization error of Equation 2.1, can be derived analytically and independently of any learning set.

daqueles em $\mathcal{L}_{\text{test}}$ e extraídos da mesma distribuição. No entanto, isso geralmente é garantido sorteando $\mathcal{L}_{\text{train}}$ e $\mathcal{L}_{\text{test}}$ simplesmente aleatoriamente de $\mathcal{L}$. Embora seja uma estimativa imparcial de $\text{Err}(\varphi_{\mathcal{L}})$, a estimativa da amostra de teste tem a desvantagem de reduzir o ta manho efetivo da amostra na qual o modelo $\varphi_{\mathcal{L}_{\text{train}}}$ é aprendido. Se $\mathcal{L}$ for grande, isso geralmente não será um problema, mas se $\mathcal{L}$ contiver ape nas algumas dezenas de amostras, essa estratégia poderá não aproximar corretamente o verdadeiro erro de generalização do modelo que teria si do aprendido em todo o conjunto de aprendizagem.

Quando $\mathcal{L}$ é pequeno, a estimativa de validação cruzada de K vezes é geralmente preferida à estimativa da amostra de teste. Consiste em dividir aleatoriamente o conjunto de aprendizagem $\mathcal{L}$ em K subconjuntos disjuntos, $\mathcal{L}_1$, ..., $\mathcal{L}_K$, e então estimar o erro de generalização como o erro médio de predição sobre as dobras $\mathcal{L}_k$ dos modelos$\varphi_{\mathcal{L}\setminus\mathcal{L}_k}$ aprendidos nos dados restantes:

$$\widehat{\text{Err}}^{\text{CV}}(\varphi_{\mathcal{L}}) = \frac{1}{K}\sum_{k=1}^{K}\overline{E}(\varphi_{\mathcal{L}\setminus\mathcal{L}_k}, \mathcal{L}_k) \tag{2.7}$$

A suposição por trás dessa abordagem é que, como cada modelo $\varphi_{\mathcal{L}\setminus\mathcal{L}_k}$é construído usando quase todos os $\mathcal{L}$, todos eles devem estar próximos do modelo$\varphi_{\mathcal{L}}$ apreendido em todo o conjunto. Como resultado, as esti mativas imparciais$E(\varphi_{\mathcal{L}\setminus\mathcal{L}_k}, \mathcal{L}_k)$ também devem estar próximas de $\text{Err}(\varphi_{\mathcal{L}})$. Embora mais intensiva em termos computacionais, a estimativa de validação cruzada K tem a vantagem de que cada par $(x, y) \in \mathcal{L}$ é usado para estimar o erro de generalização de $\varphi_{\mathcal{L}}$. Em um cenário típico, K geralmente é fixado em 10, um valor que geralmente produz estimativas estáveis e confiáveis [Kohavi et al., 1995].

Erro de generalização esperado

Como mostramos, nosso objetivo é estimar o erro de generalizaçãoErr$(\varphi_{\mathcal{L}})$ condicional ao conjunto de aprendizagem $\mathcal{L}$. Uma quantidade relacionada é o erro de generalização esperado

$$\mathbb{E}_{\mathcal{L}}\{\text{Err}(\varphi_{\mathcal{L}})\}, \tag{2.8}$$

calculando a média de tudo que é aleatório, incluindo a aleatoriedade no conjunto de aprendizagem $\mathcal{L}$usado para produzir $\varphi_{\mathcal{L}}$. Conforme discutido por Hastie et al. [2005], esta quantidade é próxima, mas diferente de $\text{Err}(\varphi_{\mathcal{L}})$. Os autores apontam que a maioria das estimativas, incluindo a validação cruzada K, estima efetivamente a Equação 2.8 em vez da Equação 2.1.

2.2.2 Modelo Bayesiano e erro residual

Em teoria, quando a distribuição de probabilidade $P(X, Y)$ é conhecida, o melhor modelo possível, ou seja, o modelo φ_B que minimiza o erro de generalização da Equação 2.1, pode ser derivado analiticamente e de forma independente.

dently of any learning set $\mathcal{L}$. By conditioning on X, the generalization error of this model can be rewritten as:

$$\mathbb{E}_{X,Y}\{L(Y, \varphi_B(X))\} = \mathbb{E}_X\{\mathbb{E}_{Y|X}\{L(Y, \varphi_B(X))\}\} \tag{2.9}$$

In this latter form, the model which minimizes Equation 2.9 is a model which minimizes the inner expectation point-wise, that is:

$$\varphi_B(\mathbf{x}) = \arg\min_{y \in \mathcal{Y}} \mathbb{E}_{Y|X=\mathbf{x}}\{L(Y, y)\} \tag{2.10}$$

In the literature, φ_B is known as the *Bayes model* and its generalization error $\text{Err}(\varphi_B)$ as the *residual error*. It represents the minimal error that any supervised learning algorithm can possibly attain, that is the irreducible error purely due to random deviations in the data.

Definition 2.7. *A model φ_B is a* Bayes model *if, for any model φ built from any learning set $\mathcal{L}$, $\text{Err}(\varphi_B) \leqslant \text{Err}(\varphi_{\mathcal{L}})$.*

In classification, when L is the zero-one loss, the Bayes model is:

$$\begin{aligned}\varphi_B(\mathbf{x}) &= \arg\min_{y \in \mathcal{Y}} \mathbb{E}_{Y|X=\mathbf{x}}\{1(Y, y)\} \\ &= \arg\min_{y \in \mathcal{Y}} P(Y \neq y | X = \mathbf{x}) \\ &= \arg\max_{y \in \mathcal{Y}} P(Y = y | X = \mathbf{x})\end{aligned} \tag{2.11}$$

Put otherwise, the best possible classifier consists in systematically predicting the most likely class $y \in \{c_1, c_2, ..., c_J\}$ given $X = \mathbf{x}$.

Similarly, for regression with the squared error loss, we have:

$$\begin{aligned}\varphi_B(\mathbf{x}) &= \arg\min_{y \in \mathcal{Y}} \mathbb{E}_{Y|X=\mathbf{x}}\{(Y - y)^2\} \\ &= \mathbb{E}_{Y|X=\mathbf{x}}\{Y\}\end{aligned} \tag{2.12}$$

In other words, the best possible regressor consists in systematically predicting the average value of Y at $X = \mathbf{x}$.

For practical problems, $P(X, Y)$ is unknown and the Bayes model cannot be derived analytically. In this context, the effectiveness of a model $\varphi_{\mathcal{L}}$ may be difficult to evaluate since (estimates of) $\text{Err}(\varphi_{\mathcal{L}})$ may not be very indicative of the goodness of $\varphi_{\mathcal{L}}$ if the lowest attainable error $\text{Err}(\varphi_B)$ is unknown. On simulated data however, where the distribution is known, deriving φ_B and $\text{Err}(\varphi_B)$ is feasible. This is beneficial as $\text{Err}(\varphi_B)$ can now be used for comparing the test set error of $\varphi_{\mathcal{L}}$ and thereby evaluate the actual effectiveness of $\varphi_{\mathcal{L}}$ with respect to the best model one can possibly build.

From a theoretical point of view, the concepts of Bayes model and residual error are also useful to study the learning capabilities of an algorithm. In particular, as $\mathcal{L}$ gets arbitrarily large, a fundamental question is to know whether it is possible to produce a model $\varphi_{\mathcal{L}}$

essencialmente de qualquer conjunto de aprendizagem $\mathcal{L}$. Ao condicionar em X, o erro de generalização deste modelo pode ser reescrito como:

$$\mathbb{E}_{X,Y}\{L(Y, \varphi_B(X))\} = \mathbb{E}_X\{\mathbb{E}_{Y|X}\{L(Y, \varphi_B(X))\}\} \tag{2.9}$$

Nesta última forma, o modelo que minimiza a Equação 2.9 é um modelo que minimiza a expectativa interna ponto a ponto, ou seja:

$$\varphi_B(\mathbf{x}) = \arg\min_{y \in \mathcal{Y}} \mathbb{E}_{Y|X=\mathbf{x}}\{L(Y, y)\} \tag{2.10}$$

Na literatura, φ_B é conhecido como modelo Bayesiano e seu erro de generalização $\text{Err}(\varphi_B)$ como erro residual. Representa o erro mínimo que qualquer algoritmo de aprendizagem supervisionada pode atingir, ou seja, o erro irredutível puramente devido a desvios aleatórios nos dados.

Definição 2.7. Um modelo φ_B é um modelo Bayesiano se, para qualquer modelo φ construído a partir de qualquer conjunto de aprendizagem $\mathcal{L}$, $\text{Err}(\varphi_B) \leqslant \text{Err}(\varphi_{\mathcal{L}})$.

Na classificação, quando L é a perda zero-um, o modelo de Bayes é:

$$\begin{aligned}\varphi_B(\mathbf{x}) &= \arg\min_{y \in \mathcal{Y}} \mathbb{E}_{Y|X=\mathbf{x}}\{1(Y, y)\} \\ &= \arg\min_{y \in \mathcal{Y}} P(Y \neq y | X = \mathbf{x}) \\ &= \arg\max_{y \in \mathcal{Y}} P(Y = y | X = \mathbf{x})\end{aligned} \tag{2.11}$$

Dito de outra forma, o melhor classificador possível consiste em prever sistematicamente a classe mais provável $y \in \{c_1, c_2, ..., c_J\}$ dado $X = \mathbf{x}$.

Da mesma forma, para regressão com perda de erro quadrático, temos:

$$\begin{aligned}\varphi_B(\mathbf{x}) &= \arg\min_{y \in \mathcal{Y}} \mathbb{E}_{Y|X=\mathbf{x}}\{(Y - y)^2\} \\ &= \mathbb{E}_{Y|X=\mathbf{x}}\{Y\}\end{aligned} \tag{2.12}$$

Em outras palavras, o melhor regressor possível consiste em prever sistematicamente o valor médio de Y em $X = \mathbf{x}$.

Para problemas práticos, $P(X, Y)$ é desconhecido e o modelo de Bayes não pode ser derivado analiticamente. Neste contexto, a eficácia de um modelo $\varphi_{\mathcal{L}}$ pode ser difícil de avaliar, uma vez que (estimativas de) $\text{Err}(\varphi_{\mathcal{L}})$ podem não ser muito indicativas da qualidade de $\varphi_{\mathcal{L}}$ se o erro mais baixo alcançável $\text{Err}(\varphi_B)$ for desconhecido. Entretanto, em dados simulados, onde a distribuição é conhecida, derivar φ_B e $\text{Err}(\varphi_B)$ é viável. Isso é benéfico, pois $\text{Err}(\varphi_B)$ agora pode ser usado para comparar o erro do conjunto de testes de $\varphi_{\mathcal{L}}$ e, assim, avaliar a eficácia real de $\varphi_{\mathcal{L}}$ em relação ao melhor modelo possível de ser construído.

Do ponto de vista teórico, os conceitos de modelo Bayesiano e erro residual também são úteis para estudar as capacidades de aprendizagem de um algoritmo. Em particular, à medida que $\mathcal{L}$ torna-se arbitrariamente grande, uma questão fundamental é saber se é possível produzir um modelo $\varphi_{\mathcal{L}}$

which is *consistent*, that is such that its generalization error gets arbitrarily close to the lowest possible generalization error $\mathrm{Err}(\varphi_B)$. Formally, consistency is defined as follows [Devroye et al., 1996]:

Definition 2.8. *A learning algorithm $\mathcal{A}$ is said to be* weakly consistent *for a certain distribution $P(X,Y)$ if $\mathbb{E}_{\mathcal{L}}\{\mathrm{Err}(\varphi_{\mathcal{L}})\} \to \mathrm{Err}(\varphi_B)$ as the size N of the learning set $\mathcal{L}$ used to build $\varphi_{\mathcal{L}}$ using $\mathcal{A}$ tends to infinity.*

Definition 2.9. *A learning algorithm $\mathcal{A}$ is said to be* strongly consistent *for a certain distribution $P(X,Y)$ if $\mathrm{Err}(\varphi_{\mathcal{L}}) \to \mathrm{Err}(\varphi_B)$ almost surely as the size N of the learning set $\mathcal{L}$ used to build $\varphi_{\mathcal{L}}$ using $\mathcal{A}$ tends to infinity.*

Note that the definition of consistency depends on the distribution $P(X,Y)$. In general, a learning algorithm $\mathcal{A}$ can be proven to be consistent for some classes of distributions, but not for others. If consistency can be proven for any distribution $P(X,Y)$, then $\mathcal{A}$ is said to be *universally* (strongly) consistent.

2.3 MODEL SELECTION

From the previous discussion in Section 2.2.2, it appears that to solve the supervised learning problem it would be sufficient to estimate $P(Y|X)$ from the learning sample $\mathcal{L}$ and then to define a model accordingly using either Equation 2.11 or Equation 2.12. Unfortunately, this approach is infeasible in practice because it requires $\mathcal{L}$ to grow exponentially with the number p of input variables in order to compute accurate estimates of $P(Y|X)$ [Geurts, 2002].

2.3.1 *Selecting the (approximately) best model*

To make supervised learning work in high-dimensional input spaces with learning sets of moderate sizes, simplifying assumptions must be made on the structure of the best model φ_B. More specifically, a supervised learning algorithm assumes that φ_B – or at least a good enough approximation – lives in a family $\mathcal{H}$ of candidate models, also known as *hypotheses* in statistical learning theory, of restricted structure. In this setting, the *model selection* problem is then defined as finding the best model among $\mathcal{H}$ on the basis of the learning set $\mathcal{L}$.

APPROXIMATION ERROR

Depending on restrictions made on the structure of the problem, the Bayes model usually does not belong to $\mathcal{H}$, but there may be models $\varphi \in \mathcal{H}$ that are sufficiently close to it. As such, the *approximation error* [Bottou and Bousquet, 2011] measures how closely the models in $\mathcal{H}$ can approximate the optimal model φ_B:

$$\mathrm{Err}(\mathcal{H}) = \min_{\varphi \in \mathcal{H}} \{\mathrm{Err}(\varphi)\} - \mathrm{Err}(\varphi_B) \tag{2.13}$$

que é consistente, isto é, tal que seu erro de generalização fica arbitrariamente próximo do menor erro de generalização possível $\mathrm{Err}(\varphi_B)$. Formalmente, a consistência é definida da seguinte forma [Devroye et al., 1996]:

Definição 2.8. Diz-se que um algoritmo de aprendizagem $\mathcal{A}$ é fracamente consistente para uma determinada distribuição $P(X,Y)$ se $\mathbb{E}_{\mathcal{L}}\{\mathrm{Err}(\varphi_{\mathcal{L}})\} \to \mathrm{Err}(\varphi_B)$ já que o tamanho N do conjunto de aprendizagem $\mathcal{L}$ usado para construir $\varphi_{\mathcal{L}}$ usando $\mathcal{A}$ tende ao infinito.

Definição 2.9. Diz-se que um algoritmo de aprendizagem $\mathcal{A}$ é fortemente consistente para uma determinada distribuição $P(X,Y)$ se $\mathrm{Err}(\varphi_{\mathcal{L}}) \to \mathrm{Err}(\varphi_B)$ é quase certo que o tamanho N do conjunto de aprendizagem $\mathcal{L}$ usado para construir $\varphi_{\mathcal{L}}$ usando $\mathcal{A}$ tende ao infinito.

Observe que a definição de consistência depende da distribuição $P(X,Y)$. Em geral, pode-se provar que um algoritmo de aprendizagem $\mathcal{A}$ é consistente para algumas classes de distribuições, mas não para outras. Se a consistência puder ser provada para qualquer distribuição $P(X,Y)$, então $\mathcal{A}$ será considerada universalmente (fortemente) consistente.

2.3 seleção de modelo

Da discussão anterior na Seção 2.2.2, parece que para resolver o problema de aprendizagem supervisionada seria suficiente estimar $P(Y|X)$ a partir da amostra de aprendizagem $\mathcal{L}$ e então definir um modelo de acordo usando a Equação 2.11 ou a Equação 2.12. Infelizmente, esta abordagem é inviável na prática porque requer $\mathcal{L}$ crescer exponencialmente com o número p de variáveis de entrada para calcular estimativas precisas de $P(Y|X)$ [Geurts, 2002].

2.3.1 Selecionando o (aproximadamente) melhor modelo

Para fazer com que a aprendizagem supervisionada funcione em espaços de entrada de alta dimensão com conjuntos de aprendizagem de tamanhos moderados, devem ser feitas suposições simplificadoras sobre a estrutura do melhor modelo φ_B. Mais especificamente, um algoritmo de aprendizagem supervisionada assume que φ_B – ou pelo menos uma aproximação suficientemente boa – reside numa família $\mathcal{H}$ de modelos candidatos, ta mbém conhecidos como hipóteses na teoria da aprendizagem estatística, de estrutura restrita. Nesta configuração, o problema de seleção de modelo é e ntão definido como encontrar o melhor modelo entre $\mathcal{H}$ com base no conjunto de aprendizagem $\mathcal{L}$.

Erro de aproximação

Dependendo das restrições feitas na estrutura do problema, o modelo Bayes geralmente não pertence a $\mathcal{H}$, mas pode haver modelos $\varphi \in \mathcal{H}$ que sejam suficientemente próximos dele. Como tal, o erro de aproximação [Bottou e Bousquet, 2011] mede o quão próximo os modelos em $\mathcal{H}$ podem aproximar-se do modelo ideal φ_B:

$$\mathrm{Err}(\mathcal{H}) = \min_{\varphi \in \mathcal{H}} \{\mathrm{Err}(\varphi)\} - \mathrm{Err}(\varphi_B) \tag{2.13}$$

To be more specific, let θ be the vector of hyper-parameters values controlling the execution of a learning algorithm $\mathcal{A}$. The application of $\mathcal{A}$ with hyper-parameters θ on the learning set $\mathcal{L}$ is a deterministic[5] process yielding a model $\mathcal{A}(\theta, \mathcal{L}) = \varphi_{\mathcal{L}} \in \mathcal{H}$. As such, our goal is to find the vector of hyper-parameters values yielding to the best model possibly learnable in $\mathcal{H}$ from $\mathcal{L}$:

$$\theta^* = \arg\min_{\theta} \operatorname{Err}(\mathcal{A}(\theta, \mathcal{L})) \tag{2.14}$$

Again, this problem cannot (usually) be solved exactly in practice since it requires the true generalization error of a model to be computable. However approximations $\widehat{\theta}^*$ can be obtained in several ways.

When $\mathcal{L}$ is large, the easiest way to find $\widehat{\theta}^*$ is to use test sample estimates (as defined by Equation 2.6) to guide the search of the hyper-parameter values, that is:

$$\widehat{\theta}^* = \arg\min_{\theta} \widehat{\operatorname{Err}}^{\text{test}}(\mathcal{A}(\theta, \mathcal{L})) \tag{2.15}$$

$$= \arg\min_{\theta} \overline{\mathrm{E}}(\mathcal{A}(\theta, \mathcal{L}_{\text{train}}), \mathcal{L}_{\text{test}}) \tag{2.16}$$

In practice, solving this later equation is also a difficult task but approximations can be obtained in several ways, e.g., using either manual tuning of θ, exhaustive exploration of the parameter space using grid search, or dedicated optimization procedures (e.g., using random search [Bergstra and Bengio, 2012]). Similarly, when $\mathcal{L}$ is scarce, the same procedure can be carried out in the exact same way but using K-fold cross-validation estimates (as defined by Equation 2.7) instead of test sample estimates. In any case, once $\widehat{\theta}^*$ is identified, the learning algorithm is run once again on the entire learning set $\mathcal{L}$, finally yielding the approximately optimal model $\mathcal{A}(\widehat{\theta}^*, \mathcal{L})$.

When optimizing θ, special care must be taken so that the resulting model is neither too simple nor too complex. In the former case, the model is indeed said to *underfit* the data, i.e., to be not flexible enough the capture the structure between X and Y. In the later case, the model is said to *overfit* the data, i.e., to be too flexible and to capture isolated structures (i.e., noise) that are specific to the learning set.

As Figure 2.1 illustrates, this phenomenon can be observed by examining the respective training and test estimates of the model with respect to its complexity[6]. When the model is too simple, both the training and test estimates are large because of underfitting. As complexity increases, the model gets more accurate and both the training and test estimates decrease. However, when the model becomes too complex, specific elements from the training set get captured, reducing the corresponding training estimates down to 0, as if the model

5 If $\mathcal{A}$ makes use of a pseudo-random generator to mimic a stochastic process, then we assume that the corresponding random seed is part of θ.

6 Unless mentioned otherwise, *complexity* here refers to the complexity of the model in function space. *Computational complexity* is studied later in Section 5.1

Para ser mais específico, seja θ o vetor de valores de hiperparâ metros que controlam a execução de um algoritmo de aprendizagem $\mathcal{A}$. A aplicação de $\mathcal{A}$ com hiperparâmetros θ no conjunto de aprendizagem $\mathcal{L}$é um processo determinístico[5]produzindo um modelo $\mathcal{A}(\theta, \mathcal{L}) = \varphi_{\mathcal{L}} \in \mathcal{H}$. Como tal, nosso objetivo é encontrar o vetor de valores de hiperparâmetros que produza o melhor modelo possivelmente aprendido em $\mathcal{H}$ a partir de $\mathcal{L}$:

$$\theta^* = \arg\min_{\theta} \operatorname{Err}(\mathcal{A}(\theta, \mathcal{L})) \tag{2.14}$$

Novamente, este problema não pode (geralmente) ser resolvido exatamente na prática, uma vez que requer que o verdadeiro erro de generalização de um modelo seja co mputável. No entanto, aproximações θ^* podem ser obtidas de diversas maneiras.

Quando $\mathcal{L}$ é grande, a maneira mais fácil de encontrar $\widehat{\theta}^*$ é usar estimativas de amostras de teste (conforme definido pela Equação 2.6) para orientar a busca dos valores dos hiperparâmetros, ou seja:

$$\widehat{\theta}^* = \arg\min_{\theta} \widehat{\operatorname{Err}}^{\text{test}}(\mathcal{A}(\theta, \mathcal{L})) \tag{2.15}$$

$$= \arg\min_{\theta} \overline{\mathrm{E}}(\mathcal{A}(\theta, \mathcal{L}_{\text{train}}), \mathcal{L}_{\text{test}}) \tag{2.16}$$

Na prática, resolver esta última equação também é uma tarefa difícil, mas as aproximações podem ser obtidas de várias maneiras, por exemplo, usando ajuste manual de θ, exploração exaustiva do espaço de parâmetros usando pesquisa em grade ou procedimentos de otimização dedicados (por exemplo, usando pesquisa aleatória [Bergstra e Bengio, 2012]). Da mesma forma, quando $\mathcal{L}$ é escasso, o mesmo procedimento pode ser realizado exatamente da mesma maneira, mas usando estimativas de validação cruzada K-fold (conforme definido pela Equação 2.7) $\widehat{}$em vez de estimativas de amostra de teste. Em qualquer caso, uma vez identificado θ^*, o algoritmo de aprendizagem é executado novamente em todo o conjunto de aprendizagem $\mathcal{L}$, $\widehat{}$ produzindo finalmente o modelo aproximadamente ideal $\mathcal{A}(\theta^*, \mathcal{L})$.

Ao otimizar θ, deve-se tomar cuidado especial para que o modelo resultante não seja muito simples nem muito complexo. No primeiro caso, diz-se que o modelo não se ajusta aos dados, ou seja, não é flexível o suficiente para capturar a estrutura entre X e Y. No último caso, diz-se que o modelo superajusta os dados, ou seja, é muito flexível e captura estruturas isoladas (ou seja, ruído) que são específicas do conjunto de aprendizagem.

Como ilustra a Figura 2.1, este fenômeno pode ser observado examinando as respectivas estimativas de treinamento e teste do modelo em relação à sua complexidade[6]. Quando o modelo é muito simples, as estimativas de treinamento e teste são grandes devido ao ajuste insuficiente. À medida que a complexidade aumenta, o modelo fica mais preciso e as estimativas de treinamento e teste diminuem. Entretanto, quando o modelo se torna muito complexo, elementos específicos do conjunto de treinamento são capturados, reduzindo as estimativas de treinamento correspondentes a 0, como se o modelo

5 Se $\mathcal{A}$ faz uso de um gerador pseudo-aleatório para imitar um processo estocástico, então assumimos que a semente aleatória correspondente faz parte de θ. 6 Salvo indicação em contrário, complexidade aqui se refere à complexidade do modelo no espaço funcional. A complexidade computacional é estudada posteriormente na Seção 5.1

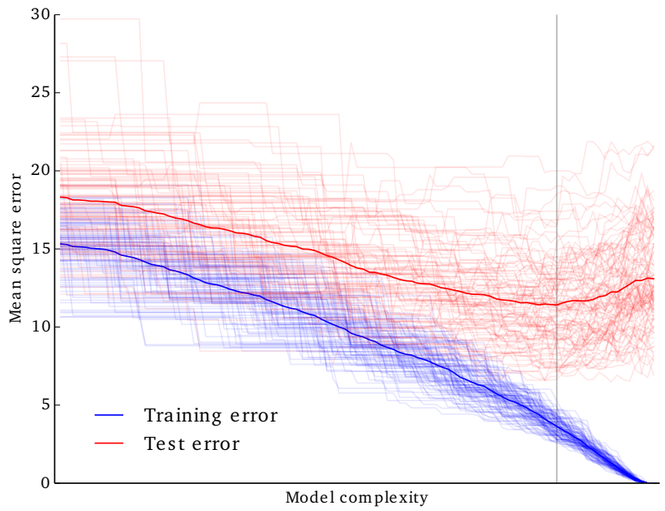


Figure 2.1: Training and test error with respect to the complexity of a model. The light blue curves show the training error over $\mathcal{L}_{\text{train}}$ while the light red curves show the test error estimated over $\mathcal{L}_{\text{test}}$ for 100 pairs of training and test sets $\mathcal{L}_{\text{train}}$ and $\mathcal{L}_{\text{test}}$ drawn at random from a known distribution. The thick blue curve is the average training error while the thick red curve is the average test error. (Figure inspired from [Hastie et al., 2005].)

were perfect. At the same time, the test estimates become worse because the structure learned from the training set is actually too specific and does not generalize. The model is overfitting. The best hyperparameter value θ is therefore the one making the appropriate trade-off and producing a model which is neither too simple nor to complex, as shown by the gray line on the figure.

As we will see later in Chapter 4, overfitting can also be explained by decomposing the generalization error in terms of bias and variance. A model which is too simple usually has high bias but low variance, while a model which is too complex usually has low bias but high variance. In those terms, finding the best model amounts to make the appropriate *bias-variance trade-off*.

2.3.2 *Selecting and evaluating simultaneously*

On real applications, and in particular when reporting results, it usually happens that one wants to do both model selection and model assessment. That is, having chosen a final model $\mathcal{A}(\widehat{\theta}^*, \mathcal{L})$, one wants to also estimate its generalization error.

A naive assessment of the generalization error of the selected model might be to simply use the test sample estimate $\widehat{\overline{\text{Err}}}^{\text{test}}(\mathcal{A}(\widehat{\theta}^*, \mathcal{L}))$ (i.e., $\overline{\text{E}}(\mathcal{A}(\widehat{\theta}^*, \mathcal{L}_{\text{train}}), \mathcal{L}_{\text{test}})$) that was minimized during model selection. The

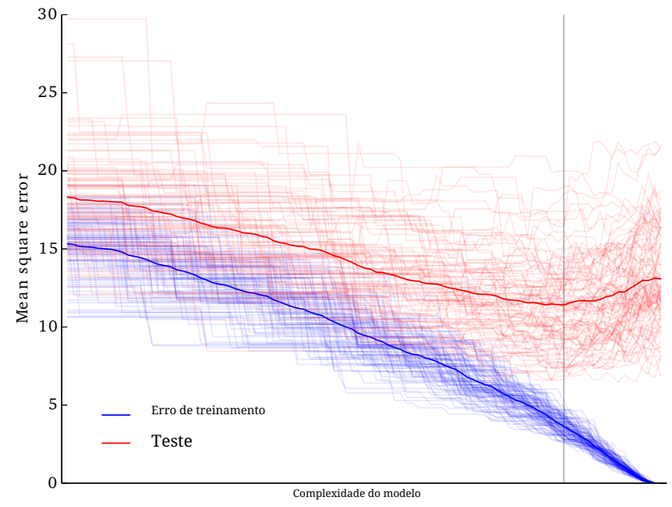


Figura 2.1: Erro de treinamento e teste em relação à complexidade de um modelo. As curvas em azul claro mostram o erro de treinamento em $\mathcal{L}_{\text{train}}$ enquanto as curvas em vermelho claro mostram o erro de teste estimado em $\mathcal{L}_{\text{test}}$ para 100 pares de conjuntos de treinamento e teste $\mathcal{L}_{\text{train}}$ e $\mathcal{L}_{\text{test}}$ extraídos aleatoriamente de uma distribuição conhecida. A curva azul espessa é o erro médio de treinamento, enquanto a curva vermelha espessa é o erro médio de teste. (Figura inspirada em [Hastie et al., 2005].)

eram perfeitos. Ao mesmo tempo, as estimativas dos testes pioram porque a estrutura aprendida no conjunto de treinamento é, na verdade, muito específica e não generaliza. O modelo está sobreajustado. O melhor valor do hiperparâmetro θ é, portanto, aquele que faz a troca apropriada e produz um modelo que não é nem muito simples nem muito complexo, como mostra a linha cinza na figura.

Como veremos mais adiante no Capítulo 4, o sobreajuste também pode ser explicado pela decomposição do erro de generalização em termos de viés e variância. Um modelo muito simples geralmente tem alto viés, mas baixa variância, enquanto um modelo muito complexo geralmente tem baixo viés, mas alta variância. Nesses termos, encontrar o melhor modelo equivale a fazer a compensação apropriada entre viés e variância.

2.3.2 Selecionando e avaliando simultaneamente

Em aplicações reais, e em particular ao relatar resultados, geralmente acontece que se deseja fazer tanto a seleção quanto a avaliação do modelo. Ou seja, tendo escolhido um modelo final $\mathcal{A}(\theta^*, \mathcal{L})$, deseja-se também estimar o seu erro de generalização.

Uma avaliação ingênua do erro de generalização do modelo selecionado pode ser simplesmente usar a estimativa da amostra de teste $\widehat{\overline{\text{Err}}}^{\text{test}}(\mathcal{A}(\widehat{\theta}^*, \mathcal{L}))$ (ou seja, $\widehat{\text{E}}(\mathcal{A}(\theta^*, \mathcal{L}_{\text{train}}), \mathcal{L}_{\text{test}})$) que foi minimizada durante a seleção do modelo. O

issue with this estimate is that the learned model is not independent from $\mathcal{L}_{\text{test}}$ since its repeated construction was precisely guided by the minimization of the prediction error over $\mathcal{L}_{\text{test}}$. As a result, the minimized test sample error is in fact a biased, optimistic, estimate of the true generalization error, sometimes leading to substantial underestimations. For the same reasons, using the K-fold cross-validation estimate does not provide a better estimate since model selection was similarly guided by the minimization of this quantity.

To guarantee an unbiased estimate, the test set on which the generalization error is evaluated should ideally be kept out of the entire model selection procedure and only be used once the final model is selected. Algorithm 2.1 details such a protocol. Similarly, per-fold estimates of the generalization error should be kept out of the model selection procedure, for example using nested cross-validation within each fold, as explicited in Algorithm 2.2.

Algorithm 2.1. *Train-Valid-Test set protocol for both model selection and evaluation.*

(a) *Divide the learning set $\mathcal{L}$ into three parts $\mathcal{L}_{train}$, $\mathcal{L}_{valid}$ and $\mathcal{L}_{test}$;*

(b) *Perform model selection on $\mathcal{L}_{train} \cup \mathcal{L}_{valid}$ using test sample estimates, i.e., find:*

$$\widehat{\theta}^* = \arg\min_{\theta} \widehat{\text{Err}}^{test}(\mathcal{A}(\theta, \mathcal{L}_{train} \cup \mathcal{L}_{valid})) \tag{2.17}$$

$$= \arg\min_{\theta} \overline{\text{E}}(\mathcal{A}(\theta, \mathcal{L}_{train}), \mathcal{L}_{valid}); \tag{2.18}$$

(c) *Evaluate the (unbiased) generalization error of the final model as*

$$\overline{\text{E}}(\mathcal{A}(\widehat{\theta}^*, \mathcal{L}_{train} \cup \mathcal{L}_{valid}), \mathcal{L}_{test}); \tag{2.19}$$

(d) *Learn the final model $\mathcal{A}(\widehat{\theta}^*, \mathcal{L})$ on the entire learning set.*

Algorithm 2.2. *Nested K-fold cross-validation protocol for both model selection and evaluation.*

(a) *Divide the learning set $\mathcal{L}$ into K folds $\mathcal{L}_1, ..., \mathcal{L}_K$;*

(b) *For each fold $k = 1, ..., K$:*

i. *Divide $\mathcal{L} \setminus \mathcal{L}_k = \mathcal{L}^{-k}$ into K folds $\mathcal{L}_1^{-k}, ..., \mathcal{L}_K^{-k}$;*

ii. *Perform model selection on the subset $\mathcal{L}^{-k}$ using nested K-fold estimates, i.e., find:*

$$\widehat{\theta}_k^* = \arg\min_{\theta} \widehat{\text{Err}}^{CV}(\mathcal{A}(\theta, \mathcal{L}^{-k})) \tag{2.20}$$

$$= \arg\min_{\theta} \frac{1}{K} \sum_{l=1}^{K} \overline{\text{E}}(\mathcal{A}(\theta, \mathcal{L}^{-k} \setminus \mathcal{L}_l^{-k}), \mathcal{L}_l^{-k}); \tag{2.21}$$

O problema com esta estimativa é que o modelo aprendido não é independente de $\mathcal{L}_{\text{test}}$, uma vez que sua construção repetida foi precisamente guiada pela minimização do erro de predição sobre $\mathcal{L}_{\text{test}}$. Como resultado, o erro minimizado da amostra de teste é, na verdade, uma estimativa tendenciosa e optimista do verdadeiro erro de generalização, conduzindo por vezes a subestimações substanciais. Pelas mesmas razões, usar a estimativa de validação cruzada K não fornece uma estimativa melhor, uma vez que a seleção do modelo foi igualmente guiada pela minimização desta quantidade.

Para garantir uma estimativa imparcial, o conjunto de testes no qual o erro de generalização é avaliado deve idealmente ser mantido fora de to do o procedimento de seleção do modelo e só ser usado quando o modelo final for selecionado. O Algoritmo 2.1 detalha tal protocolo. Da mesma forma, as estimativas por dobra do erro de generalização devem ser mantidas fora do procedimento de seleção do modelo, por exemplo, usando validação cruzada aninhada dentro de cada dobra, conforme explicitado no Algoritmo 2.2.

Algoritmo 2.1. Protocolo do conjunto Train-Valid-Test para seleção e avaliação de modelos.

(a) Divida o conjunto de aprendizagem $\mathcal{L}$ em três partes $\mathcal{L}_{train}$, $\mathcal{L}_{valid}$ e $\mathcal{L}_{test}$;

(b) Realize a seleção do modelo em $\mathcal{L}_{train} \cup \mathcal{L}_{valid}$ usando estimativas de amostra de teste, ou seja, encontre:

$$\widehat{\theta}^* = \arg\min_{\theta} \widehat{\text{Err}}^{test}(\mathcal{A}(\theta, \mathcal{L}_{train} \cup \mathcal{L}_{valid})) \tag{2.17}$$

$$= \arg\min_{\theta} \overline{\text{E}}(\mathcal{A}(\theta, \mathcal{L}_{train}), \mathcal{L}_{valid}); \tag{2.18}$$

(c) Avalie o erro de generalização (imparcial) do modelo final como

$$\overline{\text{E}}(\mathcal{A}(\widehat{\theta}^*, \mathcal{L}_{train} \cup \mathcal{L}_{valid}), \mathcal{L}_{test}); \tag{2.19}$$

(d) Aprenda o modelo final $\mathcal{A}(\theta^*, \mathcal{L})$ em todo o conjunto de aprendizagem.

Algoritmo 2.2. Protocolo de validação cruzada K-fold aninhado para seleção e avaliação de modelos.

(a) Divida o conjunto de aprendizagem $\mathcal{L}$ em K dobras $\mathcal{L}_1, ..., \mathcal{L}_K$;

(b) Para cada dobra $k = 1, ..., K$:

i. Divide $\mathcal{L} \setminus \mathcal{L}_k = \mathcal{L}^{-k}$ into K folds $\mathcal{L}_1^{-k}, ..., \mathcal{L}_K^{-k}$;

ii. Execute a seleção do modelo no subconjunto $\mathcal{L}^{-k}$ usando estimativas aninhadas K-fold, ou seja, encontre:

$$\widehat{\theta}_k^* = \arg\min_{\theta} \widehat{\text{Err}}^{CV}(\mathcal{A}(\theta, \mathcal{L}^{-k})) \tag{2.20}$$

$$= \arg\min_{\theta} \frac{1}{K} \sum_{l=1}^{K} \overline{\text{E}}(\mathcal{A}(\theta, \mathcal{L}^{-k} \setminus \mathcal{L}_l^{-k}), \mathcal{L}_l^{-k}); \tag{2.21}$$

iii. Evaluate the generalization error of the selected sub-model as

$$\overline{\mathrm{E}}(\mathcal{A}(\widehat{\theta}^*_k, \mathcal{L} \setminus \mathcal{L}_k), \mathcal{L}_k); \tag{2.22}$$

(c) Evaluate the (unbiased) generalization error of the selected model as the average generalization estimate of the sub-models selected over the folds:

$$\frac{1}{K}\sum_{k=1}^{K} \overline{\mathrm{E}}(\mathcal{A}(\widehat{\theta}^*_k, \mathcal{L} \setminus \mathcal{L}_k), \mathcal{L}_k); \tag{2.23}$$

(d) Perform model selection on the entire learning set $\mathcal{L}$ using K-fold cross-validation estimates, i.e., find:

$$\widehat{\theta}^* = \arg\min_{\theta} \widehat{\mathrm{Err}}^{\mathrm{CV}}(\mathcal{A}(\theta, \mathcal{L})) \tag{2.24}$$

$$= \arg\min_{\theta} \frac{1}{K}\sum_{k=1}^{K} \overline{\mathrm{E}}(\mathcal{A}(\theta, \mathcal{L} \setminus \mathcal{L}_k), \mathcal{L}_k); \tag{2.25}$$

(e) Learn the final model $\mathcal{A}(\widehat{\theta}^*, \mathcal{L})$ on the entire learning set.

2.4 CLASSES OF LEARNING ALGORITHMS

Before proceeding in the next chapters, and for the rest of this work, to an in-depth analysis of the class of tree-based methods, we briefly review in this section some of the other learning algorithms that have matured in the field, including linear methods, support vector machines, neural networks and nearest neighbor methods.

2.4.1 *Linear methods*

One of the oldest class of supervised learning algorithms is the class of *linear methods* from the field of statistics. In these methods, the central assumption made on $\mathcal{H}$ is that the output variable Y can be described as a linear combination of the input variables $X_1, ..., X_p$ (i.e., as an hyperplane), or at least that the linear model is a reasonable approximation. For regression, $\mathcal{H}$ includes all models φ of the form:

$$\varphi(\mathbf{x}) = b + \sum_{j=1}^{p} x_j w_j \tag{2.26}$$

For binary classification (i.e., when $\mathcal{Y} = \{c_1, c_2\}$), $\mathcal{H}$ includes all models φ of the form:

$$\varphi(\mathbf{x}) = \begin{cases} c_1 & \text{if } b + \sum_{j=1}^{p} x_j w_j > 0 \\ c_2 & \text{otherwise} \end{cases} \tag{2.27}$$

iii. Avalie o erro de generalização do submodelo selecionado como

$$\overline{\mathrm{E}}(\mathcal{A}(\widehat{\theta}^*_k, \mathcal{L} \setminus \mathcal{L}_k), \mathcal{L}_k); \tag{2.22}$$

(c) Avalie o erro de generalização (imparcial) do modelo selecionado como a estimativa média de generalização dos submodelos selecionados nas dobras:

$$\frac{1}{K}\sum_{k=1}^{K} \overline{\mathrm{E}}(\mathcal{A}(\widehat{\theta}^*_k, \mathcal{L} \setminus \mathcal{L}_k), \mathcal{L}_k); \tag{2.23}$$

(d) Realize a seleção do modelo em todo o conjunto de aprendizagem $\mathcal{L}$ usando estimativas de validação cruzada K, ou seja, encontre:

$$\widehat{\theta}^* = \arg\min_{\theta} \widehat{\mathrm{Err}}^{\mathrm{CV}}(\mathcal{A}(\theta, \mathcal{L})) \tag{2.24}$$

$$= \arg\min_{\theta} \frac{1}{K}\sum_{k=1}^{K} \overline{\mathrm{E}}(\mathcal{A}(\theta, \mathcal{L} \setminus \mathcal{L}_k), \mathcal{L}_k); \tag{2.25}$$

(e) Aprenda o modelo final $\mathcal{A}(\theta^*, \mathcal{L})$ em todo o conjunto de aprendizagem.

2.4 classes de algoritmos de aprendizagem

Antes de prosseguir nos próximos capítulos, e no restante deste trabalho, para uma análise aprofundada da classe de métodos baseados em árvores, revisamos brevemente nesta seção alguns dos outros algoritmos de aprendizagem que amadureceram na área, incluindo métodos lineares, máquinas de vetores de suporte, redes neurais e métodos de vizinhos mais próximos.

2.4.1 Métodos lineares

Uma das classes mais antigas de algoritmos de aprendizagem supervisionada é a classe de métodos lineares da área de estatística. Nestes métodos, a suposição central feita em $\mathcal{H}$ é que a variável de saída Y pode ser descrita como uma combinação linear das variáveis de entrada X_1, ..., X_p (ou seja, como um hiperplano), ou pelo menos que o modelo linear é uma aproximação razoável. Para regressão, $\mathcal{H}$ inclui todos os modelos φ da forma:

$$\varphi(\mathbf{x}) = b + \sum_{j=1}^{p} x_j w_j \tag{2.26}$$

Para classificação binária (ou seja, quando $\mathcal{Y} = \{c_1, c_2\}$), $\mathcal{H}$ inclui todos os modelos φda forma:

$$\varphi(\mathbf{x}) = \begin{cases} c_1 & \text{if } b + \sum_{j=1}^{p} x_j w_j > 0 \\ c_2 & \text{otherwise} \end{cases} \tag{2.27}$$

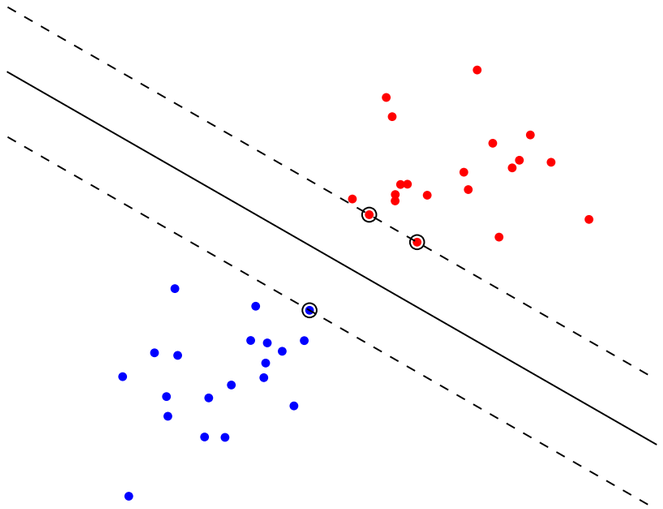


Figure 2.2: A good separating hyperplane is an hyperplane that maximizes the distance to the nearest training data points.

Linear methods come in many flavors but mostly differ from each other in the way they estimate the coefficients b and w_j (for $j = 1, \ldots, p$), usually using some specific optimization procedure to minimize a specific criterion. Among all of them, the most famous is the method of *least squares*. It consists in finding the b and w_j coefficients that minimize the resubstitution estimate (Equation 2.5) using the squared error loss.

Despite an apparent simplicity, linear methods often provide reliable predictions and an interpretable description of how the input variables affect the output. Contrary to their name, linear methods can also be used to model non-linear relations between X and Y, for example by applying these methods on (non-linear) transformations of the input variables. For more details on (generalized) linear methods, see the reviews of MacCullagh and Nelder [1989], Hastie et al. [2005], Bishop and Nasrabadi [2006] or Duda et al. [2012].

2.4.2 *Support vector machines*

When data points from the learning set are linearly separable, there exist several hyperplanes (i.e., several linear models) that are in fact equally as good when evaluated in resubstitution. In generalization however, these hyperplanes are usually not equivalent. As illustrated in Figure 2.2, a good separation is intuitively achieved when the distance (also known as the *margin*) to the nearest training data points is as large as possible, since in general the larger the margin the lower the generalization error of the model.

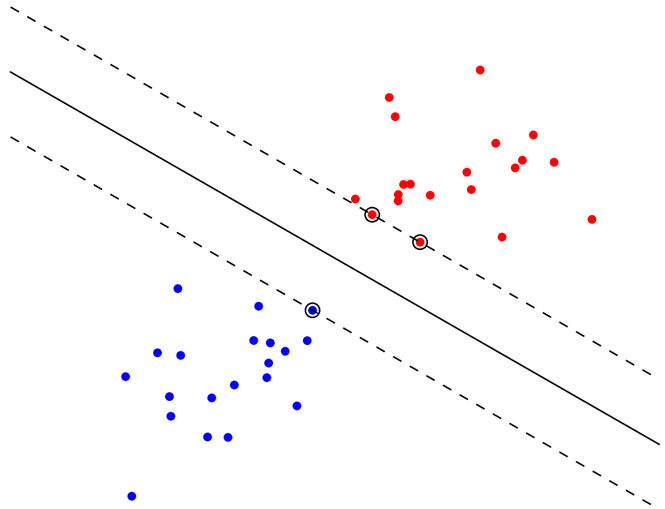


Figura 2.2: Um bom hiperplano de separação é aquele que maximiza a distância até os pontos de dados de treinamento mais próximos.

Os métodos lineares vêm em muitos sabores, mas diferem principalmente entre si na maneira como estimam os coeficientes b e w_j (para $j = 1,\ldots,p$), geralmente usando algum procedimento de otimização específico para minimizar um critério específico. Dentre todos eles, o mais famoso é o método dos mínimos quadrados. Consiste em encontrar os coeficientes b e w_j que minimizam a estimativa de substituição (Equação 2.5) utilizando a perda de erro quadrático.

Apesar de uma aparente simplicidade, os métodos lineares muitas vezes fornecem previsões confiáveis e uma descrição interpretável de como as variáveis de entrada afetam o resultado. Ao contrário do seu nome, os métodos lineares também podem ser usados para modelar relações não lineares entre X e Y, por exemplo, aplicando estes métodos em transformações (não lineares) das variáveis de entrada. Para obter mais detalhes sobre métodos lineares (generalizados), consulte as revisões de MacCullagh e Nelder [1989],Hastie et al.[2005], Bishop e Nasrabadi [2006] ou Dudaetal. [2012].

2.4.2 Máquinas de vetores de suporte

Quando os pontos de dados do conjunto de aprendizagem são linear mente separáveis, existem vários hiperplanos (ou seja, vários modelos lineares) que são de fato igualmente bons quando avaliados em resubstituição. Em generalização, entretanto, esses hiperplanos geralmente não são equivalentes. Conforme ilustrado na Figura 2.2, uma boa separação é alcançada intuitivamente quando a distância (também conhecida como margem) até os pontos de dados de treinamento mais próximos é a maior possível, pois em geral quanto maior a margem, menor o erro de generalização do modelo.

Mathematically, support vector machines [Boser et al., 1992; Cortes and Vapnik, 1995] are maximum-margin linear models of the form of Equation 2.27. Assuming without loss of generality that $\mathcal{Y} = \{-1, 1\}$ and that $b = 0$, support vector machines are learned by solving the following primal optimization problem:

$$\min_{\mathbf{w},\xi} \left\{ \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N} \xi_i \right\} \tag{2.28}$$

subject to

$$y_i(\mathbf{w} \cdot \mathbf{x_i}) \geqslant 1 - \xi_i, \quad \xi_i \geqslant 0. \tag{2.29}$$

In its dual the form, the optimization problem is

$$\max_{\alpha} \left\{ \sum_{i=1}^{N} \alpha_i - \frac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j \right\} \tag{2.30}$$

subject to

$$0 \leqslant \alpha_i \leqslant C, \tag{2.31}$$

where C is an hyper-parameter that controls the degree of misclassification of the model, in case classes are not linearly separable. From the solution of dual problem, we have

$$\mathbf{w} = \sum_{i=1}^{N} \alpha_i y_i \mathbf{x}_i, \tag{2.32}$$

from which the final linear model can finally be expressed.

Support vector machines extend to non-linear classification by projecting the original input space into a high-dimensional space (the so-called *kernel trick*), where a separating hyperplane can hopefully be found. Interestingly, the dual optimization problem is exactly the same, except that the dot product $\mathbf{x}_i \cdot \mathbf{x}_j$ is replaced by a *kernel* $K(\mathbf{x}_i, \mathbf{x}_j)$, which corresponds the dot product of $\mathbf{x}_i$ and $\mathbf{x}_j$ in the new space.

2.4.3 *Neural networks*

The family of *neural networks* methods finds its origins in attempts to identify mathematical representations of information processing in biological systems. While this objective is still far from being reached, (artificial) neural networks have nonetheless proven all their worth from a statistical point of view, and have actually grown into one of the most effective methods in machine learning.

A neural network is usually made of several units, also known as *neurons*, of the form

$$h_j(\mathbf{x}) = \sigma(w_j + \sum_{i=1}^{n} w_{ij} x_i), \tag{2.33}$$

Matematicamente, máquinas de vetores de suporte [Boser et al., 1992; Cortes e Vapnik, 1995] são modelos lineares de margem máxima na forma da Equação 2.27. Assumindo, sem perda de generalidade, que $\mathcal{Y} = \{-1, 1\}$ e que $b = 0$, máquinas de vetores de suporte são aprendidas resolvendo o seguinte problema de otimização primal:

$$\min_{\mathbf{w},\xi} \left\{ \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N} \xi_i \right\} \tag{2.28}$$

sujeito a

$$y_i(\mathbf{w} \cdot \mathbf{x_i}) \geqslant 1 - \xi_i, \quad \xi_i \geqslant 0. \tag{2.29}$$

Na sua forma dual, o problema de otimização é

$$\max_{\alpha} \left\{ \sum_{i=1}^{N} \alpha_i - \frac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j \right\} \tag{2.30}$$

sujeito a

$$0 \leqslant \alpha_i \leqslant C, \tag{2.31}$$

onde C é um hiperparâmetro que controla o grau de classificação incorreta do modelo, caso as classes não sejam linearmente separáveis. Da solução do problema duplo, temos

$$\mathbf{w} = \sum_{i=1}^{N} \alpha_i y_i \mathbf{x}_i, \tag{2.32}$$

a partir do qual o modelo linear final pode finalmente ser expresso.

Máquinas de vetores de suporte estendem a classificação não linear projetando o espaço de entrada original em um espaço de alta dimensão (o chamado truque do kernel), onde um hiperplano de separação pode ser encontrado. Curiosamente, o problema de otimização dupla é exatamente o mesmo, exceto que o produto escalar $x_i \cdot x_j$ é substituído por um kernel $K(x_i, x_j)$, que corresponde ao produto escalar de x_i e x_j no novo espaço.

2.4.3 Redes Neurais

A família de métodos de redes neurais tem sua origem nas tentativas de identificar representações matemáticas do processamento de informações em sistemas biológicos. Embora este objetivo ainda esteja longe de ser alcançado, as redes neurais (artificiais) provaram, no entanto, todo o seu valor do ponto de vista estatístico e tornaram-se, na verdade, num dos métodos mais eficazes de aprendizagem automática.

Uma rede neural geralmente é composta por várias unidades, também conhecidas como neurônios, na forma

$$h_j(\mathbf{x}) = \sigma(w_j + \sum_{i=1}^{n} w_{ij} x_i), \tag{2.33}$$

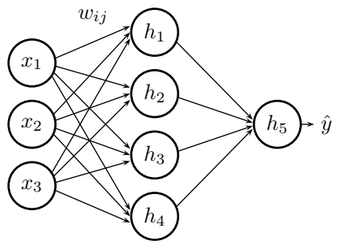


Figure 2.3: An artificial neural network.

where σ is a non-linear activation function, such as the sign function, the sigmoid function or the softmax activation. In most cases, these units are structured into successive layers, where the outputs of a layer are directed through weighted connections, also known as *synapses*, to the inputs of the next layer. As an example, Figure 2.3 illustrates a three layered neural network. The first layer is the input layer, which transmits the input values $\mathbf{x} = (x_1, ..., x_p)$ to the second layer. The second layer is made of activation units h_j, taking as inputs the weighted values of the input layer and producing non-linear transformations as outputs. The third layer is made of a single activation unit, taking as inputs the weighted outputs of the second layer and producing the predicted value $\hat{y}$. Assuming that this structure is fixed and that all units from the network make use of the same activation function σ, the hypothesis space $\mathcal{H}$ therefore includes all models φ of the form

$$\varphi(\mathbf{x}) = \sigma(w_5 + \sum_{j=1}^{4} w_{j5}\sigma(w_j + \sum_{i=1}^{p} w_{ij}x_i)). \tag{2.34}$$

As for linear methods, learning a neural network amounts to estimate the weights w_{ij} that minimize some specific loss function, using some specific optimization procedure. Among all of them, the most famous is the *backpropagation algorithm* [Bryson, 1975].

Recent advances on neural networks, now themed as *deep learning*, have recently shown the capability of these models to autonomously learn high-level and very effective representations of data. On various difficult tasks, such as image classification or speech recognition, neural networks have indeed achieved outstanding performance, outperforming both human operators and state-of-the-art methods. From a theoretical point of view however, little is still known about what makes them truly work. In particular, picking the appropriate combination of number of units, layers and types of activation functions remains a delicate task, which unfortunately makes (deep) neural networks still difficult to use for non-experts. Reviews on recent developments include the works of Hinton [2007], Arel et al. [2010] or Bengio et al. [2013].

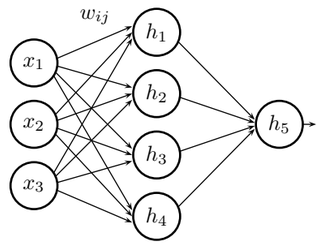


Figura 2.3: Uma rede neural artificial.

onde σé uma função de ativação não linear, como a função de sinal, a função sigmóide ou a ativação softmax. Na maioria dos casos, estas unidades são estruturadas em camadas sucessivas, onde as saídas de uma camada são direcionadas através de conexões ponderadas, também conhecidas como sinapses, para as entradas da próxima camada. Como exemplo, a Figura 2.3 ilustra uma rede neural de três camadas. A pri meira camada é a camada de entrada, que transmite os valores de entrada $\mathbf{x} = (x_1, ..., x_p)$ para a segunda camada. A segunda camada é composta por unidades de ativação h_j, tomando como entradas os valores ponderados da camada de entrada e produzindo transformações não lineares como saídas. A terceira camada é composta por uma única unidade de ativação, tomando como entradas as saídas ponderadas da segunda camada e produzindo o valor previsto $\hat{y}$. Supondo que esta estrutura seja fixa e que todas as unidades da rede utilizem a mesma função de ativação σ, o espaço de hipóteses $\mathcal{H}$ inclui, portanto, todos os modelosφda forma

$$\varphi(\mathbf{x}) = \sigma(w_5 + \sum_{j=1}^{4} w_{j5}\sigma(w_j + \sum_{i=1}^{p} w_{ij}x_i)). \tag{2.34}$$

Quanto aos métodos lineares, aprender uma rede neural equivale a estimar os pesosw_{ij} que minimizam alguma função de perda específica, usando algum procedimento de otimização específico. Dentre todos eles, o mais famoso é o algoritmo de retropropagação [Bryson, 1975].

Avanços recentes em redes neurais, agora denominadas aprendizagem profunda, mostraram recentemente a capacidade desses modelos de aprender de forma autônoma representações de dados de alto nível e muito eficazes. Em diversas tarefas difíceis, como classificação de imagens ou reconhecimento de fala, as redes neurais alcançaram de fato um desempenho excepcional, superando tanto os operadores humanos quanto os métodos de última geração. Do ponto de vista teórico, porém, pouco se sabe ainda sobre o que os faz realmente funcionar. Em particular, escolher a combinação apropriada de número de unidades, camadas e tipos de funções de ativação continua sendo uma tarefa delicada, o que infelizmente torna as redes neurais (profundas) ainda difíceis de usar para não especialistas. Revisões sobre desenvolvimentos recentes incluem os trabalhos de Hinton [2007], Arel et al. [2010] ou Bengio et al. [2013].

2.4.4 *Nearest neighbor methods*

Nearest neighbor methods belong to a class of non-parametric algorithms known as *prototype methods* [Hastie et al., 2005]. They distinguish from other learning algorithms in the sense that they are memory-based and require no model to be fit.

The principle behind nearest neighbor methods is to find a number of training samples closest in distance to the new sample, and then infer from these the value of the output variable. For regression, the k-nearest neighbor algorithm [Fix and Hodges, 1951] averages the output values from the k closest training samples, that is:

$$\varphi(\mathbf{x}) = \frac{1}{k} \sum_{(\mathbf{x}_i, y_i) \in \mathrm{NN}(\mathbf{x}, \mathcal{L}, k)} y_i, \tag{2.35}$$

where $\mathrm{NN}(\mathbf{x}, \mathcal{L}, k)$ denotes the k nearest neighbors of $\mathbf{x}$ in $\mathcal{L}$. For classification, the procedure is the same except that the predicted output value is computed as the majority class among the k nearest neighbors:

$$\varphi(\mathbf{x}) = \arg\max_{c \in \mathcal{Y}} \sum_{(\mathbf{x}_i, y_i) \in \mathrm{NN}(\mathbf{x}, \mathcal{L}, k)} 1(y_i = c). \tag{2.36}$$

In general, the distance function used to identify the k nearest neighbors can be any metric, but the standard Euclidean distance is the most common choice. A notable variant is the *radius-based neighbor algorithm*, in which the predicted output value is computed from the training samples within a radius r of the new sample. In cases where data is not uniformly sampled, this latter algorithm can be a better choice than the k-nearest neighbor method.

Despite their simplicity, nearest neighbor methods usually yield decent results in practice. They are are often successful in classification situations where the decision boundary is very irregular. From a theoretical point of view, an important result due to Cover and Hart [1967] is the proof of consistency of the method. When the size of $\mathcal{L}$ tends to infinity and k is appropriately adjusted to $\mathcal{L}$, the generalization error of the model produced by the k-nearest neighbor method is shown to converge towards the generalization error of the Bayes model.

2.4.4 Métodos do vizinho mais próximo

Os métodos do vizinho mais próximo pertencem a uma classe de algoritmos não paramétricos conhecidos como métodos protótipos [Hastie et al., 2005]. Eles se distinguem de outros algoritmos de aprendizagem no sentido de que são baseados em memória e não requerem nenhum modelo para serem ajustados.

O princípio por trás dos métodos do vizinho mais próximo é encontrar um número de amostras de treinamento mais próximas da nova amostra e, em seguida, inferir a partir delas o valor da variável de saída. Para regressão, o algoritmo k-vizinho mais próximo [Fix e Hodges, 1951] faz a média dos valores de saída das k amostras de treinamento mais próximas, ou seja:

$$\varphi(\mathbf{x}) = \frac{1}{k} \sum_{(\mathbf{x}_i, y_i) \in \mathrm{NN}(\mathbf{x}, \mathcal{L}, k)} y_i, \tag{2.35}$$

onde $\mathrm{NN}(\mathbf{x}, \mathcal{L}, k)$ denota os k vizinhos mais próximos de $\mathbf{x}$ em $\mathcal{L}$. Para classificação, o procedimento é o mesmo, exceto que o valor de saída previsto é calculado como a classe majoritária entre os k vizinhos mais próximos:

$$\varphi(\mathbf{x}) = \arg\max_{c \in \mathcal{Y}} \sum_{(\mathbf{x}_i, y_i) \in \mathrm{NN}(\mathbf{x}, \mathcal{L}, k)} 1(y_i = c). \tag{2.36}$$

Em geral, a função de distância usada para identificar os k vizinhos mais próximos pode ser qualquer métrica, mas a distância euclidiana padrão é a escolha mais comum. Uma variante notável é o algoritmo vizinho baseado em raio, no qual o valor de saída previsto é calculado a partir das amostras de treinamento dentro de um raio r da nova amostra. Nos casos em que os dados não são amostrados uniformemente, este último algoritmo pode ser uma escolha melhor do que o método do vizinho mais próximo k.

Apesar da sua simplicidade, os métodos do vizinho mais próximo geralmente produzem resultados decentes na prática. Eles são frequentemente bem sucedidos em situações de classificação onde o li mite de decisão é muito irregular. Do ponto de vista teórico, um resultado importante devido a Cover e Hart [1967] é a prova de consistência do método. Quando o tamanho de $\mathcal{L}$ tende ao infinito e k é apropriadamente ajustado para $\mathcal{L}$, o erro de generalização do modelo produzido pelo método do vizinho mais próximo k converge para o erro de generalização do modelo de Bayes.

3 CLASSIFICATION AND REGRESSION TREES

Outline

In this chapter, we present a unified framework in which we detail (single) decision trees methods. In Section 3.1, we first give an overview of the context in which these algorithms have been developed. In Section 3.2, we proceed with a mathematical presentation, introducing all necessary concepts and notations. The general learning algorithm is then presented in Section 3.3 while specific parts of the algorithm are discussed in finer details in Sections 3.4, 3.5, 3.6. As such, specific decision tree algorithms (e.g., CART, ID3 or C4.5) are described as specializations of the general framework presented here.

3.1 INTRODUCTION

Since always, artificial intelligence has been driven by the ambition to understand and uncover complex relations in data. That is, to find models that can not only produce accurate predictions, but also be used to extract knowledge in an intelligible way. Guided with this twofold objective, research in machine learning has given rise to extensive bodies of works in a myriad of directions. Among all of them however, tree-based methods stand as one of the most effective and useful method, capable to produce both reliable and understandable results, on mostly any kind of data.

Historically, the appearance of *decision trees* is due to Morgan and Sonquist [1963], who first proposed a tree-based method called *automatic interaction detector* (AID) for handling multi-variate non-additive effects in the context of survey data. Building upon AID, methodological improvements and computer programs for exploratory analysis were then proposed in the following years by several authors [Sonquist, 1970; Messenger and Mandell, 1972; Gillo, 1972; Sonquist et al., 1974]. Without contest however, the principal investigators that have driven research on the modern methodological principles are Breiman [1978a,b], Friedman [1977, 1979] and Quinlan [1979, 1986] who simultaneously and independently proposed very close algorithms for the induction of tree-based models. Most notably, the unifying work of Breiman et al. [1984], later complemented with the work of Quinlan [1993], have set decision trees into a simple and consistent methodological framework, which largely contributed in making them easy to understand and easy to use by a large audience.

Contorno

Neste capítulo, apresentamos uma estrutura unificada na qual detalha mos métodos de árvores de decisão (únicas). Na Seção 3.1, primeiro da mos uma visão geral do contexto em que esses algoritmos foram desenvolvidos. Na Seção 3.2 procedemos com uma apresentação matemática, introduzindo todos os conceitos e notações necessárias. O algoritmo geral de aprendizagem é então apresentado na Seção 3.3, enquanto partes específicas do algoritmo são discutidas em detalhes nas Seções 3.4, 3.5, 3.6. Como tal, algoritmos específicos de árvore de decisão (por exemplo, CART, ID3 ou C4.5) são descritos como especializações da estrutura geral apresentada aqui.

3.1 introdução

Desde sempre, a inteligência artificial tem sido impulsionada pela a mbição de compreender e descobrir relações complexas nos dados. Ou seja, encontrar modelos que possam não apenas produzir previsões precisas, mas também serem usados para extrair conhecimento de forma inteligível. Guiada por esse duplo objetivo, a pesquisa em aprendizado de máquina deu origem a extensos conjuntos de trabalhos em uma i nfinidade de direções. Entre todos eles, no entanto, os métodos baseados em árvore se destacam como um dos métodos mais eficazes e úteis, capazes de produzir resultados confiáveis e compreensíveis, principalmente em qualquer tipo de dados.

Historicamente, o aparecimento de árvores de decisão deve-se a Morgan e Sonquist [1963], , que primeiro propuseram um método baseado em árvore chamado detector automático de interação (AID) para lidar co m efeitos não aditivos multivariados no contexto de dados de pesquisa. Com base na AID, melhorias metodológicas e programas de computador para análise exploratória foram propostos nos anos seguintes por vários autores [Sonquist, 1970; Mensageiro e Mandell, 1972; Gillo, 1972; Sonquist et al.,1974]. Sem contestação, no entanto, os principais investigadores que impulsionaram a pesquisa sobre os princípios metodológicos modernos são Breiman[1978a,b],Friedman [1977, 1979]e Quinlan [1979, 1986] que, simultânea e independentemente, propuseram algoritmos muito próxi mos para a indução de modelos baseados em árvores. Mais notavelmente, o trabalho unificador de Breiman et al. [1984], mais tarde complementados com o trabalho de Quinlan[1993], configuraram árvores de decisão em uma estrutura metodológica simples e consistente, o que contribuiu amplamente para torná-las fáceis de entender e usar por um grande público.

As we will explore in further details all throughout this work, the success of decision trees (and by extension, of all tree-based methods) is explained by several factors that make them quite attractive in practice:

- Decision trees are non-parametric. They can model arbitrarily complex relations between inputs and outputs, without any a priori assumption;
- Decision trees handle heterogeneous data (ordered or categorical variables, or a mix of both);
- Decision trees intrinsically implement feature selection, making them robust to irrelevant or noisy variables (at least to some extent);
- Decision trees are robust to outliers or errors in labels;
- Decision trees are easily interpretable, even for non-statistically oriented users.

Most importantly, decision trees are at the foundation of many modern and state-of-the-art algorithms, including forests of randomized trees (on which this work is about, see Chapter 4) or boosting methods [Freund and Schapire, 1995; Friedman, 2001], where they are used as building blocks for composing larger models. Understanding all algorithmic details of single decision trees is therefore a mandatory prerequisite for an in-depth analysis of these methods.

3.2 TREE STRUCTURED MODELS

When the output space is a finite set of values, like in classification where $\mathcal{Y} = \{c_1, c_2, ..., c_J\}$, another way of looking at a supervised learning problem is to notice that Y defines a partition over the universe Ω, that is

$$\Omega = \Omega_{c_1} \cup \Omega_{c_2} \cup \cdots \cup \Omega_{c_J}, \tag{3.1}$$

where Ω_{c_k} is the of set objects for which Y has value c_k. Similarly, a classifier φ can also be regarded as a partition of the universe Ω since it defines an approximation $\widehat{Y}$ of Y. This partition however is defined on the input space $\mathcal{X}$ rather that directly on Ω, that is

$$\mathcal{X} = \mathcal{X}^{\varphi}_{c_1} \cup \mathcal{X}^{\varphi}_{c_2} \cup ... \cup \mathcal{X}^{\varphi}_{c_J}, \tag{3.2}$$

where $\mathcal{X}^{\varphi}_{c_k}$ is the set of description vectors $\mathbf{x} \in \mathcal{X}$ such that $\varphi(\mathbf{x}) = c_k$. Accordingly, learning a classifier can be restated as learning a partition of $\mathcal{X}$ matching as closely as possible the best possible partition, i.e., the one engendered by the Bayes model φ_B over $\mathcal{X}$:

$$\mathcal{X} = \mathcal{X}^{\varphi_B}_{c_1} \cup \mathcal{X}^{\varphi_B}_{c_2} \cup ... \cup \mathcal{X}^{\varphi_B}_{c_J}. \tag{3.3}$$

Como exploraremos com mais detalhes ao longo deste trabalho, o sucesso das árvores de decisão (e, por extensão, de todos os métodos baseados em árvores) é explicado por vários fatores que as tornam bastante atrativas na prática:

- As árvores de decisão não são paramétricas. Podem modelar relações arbitrariamente complexas entre insumos e produtos, sem qualquer suposição a priori; - Árvores de decisão lidam com dados heterogêneos (variáveis ordenadas ou categóricas, ou uma mistura de ambas); - As árvores de decisão implementam intrinsecamente a seleção de características, tornando-as robustas a variáveis irrelevantes ou ruidosas (pelo menos até certo ponto); - As árvores de decisão são robustas a outliers ou erros nos rótulos; - As árvores de decisão são facilmente interpretáveis, mesmo para utilizadores não orientados para a estatística.

Mais importante ainda, as árvores de decisão estão na base de muitos algoritmos modernos e de última geração, incluindo florestas de árvores aleatórias (sobre as quais este trabalho trata, ver Capítulo 4) ou métodos de boosting [Freund e Schapire, 1995; Friedman, 2001], onde são usados como blocos de construção para compor modelos maiores. Compreender todos os detalhes algorítmicos de árvores de decisão únicas é, portanto, um pré-requisito obrigatório para uma análise aprofundada destes métodos.

3.2 modelos estruturados em árvore

Quando o espaço de saída é um conjunto finito de valores, como na classificação onde $\mathcal{Y} = \{c_1, c_2, ..., c_J\}$, outra maneira de olhar para um problema de aprendizagem supervisionada é notar que Y define uma partição sobre o universoΩ, ou seja

$$\Omega = \Omega_{c_1} \cup \Omega_{c_2} \cup \cdots \cup \Omega_{c_J}, \tag{3.1}$$

onde Ω_{c_k} é o conjunto de objetos para os quais Y tem valor c_k. Da mesma forma, um classificador φ também pode ser considerado uma partição do universo Ω, uma vez que $\widehat{\ }$ define uma aproximação Y de Y. Esta partição, entretanto, é definida no espaço de entrada $\mathcal{X}$ e não diretamente em Ω, ou s eja

$$\mathcal{X} = \mathcal{X}^{\varphi}_{c_1} \cup \mathcal{X}^{\varphi}_{c_2} \cup ... \cup \mathcal{X}^{\varphi}_{c_J}, \tag{3.2}$$

onde $\mathcal{X}^{\varphi}_{c_k}$ é o conjunto de vetores de descrição $\mathbf{x} \in \mathcal{X}$ tais que $\varphi(\mathbf{x}) = c_k$. Conseqüentemente, aprender um classificador pode ser reafirmado como aprender uma partição de $\mathcal{X}$ correspondente o mais próximo possível à melhor partição possível, ou seja, aquela gerada pelo modelo de Bayes φ_B sobre $\mathcal{X}$:

$$\mathcal{X} = \mathcal{X}^{\varphi_B}_{c_1} \cup \mathcal{X}^{\varphi_B}_{c_2} \cup ... \cup \mathcal{X}^{\varphi_B}_{c_J}. \tag{3.3}$$

Partitioning with noise

Notice that when Y cannot be univocally determined given $X = \mathbf{x}$, e.g., when there is noise on Y, then there may exist two distinct objects from the universe Ω such that their representations $\mathbf{x}_1$ and $\mathbf{x}_2$ in the input space are equal, yet such that the corresponding output values y_1 and y_2 are different. In other words, the subsets

$$\mathcal{X}_{c_k}^{\Omega} = \{\mathbf{x}_i | i \in \Omega, Y = c_k\} \tag{3.4}$$

may not be disjoint. By contrast, since φ defines a function from $\mathcal{X}$ to $\mathcal{Y}$, any input $\mathbf{x} \in \mathcal{X}$ is mapped to exactly one output value $y \in \mathcal{Y}$ and the subsets $\mathcal{X}_{c_k}^{\varphi}$ are therefore necessarily disjoint, which means that no model will ever perfectly predict the true output value in all cases. As discussed in Section 2.2.2, this limitation is unavoidable and can in fact be viewed as the cause of the residual error.

From a geometrical point of view, the principle of tree structured models is beautifully simple. It consists in approximating the partition of the Bayes model by recursively partitioning the input space $\mathcal{X}$ into subspaces and then assign constant prediction values $\hat{y} \in \mathcal{Y}$ to all objects $\mathbf{x}$ within each terminal subspace. To make things clearer, let us first define the following concepts:

Definition 3.1. *A* tree *is a graph* $G = (V, E)$ *in which any two vertices (or* nodes*) are connected by exactly one path.*

Definition 3.2. *A* rooted tree *is a tree in which one of the nodes has been designated as the* root*. In our case, we additionally assume that a rooted tree is a* directed *graph, where all edges are directed away from the root.*

Definition 3.3. *If there exists an edge from* t_1 *to* t_2 *(i.e., if* $(t_1, t_2) \in E$*) then node* t_1 *is said to be the* parent *of node* t_2 *while node* t_2 *is said to be a* child *of node* t_1*.*

Definition 3.4. *In a rooted tree, a node is said to be* internal *if it has one or more children and* terminal *if it has no children. Terminal nodes are also known as* leaves*.*

Definition 3.5. *A* binary tree *is a rooted tree where all internal nodes have exactly two children.*

In those terms, a *tree-structured model* (or *decision tree*) can be defined as a model $\varphi : \mathcal{X} \mapsto \mathcal{Y}$ represented by a rooted tree (often binary, but not necessarily), where any node t represents a subspace $\mathcal{X}_t \subseteq \mathcal{X}$ of the input space, with the root node t_0 corresponding to $\mathcal{X}$ itself. Internal nodes t are labeled with a *split* s_t taken from a set of questions $\mathcal{Q}$. It divides the space $\mathcal{X}_t$ that node t represents into disjoint subspaces respectively corresponding to each of its children. For instance, the set of all binary splits is the set $\mathcal{Q}$ of questions s of the form *"Does*

Particionamento com ruído

Observe que quando Y não pode ser determinado univocamente dado $X = x$, por exemplo, quando há ruído em Y, então podem existir dois objetos distintos do universo Ω tais que suas representações x_1 e x_2 no espaço de entrada são iguais, mas tais que os valores de saída correspondentesy_1 e y_2 são diferentes. Em outras palavras, os subconjuntos

$$\mathcal{X}_{c_k}^{\Omega} = \{\mathbf{x}_i | i \in \Omega, Y = c_k\} \tag{3.4}$$

pode não ser disjunto. Por outro lado, como φ define uma função de $\mathcal{X}$ para$\mathcal{Y}$, qualquer entrada $x \in \mathcal{X}$ é mapeada para exatamente um valor de saída $y \in \mathcal{Y}$ e os subconjuntos $\mathcal{X}_{c_k}^{\varphi}$ são, portanto, necessariamente disjuntos, o que significa que nenhum modelo jamais preverá perfeitamente o verdadeiro valor de saída em todos os casos. Conforme discutido na Seção 2.2.2, esta limitação é inevitável e pode de fato ser vista como a causa do erro residual.

Do ponto de vista geométrico, o princípio dos modelos estruturados em árvore é maravilhosamente simples. Consiste em aproximar a partição do modelo Bayes particionando recursivamente o espaço de entrada $\mathcal{X}$em subespaços e então atribuindo valores de previsão constantes $\hat{y} \in \mathcal{Y}$ a todos os objetos x dentro de cada subespaço terminal. Para deixar as c oisas mais claras, vamos primeiro definir os seguintes conceitos:

Definição 3.1. Uma árvore é um grafo $G = (V, E)$ *no qual quaisquer dois vértices (ou nós) estão conectados por exatamente um caminho.*

Definição 3.2. Uma árvore enraizada é uma árvore na qual um dos nós foi designado como raiz. No nosso caso, assumimos adicionalmente que uma árvore enraizada é u m grafo direcionado, onde todas as arestas são direcionadas para longe da raiz.

Definição 3.3. Se existir uma aresta de t_1 *a* t_2 *(ou seja, se* $(t_1, t_2) \in E$*), então o nó* t_1 *é considerado pai do nó* t_2 *enquanto o nó* t_2 *é considerado filho do nó* t_1*.*

Definição 3.4. Em uma árvore enraizada, um nó é considerado interno se tiver um ou mais filhos e terminal se não tiver filhos. Os nós terminais também são conhecidos como folhas.

Definição 3.5. Uma árvore binária é uma árvore enraizada onde todos os nós internos têm exatamente dois filhos.

Nesses termos, um modelo estruturado em árvore (ou árvore de decisão) pode ser definido como um modelo $\varphi : \mathcal{X} \mapsto \mathcal{Y}$ representado por uma árvore enraizada (geralmente binária, mas não necessariamente), onde qualquer nó t representa um subespaço $\mathcal{X}_t \subseteq \mathcal{X}$ do espaço de entrada, com o nó raiz t_0 correspondendo a $\mathcal{X}$ próprio. Os nós internos t são rotulados com uma divisão s_t retirada de um conjunto de perguntas $\mathcal{Q}$. Ele divide o espaço $\mathcal{X}_t$ que o nó t representa em subespaços disjuntos correspondentes respectivamente a cada um de seus filhos. Por exemplo, o conjunto de todas as divisões binárias é o conjunto $\mathcal{Q}$ de perguntas s na forma "Será que

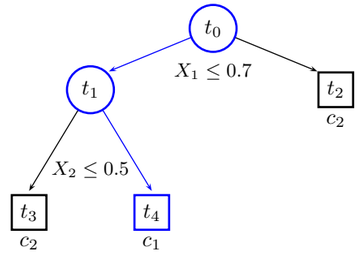


Figure 3.1: A decision tree φ built for a binary classification problem from an input space $\mathcal{X} = [0,1] \times [0,1]$. (Figure inspired from Breiman et al. [1984].)

$\mathbf{x} \in \mathcal{X}_A$?", where $\mathcal{X}_A \subset \mathcal{X}$ is some subset of the input space. Any split s of this form divides $\mathcal{X}_t$ into two subspaces respectively corresponding to $\mathcal{X}_t \cap \mathcal{X}_A$ for the left child of t and to $\mathcal{X}_t \cap (\mathcal{X} \setminus \mathcal{X}_A)$ for the right child of t. Terminal nodes are labeled with a best guess value $\hat{y}_t \in \mathcal{Y}$ of the output variable. If φ is a classification tree, then $\hat{y}_t \in \{c_1, ..., c_J\}$ while if φ is a regression tree, then $\hat{y}_t \in \mathbb{R}$. As such, the predicted output value $\varphi(\mathbf{x})$ is the label of the leaf reached by the instance $\mathbf{x}$ when it is propagated through the tree by following the splits s_t (see Algorithm 3.1).

Algorithm 3.1. *Prediction of the output value $\hat{y} = \varphi(\mathbf{x})$ in a decision tree.*

1: **function** PREDICT($\varphi, \mathbf{x}$)
2: $t = t_0$
3: **while** t is not a terminal node **do**
4: $t =$ the child node t' of t such that $\mathbf{x} \in \mathcal{X}_{t'}$
5: **end while**
6: **return** $\hat{y}_t$
7: **end function**

As an example, Figure 3.1 illustrates a decision tree φ made of five nodes and partitioning the input space $\mathcal{X} = \mathcal{X}_1 \times \mathcal{X}_2 = [0;1] \times [0;1]$ for a binary classification problem (where $\mathcal{Y} = \{c_1, c_2\}$). Node t_0 is the root node and corresponds to the whole input space $\mathcal{X}_{t_0} = \mathcal{X}$. It is labeled with the binary split $X_1 \leqslant 0.7$ (i.e., the question *"Is $X_1 \leqslant 0.7$?"*) which divides $\mathcal{X}_{t_0}$ into two disjoint subsets $\mathcal{X}_{t_1} \cup \mathcal{X}_{t_2}$. The first set corresponds to its left child t_1 and represents the set of all input vectors $\mathbf{x} \in \mathcal{X}_0$ such that $x_1 \leqslant 0.7$. Similarly, the second set corresponds to its right child t_2 and represents the set of all input vectors $\mathbf{x} \in \mathcal{X}_{t_0}$ such that $x_1 > 0.7$. Likewise, t_1 is labeled with the split $X_2 \leqslant 0.5$ which further divides $\mathcal{X}_{t_1}$ into two disjoint subsets $\mathcal{X}_{t_3} \cup \mathcal{X}_{t_4}$ respectively corresponding to the sets of all input vectors $\mathbf{x} \in \mathcal{X}_{t_1}$ such that $x_2 \leqslant 0.5$ and $x_2 > 0.5$. Terminal nodes t_2, t_3 and t_4 are represented by squares labeled with an output value $\hat{y}_t$. They form together a partition (as defined by Equation 3.2) of $\mathcal{X}$, where each set $\mathcal{X}_{c_k}^{\varphi}$ is obtained from the union of the subspaces $\mathcal{X}_t$ of all

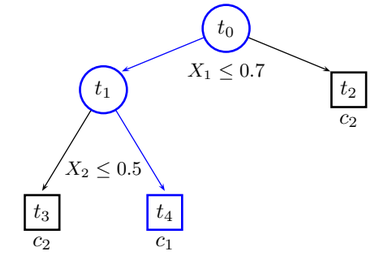


Figura 3.1: Uma árvore de decisão φ construída para um problema de classificação binária a partir de um espaço de entrada $\mathcal{X} = [0,1] \times [0,1]$. (Figura inspirada em Breiman et al. [1984].)

$x \in \mathcal{X}_A$?", onde $\mathcal{X}_A \subset \mathcal{X}$ é algum subconjunto do espaço de entrada. Qualquer divisãos desta forma divide $\mathcal{X}_t$ em dois subespaços correspondentes respectivamente a $\mathcal{X}_t \cap \mathcal{X}_A$ para o filho esquerdo de t e a $\mathcal{X}_t \cap (\mathcal{X} \setminus \mathcal{X}_A)$ para o filho direito de t. Os nós terminais são rotulados com um valor de melhor estimativa $\hat{y}_t \in \mathcal{Y}$da variável de saída. Se φ é uma árvore de classificação, então $\hat{y}_t \in \{c_1, ..., c_J\}$enquanto se φ é uma árvore de regressão, então $\hat{y}_t \in \mathbb{R}$ Como tal, o valor de saída previsto $\varphi(x)$ é o rótulo da folha alcançado pela instância x quando ela é propagada através da árvore seguindo as divisões s_t (ver Algoritmo 3.1).

Algoritmo 3.1. Previsão do valor de saída $\hat{y} = \varphi(x)$ em uma árvore de decisão.

1: function Predict(φ,x) 2: $t = t_0$**3: while** t **não é um nó terminal faça 4:** $t =$ **o nó filho** t' **de** t **tal que** $\mathbf{x} \in \mathcal{X}_{t'}$ **5: end while 6: return** $\hat{y}_t$**7: end function**

Como exemplo, a Figura 3.1 ilustra uma árvore de decisão φ composta de cinco nós e particionando o espaço de entrada $\mathcal{X} = \mathcal{X}_1 \times \mathcal{X}_2 = [0;1] \times [0;1]$ para um problema de classificação binária (onde $\mathcal{Y} = \{c_1, c_2\}$). O nó$t_0$ é o nó raiz e corresponde a todo o espaço de entrada $\mathcal{X}_{t_0} = \mathcal{X}$. É rotulado com a divisão binária $X_1 \leqslant 0.7$ (ou seja, a pergunta "É$X_1 \leqslant 0.7$?") que divide $\mathcal{X}_{t_0}$ em dois subconjuntos disjuntos $\mathcal{X}_{t_1} \cup \mathcal{X}_{t_2}$. O primeiro conjunto corresponde ao seu filho esquerdo t_1 e representa o conjunto de todos os vetores de entrada $\mathbf{x} \in \mathcal{X}_0$ tais que $x_1 \leqslant 0.7$. Da mesma forma, o segundo conjunto corresponde ao seu filho direito t_2 e representa o conjunto de todos os vetores de entrada $\mathbf{x} \in \mathcal{X}_{t_0}$ tais que $x_1 > 0.7$. Da mesma forma, t_1 é rotulado com a divisão $X_2 \leqslant 0.5$ que divide ainda $\mathcal{X}_{t_1}$ em dois subconjuntos disjuntos$\mathcal{X}_{t_3} \cup \mathcal{X}_{t_4}$ correspondendo respectivamente aos conjuntos de todos os vetores de entrada $\mathbf{x} \in \mathcal{X}_{t_1}$ tais que $x_2 \leqslant 0.5$ e $x_2 > 0.5$. Os nós terminais t_2, t_3 et$_4$ são representados por quadrados rotulados com um valor de saída $\hat{y}_t$. Eles formam juntos uma partição (conforme definido pela Equação 3.2) de $\mathcal{X}$, onde cada conjunto $\mathcal{X}_{c_k}^{\varphi}$ é obtido a partir da união dos subespaços $\mathcal{X}_t$ de todos

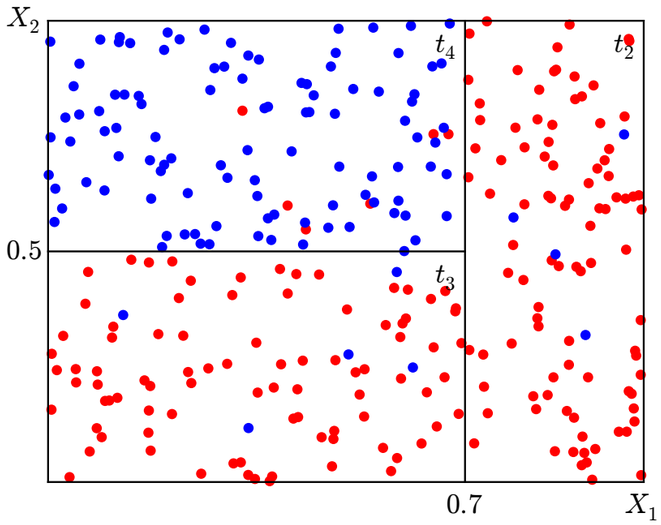


Figure 3.2: Partition of $\mathcal{X}$ induced by the decision tree φ into homogeneous subspaces. Blue dots correspond to objects of class c_1 while red dots correspond to objects of class c_2. (Figure inspired from Breiman et al. [1984].)

terminal nodes t such that $\hat{y}_t = c_k$. In this case, $\mathcal{X}_{c_1}^{\varphi} = \mathcal{X}_{t_4}$ while $\mathcal{X}_{c_2}^{\varphi} = \mathcal{X}_{t_2} \cup \mathcal{X}_{t_3}$. As shown in Figure 3.2, the partition induced by φ on $\mathcal{X}$ divides the input space into subspaces that are more and more class homogeneous, starting from $\mathcal{X}$ at the root node, then $\mathcal{X}_{t_1} \cup \mathcal{X}_{t_2}$ at the second level of tree and finally $(\mathcal{X}_{t_3} \cup \mathcal{X}_{t_4}) \cup \mathcal{X}_{t_2}$ at the leaves. As we will explore in Section 3.6, the partition is in this case made of rectangles because of the nature of the splits $s_t \in Q$ dividing the nodes. Predictions are made by propagating instances through the tree and using as output value the value labeling the terminal nodes in which they fall into. For example, as shown in blue in Figure 3.1, an object $\mathbf{x} = (x_1 = 0.2, x_2 = 0.7)$ falls into t_4 and therefore $\varphi(\mathbf{x}) = \hat{y}_{t_4} = c_1$.

Finally, let us note that from a graph theory point of view, decision trees belong to a larger family of methods known as *induction graphs* [Zighed and Rakotomalala, 2000]. In this more general framework, the structure is a directed acyclic graph, which allows for nodes to be both divided and recombined.

3.3 INDUCTION OF DECISION TREES

Learning a decision tree ideally amounts to determine the tree structure producing the partition which is closest to the partition engendered by Y over $\mathcal{X}$. Since it is unknown, the construction of a decision tree is usually driven instead with the objective of finding a model

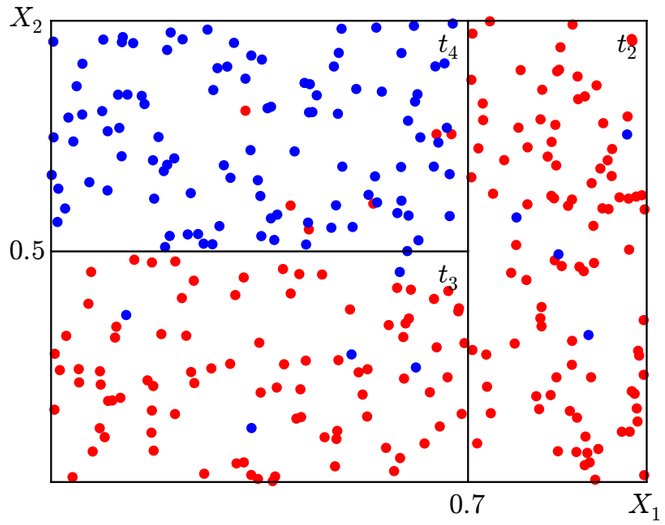


Figura 3.2:Partição de $\mathcal{X}$ induzida pela árvore de decisão φ em subespaços homogêneos. Os pontos azuis correspondem a objetos da classe c_1 enquanto os pontos vermelhos correspondem a objetos da classe c_2. (Figura inspirada em Breiman et al. [1984].)

nós terminais t de modo que $\hat{y}_t = c_k$. Neste caso, $\mathcal{X}_{c_1}^{\varphi} = \mathcal{X}_{t_4}$ enquanto $\mathcal{X}_{c_2}^{\varphi} = \mathcal{X}_{t_2} \cup \mathcal{X}_{t_3}$. Como mostrado na Figura 3.2, a partição induzida por φ em $\mathcal{X}$ divide o espaço de entrada em subespaços que são cada vez mais homogêneos de classe, começando em $\mathcal{X}$ no nó raiz, depois $\mathcal{X}_{t_1} \cup \mathcal{X}_{t_2}$no segundo nível da árvore e finalmente $(\mathcal{X}_{t_3} \cup \mathcal{X}_{t_4}) \cup \mathcal{X}_{t_2}$ nas folhas. Como exploraremos na Seção 3.6, neste caso a partição é feita de retângulos devido à natureza das divisões $s_t \in Q$ que dividem os nós. As previsões são feitas propagando instâncias através da árvore e usando como valor de saída o valor que rotula os nós terminais nos quais elas se enquadram. Por exemplo, conforme mostrado em azul na Figura 3.1, um objeto $x = (x_1 = 0.2, x_2 = 0.7)$ cai em t_4 e, portanto, $\varphi(\mathbf{x}) =_{\sim} y_{t_4} = c_1$.

Finalmente, observemos que do ponto de vista da teoria dos grafos, as árvores de decisão pertencem a uma família maior de métodos conhecidos como gráficos de indução [Zighed e Rakotomalala, 2000]. Neste quadro mais geral, a estrutura é um gráfico acíclico direcionado, que permite que os nós sejam divididos e recombinados.

3.3 indução de árvores de decisão

Aprender uma árvore de decisão idealmente equivale a determinar a estrutura da árvore que produz a partição que está mais próxima da partição gerada por Y sobre $\mathcal{X}$. Por ser desconhecido, a construção de uma árvore de decisão geralmente é conduzida com o objetivo de encontrar um modelo

which partitions the learning set $\mathcal{L}$ as well as possible. Among all decision trees $\varphi \in \mathcal{H}$ however, there may exist several of them that explain $\mathcal{L}$ equally best. Following Occam's Razor principles [Blumer et al., 1987] of preferring the explanation which makes as few assumptions as possible, that is to favor the simplest solution that fits the data, learning a decision tree from $\mathcal{L}$ is therefore usually restated as finding the smallest tree φ^* (in terms of internal nodes) minimizing its resubstitution estimate $\overline{E}(\varphi^*, \mathcal{L})$. While this assumption makes sense from a generalization point of view, it also makes sense regarding interpretability. A decision tree which is small is easier to understand than a large and complex tree.

As shown by Hyafil and Rivest [1976], finding the smallest tree φ^* that minimizes its resubstitution estimate is an NP-complete problem. As a consequence, under the assumption that $P \neq NP$, there exists no efficient algorithm for finding φ^*, thereby suggesting that finding efficient heuristics for constructing near-optimal decision trees is the best solution to keep computation requirements within realistic boundaries.

Following the framework of Breiman et al. [1984], let us broadly define an *impurity* measure $i(t)$ as a function that evaluates the goodness of any node t (we delay to Section 3.6 for a more precise definition). Let assume that the smaller $i(t)$, the *purer* the node and the better the predictions $\hat{y}_t(\mathbf{x})$ for all $\mathbf{x} \in \mathcal{L}_t$, where $\mathcal{L}_t$ is the subset of learning samples falling into t, that is all $(\mathbf{x}, y) \in \mathcal{L}$ such that $\mathbf{x} \in \mathcal{X}_t$. Starting from a single node representing the whole learning set $\mathcal{L}$, near-optimal decision trees can then be grown greedily by iteratively dividing nodes into purer nodes. That is, by iteratively dividing into smaller subsets the subsets of $\mathcal{L}$ represented by the nodes, until all terminal nodes cannot be made purer, hence guaranteeing near-optimal predictions over $\mathcal{L}$. The greedy assumption to make the resulting tree as small as possible, thereby seeking for good generalization, is then to divide each node t using the split s^* that locally maximizes the decrease of impurity of the resulting child nodes.

Formally, the decrease of impurity of a binary split s is defined as follows:

Definition 3.6. *The* impurity decrease *of a binary split* $s \in Q$ *dividing node* t *into a left node* t_L *and a right node* t_R *is*

$$\Delta i(s, t) = i(t) - p_L i(t_L) - p_R i(t_R) \tag{3.5}$$

where p_L *(resp.,* p_R*) is the proportion* $\frac{N_{t_L}}{N_t}$ *(resp.,* $\frac{N_{t_R}}{N_t}$*) of learning samples from* $\mathcal{L}_t$ *going to* t_L *(resp., to* t_R*) and where* N_t *is the size of the subset* $\mathcal{L}_t$*.*

On the basis of this concept, a general greedy procedure for the induction of decision trees can now be described formally as outlined in Algorithm 3.2. Note that for the sake of clarity, Algorithm 3.2 is formulated in terms of binary splits. However, as we will explore later in Section 3.6, it generalizes naturally to n-ary multiway splits.

que particiona o conjunto de aprendizagem $\mathcal{L}$da melhor forma possível. Entre todas as árvores de decisão $\varphi \in \mathcal{H}$ no entanto, podem existir várias delas que explicam $\mathcal{L}$ igualmente melhor. Seguindo os princípios Razor de Occam [Blumer et al., 1987] de preferir a explicação que faz o menor número possível de suposições, isto é, favorecer a solução mais simples que se ajusta aos dados, aprender uma árvore de decisão a partir de $\mathcal{L}$é, portanto, geralmente reafirmado como encontrar a menor árvore φ^* (em termos de nós internos) minimizando sua estimativa de substituição $E(\varphi^*, \mathcal{L})$. Embora esta suposição faça sentido do ponto de vista da generalização, também faz sentido no que diz respeito à i nterpretabilidade. Uma árvore de decisão pequena é mais fácil de entender do que uma árvore grande e complexa.

Conforme mostrado por Hyafil e Rivest, [1976],encontrar a menor árvore φ^*que minimize sua estimativa de ressubstituição é um problema NP-completo. Como consequência, sob a suposição de que $P \neq NP$, não existe um algoritmo eficiente para encontrar φ^*, sugerindo assim que encontrar heurísticas eficientes para construir árvores de decisão quase ótimas é a melhor solução para manter os requisitos de computação dentro de limites realistas.

Seguindo a estrutura de Breiman et al. [1984], vamos definir amplamente uma medida de impureza $i(t)$ como uma função que avalia a qualidade de qualquer nó t (passamos para a Seção 3.6 para uma definição mais precisa). Suponhamos que quanto menor $i(t)$, mais puro o nó e melhores as previsões $\hat{y}_t(x)$ para todo $x \in \mathcal{L}_t$, onde $\mathcal{L}_t$ é o subconjunto de amostras de aprendizagem que caem em t, que é tudo $(x, y) \in \mathcal{L}$ tal que $x \in \mathcal{X}_t$. Começando a partir de um único nó representando todo o conjunto de aprendizagem $\mathcal{L}$, árvores de decisão quase ideais podem então ser cultivadas avidamente dividindo iterativamente os nós em nós mais puros. Isto é, dividindo iterativamente em subconjuntos menores os subconjuntos de $\mathcal{L}$ representados pelos nós, até que todos os nós terminais não possam ser tornados mais puros, garantindo assim previsões quase ótimas sobre $\mathcal{L}$. A suposição gananciosa para tornar a árvore resultante tão pequena quanto possível, buscando assim uma boa generalização, é então dividir cada nó t usando a divisão s^* que maximiza localmente a diminuição da impureza dos nós filhos resultantes.

Formalmente, a diminuição da impureza de uma divisão binária s é definida da seguinte forma:

Definição 3.6. A diminuição de impureza de uma divisão binária $s \in Q$ *dividindo o nó* t *em um nó esquerdo* t_L *e um nó direito* t_R *é*

$$\Delta i(s, t) = i(t) - p_L i(t_L) - p_R i(t_R) \tag{3.5}$$

onde p_L *(resp.,* p_R*) é a proporção* $\frac{N_{t_L}}{N_t}$ *(resp.,* $\frac{N_{t_R}}{N_t}$*) de amostras de aprendizagem de* $\mathcal{L}_t$ *indo para* t_L *(resp., para* t_R*) e onde* N_t *é o tamanho do subconjunto* $\mathcal{L}_t$*.*

Com base neste conceito, um procedimento guloso geral para a indução de árvores de decisão pode agora ser descrito formalmente conforme descrito no Algoritmo 3.2. Observe que, para maior clareza, o Algoritmo 3.2 é formulado em termos de divisões binárias. Entretanto, como exploraremos mais adiante na Seção 3.6, ela generaliza naturalmente para divisões multivias n-árias.

Algorithm 3.2. *Greedy induction of a binary decision tree.*

1: **function** BUILDDECISIONTREE($\mathcal{L}$)
2: Create a decision tree φ with root node t_0
3: Create an empty stack S of *open* nodes $(t, \mathcal{L}_t)$
4: S.PUSH($(t_0, \mathcal{L})$)
5: **while** S is not empty **do**
6: $t, \mathcal{L}_t$ = S.POP( )
7: **if** the stopping criterion is met for t **then**
8: $\widehat{y}_t$ = some constant value
9: **else**
10: Find the split on $\mathcal{L}_t$ that maximizes impurity decrease

$$s^* = \arg\max_{s \in Q} \Delta i(s, t)$$

11: Partition $\mathcal{L}_t$ into $\mathcal{L}_{t_L} \cup \mathcal{L}_{t_R}$ according to s^*
12: Create the left child node t_L of t
13: Create the right child node t_R of t
14: S.PUSH($(t_R, \mathcal{L}_{t_R})$)
15: S.PUSH($(t_L, \mathcal{L}_{t_L})$)
16: **end if**
17: **end while**
18: **return** φ
19: **end function**

The rest of this chapter is now dedicated to a detailed discussion of specific parts of Algorithm 3.2. Section 3.4 discusses assignment rules for terminal nodes (Line 8 of Algorithm 3.2) while Section 3.5 outlines stopping criteria for deciding when a node becomes terminal (Line 7 of Algorithm 3.2). Section 3.6 then presents families Q of splitting rules, impurity criteria $i(t)$ to evaluate the goodness of splits and strategies for finding the best split $s^* \in Q$ (Line 10 of Algorithm 3.2). As we will see, the crux of the problem is in finding good splits and in knowing when to stop splitting.

LOOKAHEAD SEARCH

Obviously, the greedy assumption on which Algorithm 3.2 relies may produce trees that are suboptimal. A natural improvement of the greedy strategy is lookahead search, which consists in evaluating the goodness of a split, but also of those deeper in the tree, assuming this former split was effectively performed. As empirically investigated in [Murthy and Salzberg, 1995], such an approach is however not only more computationally intensive, it is also not significantly better than greedily induced decision trees. For this reason, more elaborate variants of Algorithm 3.2 are not considered within this work. Let us note however that trees grown with lookahead search are usually shorter, which may be a strong advantage when interpretability matters.

Algoritmo 3.2. Indução gananciosa de uma árvore de decisão binária.
1: função BuildDecisionTree($\mathcal{L}$) 2: Crie uma árvore de decisão φcom nó raiz $t_0$3: Crie uma pilha vazia S de nós abertos $(t, \mathcal{L}_t)$4: S.push((t_0, $\mathcal{L}$)) 5: enquanto Snão estiver vazio faça 6: $t, \mathcal{L}_t$ = S.pop() 7: se o critério de parada for atendido para t então 8: $\widehat{y}_t$ =algum valor constante 9: senão 10: Encontre a divisão em $\mathcal{L}_t$ que maximiza a diminuição de impureza 11: Particione $\mathcal{L}_t$ em $\mathcal{L}_{t_L} \cup \mathcal{L}_{t_R}$ de acordo com s^*12: Crie o nó filho esquerdo t_L de t13: Crie o nó filho direito t_R de t14: S.push((t_R, $\mathcal{L}_{t_R}$)) 15: S.push((t_L,$\mathcal{L}_{t_L}$)) 16: fim se 17: fim enquanto 18: retorno φ19: função final

$$s^* = \arg\max_{s \in Q} \Delta i(s, t)$$

O restante deste capítulo é agora dedicado a uma discussão detalhada de partes específicas do Algoritmo 3.2. A Seção 3.4 discute regras de atribuição para nós terminais (Linha 8 do Algoritmo 3.2), enquanto a Seção 3.5 descreve critérios de parada para decidir quando um nó se torna terminal (Linha 7 do Algoritmo 3.2). A Seção 3.6 apresenta então famílias Q de regras de divisão, critérios de impureza $i(t)$ para avaliar a qualidade das divisões e estratégias para encontrar a melhor divisão $s^* \in Q$ (Linha 10 do Algoritmo 3.2). Como veremos, o cerne do problema está em encontrar boas divisões e em saber quando parar de dividir.

Pesquisa antecipada

Obviamente, a suposição gananciosa na qual o Algoritmo 3.2 se baseia pode produzir árvores que são subótimas. Uma melhoria natural da estratégia gananciosa é a busca antecipada, que consiste em avaliar a qualidade de uma divisão, mas também daquelas mais profundas na árvore, assumindo que esta divisão anterior foi efetivamente realizada. Conforme investigado empiricamente em[Murthy e Salzberg, 1995],tal abordagem é, no entanto, não apenas mais intensiva em termos computacionais, mas também não é significativamente melhor do que árvores de decisão induzidas gananciosamente. Por esta razão, variantes mais elaboradas do Algoritmo 3.2 não são consideradas neste trabalho. Observemos, entretanto, que as árvores cultivadas com pesquisa antecipada são geralmente mais curtas, o que pode ser uma grande vantagem quando a interpretabilidade é importante.

3.4 ASSIGNMENT RULES

Let us assume that node t has been declared terminal given some stopping criterion (See next Section 3.5). The next step (Line 8 of Algorithm 3.2) in the induction procedure is to label t with a constant value $\hat{y}_t$ to be used as a prediction of the output variable Y. As such, node t can be regarded as a simplistic model defined locally on $\mathcal{X}_t \times \mathcal{Y}$ and producing the same output value $\hat{y}_t$ for all possible input vectors falling into t.

Let us first notice that, for a tree φ of *fixed* structure, minimizing the global generalization error is strictly equivalent to minimizing the local generalization error of each simplistic model in the terminal nodes. Indeed,

$$\begin{aligned} Err(\varphi) &= \mathbb{E}_{X,Y}\{L(Y, \varphi(X))\} \\ &= \sum_{t \in \tilde{\varphi}} P(X \in \mathcal{X}_t) \mathbb{E}_{X,Y|t}\{L(Y, \hat{y}_t)\} \end{aligned} \tag{3.6}$$

where $\tilde{\varphi}$ denotes the set of terminal nodes in φ and where the inner expectation[1] is the local generalization error of the model at node t. In this later form, a model which minimizes $Err(\varphi)$ is a model which minimizes the inner expectation leaf-wise. Learning the best possible decision tree (of fixed structure) therefore simply amounts to find the best constants $\hat{y}_t$ at each terminal node.

3.4.1 *Classification*

When L is the zero-one loss, the inner expectation in Equation 3.6 is minimized by the plurality rule:

$$\begin{aligned} \hat{y}_t^* &= \arg\min_{c \in \mathcal{Y}} \mathbb{E}_{X,Y|t}\{1(Y, c)\} \\ &= \arg\min_{c \in \mathcal{Y}} P(Y \neq c | X \in \mathcal{X}_t) \\ &= \arg\max_{c \in \mathcal{Y}} P(Y = c | X \in \mathcal{X}_t) \end{aligned} \tag{3.7}$$

Put otherwise, the generalization error of t is minimized by predicting the class which is the most likely for the samples in the subspace of t. Note that if the maximum is achieved by two or more different classes, then $\hat{y}_t^*$ is assigned arbitrarily as any one of the maximizing classes.

Equation 3.7 cannot be solved without the probability distribution $P(X, Y)$. However, its solution can be approximated by using estimates of the local generalization error. Let N_t denotes the number of objects in $\mathcal{L}_t$ and let N_{ct} denotes the number of objects of class c in $\mathcal{L}_t$. Then, the proportion $\frac{N_{ct}}{N_t}$ can be interpreted as the estimated probability[2]

1 The joint expectation of X and Y is taken over all objects $i \in \Omega$ such that $\mathbf{x}_i \in \mathcal{X}_t$.

2 Lower case p denotes an estimated probability while upper case P denotes a theoretical probability.

3.4 regras de atribuição

Vamos supor que o nó t foi declarado terminal dado algum critério de parada (veja a próxima Seção 3.5). O próximo passo (Linha 8 do Algoritmo 3.2) no procedimento de indução é rotular t com um valor constante $\hat{y}_t$ para ser usado como uma previsão da variável de saída Y. Como tal, o nó t pode ser considerado como um modelo simplista definido localmente em $\mathcal{X}_t \times \mathcal{Y}$ e produzindo o mesmo valor de saída $\hat{y}_t$ para todos os vetores de entrada possíveis que caem em t.

Notemos primeiro que, para uma árvore φ de estrutura fixa, minimizar o erro de generalização global é estritamente equivalente a minimizar o erro de generalização local de cada modelo simplista nos nós terminais. De fato,

$$\begin{aligned} Err(\varphi) &= \mathbb{E}_{X,Y}\{L(Y, \varphi(X))\} \\ &= \sum_{t \in \tilde{\varphi}} P(X \in \mathcal{X}_t) \mathbb{E}_{X,Y|t}\{L(Y, \hat{y}_t)\} \end{aligned} \tag{3.6}$$

onde $\tilde{\varphi}$ denota o conjunto de nós terminais em φ e onde a expectativa interna[1] é o erro de generalização local do modelo no nó t. Nesta forma posterior, um modelo que minimiza $Err(\varphi)$ é um modelo que minimiza a expectativa interna em termos de folha. Aprender a melhor árvore de decisão possível (de estrutura fixa) equivale, portanto, simplesmente a encontrar as melhores constantes $\hat{y}_t$ em cada nó terminal.

3.4.1 Classificação

Quando L é a perda zero-um, a expectativa interna na Equação 3.6 é minimizada pela regra da pluralidade:

$$\begin{aligned} \hat{y}_t^* &= \arg\min_{c \in \mathcal{Y}} \mathbb{E}_{X,Y|t}\{1(Y, c)\} \\ &= \arg\min_{c \in \mathcal{Y}} P(Y \neq c | X \in \mathcal{X}_t) \\ &= \arg\max_{c \in \mathcal{Y}} P(Y = c | X \in \mathcal{X}_t) \end{aligned} \tag{3.7}$$

Dito de outra forma, o erro de generalização de t é minimizado prevendo a classe que é mais provável para as amostras no subespaço de t. Observe que se o máximo for alcançado por duas ou mais classes diferentes, então $\hat{y}_t^*$ é atribuído arbitrariamente como qualquer uma das classes maximizadoras.

A Equação 3.7 não pode ser resolvida sem a distribuição de probabilidade $P(X, Y)$. Contudo, sua solução pode ser aproximada usando estimativas do erro de generalização local. Seja N_t denota o número de objetos em $\mathcal{L}_t$ e seja N_{ct} denota o número de objetos da classe c em $\mathcal{L}_t$. Então, a proporção $\frac{N_{ct}}{N_t}$ pode ser interpretada como a probabilidade estimada[2]

1 A expectativa conjunta de X e Y é assumida sobre todos os objetos $i \in \Omega$ tais que $\mathbf{x}_i \in \mathcal{X}_t$. 2 Minúsculas p denota uma probabilidade estimada enquanto maiúsculas P denota uma probabilidade teórica.

$p(Y = c|X \in \mathcal{X}_t)$ (shortly denoted $p(c|t)$) of class c in t and therefore be used to solve Equation 3.7:

$$\begin{aligned}\widehat{y}_t &= \arg\min_{c \in \mathcal{Y}} 1 - p(c|t) \\ &= \arg\max_{c \in \mathcal{Y}} p(c|t)\end{aligned} \tag{3.8}$$

Similarly, let us also define the proportion $\frac{N_t}{N}$ as the estimated probability $p(X \in \mathcal{X}_t)$ (shortly denoted $p(t)$). Plugging this estimate into Equation 3.6 and approximating the local generalization error with $1 - p(\widehat{y}_t|t)$ as done above, it follows:

$$\begin{aligned}\widehat{\mathrm{Err}}(\varphi) &= \sum_{t \in \widetilde{\varphi}} p(t)(1 - p(\widehat{y}_t|t)) \\ &= \sum_{t \in \widetilde{\varphi}} \frac{N_t}{N}(1 - \frac{N_{\widehat{y}_t t}}{N_t}) \\ &= \frac{1}{N} \sum_{t \in \widetilde{\varphi}} N_t - N_{\widehat{y}_t t} \\ &= \frac{1}{N} \sum_{t \in \widetilde{\varphi}} \sum_{x,y \in \mathcal{L}_t} 1(y \neq \widehat{y}_t) \\ &= \frac{1}{N} \sum_{x,y \in \mathcal{L}} 1(y \neq \varphi(\mathbf{x})) \\ &= \widehat{\mathrm{Err}}^{\text{train}}(\varphi)\end{aligned} \tag{3.9}$$

Accordingly, approximating Equation 3.6 through local probability estimates computed from class proportions in $\mathcal{L}_t$ reduces to the resubstitution estimate of φ (Equation 2.5). In other words, assignment rule 3.8 in fact minimizes the resubstitution estimate rather than the true generalization error.

An important property of assignment rule 3.8 is that the more one splits a terminal node *in any way*, the smaller the resubstitution estimate $\widehat{\mathrm{Err}}^{\text{train}}(\varphi)$ becomes.

Proposition 3.1. *For any non-empty split of a terminal node $t \in \widetilde{\varphi}$ into t_L and t_R, resulting in a new tree φ' where $\widehat{y}_{t_L}$ and $\widehat{y}_{t_R}$ are assigned with rule 3.8,*

$$\widehat{\mathrm{Err}}^{train}(\varphi) \geqslant \widehat{\mathrm{Err}}^{train}(\varphi')$$

with equality if $\widehat{y}_t = \widehat{y}_{t_L} = \widehat{y}_{t_R}$.

Proof.

$$\begin{aligned}\widehat{\mathrm{Err}}^{\text{train}}(\varphi) &\geqslant \widehat{\mathrm{Err}}^{\text{train}}(\varphi') \\ \sum_{t \in \widetilde{\varphi}} p(t)(1 - p(\widehat{y}_t|t)) &\geqslant \sum_{t \in \widetilde{\varphi'}} p(t)(1 - p(\widehat{y}_t|t)) \\ p(t)(1 - p(\widehat{y}_t|t)) &\geqslant p(t_L)(1 - p(\widehat{y}_{t_L}|t_L)) + p(t_R)(1 - p(\widehat{y}_{t_R}|t_R))\end{aligned}$$

$p(Y = c|X \in \mathcal{X}_t)$ (abreviadamente denotado como $p(c|t)$) da classe c em t e, portanto, será usado para resolver a Equação3.7:

$$\begin{aligned}\widehat{y}_t &= \arg\min_{c \in \mathcal{Y}} 1 - p(c|t) \\ &= \arg\max_{c \in \mathcal{Y}} p(c|t)\end{aligned} \tag{3.8}$$

Da mesma forma, vamos também definir a proporção $\frac{N_t}{N}$ como a probabilidade estimada $p(X \in \mathcal{X}_t)$ (denominada abreviadamente $p(t)$). Inserindo esta estimativa na Equação 3.6 e aproximando o erro de generalização local com $1 - p(\widehat{y}_t|t)$ como feito acima, segue:

$$\begin{aligned}\widehat{\mathrm{Err}}(\varphi) &= \sum_{t \in \widetilde{\varphi}} p(t)(1 - p(\widehat{y}_t|t)) \\ &= \sum_{t \in \widetilde{\varphi}} \frac{N_t}{N}(1 - \frac{N_{\widehat{y}_t t}}{N_t}) \\ &= \frac{1}{N} \sum_{t \in \widetilde{\varphi}} N_t - N_{\widehat{y}_t t} \\ &= \frac{1}{N} \sum_{t \in \widetilde{\varphi}} \sum_{x,y \in \mathcal{L}_t} 1(y \neq \widehat{y}_t) \\ &= \frac{1}{N} \sum_{x,y \in \mathcal{L}} 1(y \neq \varphi(\mathbf{x})) \\ &= \widehat{\mathrm{Err}}^{\text{train}}(\varphi)\end{aligned} \tag{3.9}$$

Conseqüentemente, aproximar a Equação 3.6 por meio de estimativas de probabilidade locais calculadas a partir de proporções de classe em $\mathcal{L}_t$ reduz-se à estimativa de resubstituição de φ (Equação 2.5). Em outras palavras, a regra de atribuição 3.8 de fato minimiza a estimativa de resubstituição em vez do verdadeiro erro de generalização.

Uma propriedade importante da regra de atribuição 3.8 é que quanto mais se divide um nó terminal de alguma forma, menor se torna a estimativa de ressubstituição $\widehat{\mathrm{Err}}^{\text{train}}(\varphi)$.

Proposição 3.1. Para qualquer divisão não vazia de um nó terminal $t \in \widetilde{\varphi}$ emt_L e t_R, resultando em uma nova árvore φ' onde $\widehat{y}_{t_L}$ e $\widehat{y}_{t_R}$ são atribuídos com a regra 3.8,

$$\widehat{\mathrm{Err}}^{train}(\varphi) \geqslant \widehat{\mathrm{Err}}^{train}(\varphi')$$

com igualdade se $\widehat{y}_t = \widehat{y}_{t_L} = \widehat{y}_{t_R}$.

Prova.

$$\begin{aligned}\widehat{\mathrm{Err}}^{\text{train}}(\varphi) &\geqslant \widehat{\mathrm{Err}}^{\text{train}}(\varphi') \\ \sum_{t \in \widetilde{\varphi}} p(t)(1 - p(\widehat{y}_t|t)) &\geqslant \sum_{t \in \widetilde{\varphi'}} p(t)(1 - p(\widehat{y}_t|t)) \\ p(t)(1 - p(\widehat{y}_t|t)) &\geqslant p(t_L)(1 - p(\widehat{y}_{t_L}|t_L)) + p(t_R)(1 - p(\widehat{y}_{t_R}|t_R))\end{aligned}$$

$$
\begin{aligned}
\frac{N_t}{N}(1-\max_{c\in\mathcal{Y}}\frac{N_{ct}}{N_t}) &\geqslant \frac{N_{t_L}}{N}(1-\max_{c\in\mathcal{Y}}\frac{N_{ct_L}}{N_{t_L}})+\frac{N_{t_R}}{N}(1-\max_{c\in\mathcal{Y}}\frac{N_{ct_R}}{N_{t_R}})\\
N_t-\max_{c\in\mathcal{Y}}N_{ct} &\geqslant N_{t_L}-\max_{c\in\mathcal{Y}}N_{ct_L}+N_{t_R}-\max_{c\in\mathcal{Y}}N_{ct_R}\\
\max_{c\in\mathcal{Y}}N_{ct} &\leqslant \max_{c\in\mathcal{Y}}N_{ct_L}+\max_{c\in\mathcal{Y}}N_{ct_R}\\
\max_{c\in\mathcal{Y}}(N_{ct_L}+N_{ct_R}) &\leqslant \max_{c\in\mathcal{Y}}N_{ct_L}+\max_{c\in\mathcal{Y}}N_{ct_R}
\end{aligned}
$$

which is true since $\max_{c\in\mathcal{Y}} N_{ct_L}$ (resp., t_R) is necessarily greater or equal to the left term N_{ct_L} (resp., to right term N_{ct_R}) in the left-hand side of the equation. Equality holds if the majority classes are the same in t, t_L and t_R. □

As a corollary of Proposition 3.1, the resubstitution estimate is minimal when terminal nodes can no longer be divided. In particular, it is equal to zero if the tree can be *fully developed*, that is if terminal nodes can be divided until they all contain exactly one object from $\mathcal{L}$. Based on this result, we will show however in Section 3.6.2 why using the resubstitution estimate to evaluate the goodness of a split s has serious defects for classification.

3.4.2 *Regression*

When L is the squared error loss, the inner expectation in Equation 3.6 is minimized by the expectation of Y in t:

$$
\begin{aligned}
\widehat{y}_t^* &= \arg\min_{\widehat{y}\in\mathcal{Y}} \mathbb{E}_{X,Y|t}\{(Y-\widehat{y})^2\}\\
&= \mathbb{E}_{X,Y|t}\{Y\} \qquad (3.10)
\end{aligned}
$$

Again, Equation 3.10 cannot be solved without the probability distribution $P(X,Y)$. However, its solution can approximated using estimates of the local generalization error:

$$
\begin{aligned}
\widehat{y}_t &= \arg\min_{\widehat{y}\in\mathcal{L}} \frac{1}{N_t}\sum_{x,y\in\mathcal{L}_t}(y-\widehat{y})^2\\
&= \frac{1}{N_t}\sum_{x,y\in\mathcal{L}_t} y \qquad (3.11)
\end{aligned}
$$

As in classification, using $p(t)$ as estimate of $P(X\in\mathcal{X}_t)$ and approximating the local generalization error with $\frac{1}{N_t}\sum_{x,y\in\mathcal{L}_t}(y-\widehat{y}_t)^2$ as done above, one can show that Equation 3.6 reduces to the resubstitution estimate of φ, thereby indicating that assignment rule 3.11 also minimizes the training error rather the true generalization error.

Most importantly, one can also show that assignment rule 3.11 behaves in the same way as 3.8. The more one splits a terminal node in any way, the smaller the resubstitution estimate $\widehat{Err}^{train}(\varphi)$ becomes.

$$
\begin{aligned}
\frac{N_t}{N}(1-\max_{c\in\mathcal{Y}}\frac{N_{ct}}{N_t}) &\geqslant \frac{N_{t_L}}{N}(1-\max_{c\in\mathcal{Y}}\frac{N_{ct_L}}{N_{t_L}})+\frac{N_{t_R}}{N}(1-\max_{c\in\mathcal{Y}}\frac{N_{ct_R}}{N_{t_R}})\\
N_t-\max_{c\in\mathcal{Y}}N_{ct} &\geqslant N_{t_L}-\max_{c\in\mathcal{Y}}N_{ct_L}+N_{t_R}-\max_{c\in\mathcal{Y}}N_{ct_R}\\
\max_{c\in\mathcal{Y}}N_{ct} &\leqslant \max_{c\in\mathcal{Y}}N_{ct_L}+\max_{c\in\mathcal{Y}}N_{ct_R}\\
\max_{c\in\mathcal{Y}}(N_{ct_L}+N_{ct_R}) &\leqslant \max_{c\in\mathcal{Y}}N_{ct_L}+\max_{c\in\mathcal{Y}}N_{ct_R}
\end{aligned}
$$

o que é verdade, pois $\max_{c\in\mathcal{Y}} N_{ct_L}$ (resp., t_R) é necessariamente maior ou igual ao termo esquerdo N_{ct_L} (resp., ao termo direito N_{ct_R}) no lado esquerdo da equação. A igualdade é válida se as classes majoritárias forem iguais em t, t_L e t_R. □

Como corolário da Proposição 3.1, a estimativa de ressubstituição é mínima quando os nós terminais não podem mais ser divididos. Em particular, é igual a zero se a árvore puder ser totalmente desenvolvida, ou seja, se os nós terminais puderem ser divididos até que todos contenham exatamente um objeto de $\mathcal{L}$. Com base neste resultado, mostraremos na Seção 3.6.2 por que o uso da estimativa de substituição para avaliar a qualidade de uma divisão s apresenta sérios defeitos de classificação.

3.4.2 Regressão

Quando L é a perda quadrática do erro, a expectativa interna na Equação 3.6 é minimizada pela expectativa de Y em t:

$$
\begin{aligned}
\widehat{y}_t^* &= \arg\min_{\widehat{y}\in\mathcal{Y}} \mathbb{E}_{X,Y|t}\{(Y-\widehat{y})^2\}\\
&= \mathbb{E}_{X,Y|t}\{Y\} \qquad (3.10)
\end{aligned}
$$

Novamente, a Equação 3.10 não pode ser resolvida sem a distribuição de probabilidade $P(X,Y)$. Contudo, sua solução pode ser aproximada usando estimativas do erro de generalização local:

$$
\begin{aligned}
\widehat{y}_t &= \arg\min_{\widehat{y}\in\mathcal{L}} \frac{1}{N_t}\sum_{x,y\in\mathcal{L}_t}(y-\widehat{y})^2\\
&= \frac{1}{N_t}\sum_{x,y\in\mathcal{L}_t} y \qquad (3.11)
\end{aligned}
$$

Tal como na classificação, usando $p(t)$ como estimativa de $P(X\in\mathcal{X}_t)$ e aproximando o erro de generalização local com $\frac{1}{N_t}\sum_{x,y\in\mathcal{L}_t}(y-\widehat{y}_t)^2$ como feito acima, pode-se mostrar que a Equação 3.6 se reduz à estimativa de resubstituição de φ, indicando assim que a regra de atribuição 3.11 também minimiza o erro de treino em vez do verdadeiro erro de generalização.

Mais importante ainda, pode-se também mostrar que a regra de atribuição 3.11 se comporta da mesma forma que a 3.8. Quanto mais se divide um nó terminal, menor se torna a estimativa de ressubstituição $\widehat{Err}^{train}(\varphi)$.

Proposition 3.2. *For any non-empty split of a terminal node* $t \in \widetilde{\varphi}$ *into* t_L *and* t_R*, resulting in a new tree* φ' *where* $\widehat{y}_{t_L}$ *and* $\widehat{y}_{t_R}$ *are assigned with rule 3.11,*

$$\widehat{Err}^{train}(\varphi) \geqslant \widehat{Err}^{train}(\varphi')$$

with equality if $\widehat{y}_{t_L} = \widehat{y}_{t_R} = \widehat{y}_t$.

Proof.

$$\widehat{Err}^{\text{train}}(\varphi) \geqslant \widehat{Err}^{\text{train}}(\varphi')$$

$$\sum_{t\in\widetilde{\varphi}} p(t)(\frac{1}{N_t}\sum_{x,y\in\mathcal{L}_t}(y-\widehat{y}_t)^2) \geqslant \sum_{t\in\widetilde{\varphi'}} p(t)(\frac{1}{N_t}\sum_{x,y\in\mathcal{L}_t}(y-\widehat{y}_t)^2)$$

$$\sum_{x,y\in\mathcal{L}_t}(y-\widehat{y}_t)^2 \geqslant \sum_{x,y\in\mathcal{L}_{t_L}}(y-\widehat{y}_{t_L})^2 + \sum_{x,y\in\mathcal{L}_{t_R}}(y-\widehat{y}_{t_R})^2$$

$$N_t\widehat{y}_t^2 \leqslant N_{t_L}\widehat{y}_{t_L}^2 + N_{t_R}\widehat{y}_{t_R}^2$$

$$\frac{1}{N_t}(\sum_{x,y\in\mathcal{L}_t} y)^2 \leqslant \frac{1}{N_{t_L}}(\sum_{x,y\in\mathcal{L}_{t_L}} y)^2 + \frac{1}{N_{t_R}}(\sum_{x,y\in\mathcal{L}_{t_R}} y)^2$$

For simplifying, let us denote $s(t) = \sum_{x,y\in\mathcal{L}_t} y = N_t\widehat{y}_t$ the sum of the output values in t. Remark that $s(t) = s(t_L) + s(t_R)$. It comes:

$$\frac{s(t)^2}{N_t} \leqslant \frac{s(t_L)^2}{N_{t_L}} + \frac{s(t_R)^2}{N_{t_R}}$$

$$\frac{(s(t_L)+s(t_R))^2}{N_{t_L}+N_{t_R}} \leqslant \frac{s(t_L)^2}{N_{t_L}} + \frac{s(t_R)^2}{N_{t_R}}$$

$$\frac{(s(t_L)N_{t_R} - s(t_R)N_{t_L})^2}{N_{t_L}N_{t_R}(N_{t_L}+N_{t_R})} \geqslant 0$$

Which is necessary true since the numerator is non-negative and the denominator is strictly positive. Equality holds if $s(t_L)N_{t_R} = s(t_R)N_{t_L}$, that is if $\widehat{y}_{t_L} = \widehat{y}_{t_R}$. □

As for Proposition 3.1, a corollary of Proposition 3.2 is that the resubstitution estimate is minimal when terminal nodes can no longer be divided.

3.5 STOPPING CRITERIA

As we have shown through Propositions 3.1 and 3.2, the deeper a decision tree, the smaller its training estimate, and the better we think it is. As discussed in Section 2.3 however, increasing model complexity, in this case by enlarging the set of terminal nodes, is likely to eventually capture noise in the learning set and to cause overfitting. In other words, it is not because the training estimate can be proved to go down to zero that the test estimate converges accordingly. On the contrary, it is very likely to diverge as complexity increases. To prevent this phenomenon it is therefore necessary to find the right

Proposição 3.2. Para qualquer divisão não vazia de um nó terminal $t \in \widetilde{\varphi}$ *em* t_L *e* t_R*, resultando em uma nova árvore* φ' *onde* $\widehat{y}_{t_L}$ *e* $\widehat{y}_{t_R}$ *são atribuídos com a regra 3.11,*

$$\widehat{Err}^{train}(\varphi) \geqslant \widehat{Err}^{train}(\varphi')$$

com igualdade se $\widehat{y}_{t_L} = \widehat{y}_{t_R} = \widehat{y}_t$.

Prova.

$$\widehat{Err}^{\text{train}}(\varphi) \geqslant \widehat{Err}^{\text{train}}(\varphi')$$

$$\sum_{t\in\widetilde{\varphi}} p(t)(\frac{1}{N_t}\sum_{x,y\in\mathcal{L}_t}(y-\widehat{y}_t)^2) \geqslant \sum_{t\in\widetilde{\varphi'}} p(t)(\frac{1}{N_t}\sum_{x,y\in\mathcal{L}_t}(y-\widehat{y}_t)^2)$$

$$\sum_{x,y\in\mathcal{L}_t}(y-\widehat{y}_t)^2 \geqslant \sum_{x,y\in\mathcal{L}_{t_L}}(y-\widehat{y}_{t_L})^2 + \sum_{x,y\in\mathcal{L}_{t_R}}(y-\widehat{y}_{t_R})^2$$

$$N_t\widehat{y}_t^2 \leqslant N_{t_L}\widehat{y}_{t_L}^2 + N_{t_R}\widehat{y}_{t_R}^2$$

$$\frac{1}{N_t}(\sum_{x,y\in\mathcal{L}_t} y)^2 \leqslant \frac{1}{N_{t_L}}(\sum_{x,y\in\mathcal{L}_{t_L}} y)^2 + \frac{1}{N_{t_R}}(\sum_{x,y\in\mathcal{L}_{t_R}} y)^2$$

Para simplificar, vamos denotar $s(t) = \sum_{x,y\in\mathcal{L}_t} y = N_t\widehat{y}_t$ a soma dos valores de saída em t. Observe que $s(t) = s(t_L) + s(t_R)$. Vem:

$$\frac{s(t)^2}{N_t} \leqslant \frac{s(t_L)^2}{N_{t_L}} + \frac{s(t_R)^2}{N_{t_R}}$$

$$\frac{(s(t_L)+s(t_R))^2}{N_{t_L}+N_{t_R}} \leqslant \frac{s(t_L)^2}{N_{t_L}} + \frac{s(t_R)^2}{N_{t_R}}$$

$$\frac{(s(t_L)N_{t_R} - s(t_R)N_{t_L})^2}{N_{t_L}N_{t_R}(N_{t_L}+N_{t_R})} \geqslant 0$$

O que é necessariamente verdade, pois o numerador não é negativo e o denominador é estritamente positivo. A igualdade é válida se $s(t_L)N_{t_R} = s(t_R)N_{t_L}$, isto é, se $\widehat{y}_{t_L} = \widehat{y}_{t_R}$. □

Quanto à Proposição 3.1, um corolário da Proposição 3.2 é que a estimativa de resubstituição é mínima quando os nós terminais não podem mais ser divididos.

3.5 critérios de parada

Como mostramos nas Proposições 3.1 e 3.2, quanto mais profunda for uma árvore de decisão, menor será sua estimativa de treinamento e melhor pensamos que ela é. Contudo, como discutido na Secção 2.3, o aumento da complexidade do modelo, neste caso através do alargamento do conjunto de nós terminais, é susceptível de eventualmente capturar ruído no conjunto de aprendizagem e causar sobreajuste. Em outras palavras, não é porque se pode provar que a estimativa de treinamento chega a zero que a estimativa de teste converge adequadamente. Pelo contrário, é muito provável que diverja à medida que a complexidade aumenta. Para evitar este fenómeno é, portanto, necessário encontrar o caminho certo

trade-off between a tree which is not too shallow nor too deep. The problem is in knowing when to stop splitting, i.e., in how to carefully formulate Line 7 of Algorithm 3.2.

Let us first consider the stopping criteria that are inherent to the iterative partition procedure, regardless of overfitting. Node t is inevitably set as a terminal node when $\mathcal{L}_t$ can no longer be split, which happens in the following cases:

(a) When the output values of the samples in $\mathcal{L}_t$ are homogeneous. That is, if $y = y'$ for all $(\mathbf{x}, y), (\mathbf{x}', y') \in \mathcal{L}_t$. In particular, this is necessarily the case when $N_t = 1$.

(b) When the input variables X_j are each locally constant in $\mathcal{L}_t$. That is, if $x_j = x'_j$ for all $(\mathbf{x}, y), (\mathbf{x}', y') \in \mathcal{L}_t$, for each input variable X_j. In this situation, it is indeed not possible to divide $\mathcal{L}_t$ into two (or more) non-empty subsets.

To prevent overfitting, the stopping criterion is then usually complemented with heuristics halting the recursive partition if $\mathcal{L}_t$ has become too small or if no sufficiently good split can be found. The most common approaches are formulated as follows:

(c) Set t as a terminal node if it contains less than N_{min} samples. (N_{min} is also known as `min_samples_split`.)

(d) Set t as a terminal node if its depth d_t is greater or equal to a threshold d_{max}. (d_{max} is also known as `max_depth`.)

(e) Set t as a terminal node if the total decrease in impurity is less than a fixed threshold β. That is, if $p(t)\Delta i(s^*, t) < \beta$.

(f) Set t as a terminal node if there is no split such that t_L and t_R both count a least N_{leaf} samples. (N_{leaf} is also known as `min_samples_leaf`.)

In all of the above, stopping criteria are defined in terms of user-defined hyper-parameters (N_{min}, d_{max}, β or N_{leaf}) that have to be tuned in order to find the right trade-off. Ideally, they need to be such that they are neither too strict nor too loose for the tree to be neither too shallow nor too deep. Too large a tree will have a higher generalization error than the right sized tree. Likewise, too small a tree will not use some of the information in $\mathcal{L}$, again resulting in a higher generalization error than the right sized tree. As described in Section 2.3, choosing the appropriate parameter values is usually performed using a dedicated model selection procedure, which can be computationally expensive, yet usually imperative for reaching good generalization performance.

trade-off entre uma árvore que não é muito rasa nem muito profunda. O problema está em saber quando parar a divisão, ou seja, em como formular cuidadosamente a Linha 7 do Algoritmo 3.2.

Consideremos primeiro os critérios de parada inerentes ao procedimento de partição iterativa, independentemente do sobreajuste. O nó t é inevitavelmente definido como um nó terminal quando $\mathcal{L}_t$ não pode mais ser dividido, o que acontece nos seguintes casos:

(a) Quando os valores de saída das amostras em $\mathcal{L}_t$ são homogêneos. Isto é, se $y = y'$ para todos $(x, y), (x', y') \in \mathcal{L}_t$. Em particular, este é necessariamente o caso quando $N_t = 1$. (b) Quando as variáveis de entrada X_j são localmente constantes em $\mathcal{L}_t$. Isto é, se $x_j = x'_j$ para t odos $(x, y), (x', y') \in \mathcal{L}_t$, para cada variável de entrada X_j. Nesta situação, de fato não é possível dividir $\mathcal{L}_t$ em dois (ou mais) subconjuntos não vazios.

Para evitar overfitting, o critério de parada é geralmente complementado com heurísticas que interrompem a partição recursiva se $\mathcal{L}_t$ se tornar muito pequeno ou se nenhuma divisão suficientemente boa puder ser encontrada. As abordagens mais comuns são formuladas da seguinte forma:

(c) Defina t como um nó terminal se contiver menos que N_{min} amostras. (N_{min} também é conhecido como min_samples_split.) (d) Defina t como um nó terminal se sua profundidade d_t for maior ou igual a um limite d_{max}. (d_{max} também é conhecido como max_profundidade.) (e) Defina t como um nó terminal se a diminuição total na impureza for menor que um limite fixo β. Isto é, se $p(t)\Delta i(s^*, t) < \beta$. (f) Defina t como um nó terminal se não houver divisão tal que t_L et_R ambos contem pelo menos N_{leaf} amostras. (N_{leaf} também é conhecido como min_samples_leaf.)

Em todos os itens acima, os critérios de parada são definidos em termos de hiperparâmetros definidos pelo usuário (N_{min}, d_{max}, β ou N_{leaf}) que devem ser ajustados para encontrar a compensação correta. Idealmente, eles precisam ser tais que não sejam nem muito rígidos nem muito soltos para que a árvore não seja nem muito rasa nem muito profunda. Uma árvore muito grande terá um erro de generalização maior do que uma árvore de tamanho correto. Da mesma forma, uma árvore muito pequena não utilizará algumas das informações em $\mathcal{L}$, resultando novamente em um erro de generalização maior do que a árvore de tamanho correto. Conforme descrito na Seção 2.3, a escolha dos valores de parâmetros apropriados geralmente é realizada usando um procedi mento de seleção de modelo dedicado, que pode ser computacionalmente caro, mas geralmente imperativo para alcançar um bom desempenho de generalização.

PRE-PRUNING AND POST-PRUNING

While the stopping criteria presented above may give good results in practice, the strategy of stopping early the induction of the tree is in general unsatisfactory. There may be nodes t for which the stopping criterion is met but whose descendants t_L and t_R may have splits that would have in fact reduced the generalization error of the tree. By declaring t as a terminal node, the good splits on t_L and t_R are never exploited.

Another way of looking at the problem of finding the right sized tree consists in fully developing all nodes and then to *prune* instead of stopping. That is, to sequentially remove the nodes that degrade the generalization error (estimated on an independent validation set) until the optimal tree is found. Since nodes are pruned after the induction of the tree, this framework is also known as *post-pruning*. By opposition, early stopping as described earlier is also referred to as *pre-pruning*.

In the context of single decision trees, post-pruning usually yields better results than pre-pruning. From an interpretability point of view, it is also a very effective framework for simplifying decision trees and better understand the structure in the data. However, in the context of ensemble of decision trees, we will see in Chapter 4 that (pre- or post-)pruning is no longer required to achieve good generalization performance. For this reason, we refer to the literature for more detailed discussion on the topic (e.g., [Breiman et al., 1984; Mingers, 1989a; Zighed and Rakotomalala, 2000]).

3.6 SPLITTING RULES

Assuming that the stopping criterion is not met, we now focus on the problem of finding the split $s^* \in Q$ of t that maximizes the impurity decrease $\Delta i(s^*, t)$ (Line 10 of Algorithm 3.2).

3.6.1 *Families Q of splitting rules*

As introduced in Section 3.2, a split s of node t is a question that divides the space $\mathcal{X}_t$ into disjoint subspaces respectively corresponding to each of the children of t. Formally, a split is defined as a partition:

Definition 3.7. *A* split s *of node* t *is a partition of* $\mathcal{X}_t$*, that is a set of non-empty subsets of* $\mathcal{X}_t$ *such that every element* $\mathbf{x} \in \mathcal{X}_t$ *is in exactly one of these subsets (i.e.,* $\mathcal{X}_t$ *is a disjoint union of the subsets).*

If s divides $\mathcal{X}_t$ into two subsets, then s is said to be a *binary* split and its left and right children are denoted t_L and t_R. In the general case however, s may divide $\mathcal{X}_t$ into more than two subsets, resulting in as many child nodes t_{i_1}, ..., t_{i_N} as subsets (See Figure 3.3).

Pré-poda e pós-poda

Embora os critérios de interrupção apresentados acima possam dar bons resultados na prática, a estratégia de interromper precocemente a indução da árvore é em geral insatisfatória. Pode haver nós t para os quais o critério de parada é atendido, mas cujos descendentes t_L e t_R podem ter divisões que teriam de fato reduzido o erro de generalização da árvore. Ao declarar t como um nó terminal, as boas divisões em t_L e t_R nunca são exploradas.

Outra forma de encarar o problema de encontrar a árvore do t amanho certo consiste em desenvolver completamente todos os nós e depois podá-los em vez de parar. Ou seja, remover sequencialmente os nós que degradam o erro de generalização (estimado em um conjunto de validação independente) até que a árvore ótima seja encontrada. Como os nós são podados após a indução da árvore, esta estrutura também é conhecida como pós-poda. Por oposição, a parada antecipada conforme descrita anteriormente também é chamada de pré-poda.

No contexto de árvores de decisão únicas, a pós-poda geralmente produz melhores resultados do que a pré-poda. Do ponto de vista da interpretabilidade, é também uma estrutura muito eficaz para simplificar árvores de decisão e compreender melhor a estrutura dos dados. Contudo, no contexto do conjunto de árvores de decisão, veremos no Capítulo 4 que a poda (pré ou pós) não é mais necessária para alcançar um bom desempenho de generalização. Por esta razão, recorremos à literatura para uma discussão mais detalhada sobre o tema (por exemplo, [Breiman et al., 1984; Mingers, 1989a; Zighed e Rakotomalala, 2000]).

3.6 regras de divisão

Supondo que o critério de parada não seja atendido, focamos agora no problema de encontrar a divisão $s^* \in Q$ de t que maximiza a diminuição de impureza $\Delta i(s^*, t)$ (Linha 10 do Algoritmo 3.2).

3.6.1 Famílias Q de regras de divisão

Conforme apresentado na Seção 3.2, uma divisão s do nó t é uma questão que divide o espaço $\mathcal{X}_t$ em subespaços disjuntos, respectivamente correspondentes a cada um dos filhos de t. Formalmente, uma divisão é definida como uma partição:

Definição 3.7. Uma divisão s *do nó* t *é uma partição de* $\mathcal{X}_t$*, que é um conjunto de subconjuntos não vazios de* $\mathcal{X}_t$ *tal que cada elemento* $x \in \mathcal{X}_t$ *está em exatamente um desses subconjuntos (ou seja,* $\mathcal{X}_t$ *é uma união disjunta dos subconjuntos).*

Se s divide $\mathcal{X}_t$ em dois subconjuntos, então s é considerado uma divisão binária e seus filhos esquerdo e direito são denotados t_L e t_R. No caso geral, entretanto, s pode dividir $\mathcal{X}_t$ em mais de dois subconjuntos, resultando em tantos nós filhos t_{i_1}, ..., t_{i_N} como subconjuntos (veja a Figura 3.3).

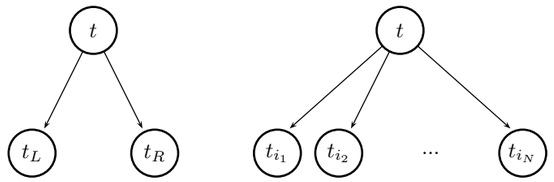


Figure 3.3: Binary split of t (left) and N-ary multiway split of t (right).

The set $\mathcal{S}$ of all possible splits s is the set of all partitions of $\mathcal{X}_t$. It is infinitely large as soon as one the input variables can take infinitely many values. For instance, if $\mathcal{X}_t = \mathbb{R}$, then there exists infinitely many ways of partitioning $\mathcal{X}_t$ into two (or more) subsets. Among all of them however, only those that yield a partition of $\mathcal{L}_t$ into non-empty subsets are worth considering. For example, in the binary case, splits resulting in one of the children being empty are not considered because their goodness cannot be evaluated on the learning set. As such, the optimization problem of finding the best split s^* of $\mathcal{X}_t$ can be roughly restated as finding the best partition of the node samples $\mathcal{L}_t$.

Assuming distinct input values for all N_t node samples, the number of partitions of $\mathcal{L}_t$ into k non-empty subsets is given by the Stirling number of the second kind [Knuth, 1992]

$$S(N_t, k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^{N_t}, \tag{3.12}$$

which reduces to $2^{N_t-1}-1$ for binary partitions. Given the exponential growth in the number of partitions, the naive strategy of enumerating all partitions and picking the best of them is often computationally intractable. For this reason, simplifying assumptions must be made on the best split s^*. More specifically, the induction algorithm usually assumes that s^* – or at least a good enough approximation – lives in a family $\mathcal{Q} \subseteq \mathcal{S}$ of candidate splits of restricted structure.

The usual family $\mathcal{Q}$ of splits is the set of binary splits defined on a single variable and resulting in non-empty subsets of $\mathcal{L}_t$:

$$\mathcal{Q} = \{s | s \in \bigcup_{j=1}^{p} \mathcal{Q}(X_j), \mathcal{L}_{t_L} \neq \phi, \mathcal{L}_{t_R} \neq \phi\} \tag{3.13}$$

- If X_j is an ordered variable taking values in $\mathcal{X}_j$, then the set of binary splits on X_j is the set of all binary non-crossing partitions s_j^v of $\mathcal{X}_j$:

$$\mathcal{Q}(X_j) = \{(\{\mathbf{x}|x_j \leqslant v\}, \{\mathbf{x}|x_j > v\}) | v \in \mathcal{X}_j\} \tag{3.14}$$

From a geometrical point of view, splits of this form partition the input space $\mathcal{X}$ with axis-parallel hyperplanes, as previously illustrated on Figure 3.2. Accordingly, the decision threshold v

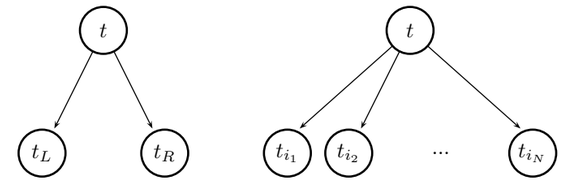


Figura 3.3: Divisão binária de t (esquerda) e divisão multiway N-ária de t (direita).

O conjunto $\mathcal{S}$ de todas as divisões possíveis s é o conjunto de todas as partições de $\mathcal{X}_t$. É infinitamente grande assim que uma das variáveis de entrada pode assumir infinitos valores. Por exemplo, se $\mathcal{X}_t = \mathbb{R}$, então existem infinitas maneiras de particionar $\mathcal{X}_t$ em dois (ou mais) subconjuntos. Entre todos eles, entretanto, apenas vale a pena considerar aqueles que produzem uma partição de $\mathcal{L}_t$ em subconjuntos não vazios. Por exemplo, no caso binário, as divisões que resultam em um dos filhos vazio não são consideradas porque a sua qualidade não pode ser avaliada no conjunto de aprendizagem. Como tal, o problema de otimização de encontrar a melhor divisão s^* de $\mathcal{X}_t$ pode ser aproximadamente reformulado como encontrar a melhor partição das amostras de nós $\mathcal{L}_t$.

Assumindo valores de entrada distintos para todas N_t amostras de nós, o número de partições de $\mathcal{L}_t$ em k subconjuntos não vazios é dado pelo número de Stirling do segundo tipo [Knuth, 1992]

$$S(N_t, k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^{N_t}, \tag{3.12}$$

que se reduz a $2^{N_t-1}-1$ para partições binárias. Dado o crescimento exponencial no número de partições, a estratégia ingênua de enumerar todas as partições e escolher a melhor delas é muitas vezes computacionalmente intratável. Por esta razão, devem ser feitas suposições simplificadoras sobre a melhor divisão s^*. Mais especificamente, o algoritmo de indução geralmente assume que s^* – ou pelo menos uma aproximação suficientemente boa – vive em uma família $\mathcal{Q} \subseteq \mathcal{S}$ de candidatos a divisões de estrutura restrita.

A família usual $\mathcal{Q}$ de divisões é o conjunto de divisões binárias definidas em uma única variável e resultando em subconjuntos não vazios de $\mathcal{L}_t$:

$$\mathcal{Q} = \{s | s \in \bigcup_{j=1}^{p} \mathcal{Q}(X_j), \mathcal{L}_{t_L} \neq \phi, \mathcal{L}_{t_R} \neq \phi\} \tag{3.13}$$

- Se X_j é uma variável ordenada que assume valores em $\mathcal{X}_j$, então o conjunto de divisões binárias em X_j é o conjunto de todas as partições binárias não cruzadass_j^v de $\mathcal{X}_j$:

$$\mathcal{Q}(X_j) = \{(\{\mathbf{x}|x_j \leqslant v\}, \{\mathbf{x}|x_j > v\}) | v \in \mathcal{X}_j\} \tag{3.14}$$

Do ponto de vista geométrico, divisões desta forma particionam o espaço de entrada $\mathcal{X}$ com hiperplanos paralelos aos eixos, conforme ilustrado anteriormente na Figura 3.2. Assim, o limite de decisão v

thus corresponds to the distance of the separating hyperplane from the origin.

- If X_j is a categorical variable taking values in $\mathcal{X}_j = \{b_1, \ldots, b_L\}$, then the set of binary splits on X_j is the set of all binary non-empty partitions of $\mathcal{X}_j$:

$$Q(X_j) = \{(\{\mathbf{x}|x_j \in \mathcal{B}\}, \{\mathbf{x}|x_j \in \overline{\mathcal{B}}\}) | \mathcal{B} \subset \{b_1, \ldots, b_L\}\} \tag{3.15}$$

where $\overline{\mathcal{B}} = \{b_1, \ldots, b_L\} \setminus \mathcal{B}$ is the complementary set of $\mathcal{B}$.

Given decomposition 3.13 into subsets $Q(X_j)$, the split $s^* \in Q$ is the best of the best splits defined on each input variable. That is,

$$s^* = \arg\max_{\substack{s_j^* \\ j=1,\ldots,p}} \Delta i(s_j^*, t) \tag{3.16}$$

$$s_j^* = \arg\max_{\substack{s \in Q(X_j) \\ \mathcal{L}_{t_L}, \mathcal{L}_{t_R} \neq \phi}} \Delta i(s, t) \tag{3.17}$$

In this framework, the crux of the problem therefore simply reduces to the implementation of Equation 3.17.

While partitions of form 3.14 and 3.15 certainly constitute the most common types of splits, alternatives proposed over the years in the literature are worth mentioning. In ID3 and C4.5, Quinlan [1986, 1993] replaces binary splits on categorical variables with multiway splits. That is, if $\mathcal{X}_j$ counts L different values $b_1, \ldots, b_L$, then splitting on X_j divides $\mathcal{X}_t$ into L child nodes – one for each value of the variable. In our framework, $Q(X_j)$ therefore reduces to the singleton

$$Q(X_j) = \{(\{\mathbf{x}|x_j = b_l\} | b_l \in \mathcal{X}_j)\}. \tag{3.18}$$

In oblique decision trees, Heath et al. [1993] propose to replace axis-parallel cutting hyperplanes with hyperplanes of any orientation, resulting into smaller, yet as accurate, decision trees. In this setting however, decomposition 3.16 no longer holds, hence making the search of the best split often more computationally intensive. More recently, several authors (e.g., [Gama, 2004; Criminisi and Shotton, 2013; Botta, 2013]) outlined more general frameworks in which splits are defined as multi-variate functions, thereby revisiting separating hyperplanes like in oblique trees or investigating more advanced models like quadratic surfaces or decision trees inside decision trees. Finally, in an another direction, several authors [Adamo, 1980; Yuan and Shaw, 1995; Olaru and Wehenkel, 2003] propose in fuzzy decision trees to redefine the notion of split as overlapping fuzzy sets (instead of disjoint subsets), hence allowing samples close to the decision threshold ν to propagate into both child nodes.

corresponde assim à distância do hiperplano de separação da origem.

- Se X_j é uma variável categórica que assume valores em $\mathcal{X}_j = \{b_1, \ldots, b_L\}$, então o conjunto de divisões binárias em X_j é o conjunto de todas as partições binárias não vazias de $\mathcal{X}_j$:

$$Q(X_j) = \{(\{\mathbf{x}|x_j \in \mathcal{B}\}, \{\mathbf{x}|x_j \in \overline{\mathcal{B}}\}) | \mathcal{B} \subset \{b_1, \ldots, b_L\}\} \tag{3.15}$$

onde $\overline{\mathcal{B}} = \{b_1, \ldots, b_L\} \setminus \mathcal{B}$ é o conjunto complementar de $\mathcal{B}$.

Dada a decomposição 3.13 em subconjuntos $Q(X_j)$, a divisão $s^* \in Q$ é a melhor das melhores divisões definidas em cada variável de entrada. Aquilo é,

$$s^* = \arg\max_{\substack{s_j^* \\ j=1,\ldots,p}} \Delta i(s_j^*, t) \tag{3.16}$$

$$s_j^* = \arg\max_{\substack{s \in Q(X_j) \\ \mathcal{L}_{t_L}, \mathcal{L}_{t_R} \neq \phi}} \Delta i(s, t) \tag{3.17}$$

Neste quadro, o cerne do problema reduz-se, portanto, simplesmente à implementação da Equação 3.17.

Embora as partições dos formatos 3.14 e 3.15 certamente constituam os tipos mais comuns de divisões, vale a pena mencionar as alternativas propostas ao longo dos anos na literatura. Em ID3 e C4.5, Quinlan [1986, 1993] substitui divisões binárias em variáveis categóricas por divisões multidirecionais. Ou seja, se $\mathcal{X}_j$ contar L valores diferentes $b_1, \ldots, b_L$ e, em seguida, a divisão em $X_j$divide $\mathcal{X}_t$ em Lnós filhos – um para cada valor da variável. Em nossa estrutura, $Q(X_j)$ portanto, reduz-se ao singleton

$$Q(X_j) = \{(\{\mathbf{x}|x_j = b_l\} | b_l \in \mathcal{X}_j)\}. \tag{3.18}$$

Em árvores de decisão oblíquas, Heath et al. [1993] propõem substituir hiperplanos de corte paralelos aos eixos por hiperplanos de qualquer orientação, resultando em árvores de decisão menores, mas tão precisas. Neste cenário, entretanto, a decomposição 3.16 não é mais válida, t ornando a busca da melhor divisão muitas vezes mais intensiva em ter mos computacionais. Mais recentemente, vários autores (por exemplo, [Gama, 2004; Criminisi e Shotton, 2013; Botta,2013]) delinearam estruturas mais gerais nas quais as divisões são definidas como funções multivariadas, revisitando assim hiperplanos de separação como em árvores oblíquas ou investigando modelos mais avançados como superfícies quadráticas ou árvores de decisão dentro de árvores de decisão. Finalmente, numa outra direção, vários autores [Adamo, 1980; Yuan e Shaw, 1995; Olaru e Wehenkel, 2003] propõem em árvores de decisão fuzzy redefinir a noção de divisão como conjuntos fuzzy sobrepostos (em vez de subconjuntos disjuntos), permitindo assim que amostras próximas ao limite de decisão ν se propaguem em ambos os nós filhos.

i	X_1	X_2	X_3	Y
0	0	0	0	c_1
1	0	0	1	c_1
2	0	1	0	c_2
3	0	1	1	c_2
4	0	1	1	c_2
5	1	0	0	c_2
6	1	0	0	c_2
7	1	0	0	c_2
8	1	0	0	c_2
9	1	1	1	c_2

Table 3.1: Toy binary classification problem

3.6.2 *Goodness of split*

In this section, we describe impurity measures $i(t)$ for evaluating the goodness of splitting rules. As they are the heart of our contributions regarding interpretability of tree-based methods (See Chapter 6), the necessary mathematical foundations that have driven their methodological development are studied here in details. Section 3.6.2.1 reviews impurity functions for classification while Section 3.6.2.2 discusses criteria for regression.

3.6.2.1 *Classification*

Let us consider a binary classification problem defined on three binary categorical input variables X_1, X_2 and X_3. Let us assume that we have a 10-sample learning set $\mathcal{L}$ as listed in Table 3.1. At the root node t_0, $\mathcal{L}_{t_0}$ contains all learning samples and we are looking for the best binary split defined on one of the input variables. Since each of them is binary and categorical, $Q(X_j)$ (for $j = 1, 2, 3$) counts exactly one single split per input variable, each partitioning $\mathcal{L}_{t_0}$ into two subsets

$$\mathcal{L}_{t_L} = \{(\mathbf{x}, y) | (\mathbf{x}, y) \in \mathcal{L}_{t_0}, x_j = 0\},$$
$$\mathcal{L}_{t_R} = \{(\mathbf{x}, y) | (\mathbf{x}, y) \in \mathcal{L}_{t_0}, x_j = 1\}.$$

As introduced in Section 3.3, our objective is to partition t_0 using the split s^* that maximizes the impurity decrease

$$\Delta i(s, t) = i(t) - p_L i(t_L) - p_R i(t_R),$$

where the impurity function $i(t)$ evaluates the goodness of node t. Since our goal is to build the decision tree that minimizes generalization error, it seems natural to take as proxy $i(t)$ the local resubstitution estimate at node t:

i	X_1	X_2	X_3	Y
0	0	0	0	c_1
1	0	0	1	c_1
2	0	1	0	c_2
3	0	1	1	c_2
4	0	1	1	c_2
5	1	0	0	c_2
6	1	0	0	c_2
7	1	0	0	c_2
8	1	0	0	c_2
9	1	1	1	c_2

Tabela 3.1: Problema de classificação binária de brinquedos

3.6.2 Bondade da divisão

Nesta seção, descrevemos medidas de impureza $i(t)$ para avaliar a qualidade das regras de divisão. Como são o centro de nossas contribuições em relação à interpretabilidade de métodos baseados em árvores (ver Capítulo 6), os fundamentos matemáticos necessários que impulsionaram seu desenvolvimento metodológico são estudados aqui e m detalhes. A Seção 3.6.2.1 revisa funções de impureza para classificação enquanto a Seção 3.6.2.2 discute critérios para regressão.

3.6.2.1 Classificação

Consideremos um problema de classificação binária definido em três variáveis de entrada categóricas binárias X_1, X_2 e X_3. Suponhamos que temos um conjunto de aprendizagem de 10 amostras $\mathcal{L}$ conforme listado na Tabela 3.1. No nó raiz t_0, $\mathcal{L}_{t_0}$ contém todas as amostras de aprendizagem e procuramos a melhor divisão binária definida em uma das variáveis de entrada. Como cada um deles é binário e categórico, $Q(X_j)$ (para $j = 1, 2, 3$) conta exatamente uma única divisão por variável de entrada, cada particionamento $\mathcal{L}_{t_0}$ em dois subconjuntos

$$\mathcal{L}_{t_L} = \{(\mathbf{x}, y) | (\mathbf{x}, y) \in \mathcal{L}_{t_0}, x_j = 0\},$$
$$\mathcal{L}_{t_R} = \{(\mathbf{x}, y) | (\mathbf{x}, y) \in \mathcal{L}_{t_0}, x_j = 1\}.$$

Conforme apresentado na Seção 3.3, nosso objetivo é particionar t_0 usando a divisão s^* que maximiza a diminuição de impurezas

$$\Delta i(s, t) = i(t) - p_L i(t_L) - p_R i(t_R),$$

onde a função de impureza $i(t)$ avalia a qualidade do nó t. Como nosso objetivo é construir a árvore de decisão que minimize o erro de generalização, parece natural tomar como proxy $i(t)$ a estimativa de substituição local no nó t:

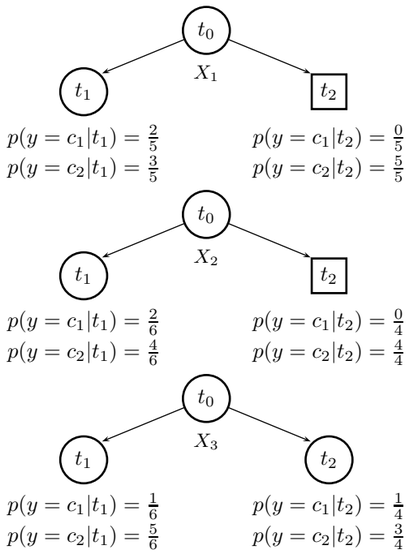


Figure 3.4: Splitting on X_1 versus splitting on either X_2 or X_3.

Definition 3.8. *In classification, the impurity function* $i_R(t)$ *based on the local resubstitution estimate defined on the zero-one loss is:*

$$i_R(t) = 1 - p(\widehat{y}_t|t) = 1 - \max_{c \in \mathcal{Y}} p(c|t) \tag{3.19}$$

With this criterion, selecting the best split would amount to pick the split that most decreases training error. In spite of its natural attractiveness however, $i_R(t)$ has two serious defects:

(a) As a corollary of Proposition 3.1, $\Delta i(s,t)$ is zero for all splits if the majority class remains the same in both child nodes, that is if $\widehat{y}_t = \widehat{y}_{t_L} = \widehat{y}_{t_R}$.

In our toy problem, splitting either on X_1, X_2 or X_3 produces child nodes in which c_2 remains the majority class. As a result, $\Delta i(s,t_0) = 0$ for all three possible splits, as if they were equally bad, or as if no further improvement could be achieved.

(b) It does not sensitively account for changes in the a posteriori class distributions $p(y|t_L)$ and $p(y|t_R)$.

In our example, none of the three splits improves training error. However, splitting on X_1 is likely to yield a tree which is simpler than if the root node had been split on either X_2 or X_3. As shown in Figure 3.4, splitting on X_1 indeed produces a right child node that is already terminal and for which no further work is required. Splitting on X_2 also results in the right child node to be pure. However, the left child node is in this

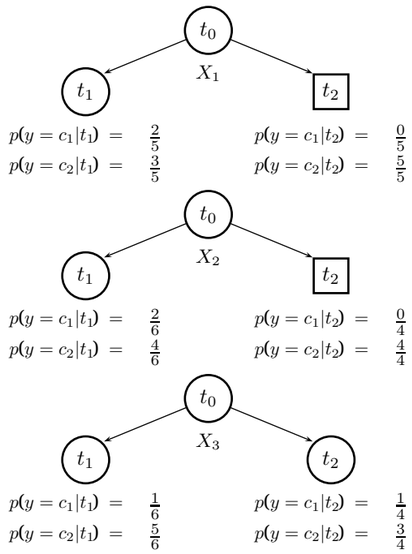


Figura 3.4: Divisão em X_1 versus divisão em X_2 ou X_3.

Definição 3.8. Na classificação, a função de impureza $i_R(t)$ *baseada na estimativa de ressubstituição local definida na perda zero-um é:*

$$i_R(t) = 1 - p(\widehat{y}_t|t) = 1 - \max_{c \in \mathcal{Y}} p(c|t) \tag{3.19}$$

Com este critério, selecionar a melhor divisão equivaleria a escolher a divisão que mais diminui o erro de treinamento. Apesar de sua atratividade natural, porém, $i_R(t)$ tem dois defeitos graves:

(a) Como corolário da Proposição 3.1, $\Delta i(s,t)$ é zero para todas as divisões se a classe majoritária permanecer a mesma em ambos os nós filhos, ou seja, se $\widehat{y}_t = \widehat{y}_{t_L} = \widehat{y}_{t_R}$.

Em nosso problema de brinquedo, a divisão em X_1, X_2 ou X_3 produz nós filhos nos quais c_2 continua sendo a classe majoritária. Como resultado,$\Delta i(s,t_0) = 0$ para todas as três divisões possíveis, como se fossem igualmente ruins ou como se nenhuma melhoria adicional pudesse ser alcançada.

(b) Não leva em conta sensivelmente as mudanças nas distribuições de classes a posteriori $p(y|t_L)$ e $p(y|t_R)$.

No nosso exemplo, nenhuma das três divisões melhora o erro de treino. No entanto, a divisão em X_1 provavelmente produzirá uma árvore que é mais simples do que se o nó raiz tivesse sido dividido em X_2 ouX_3. Como mostrado na Figura 3.4, a divisão em X_1 de fato produz um nó filho certo que já é terminal e para o qual nenhum trabalho adicional é necessário. A divisão em X_2 também faz com que o nó filho certo seja puro. No entanto, o nó filho esquerdo está neste

case larger, suggesting that more work is further required. Finally, splitting on X_3 results in child nodes that have to be both divided further. Splitting on X_1 should therefore be preferred, even if it does not reduce immediately the training error.

In fact, these issues are largely due to the fact that the tree growing algorithm 3.2 is based on a one-step optimization procedure. A good impurity criterion should ideally also take into account the possibility of further improvement deeper in the tree. To avoid the undesirable properties of $i_R(t)$, the choice of $i(t)$ should therefore be guided by ensuring that $i(t)$ gets progressively smaller when t gets more homogeneous towards one (or some) of the classes (while not being necessarily better in terms of misclassification error) and larger when t is more heterogeneous. In CART, Breiman et al. [1984] identify a class of impurity function $i(t)$ meeting these requirements:

Theorem 3.3. *Let $\Phi(p_1,\dots,p_J)$ be a strictly concave J-ary function defined on $0 \leqslant p_k \leqslant 1$, for $k = 1,\dots,J$, $\sum_{k=1}^{J} p_k = 1$ and such that*

- $\Phi(1,\dots,0) = \Phi(0,1,\dots,0) = \cdots = \Phi(0,\dots,1)$ *is minimal;*
- $\Phi(\frac{1}{J},\dots,\frac{1}{J})$ *is maximal.*

Then, for $i(t) = \Phi(p(c_1|t),\dots,p(c_J|t))$ and any split s,

$$\Delta i(s,t) \geqslant 0,$$

with equality if and only if $p(c_k|t_L) = p(c_k|t_R) = p(c_k|t)$ for $k = 1,\dots,J$.

Proof. Let us first remark that

$$\begin{aligned} p(c_k|t) &= \frac{N_{c_k t}}{N_t} \\ &= \frac{N_{c_k t_L} + N_{c_k t_R}}{N_t} \\ &= \frac{N_{t_L}}{N_t}\frac{N_{c_k t_L}}{N_{t_L}} + \frac{N_{t_R}}{N_t}\frac{N_{c_k t_R}}{N_{t_R}} \\ &= p_L p(c_k|t_L) + p_R p(c_k|t_R) \end{aligned}$$

By strict concavity, it comes

$$\begin{aligned} i(t) &= \Phi(p(c_1|t),\dots,p(c_J|t)) \\ &= \Phi(p_L p(c_1|t_L) + p_R p(c_1|t_R),\dots,p_L p(c_J|t_L) + p_R p(c_J|t_R)) \\ &\geqslant p_L \Phi(p(c_1|t_L),\dots,p(c_J|t_L)) + p_R \Phi(p(c_1|t_R),\dots,p(c_J|t_R)) \\ &= p_L i(t_L) + p_R i(t_R) \end{aligned}$$

with equality if and only if $p(c_k|t_L) = p(c_k|t_R)$ for $k = 1,\dots,J$. □

To better understand Theorem 3.3, let us consider a strictly concave function Φ that satisfies the requirements and let us evaluate on our

caso maior, sugerindo que mais trabalho é necessário. Finalmente, a divisão em X_3 resulta em nós filhos que precisam ser divididos ainda mais. Portanto, a divisão em X_1 deve ser preferida, mesmo que não reduza imediatamente o erro de treinamento.

Na verdade, esses problemas se devem em grande parte ao fato de que o algoritmo de crescimento de árvores 3.2 é baseado em um procedi mento de otimização de uma etapa. Idealmente, um bom critério de impureza também deve levar em conta a possibilidade de melhorias adicionais mais profundas na árvore. Para evitar as propriedades indesejáveis de $i_R(t)$, a escolha de $i(t)$ deve, portanto, ser orientada garantindo que $i(t)$ fique progressivamente menor quando t se torne mais homogêneo em relação a uma (ou algumas) das classes (embora não seja necessariamente melhor em termos de erro de classificação incorreta) e maior quando t é mais heterogêneo. No CART, Breiman et al. [1984] identificam uma classe de função de impureza $i(t)$ que atende a estes requisitos:

Teorema 3.3. Seja $\Phi(p_1,\dots,p_J)$ uma função J-ária estritamente côncava definida em $0 \leqslant p_k \leqslant 1$, para $k = 1,\dots,J$, $\sum_{k=1}^{J} p_k = 1$ e tal que

- $\Phi(1,\dots,0) = \Phi(0,1,\dots,0) = \cdots = \Phi(0,\dots,1)$ *is minimal;*
- $\Phi(\frac{1}{J},\dots,\frac{1}{J})$ *is maximal.*

Então, para $i(t) = \Phi(p(c_1|t),\dots,p(c_J|t))$ e qualquer divisão s,

$$\Delta i(s,t) \geqslant 0,$$

com igualdade se e somente se $p(c_k|t_L) = p(c_k|t_R) = p(c_k|t)$ para $k = 1,\dots,J$.

Prova. Observemos primeiro que

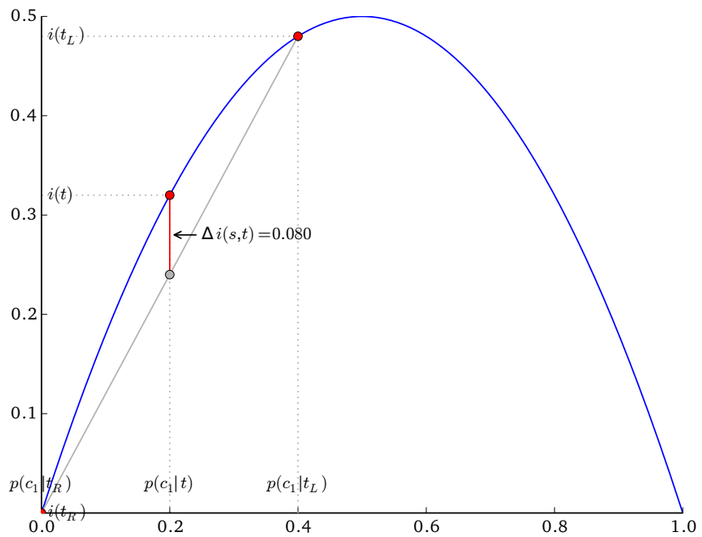
$$\begin{aligned} p(c_k|t) &= \frac{N_{c_k t}}{N_t} \\ &= \frac{N_{c_k t_L} + N_{c_k t_R}}{N_t} \\ &= \frac{N_{t_L}}{N_t}\frac{N_{c_k t_L}}{N_{t_L}} + \frac{N_{t_R}}{N_t}\frac{N_{c_k t_R}}{N_{t_R}} \\ &= p_L p(c_k|t_L) + p_R p(c_k|t_R) \end{aligned}$$

Por concavidade estrita, vem

$$\begin{aligned} i(t) &= \Phi(p(c_1|t),\dots,p(c_J|t)) \\ &= \Phi(p_L p(c_1|t_L) + p_R p(c_1|t_R),\dots,p_L p(c_J|t_L) + p_R p(c_J|t_R)) \\ &\geqslant p_L \Phi(p(c_1|t_L),\dots,p(c_J|t_L)) + p_R \Phi(p(c_1|t_R),\dots,p(c_J|t_R)) \\ &= p_L i(t_L) + p_R i(t_R) \end{aligned}$$

com igualdade se e somente se $p(c_k|t_L) = p(c_k|t_R)$ para $k = 1,\dots,J$. □

Para entender melhor o Teorema 3.3, vamos considerar uma função estritamente côncava Φ que satisfaz os requisitos e vamos avaliar com base em nossa



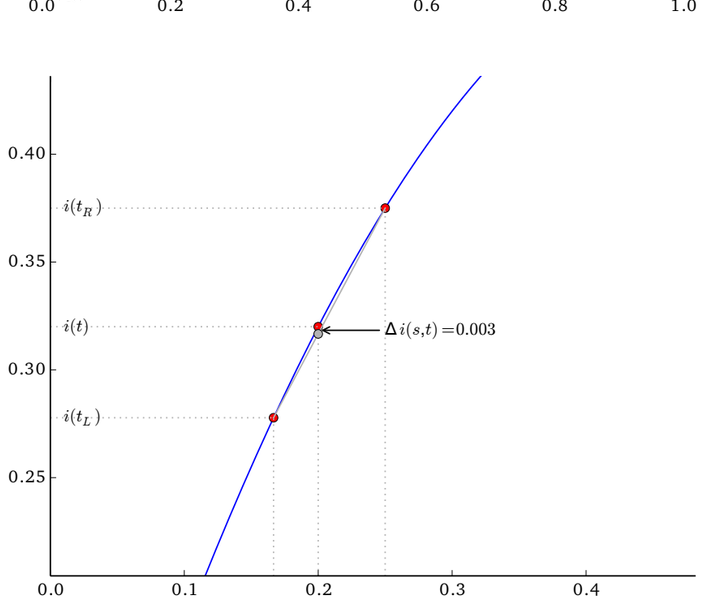
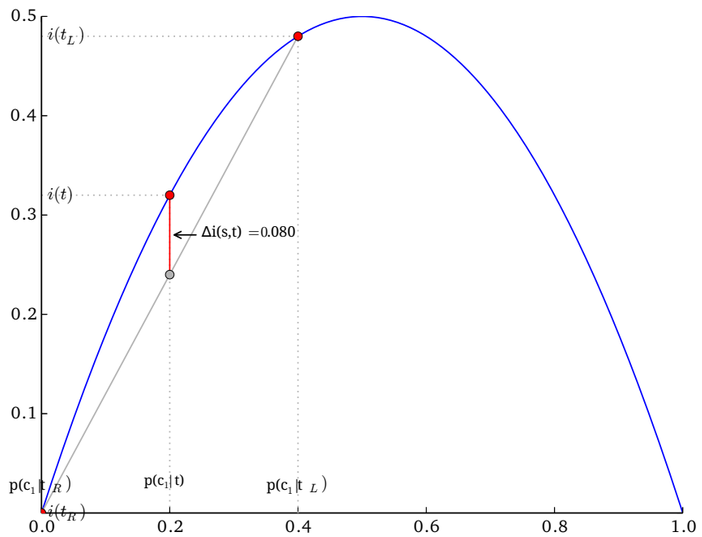


Figure 3.5: Splitting t_0 on X_1 (above) versus splitting t_0 on X_3 (below), as evaluated by a strictly concave impurity function. The figures show i with respect to the local probability $p(c_1)$ of the first class.



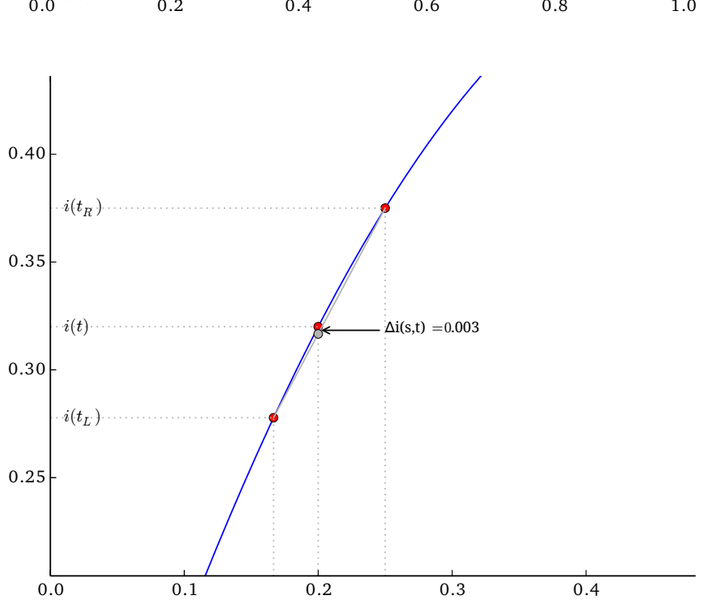


Figura 3.5: Divisão t_0 em X_1 (acima) versus divisão t_0 em X_3 (abaixo), conforme avaliado por uma função de impureza estritamente côncava. As fig uras mostram i em relação à probabilidade local $p(c_1)$ da primeira classe.

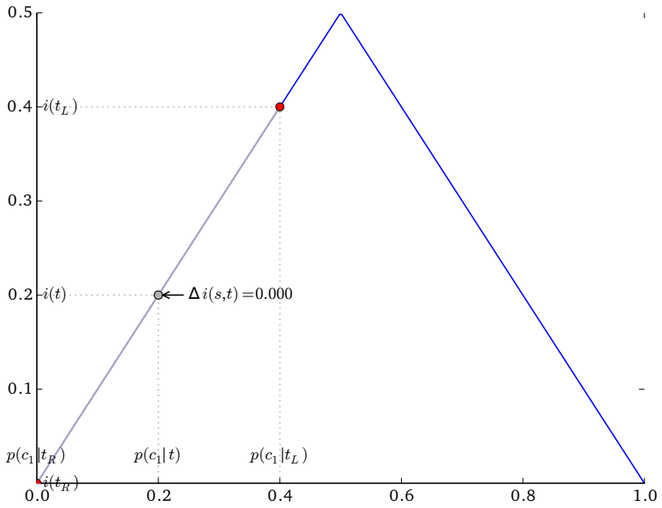


Figure 3.6: Splitting t_0 on X_1, as evaluated by the resubstitution estimate $i_R(t)$. The figure shows i_R with respect to the local probability $p(c_1)$ of the first class.

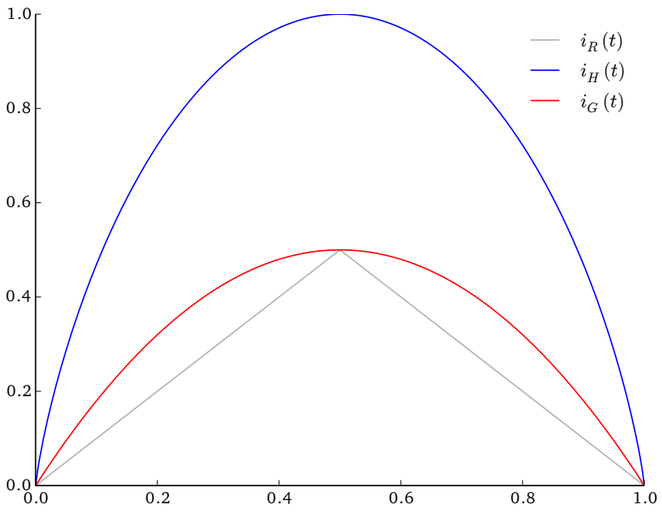


Figure 3.7: Comparison of $i_R(t)$, $i_H(t)$ and $i_G(t)$ with respect to $p(c_1|t)$.

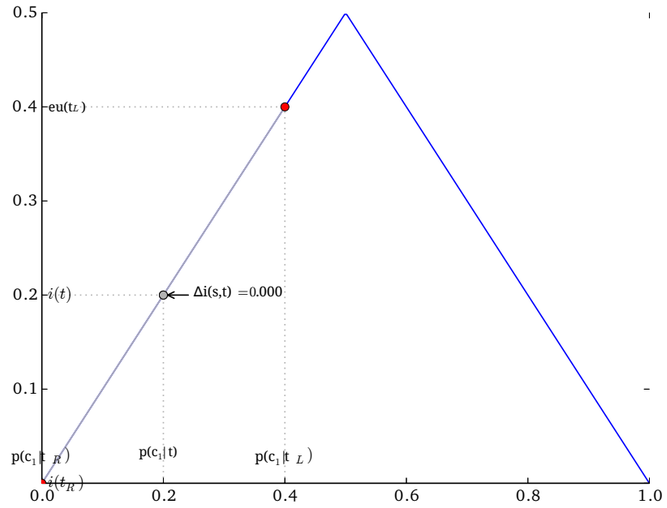


Figura 3.6: Divisão de t_0 em X_1, conforme avaliado pela estimativa de ressubstituição $i_R(t)$. A figura mostra i_R em relação à probabilidade local $p(c_1)$ da primeira classe.

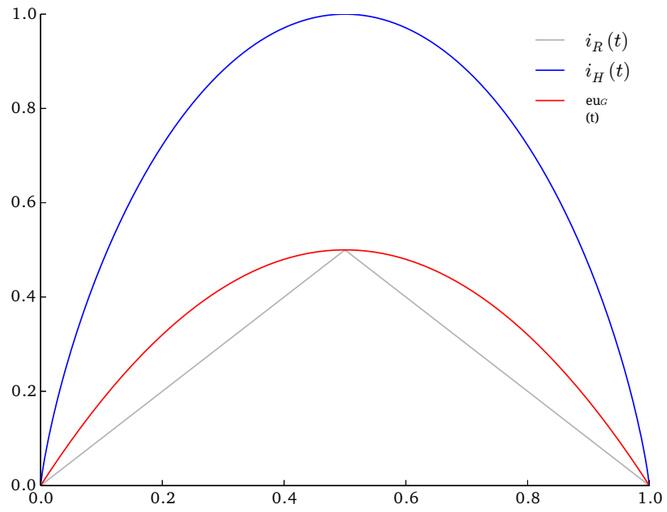


Figura 3.7: Comparação de $i_R(t)$, $i_H(t)$ e $i_G(t)$ com relação a $p(c_1|t)$.

toy problem the goodness of the split defined on X_1. As illustrated in the first plot of Figure 3.5, $i(t)$ is maximum when the uncertainty on the output value is the largest (i.e., at $p(c_1|t) = p(c_2|t) = \frac{1}{2}$) and then gets progressively smaller when certainty grows towards one of the classes (i.e., as $p(c_1|t)$ gets close to 0 or 1). As shown by Theorem 3.3, $i(t)$ is also necessarily larger than the weighted sum (illustrated by the gray dot) of impurities of the child nodes. Visually, the further away $p(c_1|t_L)$ and $p(c_1|t_R)$ from $p(c_1|t)$, the larger $\Delta i(s,t)$ (illustrated by the red line) and the better the split s. In particular, even if $i(t_L)$ is larger than $i(t)$ on the first plot of Figure 3.5, impurity is in fact globally smaller. By contrast, when s does not significantly change the a posteriori class distributions, as illustrated in the second plot of Figure 3.5 when splitting t_0 on X_3, then the closer $p(c_1|t_L)$ and $p(c_1|t_R)$ from $p(c_1|t)$ and the smaller $\Delta i(s,t)$. Yet, as long as the a posteriori class distributions do not exactly coincide with the a priori class distribution, then even the slightest changes get captured by $\Delta i(s,t)$ because of strict concavity. It is only when they exactly coincide (i.e., when $p(c_1|t_L) = p(c_1|t_R) = p(c_1|t)$) that the three red points overlap and that $\Delta i(s,t) = 0$.

Graphically, it also becomes obvious when comparing Figure 3.5 with Figure 3.6 why the resubstitution estimate $i_R(t)$ is not an appropriate impurity function. As long as the majority class within the child nodes t_L and t_R is the same as in t, the three red dots remain aligned in a perfect line and the gray dot overlap with $i(t)$. As a result, $\Delta i(s,t)$ is necessarily null no matter the a posteriori class distributions induced by the split, be them close or far from $p(c_1|t)$.

The most common impurity criteria used for classification trees are the Shannon entropy and the Gini index:

Definition 3.9. *The impurity function $i_H(t)$ based on the Shannon entropy [Shannon and Weaver, 1949] is:*

$$i_H(t) = -\sum_{k=1}^{J} p(c_k|t)\log_2(p(c_k|t)) \tag{3.20}$$

Definition 3.10. *The impurity function $i_G(t)$ based on the Gini index [Gini, 1912] is:*

$$i_G(t) = \sum_{k=1}^{J} p(c_k|t)(1 - p(c_k|t)) \tag{3.21}$$

As illustrated on Figure 3.7, both $i_H(t)$ and $i_G(t)$ satisfy the requirements of Theorem 3.3 and exhibit the properties that we are looking for. The entropy-based impurity $i_H(t)$ quantifies the uncertainty of Y within node t. With this criterion, the impurity decrease $\Delta i_H(s,t)$, also known as the *information gain*, represents the information learned about Y by splitting t into t_L and t_R. The Gini-based impurity $i_G(t)$

problema de brinquedo a qualidade da divisão definida em X_1. Conforme ilustrado no primeiro gráfico da Figura 3.5, $i(t)$ é máximo quando a incerteza no valor de saída é maior (ou seja, em $p(c_1|t) = p(c_2|t) = \frac{1}{2}$) e depois fica progressivamente menor quando a certeza cresce em direção a uma das classes (ou seja, à medida que $p(c_1|t)$ se aproxima de 0 ou 1). Conforme mostrado pelo Teorema 3.3,$i(t)$ também é necessariamente maior que a soma ponderada (ilustrada pelo ponto cinza) das impurezas dos nós filhos. Visualmente, quanto mais longe$p(c_1|t_L)$ e $p(c_1|t_R)$ de $p(c_1|t)$, maior será $\Delta i(s,t)$ (ilustrado pela linha vermelha) e melhor será a divisão s. Em particular, mesmo que $i(t_L)$ seja maior que $i(t)$ no primeiro gráfico da Figura 3.5, a impureza é de facto globalmente menor. Por outro lado, quando s não altera significativamente as distribuições de classe a posteriori, como ilustrado no segundo gráfico da Figura 3.5 ao dividir t_0 em X_3, então o mais próximo $p(c_1|t_L)$ e $p(c_1|t_R)$de $p(c_1|t)$ e o menor $\Delta i(s,t)$. No entanto, desde que as distribuições de classes a posteriori não coincidam exatamente com a distribuição de classes a priori, mesmo as menores alterações serão capturadas por $\Delta i(s,t)$ devido à concavidade estrita. Somente quando eles coincidem exatamente (ou seja, quando $p(c_1|t_L) = p(c_1|t_R) = p(c_1|t)$) é que os três pontos vermelhos se sobrepõem e isso $\Delta i(s,t) = 0$.

Graficamente, também se torna óbvio ao comparar a Figura 3.5 com a Figura 3.6 porque a estimativa de resubstituição $i_R(t)$ não é uma função de impureza apropriada. Contanto que a classe majoritária nos nós filhos t_L e t_R seja a mesma que em t, os três pontos vermelhos permanecem alinhados em uma linha perfeita e o ponto cinza se sobrepõe a $i(t)$. Como resultado,$\Delta i(s,t)$ é necessariamente nulo independentemente das distribuições de classes a posteriori induzidas pela divisão, sejam elas próximas ou distantes de $p(c_1|t)$.

Os critérios de impureza mais comuns usados para classificação de árvores são a entropia de Shannon e o índice de Gini:

Definição 3.9. A função de impureza $i_H(t)$ baseada na entropia de Shannon [Shannon e Weaver, 1949] é:

$$i_H(t) = -\sum_{k=1}^{J} p(c_k|t)\log_2(p(c_k|t)) \tag{3.20}$$

Definição 3.10. A função de impureza $i_G(t)$ baseada no índice de Gini [Gini, 1912] é:

$$i_G(t) = \sum_{k=1}^{J} p(c_k|t)(1 - p(c_k|t)) \tag{3.21}$$

Conforme ilustrado na Figura 3.7, tanto $i_H(t)$ quanto $i_G(t)$ s atisfazem os requisitos do Teorema 3.3 e exibem as propriedades que procuramos. A impureza baseada em entropia $i_H(t)$ quantifica a incerteza de$Y$ dentro do nó t. Com este critério, a diminuição de impurezas $\Delta i_H(s,t)$, também conhecida como ganho de informação, representa a informação aprendida sobre Y dividindo t em t_L e t_R. A impureza baseada em Gini $i_G(t)$

measures how often a randomly chosen object $\mathbf{x} \in \mathcal{L}_t$ would be incorrectly classified if it were randomly labeled by a class $c \in \mathcal{Y}$ according to the distribution $p(y|t)$.

While the Shannon entropy 3.20 and the Gini index 3.21 are robust and reliable impurity functions usually producing accurate decision trees, they are not exempt of defects. One of the most well-known issues is end-cut preference [Morgan and Messenger, 1973; Breiman et al., 1984], that is the tendency to favor unbalanced splits in which p_L is close to zero or one, often resulting in deep and uninterpretable decision trees. Another defect is the propensity of preferring splits based on input variables with many outcomes rather than selecting input variables without bias with respect to their cardinality [Quinlan, 1986; Strobl et al., 2007a]. Intuitively, if $|\mathcal{X}_1| > |\mathcal{X}_2|$ and that both X_1 and X_2 are independent of Y, then X_1 is indeed more likely to be selected than X_2 because of chance. To overcome these issues, numerous variants of impurity measures have been proposed in the literature, including normalization techniques, distance-based measures or causality-based impurity functions (see [Wehenkel, 1996; Zighed and Rakotomalala, 2000; Maimon and Rokach, 2005] for reviews on the topic). From an empirical point of view however, several studies [Mingers, 1989b; Miyakawa, 1989; De Mántaras, 1991] have shown that, while the impurity function may have a significant impact on the structure of the decision tree, improvements in terms of classification accuracy are usually not that significant.

3.6.2.2 *Regression*

When the output variable Y is quantitative, Proposition 3.2 shows that splitting t in any way reduces the squared error loss on the training set. In particular, the reduction is positive as long as the values assigned to the child nodes are different from the value at t. Contrary to classification, this only happens in rare circumstances (i.e., when the mean output values at the child nodes coincide with the mean output value in t), hence making the regression resubstitution estimate sensitive to changes in (the means of) the a posteriori distributions, even if these are only slight. As a result, the local resubstitution estimate does not exhibit the undesirable properties that we had in classification and in fact constitutes a good criterion for regression.

Definition 3.11. *In regression, the impurity function* $i_R(t)$ *based on the local resubstitution estimate defined on the squared error loss is:*

$$i_R(t) = \frac{1}{N_t} \sum_{x,y \in \mathcal{L}_t} (y - \hat{y}_t)^2 \tag{3.22}$$

Another way of looking at criterion 3.22 is to notice that it corresponds to the within node variance of the output value in t. Accordingly, s^* is the split that maximizes the reduction of variance $\Delta i(s,t)$ in the child nodes.

mede a frequência com que um objeto $x \in \mathcal{L}_t$ escolhido aleatoriamente seria classificado incorretamente se fosse rotulado aleatoriamente por uma classe $c \in \mathcal{Y}$ de acordo com a distribuição $p(y|t)$.

Embora a entropia de Shannon 3.20 e o índice de Gini 3.21 sejam funções de impureza robustas e confiáveis, geralmente produzindo árvores de decisão precisas, eles não estão isentos de defeitos. Uma das questões mais conhecidas é a preferência de corte final [Morgan e Messenger, 1973; Breiman et al., 1984], que é a tendência de favorecer divisões desequilibradas em que p_L é próximo de zero ou um, muitas vezes resultando em árvores de decisão profundas e não interpretáveis. Outro defeito é a propensão de preferir divisões baseadas em variáveis de entrada com muitos resultados, em vez de selecionar variáveis de entrada sem viés em relação à sua cardinalidade [Quinlan, 1986; Strobl et al., 2007a]. Intuitivamente, se $|\mathcal{X}_1| > |\mathcal{X}_2|$ e que ambosX_1 e X_2 são independentes de Y, então X_1 é de fato mais provável de ser selecionado do que X_2 por causa do acaso. Para superar esses problemas, inúmeras variantes de medidas de impureza foram propostas na literatura, i ncluindo técnicas de normalização, medidas baseadas em distância ou funções de impureza baseadas em causalidade (ver [Wehenkel, 1996; Zighed e Rakotomalala, 2000; Maimon e Rokach, 2005]para revisões sobre o tema). De um ponto de vista empírico, entretanto, vários estudos [Mingers, 1989b; Miyakawa, 1989; De Mántaras, 1991]mostraram que, embora a função de impureza possa ter um impacto significativo na estrutura da árvore de decisão, as melhorias em termos de precisão da classificação geralmente não são tão significativas.

3.6.2.2 Regressão

Quando a variável de saída Y é quantitativa, a Proposição 3.2 mostra que a divisão t de qualquer forma reduz a perda de erro quadrático no c onjunto de treinamento. Em particular, a redução é positiva desde que os valores atribuídos aos nós filhos sejam diferentes do valor em t. Ao contrário da classificação, isto só acontece em circunstâncias raras (ou seja, quando os valores médios de saída nos nós filhos coincidem com o valor médio de saída em t), tornando assim a estimativa de substituição da regressão sensível a mudanças nas (médias das) distribuições a posteriori, mesmo que estas sejam apenas ligeiras. Como resultado, a estimativa de substituição local não apresenta as propriedades i ndesejáveis que tivemos na classificação e de facto constitui um bom critério para regressão.

Definição 3.11. Na regressão, a função de impureza $i_R(t)$ *baseada na estimativa de ressubstituição local definida na perda de erro quadrático é:*

$$i_R(t) = \frac{1}{N_t} \sum_{x,y \in \mathcal{L}_t} (y - \hat{y}_t)^2 \tag{3.22}$$

Outra forma de olhar para o critério 3.22 é notar que ele corresponde à variância dentro do nó do valor de saída em t. Conseqüentemente, s^* é a divisão que maximiza a redução da variância $\Delta i(s,t)$nos nós filhos.

EQUIVALENCE BETWEEN CLASSIFICATION AND REGRESSION TREES

When $\mathcal{Y} = \{0, 1\}$, the learning problem can either be stated as a binary classification task where $c_1 = 0$ and $c_2 = 1$ or as a regression task where the goal is to predict the most accurate predictions $\hat{y} \in \mathcal{Y} \subseteq \mathbb{R}$. Interestingly, regression trees built in this setting are in fact strictly equivalent to classification trees built using the Gini index $i_G(t)$. Indeed since $\hat{y}_t = \frac{1}{N_t} \sum_{x,y \in \mathcal{L}_t} y = p(c_2|t) = 1 - p(c_1|t)$, it comes:

$$\begin{aligned} i_R(t) &= \frac{1}{N_t} \sum_{x,y \in \mathcal{L}_t} (y - \hat{y}_t)^2 \\ &= \frac{1}{N_t} \sum_{x,y \in \mathcal{L}_t} y - 2y\hat{y}_t + \hat{y}_t^2 \\ &= \hat{y}_t - 2\hat{y}_t^2 + \hat{y}_t^2 \\ &= \hat{y}_t(1 - \hat{y}_t) \\ &= p(c_2|t)(1 - p(c_2|t)) \\ &= \frac{1}{2} i_G(t) \end{aligned} \tag{3.23}$$

From a practical point of view, this means that both classification and regression trees could be implemented using the single criterion 3.22.

3.6.3 *Finding the best binary split*

Now that we have described families $\mathcal{Q}$ of splitting rules and impurity criteria to evaluate their respective goodness, the last thing which is missing to have a complete specification of the induction algorithm 3.2 is an efficient optimization procedure for finding the best split $s^* \in \mathcal{Q}$. Assuming that $\mathcal{Q}$ is the set of binary univariate splits, we showed in Section 3.6.1 that s^* is the best of the best binary splits s_j^* defined on each input variable. This leads to the following procedure:

Algorithm 3.3. *Find the best split s^* that partitions $\mathcal{L}_t$.*

```
1:  function FindBestSplit(L_t)
2:      Δ = −∞
3:      for j = 1, ..., p do
4:          Find the best binary split s_j* defined on X_j
5:          if Δi(s_j*, t) > Δ then
6:              Δ = Δi(s_j*, t)
7:              s* = s_j*
8:          end if
9:      end for
10:     return s*
11: end function
```

Let us now discuss line 4 for ordered and categorical variables.

Equivalência entre árvores de classificação e regressão

Quando $\mathcal{Y} = \{0, 1\}$, o problema de aprendizagem pode ser declarado como uma tarefa de classificação binária onde $c_1 = 0$ e $c_2 = 1$ ou como uma tarefa de regressão onde o objetivo é prever as previsões mais precisas $\hat{y} \in \mathcal{Y} \subseteq \mathbb{R}$. Curiosamente, as árvores de regressão construídas nesta configuração são, na verdade, estritamente equivalentes às árvores de classificação construídas usando o índice de Gini $i_G(t)$. Na verdade, desde $\hat{y}_t = \frac{1}{N_t} \sum_{x,y \in \mathcal{L}_t} y \equiv p(c_2|t) = 1 - p(c_1|t)$, vem:

$$\begin{aligned} i_R(t) &= \frac{1}{N_t} \sum_{x,y \in \mathcal{L}_t} (y - \hat{y}_t)^2 \\ &= \frac{1}{N_t} \sum_{x,y \in \mathcal{L}_t} y - 2y\hat{y}_t + \hat{y}_t^2 \\ &= \hat{y}_t - 2\hat{y}_t^2 + \hat{y}_t^2 \\ &= \hat{y}_t(1 - \hat{y}_t) \\ &= p(c_2|t)(1 - p(c_2|t)) \\ &= \frac{1}{2} i_G(t) \end{aligned} \tag{3.23}$$

Do ponto de vista prático, isto significa que tanto as árvores de classificação como as de regressão poderiam ser implementadas utilizando o único critério 3.22.

3.6.3 Encontrando a melhor divisão binária

Agora que descrevemos famílias $\mathcal{Q}$ de regras de divisão e critérios de impureza para avaliar sua respectiva qualidade, a última coisa que falta para ter uma especificação completa do algoritmo de indução 3.2 é um procedimento de otimização eficiente para encontrar a melhor divisão $s^* \in \mathcal{Q}$. Assumindo que $\mathcal{Q}$ é o conjunto de divisões binárias univariadas, mostramos na Seção 3.6.1 que s^* é a melhor das melhores divisões binárias s_j^*definidas em cada variável de entrada. Isto leva ao seguinte procedimento:

Algoritmo 3.3. Encontre a melhor divisão s^* que particiona $\mathcal{L}_t$.

1: função FindBestSplit($\mathcal{L}_t$) 2: $\Delta = -\infty$**3: para** $j = 1$**,. . . ,** p **faça 4: Encontre a melhor divisão binária** s_j^* **definida em** X_j**5: if** $\Delta i(s_j^*, t) > \Delta$ **then 6:** $\Delta = \Delta i(s_j^*, t)$**7:** $s^* = s_j^*$**8: end if 9: end for 10: return** s^***11: end f unction**

Vamos agora discutir a linha 4 para variáveis ordenadas e categóricas.

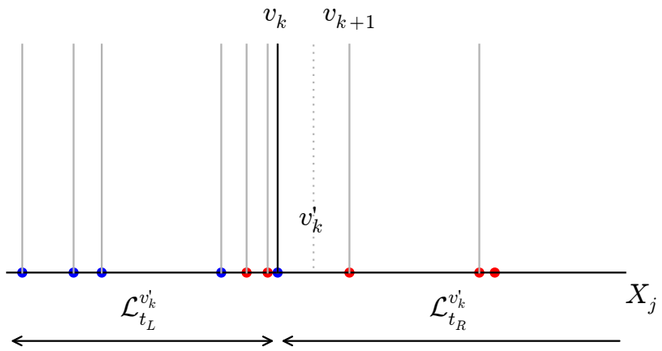


Figure 3.8: Binary partitions of $\mathcal{L}_t$ on the ordered variable X_j. Setting the decision threshold v to any value in $[v_k; v_{k+1}[$ yields identical partitions of $\mathcal{L}_t$, but not of $\mathcal{X}_t$.

3.6.3.1 *On an ordered variable*

Let X_j be an ordered variable and let $\mathcal{Q}(X_j)$ be the set of all binary non-crossing partitions of $\mathcal{X}_j$, as defined in Equation 3.14. Let also $\mathcal{X}_{j|\mathcal{L}_t} = \{x_j | \mathbf{x}, y \in \mathcal{L}_t\}$ denotes the set of unique values of X_j within the node samples in t. The best split $s_j^v \in \mathcal{Q}(X_j)$ on X_j is the best binary partition of $\mathcal{L}_t$ into two non-empty subsets:

$$\mathcal{L}_{t_L}^v = \{(\mathbf{x}, y) | (\mathbf{x}, y) \in \mathcal{L}_t, x_j \leqslant v\}$$
$$\mathcal{L}_{t_R}^v = \{(\mathbf{x}, y) | (\mathbf{x}, y) \in \mathcal{L}_t, x_j > v\}$$

where v is the decision threshold of the split. As illustrated in Figure 3.8, there exist $|\mathcal{X}_{j|\mathcal{L}_t}| - 1$ partitions of $\mathcal{L}_t$ into two such non-empty subsets, i.e., one for each value $v_k \in \mathcal{X}_{j|\mathcal{L}_t}$, except for the last one which leads to an invalid partition. In particular, if X_j is locally constant in t (i.e., if all dots overlap in Figure 3.8), then $|\mathcal{X}_{j|\mathcal{L}_t}| = 1$ and X_j cannot be used to partition t. More importantly, there usually exist several thresholds v producing the same partition of the node samples. If v_k and v_{k+1} are two immediately consecutive values in $\mathcal{X}_{j|\mathcal{L}_t}$, then all splits s_j^v for $v \in [v^k, v^{k+1}[$ indeed produce the same partition of $\mathcal{L}_t$ as $s_j^{v_k}$ does. In terms of impurity decrease Δi, all of them are thus equivalent when evaluated on $\mathcal{L}_t$. In generalization however, these splits may not be strictly the same since they do not produce the same partition of $\mathcal{X}_t$. As a compromise, the decision thresholds that are thus usually chosen are the mid-cut-points $v'_k = \frac{v_k + v_{k+1}}{2}$ between consecutive values of the variable, as shown by the dotted line in Figure 3.8. In practice, this is indeed a good-enough heuristic on most problems.

In this framework, the best split s_j^* is the split $s_j^{v'_k}$ that maximizes impurity decrease. Computationally, the exhaustive evaluation of all these splits can be carried out efficiently when considering decision thresholds v'_k in sorted order, by remarking that $\Delta i(s_j^{v'_{k+1}}, t)$ can be

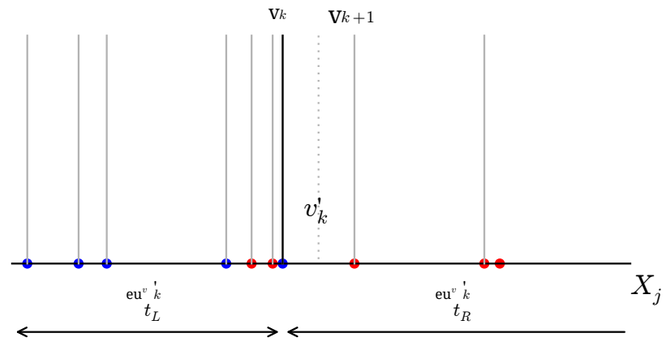


Figura 3.8: Partições binárias de $\mathcal{L}_t$ na variável ordenada X_j. Definir o limite de decisão v para qualquer valor em $[v_k; v_{k+1}[$ produz partições idênticas de $\mathcal{L}_t$, mas não de $\mathcal{X}_t$.

3.6.3.1 *Em uma variável ordenada*

Seja X_j uma variável ordenada e seja $\mathcal{Q}(X_j)$ o conjunto de todas as partições binárias não cruzadas de $\mathcal{X}_j$, conforme definido na Equação 3.14. Deixe também$\mathcal{X}_{j|\mathcal{L}_t} = \{x_j | \mathbf{x}, y \in \mathcal{L}_t\}$ denotar o conjunto de valores únicos de X_j dentro das amostras de nós em t. A melhor divisão $s_j^v \in \mathcal{Q}(X_j)$ em X_j é a melhor partição binária de $\mathcal{L}_t$ em dois subconjuntos não vazios:

$$\mathcal{L}_{t_L}^v = \{(\mathbf{x}, y) | (\mathbf{x}, y) \in \mathcal{L}_t, x_j \leqslant v\}$$
$$\mathcal{L}_{t_R}^v = \{(\mathbf{x}, y) | (\mathbf{x}, y) \in \mathcal{L}_t, x_j > v\}$$

onde v é o limite de decisão da divisão. Conforme ilustrado na Figura 3.8, existem partições $|\mathcal{X}_{j|\mathcal{L}_t}| - 1$ de $\mathcal{L}_t$ em dois desses subconjuntos não vazios, ou seja, um para cada valor $v_k \in \mathcal{X}_{j|\mathcal{L}_t}$, exceto o último que leva a uma partição inválida. Em particular, se X_j é localmente constante em t (isto é, se todos os pontos se sobrepõem na Figura 3.8), então $|\mathcal{X}_{j|\mathcal{L}_t}| = 1$ e $X_j$não podem ser usados para particionar t. Mais importante ainda, geralmente existem vários limites que produzem a mesma partição das amostras de nós. Se v_k e v_{k+1} são dois valores imediatamente consecutivos em $\mathcal{X}_{j|\mathcal{L}_t}$, então todas as divisões s_j^v para $v \in [v^k, v^{k+1}[$ de fato produzem a mesma partição de $\mathcal{L}_t$ que $s_j^{v_k}$. Em termos de diminuição de impurezas Δi, todos eles são, portanto, equivalentes quando avaliados em $\mathcal{L}_t$. Na generalização, porém, essas divisões podem não ser estritamente iguais, uma vez que não produzem a mesma partição de $\mathcal{X}_t$. Como compromisso, os limiares de decisão normalmente escolhidos são os pontos de corte médios $v'_k = \frac{v_k + v_{k+1}}{2}$ entre valores consecutivos da variável, como mostra a linha pontilhada na Figura 3.8. Na prática, esta é de fato uma heurística suficientemente boa para a maioria dos problemas.

Nesta estrutura, a melhor divisão s_j^* é a divisão $s^{v'_k}_j$ que maximiza a redução de impurezas. Computacionalmente, a avaliação exaustiva de todas essas divisões pode ser realizada de forma eficiente ao considerar os limites de decisão v'_k em ordem classificada, observando que $\Delta i(s_j^{v'_{k+1}}, t)$ pode ser

computed from $\Delta i(s_j^{\nu'_k}, t)$ in a number of operations linearly proportional the number of samples going from the right child to left child. As such, the exhaustive evaluation of all intermediate splits can be performed in linear time with respect to N_t, hence guaranteeing good performance as further shown in Chapter 5.

- In classification, evaluating either $\Delta i_H(s,t)$ or $\Delta i_G(s,t)$ boils down to the computation of $p(c|t)$, $p(c|t_L)$ and $p(c|t_R)$ for all classes $c \in \mathcal{Y}$. The necessary statistics for computing these probabilities are the numbers N_{ct}, N_{ct_L} and N_{ct_R} of samples of class c in t, t_L and t_R, along with the total numbers N_t, N_{t_L} and N_{t_R} of samples in t, t_L and t_R. N_{ct} and N_t can be computed once for all splits. The four other statistics can be computed iteratively, from $s_j^{\nu'_k}$ to $s_j^{\nu'_{k+1}}$:

$$N_{t_L}^{\nu'_{k+1}} = N_{t_L}^{\nu'_k} + |\{(\mathbf{x}, y) | (\mathbf{x}, y) \in \mathcal{L}_{t_L}^{\nu'_{k+1}} \cap \mathcal{L}_{t_R}^{\nu'_k}\}| \tag{3.24}$$

$$N_{ct_L}^{\nu'_{k+1}} = N_{ct_L}^{\nu'_k} + |\{(\mathbf{x}, y) | (\mathbf{x}, y) \in \mathcal{L}_{t_L}^{\nu'_{k+1}} \cap \mathcal{L}_{t_R}^{\nu'_k}, y = c\}| \tag{3.25}$$

$$N_{t_R}^{\nu'_{k+1}} = N_{t_R}^{\nu'_k} - |\{(\mathbf{x}, y) | (\mathbf{x}, y) \in \mathcal{L}_{t_L}^{\nu'_{k+1}} \cap \mathcal{L}_{t_R}^{\nu'_k}\}| \tag{3.26}$$

$$N_{ct_R}^{\nu'_{k+1}} = N_{ct_R}^{\nu'_k} - |\{(\mathbf{x}, y) | (\mathbf{x}, y) \in \mathcal{L}_{t_L}^{\nu'_{k+1}} \cap \mathcal{L}_{t_R}^{\nu'_k}, y = c\}| \tag{3.27}$$

where $\mathcal{L}_{t_L}^{\nu'_{k+1}} \cap \mathcal{L}_{t_R}^{\nu'_k}$ are the node samples going from the right child node to the left child node when switching from $s_j^{\nu'_k}$ to $s_j^{\nu'_{k+1}}$.

- In regression, $i_R(t)$ can be re-expressed as the difference between the mean of the squared output values and the square of the mean output value, that is

$$i_R(t) = \frac{\Sigma_{t^2}}{N_t} - (\frac{\Sigma_t}{N_t})^2 \tag{3.28}$$

where $\Sigma_t = \sum_{\mathbf{x}, y \in \mathcal{L}_t} y$ and $\Sigma_{t^2} = \sum_{\mathbf{x}, y \in \mathcal{L}_t} y^2$. In this form, the necessary statistics for computing $\Delta i_R(s,t)$ are Σ_t, Σ_{t^2}, Σ_{t_L}, $\Sigma_{t_L^2}$, Σ_{t_R} and $\Sigma_{t_R^2}$, along with the total numbers N_t, N_{t_L} and N_{t_R} of samples in t, t_L and t_R. As in classification, Σ_t, Σ_{t^2} and N_t can be computed once for all splits, while N_{t_L} and N_{t_R} can be computed from one split to another using Equations 3.24 and 3.26. The four other statistics can be computed iteratively, from $s_j^{\nu'_k}$ to $s_j^{\nu'_{k+1}}$:

$$\Sigma_{t_L}^{\nu'_{k+1}} = \Sigma_{t_L}^{\nu'_k} + \sum_{(\mathbf{x}, y) \in \mathcal{L}_{t_L}^{\nu'_{k+1}} \cap \mathcal{L}_{t_R}^{\nu'_k}} y \tag{3.29}$$

$$\Sigma_{t_L^2}^{\nu'_{k+1}} = \Sigma_{t_L^2}^{\nu'_k} + \sum_{(\mathbf{x}, y) \in \mathcal{L}_{t_L}^{\nu'_{k+1}} \cap \mathcal{L}_{t_R}^{\nu'_k}} y^2 \tag{3.30}$$

$$\Sigma_{t_R}^{\nu'_{k+1}} = \Sigma_{t_R}^{\nu'_k} - \sum_{(\mathbf{x}, y) \in \mathcal{L}_{t_L}^{\nu'_{k+1}} \cap \mathcal{L}_{t_R}^{\nu'_k}} y \tag{3.31}$$

calculado a partir de $\Delta i(s_j^{\nu'_k}, t)$ em um número de operações linearmente proporcionais ao número de amostras que vão do filho direito para o filho esquerdo. Como tal, a avaliação exaustiva de todas as divisões intermediárias pode ser realizada em tempo linear em relação a N_t, garantindo assim um bom desempenho, conforme mostrado no Capítulo 5.

- Na classificação, avaliar $\Delta i_H(s,t)$ ou $\Delta i_G(s,t)$ se resume ao cálculo de $p(c|t)$, $p(c|t_L)$ e $p(c|t_R)$ para todas as classes $c \in \mathcal{Y}$. As estatísticas necessárias para calcular essas probabilidades são os números N_{ct}, N_{ct_L} e N_{ct_R} de amostras de classec em t, t_L e t_R, juntamente com os números totais N_t, N_{t_L} e N_{t_R}de amostras em t, t_L e t_R. N_{ct} e N_t podem ser computados uma vez para todas as divisões. As outras quatro estatísticas podem ser calculadas iterativamente, de $s_j^{\nu'_k}$a $s_j^{\nu'_{k+1}}$:

$$N_{t_L}^{\nu'_{k+1}} = N_{t_L}^{\nu'_k} + |\{(\mathbf{x}, y) | (\mathbf{x}, y) \in \mathcal{L}_{t_L}^{\nu'_{k+1}} \cap \mathcal{L}_{t_R}^{\nu'_k}\}| \tag{3.24}$$

$$N_{ct_L}^{\nu'_{k+1}} = N_{ct_L}^{\nu'_k} + |\{(\mathbf{x}, y) | (\mathbf{x}, y) \in \mathcal{L}_{t_L}^{\nu'_{k+1}} \cap \mathcal{L}_{t_R}^{\nu'_k}, y = c\}| \tag{3.25}$$

$$N_{t_R}^{\nu'_{k+1}} = N_{t_R}^{\nu'_k} - |\{(\mathbf{x}, y) | (\mathbf{x}, y) \in \mathcal{L}_{t_L}^{\nu'_{k+1}} \cap \mathcal{L}_{t_R}^{\nu'_k}\}| \tag{3.26}$$

$$N_{ct_R}^{\nu'_{k+1}} = N_{ct_R}^{\nu'_k} - |\{(\mathbf{x}, y) | (\mathbf{x}, y) \in \mathcal{L}_{t_L}^{\nu'_{k+1}} \cap \mathcal{L}_{t_R}^{\nu'_k}, y = c\}| \tag{3.27}$$

onde $\mathcal{L}_{t_L}^{\nu'_{k+1}} \cap \mathcal{L}_{t_R}^{\nu'_k}$ são as amostras de nós que vão do nó filho direito para o nó filho esquerdo ao mudar de $s_j^{\nu'_k}$para $s_j^{\nu'_{k+1}}$.

- Na regressão, $i_R(t)$ pode ser reexpresso como a diferença entre a média dos valores quadrados da saída e o quadrado do valor médio da saída, ou seja

$$i_R(t) = \frac{\Sigma_{t^2}}{N_t} - (\frac{\Sigma_t}{N_t})^2 \tag{3.28}$$

onde $\Sigma_t = \sum_{\mathbf{x}, y \in \mathcal{L}_t} y$ e $\Sigma_{t^2} = \sum_{\mathbf{x}, y \in \mathcal{L}_t} y^2$. Neste formulário, as estatísticas necessárias para calcular $\Delta i_R(s,t)$ são Σ_t, Σ_{t^2}, Σ_{t_L}, $\Sigma_{t_L^2}$,Σ_{t_R} e $\Sigma_{t_R^2}$, juntamente com os números totais N_t, N_{t_L} e N_{t_R}de a mostras em t, t_L e t_R. Como na classificação, Σ_t, Σ_{t^2} e $N_t$podem ser calculados uma vez para todas as divisões, enquanto N_{t_L} e N_{t_R} podem ser calculados de uma divisão para outra usando as Equações 3.24 e 3.26. As outras quatro estatísticas podem ser calculadas iterativamente, de $s_j^{\nu'_k}$a $s_j^{\nu'_{k+1}}$:

$$\Sigma_{t_L}^{\nu'_{k+1}} = \Sigma_{t_L}^{\nu'_k} + \sum_{(\mathbf{x}, y) \in \mathcal{L}_{t_L}^{\nu'_{k+1}} \cap \mathcal{L}_{t_R}^{\nu'_k}} y \tag{3.29}$$

$$\Sigma_{t_L^2}^{\nu'_{k+1}} = \Sigma_{t_L^2}^{\nu'_k} + \sum_{(\mathbf{x}, y) \in \mathcal{L}_{t_L}^{\nu'_{k+1}} \cap \mathcal{L}_{t_R}^{\nu'_k}} y^2 \tag{3.30}$$

$$\Sigma_{t_R}^{\nu'_{k+1}} = \Sigma_{t_R}^{\nu'_k} - \sum_{(\mathbf{x}, y) \in \mathcal{L}_{t_L}^{\nu'_{k+1}} \cap \mathcal{L}_{t_R}^{\nu'_k}} y \tag{3.31}$$

$$\Sigma^{\nu'_{k+1}}_{t^2_R} = \Sigma^{\nu'_k}_{t^2_R} - \sum_{(x,y)\in\mathcal{L}^{\nu'_{k+1}}_{t_L}\cap\mathcal{L}^{\nu'_k}_{t_R}} y^2 \tag{3.32}$$

Starting from the initial partition $\nu'_0 = -\infty$, that is where t_L is empty and t_R corresponds to t, and using the above update equations to switch from ν'_k to ν'_{k+1}, the search of best split s^*_j on X_j can finally be implemented as described in Algorithm 3.4 and further illustrated in Figure 3.9.

Algorithm 3.4. *Find the best split s^*_j on X_j that partitions $\mathcal{L}_t$.*

function FINDBESTSPLIT($\mathcal{L}_t$, X_j)
  $\Delta = 0$
  $k = 0$
  $\nu'_k = -\infty$
  Compute the necessary statistics for $i(t)$
  Initialize the statistics for t_L to 0
  Initialize the statistics for t_R to those of $i(t)$
  Sort the node samples $\mathcal{L}_t$ such that $x_{1,j} \leqslant x_{2,j} \leqslant \cdots \leqslant x_{N_t,j}$
  $i = 1$
  while $i \leqslant N_t$ **do**
    while $i+1 \leqslant N_t$ and $x_{i+1,j} = x_{i,j}$ **do**
      $i = i+1$
    end while
    $i = i+1$
    if $i \leqslant N_t$ **then**
      $\nu'_{k+1} = \frac{x_{i,j}+x_{i-1,j}}{2}$
      Update the necessary statistics from ν'_k to ν'_{k+1}
      if $\Delta i(s_j^{\nu'_{k+1}}, t) > \Delta$ **then**
        $\Delta = \Delta i(s_j^{\nu'_{k+1}}, t)$
        $s^*_j = s_j^{\nu'_{k+1}}$
      end if
      $k = k+1$
    end if
  end while
  return s^*_j
end function

As we will see later in Chapter 5, the algorithm complexity is upper bounded by the complexity of the sorting operation. In the context of randomized decision trees however, where finding the very best split s^*_j is not as crucial, we will show that approximation techniques based on sub-sampling of the node samples or on the discretization of the variable X_j can help reduce the computational complexity of procedure without significant impact on accuracy.

$$\Sigma^{\nu'_{k+1}}_{t^2_R} = \Sigma^{\nu'_k}_{t^2_R} - \sum_{(x,y)\in\mathcal{L}^{\nu'_{k+1}}_{t_L}\cap\mathcal{L}^{\nu'_k}_{t_R}} y^2 \tag{3.32}$$

Começando pela partição inicial $\nu'_0 = -\infty$, que é onde t_L está vazia e t_R corresponde a t, e usando as equações de atualização acima para mudar de ν'_k para ν'_{k+1}, a busca da melhor divisão s^*_j em X_j pode finalmente ser implementada conforme descrito no Algoritmo 3.4 e ilustrado com mais detalhes na Figura 3.9.

Algoritmo 3.4. Encontre a melhor divisão s^*_j em X_j que particiona $\mathcal{L}_t$.

1: função FindBestSplit($\mathcal{L}_t$, X_j) 2: $\Delta = 0$**3:** $k = 0$**4:** $\nu'_k = -\infty$**5: Calcule as estatísticas necessárias para** $i(t)$**6: Inicialize as estatísticas de** t_L **para 0 7: Inicialize as estatísticas de** t_R **para aquelas de** $i(t)$**8: Classifique as amostras de nós** $\mathcal{L}_t$ **de modo que** $x_{1,j} \leqslant x_{2,j} \leqslant \cdots \leqslant x_{N_t,j}$**9:** $i = 1$**10: while** $i \leqslant N_t$ **faça 11: while** $i+1 \leqslant N_t$ **e** $x_{i+1,j} = x_{i,j}$ **faça 12:** $i = i+1$**13: termine while 14:** $i = i+1$**15: se** $i \leqslant N_t$ **então 16:** $\nu'_{k+1} = \frac{x_{i,j}+x_{i-1,j}}{2}$ **17: atualize as estatísticas de** t_L **e** t_R **para** ν'_k **e** ν'_{k+1} **18: if** $\Delta i(s_j^{\nu'_{k+1}}, t) > \Delta$ **então 19:** $\Delta = \Delta i(s_j^{\nu'_{k+1}}, t)$ **::** $s^*_j = s_j^{\nu'_{k+1}}$**21: end if 22:** $k = k+1$**23: end if 24: end while 25: return** s^*_j**26: end function**

Como veremos mais adiante no Capítulo 5, a complexidade do algoritmo é limitada superiormente pela complexidade da operação de classificação. No entanto, no contexto de árvores de decisão aleatórias, onde encontrar a melhor divisão s^*_j não é tão crucial, mostraremos que técnicas de aproximação baseadas na subamostragem das amostras dos nós ou na discretização da variável X_j podem ajudar a reduzir a complexidade computacional do procedimento sem impacto significativo na precisão.

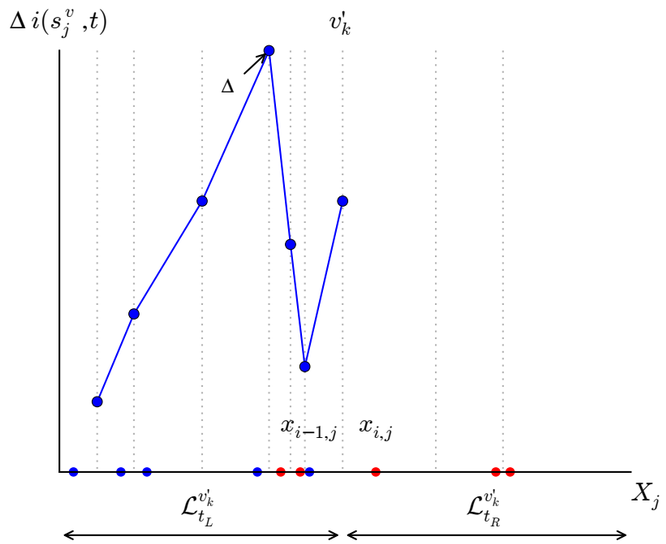


Figure 3.9: Loop invariant of Algorithm 3.4. At the end of each iteration, $\Delta i(s_j^{v'_k}, t)$ has been computed from the statistics of the previous split at v'_{k-1} and compared to the best reduction in impurity Δ found so far within the splits at $v'_0 = -\infty$ to v'_{k-1}.

3.6.3.2 *On a categorical variable*

Let X_j be a categorical variable and let $\mathcal{Q}(X_j)$ be the set of all binary partitions of $\mathcal{X}_j = \{b_1, \ldots, b_L\}$, as defined in Equation 3.15. The number of non-empty binary partitions in this set is $2^{L-1} - 1$, which may quickly become highly prohibitive to explore exhaustively when X_j counts a high number L of categories.

Fortunately, in binary classification, this exponential complexity can be reduced from looking at $2^{L-1} - 1$ partitions to $L - 1$ partitions, thanks a theoretical result due to Fisher [1958] and Breiman et al. [1984]:

Theorem 3.4. *Let reorder the categories of X_j such that*

$$p(c_1|t, X_j = b_{l_1}) \leqslant p(c_1|t, X_j = b_{l_2}) \leqslant \cdots \leqslant p(c_1|t, X_j = b_{l_L}).$$

If the impurity function $i(t)$ satisfies the requirements of Theorem 3.3, then one of the $L - 1$ subsets $\mathcal{B} = \{b_{l_1}, \ldots, b_{l_h}\}, h = 1, \ldots, L - 1$ defines a binary partition of the node samples into

$$\mathcal{L}_{t_L}^{\mathcal{B}} = \{(\mathbf{x}, y) | (\mathbf{x}, y) \in \mathcal{L}_t, x_j \in \mathcal{B}\}$$
$$\mathcal{L}_{t_R}^{\mathcal{B}} = \{(\mathbf{x}, y) | (\mathbf{x}, y) \in \mathcal{L}_t, x_j \in \overline{\mathcal{B}}\}$$

which maximizes $\Delta i(s, t)$, for $s \in \mathcal{Q}(X_j)$.

Intuitively, this result indicates that all those categories b_l leading to high probabilities of being of class c_1 should be put together into

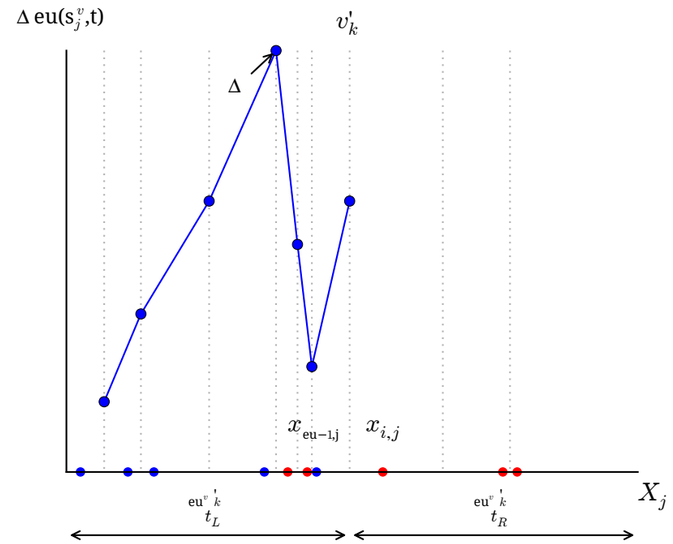


Figura 3.9: Loop invariante do Algoritmo 3.4. No final de cada iteração, $\Delta i(s_j^{v'_k}, t)$ foi calculado a partir das estatísticas da divisão anterior em v'_{k-1} e comparado com a melhor redução na impureza Δencontrada até agora nas divisões de $v'_0 = -\infty$ a v'_{k-1}.

3.6.3.2 Em uma variável categórica

Seja X_j uma variável categórica e seja $\mathcal{Q}(X_j)$ o conjunto de todas as partições binárias de $\mathcal{X}_j = \{b_1, \ldots, b_L\}$, conforme definido na Equação 3.15. O número de partições binárias não vazias neste conjunto é $2^{L-1} - 1$, o que pode rapidamente se tornar altamente proibitivo para explorar exaustivamente quando $X_j$conta um grande número Lde categorias.

Felizmente, na classificação binária, esta complexidade exponencial pode ser reduzida olhando para partições $2^{L-1} - 1$ para partições $L - 1$, graças a um resultado teórico devido a Fisher [1958] e Breiman et al. [1984]:

Teorema 3.4. Vamos reordenar as categorias de X_j de modo que

$$p(c_1|t, X_j = b_{l_1}) \leqslant p(c_1|t, X_j = b_{l_2}) \leqslant \cdots \leqslant p(c_1|t, X_j = b_{l_L}).$$

Se a função de impureza $i(t)$ satisfizer os requisitos do Teorema 3.3, então um dos $L - 1$ subconjuntos $\mathcal{B} = \{b_{l_1}, \ldots, b_{l_h}\}, h = 1, \ldots, L - 1$ define uma partição binária das amostras de nós em

$$\mathcal{L}_{t_L}^{\mathcal{B}} = \{(\mathbf{x}, y) | (\mathbf{x}, y) \in \mathcal{L}_t, x_j \in \mathcal{B}\}$$
$$\mathcal{L}_{t_R}^{\mathcal{B}} = \{(\mathbf{x}, y) | (\mathbf{x}, y) \in \mathcal{L}_t, x_j \in \overline{\mathcal{B}}\}$$

que maximiza $\Delta i(s, t)$, para $s \in \mathcal{Q}(X_j)$.

Intuitivamente, este resultado indica que todas as categorias b_l que levam a altas probabilidades de pertencer à classe c_1 devem ser reunidas em

one node and the categories leading to lower probabilities into the other. In practice, this also means that the search of the best binary split can be performed as if X_j was an ordered variable, by virtually replacing categorical values b_l with $p(c_1|t, X_j = b_l)$ and using Algorithm 3.4 to find the best binary split on these ordered values.

Unfortunately, Theorem 3.4 does not extend to multi-class classification (i.e., when $J > 2$). In this case, an exhaustive exploration of all $2^{L-1} - 1$ splits must be performed in order to find the best split. When the number L of categories is small, this can usually be done within reasonable time. However, as L gets larger, this quickly becomes infeasible in practice. As an approximation, the usual strategy [Liaw and Wiener, 2002] consists in exhaustively evaluating all splits when $L \leqslant 10$ and then switch to random sampling of the subsets $\mathcal{B} \subset \{b_1, \dots, b_L\}$ when $L > 10$. In that case, 2^8 random subsets $\mathcal{B}$ are typically drawn and evaluated.

In regression, given the equivalence of $i_R(t)$ with the Gini index, conditions of Theorem 3.4 are met and the same algorithmic trick can be applied as in binary classification to compute the optimal split. In this case, X_j is transformed into an ordered variable by virtually replacing categorical values b_l with the mean output value at b_l, i.e., $\frac{1}{N_{lt}} \sum_{\mathbf{x}, y \in \mathcal{L}_t, x_j = b_l} y$, on which Algorithm 3.4 is then applied to find the best binary split on these ordered values.

3.7 MULTI-OUTPUT DECISION TREES

Up to now, we have assumed that the response was modeled as a single target variable Y. In some contexts however, the goal may be to predict several output values $\mathbf{y} = (y_1, \dots, y_q) \in \mathcal{Y}_1 \times \cdots \times \mathcal{Y}_q$ for the same input $\mathbf{x}$. A famous example of such multi-objective task is multi-label classification (see [Tsoumakas and Katakis, 2007]), in which a sample $\mathbf{x}$ can be assigned to multiple binary labels Y_k (for $k = 1, \dots, q$). Another example is image annotation, in which the goal is to annotate (i.e., classify) every pixel of an image in order to identify regions or objects (see [Zhang et al., 2012]).

The most straightforward strategy to solve a multi-objective task is to build an independent and separate model for each of the q output variables Y_k. Distinctively, decision trees offer an alternative strategy which consists in learning a single *multi-output* model, capable of predicting all output variables at once. Algorithmically, *multi-output decision trees* closely relate to predictive clustering trees [Blockeel et al., 2000], output kernel trees [Geurts et al., 2006b] or multi-objective decision trees [Kocev et al., 2007], and can easily be implemented on the basis of the induction procedure developed in this chapter, provided the following two changes:

- Leaves are labeled with output vectors $\mathbf{y}_t = (y_{t,1}, \dots, y_{t,q})$, where each $y_{t,k}$ (for $k = 1, \dots, q$) is computed using assign-

um nó e as categorias que levam a probabilidades mais baixas no outro. Na prática, isso também significa que a busca pela melhor divisão binária pode ser realizada como se X_j fosse uma variável ordenada, substituindo virtualmente os valores categóricos b_l por $p(c_1|t, X_j = b_l)$ e usando o Algoritmo 3.4 para encontrar a melhor divisão binária nesses valores ordenados.

Infelizmente, o Teorema 3.4 não se estende à classificação multiclasse (ou seja, quando $J > 2$). Neste caso, uma exploração exaustiva de todas as divisões $2^{L-1} - 1$ deve ser realizada para encontrar a melhor divisão. Quando o número L de categorias é pequeno, isso geralmente pode ser feito dentro de um prazo razoável. No entanto, à medida que L fica maior, isso rapidamente se torna inviável na prática. Como aproximação, a estratégia usual [Liaw e Wiener, 2002] consiste em avaliar exaustivamente todas as divisões quando $L \leqslant 10$ e depois mudar para amostragem aleatória dos subconjuntos $\mathcal{B} \subset \{b_1, \dots, b_L\}$ quando $L > 10$. Nesse caso, 2^8 subconjuntos aleatórios $\mathcal{B}$ normalmente são desenhados e avaliados.

Na regressão, dada a equivalência de $i_R(t)$ com o índice de Gini, as condições do Teorema 3.4 são atendidas e o mesmo truque algorítmico pode ser aplicado como na classificação binária para calcular a divisão ótima. Neste caso, X_j é transformado em uma variável ordenada substituindo virtualmente os valores categóricos b_l pelo valor médio de saída em b_l, ou seja, $\frac{1}{N_{lt}} \sum_{\mathbf{x}, y \in \mathcal{L}_t, x_j = b_l} y$, no qual o Algoritmo 3.4 é então aplicado para encontrar a melhor divisão binária nesses valores ordenados.

3.7 árvores de decisão com múltiplas saídas

Até agora, assumimos que a resposta foi modelada como uma única variável alvo Y. Em alguns contextos, entretanto, o objetivo pode ser prever vários valores de saída $\mathbf{y} = (y_1, \dots, y_q) \in \mathcal{Y}_1 \times \cdots \times \mathcal{Y}_q$ para a mesma entrada $\mathbf{x}$. Um exemplo famoso de tal tarefa multiobjetivo é a classificação multirrótulos (ver [Tsoumakas e Katakis, 2007]), em que uma amostra $\mathbf{x}$ pode ser atribuída a vários rótulos binários Y_k (para $k = 1, \dots, q$). Outro exemplo é a anotação de imagens, em que o objetivo é anotar (ou seja, classificar) cada pixel de uma imagem para identificar regiões ou objetos (ver [Zhang et al., 2012]).

A estratégia mais direta para resolver uma tarefa multiobjetivo é construir um modelo independente e separado para cada uma das q variáveis de saída Y_k. Distintamente, as árvores de decisão oferecem uma estratégia alternativa que consiste em aprender um único modelo multi-output, capaz de prever todas as variáveis de output de uma só vez. Algoritmicamente, as árvores de decisão de múltiplas saídas estão intimamente relacionadas às árvores de agrupamento preditivo [Blockeel et al., 2000], árvores de kernel de saída [Geurts et al., 2006b] ou árvores de decisão multiobjetivo [Kocev et al., 2007], e podem ser facilmente implementadas com base no procedimento de indução desenvolvido neste capítulo, desde que o seguinte duas mudanças:

- As folhas são rotuladas com vetores de saída $\mathbf{y}_t = (y_{t,1}, \dots, y_{t,q})$, onde cada $y_{t,k}$ (para $k = 1, \dots, q$) é calculado usando atribuição-

ment rule 3.7 in classification (resp. assignment rule 3.10 in regression), as previously done in single output decision trees.

- The impurity decrease of a split is computed as the average impurity decrease over the q output variables. Accordingly, splits are optimized with respect to all output variables, thereby exploiting correlations between $Y_1, \ldots, Y_q$ whenever possible.

The main advantage of multi-output decision trees is that dependencies between output variables can be taken into account, whereas building q individual models cannot exploit such correlations, which may affect the overall accuracy of the model. A second advantage is that building a single model is often less computationally expensive than building q different models, both from a time and space complexity point of view.

regra de atribuição 3.7 na classificação (resp. regra de atribuição 3.10 na regressão), como feito anteriormente em árvores de decisão de saída única.

- A diminuição da impureza de uma divisão é calculada como a di minuição média da impureza sobre as variáveis de saída q . Conseqüentemente, as divisões são otimizadas em relação a todas as variáveis de saída, explorando assim as correlações entre $Y_1, \ldots, Y_q$ sempre que possível.

A principal vantagem das árvores de decisão com múltiplos resultados é que as dependências entre as variáveis de saída podem ser levadas em conta, enquanto a construção de modelos individuais não pode explorar tais correlações, o que pode afetar a precisão geral do modelo. Uma segunda vantagem é que construir um único modelo é muitas vezes menos dispendioso em termos computacionais do que construir q modelos diferentes, tanto do ponto de vista da complexidade do tempo como do espaço.

4 RANDOM FORESTS

Outline

In this chapter, we present the well-known family of *random forests* methods. In Section 4.1, we first describe the bias-variance decomposition of the prediction error and then present, in Section 4.2, how aggregating randomized models through ensembles reduces the prediction error by decreasing the variance term in this decomposition. In Section 4.3, we revisit random forests and its variants and study how randomness introduced into the decision trees reduces prediction errors by decorrelating the decision trees in the ensemble. Properties and features of random forests are then outlined in Section 4.4 while their consistency is finally explored in Section 4.5.

4.1 BIAS-VARIANCE DECOMPOSITION

In section 2.2, we defined the generalization error of a model $\varphi_{\mathcal{L}}$ as its expected prediction error according to some loss function L

$$Err(\varphi_{\mathcal{L}}) = \mathbb{E}_{X,Y}\{L(Y, \varphi_{\mathcal{L}}(X))\}. \tag{4.1}$$

Similarly, the expected prediction error of $\varphi_{\mathcal{L}}$ at $X = \mathbf{x}$ can be expressed as

$$Err(\varphi_{\mathcal{L}}(\mathbf{x})) = \mathbb{E}_{Y|X=\mathbf{x}}\{L(Y, \varphi_{\mathcal{L}}(\mathbf{x}))\}. \tag{4.2}$$

In regression, for the squared error loss, this latter form of the expected prediction error additively decomposes into bias and variance terms which together constitute a very useful framework for diagnosing the prediction error of a model. In classification, for the zero-one loss, a similar decomposition is more difficult to obtain. Yet, the concepts of bias and variance can be transposed in several ways to classification, thereby providing comparable frameworks for studying the prediction error of classifiers.

4.1.1 *Regression*

In regression, assuming that L is the squared error loss, the expected prediction error of a model $\varphi_{\mathcal{L}}$ at a given point $X = \mathbf{x}$ can be rewritten with respect to the Bayes model φ_B:

$$Err(\varphi_{\mathcal{L}}(\mathbf{x}))$$

4 RANDOMFORESTS

Contorno

Neste capítulo, apresentamos a conhecida família de métodos de florestas aleatórias. Na Seção 4.1, descrevemos primeiro a decomposição viés-variância do erro de predição e depois apresentamos, na Seção 4.2, como a agregação de modelos aleatórios através de conjuntos reduz o erro de predição ao diminuir o termo de variância nesta decomposição. Na Seção 4.3, revisitamos florestas aleatórias e suas variantes e estudamos como a aleatoriedade introduzida nas árvores de decisão reduz erros de previsão ao descorrelacionar as árvores de decisão no conjunto. As propriedades e características das florestas aleatórias são descritas na Seção 4.4, enquanto sua consistência é finalmente explorada na Seção 4.5.

4.1 decomposição de polarização-variância

Na seção 2.2, definimos o erro de generalização de um modelo $\varphi_{\mathcal{L}}$ como seu erro de previsão esperado de acordo com alguma função de perda L

$$Err(\varphi_{\mathcal{L}}) = \mathbb{E}_{X,Y}\{L(Y, \varphi_{\mathcal{L}}(X))\}. \tag{4.1}$$

Da mesma forma, o erro de previsão esperado de $\varphi_{\mathcal{L}}$ em $X = x$ pode ser expresso como

$$Err(\varphi_{\mathcal{L}}(\mathbf{x})) = \mathbb{E}_{Y|X=\mathbf{x}}\{L(Y, \varphi_{\mathcal{L}}(\mathbf{x}))\}. \tag{4.2}$$

Na regressão, para a perda de erro quadrático, esta última forma do erro de predição esperado se decompõe aditivamente em termos de viés e variância que juntos constituem uma estrutura muito útil para diagnosticar o erro de predição de um modelo. Na classificação, para a perda zero-um, uma decomposição semelhante é mais difícil de obter. No entanto, os conceitos de viés e variância podem ser transpostos de diversas maneiras para a classificação, fornecendo assim estruturas comparáveis para estudar o erro de predição dos classificadores.

4.1.1 Regressão

Na regressão, assumindo que L é a perda de erro quadrática, o erro de previsão esperado de um modelo $\varphi_{\mathcal{L}}$ em um determinado ponto $X = x$ pode ser reescrito em relação ao modelo de Bayes φ_B:

$$Err(\varphi_{\mathcal{L}}(\mathbf{x}))$$

$$
\begin{aligned}
&= \mathbb{E}_{Y|X=\mathbf{x}}\{(Y - \varphi_{\mathcal{L}}(\mathbf{x}))^2\} \\
&= \mathbb{E}_{Y|X=\mathbf{x}}\{(Y - \varphi_B(\mathbf{x}) + \varphi_B(\mathbf{x}) - \varphi_{\mathcal{L}}(\mathbf{x}))^2\} \\
&= \mathbb{E}_{Y|X=\mathbf{x}}\{(Y - \varphi_B(\mathbf{x}))^2\} + \mathbb{E}_{Y|X=\mathbf{x}}\{(\varphi_B(\mathbf{x}) - \varphi_{\mathcal{L}}(\mathbf{x}))^2\} \\
&\hookrightarrow + \mathbb{E}_{Y|X=\mathbf{x}}\{2(Y - \varphi_B(\mathbf{x}))(\varphi_B(\mathbf{x}) - \varphi_{\mathcal{L}}(\mathbf{x}))\} \\
&= \mathbb{E}_{Y|X=\mathbf{x}}\{(Y - \varphi_B(\mathbf{x}))^2\} + \mathbb{E}_{Y|X=\mathbf{x}}\{(\varphi_B(\mathbf{x}) - \varphi_{\mathcal{L}}(\mathbf{x}))^2\} \\
&= \mathrm{Err}(\varphi_B(\mathbf{x})) + (\varphi_B(\mathbf{x}) - \varphi_{\mathcal{L}}(\mathbf{x}))^2 \qquad (4.3)
\end{aligned}
$$

since $\mathbb{E}_{Y|X=\mathbf{x}}\{Y - \varphi_B(\mathbf{x})\} = \mathbb{E}_{Y|X=\mathbf{x}}\{Y\} - \varphi_B(\mathbf{x}) = 0$ by definition of the Bayes model in regression. In this form, the first term in the last expression of Equation 4.3 corresponds to the (irreducible) residual error at $X = \mathbf{x}$ while the second term represents the discrepancy of $\varphi_{\mathcal{L}}$ from the Bayes model. The farther from the Bayes model, the more sub-optimal the model and the larger the error.

If we further assume that the learning set $\mathcal{L}$ is itself a random variable (sampled from the population Ω) and that the learning algorithm is deterministic, then the expected discrepancy over $\mathcal{L}$ with the Bayes model can further be re-expressed in terms of the average prediction $\mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\}$ over the models learned from all possible learning sets of size N:

$$
\begin{aligned}
&\mathbb{E}_{\mathcal{L}}\{(\varphi_B(\mathbf{x}) - \varphi_{\mathcal{L}}(\mathbf{x}))^2\} \\
&= \mathbb{E}_{\mathcal{L}}\{(\varphi_B(\mathbf{x}) - \mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\} + \mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\} - \varphi_{\mathcal{L}}(\mathbf{x}))^2\} \\
&= \mathbb{E}_{\mathcal{L}}\{(\varphi_B(\mathbf{x}) - \mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\})^2\} + \mathbb{E}_{\mathcal{L}}\{(\mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\} - \varphi_{\mathcal{L}}(\mathbf{x}))^2\}\} \\
&\hookrightarrow + \mathbb{E}_{\mathcal{L}}\{2(\varphi_B(\mathbf{x}) - \mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\})(\mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\} - \varphi_{\mathcal{L}}(\mathbf{x}))\} \\
&= \mathbb{E}_{\mathcal{L}}\{(\varphi_B(\mathbf{x}) - \mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\})^2\} + \mathbb{E}_{\mathcal{L}}\{(\mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\} - \varphi_{\mathcal{L}}(\mathbf{x}))^2\}\} \\
&= (\varphi_B(\mathbf{x}) - \mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\})^2 + \mathbb{E}_{\mathcal{L}}\{(\mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\} - \varphi_{\mathcal{L}}(\mathbf{x}))^2\} \qquad (4.4)
\end{aligned}
$$

since $\mathbb{E}_{\mathcal{L}}\{\mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\} - \varphi_{\mathcal{L}}(\mathbf{x})\} = \mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\} - \mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\} = 0$. In summary, the expected generalization error additively decomposes as formulated in Theorem 4.1.

Theorem 4.1. *For the squared error loss, the bias-variance decomposition of the expected generalization error* $\mathbb{E}_{\mathcal{L}}\{\mathrm{Err}(\varphi_{\mathcal{L}}(\mathbf{x}))\}$ *at* $X = \mathbf{x}$ *is*

$$
\mathbb{E}_{\mathcal{L}}\{\mathrm{Err}(\varphi_{\mathcal{L}}(\mathbf{x}))\} = \mathit{noise}(\mathbf{x}) + \mathit{bias}^2(\mathbf{x}) + \mathit{var}(\mathbf{x}), \qquad (4.5)
$$

where

$$
\begin{aligned}
\mathit{noise}(\mathbf{x}) &= \mathrm{Err}(\varphi_B(\mathbf{x})), \\
\mathit{bias}^2(\mathbf{x}) &= (\varphi_B(\mathbf{x}) - \mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\})^2, \\
\mathit{var}(\mathbf{x}) &= \mathbb{E}_{\mathcal{L}}\{(\mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\} - \varphi_{\mathcal{L}}(\mathbf{x}))^2\}.
\end{aligned}
$$

This bias-variance decomposition of the generalization error is due to Geman et al. [1992] and was first proposed in the context of neural networks. The first term, noise($\mathbf{x}$), is the residual error. It is entirely independent of both the learning algorithm and the learning set and

$$
\begin{aligned}
&= \mathbb{E}_{Y|X=\mathbf{x}}\{(Y - \varphi_{\mathcal{L}}(\mathbf{x}))^2\} \\
&= \mathbb{E}_{Y|X=\mathbf{x}}\{(Y - \varphi_B(\mathbf{x}) + \varphi_B(\mathbf{x}) - \varphi_{\mathcal{L}}(\mathbf{x}))^2\} \\
&= \mathbb{E}_{Y|X=\mathbf{x}}\{(Y - \varphi_B(\mathbf{x}))^2\} + \mathbb{E}_{Y|X=\mathbf{x}}\{(\varphi_B(\mathbf{x}) - \varphi_{\mathcal{L}}(\mathbf{x}))^2\} \\
&\hookrightarrow + \mathbb{E}_{Y|X=\mathbf{x}}\{2(Y - \varphi_B(\mathbf{x}))(\varphi_B(\mathbf{x}) - \varphi_{\mathcal{L}}(\mathbf{x}))\} \\
&= \mathbb{E}_{Y|X=\mathbf{x}}\{(Y - \varphi_B(\mathbf{x}))^2\} + \mathbb{E}_{Y|X=\mathbf{x}}\{(\varphi_B(\mathbf{x}) - \varphi_{\mathcal{L}}(\mathbf{x}))^2\} \\
&= \mathrm{Err}(\varphi_B(\mathbf{x})) + (\varphi_B(\mathbf{x}) - \varphi_{\mathcal{L}}(\mathbf{x}))^2 \qquad (4.3)
\end{aligned}
$$

desde $\mathbb{E}_{Y|X=\mathbf{x}}\{Y - \varphi_B(\mathbf{x})\} = \mathbb{E}_{Y|X=\mathbf{x}}\{Y\} - \varphi_B(\mathbf{x}) = 0$ por definição do modelo Bayes em regressão. Nesta forma, o primeiro termo na última expressão da Equação 4.3 corresponde ao erro residual (irredutível) em $X = \mathbf{x}$ enquanto o segundo termo representa a discrepância de $\varphi_{\mathcal{L}}$do modelo de Bayes. Quanto mais distante do modelo de Bayes, mais abaixo do ideal é o modelo e maior é o erro.

Se assumirmos ainda que o conjunto de aprendizagem $\mathcal{L}$ é ele próprio uma variável aleatória (amostrada da população Ω) e que o algoritmo de aprendizagem é determinístico, então a discrepância esperada em relação a $\mathcal{L}$ com o modelo de Bayes pode ainda ser reexpressa em termos da previsão média $\mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\}$ sobre os modelos aprendidos de todos os conjuntos de aprendizagem possíveis de tamanho N:

$$
\begin{aligned}
&\mathbb{E}_{\mathcal{L}}\{(\varphi_B(\mathbf{x}) - \varphi_{\mathcal{L}}(\mathbf{x}))^2\} \\
&= \mathbb{E}_{\mathcal{L}}\{(\varphi_B(\mathbf{x}) - \mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\} + \mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\} - \varphi_{\mathcal{L}}(\mathbf{x}))^2\} \\
&= \mathbb{E}_{\mathcal{L}}\{(\varphi_B(\mathbf{x}) - \mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\})^2\} + \mathbb{E}_{\mathcal{L}}\{(\mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\} - \varphi_{\mathcal{L}}(\mathbf{x}))^2\}\} \\
&\hookrightarrow + \mathbb{E}_{\mathcal{L}}\{2(\varphi_B(\mathbf{x}) - \mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\})(\mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\} - \varphi_{\mathcal{L}}(\mathbf{x}))\} \\
&= \mathbb{E}_{\mathcal{L}}\{(\varphi_B(\mathbf{x}) - \mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\})^2\} + \mathbb{E}_{\mathcal{L}}\{(\mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\} - \varphi_{\mathcal{L}}(\mathbf{x}))^2\}\} \\
&= (\varphi_B(\mathbf{x}) - \mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\})^2 + \mathbb{E}_{\mathcal{L}}\{(\mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\} - \varphi_{\mathcal{L}}(\mathbf{x}))^2\} \qquad (4.4)
\end{aligned}
$$

desde $\mathbb{E}_{\mathcal{L}}\{\mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\} - \varphi_{\mathcal{L}}(\mathbf{x})\} = \mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\} - \mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\} = 0$. Em resumo, o erro de generalização esperado se decompõe aditivamente conforme formulado no Teorema 4.1.

Teorema 4.1. Para a perda de erro quadrático, a decomposição de polarização-variância do erro de generalização esperado $\mathbb{E}_{\mathcal{L}}\{\mathrm{Err}(\varphi_{\mathcal{L}}(\mathbf{x}))\}$ *em* $X = \mathbf{x}$ *é*

$$
\mathbb{E}_{\mathcal{L}}\{\mathrm{Err}(\varphi_{\mathcal{L}}(\mathbf{x}))\} = \mathit{noise}(\mathbf{x}) + \mathit{bias}^2(\mathbf{x}) + \mathit{var}(\mathbf{x}), \qquad (4.5)
$$

onde

$$
\begin{aligned}
\mathit{noise}(\mathbf{x}) &= \mathrm{Err}(\varphi_B(\mathbf{x})), \\
\mathit{bias}^2(\mathbf{x}) &= (\varphi_B(\mathbf{x}) - \mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\})^2, \\
\mathit{var}(\mathbf{x}) &= \mathbb{E}_{\mathcal{L}}\{(\mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\} - \varphi_{\mathcal{L}}(\mathbf{x}))^2\}.
\end{aligned}
$$

Esta decomposição viés-variância do erro de generalização é devida a Geman et al. [1992] e foi proposto pela primeira vez no contexto de redes neurais. O primeiro termo, ruído($\mathbf{x}$), é o erro residual. É totalmente independente do algoritmo de aprendizagem e do conjunto de aprendizagem e

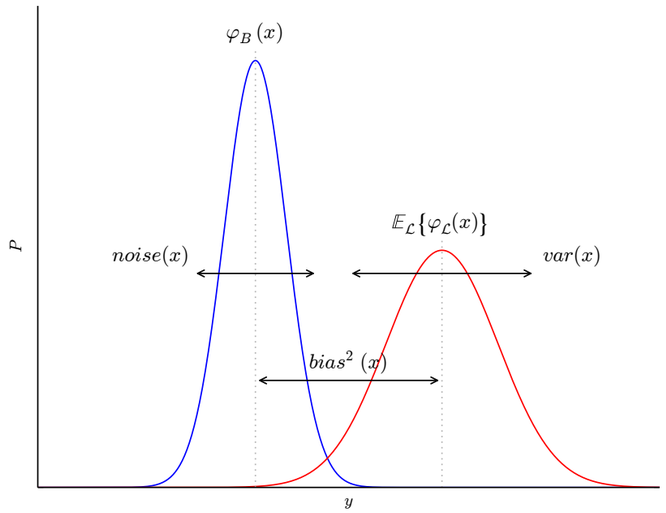


Figure 4.1: Residual error, bias and variance at $X = \mathbf{x}$. (Figure inspired from [Geurts, 2002].)

provides for any model a theoretical lower bound on its generalization error. The second term, $\text{bias}^2(\mathbf{x})$, measures the discrepancy between the average prediction and the prediction of the Bayes model. Finally, the third term, $\text{var}(\mathbf{x})$, measures the variability of the predictions at $X = \mathbf{x}$ over the models learned from all possible learning sets. All three terms are illustrated in Figure 4.1 for a toy and artificial regression problem. Both $\text{noise}(\mathbf{x})$ and $\text{var}(\mathbf{x})$ measures the spread of the two densities while $\text{bias}^2(\mathbf{x})$ is the distance between their means.

As a typical example, the bias-variance decomposition framework can be used as a tool for diagnosing underfitting and overfitting (as previously introduced in Section 2.3). The upper plots in Figure 4.2 illustrate in light red predictions $\varphi_{\mathcal{L}}(\mathbf{x})$ for polynomials of degree 1, 5 and 15 learned over random learning sets $\mathcal{L}$ sampled from a noisy cosine function. Predictions $\mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\}$ of the average model are represented by the thick red lines. Predictions for the model learned over the learning set, represented by the blue dots, are represented in gray. Predictions of the Bayes model are shown by blue lines and coincide with the unnoised cosine function that defines the regression problem. The lower plots in the figure illustrate the bias-variance decomposition of the expected generalization error of the polynomials.

Clearly, polynomials of degree 1 (left) suffer from underfitting. In terms of bias and variance, this translates into low variance but high bias as shown in the lower left plot of Figure 4.2. Indeed, due to the low degree of the polynomials (i.e., due to the low model complexity), the resulting models are almost all identical and the variability of the predictions from one model to another is therefore quite low.

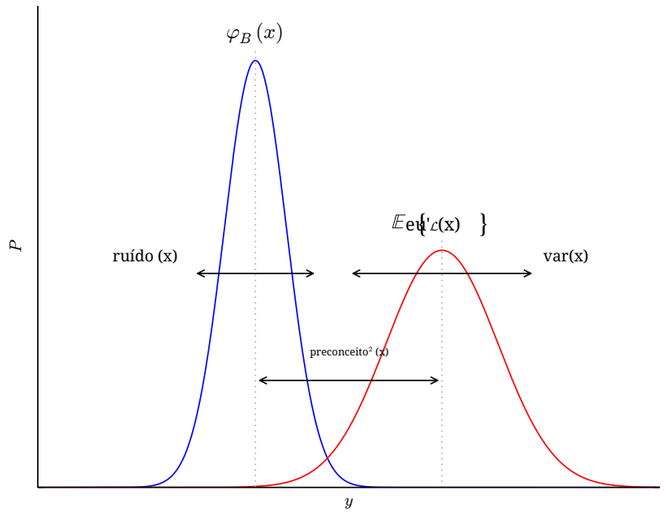


Figura 4.1: Erro residual, viés e variância em $X = \mathbf{x}$. (Figura inspirada em[Geurts, 2002].)

fornece para qualquer modelo um limite inferior teórico para seu erro de generalização. O segundo termo, $\text{viés}^2(\mathbf{x})$, mede a discrepância entre a previsão média e a previsão do modelo Bayesiano. Finalmente, o terceiro termo, $\text{var}(\mathbf{x})$, mede a variabilidade das previsões em $X = \mathbf{x}$ sobre os modelos aprendidos de todos os conjuntos de aprendizagem possíveis. Todos os três termos são ilustrados na Figura 4.1 para um problema de brinquedo e de regressão artificial. Tanto o $\text{ruído}(\mathbf{x})$ quanto o $\text{var}(\mathbf{x})$ mede a propagação das duas densidades, enquanto o $\text{viés}^2(\mathbf{x})$ é a distância entre suas médias.

Como exemplo típico, a estrutura de decomposição de viés-variância pode ser usada como uma ferramenta para diagnosticar subajuste e sobreajuste (conforme introduzido anteriormente na Seção 2.3). Os gráficos superiores na Figura 4.2 ilustram em vermelho claro as previsões $\varphi_{\mathcal{L}}(\mathbf{x})$ para polinômios de grau 1, 5 e 15 aprendidos em conjuntos de aprendizagem aleatórios $\mathcal{L}$ amostrados de uma função cosseno ruidosa. As previsões $\mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}(\mathbf{x})\}$ do modelo médio são representadas pelas linhas vermelhas grossas. As previsões para o modelo aprendido no conjunto de aprendizagem, representadas pelos pontos azuis, são representadas em cinza. As previsões do modelo de Bayes são mostradas por linhas azuis e coincidem com a função cosseno sem ruído que define o problema de regressão. Os gráficos inferiores da figura ilustram a decomposição viés-variância do erro de generalização esperado dos polinômios.

Claramente, polinômios de grau 1 (esquerda) sofrem de subajuste. Em termos de viés e variância, isso se traduz em baixa variância, mas alto viés, conforme mostrado no gráfico inferior esquerdo da Figura 4.2. Na verdade, devido ao baixo grau dos polinómios (ou seja, devido à baixa complexidade do modelo), os modelos resultantes são quase todos idênticos e a variabilidade das previsões de um modelo para outro é, portanto, bastante baixa.

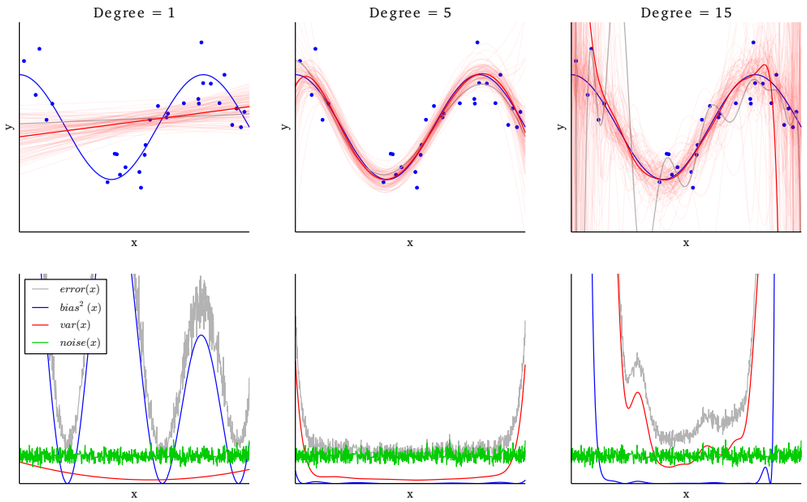


Figure 4.2: Bias-variance decomposition of the expected generalization error for polynomials of degree 1, 5 and 15.

Also, because of low complexity, none of them really fits the trend of the training points, even approximately, which implies that the average model is far from approximating the Bayes model. This results in high bias. On the other hand, polynomials of degree 15 (right) suffer from overfitting. In terms of bias and variance, the situation is the opposite. Predictions have low bias but high variance, as shown in the lower right plot of Figure 4.2. The variability of the predictions is large because the high degree of the polynomials (i.e., the high model complexity) captures noise in the learning set. Indeed, compare the gray line with the blue dots – they almost all intersect. Put otherwise, small changes in the learning set result in large changes in the obtained model and therefore in its predictions. By contrast, the average model is now quite close from the Bayes model, which results in low bias[1]. Finally, polynomials of degree 5 (middle) are neither too simple nor too complex. In terms of bias and variance, the trade-off is well-balanced between the two extreme situations. Bias and variance are neither too low nor too large.

4.1.2 *Classification*

In direct analogy with the bias-variance decomposition for the squared error loss, similar decompositions have been proposed in the literature for the expected generalization error based on the zero-one loss, i.e., for $\mathbb{E}_{\mathcal{L}}\{\mathbb{E}_{Y|X=\mathbf{x}}\{1(\varphi_{\mathcal{L}}(x) \neq Y)\}\} = P_{\mathcal{L},Y|X=\mathbf{x}}(\varphi_{\mathcal{L}}(\mathbf{x}) \neq Y)$. Most no-

1 Note however the Gibbs-like phenomenon resulting in both high variance and high bias at the boundaries of $\mathcal{X}$.

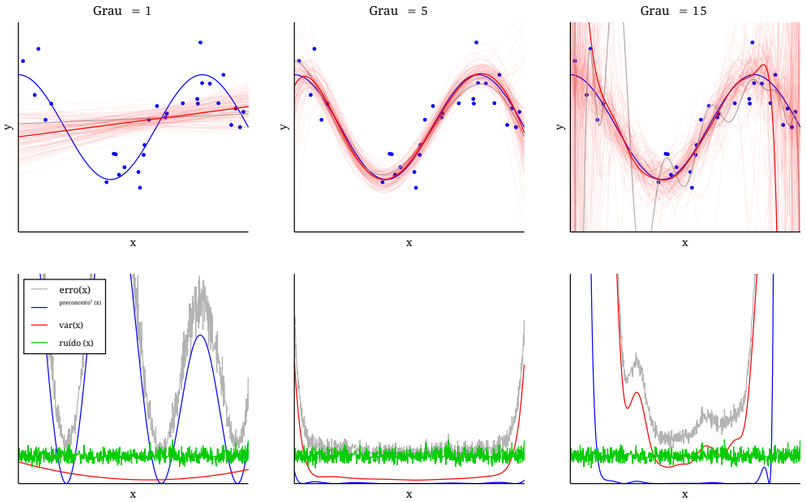


Figura 4.2: Decomposição viés-variância do erro de generalização esperado para polinômios de grau 1, 5 e 15.

Além disso, devido à baixa complexidade, nenhum deles realmente se ajusta à tendência dos pontos de treinamento, mesmo aproximadamente, o que implica que o modelo médio está longe de se aproximar do modelo de Bayes. Isso resulta em alto viés. Por outro lado, polinômios de grau 15 (direita) sofrem de overfitting. Em termos de preconceito e variância, a situação é oposta. As previsões têm baixo viés, mas alta variância, conforme mostrado no gráfico inferior direito da Figura 4.2. A variabilidade das previsões é grande porque o alto grau dos polinômios (ou seja, a alta complexidade do modelo) captura ruído no conjunto de aprendizagem. Na verdade, compare a linha cinza com os pontos azuis – quase todos eles se cruzam. Dito de outra forma, pequenas mudanças no conjunto de aprendizagem resultam em grandes mudanças no modelo obtido e, portanto, nas suas previsões. Por outro lado, o modelo médio está agora bastante próximo do modelo Bayes, o que resulta num viés baixo[1]. Finalmente, os polinômios de grau 5 (médio) não são nem muito simples nem muito complexos. Em termos de enviesamento e variância, o trade-off é bem equilibrado entre as duas situações extremas. O viés e a variância não são nem muito baixos nem muito grandes.

4.1.2 Classificação

Em analogia direta com a decomposição viés-variância para a perda de erro quadrático, decomposições semelhantes foram propostas na literat ura para o erro de generalização esperado com base na perda zero-um, ou seja, para $\mathbb{E}_{\mathcal{L}}\{\mathbb{E}_{Y|X=\mathbf{x}}\{1(\varphi_{\mathcal{L}}(x) \neq Y)\}\} = P_{\mathcal{L},Y|X=\mathbf{x}}(\varphi_{\mathcal{L}}(\mathbf{x}) \neq Y)$. A maioria não-

1 Observe, entretanto, o fenômeno do tipo Gibbs, resultando em alta variância e alto viés nos limites de $\mathcal{X}$.

tably, Dietterich and Kong [1995], Breiman [1996], Kohavi et al. [1996], Tibshirani [1996] and Domingos [2000] have all developed additive decompositions similar to Theorem 4.1 by redefining the concepts of bias and variance in the case of classification. While these efforts have all provided useful insight into the nature of classification error, none of them really have provided a seductively as simple and satisfactory framework as in regression (for reviews, see [Friedman, 1997; James, 2003; Geurts, 2005]).

An interesting connection with Theorem 4.1 however is to remark that classification algorithms usually work by computing estimates

$$\widehat{p}_{\mathcal{L}}(Y = c|X = \mathbf{x}) \tag{4.6}$$

of the conditional class probability (e.g., $\widehat{p}_{\mathcal{L}}(Y = c|X = \mathbf{x}) = p(c|t)$ in decision trees, as defined in Section 3.4) and then deriving a classification rule by predicting the class that maximizes this estimate, that is:

$$\varphi_{\mathcal{L}}(\mathbf{x}) = \arg\max_{c \in \mathcal{Y}} \widehat{p}_{\mathcal{L}}(Y = c|X = \mathbf{x}) \tag{4.7}$$

As such, a direction for studying classification models is to relate the bias-variance decomposition of these numerical estimates to the expected misclassification error of classification rule 4.7.

We now reproduce the results of Friedman [1997] who made this connection explicit for the case of binary classification. Let us first decompose the expected classification error into an irreducible part associated with the random nature of the output Y and a reducible part that depends on $\varphi_{\mathcal{L}}(\mathbf{x})$, in analogy with Equation 4.3 for the squared error loss. (Note that, to simplify notations, we assume that all probabilities based on the random variable Y is with respect to the distribution of Y at $X = \mathbf{x}$.)

$$\begin{aligned}
&\mathbb{E}_{\mathcal{L}}\{\mathbb{E}_{Y|X=\mathbf{x}}\{1(\varphi_{\mathcal{L}}(\mathbf{x}) \neq Y)\}\} \\
&= P_{\mathcal{L}}(\varphi_{\mathcal{L}}(\mathbf{x}) \neq Y) \\
&= 1 - P_{\mathcal{L}}(\varphi_{\mathcal{L}}(\mathbf{x}) = Y) \\
&= 1 - P_{\mathcal{L}}(\varphi_{\mathcal{L}}(\mathbf{x}) = \varphi_B(\mathbf{x}))P(\varphi_B(\mathbf{x}) = Y) \\
&\quad - P_{\mathcal{L}}(\varphi_{\mathcal{L}}(\mathbf{x}) \neq \varphi_B(\mathbf{x}))P(\varphi_B(\mathbf{x}) \neq Y) \\
&= P(\varphi_B(\mathbf{x}) \neq Y) + P_{\mathcal{L}}(\varphi_{\mathcal{L}}(\mathbf{x}) \neq \varphi_B(\mathbf{x})) \\
&\quad - 2P_{\mathcal{L}}(\varphi_{\mathcal{L}}(\mathbf{x}) \neq \varphi_B(\mathbf{x}))P(\varphi_B(\mathbf{x}) \neq Y) \\
&= P(\varphi_B(\mathbf{x}) \neq Y) + P_{\mathcal{L}}(\varphi_{\mathcal{L}}(\mathbf{x}) \neq \varphi_B(\mathbf{x}))(2P(\varphi_B(\mathbf{x}) = Y) - 1)
\end{aligned} \tag{4.8}$$

In this form, the first term is the irreducible error of the Bayes model. The second term is the increased error due to the misestimation of the optimal decision boundary. The probability $P_{\mathcal{L}}(\varphi_{\mathcal{L}}(\mathbf{x}) \neq \varphi_B(\mathbf{x}))$ is the probability for the model of making a decision which

principalmente, Dietterich e Kong [1995], Breiman [1996],Kohavi et al. [1996], Tibshirani [1996] e Domingos [2000] desenvolveram decomposições aditivas semelhantes ao Teorema 4.1, redefinindo os conceitos de viés e variância no caso de classificação. Embora todos estes esforços tenham fornecido informações úteis sobre a natureza do erro de classificação, nenhum deles realmente forneceu uma estrutura sedutoramente tão simples e satisfatória como na regressão (para revisões, ver [Friedman, 1997; James, 2003; Geurts, 2005]).

Uma conexão interessante com o Teorema 4.1, entretanto, é observar que algoritmos de classificação geralmente funcionam calculando estimativas

$$\widehat{p}_{\mathcal{L}}(Y = c|X = \mathbf{x}) \tag{4.6}$$

da probabilidade de classe condicional (por exemplo, $\widehat{p}_{\mathcal{L}}(Y = c|X = \mathbf{x}) = p(c|t)$ em árvores de decisão, conforme definido na Seção 3.4) e então derivar uma regra de classificação prevendo a classe que maximiza esta estimativa, isto é:

$$\varphi_{\mathcal{L}}(\mathbf{x}) = \arg\max_{c \in \mathcal{Y}} \widehat{p}_{\mathcal{L}}(Y = c|X = \mathbf{x}) \tag{4.7}$$

Como tal, uma direção para estudar modelos de classificação é relacionar a decomposição de viés-variância dessas estimativas numéricas ao erro de classificação incorreto esperado da regra de classificação4.7.

Reproduzimos agora os resultados de Friedman [1997] que tornou esta conexão explícita para o caso de classificação binária. Vamos primeiro decompor o erro de classificação esperado em uma parte irredutível associada à natureza aleatória da saída Y e uma parte redutível que depende de $\varphi_{\mathcal{L}}(\mathbf{x})$, em analogia com a Equação 4.3 para a perda de erro quadrático. (Observe que, para simplificar as notações, assumimos que todas as probabilidades baseadas na variável aleatória Y são relativas à distribuição de Y em $X = \mathbf{x}$.)

$$\begin{aligned}
&\mathbb{E}_{\mathcal{L}}\{\mathbb{E}_{Y|X=\mathbf{x}}\{1(\varphi_{\mathcal{L}}(\mathbf{x}) \neq Y)\}\} \\
&= P_{\mathcal{L}}(\varphi_{\mathcal{L}}(\mathbf{x}) \neq Y) \\
&= 1 - P_{\mathcal{L}}(\varphi_{\mathcal{L}}(\mathbf{x}) = Y) \\
&= 1 - P_{\mathcal{L}}(\varphi_{\mathcal{L}}(\mathbf{x}) = \varphi_B(\mathbf{x}))P(\varphi_B(\mathbf{x}) = Y) \\
&\quad - P_{\mathcal{L}}(\varphi_{\mathcal{L}}(\mathbf{x}) \neq \varphi_B(\mathbf{x}))P(\varphi_B(\mathbf{x}) \neq Y) \\
&= P(\varphi_B(\mathbf{x}) \neq Y) + P_{\mathcal{L}}(\varphi_{\mathcal{L}}(\mathbf{x}) \neq \varphi_B(\mathbf{x})) \\
&\quad - 2P_{\mathcal{L}}(\varphi_{\mathcal{L}}(\mathbf{x}) \neq \varphi_B(\mathbf{x}))P(\varphi_B(\mathbf{x}) \neq Y) \\
&= P(\varphi_B(\mathbf{x}) \neq Y) + P_{\mathcal{L}}(\varphi_{\mathcal{L}}(\mathbf{x}) \neq \varphi_B(\mathbf{x}))(2P(\varphi_B(\mathbf{x}) = Y) - 1)
\end{aligned} \tag{4.8}$$

Nesta forma, o primeiro termo é o erro irredutível do modelo de Bayes. O segundo termo é o erro aumentado devido à estimativa incorreta do limite de decisão ideal. A probabilidade $P_{\mathcal{L}}(\varphi_{\mathcal{L}}(\mathbf{x}) \neq \varphi_B(\mathbf{x}))$ é a probabilidade do modelo tomar uma decisão que

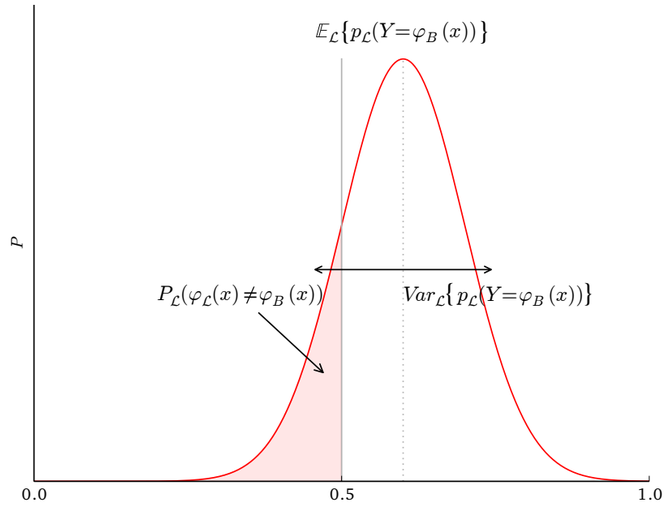


Figure 4.3: Probability distribution of the estimate $\widehat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x}))$.

is different from the decision of the Bayes model. This happens when the estimate $\widehat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x}))$ is lower than 0.5, that is:

$$P_{\mathcal{L}}(\varphi_{\mathcal{L}}(\mathbf{x}) \neq \varphi_B(\mathbf{x})) = P_{\mathcal{L}}(\widehat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x})) < 0.5) \tag{4.9}$$

As Figure 4.3 illustrates, probability 4.9 in fact corresponds to the tail area on the left side of the decision threshold (at 0.5) of the distribution of the estimate.

If we now further assume[2] that the estimate $\widehat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x}))$ is normally distributed, then probability 4.9 can be computed explicitly from its mean and variance:

$$P_{\mathcal{L}}(\widehat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x})) < 0.5) = \Phi\left(\frac{0.5 - \mathbb{E}_{\mathcal{L}}\{\widehat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x}))\}}{\sqrt{\mathbb{V}_{\mathcal{L}}\{\widehat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x}))\}}}\right) \tag{4.10}$$

where $\Phi(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x} \exp(-\frac{t^2}{2}) dt$ is the cumulative distribution function of the standard normal distribution. In summary, the expected generalization error additively decomposes as formulated in Theorem 4.2.

Theorem 4.2. *For the zero-one loss and binary classification, the expected generalization error* $\mathbb{E}_{\mathcal{L}}\{Err(\varphi_{\mathcal{L}}(\mathbf{x}))\}$ *at* $X = \mathbf{x}$ *decomposes as follows:*

$$\mathbb{E}_{\mathcal{L}}\{Err(\varphi_{\mathcal{L}}(\mathbf{x}))\} = P(\varphi_B(\mathbf{x}) \neq Y) \tag{4.11}$$
$$+ \Phi\left(\frac{0.5 - \mathbb{E}_{\mathcal{L}}\{\widehat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x}))\}}{\sqrt{\mathbb{V}_{\mathcal{L}}\{\widehat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x}))\}}}\right)(2P(\varphi_B(\mathbf{x}) = Y) - 1)$$

2 For single decision trees, the normal assumption is certainly not satisfied in all cases, but the qualitative conclusions are still generally valid. When the computations of the estimates involve some averaging process, e.g., as further developed in the case of ensemble of randomized trees, this approximation is however fairly reasonable.

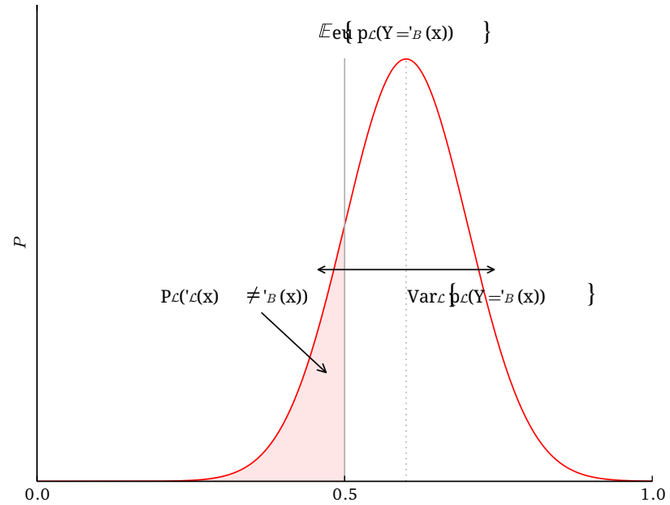


Figura 4.3: Distribuição de probabilidade da estimativa $\widehat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x}))$.

é diferente da decisão do modelo Bayesiano. Isso acontece quando a estimativa $\widehat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x}))$ é menor que 0,5, ou seja:

$$P_{\mathcal{L}}(\varphi_{\mathcal{L}}(\mathbf{x}) \neq \varphi_B(\mathbf{x})) = P_{\mathcal{L}}(\widehat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x})) < 0.5) \tag{4.9}$$

Como ilustra a Figura 4.3, a probabilidade 4,9 corresponde, de facto, à área da cauda no lado esquerdo do limiar de decisão (em 0,5) da distribuição da estimativa.

Se agora assumirmos[2] que a estimativa $\widehat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x}))$ é normalmente distribuída,então a probabilidade 4.9 pode ser calculada explicitamente a partir de sua média e variância:

$$P_{\mathcal{L}}(\widehat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x})) < 0.5) = \Phi\left(\frac{0.5 - \mathbb{E}_{\mathcal{L}}\{\widehat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x}))\}}{\sqrt{\mathbb{V}_{\mathcal{L}}\{\widehat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x}))\}}}\right) \tag{4.10}$$

onde $\Phi(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x} \exp(-\frac{t^2}{2}) dt$ é a função de distribuição cumulativa da distribuição normal padrão. Em resumo, o erro de generalização esperado se decompõe aditivamente conforme formulado no Teorema 4.2.

Teorema 4.2. Para a perda zero-um e classificação binária, o erro de generalização esperado $\mathbb{E}_{\mathcal{L}}\{Err(\varphi_{\mathcal{L}}(x))\}$ *em* $X = x$ *se decompõe da seguinte forma:*

$$\mathbb{E}_{\mathcal{L}}\{Err(\varphi_{\mathcal{L}}(\mathbf{x}))\} = P(\varphi_B(\mathbf{x}) \neq Y) \tag{4.11}$$
$$+ \Phi\left(\frac{0.5 - \mathbb{E}_{\mathcal{L}}\{\widehat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x}))\}}{\sqrt{\mathbb{V}_{\mathcal{L}}\{\widehat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x}))\}}}\right)(2P(\varphi_B(\mathbf{x}) = Y) - 1)$$

2 Para árvores de decisão únicas, a suposição normal certamente não é satisfeita em todos os casos, mas as conclusões qualitativas ainda são geralmente válidas. Quando os cálculos das estimativas envolvem algum processo de cálculo da média, por exemplo, como desenvolvido no caso do conjunto de árvores aleatórias, esta aproximação é, no entanto, bastante razoável.

As a result, Theorem 4.2 establishes a direct connection between the regression variance of the estimates and the classification error of the resulting model. In practice, this decomposition has important consequences:

- When the expected probability estimate $\mathbb{E}_{\mathcal{L}}\{\hat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x})\}$ for the true majority class is greater than 0.5, a reduction of variance of the estimate results in a decrease of the total misclassification error. If $\mathbb{V}_{\mathcal{L}}\{\hat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x}))\} \to 0$, then $\Phi \to 0$ and the expected generalization error tends to the error of the Bayes model. In particular, the generalization error can be driven to its minimum value whatever the regression bias of the estimate (at least as long as $\mathbb{E}_{\mathcal{L}}\{\hat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x})\} > 0.5$).

- Conversely, when $\mathbb{E}_{\mathcal{L}}\{\hat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x})\} < 0.5$, a decrease of variance might actually increase the total misclassification error. If $\mathbb{V}_{\mathcal{L}}\{\hat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x}))\} \to 0$, then $\Phi \to 1$ and the error is maximal.

4.2 ENSEMBLE METHODS BASED ON RANDOMIZATION

Both theorems 4.1 and 4.2 reveal the role of variance in the expected generalization error of a model. In light of these results, a sensible approach for reducing generalization error would therefore consist in driving down the prediction variance, provided the respective bias can be kept the same or not be increased too much.

As it happens, *ensemble methods* constitute a beautifully simple way to do just that. Specifically, the core principle of ensemble methods based on randomization is to introduce random perturbations into the learning procedure in order to produce several different models from a single learning set $\mathcal{L}$ and then to combine the predictions of those models to form the prediction of the ensemble. How predictions are combined and why does it help is formally studied in the next sections.

4.2.1 *Randomized models*

Given a learning set $\mathcal{L}$, a learning algorithm $\mathcal{A}$ deterministically produces a model $\mathcal{A}(\theta, \mathcal{L})$, denoted $\varphi_{\mathcal{L},\theta}$, where θ are hyper-parameters controlling the execution of $\mathcal{A}$. Let us assume that θ includes a random seed parameter for mimicking some stochastic behavior in $\mathcal{A}$, hence producing (pseudo-)randomized models that are more or less different from one random seed to another. (We defer the discussion on specific random perturbations in the case of decision trees to Section 4.3.)

In this context, the bias-variance decomposition can be extended to account for everything that is random, hence considering both $\mathcal{L}$

Como resultado, o Teorema 4.2 estabelece uma conexão direta entre a variância da regressão das estimativas e o erro de classificação do modelo resultante. Na prática, esta decomposição tem consequências importantes:

- Quando a estimativa de probabilidade esperada $\mathbb{E}_{\mathcal{L}}\{\hat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x})\}$ para a classe majoritária verdadeira é maior que 0,5, uma redução da variância da estimativa resulta em uma diminuição do erro total de classificação incorreta. Se $\mathbb{V}_{\mathcal{L}}\{\hat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x}))\} \to 0$, então $\Phi \to 0$ e o erro de generalização esperado tende ao erro do modelo de Bayes. E m particular, o erro de generalização pode ser levado ao seu valor mínimo, qualquer que seja o viés de regressão da estimativa (pelo menos enquanto $\mathbb{E}_{\mathcal{L}}\{\hat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x})\} > 0.5$).

- Por outro lado, quando $\mathbb{E}_{\mathcal{L}}\{\hat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x})\} < 0.5$, uma diminuição da variância pode na verdade aumentar o erro total de classificação incorreta. Se$\mathbb{V}_{\mathcal{L}}\{\hat{p}_{\mathcal{L}}(Y = \varphi_B(\mathbf{x}))\} \to 0$, então $\Phi \to 1$ e o erro é máximo.

4.2 métodos de conjunto baseados em randomização

Ambos os teoremas 4.1 e 4.2 revelam o papel da variância no erro de generalização esperado de um modelo. À luz destes resultados, uma abordagem sensata para reduzir o erro de generalização consistiria, portanto, em reduzir a variância da previsão, desde que o respectivo viés possa ser mantido igual ou não ser aumentado demais.

Na verdade, os métodos de conjunto constituem uma maneira maravilhosamente simples de fazer exatamente isso. Especificamente, o princípio central dos métodos de conjunto baseados em randomização é introduzir perturbações aleatórias no procedimento de aprendizagem, a fim de produzir vários modelos diferentes a partir de um único conjunto de aprendizagem $\mathcal{L}$ e então combinar as previsões desses modelos para formar a previsão do conjunto. Como as previsões são combinadas e por que isso ajuda é estudado formalmente nas próximas seções.

4.2.1 Modelos randomizados

Dado um conjunto de aprendizagem $\mathcal{L}$, um algoritmo de aprendizagem $\mathcal{A}$ produz deterministicamente um modelo $\mathcal{A}(\theta, \mathcal{L})$, denotado $\varphi_{\mathcal{L},\theta}$, onde θ são hiperparâmetros que controlam a execução de $\mathcal{A}$. Suponhamos que θ inclua um parâmetro de semente aleatório para imitar algum comportamento estocástico em $\mathcal{A}$, produzindo assim modelos (pseudo-)aleatórios que são mais ou menos diferentes de uma semente aleatória para outra. (Adiamos a discussão sobre perturbações aleatórias específicas no caso de árvores de decisão para a Seção 4.3.)

Neste contexto, a decomposição viés-variância pode ser estendida para dar conta de tudo que é aleatório, considerando, portanto, ambos $\mathcal{L}$

and θ as random variables[3]. Accordingly, theorems 4.1 and 4.2 naturally extend to the expected generalization error $\mathbb{E}_{\mathcal{L},\theta}\{Err(\varphi_{\mathcal{L},\theta}(\mathbf{x}))\}$ of the randomized model $\varphi_{\mathcal{L},\theta}$ by replacing expectations $\mathbb{E}_{\mathcal{L}}\{.\}$ and variances $\mathbb{V}_{\mathcal{L}}\{.\}$ with their respective counterparts $\mathbb{E}_{\mathcal{L},\theta}\{.\}$ and $\mathbb{V}_{\mathcal{L},\theta}\{.\}$ computed over the joint distribution of $\mathcal{L}$ and θ. In regression, the bias-variance decomposition of the squared error loss thus becomes:

$$\mathbb{E}_{\mathcal{L},\theta}\{Err(\varphi_{\mathcal{L},\theta}(\mathbf{x}))\} = noise(\mathbf{x}) + bias^2(\mathbf{x}) + var(\mathbf{x}), \tag{4.12}$$

where

$$noise(\mathbf{x}) = Err(\varphi_B(\mathbf{x})), \tag{4.13}$$
$$bias^2(\mathbf{x}) = (\varphi_B(\mathbf{x}) - \mathbb{E}_{\mathcal{L},\theta}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\})^2, \tag{4.14}$$
$$var(\mathbf{x}) = \mathbb{E}_{\mathcal{L},\theta}\{(\mathbb{E}_{\mathcal{L},\theta}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\} - \varphi_{\mathcal{L},\theta}(\mathbf{x}))^2\}. \tag{4.15}$$

In this form, variance now accounts for both the prediction variability due to the randomness of the learning set $\mathcal{L}$ and the variability due to the randomness of the learning algorithm itself. As such, the variance of a randomized algorithm is typically larger than the variance of its deterministic counterpart. Depending on the strength of randomization, bias also usually increases, but often to a smaller extent than variance.

While randomizing an algorithm might seem counter-intuitive, since it increases both variance and bias, we will show in Section 4.2.3 that combining several such randomized models might actually achieve better performance than a single non-randomized model.

4.2.2 *Combining randomized models*

Let us assume a set of M randomized models $\{\varphi_{\mathcal{L},\theta_m}|m = 1,\dots,M\}$, all learned on the same data $\mathcal{L}$ but each built from an independent random seed θ_m. Ensemble methods work by combining the predictions of these models into a new *ensemble* model, denoted $\psi_{\mathcal{L},\theta_1,\dots,\theta_M}$, such that the expected generalization error of the ensemble is (hopefully) smaller than the expected generalization error of the individual randomized models.

In regression, for the squared error loss, the most common way to combine the randomized models into an ensemble is to average their predictions to form the final prediction:

$$\psi_{\mathcal{L},\theta_1,\dots,\theta_M}(\mathbf{x}) = \frac{1}{M}\sum_{m=1}^{M}\varphi_{\mathcal{L},\theta_m}(\mathbf{x}) \tag{4.16}$$

The rationale is that the average prediction is the prediction that minimizes the average squared error with respect to the individual predictions of the models. In that sense, the average prediction is the closest prediction with respect to all individual predictions.

3 From now on, and without loss of generality, we assume that the random variable θ only controls the randomness of the learning algorithm.

e θcomo variáveis aleatórias[3]. Assim, os teoremas 4.1 e 4.2 estendem-se naturalmente ao erro de generalização esperado$\mathbb{E}_{\mathcal{L},\theta}\{Err(\varphi_{\mathcal{L},\theta}(x))\}$do modelo aleatório $\varphi_{\mathcal{L},\theta}$ substituindo expectativas $\mathbb{E}_{\mathcal{L}}\{.\}$ e variâncias $\mathbb{V}_{\mathcal{L}}\{.\}$ com suas respectivas contrapartes $\mathbb{E}_{\mathcal{L},\theta}\{.\}$ e $\mathbb{V}_{\mathcal{L},\theta}\{.\}$calculado sobre a distribuição conjunta de $\mathcal{L}$ e θ. Na regressão, a decomposição de polarização-variância da perda de erro quadrático torna-se assim:

$$\mathbb{E}_{\mathcal{L},\theta}\{Err(\varphi_{\mathcal{L},\theta}(\mathbf{x}))\} = noise(\mathbf{x}) + bias^2(\mathbf{x}) + var(\mathbf{x}), \tag{4.12}$$

onde

$$noise(\mathbf{x}) = Err(\varphi_B(\mathbf{x})), \tag{4.13}$$
$$bias^2(\mathbf{x}) = (\varphi_B(\mathbf{x}) - \mathbb{E}_{\mathcal{L},\theta}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\})^2, \tag{4.14}$$
$$var(\mathbf{x}) = \mathbb{E}_{\mathcal{L},\theta}\{(\mathbb{E}_{\mathcal{L},\theta}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\} - \varphi_{\mathcal{L},\theta}(\mathbf{x}))^2\}. \tag{4.15}$$

Nesta forma, a variância agora é responsável tanto pela variabilidade de predição devido à aleatoriedade do conjunto de aprendizagem $\mathcal{L}$ quanto pela variabilidade devido à aleatoriedade do próprio algoritmo de aprendizagem. Como tal, a variância de um algoritmo aleatório é normalmente maior do que a variância da sua contraparte deter minística. Dependendo da força da randomização, o viés também costu ma aumentar, mas muitas vezes em menor extensão que a variância.

Embora a aleatorização de um algoritmo possa parecer contra-intuitiva, uma vez que aumenta tanto a variância como o viés, mostraremos na Secção 4.2.3 que a combinação de vários desses modelos aleatorizados pode, na verdade, alcançar um melhor desempenho do que um único modelo não aleatorizado.

4.2.2 *Combinando modelos randomizados*

Vamos supor um conjunto de M modelos randomizados $\{\varphi_{\mathcal{L},\theta_m}|m = 1,\dots,M\}$, todos aprendidos com os mesmos dados $\mathcal{L}$, mas c ada um construído a partir de uma semente aleatória independente θ_m. Os métodos de conjunto funcionam combinando as previsões desses modelos em um novo modelo de conjunto, denotado $\psi_{\mathcal{L},\theta_1,\dots,\theta_M}$, de modo que o erro de generalização esperado do conjunto seja (espero) menor que o erro de generalização esperado dos modelos randomizados individuais.

Na regressão, para a perda de erro quadrático, a maneira mais comum de combinar os modelos randomizados em um conjunto é calc ular a média de suas previsões para formar a previsão final:

$$\psi_{\mathcal{L},\theta_1,\dots,\theta_M}(\mathbf{x}) = \frac{1}{M}\sum_{m=1}^{M}\varphi_{\mathcal{L},\theta_m}(\mathbf{x}) \tag{4.16}$$

A justificativa é que a previsão média é a previsão que minimiza o erro quadrático médio em relação às previsões individuais dos modelos. Nesse sentido, a previsão média é a previsão mais próxima em relação a todas as previsões individuais.

3 A partir de agora, e sem perda de generalidade, assumimos que a variável aleatória θ controla apenas a aleatoriedade do algoritmo de aprendizagem.

Ambiguity decomposition

For prediction averaging, as defined in Equation 4.16, the *ambiguity decomposition* [Krogh et al., 1995] guarantees the generalization error of the ensemble to be lower than the average generalization error of its constituents. Formally, the ambiguity decomposition states that

$$\mathrm{Err}(\psi_{\mathcal{L},\theta_1,\ldots,\theta_M}) = \overline{\mathrm{E}} - \overline{\mathrm{A}} \tag{4.17}$$

where

$$\overline{\mathrm{E}} = \frac{1}{M}\sum_{m=1}^{M}\mathrm{Err}(\varphi_{\mathcal{L},\theta_m}), \tag{4.18}$$

$$\overline{\mathrm{A}} = \mathbb{E}_X\{\frac{1}{M}\sum_{m=1}^{M}(\varphi_{\mathcal{L},\theta_m}(X) - \psi_{\mathcal{L},\theta_1,\ldots,\theta_M}(X))^2\}. \tag{4.19}$$

The first term is the average generalization error of the individual models. The second term is the ensemble ambiguity and corresponds to the variance of the individual predictions around the prediction of the ensemble. Since $\overline{A}$ is non-negative, the generalization error of the ensemble is therefore smaller than the average generalization error of its constituents.

In classification, for the zero-one loss, predictions are usually aggregated by considering the models in the ensemble as a committee and then resorting to *majority voting* to form the final prediction:

$$\psi_{\mathcal{L},\theta_1,\ldots,\theta_M}(\mathbf{x}) = \arg\max_{c\in\mathcal{Y}}\sum_{m=1}^{M}1(\varphi_{\mathcal{L},\theta_m}(\mathbf{x}) = c) \tag{4.20}$$

Similarly, the rationale is that the majority prediction is the prediction that minimizes the average zero-one error with respect to the individual predictions. Alternatively, when individual models provide class probability estimates $\widehat{p}_{\mathcal{L},\theta_m}(Y = c|X = \mathbf{x})$, *soft voting* [Zhou, 2012] consists in averaging the class probability estimates and then predict the class which is the most likely:

$$\psi_{\mathcal{L},\theta_1,\ldots,\theta_M}(\mathbf{x}) = \arg\max_{c\in\mathcal{Y}}\frac{1}{M}\sum_{m=1}^{M}\widehat{p}_{\mathcal{L},\theta_m}(Y = c|X = \mathbf{x}) \tag{4.21}$$

As empirically investigated by Breiman [1994], both approaches yield results that are nearly identical[4]. From a practical point of view however, Equation 4.21 has the advantage of providing smoother class probability estimates for the ensemble, which may prove to be useful in critical applications, e.g., when (estimates of) the certainty about

4 In the case of ensembles of fully developed decision trees that perfectly classify all samples from $\mathcal{L}$, majority voting and soft voting are exactly equivalent.

Decomposição de ambiguidade

Para a média de predição, conforme definido na Equação 4.16, a deco mposição de ambiguidade [Krogh et al., 1995] garante que o erro de generalização do conjunto seja inferior ao erro de generalização médio de seus constituintes. Formalmente, a decomposição da ambiguidade afirma que

$$\mathrm{Err}(\psi_{\mathcal{L},\theta_1,\ldots,\theta_M}) = \overline{\mathrm{E}} - \overline{\mathrm{A}} \tag{4.17}$$

onde

$$\overline{\mathrm{E}} = \frac{1}{M}\sum_{m=1}^{M}\mathrm{Err}(\varphi_{\mathcal{L},\theta_m}), \tag{4.18}$$

$$\overline{\mathrm{A}} = \mathbb{E}_X\{\frac{1}{M}\sum_{m=1}^{M}(\varphi_{\mathcal{L},\theta_m}(X) - \psi_{\mathcal{L},\theta_1,\ldots,\theta_M}(X))^2\}. \tag{4.19}$$

O primeiro termo é o erro médio de generalização dos modelos individuais. O segundo termo é a ambiguidade do conjunto e corresponde à variância das previsões individuais em torno da previsão do conjunto. Como $\overline{A}$ é não negativo, o erro de generalização do conjunto é portanto menor que o erro de generalização médio dos seus constituintes.

Na classificação, para a perda zero-um, as previsões são geralmente agregadas considerando os modelos do conjunto como um comitê e então recorrendo à votação majoritária para formar a previsão final:

$$\psi_{\mathcal{L},\theta_1,\ldots,\theta_M}(\mathbf{x}) = \arg\max_{c\in\mathcal{Y}}\sum_{m=1}^{M}1(\varphi_{\mathcal{L},\theta_m}(\mathbf{x}) = c) \tag{4.20}$$

Da mesma forma, a justificativa é que a previsão majoritária é a previsão que minimiza o erro médio zero-um em relação às previsões individuais. Alternativamente, quando modelos individuais fornecem estimativas de probabilidade de classe $\widehat{p}_{\mathcal{L},\theta_m}(Y = c|X = \mathbf{x})$, a votação suave [Zhou, 2012] consiste em calcular a média das estimativas de probabilidade de classe e então prever a classe que é a mais provável:

$$\psi_{\mathcal{L},\theta_1,\ldots,\theta_M}(\mathbf{x}) = \arg\max_{c\in\mathcal{Y}}\frac{1}{M}\sum_{m=1}^{M}\widehat{p}_{\mathcal{L},\theta_m}(Y = c|X = \mathbf{x}) \tag{4.21}$$

Conforme investigado empiricamente por Breiman, [1994], ambas as abordagens produzem resultados quase idênticos[4]. Do ponto de vista prático, entretanto, a Equação 4.21 tem a vantagem de fornecer estimativas de probabilidade de classe mais suaves para o conjunto, o que pode ser útil em aplicações críticas, por exemplo, quando (estimativas de) a certeza sobre

4 No caso de conjuntos de árvores de decisão totalmente desenvolvidas que classificam perfeitamente todas as amostras de $\mathcal{L}$, a votação majoritária e a votação suave são exatamente equivalentes.

predictions is as important as the predictions themselves. Additionally, combining predictions in this way makes it easy to study the expected generalization error of the ensemble – it suffices to plug the averaged estimates into Theorem 4.2. For these reasons, and for the rest of this work, predictions in classification are now assumed to be combined with soft voting (see Equation 4.21) unless mentioned otherwise.

CONDORCET'S JURY THEOREM

Majority voting, as defined in Equation 4.20, finds its origins in the *Condorcet's jury theorem* from the field of political science. Let consider a group of M voters that wishes to reach a decision by majority vote. The theorem states that if each voter has an independent probability $p > \frac{1}{2}$ of voting for the correct decision, then adding more voters increases the probability of the majority decision to be correct. When $M \to \infty$, the probability that the decision taken by the group is correct approaches 1. Conversely, if $p < \frac{1}{2}$, then each voter is more likely to vote incorrectly and increasing M makes things worse.

4.2.3 *Bias-variance decomposition of an ensemble*

Let us now study the bias-variance decomposition of the expected generalization error of an ensemble $\psi_{\mathcal{L},\theta_1,\ldots,\theta_M}$, first in the case in case of regression and then for classification.

To simplify notations in the analysis below, let us denote the mean prediction at $X = \mathbf{x}$ of a single randomized model $\varphi_{\mathcal{L},\theta_m}$ and its respective prediction variance as:

$$\mu_{\mathcal{L},\theta_m}(\mathbf{x}) = \mathbb{E}_{\mathcal{L},\theta_m}\{\varphi_{\mathcal{L},\theta_m}(\mathbf{x})\} \tag{4.22}$$

$$\sigma^2_{\mathcal{L},\theta_m}(\mathbf{x}) = \mathbb{V}_{\mathcal{L},\theta_m}\{\varphi_{\mathcal{L},\theta_m}(\mathbf{x})\} \tag{4.23}$$

4.2.3.1 *Regression*

From Theorem 4.1, the expected generalization error of an ensemble $\psi_{\mathcal{L},\theta_1,\ldots,\theta_M}$ made of M randomized models decomposes into a sum of $\text{noise}(\mathbf{x})$, $\text{bias}^2(\mathbf{x})$ and $\text{var}(\mathbf{x})$ terms.

The noise term only depends on the intrinsic randomness of Y. Its value stays therefore the same, no matter the learning algorithm:

$$\text{noise}(\mathbf{x}) = \mathbb{E}_{Y|X=\mathbf{x}}\{(Y - \varphi_B(\mathbf{x}))^2\} \tag{4.24}$$

The (squared) bias term is the (squared) difference between the prediction of the Bayes model and the average prediction of the model. For an ensemble, the average prediction is in fact the same as the av-

previsões são tão importantes quanto as próprias previsões. Além disso, combinar previsões desta forma facilita o estudo do erro de generalização esperado do conjunto – basta inserir as estimativas médias no Teorema 4.2. Por estas razões, e para o resto deste trabalho, assume-se agora que as previsões na classificação são combinadas com a votação suave (ver Equação 4.21), salvo indicação em contrário.

Teorema do júri de Condorcet

A votação por maioria, conforme definido na Equação 4.20, tem origem no teorema do júri de Condorcet, do campo da ciência política. Consideremos um grupo de M eleitores que desejam chegar a uma decisão por maioria de votos. O teorema afirma que se cada eleitor tiver uma probabilidade independente$p > \frac{1}{2}$ de votar na decisão correta, então adicionar mais eleitores aumenta a probabilidade de a decisão da maioria ser correta. Quando$M \to \infty$, a probabilidade de a decisão tomada pelo grupo ser correta aproxima-se de 1. Por outro lado, se $p < \frac{1}{2}$, então é mais provável que cada eleitor vote incorretamente e aumentar Mtorna as coisas piores.

4.2.3 Decomposição de polarização-variância de um conjunto

Estudemos agora a decomposição viés-variância do erro de generalização esperado de um conjunto $\psi_{\mathcal{L},\theta_1,\ldots,\theta_M}$, primeiro no caso de regressão e depois para classificação.

Para simplificar as notações na análise abaixo, vamos denotar a previsão média em $X = x$ de um único modelo randomizado $\varphi_{\mathcal{L},\theta_m}$ e sua respectiva variância de previsão como:

$$\mu_{\mathcal{L},\theta_m}(\mathbf{x}) = \mathbb{E}_{\mathcal{L},\theta_m}\{\varphi_{\mathcal{L},\theta_m}(\mathbf{x})\} \tag{4.22}$$

$$\sigma^2_{\mathcal{L},\theta_m}(\mathbf{x}) = \mathbb{V}_{\mathcal{L},\theta_m}\{\varphi_{\mathcal{L},\theta_m}(\mathbf{x})\} \tag{4.23}$$

4.2.3.1 Regressão

A partir do Teorema 4.1, o erro de generalização esperado de um conjunto $\psi_{\mathcal{L},\theta_1,\ldots,\theta_M}$ feito de M modelos randomizados se decompõe em uma soma de termos de $\text{ruído}(x)$, $\text{viés}^2(x)$ e $\text{var}(x)$.

O termo ruído depende apenas da aleatoriedade intrínseca de Y. Seu valor permanece portanto o mesmo, independentemente do algoritmo de aprendizagem:

$$\text{noise}(\mathbf{x}) = \mathbb{E}_{Y|X=\mathbf{x}}\{(Y - \varphi_B(\mathbf{x}))^2\} \tag{4.24}$$

O termo de viés (quadrado) é a diferença (quadrada) entre a previsão do modelo Bayes e a previsão média do modelo. Para um conjunto, a previsão média é de fato a mesma que a média.

erage prediction of the corresponding randomized individual model. Indeed,

$$\begin{aligned}\mathbb{E}_{\mathcal{L},\theta_1,\ldots,\theta_M}\{\psi_{\mathcal{L},\theta_1,\ldots,\theta_M}(\mathbf{x})\} &= \mathbb{E}_{\mathcal{L},\theta_1,\ldots,\theta_M}\{\frac{1}{M}\sum_{m=1}^{M}\varphi_{\mathcal{L},\theta_m}(\mathbf{x})\} \\ &= \frac{1}{M}\sum_{m=1}^{M}\mathbb{E}_{\mathcal{L},\theta_m}\{\varphi_{\mathcal{L},\theta_m}(\mathbf{x})\} \\ &= \mu_{\mathcal{L},\theta}(\mathbf{x}) \end{aligned} \tag{4.25}$$

since random variables θ_m are independent and all follow the same distribution. As a result,

$$\text{bias}^2(\mathbf{x}) = (\varphi_B(\mathbf{x}) - \mu_{\mathcal{L},\theta}(\mathbf{x}))^2, \tag{4.26}$$

which indicates that the bias of an ensemble of randomized models is the same as the bias of any of the randomized models. Put otherwise, combining randomized models has no effect on the bias.

On variance on the other hand, ensemble methods show all their raison d'etre, virtually reducing the variability of predictions to almost nothing and thereby improving the accuracy of the ensemble. Before considering the variance of $\psi_{\mathcal{L},\theta_1,\ldots,\theta_M}(\mathbf{x})$ however, let us first derive the correlation coefficient $\rho(\mathbf{x})$ between the predictions of two randomized models built on the same learning set, but grown from two independent random seeds θ' and θ''. From the definition of the Pearson's correlation coefficient, it comes:

$$\begin{aligned}\rho(\mathbf{x}) &= \frac{\mathbb{E}_{\mathcal{L},\theta',\theta''}\{(\varphi_{\mathcal{L},\theta'}(\mathbf{x}) - \mu_{\mathcal{L},\theta'}(\mathbf{x}))(\varphi_{\mathcal{L},\theta''}(\mathbf{x}) - \mu_{\mathcal{L},\theta''}(\mathbf{x}))\}}{\sigma_{\mathcal{L},\theta'}(\mathbf{x})\sigma_{\mathcal{L},\theta''}(\mathbf{x})} \\ &= \frac{\mathbb{E}_{\mathcal{L},\theta',\theta''}\{\varphi_{\mathcal{L},\theta'}(\mathbf{x})\varphi_{\mathcal{L},\theta''}(\mathbf{x}) - \varphi_{\mathcal{L},\theta'}(\mathbf{x})\mu_{\mathcal{L},\theta''}(\mathbf{x}) - \varphi_{\mathcal{L},\theta''}(\mathbf{x})\mu_{\mathcal{L},\theta'}(\mathbf{x}) + \mu_{\mathcal{L},\theta'}(\mathbf{x})\mu_{\mathcal{L},\theta''}(\mathbf{x})\}}{\sigma^2_{\mathcal{L},\theta}(\mathbf{x})} \\ &= \frac{\mathbb{E}_{\mathcal{L},\theta',\theta''}\{\varphi_{\mathcal{L},\theta'}(\mathbf{x})\varphi_{\mathcal{L},\theta''}(\mathbf{x})\} - \mu^2_{\mathcal{L},\theta}(\mathbf{x})}{\sigma^2_{\mathcal{L},\theta}(\mathbf{x})} \end{aligned} \tag{4.27}$$

by linearity of the expectation and exploiting the fact that random variables θ' and θ'' follow the same distribution. Intuitively, $\rho(\mathbf{x})$ represents the strength of the random perturbations introduced in the learning algorithm. When it is close to 1, predictions of two randomized models are highly correlated, suggesting that randomization has no sensible effect on the predictions. By contrast, when it is close to 0, predictions of the randomized models are decorrelated, hence indicating that randomization has a strong effect on the predictions. At the limit, when $\rho(\mathbf{x}) = 0$, predictions of two models built on the same learning set $\mathcal{L}$ are independent, which happens when they are perfectly random. As proved later with Equation 4.31, let us finally also remark that the correlation term $\rho(\mathbf{x})$ is non-negative, which confirms that randomization has a decorrelation effect only.

previsão de idade do modelo individual randomizado correspondente. De fato,

$$\begin{aligned}\mathbb{E}_{\mathcal{L},\theta_1,\ldots,\theta_M}\{\psi_{\mathcal{L},\theta_1,\ldots,\theta_M}(\mathbf{x})\} &= \mathbb{E}_{\mathcal{L},\theta_1,\ldots,\theta_M}\{\frac{1}{M}\sum_{m=1}^{M}\varphi_{\mathcal{L},\theta_m}(\mathbf{x})\} \\ &= \frac{1}{M}\sum_{m=1}^{M}\mathbb{E}_{\mathcal{L},\theta_m}\{\varphi_{\mathcal{L},\theta_m}(\mathbf{x})\} \\ &= \mu_{\mathcal{L},\theta}(\mathbf{x}) \end{aligned} \tag{4.25}$$

já que variáveis aleatórias θ_m são independentes e todas seguem a mesma distribuição. Como resultado,

$$\text{bias}^2(\mathbf{x}) = (\varphi_B(\mathbf{x}) - \mu_{\mathcal{L},\theta}(\mathbf{x}))^2, \tag{4.26}$$

o que indica que o viés de um conjunto de modelos randomizados é o mesmo que o viés de qualquer um dos modelos randomizados. Dito de outra forma, a combinação de modelos aleatórios não tem efeito sobre o viés.

Por outro lado, na variância, os métodos de conjunto mostram toda a sua razão de ser, reduzindo virtualmente a variabilidade das previsões a quase nada e melhorando assim a precisão do conjunto. Antes de considerar a variância de$\psi_{\mathcal{L},\theta_1,\ldots,\theta_M}(\mathbf{x})$ no entanto, vamos primeiro derivar o coeficiente de correlação $\rho(\mathbf{x})$ entre as previsões de dois modelos aleatórios construídos no mesmo conjunto de aprendizagem, mas cultivados a partir de duas sementes aleatórias independentes θ' e θ''. Da definição do coeficiente de correlação de Pearson, vem:

$$\begin{aligned}\rho(\mathbf{x}) &= \frac{\mathbb{E}_{\mathcal{L},\theta',\theta''}\{(\varphi_{\mathcal{L},\theta'}(\mathbf{x}) - \mu_{\mathcal{L},\theta'}(\mathbf{x}))(\varphi_{\mathcal{L},\theta''}(\mathbf{x}) - \mu_{\mathcal{L},\theta''}(\mathbf{x}))\}}{\sigma_{\mathcal{L},\theta'}(\mathbf{x})\sigma_{\mathcal{L},\theta''}(\mathbf{x})} \\ &= \frac{\mathbb{E}_{\mathcal{L},\theta',\theta''}\{\varphi_{\mathcal{L},\theta'}(\mathbf{x})\varphi_{\mathcal{L},\theta''}(\mathbf{x}) - \varphi_{\mathcal{L},\theta'}(\mathbf{x})\mu_{\mathcal{L},\theta''}(\mathbf{x}) - \varphi_{\mathcal{L},\theta''}(\mathbf{x})\mu_{\mathcal{L},\theta'}(\mathbf{x}) + \mu_{\mathcal{L},\theta'}(\mathbf{x})\mu_{\mathcal{L},\theta''}(\mathbf{x})\}}{\sigma^2_{\mathcal{L},\theta}(\mathbf{x})} \\ &= \frac{\mathbb{E}_{\mathcal{L},\theta',\theta''}\{\varphi_{\mathcal{L},\theta'}(\mathbf{x})\varphi_{\mathcal{L},\theta''}(\mathbf{x})\} - \mu^2_{\mathcal{L},\theta}(\mathbf{x})}{\sigma^2_{\mathcal{L},\theta}(\mathbf{x})} \end{aligned} \tag{4.27}$$

pela linearidade da expectativa e explorando o fato de que as variáveis aleatórias θ' e θ'' seguem a mesma distribuição. Intuitivamente, $\rho(\mathbf{x})$ representa a força das perturbações aleatórias introduzidas no algoritmo de aprendizagem. Quando está próximo de 1, as previsões de dois modelos randomizados estão altamente correlacionadas, sugerindo que a randomização não tem efeito sensível nas previsões. Por outro lado, quando está próximo de 0, as previsões dos modelos randomizados são desrelacionadas, indicando assim que a randomização tem um forte efeito nas previsões. No limite, quando $\rho(\mathbf{x}) = 0$, as previsões de dois modelos construídos no mesmo conjunto de aprendizagem $\mathcal{L}$ são independentes, o que acontece quando são perfeitamente aleatórias. Como provado posteriormente com a Equação 4.31, observemos finalmente que o termo de correlação $\rho(\mathbf{x})$ é não negativo, o que confirma que a randomização t em apenas um efeito de descorrelação.

From Equation 4.27, the variance of $\psi_{\mathcal{L},\theta_1,\ldots,\theta_M}(\mathbf{x})$ can now be derived as follows:

$$\begin{aligned}
\text{var}(\mathbf{x}) &= \mathbb{V}_{\mathcal{L},\theta_1,\ldots,\theta_M}\{\frac{1}{M}\sum_{m=1}^{M}\varphi_{\mathcal{L},\theta_m}(\mathbf{x})\} \\
&= \frac{1}{M^2}\left[\mathbb{E}_{\mathcal{L},\theta_1,\ldots,\theta_M}\{(\sum_{m=1}^{M}\varphi_{\mathcal{L},\theta_m}(\mathbf{x}))^2\} - \mathbb{E}_{\mathcal{L},\theta_1,\ldots,\theta_M}\{\sum_{m=1}^{M}\varphi_{\mathcal{L},\theta_m}(\mathbf{x})\}^2\right]
\end{aligned}$$

by exploiting the facts that $\mathbb{V}\{aX\} = a^2\mathbb{V}\{X\}$, $\mathbb{V}\{X\} = \mathbb{E}\{X^2\} - \mathbb{E}\{X\}^2$ and the linearity of expectation. By rewriting the square of the sum of the $\varphi_{\mathcal{L},\theta_m}(\mathbf{x})$ terms as a sum over all pairwise products $\varphi_{\mathcal{L},\theta_i}(\mathbf{x})\varphi_{\mathcal{L},\theta_j}(\mathbf{x})$, the variance can further be rewritten as:

$$\begin{aligned}
&= \frac{1}{M^2}\left[\mathbb{E}_{\mathcal{L},\theta_1,\ldots,\theta_M}\{\sum_{i,j}\varphi_{\mathcal{L},\theta_i}(\mathbf{x})\varphi_{\mathcal{L},\theta_j}(\mathbf{x})\} - (M\mu_{\mathcal{L},\theta}(\mathbf{x}))^2\right] \\
&= \frac{1}{M^2}\left[\sum_{i,j}\mathbb{E}_{\mathcal{L},\theta_i,\theta_j}\{\varphi_{\mathcal{L},\theta_i}(\mathbf{x})\varphi_{\mathcal{L},\theta_j}(\mathbf{x})\} - M^2\mu^2_{\mathcal{L},\theta}(\mathbf{x})\right] \\
&= \frac{1}{M^2}\Bigg[M\mathbb{E}_{\mathcal{L},\theta}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})^2\} \\
&\quad \hookrightarrow +(M^2-M)\mathbb{E}_{\mathcal{L},\theta',\theta''}\{\varphi_{\mathcal{L},\theta'}(\mathbf{x})\varphi_{\mathcal{L},\theta''}(\mathbf{x})\} - M^2\mu^2_{\mathcal{L},\theta}(\mathbf{x})\Bigg] \\
&= \frac{1}{M^2}\Bigg[M(\sigma^2_{\mathcal{L},\theta}(\mathbf{x}) + \mu^2_{\mathcal{L},\theta}(\mathbf{x})) \\
&\quad \hookrightarrow +(M^2-M)(\rho(\mathbf{x})\sigma^2_{\mathcal{L},\theta}(\mathbf{x}) + \mu^2_{\mathcal{L},\theta}(\mathbf{x})) - M^2\mu^2_{\mathcal{L},\theta}(\mathbf{x})\Bigg] \\
&= \frac{\sigma^2_{\mathcal{L},\theta}(\mathbf{x})}{M} + \rho(\mathbf{x})\sigma^2_{\mathcal{L},\theta}(\mathbf{x}) - \rho(\mathbf{x})\frac{\sigma^2_{\mathcal{L},\theta}(\mathbf{x})}{M} \\
&= \rho(\mathbf{x})\sigma^2_{\mathcal{L},\theta}(\mathbf{x}) + \frac{1-\rho(\mathbf{x})}{M}\sigma^2_{\mathcal{L},\theta}(\mathbf{x}) \qquad (4.28)
\end{aligned}$$

As the size of the ensemble gets arbitrarily large, i.e., as $M \to \infty$, the variance of the ensemble reduces to $\rho(\mathbf{x})\sigma^2_{\mathcal{L},\theta}(\mathbf{x})$. Under the assumption that randomization has some effect on the predictions of randomized models, i.e., assuming $\rho(\mathbf{x}) < 1$, the variance of an ensemble is therefore strictly smaller than the variance of an individual model. As a result, the expected generalization error of an ensemble is strictly smaller than the expected error of a randomized model. As such, improvements in predictions are solely the result of variance reduction, since both noise$(\mathbf{x})$ and bias$^2(\mathbf{x})$ remain unchanged. Additionally, when random effects are strong, i.e., when $\rho(\mathbf{x}) \to 0$, variance reduces to $\frac{\sigma^2_{\mathcal{L},\theta}(\mathbf{x})}{M}$, which can further be driven to 0 by increasing the size of the ensemble. On the other hand, when random effects are weak, i.e., when $\rho(\mathbf{x}) \to 1$, then variance reduces to $\sigma^2_{\mathcal{L},\theta}(\mathbf{x})$ and building an ensemble brings no benefit. Put otherwise, the stronger the

Da Equação 4.27, a variância de $\psi_{\mathcal{L},\theta_1,\ldots,\theta_M}(\mathbf{x})$ pode agora ser derivada da seguinte forma:

$$\begin{aligned}
\text{var}(\mathbf{x}) &= \mathbb{V}_{\mathcal{L},\theta_1,\ldots,\theta_M}\{\frac{1}{M}\sum_{m=1}^{M}\varphi_{\mathcal{L},\theta_m}(\mathbf{x})\} \\
&= \frac{1}{M^2}\left[\mathbb{E}_{\mathcal{L},\theta_1,\ldots,\theta_M}\{(\sum_{m=1}^{M}\varphi_{\mathcal{L},\theta_m}(\mathbf{x}))^2\} - \mathbb{E}_{\mathcal{L},\theta_1,\ldots,\theta_M}\{\sum_{m=1}^{M}\varphi_{\mathcal{L},\theta_m}(\mathbf{x})\}^2\right]
\end{aligned}$$

explorando os fatos que $\mathbb{V}\{aX\} = a^2\mathbb{V}\{X\}$, $\mathbb{V}\{X\} = \mathbb{E}\{X^2\} - \mathbb{E}\{X\}^2$e a l inearidade da expectativa. Ao reescrever o quadrado da soma dos termos $\varphi_{\mathcal{L}}$, $_{\theta_m}(\mathbf{x})$ como uma soma de todos os produtos pareados$\varphi_{\mathcal{L},\theta_i}(\mathbf{x})\varphi_{\mathcal{L},\theta_j}(\mathbf{x})$, a variância pode ainda ser reescrita como:

$$\begin{aligned}
&= \frac{1}{M^2}\left[\mathbb{E}_{\mathcal{L},\theta_1,\ldots,\theta_M}\{\sum_{i,j}\varphi_{\mathcal{L},\theta_i}(\mathbf{x})\varphi_{\mathcal{L},\theta_j}(\mathbf{x})\} - (M\mu_{\mathcal{L},\theta}(\mathbf{x}))^2\right] \\
&= \frac{1}{M^2}\left[\sum_{i,j}\mathbb{E}_{\mathcal{L},\theta_i,\theta_j}\{\varphi_{\mathcal{L},\theta_i}(\mathbf{x})\varphi_{\mathcal{L},\theta_j}(\mathbf{x})\} - M^2\mu^2_{\mathcal{L},\theta}(\mathbf{x})\right] \\
&= \frac{1}{M^2}\Bigg[M\mathbb{E}_{\mathcal{L},\theta}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})^2\} \\
&\quad \hookrightarrow +(M^2-M)\mathbb{E}_{\mathcal{L},\theta',\theta''}\{\varphi_{\mathcal{L},\theta'}(\mathbf{x})\varphi_{\mathcal{L},\theta''}(\mathbf{x})\} - M^2\mu^2_{\mathcal{L},\theta}(\mathbf{x})\Bigg] \\
&= \frac{1}{M^2}\Bigg[M(\sigma^2_{\mathcal{L},\theta}(\mathbf{x}) + \mu^2_{\mathcal{L},\theta}(\mathbf{x})) \\
&\quad \hookrightarrow +(M^2-M)(\rho(\mathbf{x})\sigma^2_{\mathcal{L},\theta}(\mathbf{x}) + \mu^2_{\mathcal{L},\theta}(\mathbf{x})) - M^2\mu^2_{\mathcal{L},\theta}(\mathbf{x})\Bigg] \\
&= \frac{\sigma^2_{\mathcal{L},\theta}(\mathbf{x})}{M} + \rho(\mathbf{x})\sigma^2_{\mathcal{L},\theta}(\mathbf{x}) - \rho(\mathbf{x})\frac{\sigma^2_{\mathcal{L},\theta}(\mathbf{x})}{M} \\
&= \rho(\mathbf{x})\sigma^2_{\mathcal{L},\theta}(\mathbf{x}) + \frac{1-\rho(\mathbf{x})}{M}\sigma^2_{\mathcal{L},\theta}(\mathbf{x}) \qquad (4.28)
\end{aligned}$$

À medida que o tamanho do conjunto fica arbitrariamente grande, ou seja, como $M \to \infty$, a variância do conjunto reduz para $\rho(\mathbf{x})\sigma^2_{\mathcal{L},\theta}(\mathbf{x})$. Supondo que a randomização tenha algum efeito nas previsões de modelos randomizados, ou seja, assumindo $\rho(\mathbf{x}) < 1$, a variância de um conjunto é, portanto, estritamente menor do que a variância de um modelo individual. Como resultado, o erro de generalização esperado de um conjunto é estritamente menor que o erro esperado de um modelo aleatório. Como t al, as melhorias nas previsões são apenas o resultado da redução da variância, uma vez que tanto o ruído$(\mathbf{x})$ quanto o viés$^2(\mathbf{x})$ permanecem inalterados. Além disso, quando os efeitos aleatórios são fortes, ou seja, quando $\rho(\mathbf{x}) \to 0$, a variância reduz para $\frac{\sigma^2_{\mathcal{L},\theta}(\mathbf{x})}{M}$, que pode ainda ser levada a 0 aumentando o tamanho do conjunto. Por outro lado, quando os efeitos aleatórios são fracos, ou seja, quando $\rho(\mathbf{x}) \to 1$, então a variância reduz para$\sigma^2_{\mathcal{L},\theta}(\mathbf{x})$ e construir um conjunto não traz nenhum benefício. Dito de outra forma, quanto mais forte for o

random effects, the larger the reduction of variance due to ensembling, and vice-versa.

In summary, the expected generalization error of an ensemble additively decomposes as stated in Theorem 4.3.

Theorem 4.3. *For the squared error loss, the bias-variance decomposition of the expected generalization error* $\mathbb{E}_{\mathcal{L}}\{\text{Err}(\psi_{\mathcal{L},\theta_1,\ldots,\theta_M}(\mathbf{x}))\}$ *at* $X = \mathbf{x}$ *of an ensemble of* M *randomized models* $\varphi_{\mathcal{L},\theta_m}$ *is*

$$\mathbb{E}_{\mathcal{L}}\{\text{Err}(\psi_{\mathcal{L},\theta_1,\ldots,\theta_M}(\mathbf{x}))\} = noise(\mathbf{x}) + bias^2(\mathbf{x}) + var(\mathbf{x}), \tag{4.29}$$

where

$$\begin{aligned} noise(\mathbf{x}) &= \text{Err}(\varphi_B(\mathbf{x})), \\ bias^2(\mathbf{x}) &= (\varphi_B(\mathbf{x}) - \mathbb{E}_{\mathcal{L},\theta}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\})^2, \\ var(\mathbf{x}) &= \rho(\mathbf{x})\sigma^2_{\mathcal{L},\theta}(\mathbf{x}) + \frac{1-\rho(\mathbf{x})}{M}\sigma^2_{\mathcal{L},\theta}(\mathbf{x}). \end{aligned}$$

In light of Theorem 4.3, the core principle of ensemble methods is thus to introduce random perturbations in order to decorrelate as much as possible the predictions of the individual models, thereby maximizing variance reduction. However, random perturbations need to be carefully chosen so as to increase bias as little as possible. The crux of the problem is to find the right trade-off between randomness and bias.

Alternative variance decomposition

Geurts [2002] (Chapter 4, Equation 4.31) alternatively decomposes the ensemble variance as

$$\text{var}(\mathbf{x}) = \mathbb{V}_{\mathcal{L}}\{\mathbb{E}_{\theta|\mathcal{L}}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\}\} + \frac{1}{M}\mathbb{E}_{\mathcal{L}}\{\mathbb{V}_{\theta|\mathcal{L}}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\}\}. \tag{4.30}$$

The first term of this decomposition is the variance due to the randomness of the learning set $\mathcal{L}$, averaged over the random perturbations due to θ. It measures the dependence of the model on the learning set, independently of θ. The second term is the expectation over all learning sets of the variance with respect to θ. It measures the strength of the random effects. As the decomposition shows, only this last part of the variance can be reduced as a result of averaging, which is consistent with our previous conclusions. The stronger the random effects, the larger the variance with respect to θ, and hence the larger of reduction of variance due to ensembling.

Proposition 4.4. *Decompositions 4.28 and 4.30 of the prediction variance for an ensemble are equivalent.*

Proof. From Equations 4.28 and 4.30, equivalence holds if Equation 4.27 is equivalent to

$$\rho(\mathbf{x}) = \frac{\mathbb{V}_{\mathcal{L}}\{\mathbb{E}_{\theta|\mathcal{L}}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\}\}}{\mathbb{V}_{\mathcal{L}}\{\mathbb{E}_{\theta|\mathcal{L}}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\}\} + \mathbb{E}_{\mathcal{L}}\{\mathbb{V}_{\theta|\mathcal{L}}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\}\}}. \tag{4.31}$$

efeitos aleatórios, maior será a redução da variância devido ao agrupamento e vice-versa.

Em resumo, o erro de generalização esperado de um conjunto decompõe-se aditivamente como afirmado no Teorema 4.3.

Teorema 4.3. Para a perda de erro quadrático, a decomposição de polarização-variância do erro de generalização esperado $\mathbb{E}_{\mathcal{L}}\{\text{Err}(\psi_{\mathcal{L},\theta_1,\ldots,\theta_M}(\mathbf{x}))\}$ *em* $X = x$ *de um conjunto de* M *modelos randomizados* $\varphi_{\mathcal{L},\theta_m}$ *é*

$$\mathbb{E}_{\mathcal{L}}\{\text{Err}(\psi_{\mathcal{L},\theta_1,\ldots,\theta_M}(\mathbf{x}))\} = noise(\mathbf{x}) + bias^2(\mathbf{x}) + var(\mathbf{x}), \tag{4.29}$$

onde

$$\begin{aligned} noise(\mathbf{x}) &= \text{Err}(\varphi_B(\mathbf{x})), \\ bias^2(\mathbf{x}) &= (\varphi_B(\mathbf{x}) - \mathbb{E}_{\mathcal{L},\theta}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\})^2, \\ var(\mathbf{x}) &= \rho(\mathbf{x})\sigma^2_{\mathcal{L},\theta}(\mathbf{x}) + \frac{1-\rho(\mathbf{x})}{M}\sigma^2_{\mathcal{L},\theta}(\mathbf{x}). \end{aligned}$$

À luz do Teorema 4.3, o princípio central dos métodos de conjunto é, portanto, introduzir perturbações aleatórias a fim de descorrelacionar tanto quanto possível as previsões dos modelos individuais, maximizando assim a redução da variância. Entretanto, perturbações aleatórias precisam ser cuidadosamente escolhidas para aumentar o viés o menos possível. O cerne do problema é encontrar o equilíbrio certo entre aleatoriedade e preconceito.

Decomposição alternativa da variância

Geurts [2002] (Capítulo 4, Equação 4.31) decompõe alternativamente a variância do conjunto como

$$\text{var}(\mathbf{x}) = \mathbb{V}_{\mathcal{L}}\{\mathbb{E}_{\theta|\mathcal{L}}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\}\} + \frac{1}{M}\mathbb{E}_{\mathcal{L}}\{\mathbb{V}_{\theta|\mathcal{L}}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\}\}. \tag{4.30}$$

O primeiro termo desta decomposição é a variância devido à aleatoriedade do conjunto de aprendizagem $\mathcal{L}$, calculada a média das perturbações aleatórias devidas a θ. Mede a dependência do modelo do conjunto de aprendizagem, independentemente de θ. O segundo termo é a expectativa sobre todos os conjuntos de aprendizagem da variância em relação a θ. Ele mede a força dos efeitos aleatórios. Como mostra a decomposição, apenas esta última parte da variância pode ser reduzida como resultado da média, o que é consistente com as nossas conclusões anteriores. Quanto mais fortes forem os efeitos aleatórios, maior será a variância em relação a θ e, portanto, maior será a redução da variância devido ao agrupamento.

Proposição 4.4. As decomposições 4.28 e 4.30 da variância de predição para um conjunto são equivalentes.

Prova. Das Equações 4.28 e 4.30, a equivalência é válida se a Equação 4.27 for equivalente a

$$\rho(\mathbf{x}) = \frac{\mathbb{V}_{\mathcal{L}}\{\mathbb{E}_{\theta|\mathcal{L}}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\}\}}{\mathbb{V}_{\mathcal{L}}\{\mathbb{E}_{\theta|\mathcal{L}}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\}\} + \mathbb{E}_{\mathcal{L}}\{\mathbb{V}_{\theta|\mathcal{L}}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\}\}}. \tag{4.31}$$

From the law of total variance, the denominator of Equation 4.27 expands to the denominator of Equation 4.31:

$$\sigma^2_{\mathcal{L},\theta}(\mathbf{x}) = \mathbb{V}_{\mathcal{L}}\{\mathbb{E}_{\theta|\mathcal{L}}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\}\} + \mathbb{E}_{\mathcal{L}}\{\mathbb{V}_{\theta|\mathcal{L}}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\}\} \tag{4.32}$$

Similarly, the numerator of Equation 4.31 can be reexpressed as the numerator of Equation 4.27, thereby proving the equivalence:

$$\begin{aligned}
&\mathbb{V}_{\mathcal{L}}\{\mathbb{E}_{\theta|\mathcal{L}}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\}\} \\
&= \mathbb{E}_{\mathcal{L}}\{(\mathbb{E}_{\theta|\mathcal{L}}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\} - \mathbb{E}_{\mathcal{L}}\{\mathbb{E}_{\theta|\mathcal{L}}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\}\})^2\} \\
&= \mathbb{E}_{\mathcal{L}}\{(\mathbb{E}_{\theta|\mathcal{L}}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\} - \mu_{\mathcal{L},\theta}(\mathbf{x}))^2\} \\
&= \mathbb{E}_{\mathcal{L}}\{\mathbb{E}_{\theta|\mathcal{L}}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\}^2\} - \mu^2_{\mathcal{L},\theta}(\mathbf{x}) \\
&= \mathbb{E}_{\mathcal{L}}\{\mathbb{E}_{\theta'|\mathcal{L}}\{\varphi_{\mathcal{L},\theta'}(\mathbf{x})\}\mathbb{E}_{\theta''|\mathcal{L}}\{\varphi_{\mathcal{L},\theta''}(\mathbf{x})\}\} - \mu^2_{\mathcal{L},\theta}(\mathbf{x}) \\
&= \mathbb{E}_{\mathcal{L},\theta',\theta''}\{\varphi_{\mathcal{L},\theta'}(\mathbf{x})\varphi_{\mathcal{L},\theta''}(\mathbf{x})\} - \mu^2_{\mathcal{L},\theta}(\mathbf{x}).
\end{aligned} \tag{4.33}$$

□

In this form, $\rho(\mathbf{x})$ is interpreted as the ratio between the variance due to the learning set and the total variance, accounting for random effects due to both the learning set and the random perturbations. It is close to 1 when variance is mostly due to the learning set, hence yielding correlated predictions. Conversely, it is close to 0 when variance is mostly due to the random perturbations induced by θ, hence decorrelating the predictions.

4.2.3.2 *Classification*

The decomposition of the expected generalization error of an ensemble in classification directly follows from theorems 4.2 and 4.3. Building an ensemble always reduces the variance of the class probability estimate $\mathbb{E}_{\mathcal{L},\theta}\{\hat{p}_{\mathcal{L},\theta}(Y = \varphi_B(\mathbf{x})\}$ (as shown in Equation 4.28), which results in a decrease of the misclassification error if the expected estimate remains strictly greater than 0.5 in a randomized model.

Shortcomings addressed by ensembles

In complement with the formal bias-variance analysis carried out in the previous section, Dietterich [2000b] identifies three fundamental reasons intuitively explaining why ensembles often work better than single models.

The first reason is statistical. When the learning set is too small, a learning algorithm can typically find several models in the hypothesis space $\mathcal{H}$ that all give the same performance on the training data. Provided their predictions are uncorrelated, averaging several models reduces the risk of choosing the wrong hypothesis.

A partir da lei da variância total, o denominador da Equação 4.27 se expande para o denominador da Equação 4.31:

$$\sigma^2_{\mathcal{L},\theta}(\mathbf{x}) = \mathbb{V}_{\mathcal{L}}\{\mathbb{E}_{\theta|\mathcal{L}}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\}\} + \mathbb{E}_{\mathcal{L}}\{\mathbb{V}_{\theta|\mathcal{L}}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\}\} \tag{4.32}$$

Da mesma forma, o numerador da Equação 4.31 pode ser reexpresso como o numerador da Equação 4.27, provando assim a equivalência:

$$\begin{aligned}
&\mathbb{V}_{\mathcal{L}}\{\mathbb{E}_{\theta|\mathcal{L}}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\}\} \\
&= \mathbb{E}_{\mathcal{L}}\{(\mathbb{E}_{\theta|\mathcal{L}}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\} - \mathbb{E}_{\mathcal{L}}\{\mathbb{E}_{\theta|\mathcal{L}}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\}\})^2\} \\
&= \mathbb{E}_{\mathcal{L}}\{(\mathbb{E}_{\theta|\mathcal{L}}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\} - \mu_{\mathcal{L},\theta}(\mathbf{x}))^2\} \\
&= \mathbb{E}_{\mathcal{L}}\{\mathbb{E}_{\theta|\mathcal{L}}\{\varphi_{\mathcal{L},\theta}(\mathbf{x})\}^2\} - \mu^2_{\mathcal{L},\theta}(\mathbf{x}) \\
&= \mathbb{E}_{\mathcal{L}}\{\mathbb{E}_{\theta'|\mathcal{L}}\{\varphi_{\mathcal{L},\theta'}(\mathbf{x})\}\mathbb{E}_{\theta''|\mathcal{L}}\{\varphi_{\mathcal{L},\theta''}(\mathbf{x})\}\} - \mu^2_{\mathcal{L},\theta}(\mathbf{x}) \\
&= \mathbb{E}_{\mathcal{L},\theta',\theta''}\{\varphi_{\mathcal{L},\theta'}(\mathbf{x})\varphi_{\mathcal{L},\theta''}(\mathbf{x})\} - \mu^2_{\mathcal{L},\theta}(\mathbf{x}).
\end{aligned} \tag{4.33}$$

□

Nesta forma, $\rho(\mathbf{x})$ é interpretado como a razão entre a variância devida ao conjunto de aprendizagem e a variância total, contabilizando os efeitos aleatórios devidos tanto ao conjunto de aprendizagem quanto às perturbações aleatórias. É próximo de 1 quando a variância se deve principalmente ao conjunto de aprendizagem, produzindo assim previsões correlacionadas. Por outro lado, é próximo de 0 quando a variância se deve principalmente às perturbações aleatórias induzidas por θ, descorrelacionando assim as previsões.

4.2.3.2 Classificação

A decomposição do erro de generalização esperado de um conjunto na classificação segue diretamente dos teoremas 4.2 e 4.3. Construir um conjunto sempre reduz a variância da estimativa de probabilidade de classe $\mathbb{E}_{\mathcal{L},\theta}\{\hat{p}_{\mathcal{L},\theta}(Y = \varphi_B(\mathbf{x})\}$ (como mostrado na Equação 4.28), o que resulta em uma diminuição do erro de classificação incorreta se a estimativa esperada permanecer estritamente maior que 0,5 em um modelo aleatório.

Deficiências abordadas pelos conjuntos

Em complemento à análise formal de viés-variância realizada na seção anterior, Dietterich [2000b]identifica três razões fundamentais que explicam intuitivamente por que conjuntos geralmente funcionam melhor do que modelos únicos.

A primeira razão é estatística. Quando o conjunto de aprendizagem é muito pequeno, um algoritmo de aprendizagem normalmente pode encontrar vários modelos no espaço de hipóteses $\mathcal{H}$ que fornecem o mesmo desempenho nos dados de treinamento. Desde que as suas previsões não estejam correlacionadas, a média de vários modelos reduz o risco de escolher a hipótese errada.

The second reason is computational. Many learning algorithms rely on some greedy assumption or local search that may get stuck in local optima. As such, an ensemble made of individual models built from many different starting points may provide a better approximation of the true unknown function that any of the single models.

Finally, the third reason is representational. In most cases, for a learning set of finite size, the true function cannot be represented by any of the candidate models in $\mathcal{H}$. By combining several models in an ensemble, it may be possible to expand the space of representable functions and to better model the true function.

4.3 RANDOM FORESTS

Random forests[5] form a family of methods that consist in building an ensemble (or *forest*) of decision trees grown from a randomized variant of the tree induction algorithm (as described in Chapter 3). Decision trees are indeed ideal candidates for ensemble methods since they usually have low bias and high variance, making them very likely to benefit from the averaging process. As we will review in this section, random forests methods mostly differ from each other in the way they introduce random perturbations into the induction procedure. As highlighted in the previous section, the difficulty is to inject randomness while minimizing $\rho(\mathbf{x})$ and simultaneously maintaining a low bias in the randomized decision trees.

4.3.1 *Randomized induction algorithms*

Kwok and Carter [1990]:

> Historically, the earliest mention of ensemble of decision trees is due to Kwok and Carter [1990]. In this work, the authors empirically observe that averaging multiple decision trees with different structure consistently produces better results than any of the constituents of the ensemble. This approach however was not based on randomization nor was fully automatic: decision trees were generated by first manually selecting at the top of the tree splits that were almost as good as the optimal splits, and then expanded using the classical ID3 induction procedure.

Breiman [1994]:

> In a now famous technical report, Breiman [1994] was one of the earliest to show, both theoretically and empirically, that ag-

5 The term *random forests*, without capitals, is used to denote any ensemble of decision trees. Specific variants are referred to using their original designation, denoted with capitals. In particular, the ensemble method due to Breiman [2001] is denoted as *Random Forests*, with capitals.

A segunda razão é computacional. Muitos algoritmos de aprendizagem dependem de algumas suposições gananciosas ou de busca local que podem ficar presas em ótimos locais. Como tal, um conjunto feito de modelos individuais construídos a partir de muitos pontos de partida diferentes pode fornecer uma melhor aproximação da verdadeira função desconhecida do que qualquer um dos modelos individuais.

Finalmente, a terceira razão é representacional. Na maioria dos casos, para um conjunto de aprendizagem de tamanho finito, a função verdadeira não pode ser representada por nenhum dos modelos candidatos em $\mathcal{H}$. Ao combinar vários modelos num conjunto, pode ser possível expandir o espaço de funções representáveis e modelar melhor a função verdadeira.

4.3 florestas aleatórias

Florestas aleatórias[5] formam uma família de métodos que consiste na construção de um conjunto (ou floresta) de árvores de decisão cultivadas a partir de uma variante aleatória do algoritmo de indução de árvores (conforme descrito no Capítulo 3). As árvores de decisão são de fato candidatas ideais para métodos de conjunto, uma vez que geralmente têm baixo viés e alta variância, tornando-as muito propensas a se beneficiare m do processo de cálculo da média. Como revisaremos nesta seção, os métodos de florestas aleatórias diferem principalmente entre si na maneira como introduzem perturbações aleatórias no procedimento de indução. Conforme destacado na seção anterior, a dificuldade é injetar aleatoriedade enquanto minimiza $\rho(\mathbf{x})$ e simultaneamente mantém um viés baixo nas árvores de decisão aleatórias.

4.3.1 Algoritmos de indução randomizados

Kwok e Carter [1990]:

> Historicamente, a primeira menção ao conjunto de árvores de decisão se deve a Kwok e Carter [1990]. Neste trabalho, os autores observam empiricamente que a média de múltiplas árvores de decisão com estruturas diferentes produz consistentemente melhores resultados do que qualquer um dos constituintes do conjunto. Esta abordagem, no entanto, não foi baseada na randomização nem foi totalmente automática: as árvores de decisão foram geradas selecionando primeiro manualmente no topo da árvore as divisões que eram quase tão boas quanto as divisões ideais e depois expandidas usando o procedimento clássico de indução ID3.

Breiman [1994]:

> Num relatório técnico agora famoso, Breiman [1994]foi um dos primeiros a mostrar, tanto teórica como empiricamente, que ag-

5 O termo florestas aleatórias, sem letras maiúsculas, é usado para denotar qualquer conjunto de árvores de decisão. Variantes específicas são referidas usando sua designação original, indicada em maiúsculas. Em particular, o método ensemble devido a Breiman [2001] é denotado como Random Forests, com letras maiúsculas.

gregating multiple versions of an estimator into an ensemble can give substantial gains in accuracy. He notes and shows that the average model $\mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}\}$ has a lower expected generalization error than $\varphi_{\mathcal{L}}$. As such, *Bagging* consists in approximating $\mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}\}$ by combining models built from *bootstrap samples* [Efron, 1979] $\mathcal{L}^m$ (for $m = 1, \ldots, M$) of the learning set $\mathcal{L}$. The $\{\mathcal{L}^m\}$ form replicates of $\mathcal{L}$, each consisting of N cases $(\mathbf{x}, y)$, drawn at random but *with replacement* from $\mathcal{L}$.

Note that even though $|\mathcal{L}| = |\mathcal{L}^m| = N$, 37% of the couples $(\mathbf{x}, y)$ from $\mathcal{L}$ are on average missing in the bootstrap replicates. Indeed, after N draws with replacement, the probability of never have been selected is

$$(1 - \frac{1}{N})^N \approx \frac{1}{e} \approx 0.368. \tag{4.34}$$

When the learning algorithm is unstable (i.e., when small changes in the learning set can cause large changes in the learned models), Bagging generates individual models that are different from one bootstrap sample to another, hence making them likely to benefit from the averaging process. In some cases however, when the learning set $\mathcal{L}$ is small, subsampling 67% of the objects might lead to an increase of bias (e.g., because of a decrease in model complexity) which is too large to be compensated by a decrease of variance, hence resulting in overall poorer performance. Despite this defect, Bagging has proven to be an effective method in numerous applications, one of its strengths being that it can be used to combine any kind of models – i.e., not only decision trees.

Dietterich and Kong [1995]:

> Building upon [Kwok and Carter, 1990], Dietterich and Kong [1995] propose to randomize the choice of the best split at a given node by selecting uniformly at random one of the 20 best splits of node t. The authors empirically show in [Dietterich and Kong, 1995; Dietterich, 2000a] that randomizing in this way gives results that are slightly better than Bagging in low noise settings. Experiments show however that when noise is important, Bagging usually yield better results. From a bias-variance point of view, this method virtually does not change bias but increases variance due to randomization.

Amit et al. [1997]:

> In the context of handwritten character recognition, where the number p of input variables is typically very large, Amit et al. [1997] propose a randomized variant of the tree induction algorithm that consists in searching for the best split at each node over a random subsample of the variables.

agregar múltiplas versões de um estimador em um conjunto pode proporcionar ganhos substanciais em precisão. Ele observa e mostra que o modelo médio $\mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}\}$tem um erro de generalização esperado menor que $\varphi_{\mathcal{L}}$. Como tal, Bagging consiste em aproximar $\mathbb{E}_{\mathcal{L}}\{\varphi_{\mathcal{L}}\}$ combinando modelos construídos a partir de amostras de bootstrap [Efron, 1979] $\mathcal{L}^m$ (para $m = 1, \ldots, M$) do conjunto de aprendizagem $\mathcal{L}$. A forma $\{\mathcal{L}^m\}$ é replicada de $\mathcal{L}$, cada uma consistindo em N casos (x, y), sorteados aleatoriamente, mas com substituição de $\mathcal{L}$.

Observe que, embora $|\mathcal{L}| = |\mathcal{L}^m| = N$, 37% dos casais (x, y)de $\mathcal{L}$estão, em média, ausentes nas réplicas de bootstrap. Na verdade, após N empates com reposição, a probabilidade de nunca ter sido selecionado é

$$(1 - \frac{1}{N})^N \approx \frac{1}{e} \approx 0.368. \tag{4.34}$$

Quando o algoritmo de aprendizagem é instável (ou seja, quando pequenas mudanças no conjunto de aprendizagem podem causar grandes mudanças nos modelos aprendidos), o Bagging gera modelos individuais que são diferentes de uma amostra de bootstrap para outra, tornando-os assim propensos a se beneficiarem do processo de cálculo da média. Em alguns casos, porém, quando o conjunto de aprendizagem $\mathcal{L}$ é pequeno, a subamostragem de 67% dos objetos pode levar a um aumento de viés (por exemplo, devido a uma diminuição na complexidade do modelo), o que é muito grande será compensado por uma diminuição da variância, resultando, portanto, em um desempenho geral pior. Apesar deste defeito, o Bagging provou ser um método eficaz em inúmeras aplicações, sendo um dos seus pontos fortes o facto de poder ser utilizado para combinar qualquer tipo de modelos – ou seja, não apenas árvores de decisão.

Dietterich e Kong [1995]:

> Com base em [Kwok e Carter, 1990], Dietterich e Kong[1995] propõem randomizar a escolha da melhor divisão em um determinado nó, selecionando uniformemente e aleatoriamente uma das 20 melhores divisões do nó t. Os autores mostram empiricamente em [Dietterich e Kong, 1995; Dietterich, 2000a] que a randomização dessa forma fornece resultados ligeiramente melhores do que o Bagging em configurações de baixo ruído. As experiências mostram, no entanto, que quando o ruído é i mportante, o Bagging normalmente produz melhores resultados. Do ponto de vista do viés-variância, este método praticamente não altera o viés, mas aumenta a variância devido à randomização.

Amit et al. [1997]:

> No contexto do reconhecimento de caracteres manuscritos, onde o número p de variáveis de entrada é tipicamente muito grande, A mit et al.[1997] propõem uma variante aleatória do algoritmo de indução de árvore que consiste em procurar a melhor divisão em cada nó sobre uma subamostra aleatória das variáveis.

Denoting $K \leqslant p$ (also known as `mtry` or `max_features`) the number of variables effectively considered at each node, this variant replaces Algorithm 3.3 with the following randomized alternative:

Algorithm 4.1. *Find the best split s^* that partitions $\mathcal{L}_t$, among a random subset of $K \leqslant p$ input variables.*

function FINDBESTSPLITRANDOM($\mathcal{L}_t$, K)
  $\Delta = -\infty$
  Draw K random indices j_k from $1, \dots, p$
  for $k = 1, \dots, K$ **do**
    Find the best binary split $s^*_{j_k}$ defined on X_{j_k}
    if $\Delta i(s^*_{j_k}, t) > \Delta$ **then**
      $\Delta = \Delta i(s^*_{j_k}, t)$
      $s^* = s^*_{j_k}$
    end if
  end for
  return s^*
end function

When the output Y can be explained in several ways, this randomized algorithm generates trees that are each structurally different, yet individually good. As a result, bias usually increases only slightly, while the increased variance due to randomization can be cancelled out through averaging. The optimal trade-off between these quantities can otherwise be adjusted by tuning the value of K. As $K \to 1$, the larger the bias but the larger the variance of the individual models and hence the more effective the averaging process. Conversely, as $K \to p$, the smaller the bias but also the smaller the variance of the individual models and therefore the less beneficial the ensemble.

Ho [1998]:

Inspired from the principles of Bagging [Breiman, 1994] and random subsets of variables [Amit et al., 1997], Ho [1998] proposes with the *Random Subspace* (RS) method to build a *decision forest* whose trees are grown on random subsets of the input variables – drawn once, prior to the construction of each tree – rather than on all p variables. As empirically evaluated at several occasions [Ho, 1998; Panov and Džeroski, 2007; Louppe and Geurts, 2012], the Random Subspace method is a powerful generic ensemble method that can achieve near state-of-the-art performance on many problems. Again, the optimal trade-off between the variance due to randomization and the increase of bias can be controlled by tuning the size of the random subset.

Breiman [2001]:

In his seminal *Random Forests* (RF) paper, Breiman [2001] com-

Denotando $K \leqslant p$ (também conhecido como mtry ou max_features) o número de variáveis efetivamente consideradas em cada nó, esta variante substitui o Algoritmo 3.3 pela seguinte alternativa aleatória:

Algoritmo 4.1. Encontre a melhor divisão s^* que particiona $\mathcal{L}_t$, entre um subconjunto aleatório de $K \leqslant p$ variáveis de entrada.

1: função FindBestSplitRandom($\mathcal{L}_t$, K) 2: $\Delta = -\infty$3: Desenhe K índices aleatórios j_k de 1, . . . , p4: para $k = 1$, . . . , K faça 5: Encontre a melhor divisão binária $s^*_{j_k}$ definida em X_{j_k}6: if $\Delta i(s^*_{j_k}, t) > \Delta$ then 7: $\Delta = \Delta i(s^*_{j_k}, t)$8: $s^* = s^*_{j_k}$ 9: end if 10: end for 11: return s^*12: end function

Quando a saída Y pode ser explicada de diversas maneiras, esse algoritmo randomizado gera árvores que são estruturalmente diferentes, mas individualmente boas. Como resultado, o viés geralmente aumenta apenas ligeiramente, enquanto o aumento da variância devido à randomização pode ser cancelado através da média. O equilíbrio ideal entre essas quantidades pode ser ajustado ajustando o valor de K. Como $K \to 1$, quanto maior for o viés, mas maior será a variância dos modelos individuais e, portanto, mais eficaz será o processo de cálculo da média. Por outro lado, como $K \to p$, quanto menor for o viés, mas também menor será a variância dos modelos individuais e, portanto, menos benéfico será o conjunto.

Ho [1998]:

Inspirado nos princípios de Bagging [Breiman, 1994] e subconjuntos aleatórios de variáveis [Amit et al., 1997], Ho [1998] propõe com o método RandomSubspace (RS) construir uma floresta de decisão cujas árvores são cultivadas em subconjuntos aleatórios das variáveis de entrada – desenhadas uma vez, antes da construção de cada árvore – em vez de em todas as variáveis p. Conforme avaliado empiricamente em diversas ocasiões [Ho, 1998; Panov e Džeroski, 2007; Louppe e Geurts, 2012], o método Random Subspace é um método de conjunto genérico poderoso que pode alcançar um desempenho próximo do estado da arte em muitos problemas. Novamente, o equilíbrio ideal entre a variância devido à randomização e o aumento do viés pode ser controlado ajustando o tamanho do subconjunto aleatório.

Breiman [2001]:

Em seu artigo seminal sobre Random Forests (RF), Breiman [2001] com-

bines Bagging [Breiman, 1994] with random variable selection at each node [Amit et al., 1997]. Injecting randomness simultaneously with both strategies yields one the most effective off-the-shelf methods in machine learning, working surprisingly well for almost any kind of problems. The author empirically shows that Random Forests give results that are competitive with boosting [Freund and Schapire, 1995] and arcing algorithms [Breiman, 1996], which both are designed to reduce bias while forests focus on variance reduction.

While the original principles are due to several authors (as discussed above), Breiman is often cited as the father of forests of randomized trees. Parts of this recognition are certainly due to the pioneer theoretical analysis that has always complemented his empirical analysis of algorithms. In contrast with other authors, another reason might also be his efficient software implementation [Breiman, 2002] that was made freely available, allowing users outside of the machine learning community to quickly and easily apply Random Forests to their problems.

Cutler and Zhao [2001]:

With *Perfect Random Tree Ensembles* (PERT), Cutler and Zhao [2001] propose to grow a forest of perfectly fit decision trees in which both the (ordered) variable to split on and the discretization threshold are chosen at random. More specifically, given a node t, the split variable X_j is drawn at random using Algorithm 4.1 with $K = 1$ while the cut-point v is set midway between two randomly drawn samples using the following procedure (instead of Algorithm 3.4):

Algorithm 4.2. *Draw a random split on X_j that partitions $\mathcal{L}_t$.*

1: **function** FINDRANDOMSPLIT-PERT($\mathcal{L}_t$, X_j)
2: Draw $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2) \in \mathcal{L}_t$ such that $y_1 \neq y_2$
3: Draw α uniformly at random from $[0, 1]$
4: $v = \alpha x_{1,j} + (1 - \alpha)x_{2,j}$
5: **return** s_j^v
6: **end function**

The induction of the tree proceeds using such random splits until all nodes become pure or until it is no longer possible to draw samples of different output values.

From a practical point of view, PERT is an easily coded and a very efficient ensemble method since there is no impurity criterion to evaluate when splitting the nodes of a tree. Regarding accuracy, experimental comparisons in [Cutler and Zhao, 2001] show that PERT is often nearly as good as Random Forests [Breiman, 2001], while resulting however in random trees that

bines Bagging [Breiman, 1994]com seleção de variáveis aleatórias em cada nó [Amit et al., 1997]. Injetar aleatoriedade simultaneamente com ambas as estratégias produz um dos métodos disponíveis no mercado mais eficazes em aprendizado de máquina, funcionando surpreendentemente bem para quase todos os tipos de problemas. O autor mostra empiricamente que as Florestas Aleatórias fornecem resultados que são competitivos com o reforço [Freund e Schapire, 1995] e os algoritmos de arco[Breiman, 1996],que são projetados para reduzir o viés enquanto as florestas se concentram na redução da variância.

Embora os princípios originais sejam devidos a vários autores (conforme discutido acima), Breiman é frequentemente citado co mo o pai das florestas de árvores aleatórias. Parte desse reconhecimento se deve certamente à análise teórica pioneira que sempre complementou sua análise empírica de algoritmos. Em contraste com outros autores, outra razão também pode ser sua eficiente implementação de software [Breiman, 2002] que foi disponibilizada gratuitamente, permitindo que usuários fora da comunidade de aprendizado de máquina aplicassem rápida e facilmente RandomForests aos seus problemas.

Cutler e Zhao [2001]:

Com Perfect Random Tree Ensembles (PERT), Cutler e Zhao[2001] propõem o crescimento de uma floresta de árvores de decisão perfeitamente ajustadas nas quais tanto a variável (ordenada) a ser dividida quanto o limite de discretização são escolhidos aleatoriamente. Mais especificamente, dado um nó t, a variável de divisão X_j é sorteada aleatoriamente usando o Algoritmo 4.1 com $K = 1$ enquanto o ponto de corte v é definido no meio do caminho entre duas amostras sorteadas aleatoriamente usando o seguinte procedimento (em vez do Algoritmo 3.4):

Algoritmo 4.2. Desenhe uma divisão aleatória X_j nessas partições $\mathcal{L}_t$.

1: função FindRandomSplit-PERT($\mathcal{L}_t$, X_j) 2: Desenhe (**x1,** y_1**), (x2,** y_2**)** $\in \mathcal{L}_t$ **tal que** $y_1 \neq y_2$**3: Desenhe** α **uniformemente aleatoriamente de** $[0, 1]$**4:** $v = \alpha x_1$**,** $_j + (1 - \alpha)x_{2,j}$**5: retorno** s_j^v**6: função final**

A indução da árvore prossegue usando tais divisões aleatórias até que todos os nós se tornem puros ou até que não seja mais possível extrair amostras de diferentes valores de saída.

Do ponto de vista prático, PERT é um método de conjunto de fácil codificação e muito eficiente, uma vez que não existe nenhum critério de impureza a ser avaliado ao dividir os nós de uma árvore. Em relação à precisão, comparações experimentais em [Cutler e Zhao 2001]mostra que o PERT é muitas vezes quase tão bom quanto as Florestas Aleatórias[Breiman, 2001], ao mesmo tempo que resulta em árvores aleatórias que

are typically larger than decision trees grown with less randomization. The simplicity of the method also allows for an amenable theoretical analysis of forests of randomized trees, as carried out in [Zhao, 2000].

Geurts et al. [2006a]:

As investigated in [Wehenkel, 1997; Geurts and Wehenkel, 2000], the notoriously high variance of decision trees partly finds its origins from the high dependence of the splits with the random nature of the learning set. The authors empirically show that the variance of the optimal cut-point v (in the case of ordered input variables) may indeed be very high, even for large sample sizes. In particular, Geurts [2002] shows that cut-point variance appears to be responsible for a significant part of the generalization error of decision trees. As a way to smoothen the decision boundary, Geurts et al. [2006a] hence propose in *Extremely Randomized Trees* (ETs) to combine random variable selection [Amit et al., 1997] with random discretization thresholds. As a drop-in replacement of Algorithm 3.4, the authors propose the following simplistic but effective procedure for drawing splits at random:

Algorithm 4.3. *Draw a random split on* X_j *that partitions* $\mathcal{L}_t$.

1: **function** FINDRANDOMSPLIT-ETS($\mathcal{L}_t$, X_j)
2: $\quad \min_j = \min(\{x_{i,j} | (\mathbf{x}_i, y_i) \in \mathcal{L}_t\})$
3: $\quad \max_j = \max(\{x_{i,j} | (\mathbf{x}_i, y_i) \in \mathcal{L}_t\})$
4: $\quad$ Draw v uniformly at random from $[\min_j, \max_j[$
5: $\quad$ **return** s_j^v
6: **end function**

With respect to decomposition 4.30 of variance, extremely randomized trees can therefore be seen as a way to transfer cut-point variance from the variance term due to the learning set to the (reducible) variance term that is due to random effects.

In the special case where $K = 1$, Extremely Randomized Trees reduce to *Totally Randomized Trees*, in which both a single variable X_j and a discretization threshold v are drawn at random at each node. As a result, the structure of such trees can be learned in an unsupervised way, independently of the output variable Y. In this setting, Totally Randomized Trees are very close to Perfect Random Tree Ensembles [Cutler and Zhao, 2001] since both draw X_j and v at random. These methods are however not strictly equivalent since they do not draw the random discretization thresholds v with respect to the same probability distribution.

Rodriguez et al. [2006]:

In a different direction, Rodriguez et al. [2006] propose in *Ro-*

são tipicamente maiores do que árvores de decisão cultivadas com menos randomização. A simplicidade do método também permite uma análise teórica favorável de florestas de árvores aleatórias, conforme realizado em [Zhao, 2000].

Geurts et al. [2006a]:

Conforme investigado em [Wehenkel, 1997; Geurts e Wehenkel, 2000], a variância notoriamente alta das árvores de decisão tem origem, em parte, na alta dependência das divisões com a natureza aleatória do conjunto de aprendizagem. Os autores mostram empiricamente que a variância do ponto de corte ideal v (no caso de variáveis de entrada ordenadas) pode de fato ser muito alta, mesmo para amostras grandes. Em particular, Geurts [2002]mostra que a variância do ponto de corte parece ser responsável por uma parte significativa do erro de generalização das árvores de decisão. Como forma de suavizar o limite de decisão, Geurts et al. [2006por isso] propõem em Árvores Extremamente Randomizadas (ETs) combinar a seleção de variáveis aleatórias [Amit et al., 1997] com limites de discretização aleatória. Como substituto imediato do Algoritmo 3.4, os autores propõem o seguinte procedimento simplista, mas eficaz, para desenhar divisões aleatoriamente:

A Algoritmo 4.3. Desenhe uma divisão aleatória X_j **nessas partições** $\mathcal{L}_t$**. 1: função FindRandomSplit-ETs(**$\mathcal{L}_t$**,** X_j**) 2: min**^{j} **=** $\min(\{x_{i,j} | (\mathbf{x}_i, y_i) \in \mathcal{L}_t\})$**3: max**^{j} **=** $\max(\{x_{i,j} | (\mathbf{x}_i, y_i) \in \mathcal{L}_t\})$**4: Desenhe** v **uniformemente aleatoriamente de** $[$**min**^{j}**,max**$_j[$**5: return** s_j^v**6: função final**

Com relação à decomposição 4.30 da variância, árvores extremamente aleatórias podem, portanto, ser vistas como uma forma de transferir a variância do ponto de corte do termo de variância devido ao conjunto de aprendizagem para o termo de variância (redutível) que é devido a efeitos aleatórios.

No caso especial em que $K = 1$, Árvores Extremamente Randomizadas são reduzidas a Árvores Totalmente Randomizadas, nas quais tanto uma única variável X_j quanto um limite de discretização v são sorteados aleatoriamente em cada nó. Como resultado, a estrutura de tais árvores pode ser aprendida de forma não supervisionada, independentemente da variável de saídaY. Neste cenário, as árvores totalmente aleatórias estão muito próximas dos conjuntos de árvores aleatórias perfeitas [Cutler e Zhao, 2001]já que ambos desenham X_j e v aleatoriamente. Estes métodos, contudo, não são estritamente equivalentes, uma vez que não traçam os limiares de discretização aleatória v em relação à mesma distribuição de probabilidade.

Rodríguez et al. [2006]:

Em uma direção diferente, Rodriguez et al. [2006]propor em Ro-

tation Forests to generate randomized decision trees based on feature extraction. As in Bagging [Breiman, 1994], individual decision trees are built on bootstrap replicates $\mathcal{L}^m$ of the learning set $\mathcal{L}$. To further enhance diversity (i.e., to further decorrelate the predictions of the constituents of the ensemble), for each of the M bootstrap replicates, input variables are randomly partitioned into q subsets of $\frac{p}{q}$ variables, principal component analysis (PCA) is run separately on each subset, and a new set $\widetilde{\mathcal{L}^m}$ of p extracted input variables is constructed by pooling all principal components from the q projections. In this way, bootstrap replicates $\mathcal{L}^m$ are each independently transformed linearly into a new input space using q axis rotations. In this framework, decision trees are particularly suited because they are sensitive to changes in the input space and still can be very accurate. As reported in [Rodriguez et al., 2006; Kuncheva and Rodríguez, 2007], Rotation Forests favorably compare with other tree-based ensemble methods, yielding results that are often as good, sometimes better, than Random Forests [Breiman, 2001]. In terms of complexity however, the computational overhead due to the q axis rotations should not be overlooked when resources are limited.

4.3.2 *Illustration*

As a summary and illustrative example, let us consider a simulated toy regression problem such that

$$Y = \sum_{j=1}^{5} X_j, \tag{4.35}$$

where all input variables $X_1, \dots, X_5$ are independent random Gaussian variables of zero mean and unit variance. To simulate noise in the data, 5 additional random Gaussian input variables $X_6, \dots, X_{10}$, all independent from Y, are further appended to the learning set.

Let us compare for this problem the bias-variance decomposition of the expected generalization error of a Random Forest [Breiman, 2001] (RF) and of Extremely Randomized Trees [Geurts et al., 2006a] (ETs). For both methods, the error is estimated as derived from Theorem 4.3, using 100 randomly drawn learning sets $\mathcal{L}$ of 50 samples, on which ensembles of 10 trees are built. Their generalization error is estimated on an independent test set of 1000 samples.

As the left plot in Figure 4.4 shows, the expected generalization error additively decomposes into (squared) bias and variance terms. For small values of K, random effects are strong, leading to high bias and low variance. By contrast, for larger values of K, random effects are less important, reducing bias but increasing variance. For both methods, too small a value of K appears however to lead to a too

tação Florestas para gerar árvores de decisão aleatórias com base na extração de recursos. Como em Bagging [Breiman, 1994],árvores de decisão individuais são construídas em réplicas de bootstrap $\mathcal{L}^m$ do conjunto de aprendizagem $\mathcal{L}$. Para aumentar ainda mais a diversidade (ou seja, para descorrelacionar ainda mais as previsões dos constituintes do conjunto), para cada uma das réplicas de bootstrap M, as variáveis de entrada são particionadas aleatoriamente em subconjuntos q de variáveis $\frac{p}{q}$, a análise de componentes principais (PCA) é executada separadamente em cada subconjunto, e um novo conjunto $\mathcal{L}^m$ de variáveis de entrada extraídas p é construído agrupando todos os princípios. componentes principais das q projeções. Dessa forma, as réplicas de bootstrap $\mathcal{L}^m$ são transformadas de forma independente e linear e m um novo espaço de entrada usando q rotações de eixo. Neste contexto, as árvores de decisão são particularmente adequadas porque são sensíveis a mudanças no espaço de entrada e ainda podem ser muito precisas. Conforme relatado em [Rodriguez et al., 2006; Kuncheva e Rodríguez,2007], Rotation Forests comparam-se favoravelmente com outros métodos de conjuntos baseados em árvores, produzindo resultados que são muitas vezes tão bons, às vezes melhores, que Random Forests [Breiman, 2001]. No entanto, em termos de complexidade, a sobrecarga computacional devido às rotações dos eixos não deve ser ignorada quando os recursos são limitados.

4.3.2 Ilustração

Como exemplo resumido e ilustrativo, consideremos um problema de regressão de brinquedo simulado tal que

$$Y = \sum_{j=1}^{5} X_j, \tag{4.35}$$

onde todas as variáveis de entrada $X_1, \dots, X_5$ são variáveis gaussianas aleatórias independentes de média zero e variância unitária. Para simular ruído nos dados, 5 variáveis de entrada gaussianas aleatórias adicionais $X_6, \dots, X_{10}$, todos independentes de Y, são posteriormente anexados ao conjunto de aprendizagem.

Vamos comparar para este problema a decomposição viés-variância do erro de generalização esperado de uma Floresta Aleatória [Breiman, 2001](RF) e de Árvores Extremamente Randomizadas [Geurts et al., 2006a] (ETs). Para ambos os métodos, o erro é estimado como derivado do Teorema 4.3, usando 100 conjuntos de aprendizagem sorteados aleatoriamente $\mathcal{L}$ de 50 amostras, nos quais são construídos conjuntos de 10 árvores. Seu erro de generalização é estimado em um conjunto de t estes independente de 1.000 amostras.

Como mostra o gráfico à esquerda na Figura 4.4, o erro de generalização esperado decompõe-se aditivamente em termos de tendência (quadrado) e variância. Para valores pequenos de K, os efeitos aleatórios são fortes, levando a alto viés e baixa variância. Por outro lado, para valores maiores de K, os efeitos aleatórios são menos importantes, reduzindo o viés, mas aumentando a variância. Para ambos os métodos, um valor muito pequeno de K parece, no entanto, levar a um valor muito

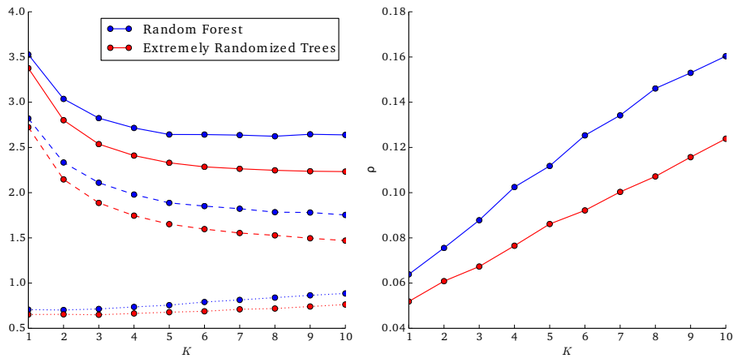


Figure 4.4: (Left) Bias-variance decomposition of the generalization error with respect to the hyper-parameter K. The total error is shown by the plain lines. Bias and variance terms are respectively shown by the dashed and dotted lines. (Right) Average correlation coefficient $\rho(\mathbf{x})$ over the predictions of two randomized trees grown from the same learning set but with different random seeds.

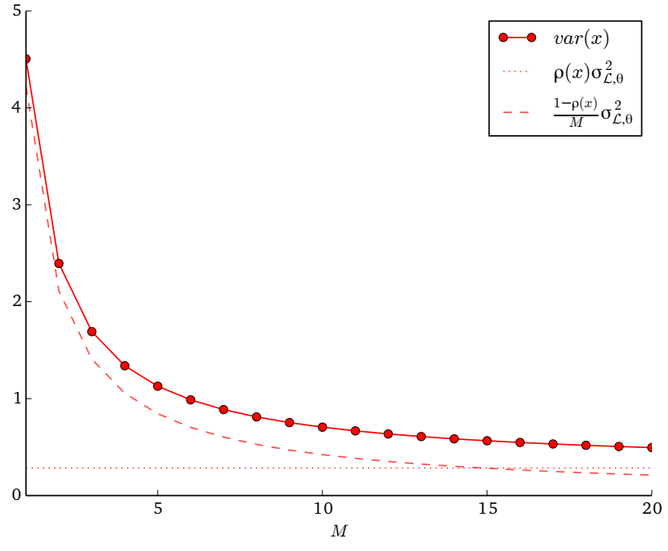


Figure 4.5: Decomposition of the variance term $\text{var}(\mathbf{x})$ with respect to the number M of trees in the ensemble.

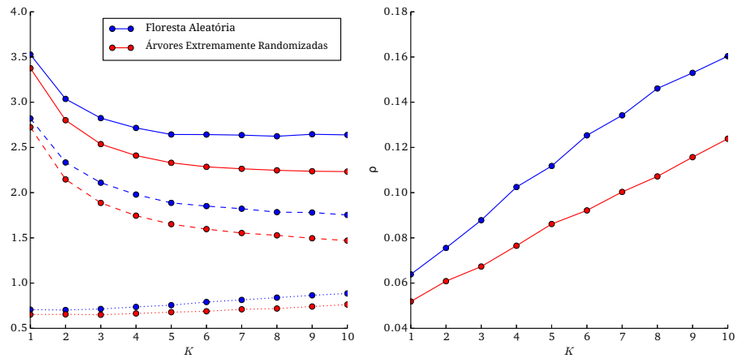


Figura 4.4: (Esquerda) Decomposição de polarização-variância do erro de generalização em relação ao hiperparâmetro K. O erro total é mostrado pelas linhas simples. Os termos de tendência e variância são mostrados respectivamente pelas linhas tracejadas e pontilhadas. (Direita) Coeficiente de correlação médio $\rho(\mathbf{x})$ sobre as previsões de duas árvores aleatórias cultivadas a partir do mesmo conjunto de aprendizagem, mas com sementes aleatórias diferentes.

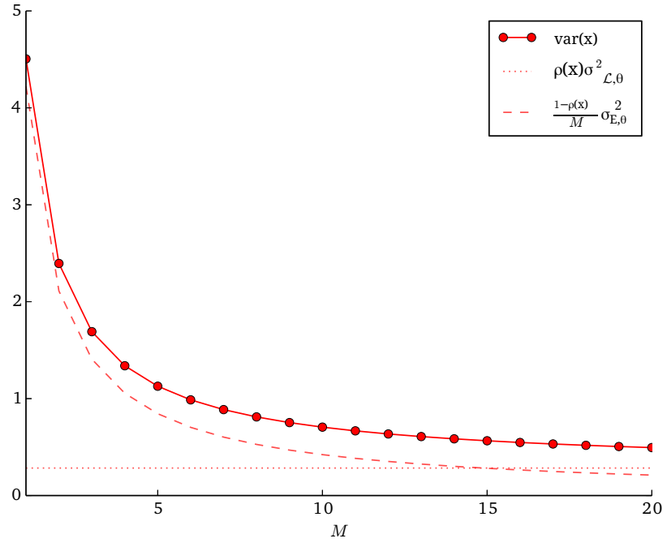


Figura 4.5: Decomposição do termo de variância $\text{var}(\mathbf{x})$ em relação ao número M de árvores no conjunto.

large increase of bias, for which averaging is not able to compensate. For RF, the optimal trade-off is at $K = 8$, which indicates that bagging and random variable selection are here complementary with regard to the level of randomness injected into the trees. Indeed, using bagging only (at $K = 10$) for randomizing the construction of the trees yield results that are slightly worse than when both techniques are combined. For ETs, the optimal trade-off is at $K = 10$, suggesting that random thresholds v provide by themselves enough randomization on this problem.

The right plot of Figure 4.4 illustrates the (averaged) correlation coefficient $\rho(\mathbf{x})$ between the predictions of two randomized trees grown on the same learning set. As expected, the smaller K, the stronger the random effects, therefore the less correlated the predictions and the more variance can be reduced from averaging. The plot also confirms that ETs are inherently less correlated than trees built in RF, which is not surprising given the fact that the former method randomizes the choice of the discretization threshold while the latter does not. In cases where such a randomization does not induce too large an increase of bias, as in this problem, ETs are therefore expected to yield better results than RF. (The choice of the optimal randomization strategy is however highly dependent on the problem and no general conclusion should be drawn from this toy regression problem.)

As confirmed by Figure 4.5 for RF, variance also additively decomposes into

$$\text{var}(\mathbf{x}) = \rho(\mathbf{x})\sigma^2_{\mathcal{L},\theta}(\mathbf{x}) + \frac{1-\rho(\mathbf{x})}{M}\sigma^2_{\mathcal{L},\theta}(\mathbf{x}). \tag{4.36}$$

The first term is the variance due to the learning set $\mathcal{L}$ and remains constant as the number M of trees increases. The second term is the variance due to random effects and decreases as M increases. Of particular interest is variance at $M = 1$, which corresponds to the variance of a single decision tree. As the figure clearly shows, averaging several decision trees into an ensemble allows to significantly reduce this quantity. At the limit, when $M \to \infty$, variance tends to $\rho(\mathbf{x})\sigma^2_{\mathcal{L},\theta}(\mathbf{x})$, as shown by the dotted line and as expected from Theorem 4.3.

4.4 PROPERTIES AND FEATURES

4.4.1 *Out-of-bag estimates*

An interesting feature of ensemble methods that construct models on bootstrap samples, like Bagging or Random Forests, is the built-in possibility of using the left-out samples $\mathcal{L} \setminus \mathcal{L}^m$ to form estimates of important statistics. In the case of generalization error, the *out-of-bag* estimate at $(\mathbf{x}_i, y_i)$ consists in evaluating the prediction of the

grande aumento de viés, que a média não é capaz de compensar. Para RF, o trade-off ideal está em $K = 8$, o que indica que o bagging e a seleção de variáveis aleatórias são aqui complementares em relação ao nível de aleatoriedade injetado nas árvores. Na verdade, usar apenas bagging (em $K = 10$) para randomizar a construção das árvores produz resultados ligeiramente piores do que quando ambas as técnicas são combinadas. Para ETs, o trade-off ideal está em $K = 10$, sugerindo que limites aleatórios v fornecem por si só randomização suficiente neste problema.

O gráfico à direita da Figura 4.4 ilustra o coeficiente de correlação (média) $\rho(\mathbf{x})$ entre as previsões de duas árvores aleatórias cultivadas no mesmo conjunto de aprendizagem. Como esperado, quanto menor K, mais fortes serão os efeitos aleatórios, portanto, menos correlacionadas serão as previsões e mais variância poderá ser reduzida a partir da média. O gráfico também confirma que as ETs são inerentemente menos correlacionadas do que as árvores construídas em RF, o que não é surpreendente dado o fato de que o primeiro método randomiza a escolha do limiar de discretização enquanto o último não. Nos casos em que tal randomização não induz um aumento muito grande de viés, como neste problema, espera-se que os ETs produzam melhores resultados do que os FR. (A escolha da estratégia de randomização ideal é, no entanto, altamente dependente do problema e nenhuma conclusão geral deve ser tirada deste problema de regressão do brinquedo.)

Conforme confirmado pela Figura 4.5 para RF, a variância também se decompõe aditivamente em

$$\text{var}(\mathbf{x}) = \rho(\mathbf{x})\sigma^2_{\mathcal{L},\theta}(\mathbf{x}) + \frac{1-\rho(\mathbf{x})}{M}\sigma^2_{\mathcal{L},\theta}(\mathbf{x}). \tag{4.36}$$

O primeiro termo é a variância devido ao conjunto de aprendizagem $\mathcal{L}$ e permanece constante à medida que o número M de árvores aumenta. O segundo termo é a variância devido a efeitos aleatórios e diminui à medida que M aumenta. De particular interesse é a variância em $M = 1$, que corresponde à variância de uma única árvore de decisão. Como a figura mostra claramente, a média de várias árvores de decisão num conjunto permite reduzir significativamente esta quantidade. No limite, quando $M \to \infty$, a variância tende a $\rho(\mathbf{x})\sigma^2_{\mathcal{L},\theta}(\mathbf{x})$, como mostrado pelas linhas pontilhadas e como esperado do Teorema 4.3.

4.4 propriedades e recursos

4.4.1 Estimativas prontas

Uma característica interessante dos métodos de conjunto que constroem modelos em amostras de bootstrap, como Bagging ou Random Forests, é a possibilidade incorporada de usar amostras deixadas de fora $\mathcal{L} \setminus \mathcal{L}^m$ para formar estimativas de estatísticas importantes. No caso de erro de generalização, a estimativa out-of-bag em $(\mathbf{x}_i, y_i)$ consiste em avaliar a previsão do

ensemble using only the individual models $\varphi_{\mathcal{L}^m}$ whose bootstrap samples $\mathcal{L}^m$ did not include $(\mathbf{x}_i, y_i)$. That is, in regression,

$$\widehat{\text{Err}}^{\text{OOB}}(\psi_{\mathcal{L}}) = \frac{1}{N} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{L}} L\left(\frac{1}{M^{-i}} \sum_{l=1}^{M^{-i}} \varphi_{\mathcal{L}^{m_{k_l}}}(\mathbf{x}_i), y_i\right), \tag{4.37}$$

where $m_{k_1}, \ldots, m_{k_{M^{-i}}}$ denote the indices of M^{-i} the trees that have been built from bootstrap replicates that do not include $(\mathbf{x}_i, y_i)$. For classification, the out-of-bag estimate of the generalization error is similar to Equation 4.37, except that the out-of-bag average prediction is replaced with the class which is the most likely, as computed from the out-of-bag class probability estimates.

Out-of-bag estimates provide accurate estimates of the generalization error of the ensemble, often yielding statistics that are as good or even more precise than K-fold cross-validation estimates [Wolpert and Macready, 1999]. In practice, out-of-bag estimates also constitute a computationally efficient alternative to K-fold cross-validation, reducing to M the number of invocations of the learning algorithm, instead of otherwise having to build $K \times M$ base models.

While out-of-bag estimates constitute an helpful tool, their benefits should however be put in balance with the potential decrease of accuracy that the use of bootstrap replicates may induce. As shown experimentally in [Louppe and Geurts, 2012], bootstrapping is in fact rarely crucial for random forests to obtain good accuracy. On the contrary, not using bootstrap samples usually yield better results.

4.4.2 *Variable importances*

In most machine learning tasks, the goal is not only to find the most accurate model of the response but also to identify which of the input variables are the most important to make the predictions, e.g., in order to lead to a deeper understanding of the problem under study.

In this context, random forests offer several mechanisms for assessing the *importance* of an input variable, and therefore enhance the interpretability of the model. These are the object of Chapter 6, in which we study variable importance measures and develop original contributions to further improve their understanding.

4.4.3 *Proximity measures*

Another helpful built-in feature of tree-based ensemble methods is the *proximity measure* [Breiman, 2002] between two sample points. Formally, the proximity between $(\mathbf{x}_1, y_1)$ and $(\mathbf{x}_2, y_2)$ is defined as the number of times both samples reach the same leaf t within each de-

conjunto usando apenas os modelos individuais $\varphi_{\mathcal{L}^m}$ cujas amostras de bootstrap $\mathcal{L}^m$ não incluíram (x_i, y_i). Ou seja, na regressão,

$$\widehat{\text{Err}}^{\text{OOB}}(\psi_{\mathcal{L}}) = \frac{1}{N} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{L}} L\left(\frac{1}{M^{-i}} \sum_{l=1}^{M^{-i}} \varphi_{\mathcal{L}^{m_{k_l}}}(\mathbf{x}_i), y_i\right), \tag{4.37}$$

onde $m_{k_1}, \ldots, m_{k_{M^{-i}}}$ denota os índices das M^{-i} árvores que foram construídas a partir de réplicas de bootstrap que não incluem (x_i, y_i). Para classificação, a estimativa out-of-bag do erro de generalização é semelhante à Equação 4.37, exceto que a previsão média out-of-bag é substituída pela classe que é mais provável, conforme calculada a partir das estimativas de probabilidade da classe out-of-bag.

Estimativas prontas fornecem estimativas precisas do erro de generalização do conjunto, muitas vezes produzindo estatísticas que são tão boas ou até mais precisas do que estimativas de validação cruzada K-fold [Wolpert e Macready, 1999]. Na prática, estimativas prontas também constituem uma alternativa computacionalmente eficiente à validação cruzada K-fold, reduzindo para M o número de invocações do algoritmo de aprendizagem, em vez de ter que construir modelos base $K \times M$.

Embora as estimativas prontas constituam uma ferramenta útil, os seus benefícios devem, no entanto, ser equilibrados com a potencial diminuição da precisão que o uso de réplicas de bootstrap pode induzir. Conforme mostrado experimentalmente em [Louppe e Geurts, 2012], o bootstrapping raramente é crucial para que florestas aleatórias obtenha m boa precisão. Pelo contrário, não usar amostras de bootstrap geralmente produz melhores resultados.

4.4.2 Importâncias variáveis

Na maioria das tarefas de aprendizagem automática, o objetivo não é apenas encontrar o modelo mais preciso da resposta, mas também identificar quais das variáveis de entrada são as mais importantes para fazer as previsões, por exemplo, a fim de levar a uma compreensão mais profunda do problema em estudo.

Neste contexto, as florestas aleatórias oferecem vários mecanismos para avaliar a importância de uma variável de entrada e, portanto, melhoram a interpretabilidade do modelo. Estas são o objeto do Capítulo 6, no qual estudamos medidas de importância variável e desenvolvemos contribuições originais para melhorar ainda mais a sua compreensão.

4.4.3 Medidas de proximidade

Outro recurso integrado útil dos métodos de conjunto baseados em árvore é a medida de proximidade [Breiman, 2002] entre dois pontos de amostra. Formalmente, a proximidade entre $(x1, y_1)$ e $(x2, y_2)$ é definida como o número de vezes que ambas as amostras alcançam a mesma folha t dentro de cada de-

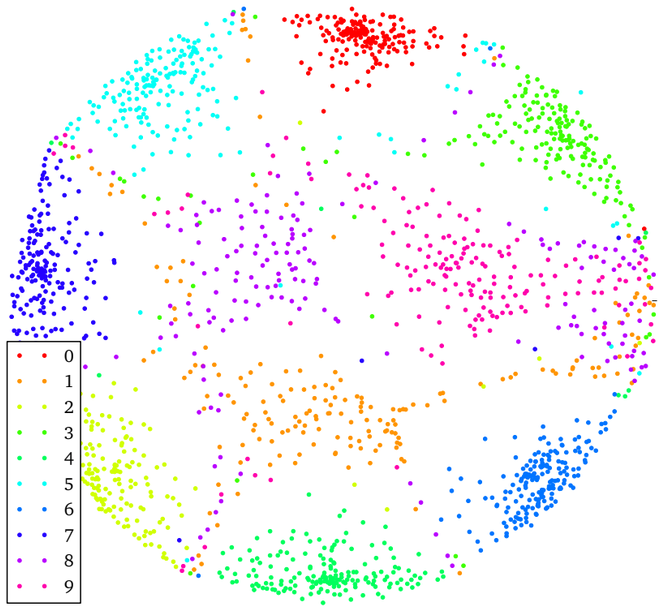


Figure 4.6: Proximity plot for a 10-class handwritten digit classification task.

cision tree, normalized by the number of trees in the forest. That is,

$$\text{proximity}(\mathbf{x}_1, \mathbf{x}_2) = \frac{1}{M} \sum_{m=1}^{M} \sum_{t \in \widetilde{\varphi}_{\mathcal{L},\theta_m}} \mathbb{1}(\mathbf{x}_1, \mathbf{x}_2 \in \mathcal{X}_t) \tag{4.38}$$

where $\widetilde{\varphi}_{\mathcal{L},\theta_m}$ denotes the set of terminal nodes in $\varphi_{\mathcal{L},\theta_m}$. The idea is that the proximity measure gives an indication of how close the samples are in the eyes of the random forest [Hastie et al., 2005], even if the data is high-dimensional or involves mixed input variables. When proximity is close to 1, samples propagate into the same leaves and are therefore similar according to the forest. On the other hand, when it is close to 0, samples reach different leaves, suggesting that they are structurally different from each other. The proximity measure depends on both the depth and the number of trees in the forest. When trees are shallow, samples are more likely to end up in the same leaves than when trees are grown more deeply, thereby impacting on the spread of the measure. Likewise, the more trees in the forest, the smoother the measure since the larger the number $M + 1$ of values the proximity measure can take.

For exploratory purposes, the $N \times N$ proximity matrix P such that

$$P_{ij} = \text{proximity}(\mathbf{x}_i, \mathbf{x}_j), \tag{4.39}$$

can be used to visually represent how samples are close together with respect to the random forest. Using $1 - P$ as a dissimilarity matrix, the

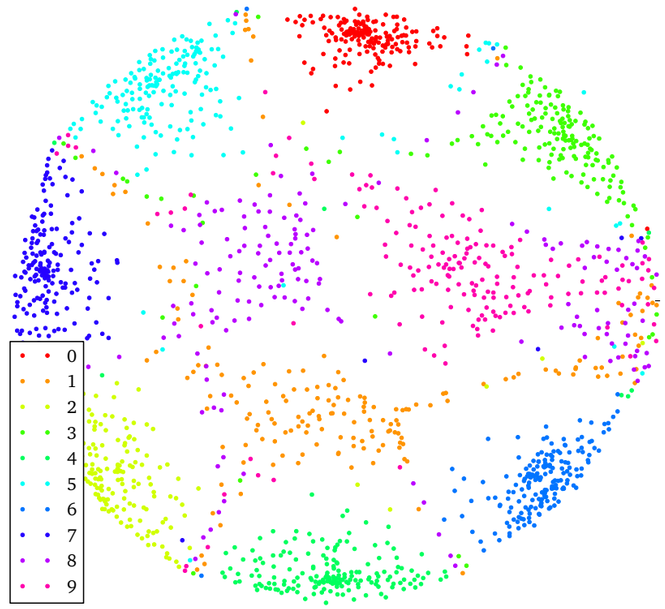


Figura 4.6: Gráfico de proximidade para uma tarefa de classificação de dígitos manuscritos de 10 classes.

árvore de decisão, normalizada pelo número de árvores na floresta. Aquilo é,

$$\text{proximity}(\mathbf{x}_1, \mathbf{x}_2) = \frac{1}{M} \sum_{m=1}^{M} \sum_{t \in \widetilde{\varphi}_{\mathcal{L},\theta_m}} \mathbb{1}(\mathbf{x}_1, \mathbf{x}_2 \in \mathcal{X}_t) \tag{4.38}$$

onde $\widetilde{\varphi}_{\mathcal{L},\theta_m}$ denota o conjunto de nós terminais em $\varphi_{\mathcal{L},\theta_m}$. A ideia é que a medida de proximidade dê uma indicação de quão próximas as amostras estão aos olhos da floresta aleatória [Hastie et al., 2005],mesmo que os dados sejam de alta dimensão ou envolvam variáveis de entrada mistas. Quando a proximidade é próxima de 1, as amostras se propagam nas mesmas folhas e, portanto, são semelhantes de acordo com a floresta. Por outro lado, quando está próximo de 0, as amostras atingem folhas diferentes, sugerindo que são estruturalmente diferentes umas das outras. A medida de proximidade depende tanto da profundidade quanto do número de árvores na floresta. Quando as árvores são rasas, é mais provável que as amostras acabem nas mesmas folhas do que quando as árvores crescem mais profundamente, impactando assim na difusão da medida. Da mesma forma, quanto mais árvores houver na floresta, mais suave será a medida, pois maior será o número $M + 1$de valores que a medida de proximidade pode assumir.

Para fins exploratórios, a $N \times N$ matriz de proximidade P tal que

$$P_{ij} = \text{proximity}(\mathbf{x}_i, \mathbf{x}_j), \tag{4.39}$$

pode ser usado para representar visualmente como as amostras estão próximas em relação à floresta aleatória. Usando $1 - P$ como matriz de dissimilaridade, o

level of similarity between individual samples can be visualized e.g., by projecting them on a d-dimensional space (e.g., on a plane) such that the distances between any pair of samples in that space correspond as best as possible to the dissimilarities in $1 - P$. As an illustrative example, Figure 4.6 represents the proximity matrix learned for a 10-class handwritten digit classification task, as projected on a plane using Multidimensional Scaling [Kruskal, 1964]. Samples from a same class form identifiable clusters, which suggests that they share similar structure (since they end up in the same leaves). The figure also highlights classes for which the random forest makes errors. In this case, digits 1 and 8 are the more dispersed, suggesting high within-class variance, but also overlap the most with samples of other classes, indicating that the random forest fails to identify the true class for these samples.

Additionally, proximity measures can be used for identifying outliers within a learning set $\mathcal{L}$. A sample $\mathbf{x}$ can be considered as an outlier if its average proximity with respect to all other samples is small, which indeed indicates that $\mathbf{x}$ is structurally different from the other samples (since they do not end up in the same leaves). In the same way, in classification, within-class outliers can be identified by computing the average proximity of a sample with respect to all other samples of the same class. Conversely, proximity measures can be used for identifying class prototypes, by considering samples whose proximity with all other samples of the same class is the largest.

An alternative formulation of proximity measures is to consider a random forest as a mapping $\phi : \mathcal{X} \mapsto \mathcal{X}'$ which transforms a sample $\mathbf{x}$ into the (one-hot-encoded) indices of the leaves it ends up in. That is, $\mathbf{x}'_t$ is 1 for all leaves t of the forest in which $\mathbf{x}$ falls in, and 0 for all the others:

$$\phi(\mathbf{x}) = \Big(1(\mathbf{x} \in \mathcal{X}_{t_1}), \ldots, 1(\mathbf{x} \in \mathcal{X}_{t_L})\Big) \tag{4.40}$$

where $t_1, \ldots, t_L \in \widetilde{\psi}$ denote the leafs of all M trees in the forest ψ. In this formalism, the proximity between $\mathbf{x}_1$ and $\mathbf{x}_2$ corresponds to a *kernel* [Scholkopf and Smola, 2001], that can be defined as the normalized dot product of the samples, as represented in $\mathcal{X}'$:

$$\text{proximity}(\mathbf{x}_1, \mathbf{x}_2) = \frac{1}{M}\phi(\mathbf{x}_1) \cdot \phi(\mathbf{x}_2) \tag{4.41}$$

Interestingly, ϕ provides a non-linear transformation to a sparse very high-dimensional space, taking somehow into account the structure of the problem. If two samples are structurally similar, then they will end up in the same leafs and their representations in the projected space will be close, even if they may in fact appear quite dissimilar in the original space.

In close connection, [Geurts et al., 2006a] show that a regression tree φ can be expressed as a kernel-based model by formulating the

O nível de similaridade entre amostras individuais pode ser visualizado, por exemplo, projetando-as em um espaço dimensional d (por exemplo, em um plano) de modo que as distâncias entre qualquer par de amostras nesse espaço correspondam da melhor forma possível às dissimilaridades em $1 - P$. Como exemplo ilustrativo, a Figura 4.6 representa a matriz de proximidade aprendida para uma tarefa de classificação de dígitos manuscritos de 10 classes, projetada em um plano usando Escala Multidimensional [Kruskal, 1964]. Amostras de uma mesma classe for mam clusters identificáveis, o que sugere que compartilham estrutura semelhante (uma vez que terminam nas mesmas folhas). A figura ta mbém destaca classes para as quais a floresta aleatória comete erros. Nesse caso, os dígitos 1 e 8 são os mais dispersos, sugerindo alta variância dentro da classe, mas também são os que mais se sobrepõem a amostras de outras classes, indicando que a floresta aleatória não consegue identificar a verdadeira classe para essas amostras.

Além disso, medidas de proximidade podem ser usadas para identificar valores discrepantes dentro de um conjunto de aprendizagem $\mathcal{L}$. Uma a mostra x pode ser considerada um outlier se a sua proximidade média em relação a todas as outras amostras for pequena, o que de facto indica que x é estruturalmente diferente das outras amostras (uma vez que não terminam nas mesmas folhas). Da mesma forma, na classificação, os outliers dentro da classe podem ser identificados calculando a proximidade média de uma amostra em relação a todas as outras amostras da mesma classe. Por outro l ado, medidas de proximidade podem ser utilizadas para identificar protótipos de classes, considerando amostras cuja proximidade com todas as outras amostras da mesma classe seja maior.

Uma formulação alternativa de medidas de proximidade é considerar um mapeamento aleatório de florestas $\phi : \mathcal{X} \mapsto \mathcal{X}'$ que transforma uma amostra x nos índices (codificados em um ponto) das folhas em que ela termina. Ou seja, x'_t é 1 para todas as folhas t da floresta em que x cai, e 0 para todas as outras:

$$\phi(\mathbf{x}) = \Big(1(\mathbf{x} \in \mathcal{X}_{t_1}), \ldots, 1(\mathbf{x} \in \mathcal{X}_{t_L})\Big) \tag{4.40}$$

$\widetilde{\text{o}}$nde $t_1, \ldots, t_L \in \psi$ denota as folhas de todas as M árvores da florestaψ. Neste formalismo, a proximidade entre x_1 e x_2 corresponde a um kernel [Scholkopf e Smola, 2001], que pode ser definido como o produto escalar normalizado das amostras, conforme representado em $\mathcal{X}'$:

$$\text{proximity}(\mathbf{x}_1, \mathbf{x}_2) = \frac{1}{M}\phi(\mathbf{x}_1) \cdot \phi(\mathbf{x}_2) \tag{4.41}$$

Curiosamente, ϕ fornece uma transformação não linear para um espaço esparso de dimensões muito altas, levando de alguma forma em consideração a estrutura do problema. Se duas amostras forem estruturalmente semelhantes, então terminarão nas mesmas folhas e as suas representações no espaço projectado serão próximas, mesmo que possam de facto parecer bastante diferentes no espaço original.

Em estreita conexão, [Geurts et al., 2006a] mostra que uma árvore de regressão φpode ser expressa como um modelo baseado em kernel, formulando o

prediction $\varphi(\mathbf{x})$ as a scalar product over the input space defined by the normalized characteristic function ϕ' of the leaf nodes. That is,

$$\varphi(\mathbf{x}) = \sum_{(\mathbf{x}_i, y_i) \in \mathcal{L}} y_i K_\varphi(\mathbf{x}_i, \mathbf{x}) \tag{4.42}$$

where

$$K_\varphi(\mathbf{x}_i, \mathbf{x}) = \phi'(\mathbf{x}_i) \cdot \phi'(\mathbf{x}), \tag{4.43}$$

$$\phi'(\mathbf{x}) = \Big(\frac{1(\mathbf{x} \in \mathcal{X}_1)}{\sqrt{N_1}}, \dots, \frac{1(\mathbf{x} \in \mathcal{X}_{t_L})}{\sqrt{N_{t_L}}} \Big), \tag{4.44}$$

and where $t_1, \dots, t_L \in \widetilde{\varphi}$ denote the leafs of φ. Likewise, the formulation can be extended to an ensemble ψ of M decision trees by defining K_ψ as the average kernel over K_{φ_m} (for $m = 1, \dots, M$).

From a more practical point of view, such forest-based transforms ϕ and ϕ' have proven to be helpful and efficient embeddings, e.g., when combined with linear methods or support vector machines [Moosmann et al., 2006; Marée et al., 2013]. In particular, they find useful applications in computer vision for transforming the raw pixel input space into a new feature space hopefully capturing structures and patterns in images.

4.4.4 *Missing data*

Because of practical limitations, physical constraints or for privacy reasons, real-world data are often imperfect, erroneous or incomplete. In particular, most machine learning algorithms are often not applicable on data containing missing values because they implicitly assume an ideal scenario in which all values are known for all input variables. Fortunately, random forests offer several mechanisms for dealing with this issue.

Ternary decision trees.

The simplest strategy is to explicitly model missing values in the structure of decision trees by considering ternary splits instead of binary splits. That is, partition t into t_L, t_R and t_M, such that t_L and t_R are defined from a binary split $s_j^\nu : X_j \leqslant \nu$, for all node samples where the value of X_j is known, and such that t_M contain all node samples for which the value of X_j is missing.

Propagate in both child nodes

An alternative strategy is to propagate samples for which the value of the split variable is missing into both the left and right child nodes. Accordingly, samples going into both child nodes should be re-weighted by half their sample weight (see Chapter 5), so that they are not unfairly taken into account more than the other samples.

previsão $\varphi(\mathbf{x})$ como um produto escalar sobre o espaço de entrada definido pela função característica normalizada ϕ' dos nós folha. Aquilo é,

$$\varphi(\mathbf{x}) = \sum_{(\mathbf{x}_i, y_i) \in \mathcal{L}} y_i K_\varphi(\mathbf{x}_i, \mathbf{x}) \tag{4.42}$$

onde

$$K_\varphi(\mathbf{x}_i, \mathbf{x}) = \phi'(\mathbf{x}_i) \cdot \phi'(\mathbf{x}), \tag{4.43}$$

$$\phi'(\mathbf{x}) = \Big(\frac{1(\mathbf{x} \in \mathcal{X}_1)}{\sqrt{N_1}}, \dots, \frac{1(\mathbf{x} \in \mathcal{X}_{t_L})}{\sqrt{N_{t_L}}} \Big), \tag{4.44}$$

e onde $t_1, \dots, t_L \in \widetilde{\varphi}$ denota as folhas de φ. Da mesma forma, a formulação pode ser estendida a um conjunto ψde M árvores de decisão definindo K_ψ como o kernel médio sobre K_{φ_m}(para $m = 1,..., M$).

De um ponto de vista mais prático, tais transformações baseadas em florestaϕ e ϕ' provaram ser incorporações úteis e eficientes, por exemplo, quando combinadas com métodos lineares ou máquinas de vetores de suporte [Moosmann et al., 2006; Marée et al., 2013]. Em particular, eles encontram aplicações úteis em visão computacional para transformar o espaço de entrada de pixels brutos em um novo espaço de recursos, capturando estruturas e padrões em imagens.

4.4.4 Dados faltantes

Devido a limitações práticas, restrições físicas ou por razões de privacidade, os dados do mundo real são frequentemente imperfeitos, erróneos ou incompletos. Em particular, a maioria dos algoritmos de aprendizagem automática muitas vezes não são aplicáveis em dados que contêm valores em falta porque assumem implicitamente um cenário i deal em que todos os valores são conhecidos para todas as variáveis de entrada. Felizmente, as florestas aleatórias oferecem vários mecanismos para lidar com este problema.

Árvores de decisão ternárias.

A estratégia mais simples é modelar explicitamente os valores ausentes na estrutura das árvores de decisão, considerando divisões ternárias em vez de divisões binárias. Ou seja, particione t em t_L, t_R e t_M, de modo que t_L e t_R sejam definidos a partir de uma divisão binária $s_j^\nu : X_j \leqslant \nu$, para todas as amostras de nós onde o valor de X_j é conhecido, e de modo que $t_M$contenha todas as amostras de nós para as quais o valor de X_j está faltando.

Propague em ambos os nós filhos

Uma estratégia alternativa é propagar amostras para as quais o valor da variável de divisão está faltando nos nós filhos esquerdo e direito. Conseqüentemente, as amostras que vão para ambos os nós filhos devem ser reponderadas pela metade do seu peso amostral (ver Capítulo 5), para que não sejam injustamente levadas em conta mais do que as outras amostras.

Imputation.

Finally, random forests also offer several mechanisms for imputing missing values. A simple approach, due to [Breiman, 2002], consists first in filling missing values with a rough and inaccurate approximation (e.g., the median). Then build a random forest on the completed data and update the missing values of each sample by the weighted mean value over the samples that are the closest (as defined by the proximity measure). The procedure is then repeated until convergence, typically after 4 to 6 iterations.

Alternatively, missing data imputation can be considered as a supervised learning problem in itself, where the response variable is the input variable for which values are missing. As such, the MissForest algorithm [Stekhoven and Bühlmann, 2012] consists in iteratively building a random forest on the observed parts of the data in order to predict the missing values for a given variable.

4.5 CONSISTENCY

Despite their extensive use in practice, excellent performance and relative algorithmic simplicity, the mathematical mechanisms that drive random forests are still not well understood. More specifically, the fundamental theoretical question of the consistency (see definitions 2.8 and 2.9) of random forests, i.e., whether convergence towards an optimal model is guaranteed provided an infinitely large learning set, remains an open and difficult problem. In this section, we review theoretical works that have investigated simplified versions of the algorithm, for which the construction procedure is often made data-independent, hence making the theoretical analysis typically more tractable. With the hope that results obtained for these simplified models will provide insights on the mathematical properties of the actual algorithm, the long-term objective of this line of research is usually to prove the consistency of the original Random Forest algorithm [Breiman, 2001], hence bridging the gap between theory and practice.

Breiman et al. [1984]:

Single decision trees are proved to be consistent, both in regression and classification.

Note that these results do not extend to the Random Forest algorithm for the following reasons:

- In single decision trees, the number of samples in terminal nodes is let to become large, while trees in random forests are usually fully developed;

Imputação.

Finalmente, as florestas aleatórias também oferecem vários mecanismos para imputar valores faltantes. Uma abordagem simples, devida a [Breiman, 2002],consiste primeiro em preencher os valores faltantes com uma aproximação grosseira e imprecisa (por exemplo, a mediana). Em seguida, construa uma floresta aleatória com base nos dados concluídos e atualize os valores ausentes de cada amostra pelo valor médio ponderado sobre as amostras mais próximas (conforme definido pela medida de proximidade). O procedimento é então repetido até a convergência, normalmente após 4 a 6 iterações.

Alternativamente, a imputação de dados faltantes pode ser considerada como um problema de aprendizagem supervisionada em si, onde a variável resposta é a variável de entrada para a qual faltam valores. Como tal, o algoritmo MissForest [Stekhoven e Bühlmann, 2012] consiste em construir iterativamente uma floresta aleatória nas partes observadas dos dados, a fim de prever os valores faltantes para uma determinada variável.

4,5 consistência

Apesar de seu amplo uso na prática, excelente desempenho e relativa simplicidade algorítmica, os mecanismos matemáticos que orientam florestas aleatórias ainda não são bem compreendidos. Mais especificamente, a questão teórica fundamental da consistência (ver definições 2.8 e 2.9) de florestas aleatórias, isto é, se a convergência para um modelo ideal é garantida desde que haja um conjunto de aprendizagem infinitamente grande, permanece um problema aberto e difícil. Nesta seção, revisamos trabalhos teóricos que investigaram versões simplificadas do algoritmo, para as quais o procedimento de construção é muitas vezes independente dos dados, tornando a análise teórica tipicamente mais tratável. Com a esperança de que os resultados obtidos para estes modelos simplificados forneçam insights sobre as propriedades matemáticas do algoritmo real, o objetivo de longo prazo desta linha de pesquisa é geralmente provar a consistência do algoritmo Random Forest original [Breiman, 2001], preenchendo assim a lacuna entre a teoria e a prática.

Breiman et al. [1984]:

As árvores de decisão única provaram ser consistentes, tanto na regressão como na classificação.

Observe que esses resultados não se estendem ao algoritmo Rando m Forest pelas seguintes razões:

- Em árvores de decisão única, o número de amostras em nós terminais pode se tornar grande, enquanto árvores em florestas aleatórias geralmente estão totalmente desenvolvidas;

- Single decision trees do not make use of bootstrap sampling;
- The splitting strategy in single decision trees consists in selecting the split that maximizes the Gini criterion. By contrast, in random forests, the splitting strategy is randomized.

Zhao [2000]:

One of the earliest works studying the consistency of ensembles of randomized trees is due to Zhao [2000]. In classification, the author conjectures that PERT is (weakly) consistent, but establishes its strong consistency (Theorem 4.4.2) provided the construction of the trees stops early. More specifically, strong consistency is guaranteed provided:

(a) Trees are grown infinitely deeply while forming leaves with infinitely many node samples, hence making empirical class proportions in terminal nodes converge towards their theoretical counterparts. This is guaranteed, e.g., by stopping the construction when $N_t < N_{min}^{(N)}$, such that $N_{min}^{(N)} \to 0$ and $N \times N_{min}^{(N)} \to \infty$ as $N \to \infty$ (where $N_{min}^{(N)}$ is the value of the N_{min} parameter for a forest grown on a learning set of size N);

(b) The posterior class probabilities induced by the ensemble are all continuous in the input space $\mathcal{X}$.

Given the close formulations of the methods, results from [Zhao, 2000] regarding PERT extend to Extremely Randomized Trees provided the posterior class probabilities are continuous. In Appendix F of [Geurts et al., 2006a], Extremely Randomizes Trees (for $K = 1$ and $M \to \infty$) are shown to be a continuous piecewise multilinear function of its arguments, which should therefore suffice to establish the strong consistency of the method when trees are built totally at random.

Breiman [2004]:

In this work, consistency is studied for a simplified variant of the Random Forest algorithm, assuming (i) no bootstrap sampling, (ii) that variables are selected as split variables with probability $p(m)$ (for $m = 1, \dots, p$), (iii) that splits on relevant variables are set at the midpoint of the values of the selected variable, (iv) that splits on irrelevant variables are set at random points along the values of the selected variable and (v) that trees are balanced. Under these assumptions, it can be shown that this variant reduces to an (adaptive) nearest neighbor algorithm [Lin and Jeon, 2002], for which (weak) consistency conditions are met (both in regression and classification). Additionally, this work studies the bias-variance decomposition of this

- Árvores de decisão única não utilizam amostragem bootstrap; - A estratégia de divisão em árvores de decisão única consiste em selecionar a divisão que maximiza o critério de Gini. Em contraste, em florestas aleatórias, a estratégia de divisão é aleatória.

Zhao [2000]:

Um dos primeiros trabalhos estudando a consistência de conjuntos de árvores aleatórias é devido a Zhao [2000]. Na classificação, o autor conjectura que o PERT é (fracamente) consistente, mas estabelece a sua consistência forte (Teorema 4.4.2) desde que a construção das árvores pare precocemente. Mais especificamente, uma consistência forte é garantida desde que:

(a) As árvores crescem infinitamente profundamente enquanto formam folhas com um número infinito de amostras de nós, fazendo com que as proporções de classes empíricas nos nós terminais convirjam para suas contrapartes teóricas. Isso é garantido, por exemplo, interrompendo a construção quando $N_t < N^{(N)}\,\min$, tal que $N^{(N)}\,\min \to 0$e $N \times N^{(N)}\,\min \to \infty$ como $N \to \infty$ (onde $N^{(N)}\,\min$ é o valor do parâmetro N_{min} para uma floresta cultivada em um conjunto de aprendizagem de tamanho N); (b) As probabilidades de classe posterior induzidas pelo conjunto são todas contínuas no espaço de entrada $\mathcal{X}$.

Dadas as formulações próximas dos métodos, os resultados de [Zhao, 2000] em relação ao PERT estendem-se a Árvores Extremamente Randomizadas, desde que as probabilidades da classe posterior sejam contínuas. No Apêndice F de [Geurts et al., 2006a], Extremely Randomizes Trees (para $K = 1$ e $M \to \infty$) são mostrados como uma função multilinear contínua por partes de seus argumentos, o que deve, portanto, ser suficiente para estabelecer a forte consistência do método quando as árvores são construídas totalmente aleatoriamente.

Breiman [2004]:

Neste trabalho, a consistência é estudada para uma variante simplificada do algoritmo Random Forest, assumindo (i) nenhuma amostragem bootstrap, (ii) que as variáveis são selecionadas como variáveis divididas com probabilidade $p(m)$ (para $m = 1, \dots, p$), (iii) que as divisões em variáveis relevantes são definidas no ponto médio dos valores da variável selecionada, (iv) que as divisões em variáveis irrelevantes são definidas em pontos aleatórios ao longo dos valores. da variável selecionada e (v) que as árvores estão balanceadas. Sob estas suposições, pode ser mostrado que esta variante se reduz a um algoritmo de vizinho mais próximo (adaptativo) [Lin e Jeon, 2002], para o qual são satisfeitas condições de consistência (fraca) (tanto na regressão como na classificação). Além disso, este trabalho estuda a decomposição viés-variância deste

simplified method and shows that the rate of convergence towards the Bayes error only depends on the number r of relevant variables, hence explaining why random forests work well even with many noise variables.

Biau et al. [2008]:

In binary classification, Biau et al. [2008] proves that if the randomized base models in an ensemble are consistent, then the corresponding majority or soft voting ensembles are also consistent. (This result was later expanded both for multi-class classification [Denil et al., 2013b] and regression [Denil et al., 2013a].)

From this proposition, the consistency of Purely Random Forests [Breiman, 2000] is established. Let us assume that the input space $\mathcal{X}$ is supported on $[0,1]^p$ and that terminal nodes represent hyper rectangles of $[0,1]^p$, called cells, and forming together a partition of $[0,1]^p$. At each step, one of the current terminal nodes t and one the p input variables are chosen uniformly at random. The selected node t is then split along the chosen variable at a random location, along the length of the chosen side in t. This procedure is repeated $k \geqslant 1$ times, which amounts to developing trees in random order. In this setting, and similarly to PERT, (strong) consistency of Purely Random Forests is guaranteed whenever $k^{(N)} \to \infty$ and $\frac{k^{(N)}}{N} \to 0$ as $N \to \infty$ (where $k^{(N)}$ is the value of k for a forest grown on learning set of size N) – which is equivalent to letting the number of points in terminal nodes grow to infinity.

In Purely Random Forests, let us remark that trees are built in a data-independent manner, without even looking at the samples in $\mathcal{L}$. In this same work, and assuming no bootstrap sampling, the authors show that consistency is however also guaranteed when the position of the cut is chosen in a data-dependent manner, by selecting a random gap between consecutive node samples (ordered along the chosen variable) and then sampling uniformly within the gap.

Biau [2012]:

In this work, the authors more closely approaches the consistency of the actual Random Forest algorithm and prove the consistency of the following variant.

Again, let us assume that the input space $\mathcal{X}$ is supported on $[0,1]^p$ and that terminal nodes represent hyper rectangles of $[0,1]^p$, forming together a partition of the input space. At each step, all current terminal nodes are independently split using one of the p input variables X_j, drawn with probability $p_j^{(N)}$, and using as threshold the mid-point of the chosen side in t. This procedure is repeated $\lceil \log_2 k \rceil$ times, which amounts

método simplificado e mostra que a taxa de convergência para o erro de Bayes depende apenas do número r de variáveis relevantes, explicando assim por que florestas aleatórias funcionam bem mesmo com muitas variáveis de ruído.

Biau et al. [2008]:

Na classificação binária, Biau et al. [2008] prova que se os modelos básicos randomizados em um conjunto são consistentes, então os conjuntos de votação majoritária ou de votação suave correspondentes também são consistentes. (Este resultado foi posteriormente expandido tanto para classificação multiclasse [Denil et al., 2013b]quanto para regressão [Denil et al., 2013a].)

A partir desta proposição, a consistência das Florestas Puramente Aleatórias[Breiman, 2000] é estabelecida. Suponhamos que o espaço de entrada $\mathcal{X}$ seja suportado em $[0,1]^p$ e que os nós terminais representem hiperretângulos de $[0,1]^p$, chamados células, e formando juntos uma partição de $[0,1]^p$. Em cada etapa, um dos nós terminais atuais t e uma das variáveis de entrada p são escolhidos uniformemente e aleatoriamente. O nó selecionado t é então dividido ao longo da variável escolhida em um local aleatório, ao longo do comprimento do lado escolhido em t. Este procedimento é repetido $k \geqslant 1$ vezes, o que equivale ao desenvolvimento de árvores em ordem aleatória. Nesta configuração, e de forma semelhante ao PERT, a consistência (forte) de Florestas Puramente Aleatórias é garantida sempre que $k^{(N)} \to \infty$e $\frac{k^{(N)}}{N} \to 0$ como $N \to \infty$ (onde $k^{(N)}$ é o valor de k para uma floresta cultivada em um conjunto de aprendizagem de tamanho N) – o que equivale a deixar o número de pontos em nós terminais crescer até o infinito.

Em Florestas Puramente Aleatórias, observemos que as árvores são construídas de maneira independente dos dados, sem sequer olhar para as amostras em $\mathcal{L}$. Neste mesmo trabalho, e assumindo que não existe amostragem bootstrap, os autores mostram que a consistência também é garantida quando a posição do corte é escolhida de forma dependente dos dados, selecionando um intervalo aleatório entre amostras de nós consecutivas (ordenadas ao longo da variável escolhida) e depois amostrando u niformemente dentro do intervalo.

Biau [2012]:

Neste trabalho, os autores abordam mais de perto a consistência do algoritmo Random Forest real e provam a consistência da seguinte variante.

Novamente, vamos supor que o espaço de entrada $\mathcal{X}$ seja suportado em$[0,1]^p$ e que os nós terminais representem hiperretângulos de$[0,1]^p$, formando juntos uma partição do espaço de entrada. Em cada etapa, todos os nós terminais atuais são divididos independentemente usando uma das p variáveis de entrada X_j, desenhada com probabilidade $p_j^{(N)}$, e usando como limite o ponto médio do lado escolhido em$t$. Este procedimento é repetido $\lceil \log_2 k \rceil$ vezes, o que equivale

to developing trees in breadth-first order until depth $\lceil \log_2 k \rceil$. (Strong) Consistency is then guaranteed whenever $p_j^{(N)} k^{(N)} \to \infty$ (for $j = 1, \ldots, p$) and $\frac{k^{(N)}}{N} \to 0$ as $N \to \infty$.

In particular, by properly defining the probabilities p_j, this result can be shown to include the situation where, at each node, randomness is introduced by selecting at random a subset of K variables and splitting along the one that maximizes some impurity criterion, like in Random Forest. (Note however that best cut-points coincide with the mid-points only for some probability distributions.) Assuming no bootstrap sampling and provided that two independent datasets are used for evaluating the goodness of the splits and fitting the prediction values at leafs, consistency of the method is then also established.

Interestingly, and corroborating results of [Breiman, 2004], this work also highlights the fact that performance of random forests only depends on the number r of relevant variables, and not on p, making the method robust to overfitting.

Denil et al. [2013a]:

Building upon [Biau, 2012], Denil et al. [2013a] narrowed the gap between theory and practice by proving the consistency of the following variant.

For each tree in the forest, the learning set is partitioned randomly into structure points (used for determining splits) and estimation points (used for fitting values at leafs). Again, let us assume that the input space $\mathcal{X}$ is supported on $[0,1]^p$ and that terminal nodes represent hyper rectangles of $[0,1]^p$, forming together a partition of the input space. At each step, current terminal nodes are expanded by drawing at random $\min(1 + \text{Poisson}(\lambda), p)$ variables and then looking for the cut-point that maximizes the impurity criterion, as computed over m randomly drawn structure points. The construction halts when no split leading to child nodes with less than k node samples can be found.

In regression, assuming not bootstrap sampling, (strong) consistency of this variant is guaranteed whenever $k^{(N)} \to \infty$ and $\frac{k^{(N)}}{N} \to 0$ as $N \to \infty$ (where $k^{(N)}$ is the value of k for a forest grown on learning set of size N).

Scornet et al. [2014]:

This work establishes the first consistency result for the original Random Forest algorithm. In particular, (strong) consistency is obtained in the context of regression additive models, assuming subsampling without replacement (instead of bootstrap sampling). This work is the first result establishing consistency

para desenvolver árvores em ordem de largura até a profundidade $\lceil \log_2 k \rceil$. (Forte) A consistência é então garantida sempre que $p_j^{(N)} k^{(N)} \to \infty$ (para $j = 1, \ldots, p$) e $\frac{k^{(N)}}{N} \to 0$ como $N \to \infty$.

Em particular, ao definir adequadamente as probabilidades p_j, este resultado pode ser mostrado para incluir a situação onde, em cada nó, a aleatoriedade é introduzida selecionando aleatoriamente um subconjunto de variáveis K e dividindo ao longo daquele que maximiza algum critério de impureza, como em Random Forest. (Observe, no entanto, que os melhores pontos de corte coincidem com os pontos médios apenas para algumas distribuições de probabilidade.) Assumindo que não há amostragem bootstrap e desde que dois conjuntos de dados independentes sejam usados para avaliar a qualidade das divisões e ajustar os valores de previsão nas folhas, a consistência do método também é então estabelecida.

Curiosamente, e corroborando os resultados de [Breiman, 2004],este trabalho também destaca o fato de que o desempenho de florestas aleatórias depende apenas do número r de variáveis relevantes, e não de$p$, tornando o método robusto ao overfitting.

Denil et al. [2013a]:

Com base em [Biau, 2012], Denil et al. [2013a] diminuiu a lacuna entre a teoria e a prática provando a consistência da seguinte variante.

Para cada árvore na floresta, o conjunto de aprendizagem é particionado aleatoriamente em pontos de estrutura (usados para determinar divisões) e pontos de estimativa (usados para ajustar valores nas folhas). Novamente, vamos supor que o espaço de entrada $\mathcal{X}$ seja suportado em $[0,1]^p$ e que os nós terminais representem hiperretângulos de $[0,1]^p$, formando juntos uma partição do espaço de entrada. Em cada etapa, os nós terminais atuais são expandidos desenhando aleatoriamente variáveis $\min(1 + \text{Poisson}(\lambda), p)$ e, em seguida, procurando o ponto de corte que maximiza o critério de impureza, conforme calculado em m pontos de estrutura desenhados aleatoriamente. A construção é interrompida quando nenhuma divisão que leva a nós filhos com menos de k amostras de nó pode ser encontrada.

Na regressão, assumindo que não é uma amostragem bootstrap, a consistência (forte) desta variante é garantida sempre que $k^{(N)} \to \infty$ e$\frac{k^{(N)}}{N} \to 0$ como $N \to \infty$ (onde $k^{(N)}$ é o valor de k para um conjunto de tamanho de floresta cultivada em aprendizagem N).

Scornet et al. [2014]:

Este trabalho estabelece o primeiro resultado de consistência para o Algoritmo Florestal Aleatório original. Em particular, a consistência (forte) é obtida no contexto de modelos aditivos de regressão, assumindo subamostragem sem substituição (em vez de amostragem bootstrap). Este trabalho é o primeiro resultado que estabelece consistência

when (i) splits are chosen in a data-dependent manner and (ii) leafs are not let to grow to an infinite number of node samples.

In conclusions, despite the difficulty of the mathematical analysis of the method, these theoretical works provide together converging arguments all confirming why random forests – including the Random Forest algorithm but also Extremely Randomized Trees – appear to work so well in practice.

Finally, let us complete this review by mentioning consistency results in the case of domain-specific adaptations of random forests, including quantile regression [Meinshausen, 2006], survival analysis [Ishwaran and Kogalur, 2010] and online forest construction [Denil et al., 2013b].

quando (i) as divisões são escolhidas de maneira dependente dos dados e (ii) as folhas não podem crescer até um número infinito de amostras de nós.

Em conclusão, apesar da dificuldade da análise matemática do método, estes trabalhos teóricos fornecem, em conjunto, argumentos convergentes que confirmam porque é que as florestas aleatórias – incluindo o algoritmo Floresta Aleatória, mas também as Árvores Extremamente Randomizadas – parecem funcionar tão bem na prática.

Finalmente, vamos completar esta revisão mencionando resultados de consistência no caso de adaptações específicas de domínio de florestas aleatórias, incluindo regressão quantílica [Meinshausen, 2006], análise de sobrevivência [Ishwaran e Kogalur, 2010] e construção florestal on-line [Denil etal., 2013b].

5 COMPLEXITY ANALYSIS AND IMPLEMENTATION

Outline

In this chapter, we study the computational efficiency of tree-based ensemble methods. In sections 5.1 and 5.2, we derive and discuss the time complexity of random forests, first for building them from data and then for making predictions. In Section 5.3, we discuss technical details that are critical for efficiently implementing random forests. Finally, we conclude in Section 5.4 with benchmarks of the random forest implementation developed within this work and compare our solution with alternative implementations.

5.1 COMPLEXITY OF THE INDUCTION PROCEDURE

The dominant objective of most machine learning methods is to find models that maximize accuracy. For models of equivalent performance, a secondary objective is usually to minimize complexity, both functional, as studied in the previous chapters, and computational. With regards to the later, a first and immediate aspect of computational complexity in decision trees and random forests is the *time complexity* for learning models, that is the number of operations required for building models from data.

Given the exponential growth in the number of possible partitions of N samples, we chose in Section 3.6 to restrict splitting rules to binary splits defined on single variables. Not only this is sufficient for reaching good accuracy (as discussed in the previous chapter), it also allows for time complexity to effectively remain within reasonable bounds.

Formally, let $T(N)$ denotes the time complexity for building a decision tree from a learning set $\mathcal{L}$ of N samples. From Algorithm 3.2, $T(N)$ corresponds to the number of operations required for splitting a node of N samples and then for recursively building two sub-trees respectively from N_{t_L} and N_{t_R} samples. Without loss of generality, let us assume that learning samples all have distinct input values, such that it is possible to build a fully developed decision tree where each leaf contains exactly one sample. Under this assumption, determining time complexity therefore boils down to solve the recurrence equation

$$\begin{cases} T(1) = c_1 \\ T(N) = C(N) + T(N_{t_L}) + T(N_{t_R}), \end{cases} \tag{5.1}$$

5 COMPLEXITYANALYSÉEDIMPLEMENTATION

Contorno

Neste capítulo, estudamos a eficiência computacional de métodos de conjunto baseados em árvores. Nas seções 5.1 e 5.2, derivamos e discutimos a complexidade temporal de florestas aleatórias, primeiro para construí-las a partir de dados e depois para fazer previsões. Na Seção 5.3, discutimos detalhes técnicos que são críticos para a implementação eficiente de florestas aleatórias. Finalmente, concluímos na Seção 5.4 com benchmarks da implementação de floresta aleatória desenvolvida neste trabalho e comparamos nossa solução com implementações alternativas.

5.1 complexidade do procedimento de indução

O objetivo dominante da maioria dos métodos de aprendizado de máquina é encontrar modelos que maximizem a precisão. Para modelos de desempenho equivalente, um objetivo secundário geralmente é minimizar a complexidade, tanto funcional, como estudado nos capítulos anteriores, quanto computacional. Com relação a este último, um primeiro e imediato aspecto da complexidade computacional em árvores de decisão e florestas aleatórias é a complexidade temporal para modelos de aprendizagem, ou seja, o número de operações necessárias para construir modelos a partir de dados.

Dado o crescimento exponencial no número de partições possíveis de amostras N, optamos na Seção 3.6 por restringir as regras de divisão a divisões binárias definidas em variáveis únicas. Isso não é apenas suficiente para alcançar uma boa precisão (conforme discutido no capítulo anterior), mas também permite que a complexidade do tempo permaneça efetivamente dentro de limites razoáveis.

Formalmente, vamos $T(N)$ denotar a complexidade de tempo para construir uma árvore de decisão a partir de um conjunto de apre ndizagem $\mathcal{L}$ de N amostras. Do Algoritmo 3.2,$T(N)$ corresponde ao número de operações necessárias para dividir um nó de N amostras e depois para construir recursivamente duas subárvores respectivamente a partir de amostras N_{t_L} e N_{t_R}. Sem perda de generalidade, vamos supor que todas as amostras de aprendizagem tenham valores de entrada distintos, de modo que seja possível construir uma árvore de decisão totalmente desenvolvida onde cada folha contenha exatamente uma amostra. Sob esta suposição, a determinação da complexidade do tempo se resume, portanto, a resolver a equação de recorrência

$$\begin{cases} T(1) = c_1 \\ T(N) = C(N) + T(N_{t_L}) + T(N_{t_R}), \end{cases} \tag{5.1}$$

where c_1 is the constant cost for making a leaf node out of a single sample and $C(N)$ is the runtime complexity for finding a split s and then partitioning the N node samples into t_L and t_R. In particular, this later operation requires at least to iterate over all N samples, which sets a linear lower bound on the time complexity within a node, i.e., $C(N) = \Omega(N)$. As for finding the split s, this can be achieved in several ways, as outlined in the previous chapters for several randomized variants of the induction procedure. As we will see, not only this has an impact on the accuracy of the resulting model, it also drives the time complexity of the induction procedure.

Big O notations

Time complexity analyzes the asymptotic behavior of an algorithm with respect to the size N of its input and its hyper-parameters. In this way, big O notations are used to formally express an asymptotic upper bound on the growth rate of the number $f(N)$ of steps in the algorithm. Formally, we write that

$$f(N) = O(g(N)) \text{ if } \exists c > 0, N_0 > 0, \forall N > N_0, f(N) \leqslant cg(N) \quad (5.2)$$

to express that $f(N)$ is asymptotically upper bounded by $g(N)$, up to some negligible constant factor c. Similarly, big Ω notations are used to express an asymptotic lower bound on the growth rate of the number of steps in the algorithm. Formally, we write that

$$f(N) = \Omega(g(N)) \text{ if } \exists c > 0, N_0 > 0, \forall N > N_0, cg(N) \leqslant f(N) \quad (5.3)$$

to express that $f(N)$ is asymptotically lower bounded by $g(N)$. Consequently, if $f(N)$ is both $O(g(N))$ and $\Omega(g(N))$ then $f(N)$ is both lower bounded and upper bounded asymptotically by $g(N)$ (possibly for different constants), which we write using big Θ notations:

$$f(N) = \Theta(g(N)) \text{ if } f(N) = O(g(N)) = \Omega(g(N)). \quad (5.4)$$

In the original CART induction procedure [Breiman et al., 1984] (Algorithms 3.3 and 3.4), finding a split s requires, for each of the p variables X_j (for $j = 1, \dots, p$), to sort the values $x_{i,j}$ of all N node samples (for $i = 1, \dots, N$) and then to iterate over these in order to find the best threshold v. Assuming that the value of the impurity criterion can be computed iteratively for consecutive thresholds, the most costly operation is sorting, whose time complexity is at worst $O(N \log N)$ using an optimal algorithm. As a result, the overall within-node complexity is $C(N) = O(pN \log N)$. In a randomized tree as built with the Random Forest algorithm [Breiman, 2001] (RF, Algorithm 4.1), the search of the best split s is carried out in the same way, but only on $K \leqslant p$ of the input variables, resulting in a within-node complexity

onde c_1 é o custo constante para criar um nó folha a partir de uma única amostra e $C(N)$ é a complexidade do tempo de execução para encontrar uma divisão s e então particionar as amostras de nó N em t_L e t_R. Em particular, esta operação posterior requer pelo menos iterar todas as amostras N , o que define um limite inferior linear para a complexidade do tempo dentro de um nó, ou seja, $C(N) = \Omega(N)$. Quanto a encontrar a divisão s, isto pode ser conseguido de diversas maneiras, conforme descrito nos capítulos anteriores para diversas variantes aleatórias do procedimento de indução. Como veremos, isto não só tem impacto na precisão do modelo resultante, como também determina a complexidade temporal do procedimento de indução.

Notações O grandes

A complexidade de tempo analisa o comportamento assintótico de um algoritmo em relação ao tamanho N de sua entrada e seus hiperparâmetros. Dessa forma, notações O grandes são usadas para expressar formalmente um limite superior assintótico na taxa de crescimento do número $f(N)$ de etapas no algoritmo. Formalmente, escrevemos que

$$f(N) = O(g(N)) \text{ if } \exists c > 0, N_0 > 0, \forall N > N_0, f(N) \leqslant cg(N) \quad (5.2)$$

para expressar que $f(N)$ é assintoticamente limitado superiormente por $g(N)$, até algum fator constante desprezível c. Da mesma forma, notações grandes Ω são usadas para expressar um limite inferior assintótico na taxa de crescimento do número de etapas no algoritmo. Formalmente, escrevemos que

$$f(N) = \Omega(g(N)) \text{ if } \exists c > 0, N_0 > 0, \forall N > N_0, cg(N) \leqslant f(N) \quad (5.3)$$

para expressar que $f(N)$ é assintoticamente limitado inferiormente por $g(N)$. Conseqüentemente, se $f(N)$ é $O(g(N))$ e $\Omega(g(N))$ então $f(N)$ é tanto limitado inferior quanto superior assintoticamente por $g(N)$ (possivelmente para constantes diferentes), que escrevemos usando notações Θ grandes:

$$f(N) = \Theta(g(N)) \text{ if } f(N) = O(g(N)) = \Omega(g(N)). \quad (5.4)$$

No procedimento de indução CART original [Breiman et al., 1984] (Algoritmos 3.3 e 3.4), encontrar uma divisão s requer, para cada uma das p variáveis X_j (para $j = 1, \dots, p$), classificar os valores $x_{i,j}$ de todas N amostras de nós (para $i = 1, ., N$) e, em seguida, iterar sobre eles para encontrar o melhor limite v. Assumindo que o valor do critério de impureza pode ser calculado iterativamente para limites consecutivos, a operação mais dispendiosa é a classificação, cuja complexidade de tempo é, na pior das hipóteses, $O(N \log N)$ usando um algoritmo ideal. Como resultado, a complexidade geral dentro do nó é $C(N) = O(pN \log N)$. Em uma árvore aleatória construída com o algoritmo Random Forest [Breiman, 2001](RF, Algoritmo 4.1), a busca da melhor divisão s é realizada da mesma maneira, mas apenas em$K \leqslant p$ das variáveis de entrada, resultando em uma complexidade dentro do nó

$C(N) = O(KN \log N)$. In Extremely Randomized Trees [Geurts et al., 2006a] (ETs, Algorithm 4.3), discretization thresholds are drawn at random within the minimum and maximum node sample values of X_j, making sort no longer necessary. As such, the within-node complexity reduces to the time required for finding these lower and upper bounds, which can be done in linear time, hence $C(N) = O(KN)$. Finally, in Perfect Random Tree Ensembles [Cutler and Zhao, 2001] (PERT, Algorithm 4.2), cut-points v are set midway between two randomly drawn samples, which can be done in constant time, independently of the number N of node samples. Yet, the within-node complexity is lower bounded by the time required for partitioning the node samples into t_L and t_R, hence $C(N) = O(N)$.

Since both CART and PERT can respectively be considered as special cases of RF and ETs with regards to time complexity (i.e., for $K = p$ in CART, for $K = 1$ in PERT), let us consider the overall complexity $T(N)$ when either $C(N) = O(KN \log N)$ or $C(N) = O(KN)$ (for $K = 1, \dots, p$). To make our analysis easier, let us further assume that it exists $c_2, c_3 > 0$ such that $c_2 KN \log N \leqslant C(N) \leqslant c_3 KN \log N$ for all $N \geqslant 1$ (resp. $c_2 KN \leqslant C(N) \leqslant c_3 KN$ for all $N \geqslant 1$). Results presented below extend however to the case where $C(N) = \Theta(KN)$ (resp. $\Theta(KN \log N)$), at the price of a more complicated mathematical analysis. Time complexity is studied in three cases:

Best case.

The induction procedure is the most efficient when node samples can always be partitioned into two balanced subsets of $\frac{N}{2}$ samples.

Worst case.

By contrast, the induction procedure is the least efficient when splits are unbalanced. In the worst case, this results in splits that move a single sample in the first sub-tree and put all $N - 1$ others in the second sub-tree.

Average case.

The average case corresponds to the average time complexity, as taken over all possible learning sets $\mathcal{L}$ (of a given size and for a given distribution) and all random seeds θ. Since the analysis of this case is hardly feasible without strong assumptions on the structure of the problem, we consider instead the case where the sizes of the possible partitions of a node are all equiprobable. We believe this is a good enough approximation, that should at least help us derive the order of magnitude of the complexity (instead of deriving it exactly).

Since the induction algorithm looks for splits that most decrease impurity (in particular, as $K \to p$), we assume that balanced partitions of the nodes samples should be more likely than partitions that are

$C(N) = O(KN$ registro $N)$. Em Árvores Extremamente Randomizadas [Geurts et al., 2006a] (ETs, Algoritmo 4.3), os limites de discretização são desenhados aleatoriamente dentro dos valores mínimo e máximo de amostra de nós de X_j, tornando a classificação não mais necessária. Como tal, a complexidade dentro do nó reduz-se ao tempo necessário para encontrar estes limites inferior e superior, o que pode ser feito em tempo linear, daí $C(N) = O(KN)$. Finalmente, em Perfect Random Tree Ensembles [Cutler e Zhao, 2001](PERT, Algoritmo 4.2), os pontos de corte v são definidos no meio do caminho entre duas amostras sorteadas aleatoriamente, o que pode ser feito em tempo constante, independentemente do número N de amostras de nós. No entanto, a complexidade dentro do nó é limitada pelo tempo necessário para particionar as amostras do nó em t_L e t_R, portanto, $C(N) = O(N)$.

Como tanto CART quanto PERT podem ser considerados, respectivamente, como casos especiais de RF e ETs no que diz respeito à complexidade de tempo (ou seja, para$K = p$ no CART, para $K = 1$ no PERT), vamos considerar a complexidade geral $T(N)$ quando $C(N) = O(KN \log N)$ ou $C(N) = O(KN)$(para $K = 1, \dots, p$). Para facilitar nossa análise, vamos supor ainda que ele existe $c_2, c_3 > 0$ tal que $c_2 KN \log N \leqslant C(N) \leqslant c_3 KN \log N$para todos $N \geqslant 1$ (resp. $c_2 KN \leqslant C(N) \leqslant c_3 KN$ para todos $N \geqslant 1$). Os resultados apresentados abaixo estendem-se, no entanto, ao caso em que $C(N) = \Theta(KN)$(resp. $\Theta(KN \log N)$), ao preço de uma análise matemática mais complicada. A complexidade do tempo é estudada em três casos:

Melhor caso.

O procedimento de indução é mais eficiente quando as amostras de nós sempre podem ser particionadas em dois subconjuntos balanceados de amostras $\frac{N}{2}$.

Pior caso.

Por outro lado, o procedimento de indução é o menos eficiente quando as divisões são desequilibradas. Na pior das hipóteses, isso resulta em divisões que movem uma única amostra na primeira subárvore e colocam todas as outras na segunda subárvore.

Caso médio.

O caso médio corresponde à complexidade de tempo média, considerada todos os conjuntos de aprendizagem possíveis $\mathcal{L}$(de u m determinado tamanho e para uma determinada distribuição) e todas as sementes aleatórias θ. Como a análise deste caso é dificilmente viável sem fortes suposições sobre a estrutura do problema, consideramos, em vez disso, o caso em que os tamanhos das possíveis partições do ânodo são todos equiprováveis. Acreditamos que esta é uma aproximação boa o suficiente, que deveria pelo menos nos ajudar a derivar a ordem de grandeza da complexidade (em vez de derivá-la com exatidão).

Como o algoritmo de indução procura divisões que mais diminuam a impureza (em particular, como $K \to p$), assumimos que partições balanceadas das amostras de nós deveriam ser mais prováveis do que partições que são

unbalanced (such a claim is however highly data dependent). Under this assumption, random forests should therefore approach the best case rather than the average case (for which partitions are equally likely). However, as randomization increases (e.g., as $K \to 1$), splits yield partitions that tend to be more equiprobable, hence approaching the average case. As such, we believe the complexity for the true average case should lie in-between the complexity of the best case and the complexity of the average case as defined above. As for the worst case, it should only arise in exceptional circumstances.

Let us mention that due to the end-cut preference of classical impurity criteria, worst splits are in fact likely to be selected (see [Breiman et al., 1984], Theorem 11.1) if none of the K selected variables are informative with respect to the output. (Note that this effect is less pronounced in ETs because thresholds are drawn at random rather than being optimized locally, as in RF.) In the analysis carried below, intermingling effects due to good splits at the top of trees and such degenerated splits due to end-cut preference when input variables become independent of the output are not explicitly taken into account. Yet, the simplifying assumptions defining our average case appear to constitute a good-enough approximation, as further confirmed in empirical benchmarks in Section 5.4.

Master Theorem

Some of the results outlined in the remaining of this section make use of the Master Theorem [Bentley et al., 1980; Goodrich and Tamassia, 2006], which is recalled below to be self-contained.

Theorem 5.1. *Let consider the recurrence equation*

$$\begin{cases} T(1) = c & \text{if} \quad n < d \\ T(n) = aT(\frac{n}{b}) + f(n) & \text{if} \quad n \geqslant d \end{cases} \tag{5.5}$$

where $d \geqslant 1$ *is an integer constant,* $a > 0$, $c > 0$ *and* $b > 1$ *are real constants, and* $f(n)$ *is a function that is positive for* $n \geqslant d$.

(a) If there is a small constant $\epsilon > 0$, *such that* $f(n)$ *is* $O(n^{\log_b a - \epsilon})$, *then* $T(n)$ *is* $\Theta(n^{\log_b a})$.

(b) If there is a constant $k \geqslant 0$, *such that* $f(n)$ *is* $\Theta(n^{\log_b a} \log^k n)$, *then* $T(n)$ *is* $\Theta(n^{\log_b a} \log^{k+1} n)$.

(c) If there are small constants $\epsilon > 0$ *and* $\delta < 1$, *such that* $f(n)$ *is* $\Omega(n^{\log_b a + \epsilon})$ *and* $af(\frac{n}{b}) \leqslant \delta f(n)$, *for* $n \geqslant d$, *then* $T(n)$ *is* $\Theta(f(n))$.

desequilibrado (tal afirmação é, no entanto, altamente dependente dos dados). Sob esta suposição, as florestas aleatórias deveriam, portanto, aproximar-se do melhor caso e não do caso médio (para o qual as partições são igualmente prováveis). No entanto, à medida que a randomização aumenta (por exemplo, como $K \to 1$), as divisões produze m partições que tendem a ser mais equiprováveis, aproximando-se, portanto, do caso médio. Como tal, acreditamos que a complexidade do verdadeiro caso médio deve situar-se entre a complexidade do melhor caso e a complexidade do caso médio conforme definido acima. Quanto ao pior caso, só deverá surgir em circunstâncias excepcionais.

Mencionemos que, devido à preferência de corte final dos critérios clássicos de impureza, é provável que as piores divisões sejam selecionadas (ver [Breiman et al., 1984], Teorema 11.1) se nenhuma das K variáveis selecionadas for informativa em relação à saída. (Observe que este efeito é menos pronunciado em ETs porque os limites são traçados aleatoriamente em vez de serem otimizados localmente, como em RF.) Na análise realizada abaixo, os efeitos de mistura devido a boas divisões no topo das árvores e tais divisões degeneradas devido à preferência de corte final quando as variáveis de entrada se tornam independentes da saída não são explicitamente levados em conta. No entanto, os pressupostos simplificadores que definem o nosso caso médio parecem constituir uma aproximação suficientemente boa, conforme confirmado nos parâmetros de referência empíricos na Secção 5.4.

Teorema Mestre

Alguns dos resultados descritos no restante desta seção fazem uso do Teorema Mestre [Bentley et al., 1980; Goodrich e Tamassia,2006], que é lembrado abaixo como sendo independente.

Teorema 5.1. Vamos considerar a equação de recorrência

$$\begin{cases} T(1) = c & \text{if} \quad n < d \\ T(n) = aT(\frac{n}{b}) + f(n) & \text{if} \quad n \geqslant d \end{cases} \tag{5.5}$$

onde $d \geqslant 1$ *é uma constante inteira,* $a > 0$, $c > 0$ *e* $b > 1$ *são constantes reais e* $f(n)$ *é uma função positiva para* $n \geqslant d$.

(a) Se houver uma pequena constante $\epsilon > 0$, *tal que* $f(n)$ *é* $O(n^{\log_b a - \epsilon})$, *então* $T(n)$ *é* $\Theta(n^{\log_b a})$. *(b) Se existe uma constante* $k \geqslant 0$, *tal que* $f(n)$ *é* $\Theta(n^{\log_b a} \log^k n)$, *então*$T(n)$ *é* $\Theta(n^{\log_b a} \log^{k+1} n)$. *(c) Se houver pequenas constantes* $\epsilon > 0$ *e* $\delta < 1$, *tais que* $f(n)$ *é*$\Omega(n^{\log_b a + \epsilon})$ *e* $af(\frac{n}{b}) \leqslant \delta f(n)$, *para* $n \geqslant d$, *então* $T(n)$ *é* $\Theta(f(n))$.

–

Theorem 5.2. *For* $c_2 KN \log N \leqslant C(N) \leqslant c_3 KN \log N$ *(for all* $N \geqslant 1$*), the time complexity for building a decision tree in the best case is* $\Theta(KN \log^2 N)$.

Proof. Let us rewrite $T(N)$ as $KT'(N)$ where, in the best case,

$$\begin{cases} T'(1) = c'_1 \\ T'(N) = C'(N) + 2T'(\frac{N}{2}), \end{cases} \tag{5.6}$$

with $c'_1 = \frac{c_1}{K}$ and $C'(N) = \frac{C(N)}{K}$. In this form, the second case of the Master Theorem applies for $f(N) = C'(N) = \Theta(N \log N)$, $a = 2$, $b = 2$ and $k = 1$. Accordingly, $T'(N) = \Theta(N \log^{k+1} N) = \Theta(N \log^2 N)$ and $T(N) = \Theta(KN \log^2 N)$. □

Theorem 5.3. *For* $c_2 KN \leqslant C(N) \leqslant c_3 KN$ *(for all* $N \geqslant 1$*), the time complexity for building a decision tree in the best case is* $\Theta(KN \log N)$.

Proof. In the best case, $T(N) = KT'(N)$, where

$$\begin{cases} T'(1) = c'_1 \\ T'(N) = C'(N) + 2T'(\frac{N}{2}). \end{cases} \tag{5.7}$$

Again, the second case of the Master Theorem applies, this time for $f(N) = C'(N) = \Theta(N)$, $a = 2$, $b = 2$ and $k = 0$. Accordingly, $T'(N) = \Theta(N \log^{k+1} N) = \Theta(N \log N)$ and $T(N) = \Theta(KN \log N)$. □

Theorem 5.4. *For* $c_2 KN \log N \leqslant C(N) \leqslant c_3 KN \log N$ *(for all* $N \geqslant 1$*), the time complexity for building a decision tree in the worst case is* $\Omega(KN^2)$ *and* $O(KN^2 \log N)$.

Proof. In the worst case, $T(N) = KT'(N)$, where

$$\begin{cases} T'(1) = c'_1 \\ T'(N) = C'(N) + T'(1) + T'(N-1). \end{cases} \tag{5.8}$$

Let us first consider an upper bound on $T'(N)$. By expanding the recurrence, we have

$$\begin{aligned} T'(N) &\leqslant c_3 N \log N + c'_1 + T'(N-1) \\ &= \sum_{i=1}^{N} c'_1 + c_3 i \log i \\ &= c'_1 N + \sum_{i=1}^{N} c_3 i \log i \\ &\leqslant c'_1 N + c_3 \log N \sum_{i=1}^{N} i \\ &= c'_1 N + c_3 \log N \frac{N(N+1)}{2} \end{aligned}$$

Teorema 5.2. Para $c_2 KN \log N \leqslant C(N) \leqslant c_3 KN \log N$ *(para todos* $N \geqslant 1$*), a complexidade de tempo para construir uma árvore de decisão na melhor das hipóteses é* $\Theta(KN \log^2 N)$.

Prova. Vamos reescrever $T(N)$ como $KT'(N)$ onde, na melhor das hipóteses,

$$\begin{cases} T'(1) = c'_1 \\ T'(N) = C'(N) + 2T'(\frac{N}{2}), \end{cases} \tag{5.6}$$

com $c'_1 = \frac{c_1}{K}$ e $C'(N) = \frac{C(N)}{K}$. Nesta forma, o segundo caso do Teorema Mestre se aplica a $f(N) = C'(N) = \Theta(N \log N)$, $a = 2$, $b = 2$ e $k = 1$. Da mesma forma, $T'(N) = \Theta(N \log^{k+1} N) = \Theta(N \log^2 N)$ e $T(N) = \Theta(KN \log^2 N)$.

□

Teorema 5.3. Para $c_2 KN \leqslant C(N) \leqslant c_3 KN$ *(para todos* $N \geqslant 1$*), a complexidade de tempo para construir uma árvore de decisão na melhor das hipóteses é* $\Theta(KN \log N)$.

Prova. Na melhor das hipóteses, $T(N) = KT'(N)$, onde

$$\begin{cases} T'(1) = c'_1 \\ T'(N) = C'(N) + 2T'(\frac{N}{2}). \end{cases} \tag{5.7}$$

Novamente, o segundo caso do Teorema Mestre se aplica, desta vez para $f(N) = C'(N) = \Theta(N)$, $a = 2$, $b = 2$ e $k = 0$. Da mesma forma, $T'(N) = \Theta(N \log^{k+1} N) = \Theta(N \log N)$ e $T(N) = \Theta(KN \log N)$.

□

Teorema 5.4. Para $c_2 KN \log N \leqslant C(N) \leqslant c_3 KN \log N$ *(para todos* $N \geqslant 1$*), a complexidade de tempo para construir uma árvore de decisão no pior caso é* $\Omega(KN^2)$ *e* $O(KN^2 \log N)$.

Prova. No pior caso, $T(N) = KT'(N)$, onde

$$\begin{cases} T'(1) = c'_1 \\ T'(N) = C'(N) + T'(1) + T'(N-1). \end{cases} \tag{5.8}$$

Consideremos primeiro um limite superior em $T'(N)$. Expandindo a recorrência, temos

$$\begin{aligned} T'(N) &\leqslant c_3 N \log N + c'_1 + T'(N-1) \\ &= \sum_{i=1}^{N} c'_1 + c_3 i \log i \\ &= c'_1 N + \sum_{i=1}^{N} c_3 i \log i \\ &\leqslant c'_1 N + c_3 \log N \sum_{i=1}^{N} i \\ &= c'_1 N + c_3 \log N \frac{N(N+1)}{2} \end{aligned}$$

$$= O(N^2 \log N). \tag{5.9}$$

Similarly, a lower bound on $T'(N)$ can be derived in the following way:

$$\begin{aligned} T'(N) &\geqslant c_2 N \log N + c_1' + T'(N-1) \\ &= \sum_{i=1}^{N} c_1' + c_2 i \log i \\ &= c_1' N + \sum_{i=1}^{N} c_2 i \log i \\ &\geqslant c_1' N + c_2 \sum_{i=2}^{N} i \\ &= c_1' N + c_2 \frac{N(N+1)}{2} - c_2 \\ &= \Omega(N^2). \end{aligned} \tag{5.10}$$

As such, $T(N) = \Omega(KN^2)$ and $O(KN^2 \log N)$ in the worst case. □

Let us remark that the lower and upper bounds derived for $T(N)$ in Theorem 5.4 are not tight. Nevertheless, given the fact that

$$\sum_{i=1}^{N} i \log i = \log(H(N)), \tag{5.11}$$

where $H(N)$ is the hyperfactorial function, complexity could be re-expressed tightly as $T(N) = \Theta(K \log(H(N)))$.

Theorem 5.5. *For $c_2 KN \leqslant C(N) \leqslant c_3 KN$ (for all $N \geqslant 1$), the time complexity for building a decision tree in the worst case is $\Theta(KN^2)$.*

Proof. In the worst case, $T(N) = KT'(N)$, where

$$\begin{cases} T'(1) = c_1' \\ T'(N) = C'(N) + T'(1) + T'(N-1). \end{cases} \tag{5.12}$$

By expanding the recurrence, we have

$$\begin{aligned} T'(N) &\leqslant c_3 N + c_1' + T'(N-1) \\ &= c_1' + \sum_{i=2}^{N} (c_1' + c_3 i) \\ &= c_1' N + \frac{c_3}{2}(N^2 + 3N - 4) \\ &= O(N^2). \end{aligned} \tag{5.13}$$

In the exact same way, $T'(N)$ can be lower bounded by $c_2 N + c_1' + T'(N-1)$ and shown to be $\Omega(N^2)$. As a result, $T(N) = \Theta(KN^2)$ in the worst case. □

$$= O(N^2 \log N). \tag{5.9}$$

Da mesma forma, um limite inferior em $T'(N)$ pode ser derivado da seguinte maneira:

$$\begin{aligned} T'(N) &\geqslant c_2 N \log N + c_1' + T'(N-1) \\ &= \sum_{i=1}^{N} c_1' + c_2 i \log i \\ &= c_1' N + \sum_{i=1}^{N} c_2 i \log i \\ &\geqslant c_1' N + c_2 \sum_{i=2}^{N} i \\ &= c_1' N + c_2 \frac{N(N+1)}{2} - c_2 \\ &= \Omega(N^2). \end{aligned} \tag{5.10}$$

Dessa forma, $T(N) = \Omega(KN^2)$ e $O(KN^2$ registre $N)$ no pior caso. □

Observemos que os limites inferior e superior derivados para $T(N)$no Teorema 5.4 não são rígidos. Não obstante, dado o facto de

$$\sum_{i=1}^{N} i \log i = \log(H(N)), \tag{5.11}$$

onde $H(N)$ é a função hiperfatorial, a complexidade pode ser reexpressa firmemente como $T(N) = \Theta(K \log(H(N)))$.

Teorema 5.5. Para $c_2 KN \leqslant C(N) \leqslant c_3 KN$ (para todos $N \geqslant 1$), a co mplexidade de tempo para construir uma árvore de decisão no pior caso é $\Theta(KN^2)$.

Prova. No pior caso, $T(N) = KT'(N)$, onde

$$\begin{cases} T'(1) = c_1' \\ T'(N) = C'(N) + T'(1) + T'(N-1). \end{cases} \tag{5.12}$$

Expandindo a recorrência, temos

$$\begin{aligned} T'(N) &\leqslant c_3 N + c_1' + T'(N-1) \\ &= c_1' + \sum_{i=2}^{N} (c_1' + c_3 i) \\ &= c_1' N + \frac{c_3}{2}(N^2 + 3N - 4) \\ &= O(N^2). \end{aligned} \tag{5.13}$$

Exatamente da mesma maneira, $T'(N)$ pode ser limitado inferiormente por $c_2 N + c_1' + T'(N-1)$ e mostrado como $\Omega(N^2)$. Como resultado, $T(N) = \Theta(KN^2)$ na pior das hipóteses. □

Theorem 5.6. *For* $c_2 KN \log N \leqslant C(N) \leqslant c_3 KN \log N$ *(for all* $N \geqslant 1$*), the time complexity for building a decision tree in the average case is* $\Theta(KN \log^2 N)$.

Proof. In the average case, $T(N) = KT'(N)$, where

$$\begin{cases} T'(1) = c'_1 \\ T'(N) = C'(N) + \frac{1}{N-1} \sum_{i=1}^{N-1} (T'(i) + T'(N-i)). \end{cases} \tag{5.14}$$

Let us first consider an upper bound on $T'(N)$ by assuming that $C'(N) = c_3 N \log N$. By symmetry and by multiplying both sides of the last equation by $(N-1)$, it comes

$$(N-1)T'(N) = (N-1)(c_3 N \log N) + 2 \sum_{i=1}^{N-1} T'(i). \tag{5.15}$$

For $N \geqslant 3$, substituting N with $N-1$ similarly yields

$$(N-2)T'(N-1) = (N-2)(c_3(N-1)\log(N-1)) + 2 \sum_{i=1}^{N-2} T'(i). \tag{5.16}$$

Subtracting Equation 5.16 from 5.15, it comes after simplifications and division of both sides by $N(N-1)$:

$$\frac{T'(N)}{N} = \frac{T'(N-1)}{N-1} + c_3 \frac{2}{N} \log(N-1) + c_3 \log \frac{N}{N-1}. \tag{5.17}$$

Let us now introduce $S(N) = \frac{T'(N)}{N}$. From the last equation, it comes

$$\begin{aligned} S(N) &= S(N-1) + c_3 \frac{2}{N} \log(N-1) + c_3 \log \frac{N}{N-1} \\ &= c'_1 + c_3 \sum_{i=2}^{N} \frac{2}{i} \log(i-1) + \log \frac{i}{i-1} \\ &= c'_1 + c_3 \log N + 2c_3 \sum_{i=2}^{N} \frac{1}{i} \log(i-1) \\ &\leqslant c'_1 + c_3 \log N + 2c_3 \log N \sum_{i=2}^{N} \frac{1}{i} \\ &= c'_1 + c_3 \log N + 2c_3 \log N (H_N - 1) \\ &= O(H_N \log N) \end{aligned} \tag{5.18}$$

where H_N is the N-th harmonic number. Using

$$H_N \approx \log N + \gamma + \frac{1}{2N} + O(\frac{1}{N^2}) \tag{5.19}$$

as approximation (where γ is the Euler-Mascheroni constant), we have

$$T'(N) = O(NH_N \log N)$$

Teorema 5.6. Para $c_2 KN \log N \leqslant C(N) \leqslant c_3 KN \log N$ *(para todos* $N \geqslant 1$*), a complexidade de tempo para construir uma árvore de decisão no caso médio é* $\Theta(KN \log^2 N)$.

Prova. No caso médio, $T(N) = KT'(N)$, onde

$$\begin{cases} T'(1) = c'_1 \\ T'(N) = C'(N) + \frac{1}{N-1} \sum_{i=1}^{N-1} (T'(i) + T'(N-i)). \end{cases} \tag{5.14}$$

Vamos primeiro considerar um limite superior em $T'(N)$ assumindo esse $C'(N) = c_3 N \log N$. Por simetria e multiplicando ambos os lados da última equação por $(N-1)$, chega-se

$$(N-1)T'(N) = (N-1)(c_3 N \log N) + 2 \sum_{i=1}^{N-1} T'(i). \tag{5.15}$$

Para $N \geqslant 3$, substituindo N por $N-1$ de forma semelhante produz

$$(N-2)T'(N-1) = (N-2)(c_3(N-1)\log(N-1)) + 2 \sum_{i=1}^{N-2} T'(i). \tag{5.16}$$

Subtraindo a Equação 5.16 da 5.15, vem após simplificações e divisão de ambos os lados por $N(N-1)$:

$$\frac{T'(N)}{N} = \frac{T'(N-1)}{N-1} + c_3 \frac{2}{N} \log(N-1) + c_3 \log \frac{N}{N-1}. \tag{5.17}$$

Vamos agora apresentar $S(N) = \frac{T'(N)}{N}$. Da última equação, vem

$$\begin{aligned} S(N) &= S(N-1) + c_3 \frac{2}{N} \log(N-1) + c_3 \log \frac{N}{N-1} \\ &= c'_1 + c_3 \sum_{i=2}^{N} \frac{2}{i} \log(i-1) + \log \frac{i}{i-1} \\ &= c'_1 + c_3 \log N + 2c_3 \sum_{i=2}^{N} \frac{1}{i} \log(i-1) \\ &\leqslant c'_1 + c_3 \log N + 2c_3 \log N \sum_{i=2}^{N} \frac{1}{i} \\ &= c'_1 + c_3 \log N + 2c_3 \log N (H_N - 1) \\ &= O(H_N \log N) \end{aligned} \tag{5.18}$$

onde H_N é o N-ésimo número harmônico. Usando

$$H_N \approx \log N + \gamma + \frac{1}{2N} + O(\frac{1}{N^2}) \tag{5.19}$$

como aproximação (onde γé a constante de Euler-Mascheroni), temos

$$T'(N) = O(NH_N \log N)$$

$$= O(N(\log N + \gamma + \frac{1}{2N} + \frac{1}{N^2}) \log N)$$
$$= O(N \log^2 N).$$

Given Theorem 5.2, we also know that $T'(N)$ cannot be faster than $\Omega(N \log^2 N)$, thereby setting a lower bound on the complexity of the average case.

As a result, $T(N) = \Theta(KN \log^2 N)$ in the average case. □

Theorem 5.7. *For $c_2 KN \leqslant C(N) \leqslant c_3 KN$ (for all $N \geqslant 1$), the time complexity for building a decision tree in the average case is $\Theta(KN \log N)$.*

Proof. In the average case, $T(N) = KT'(N)$, where

$$\begin{cases} T'(1) = c'_1 \\ T'(N) = C'(N) + \frac{1}{N-1} \sum_{i=1}^{N-1} T'(i) + T'(N-i). \end{cases} \tag{5.20}$$

Let us first consider an upper bound on $T'(N)$ by assuming that $C'(N) = c_3 N$. By symmetry and by multiplying both sides of the last equation by $(N-1)$, it comes

$$(N-1)T'(N) = c_3 N(N-1) + 2 \sum_{i=1}^{N-1} T'(i). \tag{5.21}$$

For $N \geqslant 3$, substituting N with $N-1$ similarly yields

$$(N-2)T'(N-1) = c_3(N-1)(N-2) + 2 \sum_{i=1}^{N-2} T'(i). \tag{5.22}$$

Subtracting Equation 5.22 from 5.21, it comes after simplifications and division of both sides by $N(N-1)$:

$$\frac{T'(N)}{N} = \frac{T'(N-1)}{N-1} + \frac{2c_3}{N}. \tag{5.23}$$

Let us now introduce $S(N) = \frac{T'(N)}{N}$. From the last equation, it comes

$$\begin{aligned} S(N) &= S(N-1) + \frac{2c_3}{N} \\ &= c'_1 + 2c_3 \sum_{i=2}^{N} \frac{1}{i} \\ &= O(2H_N) \end{aligned} \tag{5.24}$$

where H_N is the N-th harmonic number. Using

$$H_N \approx \log N + \gamma + \frac{1}{2N} + O(\frac{1}{N^2}) \tag{5.25}$$

as approximation (where γ is the Euler-Mascheroni constant), we have

$$T'(N) = O(2H_N N)$$

$$= O(N(\log N + \gamma + \frac{1}{2N} + \frac{1}{N^2}) \log N)$$
$$= O(N \log^2 N).$$

Dado o Teorema 5.2, também sabemos que $T'(N)$ não pode ser mais rápido que $\Omega(N \log^2 N)$, estabelecendo assim um limite inferior para a complexidade do caso médio.

Como resultado, $T(N) = \Theta(KN \text{ registre}^2 N)$ no caso médio. □

Teorema 5.7. Para $c_2 KN \leqslant C(N) \leqslant c_3 KN$ (para todos $N \geqslant 1$), a complexidade de tempo para construir uma árvore de decisão no caso médio é $\Theta(KN \log N)$.

Prova. No caso médio, $T(N) = KT'(N)$, onde

$$\begin{cases} T'(1) = c'_1 \\ T'(N) = C'(N) + \frac{1}{N-1} \sum_{i=1}^{N-1} T'(i) + T'(N-i). \end{cases} \tag{5.20}$$

Vamos primeiro considerar um limite superior em $T'(N)$ assumindo que $C'(N) = c_3 N$. Por simetria e multiplicando ambos os lados da última equação por $(N-1)$, chega-se

$$(N-1)T'(N) = c_3 N(N-1) + 2 \sum_{i=1}^{N-1} T'(i). \tag{5.21}$$

Para $N \geqslant 3$, substituindo N por $N-1$ de forma semelhante produz

$$(N-2)T'(N-1) = c_3(N-1)(N-2) + 2 \sum_{i=1}^{N-2} T'(i). \tag{5.22}$$

Subtraindo a Equação 5.22 da 5.21, vem após simplificações e divisão de ambos os lados por $N(N-1)$:

$$\frac{T'(N)}{N} = \frac{T'(N-1)}{N-1} + \frac{2c_3}{N}. \tag{5.23}$$

Vamos agora apresentar $S(N) = \frac{T'(N)}{N}$. Da última equação, vem

$$\begin{aligned} S(N) &= S(N-1) + \frac{2c_3}{N} \\ &= c'_1 + 2c_3 \sum_{i=2}^{N} \frac{1}{i} \\ &= O(2H_N) \end{aligned} \tag{5.24}$$

onde H_N é o N-ésimo número harmônico. Usando

$$H_N \approx \log N + \gamma + \frac{1}{2N} + O(\frac{1}{N^2}) \tag{5.25}$$

como aproximação (onde γ é a constante de Euler-Mascheroni), temos

$$T'(N) = O(2H_N N)$$

$$= O(2N \log N + 2N\gamma + 1 + \frac{2}{N})$$
$$= O(N \log N). \quad (5.26)$$

In the exact same way, when assuming that $C'(N) = c_2 N$, $T'(N)$ can be shown to be lower bounded by $\Omega(N \log N)$. As a result, $T(N) = \Theta(KN \log N)$ in the average case. □

From Theorems 5.2-5.7, the total time complexity for building an ensemble of M randomized trees can finally be derived. As summarized in Table 5.1, complexity remains within polynomial time for all variants studied in this work. In the best case, complexity is linear with the number of split variables considered at each node (i.e., $O(K)$) and linearithmic or quasilinear with the number of (unique) samples effectively used to build each tree (i.e., $O(N \log N)$ or $O(N \log^2 N)$). In the very worst case however, this later dependency becomes quadratic (i.e., $O(N^2)$ or $O(N^2 \log N)$), which might greatly affect performance in the context of large datasets. Reassuringly, the analysis of the average case shows that pathological cases are not dominant and that, on average, for all methods, complexity behaves as in the best case.

As expected, the complexity of Bagging [Breiman, 1994] is about M times as large as the complexity of building a single decision tree. Yet, by taking into account the fact that bagged decision trees are built on bootstrap replicates $\mathcal{L}^m$ that contain about 63.2% of unique samples, complexity can be expressed with respect to $\widetilde{N} = 0.632N$ instead of N. From an implementation point of view, repeated occurrences of the same input vector can indeed be accounted for using sample weights for the evaluation of the impurity criterion, thereby simulating a learning set of N samples from $\widetilde{N}$ unique samples while reducing the effective size of the learning set. Assuming close constant factors, building a single bagged decision tree is therefore asymptotically

$$\lim_{N \to \infty} \frac{N \log^2 N}{0.632N \log^2 0.632N} = 1.582 \quad (5.27)$$

times faster than building a regular decision tree. The complexity of RF is identical to Bagging, except that the dependency to p becomes a dependency to K, resulting in an average speedup of $\frac{p}{K}$. In other words, not only choosing $K \leqslant p$ improves accuracy, it also significantly decreases the time required for building trees in a forest. In ETs, due to the fact that samples are no longer required to be sorted at each node, the dependency to N becomes linearithmic instead of quasilinear. With respect to RF, speedup is asymptotically $O(\log N)$ in the best and average cases. Finally, PERT shows to be the fastest method of all, which is due to the fact that only a single split variable is considered at each split. For $K = 1$ however, ETs and PERT have identical time complexity. Note that the analysis presented here

$$= O(2N \log N + 2N\gamma + 1 + \frac{2}{N})$$
$$= O(N \log N). \quad (5.26)$$

Exatamente da mesma maneira, ao assumir que $C'(N) = c_2 N$, $T'(N)$ pode ser mostrado como sendo limitado inferiormente por $\Omega(N \log N)$. Como resultado, $T(N) = \Theta(KN \log N)$ no caso médio. □

A partir dos Teoremas 5.2-5.7, a complexidade de tempo total para construir um conjunto de M árvores aleatórias pode finalmente ser derivada. Conforme resumido na Tabela 5.1, a complexidade permanece dentro do tempo polinomial para todas as variantes estudadas neste trabalho. Na melhor das hipóteses, a complexidade é linear com o número de variáveis de divisão consideradas em cada nó (ou seja, $O(K)$) e linearítmica ou quase linear com o número de amostras (únicas) efetivamente usadas para construir cada árvore (ou seja, $O(N \log N)$ ou $O(N \log^2 N)$). No pior dos casos, porém, essa dependência posterior se torna quadrática (ou seja, $O(N^2)$ ou $O(N^2 \log N)$), o que pode afetar bastante o desempenho no contexto de grandes conjuntos de dados. De forma tranquilizadora, a análise do caso médio mostra que os casos patológicos não são dominantes e que, em média, para todos os métodos, a complexidade se comporta como no melhor caso.

Como esperado, a complexidade do Bagging [Breiman, 1994] é quase M vezes maior que a complexidade de construir uma única árvore de decisão. No entanto, levando em consideração o fato de que as árvores de decisão empacotadas são construídas em réplicas de bootstrap $\mathcal{L}^m$ que contêm cerca de 63,2% de amostras únicas $\widetilde{N}$, a complexidade pode ser expressa em relação a $N = 0.632N$ em vez de N. Do ponto de vista da implementação, ocorrências repetidas do mesmo vetor de entrada podem de fato ser contabilizadas usando pesos amostrais para a avaliação do critério de impureza, simulando assim um conjunto de aprendizagem de N amostras de N amostras únicas enquanto reduz o tamanho efetivo do conjunto de aprendizagem. Assumindo fatores constantes próximos, construir uma única árvore de decisão ensacada é, portanto, assintoticamente

$$\lim_{N \to \infty} \frac{N \log^2 N}{0.632N \log^2 0.632N} = 1.582 \quad (5.27)$$

vezes mais rápido do que construir uma árvore de decisão normal. A complexidade do RF é idêntica à do Bagging, exceto que a dependência de p se torna uma dependência de K, resultando em uma aceleração média de $\frac{p}{K}$. Em outras palavras, escolher $K \leqslant p$ não apenas melhora a precisão, mas também diminui significativamente o tempo necessário para construir árvores em uma floresta. Em ETs, devido ao fato de que as amostras não precisam mais ser classificadas em cada nó, a dependência de N torna-se linearítmica em vez de quase linear. Com relação à RF, a aceleração é assintoticamente $O(\log N)$ nos casos melhores e médios. Finalmente, o PERT mostra-se o método mais rápido de todos, o que se deve ao facto de apenas uma única variável de divisão ser considerada e m cada divisão. Para $K = 1$ no entanto, ETs e PERT têm complexidade de tempo idêntica. Observe que a análise aqui apresentada

	Best case	*Worst case*	*Average case*
CART	$\Theta(pN\log^2 N)$	$O(pN^2\log N)$	$\Theta(pN\log^2 N)$
Bagging	$\Theta(Mp\widetilde{N}\log^2\widetilde{N})$	$O(Mp\widetilde{N}^2\log\widetilde{N})$	$\Theta(Mp\widetilde{N}\log^2\widetilde{N})$
RF	$\Theta(MK\widetilde{N}\log^2\widetilde{N})$	$O(MK\widetilde{N}^2\log\widetilde{N})$	$\Theta(MK\widetilde{N}\log^2\widetilde{N})$
ETs	$\Theta(MKN\log N)$	$\Theta(MKN^2)$	$\Theta(MKN\log N)$
PERT	$\Theta(MN\log N)$	$\Theta(MN^2)$	$\Theta(MN\log N)$

Table 5.1: Time complexity for building forests of M randomized trees. N denotes the number of samples in $\mathcal{L}$, p the number of input variables and K the number of variables randomly drawn at each node. $\widetilde{N} = 0.632N$, due to the fact that bootstrap samples draw, on average, 63.2% of unique samples.

is only valid asymptotically. In practice, constant factors might lead to different observed results, though they should not significantly deviate from our conclusions if algorithms are all implemented from a common code-base.

5.2 COMPLEXITY OF THE PREDICTION PROCEDURE

A second facet of computational complexity in random forests is the time required for making predictions. For a single test point, this represents the number of operations for traversing each of the M trees, from the root to one of the leaves. As such, computational complexity is directly related to the average depth of the leaves reached by the test samples. Assuming that both the learning set and the test set are drawn from the same probability distribution, the complexity analysis of the prediction procedure therefore reduces to the analysis of the average depth of the leaves induced by a learning set $\mathcal{L}$.

As in Section 5.1, let us assume a fully developed decision tree, where each leaf contains exactly one sample from $\mathcal{L}$. Likewise let us derive the average depth $D(N)$ of the leaves in the best, worst and average cases.

Theorem 5.8. *In the best case, the average depth of the leaves is* $\Theta(\log N)$.

Proof. For a decision tree which is perfectly balanced, node samples are recursively split into two subsets of $\frac{N_t}{2}$ samples at each node, which leads to the following recurrence equation in the best case:

$$\begin{cases} D(1) = 1 \\ D(N) = 1 + \frac{1}{2}D(\frac{N}{2}) + \frac{1}{2}D(\frac{N}{2}) = 1 + D(\frac{N}{2}). \end{cases} \tag{5.28}$$

With this equation, the second case of the Master Theorem applies for $f(N) = 1$, $a = 1$, $b = 2$ and $k = 0$. Accordingly, $D(N) = \Theta(\log^{k+1} N) = \Theta(\log N)$. □

	Melhor caso	*Pior caso*	*Caso médio*
CART	$\Theta(pN\text{registro}^2 N)$	$O(pN^2\text{ registro }N)$	$\Theta(pN\text{registro}^2 N)$
Ensacamento	$\Theta(Mp\widetilde{N}\text{ registro}^2\widetilde{N})$	$O(Mp\widetilde{N}^2\text{ registro }\widetilde{N})$	$\Theta(Mp\widetilde{N}\text{ registro}^2\widetilde{N})$
RF	$\Theta(MK\widetilde{N}\text{ registro}^2\widetilde{N})$	$O(MK\widetilde{N}^2\text{ registro }\widetilde{N})$	$\Theta(MK\widetilde{N}\text{ registro}^2\widetilde{N})$
ETs	$\Theta(MKN\text{ registro }N)$	$\Theta(MKN^2)$	$\Theta(MKN\text{ registro }N)$
PERT	$\Theta(MN\text{ registro }N)$	$\Theta(MN^2)$	$\Theta(MN\text{ registro }N)$

Tabela 5.1: Complexidade de tempo para construção de florestas de M árvores aleatórias. N denota o número de amostras em $\mathcal{L}$, p o número de variáveis de entrada e K o número de variáveis sorteadas aleatoriamente em cada nó. $\widetilde{N} = 0.632N$, devido ao fato de que amostras bootstrap extraem, em média, 63,2% de amostras únicas.

só é válido assintoticamente. Na prática, fatores constantes podem levar a diferentes resultados observados, embora não devam desviar-se significativamente das nossas conclusões se os algoritmos forem todos implementados a partir de uma base de código comum.

5.2 complexidade do procedimento de previsão

Uma segunda faceta da complexidade computacional em florestas aleatórias é o tempo necessário para fazer previsões. Para um único ponto de teste, isso representa o número de operações para percorrer cada uma das árvores M, da raiz até uma das folhas. Assim, a complexidade computacional está diretamente relacionada à profundidade média das folhas atingidas pelas amostras de teste. Assumindo que tanto o conjunto de aprendizagem como o conjunto de teste são extraídos da mesma distribuição de probabilidade, a análise da complexidade do procedimento de predição reduz-se, portanto, à análise da profundidade média das folhas induzida por um conjunto de aprendizagem $\mathcal{L}$.

Como na Seção 5.1, vamos assumir uma árvore de decisão totalmente desenvolvida, onde cada folha contém exatamente uma amostra de $\mathcal{L}$. Da mesma forma, vamos derivar a profundidade média $D(N)$ das folhas nos melhores, piores e médios casos.

Teorema 5.8. Na melhor das hipóteses, a profundidade média das folhas é $\Theta(\log N)$.

Prova. Para uma árvore de decisão perfeitamente balanceada, as amostras de nós são divididas recursivamente em dois subconjuntos de amostras $\frac{N_t}{2}$ em cada nó, o que leva à seguinte equação de recorrência no melhor caso:

$$\begin{cases} D(1) = 1 \\ D(N) = 1 + \frac{1}{2}D(\frac{N}{2}) + \frac{1}{2}D(\frac{N}{2}) = 1 + D(\frac{N}{2}). \end{cases} \tag{5.28}$$

Com esta equação, o segundo caso do Teorema Mestre se aplica para $f(N) = 1$, $a = 1$, $b = 2$ e $k = 0$. Da mesma forma, $D(N) = \Theta(\log^{k+1} N) = \Theta(\log N)$.

□

Theorem 5.9. *In the worst case, the average depth of the leaves is* $\Theta(N)$.

Proof. In the worst case, the recurrence equation of the average depth of the leaves is:

$$\begin{cases} D(1) = 1 \\ D(N) = 1 + \frac{1}{N} D(1) + \frac{N-1}{N} D(N-1). \end{cases} \tag{5.29}$$

Let us now introduce $S(N) = ND(N)$. From the previous equation, it comes

$$\begin{aligned} S(N) &= N + 1 + S(N-1) \\ &= 1 + \sum_{i=2}^{N} (i+1) \\ &= \frac{N^2}{2} + \frac{3N}{2} - 1. \end{aligned} \tag{5.30}$$

Since $D(N) = \frac{S(N)}{N}$, we have $D(N) = \frac{1}{2}(N - \frac{2}{N} + 3) = \Theta(N)$. □

Theorem 5.10. *In the average case, the average depth of the leaves is* $\Theta(\log N)$.

Proof. In the average case, the recurrence equation of the average depth of the leaves is:

$$\begin{cases} D(1) = 1 \\ D(N) = 1 + \frac{1}{N-1} \sum_{i=1}^{N-1} (\frac{i}{N} D(i) + \frac{N-i}{N} D(N-i)). \end{cases} \tag{5.31}$$

By symmetry and by multiplying both sides by $N(N-1)$, it comes

$$N(N-1)D(N) = N(N-1) + 2 \sum_{i=1}^{N-1} iD(i). \tag{5.32}$$

For $N \geqslant 3$, substituting N with $N-1$ similarly yields

$$(N-1)(N-2)D(N-1) = (N-1)(N-2) + 2 \sum_{i=1}^{N-2} iD(i). \tag{5.33}$$

Subtracting Equation 5.33 from 5.32, it comes after simplifications and division of both sides by $N(N-1)$

$$\begin{aligned} D(N) &= D(N-1) + \frac{2}{N} \\ &= 1 + 2 \sum_{i=2}^{N} \frac{1}{i} \\ &= 2H_N - 1 \end{aligned} \tag{5.34}$$

where H_N is the N-th harmonic number. Using

$$H_N \approx \log N + \gamma + \frac{1}{2N} + O(\frac{1}{N^2}) \tag{5.35}$$

as approximation (where γ is the Euler-Mascheroni constant), we have $D(N) = \Theta(\log N)$. □

Teorema 5.9. Na pior das hipóteses, a profundidade média das folhas é $\Theta(N)$.

Prova. No pior caso, a equação de recorrência da profundidade média das folhas é:

$$\begin{cases} D(1) = 1 \\ D(N) = 1 + \frac{1}{N} D(1) + \frac{N-1}{N} D(N-1). \end{cases} \tag{5.29}$$

Vamos agora apresentar $S(N) = ND(N)$. Da equação anterior, vem

$$\begin{aligned} S(N) &= N + 1 + S(N-1) \\ &= 1 + \sum_{i=2}^{N} (i+1) \\ &= \frac{N^2}{2} + \frac{3N}{2} - 1. \end{aligned} \tag{5.30}$$

Desde $D(N) = \frac{S(N)}{N}$, temos $D(N) = \frac{1}{2}(N - \frac{2}{N} + 3) = \Theta(N)$. □

Teorema 5.10. No caso médio, a profundidade média das folhas é $\Theta(\log N)$.

Prova. No caso médio, a equação de recorrência da profundidade média das folhas é:

$$\begin{cases} D(1) = 1 \\ D(N) = 1 + \frac{1}{N-1} \sum_{i=1}^{N-1} (\frac{i}{N} D(i) + \frac{N-i}{N} D(N-i)). \end{cases} \tag{5.31}$$

Por simetria e multiplicando ambos os lados por $N(N-1)$, vem

$$N(N-1)D(N) = N(N-1) + 2 \sum_{i=1}^{N-1} iD(i). \tag{5.32}$$

Para $N \geqslant 3$, substituindo N por $N-1$ de forma semelhante produz

$$(N-1)(N-2)D(N-1) = (N-1)(N-2) + 2 \sum_{i=1}^{N-2} iD(i). \tag{5.33}$$

Subtraindo a Equação 5.33 de 5.32, vem após simplificações e divisão de ambos os lados por $N(N-1)$

$$\begin{aligned} D(N) &= D(N-1) + \frac{2}{N} \\ &= 1 + 2 \sum_{i=2}^{N} \frac{1}{i} \\ &= 2H_N - 1 \end{aligned} \tag{5.34}$$

onde H_N é o N-ésimo número harmônico. Usando

$$H_N \approx \log N + \gamma + \frac{1}{2N} + O(\frac{1}{N^2}) \tag{5.35}$$

como aproximação (onde γ é a constante de Euler-Mascheroni), temos $D(N) = \Theta(\log N)$. □

From Theorems 5.8-5.10, the total time complexity for computing predictions out of a forest of M trees is therefore $\Theta(M \log N)$ in the best case and $\Theta(MN)$ is the very worst case. In accordance with previous results, the analysis of the average case shows however that pathological cases are not dominant and that, on average, complexity behaves once again as in the best case.

Randomized trees are not equiprobable

When trees are built totally at random, that is when all sizes of partitions are equiprobable, the analysis of the average case shows that the average depth of the leaves remains logarithmic with size of the learning set. Assuming unique input values for all samples and considering the special case $p = 1$, the random construction of a decision tree is in fact equivalent to the insertion in random order of N unique keys into a binary search tree. As shown exactly in [Sedgewick and Flajolet, 2013], the average depth of a binary search tree built from equiprobable permutations of N unique keys is $O(\log N)$, which corroborates our results. By contrast, if trees were drawn uniformly at random from the set of all possible trees (i.e., in the case of *Catalan* trees), then the average depth can be shown to be $O(\sqrt{N})$. The fundamental difference is that binary search trees built from random permutations are *not* all equiprobable. There may exist several permutations mapping to the same binary search tree. In particular, short trees are more likely to occur than degeneratedly deep trees. Accordingly, decision trees, even built totally at random, are also not all equiprobable. Because of the recursive construction procedure, deep degenerated decision trees are less likely than shorter decision trees.

5.3 IMPLEMENTATION

Implementing decision trees and random forests involves many issues that are easily overlooked if not considered with care. In this section, we describe the random forest implementation that has been developed in this work and deployed within the Scikit-Learn machine learning library [Pedregosa et al., 2011]. *The first part of this section is based on previous work published in [Buitinck et al., 2013].*

5.3.1 *Scikit-Learn*

The Scikit-Learn library provides a comprehensive suite of machine learning tools for Python. It extends this general-purpose programming language with machine learning operations: learning algorithms, preprocessing tools, model selection procedures and a composition mechanism to produce complex machine learning work-flows. The ambition of the library is to provide efficient implementations of well-

A partir dos Teoremas 5.8-5.10, a complexidade de tempo total para calcular previsões de uma floresta de M árvores é, portanto, $\Theta(M \log N)$ no melhor caso e $\Theta(MN)$ é o pior caso. De acordo com resultados anteriores, a análise do caso médio mostra, no entanto, que os casos patológicos não são dominantes e que, em média, a complexidade comporta-se mais uma vez como no melhor caso.

Árvores aleatórias não são equiprováveis

Quando as árvores são construídas de forma totalmente aleatória, ou seja, quando todos os tamanhos de partições são equiprováveis, a análise do caso médio mostra que a profundidade média das folhas permanece logarítmica com o tamanho do conjunto de aprendizagem. Assumindo valores de entrada únicos para todas as amostras e considerando o caso especial $p = 1$, a construção aleatória de uma árvore de decisão é de facto equivalente à inserção em ordem aleatória de N chaves únicas numa árvore de pesquisa binária. Como mostrado exatamente em [Sedgewick e Flajolet, 2013], a profundidade média de uma árvore de busca binária construída a partir de permutações equiprováveis de N chaves únicas é $O(\log N)$, o que corrobora nossos resultados. Por outro lado, se as árvores fossem extraídas uniformemente e aleatoriamente do conjunto de todas as árvores possíveis (ou seja, no caso das árvores catalãs), então a profundidade média pode ser mostrada como $O(\sqrt{N})$. A diferença fundamental é que as árvores de busca binária construídas a partir de permutações aleatórias não são todas equiprováveis. Podem existir diversas permutações mapeadas para a mesma árvore binária de busca. Em particular, árvores baixas são mais prováveis de ocorrer do que árvores degeneradas e profundas. Conseqüentemente, as árvores de decisão, mesmo construídas de forma totalmente aleatória, tambem não são todas equiprováveis. Devido ao procedimento de construção recursivo, árvores de decisão degeneradas profundas são menos prováveis do que árvores de decisão mais curtas.

5.3 implementação

A implementação de árvores de decisão e florestas aleatórias envolve muitas questões que são facilmente ignoradas se não forem consideradas com cuidado. Nesta seção, descrevemos a implementação de floresta aleatória que foi desenvolvida neste trabalho e implantada na biblioteca de aprendizado de máquina Scikit-Learn [Pedregosa et al., 2011]. A primeira parte desta seção é baseada em trabalhos anteriores publicados em *[Buitinck et al., 2013]*.

5.3.1 Scikit-Learn

A biblioteca Scikit-Learn fornece um conjunto abrangente de ferramentas de aprendizado de máquina para Python. Ele estende esta linguagem de programação de uso geral com operações de aprendizado de máquina: algoritmos de aprendizado, ferramentas de pré-processamento, procedimentos de seleção de modelos e um mecanismo de composição para produzir fluxos de trabalho complexos de aprendizado de máquina. A ambição da biblioteca é fornecer implementações eficientes de

established algorithms within a programming environment that is accessible to non-experts and reusable in various scientific areas. The library is distributed under the simplified BSD license to encourage its use in both academia and industry.

Scikit-Learn is designed to tie in with the set of numeric and scientific packages centered around the NumPy [Oliphant, 2007] and SciPy [Van der Walt et al., 2011] libraries. NumPy augments Python with a contiguous numeric array datatype and fast array computing primitives, while SciPy extends it further with common numerical operations, either by implementing these in Python/NumPy or by wrapping existing C/C++/Fortran code. The name "scikit" derives from "SciPy toolkit" and is shared with *scikit-image*. IPython [Perez and Granger, 2007] and Matplotlib [Hunter, 2007] complement SciPy to provide a MATLAB-like working environment suited for scientific use.

Since 2007, Scikit-Learn has been developed by more than a dozen core developers, mostly researchers in fields ranging from neuroscience to astrophysics. It also benefits from occasional contributors proposing small bug-fixes or improvements. Development proceeds on GitHub[1], a platform which greatly facilitates this kind of collaboration. Because of the large number of developers, emphasis is put on keeping the project maintainable. In particular, code must follow specific quality guidelines, such as style consistency and unit-test coverage. Documentation and examples are required for all features, and major changes must pass code review by at least two other developers. The popularity of the project can be gauged from various indicators such as the hundreds of citations in scientific publications, successes in various machine learning challenges (e.g., Kaggle), and statistics derived from our repositories and mailing list. At the time of writing[2], the project is watched by 3,445 people and forked 1,867 times on GitHub; the mailing list receives more than 300 mails per month; version control logs show more than 200 unique contributors to the code-base and the online documentation receives 37,000 unique visitors and 295,000 pageviews per month.

Our implementation guidelines emphasize writing efficient but readable code. In particular, we focus on making the code-base maintainable and understandable in order to favor external contributions. Whenever practical, algorithms implemented in Scikit-Learn are written in Python, using NumPy vector operations for numerical work. This allows the code to remain concise, readable and efficient. For critical algorithms that cannot be easily and efficiently expressed as NumPy operations, we rely on Cython [Behnel et al., 2011] to achieve competitive performance and scalability. Cython is a compiled programming language that extends Python with static typing. It pro-

1 https://github.com/scikit-learn
2 July 2014.

algoritmos estabelecidos num ambiente de programação acessível a leigos e reutilizável em diversas áreas científicas. A biblioteca é distribuída sob a licença BSD simplificada para incentivar seu uso tanto na academia quanto na indústria.

O Scikit-Learn foi projetado para se conectar ao conjunto de pacotes numéricos e científicos centrados nas bibliotecas NumPy [Oliphant 2007] e SciPy [Van der Walt et al., 2011]. NumPy aumenta o Python com um tipo de dados numérico contíguo e primitivas de computação fastarray, enquanto o SciPy o estende ainda mais com operações numéricas comuns, seja implementando-as em Python/NumPy ou agrupando código C/C++/Fortran existente. O nome "scikit" deriva de "kit de ferramentas SciPy" e é compartilhado com scikit-image. IPython [Perez e Granger, 2007] e Matplotlib [Hunter, 2007] complementam o SciPy para fornecer um ambiente de trabalho semelhante ao matlab adequado para uso científico.

Desde 2007, o Scikit-Learn foi desenvolvido por mais de uma dúzia de desenvolvedores principais, principalmente pesquisadores em áreas que vão da neurociência à astrofísica. Também se beneficia de contribuidores ocasionais que propõem pequenas correções de bugs ou melhorias. O desenvolvimento continua no GitHub[1],, uma plataforma que facilita muito esse tipo de colaboração. Devido ao grande número de desenvolvedores, a ênfase é colocada em manter o projeto sustentável. Em particular, o código deve seguir diretrizes específicas de qualidade, como consistência de estilo e cobertura de testes unitários. Documentação e exemplos são necessários para todos os recursos, e mudanças importantes devem passar pela revisão do código por pelo menos dois outros desenvolvedores. A popularidade do projeto pode ser avaliada a partir de vários indicadores, como centenas de citações em publicações científicas, sucessos em vários desafios de aprendizado de máquina (por exemplo, Kaggle) e estatísticas derivadas de nossos repositórios e lista de discussão. No momento em que este artigo foi escrito,[2], o projeto era assistido por 3.445 pessoas e bifurcado 1.867 vezes no GitHub; a mailing list recebe mais de 300 mails por mês; os logs de controle de versão mostram mais de 200 contribuidores únicos para a base de código e a documentação on-line recebe 37.000 visitantes únicos e 295.000 visualizações de página por mês.

Nossas diretrizes de implementação enfatizam a escrita de código eficiente, mas legível. Em particular, nos concentramos em tornar a base de código sustentável e compreensível, a fim de favorecer contribuições externas. Sempre que prático, os algoritmos implementados no Scikit-Learn são escritos em Python, usando operações vetoriais NumPy para trabalho numérico. Isso permite que o código permaneça conciso, legível e eficiente. Para algoritmos críticos que não podem ser expressos de maneira fácil e eficiente como operações NumPy, contamos com Cython [Behnel etal., 2011] para alcançar desempenho competitivo e escalabilidade. Cython é uma linguagem de programação compilada que estende Python com tipagem estática. É pró-

1 https://github.com/scikit-learn 2 de julho de 2014.

duces efficient C extension modules that are importable from the Python run-time system. Examples of Cython code in Scikit-Learn are stochastic gradient descent for linear models, some clustering algorithms, and decision trees.

5.3.1.1 *Data*

Machine learning revolves around data, so good data structures are paramount to designing software for it. In most tasks, data is modeled by a set of p numerical variables, so that a single *sample* is a vector $\mathbf{x} \in \mathbb{R}^p$. A collection of such samples is naturally regarded as the rows in a matrix $\mathbf{X}$ of size $N \times p$. In the common case of supervised learning (classification, regression), we have an additional vector $\mathbf{y}$ of length N at training time and want an algorithm to produce such a $\mathbf{y}$ for new data.

Scikit-Learn's data representation is kept as close as possible to this matrix formulation: datasets are encoded as two-dimensional NumPy arrays or SciPy sparse matrices [Van der Walt et al., 2011], while target vectors are flat arrays of numbers or strings. While these may seem rather unsophisticated when compared to more object-oriented constructs, such as the ones used by Weka [Hall et al., 2009], they allow us to rely on efficient NumPy and SciPy vector operations while keeping the code close to the textbook. Given the pervasiveness of NumPy and SciPy in many other scientific Python packages, many scientific users of Python will already be familiar with these data structures, and a collection of available data loading and conversion tools facilitate interoperability. For tasks where the inputs are text files or semi-structured objects, we provide *vectorizer* objects that efficiently convert such data to the NumPy or SciPy formats.

The public interface is oriented towards processing a batch of samples, rather than a single sample, in each API call. While classification and regression algorithms can make predictions for single samples, Scikit-Learn objects are not optimized for this use case. (The online learning algorithms in the library are intended to take mini-batches.) Batch processing makes optimal use of NumPy and SciPy by preventing the overhead inherent to Python function calls or due to per-element dynamic type checking. Although this might seem to be an artifact of the Python language, and therefore an implementation detail that leaks into the API, we argue that APIs should be designed so as not to tie a library to a suboptimal implementation strategy. Batch processing also enables fast implementations in lower-level languages, where memory hierarchy effects and the possibility of internal parallelization come into play.

produz módulos de extensão C eficientes que são importáveis do sistema de tempo de execução Python. Exemplos de código Cython no Scikit-Learn são descida gradiente estocástica para modelos lineares, alguns algorit mos de agrupamento e árvores de decisão.

5.3.1.1 Dados

O aprendizado de máquina gira em torno de dados, portanto, boas estruturas de dados são fundamentais para projetar software para ele. Na maioria das tarefas, os dados são modelados por um conjunto de p variáveis numéricas, de modo que uma única amostra é um vetor $\mathbf{x} \in \mathbb{R}^p$. Uma coleção de tais amostras é naturalmente considerada como as linhas de uma matriz X de tamanho $N \times p$. No caso comum de aprendizagem supervisionada (classificação, regressão), temos um vetor y adicional de comprimento N no momento do treinamento e queremos que um algoritmo produza tal y para novos dados.

A representação de dados do Scikit-Learn é mantida o mais próximo possível desta formulação de matriz: os conjuntos de dados são co dificados como matrizes NumPy bidimensionais ou matrizes esparsas SciPy [Van der Walt etal., 2011], enquanto os vetores alvo são matrizes planas de números ou strings. Embora possam parecer pouco sofisticados quando comparados com construções mais orientadas a objetos, como as usadas por Weka [Hall et al., 2009], elas nos permitem confiar em operações eficientes de vetores NumPy e SciPy, mantendo o código próximo ao livro didático. Dada a difusão do NumPy e do SciPy em muitos outros pacotes científicos do Python, muitos usuários científicos do Python já estarão familiarizados com essas estruturas de dados, e uma coleção de ferramentas de carregamento e conversão de dados disponíveis facilita a interoperabilidade. Para tarefas onde as entradas são arquivos de texto ou objetos semiestruturados, fornecemos objetos vetorizadores que convertem esses dados com eficiência para os formatos NumPy ou SciPy.

A interface pública é orientada para processar um lote de amostras, em vez de uma única amostra, em cada chamada de API. Embora os algoritmos de classificação e regressão possam fazer previsões para amostras únicas, os objetos do Scikit-Learn não são otimizados para este caso de uso. (Os algoritmos de aprendizagem on-line na biblioteca destinam-se a receber minilotes.) O processamento em lote faz uso ideal de NumPy e SciPy, evitando a sobrecarga inerente às chamadas de função Python ou devido à verificação dinâmica de tipo por elemento. Embora i sso possa parecer um artefato da linguagem Python e, portanto, um detalhe de implementação que vaza para a API, argumentamos que as APIs devem ser projetadas de modo a não vincular uma biblioteca a uma estratégia de implementação abaixo do ideal. O processamento em lote também permite implementações rápidas em linguagens de nível inferior, onde os efeitos da hierarquia de memória e a possibilidade de paralelização interna entram em jogo.

5.3.1.2 *Estimators*

The *estimator* interface is at the core of the library. It defines instantiation mechanisms of objects and exposes a `fit` method for learning a model from training data. All supervised and unsupervised learning algorithms (e.g., for classification, regression or clustering) are offered as objects implementing this interface. Machine learning tasks like feature extraction and selection are also provided as estimators.

Estimator initialization and actual learning are strictly separated, in a way that is similar to partial function application: an estimator is initialized from a set of named hyper-parameter values (e.g., the number of trees in a forest) and can be considered a function that maps these values to actual learning algorithms. The constructor does not see any actual data. All it does is attach the given parameters to the object. For the sake of model inspection, hyper-parameters are set as public attributes, which is especially important in model selection settings. Default hyper-parameter values are provided for all built-in estimators. These defaults are set to be relevant in many common situations in order to make estimators effective *out-of-the-box*.

Actual learning is performed by the `fit` method. This method is called with training data (e.g., supplied as two arrays `X_train` and `y_train` in supervised learning estimators). Its task is to run a learning algorithm and to determine model-specific parameters from the training data and set these as attributes on the estimator object. As a convention, the parameters learned by an estimator are exposed as public attributes with names suffixed with a trailing underscore (e.g., `coef_` for the learned coefficients of a linear model), again to facilitate model inspection. In the partial application view, `fit` is a function from data to a model of that data. It always returns the estimator object it was called on, which now serves as a model and can be used to perform predictions or transformations of new data.

The choice to let a single object serve dual purpose as estimator and model has been driven by usability and technical considerations. Having two coupled instances (an estimator object used as a factory, and a model object produced by the estimator) makes a library harder to learn and use. From the developer point of view, decoupling estimators from models would create parallel class hierarchies and increases the overall maintenance burden. A good reason for decoupling would be to make it possible to ship a model to a new environment where the full library cannot be installed. However, our inspectable setup where model parameters are documented public attributes and prediction formulas follow standard textbooks, goes a long way in solving this problem.

To illustrate the initialize-fit sequence, let us consider a supervised learning task using a single decision tree. Given the API defined above, solving this problem is as simple as the following example.

5.3.1.2 Estimadores

A interface do estimador está no centro da biblioteca. Ele define mecanismos de instanciação de objetos e expõe um método adequado para aprender um modelo a partir de dados de treinamento. Todos os algoritmos de aprendizage m supervisionados e não supervisionados (por exemplo, para classificação, regressão ou agrupamento) são oferecidos como objetos que implementam esta interface. Tarefas de aprendizado de máquina, como extração e seleção de recursos, também são fornecidas como estimadores.

A inicialização do estimador e o aprendizado real são estritamente separados, de uma forma semelhante à aplicação de função parcial: um estimador é inicializado a partir de um conjunto de valores de hiperparâmetros nomeados (por exemplo, o número de árvores em uma floresta) e pode ser considerado uma função que mapeia esses valores para algoritmos de aprendizado reais. O construtor não vê nenhum dado real. Tudo o que ele faz é anexar os parâmetros fornecidos ao objeto. Para fins de i nspeção do modelo, os hiperparâmetros são definidos como atributos públicos, o que é especialmente importante nas configurações de seleção de modelos. Valores padrão de hiperparâmetros são fornecidos para todos os estimadores integrados. Esses padrões são definidos para serem relevantes em muitas situações comuns, a fim de tornar os estimadores eficazes desde o início.

A aprendizagem real é realizada pelo método fit. Este método é chamado com dados de treinamento (por exemplo, fornecidos como duas matrizes X_train e y_train em estimadores de aprendizagem supervisionada). Sua tarefa é executar um algoritmo de aprendizado e determinar parâmetros específicos do modelo a partir dos dados de treinamento e defini-los como atributos no objeto estimador. Como convenção, os parâmetros aprendidos por um estimador são expostos c omo atributos públicos com nomes sufixados com um sublinhado à direita (por exemplo, coef_ para os coeficientes aprendidos de um modelo linear), novamente para facilitar a inspeção do modelo. Na visão parcial da aplicação, ajuste é uma função dos dados para um modelo desses dados. Ele sempre retorna o objeto estimador ao qual foi chamado, que agora serve como modelo e pode ser usado para realizar previsões ou transformações de novos dados.

A escolha de deixar um único objeto servir a dupla finalidade como estimador e modelo foi motivada pela usabilidade e por considerações técnicas. Ter duas instâncias acopladas (um objeto estimador usado como fábrica e um objeto modelo produzido pelo estimador) torna uma biblioteca mais difícil de aprender e usar. Do ponto de vista do desenvolvedor, dissociar os estimadores dos modelos criaria hierarquias de classes paralelas e aumentaria a carga geral de manutenção. Uma boa razão para a dissociação seria tornar possível enviar um modelo para um novo ambiente onde a biblioteca completa não possa ser instalada. No entanto, nossa configuração inspecionável, onde os parâmetros do modelo são atributos públicos documentados e as fórmulas de previsão seguem livros-texto padrão, contribui muito para resolver esse problema.

Para ilustrar a sequência de inicialização-ajuste, consideremos uma tarefa de aprendizagem supervisionada usando uma única árvore de decisão. Dada a API definida acima, resolver este problema é tão simples quanto o exemplo a seguir.

```
# Import the tree module
from sklearn.tree import DecisionTreeClassifier
# Instantiate and set hyper-parameters
clf = DecisionTreeClassifier(min_samples_split=5)
# Learn a model from data
clf.fit(X_train, y_train)
```

In this snippet, a `DecisionTreeClassifier` estimator is first initialized by setting the `min_samples_split` hyper-parameter to 5 (See Section 3.5). Upon calling `fit`, a model is learned from the training arrays `X_train` and `y_train`, and stored on the object for later use. Since all estimators share the same interface, using a different learning algorithm is as simple as replacing the constructor (the class name). To build a random forest on the same data, one would simply replace `DecisionTreeClassifier(min_samples_split=5)` in the snippet above by `RandomForestClassifier()`.

In Scikit-Learn, classical learning algorithms are not the only objects to be implemented as estimators. For example, preprocessing routines (e.g., scaling of features) or feature extraction techniques (e.g., vectorization of text documents) also implement the *estimator* interface. Even stateless processing steps, that do not require the `fit` method to perform useful work, implement the estimator interface. This design pattern is of prime importance for consistency, composition and model selection reasons, as further illustrated in [Buitinck et al., 2013].

5.3.1.3 *Predictors*

The *predictor* interface extends the notion of an estimator by adding a `predict` method that takes an array `X_test` and produces predictions for `X_test`, based on the learned parameters of the estimator (we call the input to `predict` "`X_test`" in order to emphasize that `predict` generalizes to new data). In the case of supervised learning estimators, this method returns the predicted labels or values computed by the model:

```
# Make predictions on new data
y_pred = clf.predict(X_test)
```

Apart from `predict`, predictors may also implement methods that quantify the confidence of predictions. In the case of linear models, the `decision_function` method returns the distance of samples to the separating hyperplane. Some predictors also provide a `predict_proba` method which returns class probability estimates.

Finally, supervised predictors provide a `score` function to assess their performance on a batch of input data. This method takes as input arrays `X_test` and `y_test` and typically computes the coefficient of determination between `y_test` and `predict(X_test)` in regression, or the accuracy in classification. The only requirement is

Importe o módulo de árvore de sklearn.tree import DecisionTreeClassifier # Instancie e defina hiperparâmetros clf =DecisionTreeClassifier(min_samples_split=5) # Aprenda um modelo a partir de dados clf.fit(X_train, y_train)

Neste trecho, um estimador DecisionTreeClassifier é inicializado primeiro definindo o hiperparâmetro min_samples_split como 5 (consulte a Seção 3.5). Ao chamar fit, um modelo é aprendido a partir dos arrays de treinamento X_train e y_train e armazenado no objeto para uso posterior. Como todos os estimadores compartilham a mesma interface, usar um algoritmo de aprendizado diferente é tão simples quanto substituir o construtor (o nome da classe). Para construir uma floresta aleatória com os mesmos dados, bastaria substituir DecisionTreeClassifier(min_samples_split=5) no trecho acima por RandomForestClassifier().

No Scikit-Learn, algoritmos de aprendizagem clássicos não são os únicos objetos a serem implementados como estimadores. Por exemplo, rotinas de pré-processamento (por exemplo, dimensionamento de recursos) ou técnicas de extração de recursos (por exemplo, vetorização de documentos de texto) também implementam a interface do estimador. Mesmo as etapas de processamento sem estado, que não requerem o método fit para realizar um trabalho útil, implementam a interface do estimador. Este padrão de design é de primordial importância por razões de consistência, composição e seleção de modelo, conforme ilustrado em [Buitinck et al., 2013].

5.3.1.3 Preditores

A interface do preditor estende a noção de um estimador adicionando um método de previsão que pega uma matriz X_test e produz previsões para X_test, com base nos parâmetros aprendidos do estimador (chamamos a entrada para prever "X_test" para enfatizar que a previsão generaliza para novos dados). No caso de estimadores de aprendizagem supervisionada, este método retorna os rótulos ou valores previstos calculados pelo modelo:

Faça previsões sobre novos dados y_pred =clf.predict(X_test)

Além de prever, os preditores também podem implementar métodos que quantificam a confiança das previsões. No caso de modelos lineares, o método Decision_function retorna a distância das amostras ao hiperplano de separação. Alguns preditores também fornecem um método predict_proba que retorna estimativas de probabilidade de classe.

Finalmente, os preditores supervisionados fornecem uma função de pontuação para avaliar seu desempenho em um lote de dados de entrada. Este método toma como entrada os arrays X_test e y_test e normalmente calcula o coeficiente de determinação entre y_test e predizer(X_test) em regressão, ou a precisão na classificação. O único requisito é

that the `score` method return a value that quantifies the quality of its predictions (the higher, the better). An unsupervised estimator may also expose a `score` function to compute, for instance, data likelihood under its model.

5.3.1.4 *API for random forests*

Scikit-Learn provides efficient implementations of decision trees and random forests, all offered as objects implementing the estimator and predictor interfaces presented above. Most of the hyper-parameters described in this work are supported.

`DecisionTreeClassifier`, `DecisionTreeRegressor`:
: Implement single decision trees [Breiman et al., 1984], as described in Chapter 3.

`BaggingClassifier`, `BaggingRegressor`:
: Implement Bagging [Breiman, 1994], Random Subspaces [Ho, 1998] and Ensembles of Random Patches [Louppe and Geurts, 2012], as described in Chapters 4 and 8.

`RandomForestClassifier`, `RandomForestRegressor`:
: Implement Random Forest [Breiman, 2001], as described in Chapter 4.

`ExtraTreesClassifier`, `ExtraTreesRegressor`:
: Implement Extremely Randomized Trees [Geurts et al., 2006a], as described in Chapter 4.

5.3.2 *Internal data structures*

Among all possible ways of representing a decision tree, one of the simplest and most direct representations is to adopt an object-oriented approach. In this paradigm, a decision tree is naturally represented as a hierarchy of high-level objects, where each object corresponds to a node of the tree and comprises attributes referencing its children or storing its split and value. Such a representation would make for a correct, intuitive and flexible implementation but may in fact not be the most appropriate when aiming for high-performance. One of the biggest issues indeed is that object-oriented programming usually fragments complex and nested objects into non-contiguous memory blocks, thereby requiring multiple levels of indirection for traversing the structure. In particular, this design can easily impair computing times in performance-critical applications, e.g., by not making it possible to leverage CPU cache or pre-fetching mechanisms. At the price of less abstraction and flexibility, we adopt instead in this work a compact low-level representation of decision trees, allowing us for

que o método de pontuação retorne um valor que quantifique a qualidade de suas previsões (quanto maior, melhor). Um estimador não supervisionado também pode expor uma função de pontuação para calcular, por exemplo, a probabilidade dos dados em seu modelo.

5.3.1.4 API para florestas aleatórias

O Scikit-Learn fornece implementações eficientes de árvores de decisão e florestas aleatórias, todas oferecidas como objetos que implementam as interfaces de estimador e preditor apresentadas acima. A maioria dos hiperparâmetros descritos neste trabalho são suportados.

DecisionTreeClassifier, DecisionTreeRegressor:
: Implemente árvores de decisão únicas [Breiman et al., 1984], conforme descrito no Capítulo 3.

BaggingClassifier, BaggingRegressor: implemente Bagging [Breiman, 1994], subespaços aleatórios [Ho1998]e conjuntos de patches aleatórios [Louppe e Geurts,2012], conforme descrito nos capítulos 4 e 8.

RandomForestClassifier, RandomForestRegressor: Implemente Random Forest [Breiman, 2001], conforme descrito no Capítulo 4.

ExtraTreesClassifier, ExtraTreesRegressor: Implemente árvores extremamente aleatórias [Geurts et al., 2006a],conforme descrito no Capítulo 4.

5.3.2 Estruturas de dados internas

Dentre todas as formas possíveis de representar uma árvore de decisão, uma das representações mais simples e diretas é adotar uma abordagem orientada a objetos. Neste paradigma, uma árvore de decisão é naturalmente representada como uma hierarquia de objetos de alto nível, onde cada objeto corresponde a um nó da árvore e compreende atributos que referenciam seus filhos ou armazenam sua divisão e valor. Tal representação proporcionaria uma implementação correta, intuitiva e flexível, mas pode, na verdade, não ser a mais apropriada quando se pretende um alto desempenho. De fato, um dos maiores problemas é que a programação orientada a objetos geralmente fragmenta objetos co mplexos e aninhados em blocos de memória não contíguos, exigindo assi m vários níveis de indireção para percorrer a estrutura. Em particular, este design pode facilmente prejudicar os tempos de computação em aplicações críticas de desempenho, por exemplo, ao não tornar possível aproveitar o cache da CPU ou mecanismos de pré-busca. Ao preço de menos abstração e flexibilidade, adotamos neste trabalho uma representação compacta de baixo nível de árvores de decisão, permitindo-nos

a fine-grained and complete control over memory management and CPU mechanisms.

Let $t \in \{0, \dots, T-1\}$ denote unique node identifiers and let $t = 0$ corresponds to the identifier of the root node. The data structure that we propose for representing decision trees consists in using contiguous arrays for simultaneously storing the content of all nodes, as defined below:

`left_child` (array of T integers):
: Store the node identifier `left_child[t]` $\in \{0, \dots, T-1\}$ of the left child of node `t`.

`right_child` (array of T integers):
: Store the node identifier `right_child[t]` $\in \{0, \dots, T-1\}$ of the right child of node `t`.

`feature` (array of T integers):
: Store the identifier `feature[t]` $\in \{1, \dots, p\}$ of the variable used for splitting node `t`.

`threshold` (array of T reals):
: Store the cut-off value `threshold[t]` $\in \mathbb{R}$ used for splitting node `t`.

`impurity` (array of T reals):
: Store the impurity $i(t)$ at node `t`, as computed on the learning set $\mathcal{L}$.

`n_samples` (array of T integers or reals):
: Store the (weighted) number `n_samples[t]` of learning samples reaching node `t`.

`value` (array of $T \times J$ reals or T reals):
: Store the class probability estimates (i.e., the number of samples of each class) or the mean regression values, as computed from the learning samples reaching node `t`.

As an example, Table 5.2 illustrates the array representation of the decision tree shown in Figure 5.2, as built from the learning set $\mathcal{L}$ of Figure 5.1. Internal nodes (t_0, t_2, t_3 and t_6) are nodes for which the corresponding values in `left_child`, `right_child`, `feature` and `threshold` are not empty, while leaves (t_1, t_4, t_7 and t_8) correspond to nodes for which these values are not defined. In the case of classification, the `value` array contains the number of samples of each class for each node. From these, class probability estimates $p_{\mathcal{L}}(Y = c|t)$ can be computed by dividing `value[t][c]` by the number of samples `n_samples[t]` in t. In the case of regression, `value[t]` would contain the average output value for the samples in t. Finally, let us note that storing node impurities in the `impurity` array is not strictly

um controle completo e refinado sobre o gerenciamento de memória e mecanismos de CPU.

Seja $t \in \{0, \dots, T-1\}$ denota identificadores de nó exclusivos e seja $t = 0$ corresponde ao identificador do nó raiz. A estrutura de dados que propomos para representar árvores de decisão consiste na utilização de arrays contíguos para armazenar simultaneamente o conteúdo de todos os nós, conforme definido abaixo:

left_child (matriz de T inteiros):
: Armazene o identificador do nó left_child`[t]` $\in \{0, \dots, T-1\}$ do filho esquerdo do nó t.

right_child (matriz de T inteiros):
: Armazene o identificador do nó right_child`[t]` $\in \{0, \dots, T-1\}$ do filho direito do nó t.

recurso (matriz de T números inteiros):
: Armazene o recurso identificador`[t]` $\in \{1, \dots, p\}$ da variável usada para dividir o nó t.

limite (matriz de T reais):
: Armazene o limite do valor de corte`[t]` $\in \mathbb{R}$ usado para dividir o nó t.

impureza (matriz de T reais):
: Armazene a impureza $i(t)$ no nó t, conforme calculado no conjunto de aprendizagem $\mathcal{L}$.

n_samples (matriz de T inteiros ou reais):
: Armazene o número (ponderado) n_samples`[t]` de amostras de aprendizagem que atingem o nó t.

valor (matriz de $T \times J$ reais ou T reais):
: Armazene as estimativas de probabilidade de classe (ou seja, o número de amostras de cada classe) ou os valores médios de regressão, calculados a partir das amostras de aprendizagem que atingem o nó t.

Como exemplo, a Tabela 5.2 ilustra a representação de array da árvore de decisão mostrada na Figura 5.2, construída a partir do conjunto de aprendizagem $\mathcal{L}$da Figura 5.1. Nós internos (t_0, t_2, t_3 e t_6) são nós para os quais os valores correspondentes em left_child, right_child, recurso e limite não estão vazios, enquanto folhas (t_1, t_4, t_7 e t_8) correspondem a nós para os quais esses valores não estão definidos. No caso de classificação, a matriz de valores contém o número de amostras de cada classe para cada nó. A partir destes, as estimativas de probabilidade de classe $p_{\mathcal{L}}(Y = c|t)$ podem ser calculadas dividindo o valor`[t][c]` pelo número de amostras n_samples`[t]` em t. No caso de regressão, o valor`[t]` conteria o valor médio de saída para as amostras em t. Finalmente, observemos que armazenar impurezas de nós na matriz de impurezas não é estritamente

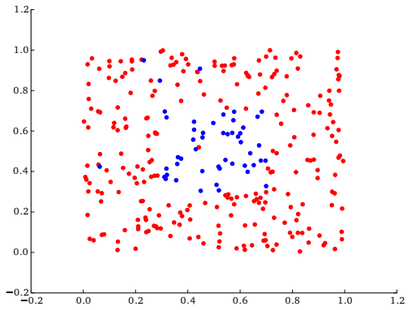


Figure 5.1: Binary classification task.

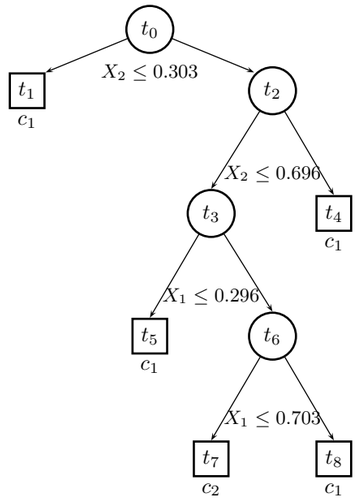


Figure 5.2: Decision tree learned from Figure 5.1.

t	0	1	2	3	4	5	6	7	8
left_child	1	–	3	5	–	–	7	–	–
right_child	2	–	4	6	–	–	8	–	–
feature	2	–	2	1	–	–	1	–	–
threshold	0.303	–	0.696	0.296	–	–	0.703	–	–
impurity	0.273	0.	0.368	0.482	0.065	0.051	0.48	0.042	0.
n_samples	300	99	201	113	88	38	75	46	29
value	251/49	99/0	152/49	67/46	85/3	37/1	30/45	1/45	29/0

Table 5.2: Array representation of the decision tree in Figure 5.2.

necessary, since these values are only used during construction. Yet, storing them can prove to be valuable, e.g., for later inspection of the decision tree or for computing variable importances (see Chapter 6) more readily.

By implementing the array representation as dynamic tables [Cormen et al., 2001], insertion of new nodes has an $O(1)$ amortized cost, while keeping memory allocations at a minimum and thereby reducing the overhead that would otherwise occur using per-node data structures. Maintaining the representation contiguously in memory also brings the benefits of leveraging CPU caches, which might greatly improve performance when fast predictions are critical. Since nodes are accessed successively and repeatedly at test time, storing them in nearby memory locations is indeed highly recommended.

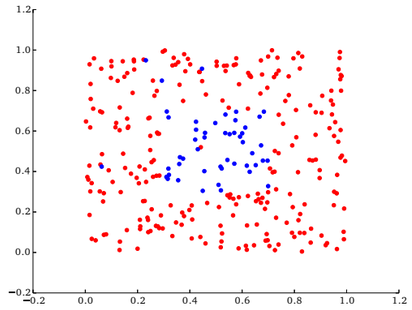


Figura 5.1: Tarefa de classificação binária.

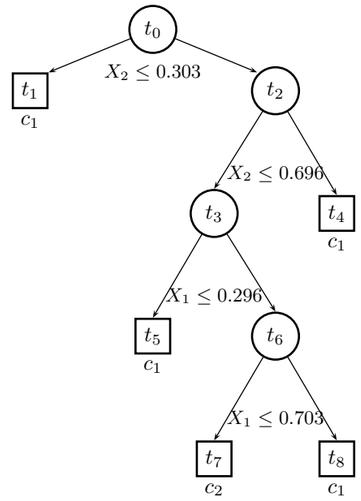


Figura 5.2: Árvore de decisão aprendida com a Figura 5.1.

t	0	1	2	3	4	5	6	7	8
filho_esquerdo	1	–	3	5	–	–	7	–	–
filho_direito	2	–	4	6	–	–	8	–	–
recurso	2	–	2	1	–	–	1	–	–
limite	0.303	–	0.696	0.296	–	–	0.703	–	–
impureza	0.273	0.	0.368	0.482	0.065	0.051	0.48	0.042	0.
n_amostras	300	99	201	113	88	38	75	46	29
valor	251/49	99/0	152/49	67/46	85/3	37/1	30/45	1/45	29/0

Tabela 5.2: Representação em array da árvore de decisão na Figura 5.2.

necessário, uma vez que estes valores são utilizados apenas durante a construção. No entanto, armazená-los pode ser valioso, por exemplo, para inspeção posterior da árvore de decisão ou para calcular importâncias variáveis (ver Capítulo 6) mais rapidamente.

Ao implementar a representação de array como tabelas dinâmicas [Cormen et al., 2001], a inserção de novos nós tem um custo $O(1)$ amortizado, enquanto mantém as alocações de memória no mínimo e, assim, reduz a sobrecarga que de outra forma ocorreria usando estruturas de dados por nó. Manter a representação de forma contígua na memória também traz os benefícios de aproveitar os caches da CPU, o que pode melhorar bastante o desempenho quando previsões rápidas são críticas. Como os nós são acessados sucessivamente e repetidamente no momento do teste, armazená-los em locais de memória próximos é altamente recomendado.

5.3.3 *Builders*

The implementation of the induction procedure in Scikit-Learn revolves around three nested components:

(a) The first component is a `Builder` object whose role is to effectively build the tree array representation presented above, by recursively partitioning nodes using splits found by a `Splitter` object.

(b) The second component is a `Splitter` object whose role is to find splits on internal nodes.

(c) The third component is a `Criterion` object whose role is to evaluate the goodness of a split.

The most direct `Builder` implementation is the depth-first greedy induction procedure as originally outlined in Algorithm 3.2. A critical aspect of this algorithm is how to keep account of the the input samples $(\mathbf{x}, y) \in \mathcal{L}_t$ that arrive at node t. A straightforward solution is to store lists of indices of the input samples at node t. As pointed out in [Criminisi and Shotton, 2013], building many such lists is however inefficient because of repeated memory allocations and deallocations. A better solution is to have instead all indices stored in a single static array and to use in-place reordering operations when partitioning t. More specifically, let assume that `samples` is the list of all indices and that `start` and `end` are bounds such that `samples[start:end]` contains the sample indices at the current node t. If t is split on feature `feature[t]` and threshold `threshold[t]`, then $\mathcal{L}_t$ can be partitioned into $\mathcal{L}_{t_L}$ and $\mathcal{L}_{t_R}$ by reorganizing `samples[start:end]` into `samples[start:pos]` and `samples[pos:end]`, such that all samples in the first part are on the left side of the split while all samples in the second part are on the right side of the split. The induction procedure then proceeds by pushing the bounds `start:pos` and `pos:end` on the stack S, as an efficient way to effectively represent $\mathcal{L}_{t_L}$ and $\mathcal{L}_{t_R}$.

An alternative implementation of the `Builder` interface consists in replacing the stack S in Algorithm 3.2 by a priority queue, hence changing the order in which the nodes are split. In particular, if nodes t are prioritized by weighted potential impurity decrease $p(t)\Delta i(s^*, t)$ (where s^* is the best split on t), then the greedy induction procedure switches from a depth-first construction of the tree to a *best-first* induction procedure. In this way, an additional stopping criterion can be defined as the maximum number `max_leaf_nodes` of leaves in the tree. If nodes are expanded in best-first order, then the resulting tree only includes the most significant splits, thereby pre-pruning all (seemingly) irrelevant branches without wasting computational resources. Alternatives also include prioritizing nodes according to their impurity $i(t)$ or their size N_t.

5.3.3 Construtores

A implementação do procedimento de indução no Scikit-Learn gira em torno de três componentes aninhados:

(a) O primeiro componente é um objeto Builder cuja função é construir efetivamente a representação do array em árvore apresentada acima, particionando recursivamente os nós usando divisões encontradas por um objeto Splitter. (b) O segundo componente é um objeto Splitter cuja função é encontrar divisões em nós internos. (c) O terceiro componente é um objeto Critério cuja função é avaliar a qualidade de uma divisão.

A implementação mais direta do Builder é o procedimento de indução gananciosa em profundidade, conforme originalmente descrito no Algoritmo 3.2. Um aspecto crítico deste algoritmo é como contabilizar as amostras de entrada $(\mathbf{x}, y) \in \mathcal{L}_t$ que chegam ao nó t. Uma solução simples é armazenar listas de índices das amostras de entrada no nó t. Conforme apontado em [Criminisi e Shotton, 2013], construir muitas dessas listas é, no entanto, ineficiente devido a repetidas alocações e desalocações de memória. Uma solução melhor é ter todos os índices armazenados em uma única matriz estática e usar operações de reordenação no local ao particionar t. Mais especificamente, vamos supor que samples sejam a lista de todos os índices e que o início e o fim sejam limites de modo que sa mples`[início:fim]`contenha os índices de amostra no nó atual t. Se t for dividido no recurso feature`[t]` e no limite`[t]`, então $\mathcal{L}_t$ pode ser particionado em $\mathcal{L}_{t_L}$ e $\mathcal{L}_{t_R}$ reorganizando amostras`[start:end]` em amostras`[start:pos]` e amostras`[pos:end]`,de modo que todas as amostras na primeira parte estejam à esquerda lado da divisão, enquanto todas as amostras na segunda parte estão no lado direito da divisão. O procedimento de indução então prossegue empurrando os limites start:pos e pos:end na pilha S, como uma forma eficiente de representar efetivamente $\mathcal{L}_{t_L}$ e $\mathcal{L}_{t_R}$.

Uma implementação alternativa da interface Builder consiste em substituir a pilha S do Algoritmo 3.2 por uma fila de prioridade, alterando assim a ordem em que os nós são divididos. Em particular, se os nóst são priorizados pela redução potencial ponderada de impureza $p(t)\Delta i(s^*, t)$(onde s^* é a melhor divisão em t), então o procedimento de indução ganancioso muda de uma construção da árvore em profundidade para um procedimento de indução do melhor primeiro. Desta forma, um critério de parada adicional pode ser definido como o número máximo max_leaf_nodes de folhas na árvore. Se os nós forem expandidos na melhor primeira ordem, então a árvore resultante incluirá apenas as divisões mais significativas, pré-podando assim todos os ramos (aparentemente) irrelevantes sem desperdiçar recursos computacionais. As alternativas também incluem priorizar nós de acordo com sua i mpureza $i(t)$ ou seu tamanho N_t.

Likewise, the stack S in Algorithm 3.2 can be replaced by a deque data structure, in which case the induction procedure switches to a *breadth-first* construction of the tree. With some modifications, breadth-first construction shows to be more efficient when it is expensive to randomly access the data [Criminisi and Shotton, 2013], e.g., when data are too big to fit into memory and must be streamed from disk. Breadth-first induction also proves to be an efficient strategy when combined with parallelism, e.g., for building a whole level of the decision tree simultaneously using multiple cores [Liao et al., 2013].

5.3.4 *Splitters*

In Scikit-Learn, `Splitter` objects implement search procedures for finding splits on internal nodes. In the case of decision trees, they implement algorithms 3.3 and 3.4 for finding the best split over all p input variables. In the case of randomized trees, they implement the search procedure 4.1 over $K \leqslant p$ randomly drawn input variables, combined with either algorithm 3.4 or 4.3 to respectively obtain Random Forests or Extremely Randomized Trees. Again, several aspects of the algorithm should be considered with care to obtain an efficient implementation:

Data access.
Looking for the best split on the input variable X_j and partitioning $\mathcal{L}_t$ requires to repeatedly access over the node sample values $x_{i,j}$ (i.e., over values in the j-th column if samples are represented as a two dimensional array), which may comes at a non negligible cost if data is not ordered properly. However, due to CPU caching and pre-fetching effects, the closer the values in memory, usually the lower it takes to access them. In our case, this can be exploited in two ways to minimize the cost of data access:

- By storing the learning set $\mathcal{L}$ using column-major order (i.e., Fortran array memory layout), hence storing the values $x_{i,j}$ of X_j (for all samples $i = 1, \dots, N$) contiguously in memory.
- By pre-fetching the node sample values $x_{i,j}$ (for $i = 1, \dots, N_t$) manually into a static and contiguous buffer, on which the search procedure is then applied.

Sort algorithm.
In CART and Random Forests, finding the best split on X_j requires to sort the sample indices `samples[start:end]` along their respective values on X_j, i.e., such that

$$\texttt{X[samples[start], j]} \leqslant \cdots \leqslant \texttt{X[samples[start+i], j]}$$

Da mesma forma, a pilha `S` no Algoritmo 3.2 pode ser substituída por uma estrutura de dados deque, caso em que o procedimento de indução muda para uma construção da árvore em largura. Com algumas modificações, a construção em largura mostra-se mais eficiente quando é caro acessar aleatoriamente os dados [Criminisi e Shotton, 2013], por exemplo, quando os dados são grandes demais para caber na memória e precisam ser transmitidos do disco. A indução em largura também prova ser uma estratégia eficiente quando combinada com paralelismo, por exemplo, para construir um nível inteiro da árvore de decisão simultaneamente usando múltiplos núcleos [Liao et al., 2013].

5.3.4 Divisores

No Scikit-Learn, os objetos Splitter implementam procedimentos de pesquisa para encontrar divisões em nós internos. No caso de árvores de decisão, eles implementam os algoritmos 3.3 e 3.4 para encontrar a melhor divisão entre todas p as variáveis de entrada. No caso de árvores aleatórias, eles implementam o procedimento de busca 4.1 sobre $K \leqslant p$ variáveis de entrada desenhadas aleatoriamente, combinado com o algoritmo 3.4 ou 4.3 para obter respectivamente Florestas Aleatórias ou Árvores Extremamente Randomizadas. Novamente, vários aspectos do algoritmo devem ser considerados com cuidado para obter uma implementação eficiente:

Acesso a dados.
Procurar a melhor divisão na variável de entrada X_j e particionamento $\mathcal{L}_t$ requer acessar repetidamente os valores de amostra do nó $x_{i,j}$ (ou seja, sobre os valores na j-ésima coluna se as amostras forem representadas como uma matriz bidimensional), o que pode ter um custo não negligenciável se os dados não forem ordenados corretamente. Entretanto, devido ao cache da CPU e aos efeitos de pré-busca, quanto mais próximos os valores na memória, geralmente menor será o tempo necessário para acessá-los. No nosso caso, isto pode ser explorado de duas maneiras para minimizar o custo de acesso aos dados:

- Armazenando o conjunto de aprendizagem $\mathcal{L}$ usando a ordem principal da coluna (ou seja, layout de memória de array Fortran), armazenando assim os valores $x_{i,j}$ de X_j (para todas as amostras $i = 1,\dots, N$) contiguamente na memória. - Pré-buscando os valores de amostra do nó $x_{i,j}$ (para $i = 1, \dots, N_t$) manualmente em um buffer estático e contíguo, no qual o procedimento de pesquisa é então aplicado.

Algoritmo de classificação.
Em CART e Random Forests, encontrar a melhor divisão em X_j requer classificar as amostras de índices de amostra `[start:end]` ao longo de seus respectivos valores em X_j, ou seja, de modo que

$$\texttt{X[samples[start], j]} \leqslant \cdots \leqslant \texttt{X[samples[start+i], j]}$$

$$\leqslant \cdots \leqslant \texttt{X[samples[end-1], j]},$$

for $\texttt{i} = 0, \ldots, N_t - 1$. As shown in Section 5.1, this operation drives the complexity of the overall induction procedure and is therefore critical in the implementation of the algorithm.

To guarantee the lowest time complexity in all cases, we rely on Introsort [Musser, 1997] which combines Quicksort and Heapsort into a hybrid sorting algorithm. Basically, sorting in Introsort begins with Quicksort and then switches to Heapsort when the depth of partitioning exceeds $\Theta(\log N)$. As such, the practical $\Theta(N \log N)$ performance of Introsort is comparable to the average performance of Quicksort, except that performance in the worst case is bounded to $\Theta(N \log N)$ due to Heapsort, instead of $\Theta(N^2)$ in the worst case for Quicksort. Additionally, Quicksort is implemented using the median-of-three as pivot [Bentley and McIlroy, 1993], hence yielding better partitions and further improving performance.

In the context of the tree induction procedure, let us finally notice that partitioning node samples $\mathcal{L}_t$ into subsets $\mathcal{L}_{t_L}$ and $\mathcal{L}_{t_R}$ not only makes them purer in terms of the output variable Y, it also indirectly makes node samples more and more identical in terms of the values taken by the input variables. As we get deeper into the tree, this property can be leveraged by implementing Quicksort as a 3-way recursive partition [Bentley and McIlroy, 1993], as illustrated in Figure 5.3. In this variant, elements that are identical to the pivot are all set at their final positions in a single recursive call, hence accelerating the sorting procedure. If the number of distinct values is constant, 3-way Quicksort can be shown [Sedgewick and Wayne, 2011] to reduce running times from $O(N \log N)$ to $O(NH)$, where H is the Shannon entropy defined on the frequencies of the values to be sorted. As a consequence, in the case of many duplicates (i.e., when $H = O(1)$), $C(N) = O(N)$ and using 3-way Quicksort for sorting the sample indices in fact reduces the overall complexity of Random Forests to the asymptotic complexity of Extremely Randomized Trees (see Section 5.1).

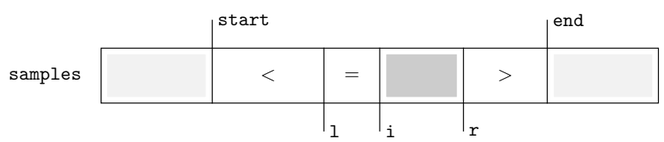


Figure 5.3: 3-way partition in Quicksort. Once `i=r`, elements that are identical to the pivot are all set at their final positions `samples[l:r]`, in a single recursive call. The sub-arrays `samples[start:l]` and `samples[r:end]` are then recursively sorted.

$$\leqslant \cdots \leqslant \texttt{X[amostras[fim-1], j]},$$

para $\texttt{i} = 0, \ldots, N_t - 1$. Conforme mostrado na Seção 5.1, esta operação aumenta a complexidade do procedimento geral de indução e é, portanto, crítica na implementação do algoritmo.

Para garantir a menor complexidade de tempo em todos os casos, contamos com o Introsort [Musser, 1997] que combina Quicksort e Heapsort em um algoritmo de classificação híbrido. Basicamente, a classificação no Introsort começa com Quicksort e depois muda para Heapsort quando a profundidade do particionamento excede $\Theta(\log N)$. Dessa forma, o desempenho prático de $\Theta(N \log N)$ do Introsort é comparável ao desempenho médio do Quicksort, exceto que o desempenho no pior caso é limitado ao $\Theta(N \log N)$ devido ao Heapsort, em vez de $\Theta(N^2)$ no pior caso do Quicksort. Além disso, o Quicksort é implementado usando a mediana de três como pivô [Bentley e McIlroy, 1993], produzindo partições melhores e melhorando ainda mais o desempenho.

No contexto do procedimento de indução de árvore, observemos finalmente que particionar amostras de nós $\mathcal{L}_t$ em subconjuntos $\mathcal{L}_{t_L}$ e $\mathcal{L}_{t_R}$não apenas as torna mais puras em termos da variável de saída Y, mas também indiretamente torna as amostras de nós cada vez mais idênticas em termos dos valores obtidos pelas variáveis de entrada. À medida que nos aprofundamos na árvore, essa propriedade pode ser aproveitada implementando o Quicksort com uma partição recursiva de três vias [Bentley e McIlroy, 1993], conforme ilustrado na Figura 5.3. Nesta variante, os elementos idênticos ao pivô são todos colocados nas suas posições finais numa única chamada recursiva, acelerando assim o procedimento de ordenação. Se o número de valores distintos for constante, o Quicksort de 3 vias pode ser mostrado [Sedgewick e Wayne, 2011] para reduzir os tempos de execução de $O(N \log N)$ para $O(NH)$, onde H é a entropia de Shannon definida nas frequências dos valores a serem classificados. Como consequência, no caso de muitas duplicatas (ou seja, quando $H = O(1)$), $C(N) = O(N)$ e o uso do Quicksort de 3 vias para classificar os índices de amostra, de fato reduz a complexidade geral das Florestas Aleatórias à co mplexidade assintótica das Árvores Extremamente Randomizadas (consulte a Seção 5.1).

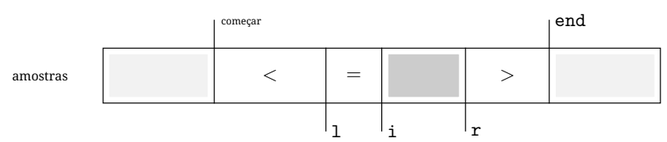


Figura 5.3: Partição de 3 vias no Quicksort. Uma vez `i=r`, os elementos que são idênticos ao pivô são todos definidos em suas posições finais amostras`[l:r]`, em uma única chamada recursiva. As submatrizes samples`[start:l]` e samples`[r:end]` são então classificadas recursivamente.

Skipping locally constant input variables.

In the extreme case, recursively partitioning node samples might result in input variables X_j to become locally constant at t. That is, $x_j = x'_j$ for all $(\mathbf{x}, y), (\mathbf{x}', y) \in \mathcal{L}_t$. In such a case, there is no valid split on X_j and trying to find one at t, but also at any of the descendant nodes, is a waste of computational resources. Accordingly, Algorithm 4.1 can be extended to account for variables that are known to be constant, thereby skipping non-informative variables if we know in advance that no valid split can be found. To stay as close as possible to the original algorithm, note however that constant variables are still included when drawing K variables at random.

Split approximations.

In Chapter 4, we showed that an increase of bias due to randomization is beneficial as long as it sufficiently decorrelates the predictions of the individual trees. Given this result, a legitimate question is to wonder if finding the very best splits s_j^* really is imperative in the first place to achieve good accuracy in random forests. As proved by the good results usually obtained by Extremely Randomized Trees, even splits that are drawn at random are in fact often sufficient to reach good (sometimes even better) performance. More importantly, not only this yields comparable accuracy, it is also more computationally efficient. At midway between exact best splits (as in Algorithm 3.4) and random splits (as in Algorithm 4.3), an intermediate and pragmatic solution is therefore to consider approximations of the best splits. The two most common strategies are based on *subsampling* and *binning*:

- Let N_0 denote an upper limit on the node sample size. If $N_t > N_0$, the subsampling approximation of the best split s_j^* at t on X_j consists in looking for the best split using only N_0 randomly drawn node samples from $\mathcal{L}_t$, and all node samples otherwise. The rationale behind this strategy is that, in the first nodes of the tree, the best splits are often so markedly superior to all other splits that this clear-cut information also reflects in a subsample. For nodes further down in the tree (i.e., for $N_t \leqslant N_0$), splits are usually less marked but then all node samples in $\mathcal{L}_t$ are used, hence guaranteeing to find the same splits as in the original procedure. Historically, the subsampling strategy was first proposed in [Breiman et al., 1984] in the context of single decision trees, for datasets too large to be held in memory.
- An alternative but close strategy is to consider only a subset of the possible discretization thresholds v'_k instead of exhaustively evaluating all candidate splits. In binning, this

Ignorando variáveis de entrada localmente constantes.

No caso extremo, o particionamento recursivo de amostras de nós pode fazer com que variáveis de entrada X_j se tornem localmente constantes em t. Ou seja, $x_j = x'_j$ para todo $(\mathbf{x}, y), (\mathbf{x}', y) \in \mathcal{L}_t$. Nesse caso, não há divisão válida em X_j e tentar encontrar uma em t, mas também em qualquer um dos nós descendentes, é um desperdício de recursos computacionais. Conseqüentemente, o Algoritmo 4.1 pode ser estendido para levar em conta variáveis que são conhecidas como constantes, ignorando assim variáveis não informativas se soubermos antecipadamente que nenhuma divisão válida pode ser encontrada. Para ficar o mais próximo possível do algoritmo original, observe, entretanto, que variáveis constantes ainda são incluídas ao desenhar variáveis aleatoriamente.

Aproximações divididas.

No Capítulo 4, mostramos que um aumento do viés devido à randomização é benéfico desde que descorrelacione suficientemente as previsões das árvores individuais. Dado esse resultado, uma questão legítima é perguntar se encontrar as melhores divisões s_j^* é realmente imperativo em primeiro lugar para obter boa precisão em florestas aleatórias. Como provado pelos bons resultados geralmente obtidos por Árvores Extremamente Randomizadas, mesmo as divisões sorteadas aleatoriamente são, na verdade, muitas vezes suficientes para alcançar um bom (às vezes até melhor) desempenho. Mais importante ainda, isso não apenas produz uma precisão comparável, mas também é mais eficiente do ponto de vista computacional. A meio caminho entre as melhores divisões exactas (como no Algoritmo 3.4) e as divisões aleatórias (como no Algoritmo 4.3), uma solução intermédia e pragmática é, portanto, considerar aproximações das melhores divisões. As duas estratégias mais comuns são baseadas em subamostragem e binning:

- Deixe N_0 denotar um limite superior no tamanho da amostra do nó. Se $N_t > N_0$, a aproximação de subamostragem da melhor divisão s_j^* em t em X_j consiste em procurar a melhor divisão usando apenas N_0 amostras de nós sorteadas aleatoriamente de $\mathcal{L}_t$, e todas as amostras de nós caso contrário. A lógica por trás desta estratégia é que, nos primeiros nós da árvore, as melhores divisões são muitas vezes tão marcadamente superiores a todas as outras divisões que esta informação clara também se reflecte numa subamostra. Para nós mais abaixo na árvore (ou seja, para $N_t \leqslant N_0$), as divisões são geralmente menos marcadas, mas então todas as amostras de nós em $\mathcal{L}_t$ são usadas, garantindo assim encontrar as mesmas divisões do procedimento original. Historicamente, a estratégia de subamostragem foi proposta pela primeira vez em [Breiman et al., 1984] no contexto de árvores de decisão únicas, para conjuntos de dados grandes demais para serem mantidos na memória.

- Uma estratégia alternativa, mas próxima, é considerar apenas um subconjunto dos possíveis limites de discretização v'_k em vez de avaliar exaustivamente todas as divisões candidatas. No binning, isso

is performed by grouping together node samples that are close to each other into bins and then evaluating only the thresholds in-between these groups. The simplest strategy of all is to divide the value interval of X_j at t into D intervals of equal length. Another strategy is to sort the node samples along their respective values of X_j and to partition them into D subsets of equal size. More elaborate algorithms, also known as *attribute discretization* procedures, are reviewed in [Zighed and Rakotomalala, 2000].

Pre-sorting data.

As shown earlier, sorting node samples for each candidate split significantly impacts the overall complexity of the induction procedure. An alternative strategy is possible [Breiman, 2002] and consists in presorting the samples for all p input variables before the construction of the tree. More specifically, let $i = 1, \dots, \widetilde{N}$ denote the original indices of the unique input samples in $\mathcal{L}^m$. For each input variable X_j (for $j = 1, \dots, p$), the sorted indices $\sigma^j_1, \dots, \sigma^j_{\widetilde{N}}$, such that

$$x_{\sigma^j_1, j} \leqslant \cdots \leqslant x_{\sigma^j_{\widetilde{N}}, j}, \tag{5.36}$$

can be computed in $O(p\widetilde{N} \log \widetilde{N})$. Given these indices, finding the best split at a node then simply amounts to iterate, in that order, over the samples $\sigma^j_1, \dots, \sigma^j_{\widetilde{N}}$ (for all K of the split variables X_j), which reduces complexity to $O(K\widetilde{N})$ instead of $O(K\widetilde{N} \log \widetilde{N})$. Partitioning the node samples into t_L and t_R then requires to partition all p lists of sorted indices into $2p$ sub-lists of $\widetilde{N}_{t_L}$ and $\widetilde{N}_{t_R}$ sorted indices $\sigma^j_1, \dots, \sigma^j_{\widetilde{N}_{t_L}}$ and $\sigma^j_1, \dots, \sigma^j_{\widetilde{N}_{t_R}}$, which can be done in $O(p\widetilde{N})$. In total, the within-node complexity is

$$C(\widetilde{N}) = O(K\widetilde{N} + p\widetilde{N}) = O(p\widetilde{N}). \tag{5.37}$$

Using this alternative strategy, the time complexity of RF for the best, the worst and the average cases is respectively $O(Mp\widetilde{N} \log \widetilde{N})$, $O(Mp\widetilde{N}^2)$ and $O(Mp\widetilde{N} \log \widetilde{N})$, as derived from theorems 5.3, 5.5 and 5.7 for $K = p$. Neglecting constant factors, the ratio between the two implementations is $O(\frac{p}{K \log \widetilde{N}})$, which might not necessarily lead to faster building times depending on K and the size of the problem. For this reason, pre-sorting is in fact not used within the Scikit-Learn implementation of random forests.

5.3.5 *Criteria*

The last components of the implementation of decision trees in Scikit-Learn are `Criterion` objects for evaluating the goodness of splits. Supported criteria are `"gini"` and `"entropy"` in classification, and

é realizado agrupando amostras de nós que estão próximos uns dos outros em compartimentos e, em seguida, avaliando apenas os limites entre esses grupos. A estratégia mais simples de todas é dividir o intervalo de valor de X_j em t em D intervalos de igual comprimento. Outra estratégia é classificar as amostras de nós de acordo com seus respectivos valores de X_j e particioná-las em Dsubconjuntos de tamanho igual. Algoritmos mais elaborados, também conhecidos como procedimentos de cretização atribuída, são revisados em [Zighed e Rakotomalala, 2000].

Pré-classificação de dados.

Conforme mostrado anteriormente, a classificação de amostras de nós para cada divisão candidata impacta significativamente a complexidade geral do procedimento de indução. Uma estratégia alternativa é possível [Breiman, 2002]e consiste em pré-classificar as amostras para todas p variáveis de entrada antes da construção da árvore. Mais especificamente, seja $i = \widetilde{1}, \dots, N$ denota os índices originais das amostras de entrada exclusivas em $\mathcal{L}^m$. Para cada variável de entrada X_j (para $j = 1, \dots, p$), os índices classificados $\sigma^j_1, \dots, \sigma^j_{\widetilde{N}}$, tal que

$$x_{\sigma^j_1, j} \leqslant \cdots \leqslant x_{\sigma^j_{\widetilde{N}}, j}, \tag{5.36}$$

$\widetilde{\ \ \ \ \ \ \ \ }$pode ser calculado em $O(pN \log N)$. Dados esses índices, encontrar a melhor divisão em um nó equivale simplesmente a iterar, nessa ordem, sobre as amostras $\sigma^j_1, \dots, \sigma^j_{\widetilde{N}}$ (para todas K das variáveis de divisão X_j), o que reduz a complexidade para $O(K\widetilde{N})$ e m vez de $O(K\widetilde{N} \log \widetilde{N})$. O particionamento das amostras de nós em t_L e t_R requer então $\widetilde{\ \ \ \ \ \ \ \ }$particionar todas as p listas de índices classificados em $2p$ sublistas de N_{t_L} e$\widetilde{N}_{t_R}$índices classificados $\sigma^j_1, \dots, \sigma^j_{\widetilde{N}_{t_L}}$ e $\sigma^j_1, \dots, \sigma^j_{\widetilde{N}_{t_R}}$, o que pode ser feito em $O(p\widetilde{N})$. No total, a complexidade dentro do nó é

$$C(\widetilde{N}) = O(K\widetilde{N} + p\widetilde{N}) = O(p\widetilde{N}). \tag{5.37}$$

Usando esta estratégia alternativa, a complexidade de tempo de RF para o $\widetilde{\ \ \ \ \ \ \ \ }$melhor, o pior e o caso médio é respectivamente $O(MpN \log N)$,$O(Mp\widetilde{N}^2)$ $O(Mp\widetilde{N} \ \ \widetilde{N})$ 5 3, 5 5 e log, conforme derivado dos teoremas. . e 5,7 para $K = p$. Desprezando os fatores constantes, a proporção entre as duas implementações é $O(\frac{p}{K \log \widetilde{N}})$, o que pode não levar necessariamente a tempos de construção mais rápidos dependendo de K e do tamanho do problema. Por esse motivo, a pré-classificação não é de fato usada na implementação do Scikit-Learn de florestas aleatórias.

5.3.5 Critérios

Os últimos componentes da implementação de árvores de decisão no Scikit-Learn são objetos Criterion para avaliar a qualidade das divisões. Os critérios suportados são "gini" e "entropia" na classificação, e

the reduction of variance "mse" in regression. All of them implement the update mechanisms described in Section 3.6.3.1, such that the iterative evaluation of $N_t - 1$ splits on an ordered variable remains a $O(N_t)$ operation.

5.3.6 *Parallelization*

Scikit-Learn implementation of random forests supports parallelism to distribute computations on multi-core architectures. The simple, yet effective, approach that we implement is to consider the construction of a forest as an embarrassingly parallel problem, in which the M decision trees are built concurrently and independently on multiple processors. Assuming no overhead due to parallelization, the theoretical speedup of this strategy is equal to the number N_p of processors used for building the forest. In practice however, this upper bound is not always strictly reached due to the variance of the time required for building randomized trees. It may indeed happen that building trees take longer than building some others, thereby under-utilizing computational resources since jobs may have finished sooner than others.

More fine-grained approaches consist in building decision trees one at time, but using multiple processors. In the *node-based decomposition*, the construction of a tree is distributed by building multiple nodes concurrently. In this case, workload can be divided between processors using a breadth-first induction procedure and assigning whole sub-trees to develop once the waiting queue contains N_p nodes. Once again however, sub-trees may take longer to build than others, hence under-utilizing computational resources if the resulting decision tree is not balanced. To alleviate this issue, processors may be assigned single nodes to develop (that is, one at a time) using a producer-consumer scheme to balance workload in a fair way. Unfortunately, this latter approach often proves to be dominated in practice by the overhead due to task assignment and bookkeeping activities. Alternatively, the construction of a tree can also be distributed using an *attribute-based decomposition*. In this case, the workload is divided by parallelizing the search of the K best splits at a given node (see Algorithm 4.1). For detailed reviews on the topic, see [Kufrin, 1997; Srivastava et al., 2002].

Using similar or hybrid approaches, let us finally note that the parallel induction of random forests extend to other distributed architectures, including GPUs [Sharp, 2008; Liao et al., 2013], FPGA [Narayanan et al., 2007] or clusters [Mitchell et al., 2011].

a redução da variância "mse" na regressão. Todos eles implementam os mecanismos de atualização descritos na Seção 3.6.3.1, de modo que a avaliação iterativa de divisões $N_t - 1$ em uma variável ordenada permaneça uma operação$O(N_t)$.

5.3.6 Paralelização

A implementação do Scikit-Learn de florestas aleatórias suporta paralelismo para distribuir cálculos em arquiteturas multi-core. A abordagem simples, porém eficaz, que implementamos é considerar a construção de uma floresta como um problema embaraçosamente paralelo, no qual as árvores de decisão são construídas simultânea e independentemente em múltiplos processadores. Assumindo que não há sobrecarga devido à paralelização, a aceleração teórica desta estratégia é igual ao número N_p de processadores usados para construir a floresta. Na prática, porém, este limite superior nem sempre é estritamente alcançado devido à variação do tempo necessário para a construção de árvores aleatórias. Na verdade, pode acontecer que a construção de árvores demore mais do que a construção de outras, subutilizando assim os recursos computacionais, uma vez que os trabalhos podem ter terminado mais cedo do que outros.

Abordagens mais refinadas consistem em construir árvores de decisão uma de cada vez, mas usando vários processadores. Na decomposição baseada em nós, a construção de uma árvore é distribuída pela construção de vários nós simultaneamente. Nesse caso, a carga de trabalho pode ser dividida entre processadores usando um procedimento de indução em largura e atribuindo subárvores inteiras para desenvolvimento, uma vez que a fila de espera contenha nós N_p . Mais uma vez, no entanto, as subárvores podem demorar mais para serem construídas do que outras, subutilizando assim os recursos computacionais se a árvore de decisão resultante não estiver equilibrada. Para aliviar esse problema, os processadores podem receber nós únicos para desenvolver (ou seja, um de cada vez) usando um esquema produtor-consumidor para equilibrar a carga de trabalho de maneira justa. Infelizmente, esta última abordagem revela-se frequentemente dominada na prática pelas despesas gerais devidas à atribuição de tarefas e às atividades de contabilidade. Alternativamente, a construção de uma árvore também pode ser distribuída usando uma decomposição baseada em atributos. Neste caso, a carga de trabalho é dividida paralelizando a busca das K melhores divisões em um determinado nó (ver Algoritmo 4.1). Para análises detalhadas sobre o tema, consulte [Kufrin, 1997; Srivastava et al., 2002].

Usando abordagens similares ou híbridas, observemos finalmente que a indução paralela de florestas aleatórias se estende a outras arquiteturas distribuídas, incluindo GPUs [Sharp, 2008; Liao et al., 2013], FPGA[Narayanan et al., 2007] ou clusters [Mitchell et al., 2011].

5.4 BENCHMARKS

5.4.1 *Complexity on artificial data*

In this section, we study the empirical complexity of random forests on artificial data. Our goal is to investigate whether the theoretical results that have been derived in Sections 5.1 and 5.2 hold true in practice, even without our assumptions necessarily satisfied.

For our experiments on simulated data, we consider the Friedman #1 artificial regression problem [Friedman, 1991]. It consists in 10 independent input variables $X_1, \dots, X_{10}$, each uniformly distributed over $[0, 1]$. The output Y is a real-valued variable and is given by

$$Y = 10 \sin(\pi X_1 X_2) + 20(X_3 - 0.48)^2 + 10X_4 + 5X_5 + \epsilon \qquad (5.38)$$

where ϵ is $\mathcal{N}(0, 1)$. We study the effects of the number M of trees in the forest, the number N of input samples, the number p of input variables (by adding new irrelevant variables), the number K of variables drawn at each node and the effects of using bootstrap replicates $\mathcal{L}^m$ instead of $\mathcal{L}$. Hyper-parameters are studied independently for both (fully developed) Random Forests (RF) and Extremely Randomized Trees (ETs) built on $N = 1000$ learning samples of $p = 10$ input variables, with $M = 250$ trees and $K = \sqrt{p}$ set by default. Statistics reported below are averages over 10 runs. Figures 5.4-5.8 all illustrate the effect of one the hyper-parameters with respect to the time required for building a forest (on the left plots), the average depth of the trees (in the middle plots) and the mean squared error of the ensemble (on the right plots).

Figure 5.4 first demonstrates the effect of the number M of trees. As expected, the time complexity of the induction procedure is $O(M)$ while the average depth of the leaves is constant with M, since the later has no effect on the structure of the individual trees. The left plot also confirms that ETs are faster to build than RF, even if trees in ETs are on average one level deeper that trees in RFs, as shown in the middle plot. Finally, the mean squared error of the model shows to be inversely proportional to the number of trees in the forest, which confirms our previous results on variance reduction from Chapter 4.

Figure 5.5 considers the effect of the number N of training samples. As shown on the left plot, building times grow slightly faster than linearly, which appears to confirm the respective $O(N \log^2 N)$ and $O(N \log N)$ dependencies of RF and ETs with the size of the learning set (as derived in Section 5.1). Similarly, the middle plot confirms that the average depth of the leaves grows in $O(\log N)$ with N (as shown in Section 5.2). As expected, the right plot also illustrates that the more the data, the better the resulting model. Additionally, it shows that RF and ETs tend towards the same error rate as N increases, even if RF was strictly better on less data. This confirms once again that finding the very best splits is not strictly necessary to achieve good results.

5.4 parâmetros de referência

5.4.1 Complexidade em dados artificiais

Nesta seção, estudamos a complexidade empírica de florestas aleatórias em dados artificiais. Nosso objetivo é investigar se os resultados teóricos obtidos nas Seções 5.1 e 5.2 são verdadeiros na prática, mesmo sem que nossas suposições sejam necessariamente satisfeitas.

Para nossos experimentos com dados simulados, consideramos o problema de regressão artificial de Friedman nº 1 [Friedman, 1991]. Consiste em 10 variáveis de entrada independentes $X_1, \dots, X_{10}$, cada um distribuído uniformemente em $[0, 1]$. A saída Y é uma variável de valor real e é dada por

$$Y = 10 \sin(\pi X_1 X_2) + 20(X_3 - 0.48)^2 + 10X_4 + 5X_5 + \epsilon \qquad (5.38)$$

onde ϵ é $\mathcal{N}(0, 1)$. Estudamos os efeitos do número M de árvores na floresta, o número N de amostras de entrada, o número p de variáveis de entrada (adicionando novas variáveis irrelevantes), o número K de variáveis desenhadas em cada nó e os efeitos do uso de réplicas de bootstrap$\mathcal{L}^m$ em vez de $\mathcal{L}$. Os hiperparâmetros são estudados independentemente para Florestas Aleatórias (RF) (totalmente desenvolvidas) e Árvores Extremamente Randomizadas (ETs) construídas em $N = 1000$ amostras de aprendizagem de $p = 10$ variáveis de entrada, com $M = 250$ árvores e $K = \sqrt{p}$ definidas por padrão. As estatísticas relatadas abaixo são médias de 10 execuções. Todas as Figuras 5.4-5.8 ilustram o efeito de um dos hiperparâmetros em relação ao tempo necessário para construir uma floresta (nas parcelas da esquerda), a profundidade média das árvores (nas parcelas do meio) e o erro quadrático médio do conjunto (nas parcelas da direita).

A Figura 5.4 demonstra primeiro o efeito do número M de árvores. Como esperado, a complexidade temporal do procedimento de indução é $O(M)$ enquanto a profundidade média das folhas é constante com M, uma vez que esta última não tem efeito na estrutura das árvores individuais. O gráfico da esquerda também confirma que os ETs são mais rápidos de construir do que os RF, mesmo que as árvores nos ETs estejam, em média, um nível mais profundo que as árvores nos RFs, como mostrado no gráfico do meio. Finalmente, o erro quadrático médio do modelo mostra-se inversamente proporcional ao número de árvores na floresta, o que confirma nossos resultados anteriores sobre redução de variância do Capítulo 4.

A Figura 5.5 considera o efeito do número N de amostras de treinamento. Conforme mostrado no gráfico à esquerda, os tempos de construção crescem um pouco mais rápido do que linearmente, o que parece confirmar as respectivas dependências $O(N\log^2 N)$ e$O(N \log N)$ de RF e ETs com o tamanho do conjunto de aprendizagem (conforme derivado na Seção 5.1). Da mesma forma, o gráfico do meio confirma que a profundidade média das folhas cresce em $O(\log N)$ com N (como mostrado na Seção 5.2). Como esperado, o gráfico à direita também ilustra que quanto mais dados, melhor será o modelo resultante. Além disso, mostra que RF e ETs tendem à mesma taxa de erro à medida que N aumenta, mesmo que RF fosse estritamente melhor com menos dados. Isto confirma mais uma vez que encontrar as melhores divisões não é estritamente necessário para alcançar bons resultados.

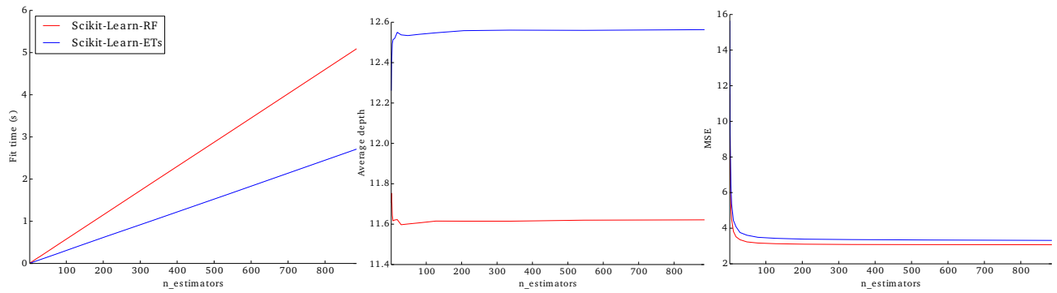


Figure 5.4: Effect of the number M (= `n_estimators`) of trees on training time (left), average depth (middle) and mean squared error (right). N = 1000, p = 10, K = $\sqrt{p}$.

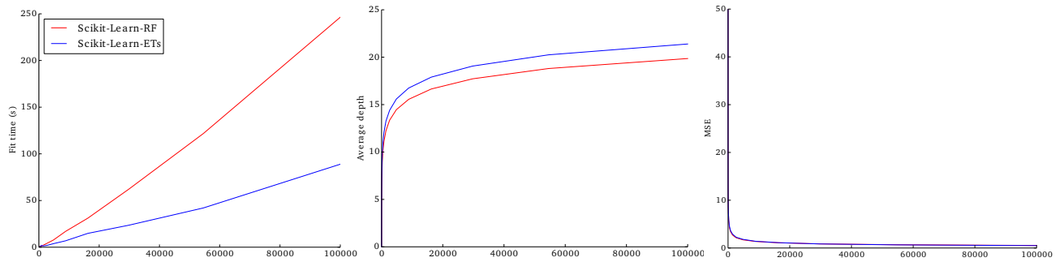


Figure 5.5: Effect of the size N (= `n_train`) of the learning set on training time (left), average depth (middle) and mean squared error (right). M = 250, p = 10, K = $\sqrt{p}$.

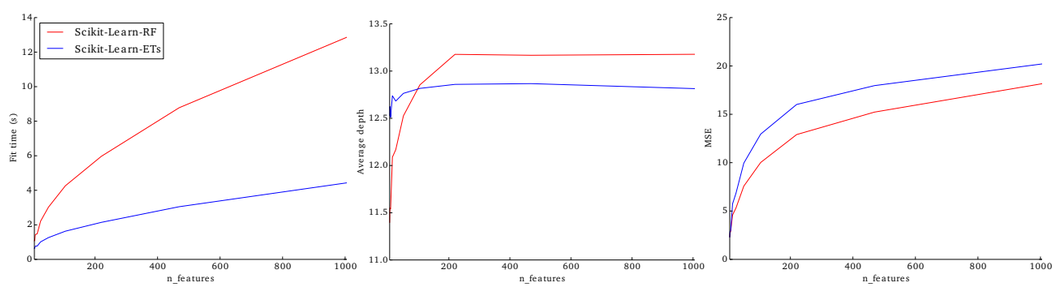


Figure 5.6: Effect of the number p (= `n_features`) of input variables on training time (left), average depth (middle) and mean squared error (right). N = 1000, M = 250, K = $\sqrt{p}$.

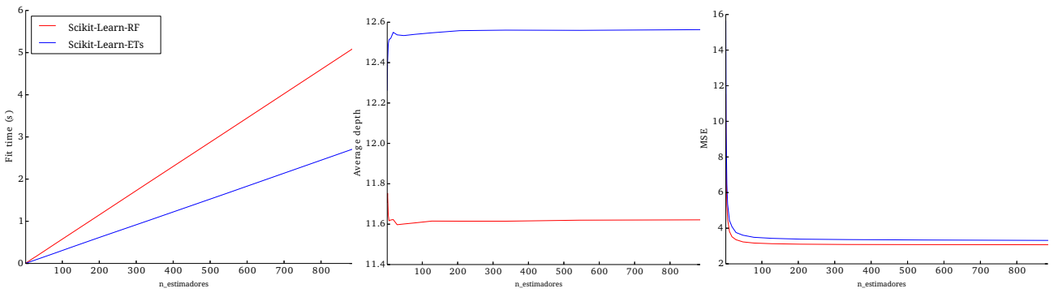


Figura 5.4: Efeito do número M (= n_estimators) de árvores no tempo de treinamento (esquerda), profundidade média (meio) e erro quadrático médio (direita). N = 1000, p = 10, K = $\sqrt{p}$. —

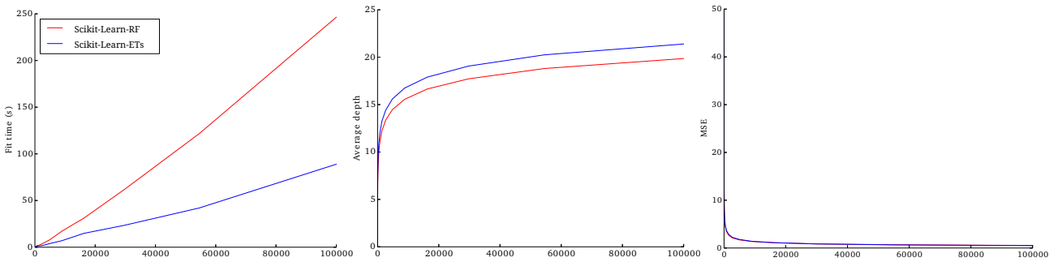


Figura 5.5: Efeito do tamanho N (= n_train) do conjunto de aprendizagem no tempo de treinamento (esquerda), profundidade média (meio) e erro quadrático médio (direita). M = 250, p = 10, K = $\sqrt{p}$. —

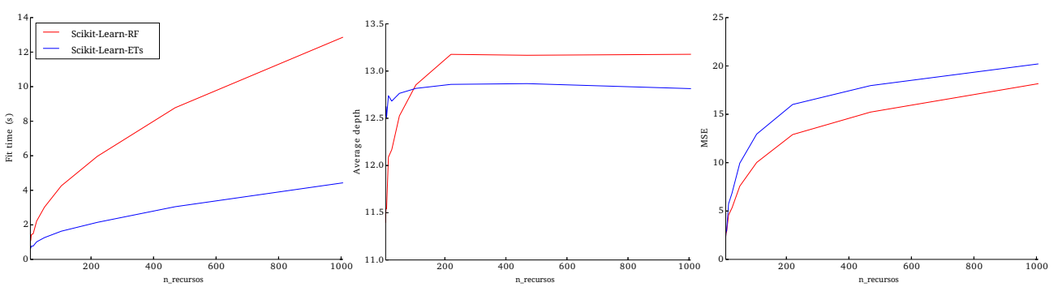


Figura 5.6: Efeito do número p (= n_features) de variáveis de entrada no tempo de treinamento (esquerda), profundidade média (meio) e erro quadrático médio (direita). N = 1000, M = 250, K = $\sqrt{p}$. —

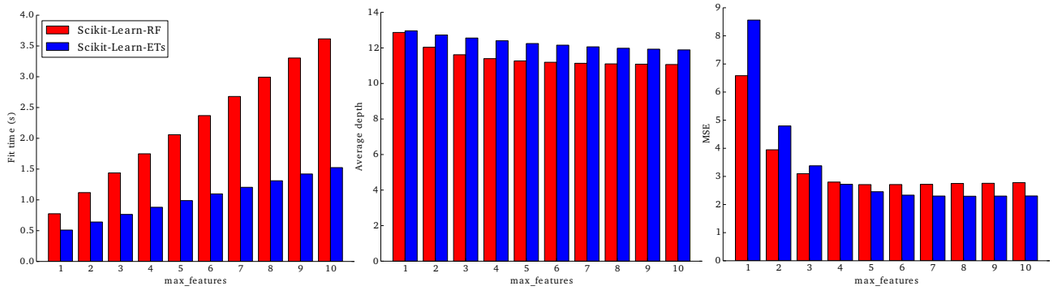


Figure 5.7: Effect of K (= `max_features`) in random variable selection on training time (left), average depth (middle) and mean squared error (right). $N = 1000$, $M = 250$, $p = 10$.

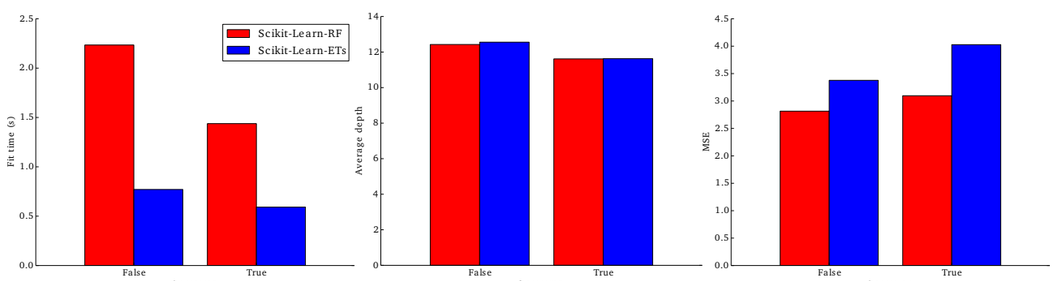


Figure 5.8: Effect of bootstrap replicates on training time (left), average depth (middle) and mean squared error (right). $N = 1000$, $M = 250$, $p = 10$, $K = \sqrt{p}$.

Next, Figure 5.6 shows the effect of irrelevant and noisy variables. The right plot first indicates that adding irrelevant variables deteriorates the accuracy of the forests. This is expected since the more the irrelevant variables, the higher the probability of splitting on one of them and therefore of fitting noise in the data. More importantly, the left and middle plots show that RF is more affected than ETs by the addition of noisy variables. This is actually a consequence of the end-cut preference of the classical impurity criteria. When splitting on noisy variables and looking for the best threshold, partitions of the node samples are more likely to be unbalanced rather than the opposite. This explains why on the middle plot the average depth of RF eventually gets larger than the depth of ETs – which suffers less from end-cut preference since thresholds are not optimized locally.

Figure 5.7 illustrates the effect of random variable selection. On the left plot, building times grow as expected in $O(K)$ with the number K of variables randomly drawn at each node. The average depth of the tree shows that increasing K slightly reduces the depth of the trees, which is explained by the fact that better splits are selected as

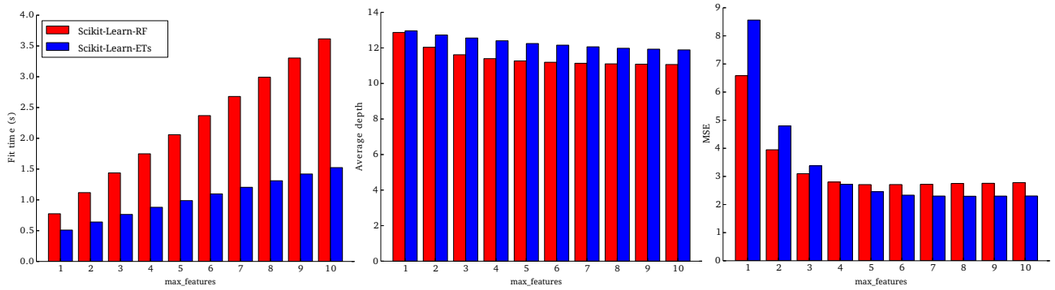


Figura 5.7: Efeito de K (= max_features) na seleção de variáveis aleatórias no tempo de treinamento (esquerda), profundidade média (meio) e erro quadrático médio (direita). $N = 1000$, $M = 250$, $p = 10$.

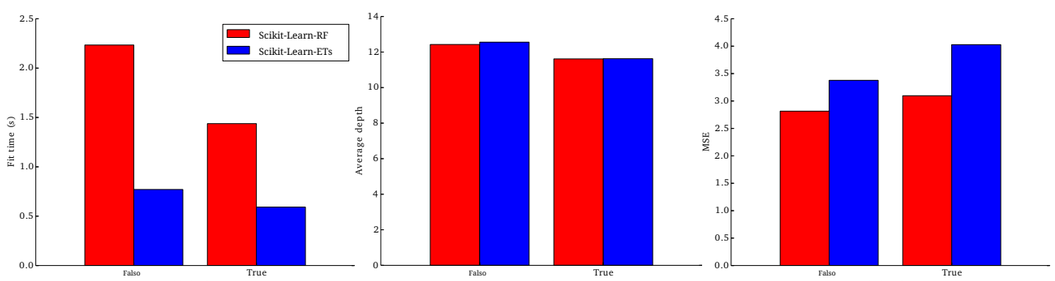


Figura 5.8: Efeito das réplicas de bootstrap no tempo de treinamento (esquerda), profundidade média (meio) e erro quadrático médio (direita). $N = 1000$,$M = 250$, $p = 10$, $K = \sqrt{p}$.

A seguir, a Figura 5.6 mostra o efeito de variáveis irrelevantes e ruidosas. O gráfico da direita indica primeiro que a adição de variáveis irrelevantes deteriora a precisão das florestas. Isto é esperado, pois quanto mais variáveis irrelevantes, maior a probabilidade de divisão em uma delas e, portanto, de ajuste de ruído nos dados. Mais importante ainda, os gráficos da esquerda e do meio mostram que o RF é mais afetado que os ETs pela adição de variáveis ruidosas. Na verdade, isso é uma consequência da preferência de corte final dos critérios clássicos de impureza. Ao dividir em variáveis ruidosas e procurar o melhor limite, as partições das amostras de nós têm maior probabilidade de serem desequilibradas, e não o contrário. Isso explica por que no gráfico do meio a profundidade média de RF eventualmente fica maior que a profundidade dos ETs – que sofre menos com a preferência de corte final, uma vez que os limites não são otimizados localmente.

A Figura 5.7 ilustra o efeito da seleção de variáveis aleatórias. No gráfico à esquerda, os tempos de construção crescem conforme esperado em $O(K)$ com o número$K$ de variáveis sorteadas aleatoriamente em cada nó. A profundidade média da árvore mostra que o aumento de K reduz ligeiramente a profundidade das árvores, o que é explicado pelo fato de que melhores divisões são selecionadas como

more variables are evaluated. Finally, the right plot again confirms results from Chapter 4. Random perturbations negatively impact the resulting model when they are too strong (at $K = 1$) but then become beneficial as K increases. For RF, the optimal trade-off is at $K = 5$ while the lowest error can be achieved in ETs for $K = 7$.

Finally, Figure 5.8 studies the effect of using bootstrap replicates. For RF, learning trees from learning sets $\mathcal{L}^m$ drawn with replacement from $\mathcal{L}$ (`bootstrap=True`) is about 1.5 faster than building trees directly on $\mathcal{L}$ (`bootstrap=False`) , which is in accordance with our theoretical results from Section 5.1. The effect for ETs is less strong, but still leads to a non-negligible speedup. The average depth for RF and ETs is roughly 0.5 lower when using bootstrap replicates. Again, this is expected since the average depth grows with $O(\log N)$ and that reducing the effective size of the learning set to $\tilde{N} = 0.632N$ reduces its logarithm by $-\log(0.632) = 0.662$. Finally, the right plot shows that using bootstrap replicates actually impairs performance on this problem.

While not discussed here, further experiments carried out on other artificial data, for both regression and classification, lead to similar conclusions. They all corroborate the theoretical results presented earlier.

5.4.2 *Comparison with other implementations*

Due to their apparent simplicity, random forests have been reimplemented at multiple occasions in various machine learning libraries and programming languages. In this section, we compare the Scikit-Learn implementation developed within this work with popular alternatives available in other machine learning packages.

Table 5.3 summarizes the main open source implementations of random forests along with some of their supported features. All implement the original Random Forest algorithm [Breiman, 2001] and therefore also provide an interface for Bagging [Breiman, 1994] (since it corresponds to the special case $K = p$). Scikit-Learn, OpenCV and OK3 also offer variants like Extremely Randomized Trees [Geurts et al., 2006a] or Random Subspaces [Ho, 1998]. All support both classification and regression tasks, with the exceptions of Weka and H2O which appear to only support classification. From a parameter point of view, implementations mostly differ from each other in the impurity and stopping criteria they support. The most common impurity criterion is Gini (or, equivalently, MSE) but some packages, like Orange, include alternative measures such as entropy, gain ratio or Relief [Kira and Rendell, 1992]. The most common stopping criteria are an upper bound on the depth of the trees (e.g., `max_depth` in Scikit-Learn) and a minimum number of samples required for splitting a node (e.g., `nmin` in OK3). Some packages also allow to set an upper

mais variáveis são avaliadas. Finalmente, o gráfico à direita confirma novamente os resultados do Capítulo 4. Perturbações aleatórias impactam negativamente o modelo resultante quando são muito fortes (em $K = 1$), mas tornam-se benéficas à medida que K aumenta. Para RF, o trade-off ideal está em $K = 5$ enquanto o erro mais baixo pode ser alcançado em ETs para $K = 7$.

Finalmente, a Figura 5.8 estuda o efeito do uso de réplicas de bootstrap. Para RF, aprender árvores a partir de conjuntos de aprendizagem $\mathcal{L}^m$ desenhados com substituição de $\mathcal{L}$ (bootstrap=True) é cerca de 1,5 mais rápido do que construir árvores diretamente em $\mathcal{L}$ (bootstrap=False) , o que está de acordo com nossos resultados teóricos da Seção 5.1. O efeito para os ETs é menos forte, mas ainda leva a uma aceleração não negligenciável. A profundidade média para RF e ETs é aproximadamente 0,5 menor ao usar réplicas de bootstrap. Novamente, isso é esperado, já que a profundidade média cresce com $O(\log N)$ e isso reduz $\tilde{}$o tamanho efetivo do conjunto de aprendizagem para $N = 0.632N$ reduz seu logaritmo em $-\log(0.632) = 0.662$. Finalmente, o gráfico à direita mostra que o uso de réplicas de bootstrap realmente prejudica o desempenho neste problema.

Embora não sejam discutidos aqui, outros experimentos realizados com outros dados artificiais, tanto para regressão quanto para classificação, levam a conclusões semelhantes. Todos corroboram os resultados teóricos apresentados anteriormente.

5.4.2 *Comparação com outras implementações*

Devido à sua aparente simplicidade, as florestas aleatórias foram reimplementadas em diversas ocasiões em diversas bibliotecas de aprendizado de máquina e linguagens de programação. Nesta seção, comparamos a implementação do Scikit-Learn desenvolvida neste trabalho com alternativas populares disponíveis em outros pacotes de aprendizado de máquina.

A Tabela 5.3 resume as principais implementações de código aberto de florestas aleatórias juntamente com alguns de seus recursos suportados. Todos implementam o algoritmo Random Forest original [Breiman, 2001] e portanto também fornecem uma interface para Bagging [Breiman, 1994] (uma vez que corresponde ao caso especial $K = p$). Scikit-Learn, OpenCV e OK3 também oferecem variantes como Árvores Extremamente Randomizadas [Geurts et al., 2006a] ou Subespaços Aleatórios [Ho, 1998]. Todos suportam tarefas de classificação e regressão, com exceção de Weka e H2O que parecem suportar apenas classificação. Do ponto de vista dos parâmetros, as implementações diferem principalmente umas das outras na impureza e nos critérios de parada que suportam. O critério de impureza mais comum é Gini (ou, equivalentemente, MSE), mas alguns pacotes, como Orange, incluem medidas alternativas como entropia, taxa de ganho ou Alívio [Kira e Rendell, 1992]. Os critérios de parada mais comuns são um limite superior para a profundidade das árvores (por exemplo, max_profundidade no Scikit-Learn) e um número mínimo de amostras necessárias para dividir um nó (por exemplo, nmin em OK3). Alguns pacotes também permitem definir um valor superior

Library	Algorithms	Tasks	Impurity	Stopping criteria	Variable importances	Multi-thread	Open source	Language	Reference
Scikit-Learn	Bagging, RF, ETs, RS, RP	C/R	Gini, Entropy, MSE	`max_depth`, `min_samples_split`, `min_samples_leaf`, `max_leaf_nodes`	✓	✓	BSD	Python	[Pedregosa et al., 2011]
OpenCV	Bagging, RF, ETs	C/R	Gini	`max_depth`, `min_samples_count`, `forest_accuracy`	✓	✗	BSD	C/C++	[Bradski and Kaehler, 2008]
OK3	Bagging, RF, ETs	C/R	MSE	`varmin`, `nmin`, `maxnbsplits`	✓	✗	Source-available	C	[Geurts et al., 2006a]
Weka	Bagging, RF	C	Gini	`depth`	✗	✓	GPL	Java	[Hall et al., 2009]
randomForest	Bagging, RF	C/R	Gini	`nodesize`, `maxnodes`	✓	✗	GPL	R	[Liaw and Wiener, 2002]
Orange	Bagging, RF	C/R	GainRatio, Gini, Relief, MSE	`worst_acceptable`, `min_susbet`, `min_instances`, `max_depth`, `max_majority`	✗	✗	GPL	Python	[Demšar et al., 2013]
H2O	Bagging, RF	C	Gini, Entropy	`maxDepth`	✗	✓	Apache	Java	[Vitek, 2013]

Table 5.3: Popular libraries for random forests.

Library	Algorithms	Tasks	Impurity	Stopping criteria	Variable importances	Multi-thread	Open source	Language	Reference
Scikit-Learn	Bagging, RF, ETs, RS, RP	C/R	Gini, Entropy, MSE	`max_depth`, `min_samples_split`, `min_samples_leaf`, `max_leaf_nodes`	✓	✓	BSD	Python	[Pedregosa et al., 2011]
OpenCV	Bagging, RF, ETs	C/R	Gini	`max_depth`, `min_samples_count`, `forest_accuracy`	✓	✗	BSD	C/C++	[Bradski and Kaehler, 2008]
OK3	Bagging, RF, ETs	C/R	MSE	`varmin`, `nmin`, `maxnbsplits`	✓	✗	Source-available	C	[Geurts et al., 2006a]
Weka	Bagging, RF	C	Gini	`depth`	✗	✓	GPL	Java	[Hall et al., 2009]
randomForest	Bagging, RF	C/R	Gini	`nodesize`, `maxnodes`	✓	✗	GPL	R	[Liaw and Wiener, 2002]
Orange	Bagging, RF	C/R	GainRatio, Gini, Relief, MSE	`worst_acceptable`, `min_susbet`, `min_instances`, `max_depth`, `max_majority`	✗	✗	GPL	Python	[Demšar et al., 2013]
H2O	Bagging, RF	C	Gini, Entropy	`maxDepth`	✗	✓	Apache	Java	[Vitek, 2013]

Tabela 5.3: Bibliotecas populares para florestas aleatórias.

limit on the number of nodes in the trees or to stop the construction when a fixed accuracy is reached. Despite the embarrassing parallelism, only Scikit-Learn, Weka and H2O make it possible to build random forests using multiple cores. Most notably, H2O builds on top of Hadoop to provide a fully distributed implementation of random forests (i.e., on a cluster).

For performance reasons, implementations listed in Table 5.3 have mostly been written in low-level programming languages like C, C++ or Java, but usually interface with higher level languages, like Python, MATLAB or R, for convenience reasons. In the case of OpenCV for example, the core implementation is written in C++ but can easily been called from Python through bindings shipped with the library. Let us finally note that Table 5.3 is by no means an exhaustive list of all implementations of random forests. Domain-specific implementations (e.g., Random Jungle [Schwarz et al., 2010] for GWAS or TMVA [Hoecker et al., 2007] in particle physics) and proprietary software (e.g., Salford Systems, which owns the *Random Forests* trademark) are not listed nor evaluated here.

In the experiments carried out below, we benchmark the Scikit-Learn implementations of RF and ETs against all libraries listed in Table 5.3. We do not compare against H2O since it requires a dedicated cloud environment. Implementations are all benchmarked on 29 well-known and publicly available classification datasets that were chosen a priori and independently of the results obtained. Overall, these datasets cover a wide range of conditions (including both artificial and real data), with a sample size N ranging from 208 to 70000 and a number p of variables varying from 6 to 24496. Implementations are evaluated for (fully developed) Random Forests (RF) and Extremely Randomized Trees (ETs) whenever available, using respectively 75% and 25% of the original dataset as learning and test sets, $M = 250$ trees and $K = \sqrt{p}$ set by default. Results reported below are averages over 10 runs, with the learning and test sets resampled at each fold. Tables 5.4 and 5.5 respectively report the average time required for building a random forest and the average time required making predictions, on all 29 datasets and for all implementations. Computing times are reported using the Scikit-Learn implementation of RF as a baseline. Results in bold indicate that the implementation is the fastest on the dataset. Remark that times are not reported on CIFAR10 for R and Orange because these implementations failed at building a single forest within 96 hours for CPU time.

With respect to RF, Table 5.4 shows that Scikit-Learn is the fastest on average, far before all others. OK3 is the second fastest implementation (2.63× slower on average), Weka is third (2.92× slower), OpenCV is fourth (12.37× slower), Orange is fifth (17.80× slower) and the R implementation is last (30.73× slower). In particular, the R implementation shows to be very ineffective on large datasets, when

limitar o número de nós nas árvores ou interromper a construção quando uma precisão fixa for atingida. Apesar do paralelismo embaraçoso, apenas Scikit-Learn, Weka e H2O tornam possível construir florestas aleatórias usando múltiplos núcleos. Mais notavelmente, o H2O baseia-se no Hadoop para fornecer uma implementação totalmente distribuída de florestas aleatórias (ou seja, em um cluster).

Por razões de desempenho, as implementações listadas na Tabela 5.3 foram escritas principalmente em linguagens de programação de baixo nível, como C, C++ou Java, mas geralmente fazem interface com li nguagens de nível superior, como Python, MATLAB ou R, por razões de conveniência. No caso do OpenCV, por exemplo, a implementação principal é escrita em C++ , mas pode ser facilmente chamada do Python por meio de ligações enviadas com a biblioteca. Notemos finalmente que a Tabela 5.3 não é de forma alguma uma lista exaustiva de todas as implementações de florestas aleatórias. Implementações específicas de domínio (por exemplo, Random Jungle [Schwarz et al., 2010] para GWAS ou TMVA [Hoecker et al., 2007]em física de partículas) e software proprietário (por exemplo, Salford Systems, que possui a marca registrada Random Forests) não são listadas nem avaliadas aqui.

Nos experimentos realizados abaixo, comparamos as implementações de RF e ETs do Scikit-Learn com todas as bibliotecas listadas na Tabela 5.3. Não comparamos com H2O, pois requer um ambiente de nuvem dedicado. As implementações são todas avaliadas em 29 conjuntos de dados de classificação bem conhecidos e disponíveis publicamente que foram escolhidos a priori e independentemente dos resultados obtidos. No geral, esses conjuntos de dados cobrem uma ampla gama de condições (i ncluindo dados artificiais e reais), com um tamanho de amostra N variando de 208 a 70.000 e um número p de variáveis variando de 6 a 24.496. As implementações são avaliadas para Florestas Aleatórias (RF) (RF) e Árvores Extremamente Randomizadas (ETs) (totalmente desenvolvidas), sempre que disponíveis, usando respectivamente 75% e 25 % do original. conjunto de dados como conjuntos de aprendizado e teste, $M = 250$ árvores e$K = \sqrt{p}$ conjuntos por padrão. Os resultados relatados abaixo são médias de 10 execuções, com os conjuntos de aprendizado e teste reamostrados em cada dobra. As Tabelas 5.4 e 5.5 relatam, respectivamente, o tempo médio necessário para construir uma floresta aleatória e o tempo médio necessário para fazer previsões, em todos os 29 conjuntos de dados e para todas as implementações. Os tempos de computação são relatados usando a implementação de RF do Scikit-Learn como linha de base. Os resultados em negrito indicam que a implementação é a mais rápida no conjunto de dados. Observe que os tempos não são relatados no cifar10 para R e Orange porque essas implementações falharam na construção de uma única floresta em 96 horas para o tempo de CPU.

Com relação à RF, a Tabela 5.4 mostra que o Scikit-Learn é o mais rápido em média, muito antes de todos os outros. OK3 é a segunda implementação mais rápida (2.63× mais lenta em média), Weka é a terceira (2.92× mais lenta), OpenCV é a quarta (12.37× mais lenta), Orange é a quinta (17.80× mais lenta) e a implementação R é a última (30. 73× mais lenta). Em particular, a implementação R mostra-se muito ineficaz em grandes conjuntos de dados, quando

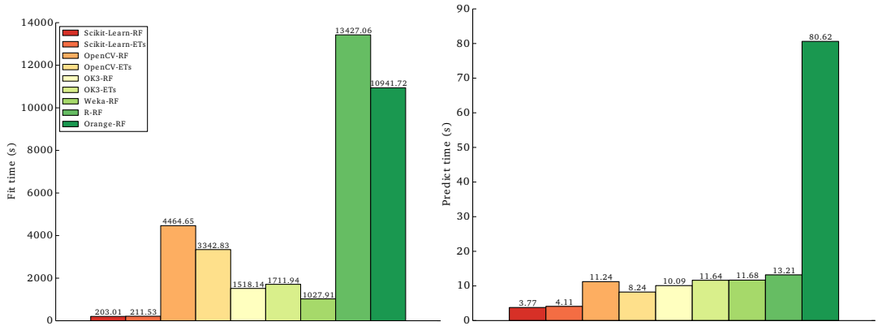


Figure 5.9: Average time required for building a forest on the MNIST dataset (left) and average time required for making predictions (right).

both N and p are moderately large (e.g., on MNIST, as further illustrated in Figure 5.9). A possible explanation is that the R implementation uses pre-sorting (see Section 5.3.4), which makes complexity linear with p rather than with K. Regarding Orange, results can be explained by the fact that the implementation is purely written in Python (i.e., an interpreted high-level programming language), while all others are written in optimized low-level programming languages. With respect to ETs, the Scikit-Learn implementation is again the fastest, far before all others. More interestingly, the table also shows that ETs are usually faster to build than RF, which confirms the theoretical results presented earlier. For the Scikit-Learn implementation, ETs are indeed empirically $\frac{1}{0.71} = 1.41\times$ faster than RF on average.

Table 5.5 reports results regarding the average time required for making predictions on the test set. Again, the Scikit-Learn implementation of RF is the fastest implementation on average. OpenCV is second (2.07× slower on average), OK3 is third (3.71× slower), Weka is fourth (4.31× slower), R is fifth (9.64× slower) and Orange is last (19.82× slower). By contrast with fitting times, making predictions from ETs is now slightly slower than RF, which is explained by the fact that trees in ETs are on average deeper than in RF.

In conclusions, benchmarks show that the Scikit-Learn implementation is on average significantly faster than OpenCV, OK3, Weka, R and Orange. While these results suffer from a selection bias (i.e., they depend on the 29 selected datasets), we believe that the general conclusions extend to other datasets. More importantly, the careful design and implementation of each and every component of random forests, as discussed all throughout this work, eventually shows to be highly rewarding in terms of computational performance.

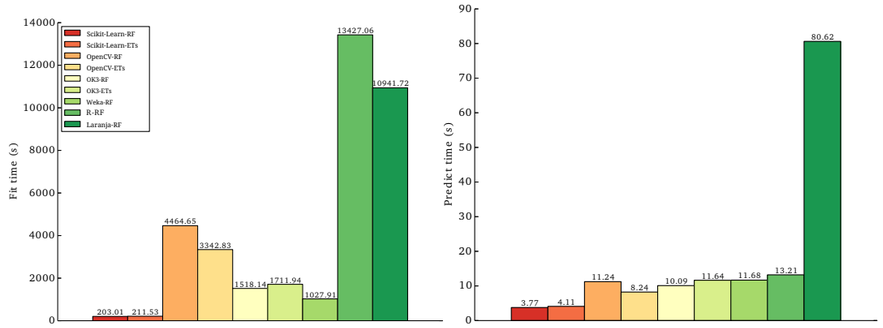


Figura 5.9: Tempo médio necessário para construir uma floresta no mnistdataset (esquerda) e tempo médio necessário para fazer previsões (direita).

tanto N quanto p são moderadamente grandes (por exemplo, em mnist, conforme ilustrado mais detalhadamente na Figura 5.9). Uma possível explicação é que a implementação R usa pré-classificação (ver Seção 5.3.4), o que torna a complexidade linear com p em vez de com K. Em relação ao Orange, os resultados podem ser explicados pelo fato de que a implementação é puramente escrita em Python (ou seja, uma linguagem de programação interpretada de alto nível), enquanto todas as outras são escritas em linguagens de programação otimizadas de baixo nível. No que diz respeito aos ETs, a implementação do Scikit-Learn é novamente a mais rápida, muito antes de todas as outras. O mais interessante é que a tabela também mostra que os ETs são geralmente mais rápidos de construir do que os RF, o que confirma os resultados teóricos apresentados anteriormente. Para a implementação do Scikit-Learn, os ETs são de fato empiricamente 1 0.71 = 1.41× ~~mais~~ rápidos que o RF, em média.

A Tabela 5.5 relata os resultados relativos ao tempo médio necessário para fazer previsões no conjunto de testes. Novamente, a implementação de RF do Scikit-Learn é a implementação mais rápida, em média. OpenCV é o segundo (2.07× mais lento em média), OK3 é o terceiro (3.71× mais lento), Weka é o quarto (4.31× mais lento), R é o quinto (9.64× mais lento) e Orange é o último (19.82× mais lento). Em contraste com os tempos de ajuste, fazer previsões a partir de ETs é agora um pouco mais lento do que RF, o que é explicado pelo fato de que as árvores em ETs são, em média, mais profundas do que em RF.

Nas conclusões, os benchmarks mostram que a implementação do Scikit-Learn é, em média, significativamente mais rápida que OpenCV, OK3, Weka, R e Orange. Embora estes resultados sofram de um viés de seleção (ou seja, dependem dos 29 conjuntos de dados selecionados), acreditamos que as conclusões gerais se estendem a outros conjuntos de dados. Mais importante ainda, o projeto e a implementação cuidadosos de cada um dos componentes das florestas aleatórias, conforme discutido ao longo deste trabalho, eventualmente se mostram altamente recompensadores em termos de desempenho computacional.

Table 5.4: Average time required for building a random forest, relative to the Scikit-Learn implementation of the Random Forest algorithm (first column). The lower, the better.

Dataset	Sklearn-RF	Sklearn-ETs	OpenCV-RF	OpenCV-ETs	OK3-RF	OK3-ETs	Weka-RF	R-RF	Orange-RF
ARCENE	1.00	**0.77**	26.33	16.73	0.81	0.85	8.70	22.84	4.78
BREAST2	1.00	**0.41**	35.10	19.03	1.20	0.99	3.88	33.41	11.42
CIFAR10	1.00	**0.52**	22.82	13.76	3.78	2.30	3.29	–	–
DIABETES	1.00	0.88	0.73	0.75	**0.59**	0.65	2.25	1.07	2.42
DIG44	1.00	**0.39**	1.01	0.71	2.68	2.03	1.70	1.60	10.85
IONOSPHERE	1.00	0.71	0.79	0.56	0.53	**0.34**	1.87	0.80	3.01
ISOLET	1.00	**0.36**	5.39	3.39	4.48	2.76	2.24	8.70	16.58
LETTER	1.00	**0.94**	2.34	1.82	9.05	10.57	3.13	4.43	21.04
LIVER	1.00	0.95	0.38	0.46	**0.33**	0.42	2.80	0.58	1.42
MADELON	1.00	**0.46**	5.82	4.07	1.38	0.82	2.06	6.18	48.48
MARTI0	1.00	**0.31**	5.70	4.45	1.08	0.55	1.36	18.99	7.16
MNIST	**1.00**	1.04	21.99	16.47	7.48	8.43	5.06	66.14	53.90
MNIST3VS8	1.00	**0.95**	18.74	13.90	3.32	3.04	3.68	54.89	27.80
MNIST4VS9	1.00	**0.97**	20.21	14.70	3.46	3.22	4.09	57.10	29.42
MUSK2	1.00	**0.44**	4.59	2.44	2.00	0.95	1.97	5.48	22.63
PENDIGITS	1.00	**0.61**	1.60	1.12	3.61	3.09	2.30	2.64	9.24
REGED0	1.00	**0.43**	6.65	4.29	1.18	0.64	1.63	10.02	7.04
RING-NORM	1.00	**0.22**	0.63	0.43	1.13	0.42	1.11	1.21	9.92
SATELLITE	1.00	**0.56**	2.10	1.36	2.69	1.97	1.94	2.56	13.88
SECOM	1.00	**0.21**	5.59	2.61	1.54	0.48	1.00	5.46	12.88
SEGMENT	1.00	**0.76**	1.09	0.81	1.91	1.79	1.83	1.64	5.97
SIDO0	**1.00**	1.55	133.11	140.63	9.70	8.62	9.69	468.85	49.11
SONAR	1.00	0.78	0.80	0.64	0.39	**0.30**	1.82	0.81	2.61
SPAMBASE	**1.00**	1.41	4.06	3.36	2.24	3.25	2.90	4.59	15.85
TIS	**1.00**	1.84	26.83	24.59	3.73	4.31	5.41	74.77	69.51
TWO-NORM	1.00	**0.30**	0.98	0.66	1.12	0.55	1.21	1.33	17.12
VEHICLE	1.00	**0.74**	1.40	1.08	1.63	1.46	2.23	1.69	6.51
VOWEL	1.00	0.59	0.67	**0.59**	1.87	1.50	1.99	1.08	8.18
WAVEFORM	1.00	**0.43**	1.20	0.94	1.49	0.98	1.49	1.50	9.79
Average	1.00	**0.71**	12.37	10.22	2.63	2.32	2.92	30.73	17.80
Median	1.00	**0.61**	4.06	2.44	1.87	1.46	2.23	4.51	11.14

Tabela 5.4: Tempo médio necessário para construir uma floresta aleatória, relativo à implementação do algoritmo Random Forest no Scikit-Learn (primeira coluna). Quanto mais baixo, melhor.

Dataset	Sklearn-RF	Sklearn-ETs	OpenCV-RF	OpenCV-ETs	OK3-RF	OK3-ETs	Weka-RF	R-RF	Orange-RF
ARCENE	1.00	**0.77**	26.33	16.73	0.81	0.85	8.70	22.84	4.78
BREAST2	1.00	**0.41**	35.10	19.03	1.20	0.99	3.88	33.41	11.42
CIFAR10	1.00	**0.52**	22.82	13.76	3.78	2.30	3.29	–	–
DIABETES	1.00	0.88	0.73	0.75	**0.59**	0.65	2.25	1.07	2.42
DIG44	1.00	**0.39**	1.01	0.71	2.68	2.03	1.70	1.60	10.85
IONOSPHERE	1.00	0.71	0.79	0.56	0.53	**0.34**	1.87	0.80	3.01
ISOLET	1.00	**0.36**	5.39	3.39	4.48	2.76	2.24	8.70	16.58
LETTER	1.00	**0.94**	2.34	1.82	9.05	10.57	3.13	4.43	21.04
LIVER	1.00	0.95	0.38	0.46	**0.33**	0.42	2.80	0.58	1.42
MADELON	1.00	**0.46**	5.82	4.07	1.38	0.82	2.06	6.18	48.48
MARTI0	1.00	**0.31**	5.70	4.45	1.08	0.55	1.36	18.99	7.16
MNIST	**1.00**	1.04	21.99	16.47	7.48	8.43	5.06	66.14	53.90
MNIST3VS8	1.00	**0.95**	18.74	13.90	3.32	3.04	3.68	54.89	27.80
MNIST4VS9	1.00	**0.97**	20.21	14.70	3.46	3.22	4.09	57.10	29.42
MUSK2	1.00	**0.44**	4.59	2.44	2.00	0.95	1.97	5.48	22.63
PENDIGITS	1.00	**0.61**	1.60	1.12	3.61	3.09	2.30	2.64	9.24
REGED0	1.00	**0.43**	6.65	4.29	1.18	0.64	1.63	10.02	7.04
RING-NORM	1.00	**0.22**	0.63	0.43	1.13	0.42	1.11	1.21	9.92
SATELLITE	1.00	**0.56**	2.10	1.36	2.69	1.97	1.94	2.56	13.88
SECOM	1.00	**0.21**	5.59	2.61	1.54	0.48	1.00	5.46	12.88
SEGMENT	1.00	**0.76**	1.09	0.81	1.91	1.79	1.83	1.64	5.97
SIDO0	**1.00**	1.55	133.11	140.63	9.70	8.62	9.69	468.85	49.11
SONAR	1.00	0.78	0.80	0.64	0.39	**0.30**	1.82	0.81	2.61
SPAMBASE	**1.00**	1.41	4.06	3.36	2.24	3.25	2.90	4.59	15.85
TIS	**1.00**	1.84	26.83	24.59	3.73	4.31	5.41	74.77	69.51
TWO-NORM	1.00	**0.30**	0.98	0.66	1.12	0.55	1.21	1.33	17.12
VEHICLE	1.00	**0.74**	1.40	1.08	1.63	1.46	2.23	1.69	6.51
VOWEL	1.00	0.59	0.67	**0.59**	1.87	1.50	1.99	1.08	8.18
WAVEFORM	1.00	**0.43**	1.20	0.94	1.49	0.98	1.49	1.50	9.79
Average	1.00	**0.71**	12.37	10.22	2.63	2.32	2.92	30.73	17.80
Median	1.00	**0.61**	4.06	2.44	1.87	1.46	2.23	4.51	11.14

Dataset	Sklearn-RF	Sklearn-ETs	OpenCV-RF	OpenCV-ETs	OK3-RF	OK3-ETs	Weka-RF	R-RF	Orange-RF
ARCENE	1.00	1.02	**0.38**	0.40	0.55	0.78	5.65	5.46	6.70
BREAST2	**1.00**	1.11	2.00	1.76	3.63	4.97	33.92	33.26	44.73
CIFAR10	**1.00**	1.13	3.17	2.25	2.82	3.05	6.23	–	–
DIABETES	1.00	1.25	1.00	1.78	1.34	2.47	1.49	**0.99**	11.41
DIG44	**1.00**	1.25	3.60	4.40	4.62	5.86	3.25	1.44	21.86
IONOSPHERE	1.00	1.05	**0.35**	0.61	0.47	0.74	1.36	0.45	6.56
ISOLET	**1.00**	1.19	2.21	2.25	2.03	2.63	2.92	2.17	12.28
LETTER	1.00	1.15	2.25	2.71	3.18	3.87	2.69	**0.85**	14.91
LIVER	1.00	1.14	**0.49**	0.93	0.62	1.22	1.54	0.58	6.01
MADELON	**1.00**	1.29	2.83	4.01	7.28	11.03	3.58	3.01	33.41
MARTI0	1.00	1.10	**0.68**	1.00	0.73	1.09	3.23	2.42	9.80
MNIST	**1.00**	1.09	2.98	2.18	2.68	3.09	3.10	3.50	21.39
MNIST3VS8	**1.00**	1.05	1.74	1.71	1.90	2.37	2.55	3.46	15.76
MNIST4VS9	**1.00**	1.21	2.29	2.24	2.42	2.92	3.21	5.21	19.46
MUSK2	**1.00**	1.15	2.62	3.31	3.10	4.18	2.88	2.07	29.81
PENDIGITS	**1.00**	1.18	2.70	3.72	3.66	4.97	2.89	1.03	22.49
REGED0	1.00	1.09	**0.48**	0.69	0.51	0.81	2.84	2.37	10.70
RING-NORM	**1.00**	1.48	4.35	6.24	5.74	8.29	3.30	1.59	28.55
SATELLITE	**1.00**	1.07	2.64	3.43	3.46	4.51	2.63	1.16	23.14
SECOM	**1.00**	1.07	1.87	1.98	2.09	2.22	3.06	2.74	17.35
SEGMENT	1.00	1.49	1.25	2.15	1.66	2.86	2.23	**0.82**	18.98
SIDO0	**1.00**	1.12	2.02	2.18	2.52	2.96	11.25	184.54	46.13
SONAR	1.00	1.03	**0.20**	0.33	0.23	0.41	1.01	0.39	4.70
SPAMBASE	**1.00**	1.38	2.91	3.90	4.32	8.89	3.67	1.73	26.51
TIS	**1.00**	1.33	2.70	2.66	2.82	3.66	3.40	3.70	16.65
TWO-NORM	**1.00**	1.33	4.09	5.93	5.35	7.57	3.52	1.52	32.80
VEHICLE	**1.00**	1.17	1.74	2.50	2.26	3.21	2.57	1.07	17.61
VOWEL	1.00	1.18	1.07	1.60	1.43	2.12	1.53	**0.76**	10.83
WAVEFORM	**1.00**	1.35	3.48	4.95	4.37	6.34	3.40	1.52	24.53
Average	**1.00**	1.19	2.07	2.54	2.68	3.76	4.31	9.64	19.82
Median	**1.00**	1.15	2.21	2.24	2.52	3.05	3.06	1.66	18.29

Table 5.5: Average time required for making predictions, relative to the Scikit-Learn implementation of the Random Forest algorithm (first column). The lower, the better.

Dataset	Sklearn-RF	Sklearn-ETs	OpenCV-RF	OpenCV-ETs	OK3-RF	OK3-ETs	Weka-RF	R-RF	Orange-RF
ARCENE	1.00	1.02	**0.38**	0.40	0.55	0.78	5.65	5.46	6.70
BREAST2	**1.00**	1.11	2.00	1.76	3.63	4.97	33.92	33.26	44.73
CIFAR10	**1.00**	1.13	3.17	2.25	2.82	3.05	6.23	–	–
DIABETES	1.00	1.25	1.00	1.78	1.34	2.47	1.49	**0.99**	11.41
DIG44	**1.00**	1.25	3.60	4.40	4.62	5.86	3.25	1.44	21.86
IONOSPHERE	1.00	1.05	**0.35**	0.61	0.47	0.74	1.36	0.45	6.56
ISOLET	**1.00**	1.19	2.21	2.25	2.03	2.63	2.92	2.17	12.28
LETTER	1.00	1.15	2.25	2.71	3.18	3.87	2.69	**0.85**	14.91
LIVER	1.00	1.14	**0.49**	0.93	0.62	1.22	1.54	0.58	6.01
MADELON	**1.00**	1.29	2.83	4.01	7.28	11.03	3.58	3.01	33.41
MARTI0	1.00	1.10	**0.68**	1.00	0.73	1.09	3.23	2.42	9.80
MNIST	**1.00**	1.09	2.98	2.18	2.68	3.09	3.10	3.50	21.39
MNIST3VS8	**1.00**	1.05	1.74	1.71	1.90	2.37	2.55	3.46	15.76
MNIST4VS9	**1.00**	1.21	2.29	2.24	2.42	2.92	3.21	5.21	19.46
MUSK2	**1.00**	1.15	2.62	3.31	3.10	4.18	2.88	2.07	29.81
PENDIGITS	**1.00**	1.18	2.70	3.72	3.66	4.97	2.89	1.03	22.49
REGED0	1.00	1.09	**0.48**	0.69	0.51	0.81	2.84	2.37	10.70
RING-NORM	**1.00**	1.48	4.35	6.24	5.74	8.29	3.30	1.59	28.55
SATELLITE	**1.00**	1.07	2.64	3.43	3.46	4.51	2.63	1.16	23.14
SECOM	**1.00**	1.07	1.87	1.98	2.09	2.22	3.06	2.74	17.35
SEGMENT	1.00	1.49	1.25	2.15	1.66	2.86	2.23	**0.82**	18.98
SIDO0	**1.00**	1.12	2.02	2.18	2.52	2.96	11.25	184.54	46.13
SONAR	1.00	1.03	**0.20**	0.33	0.23	0.41	1.01	0.39	4.70
SPAMBASE	**1.00**	1.38	2.91	3.90	4.32	8.89	3.67	1.73	26.51
TIS	**1.00**	1.33	2.70	2.66	2.82	3.66	3.40	3.70	16.65
TWO-NORM	**1.00**	1.33	4.09	5.93	5.35	7.57	3.52	1.52	32.80
VEHICLE	**1.00**	1.17	1.74	2.50	2.26	3.21	2.57	1.07	17.61
VOWEL	1.00	1.18	1.07	1.60	1.43	2.12	1.53	**0.76**	10.83
WAVEFORM	**1.00**	1.35	3.48	4.95	4.37	6.34	3.40	1.52	24.53
Average	**1.00**	1.19	2.07	2.54	2.68	3.76	4.31	9.64	19.82
Median	**1.00**	1.15	2.21	2.24	2.52	3.05	3.06	1.66	18.29

Tabela 5.5: Tempo médio necessário para fazer previsões, relativo à implementação Scikit-Learn do algoritmo Random Forest (primeira coluna). Quanto mais baixo, melhor.

Part II

INTERPRETING RANDOM FORESTS

Parte II

INTERPRETINGRANDOMFORESTS

6 UNDERSTANDING VARIABLE IMPORTANCES

Outline

In this chapter, we study variable importance measures as computed from forests of randomized trees. In Section 6.1, we first present how random forests can be used to assess the importance of input variables. We then derive in Section 6.2 a characterization in asymptotic conditions and show that variable importances derived from totally randomized trees offer a three-level decomposition of the information jointly contained in the input variables about the output. In Section 6.3, we show that this characterization only depends on the relevant variables and then discuss these ideas in Section 6.4 in the context of variants closer to the Random Forest algorithm. Finally, we illustrate these results on an artificial problem in Section 6.5. *This chapter is based on previous work published in [Louppe et al., 2013].*

An important task in many scientific fields is the prediction of a response variable based on a set of predictor variables. In many situations though, the aim is not only to make the most accurate predictions of the response but also to identify which predictor variables are the most important to make these predictions, e.g. in order to understand the underlying process. Because of their applicability to a wide range of problems and their capability to build accurate models and, at the same time, to provide variable importance measures, random forests have become a major data analysis tool used with success in various scientific areas.

Despite their extensive use in applied research, only a couple of works have studied the theoretical properties and statistical mechanisms of these algorithms. Zhao [2000], Breiman [2004], Meinshausen [2006], Lin and Jeon [2002] and Biau et al. [2008]; Biau [2012] investigated simplified to very close variants of these algorithms and proved the consistency of those variants. Little is known however regarding the variable importances computed by random forests, and – as far as we know – the work of Ishwaran [2007] is indeed the only theoretical study of tree-based variable importance measures. In this chapter, we aim at filling this gap and present a theoretical analysis of the Mean Decrease Impurity importance derived from ensembles of randomized trees.

6 ENTENDENDO OS VARIÁVEIS IMPORTÂNCIAS

Contorno

Neste capítulo, estudamos medidas de importância variável calculadas a partir de florestas de árvores aleatórias. Na Seção 6.1, apresentamos primeiro como florestas aleatórias podem ser usadas para avaliar a importância das variáveis de entrada. Derivamos então na Secção 6.2 u ma caracterização em condições assintóticas e mostramos que as importâncias das variáveis derivadas de árvores totalmente aleatórias oferecem uma decomposição em três níveis da informação contida conjuntamente nas variáveis de entrada sobre o resultado. Na Seção 6.3, mostramos que essa caracterização depende apenas das variáveis relevantes e depois discutimos essas ideias na Seção 6.4 no contexto de variantes mais próximas do algoritmo Random Forest. Finalmente, ilustramos esses resultados em um problema artificial na Seção 6.5. Este capítulo é baseado em trabalhos anteriores publicados em [Louppe et al., 2013].

Uma tarefa importante em muitos campos científicos é a previsão de uma variável de resposta baseada em um conjunto de variáveis preditoras. Em muitas situações, porém, o objectivo não é apenas fazer as previsões mais precisas da resposta, mas também identificar quais as variáveis preditoras que são mais importantes para fazer essas previsões, por ex. para compreender o processo subjacente. Devido à sua aplicabilidade a uma ampla gama de problemas e à sua capacidade de construir modelos precisos e, ao mesmo tempo, de fornecer medidas de importância variável, as florestas aleatórias tornaram-se uma importante ferramenta de análise de dados utilizada com sucesso em diversas áreas científicas.

Apesar de seu amplo uso em pesquisa aplicada, apenas alguns trabalhos estudaram as propriedades teóricas e os mecanismos estatísticos desses algoritmos. Zhao [2000], Breiman [2004], Meinshausen[2006], Lin e Jeon [2002] e Biau et al. [2008]; Biau [2012] investigou variantes simplificadas a muito próximas desses algoritmos e provou a consistência dessas variantes. No entanto, pouco se sabe sobre as importâncias variáveis calculadas por florestas aleatórias e – tanto quanto sabemos – o trabalho de Ishwaran [2007]é de facto o único estudo teórico de medidas de importância variável baseadas em árvores. Neste capítulo, pretendemos preencher esta lacuna e apresentar uma análise teórica da importância da diminuição média da impureza derivada de conjuntos de árvores aleatórias.

6.1 VARIABLE IMPORTANCES

6.1.1 *Importances in single decision trees*

In the context of single decision trees, Breiman et al. [1984] first defined the measure of importance of a variable X_j as

$$\text{Imp}(X_j) = \sum_{t \in \varphi} \Delta I(\tilde{s}_t^j, t), \tag{6.1}$$

where $\tilde{s}_t^j$ is the best *surrogate split* for s_t, that is the closest split defined on variable X_j that can mimic the actual split s_t defined at node t. The use of surrogate splits was proposed to account for masking effects: it may indeed happen that some variable X_{j_2} never occurs in any split because it leads to splits that are slightly worse, and therefore not selected, than those of some other variable X_{j_1}. However, if X_{j_1} is removed and another tree is grown, X_{j_2} may now occur prominently within the splits and the resulting tree may be nearly as good as the original tree. In such a case, a relevant measure should detect the importance of X_{j_2}. Accordingly, if X_{j_2} is being masked at t by X_{j_1} (i.e., if X_{j_1} is used to split t), but if $\tilde{s}_t^{j_2}$ is similar to s_t, but not quite as good, then $\Delta I(\tilde{s}_t^{j_2}, t)$ will be nearly as large as $\Delta I(s_t, t)$ and therefore the proposed measure will indeed account for the importance of X_{j_2}.

6.1.2 *Importances in forests*

Thanks to randomization, masking effects are dampened within forests of randomized trees. Even if X_{j_2} is being masked by X_{j_1} there is indeed still a chance for X_{j_2} to be chosen as a split if X_{j_1} is not selected among the K variables chosen at random. Depending on the value K, masking effects do not disappear entirely though. The use of bootstrap samples also helps reduce masking effects, making X_{j_1} or X_{j_2} just slightly better than the other due to the variations in the bootstrap samples.

For this reason, Breiman [2001, 2002] proposed to evaluate the importance of a variable X_j for predicting Y by adding up the weighted impurity decreases $p(t)\Delta i(s_t, t)$ for all nodes t where X_j is used, averaged over all trees φ_m (for $m = 1, \dots, M$) in the forest:

$$\text{Imp}(X_j) = \frac{1}{M} \sum_{m=1}^{M} \sum_{t \in \varphi_m} 1(j_t = j) \Big[p(t)\Delta i(s_t, t) \Big], \tag{6.2}$$

where $p(t)$ is the proportion $\frac{N_t}{N}$ of samples reaching t and where j_t denotes the identifier of the variable used for splitting node t. When using the Gini index as impurity function, this measure is known as the *Gini importance* or *Mean Decrease Gini*. However, since it can be defined for any impurity measure $i(t)$, we will refer to Equation 6.2 as

6.1 importâncias variáveis

6.1.1 Importâncias em árvores de decisão única

No contexto de árvores de decisão únicas, Breiman et al. [1984] primeiro definiu a medida de importância de uma variável X_j como

$$\text{Imp}(X_j) = \sum_{t \in \varphi} \Delta I(\tilde{s}_t^j, t), \tag{6.1}$$

onde $\tilde{s}_t^j$ é a melhor divisão substituta para s_t, que é a divisão mais próxima definida na variável X_j que pode imitar a divisão real s_t definida no nó t. O uso de divisões substitutas foi proposto para explicar os efeitos de mascaramento: pode de fato acontecer que alguma variável X_{j_2} nunca ocorra em qualquer divisão porque leva a divisões que são ligeiramente piores e, portanto, não selecionadas, do que aquelas de alguma outra variável X_{j_1}. No entanto, se X_{j_1} for removido e outra árvore for cultivada, X_{j_2} pode agora ocorrer de forma proeminente dentro das divisões e a árvore resultante pode ser quase tão boa quanto a árvore original. Nesse caso, uma medida relevante deverá detectar a importância de X_{j_2}. Assim, se X_{j_2} está sendo mascarado em t por X_{j_1} (ou seja, se X_{j_1} é usado para dividir t), mas se $\tilde{s}_t^{j_2}$é semelhante a s_t, mas não tão bom, então $\Delta I(\tilde{s}_t^{j_2}, t)$ será quase tão grande quanto $\Delta I(s_t, t)$ e, portanto, a medida proposta de fato levará em conta a importância de X_{j_2}.

6.1.2 Importâncias nas florestas

Graças à randomização, os efeitos de mascaramento são atenuados nas florestas de árvores aleatórias. Mesmo que X_{j_2} esteja sendo mascarado por X_{j_1} , ainda há uma chance de X_{j_2} ser escolhido como uma divisão se X_{j_1} não for selecionado entre as variáveis K escolhidas aleatoriamente. Dependendo do valor K, os efeitos de mascaramento não desaparecem completamente. O uso de amostras de bootstrap também ajuda a reduzir os efeitos de mascaramento, tornando X_{j_1} ou X_{j_2}apenas um pouco melhor que o outro devido às variações nas amostras de bootstrap.

Por esta razão, Breiman [2001, 2002] propôs avaliar a importância de uma variável X_j para prever Y somando as diminuições ponderadas de impureza $p(t)\Delta i(s_t, t)$ para todos os nós t onde X_j é usado, com média de todas as árvores φ_m (para $m = 1, \dots, M$) na floresta:

$$\text{Imp}(X_j) = \frac{1}{M} \sum_{m=1}^{M} \sum_{t \in \varphi_m} 1(j_t = j) \Big[p(t)\Delta i(s_t, t) \Big], \tag{6.2}$$

onde $p(t)$ é a proporção $\frac{N_t}{N}$ de amostras que atingem t e onde $j_t$denota o identificador da variável usada para dividir o nó t. Ao utilizar o índice de Gini como função de impureza, esta medida é conhecida como importância de Gini ou diminuição média de Gini. Entretanto, como pode ser definida para qualquer medida de impureza $i(t)$, nos referiremos à Equação 6.2 como

the *Mean Decrease Impurity* importance (MDI), no matter the impurity measure $i(t)$. We will characterize and derive results for this measure in the rest of this text.

In addition to MDI, Breiman [2001, 2002] also proposed to evaluate the importance of a variable X_j by measuring the *Mean Decrease Accuracy* (MDA) or, equivalently, by measuring the *Mean Increase Error*, of the forest when the values of X_j are randomly permuted in the out-of-bag samples. For that reason, this latter measure is also known as the *permutation importance*. Formally, in regression, the permutation importance of X_j is given as:

$$\begin{aligned} \operatorname{Imp}(X_j) &= \mathbb{E}_{\pi_j}\left\{\frac{1}{N}\sum_{(\mathbf{x}'_i,y_i)\in\pi_j(\mathcal{L})} L(\frac{1}{M^{-i}}\sum_{l=1}^{M^{-i}}\varphi_{\mathcal{L}^{m_{k_l}}}(\mathbf{x}'_i), y_i)\right\} \\ &\hookrightarrow -\frac{1}{N}\sum_{(\mathbf{x}_i,y_i)\in\mathcal{L}} L(\frac{1}{M^{-i}}\sum_{l=1}^{M^{-i}}\varphi_{\mathcal{L}^{m_{k_l}}}(\mathbf{x}_i), y_i) \end{aligned} \tag{6.3}$$

where $\pi_j(\mathcal{L})$ denotes a replicate of $\mathcal{L}$ in which in the values of X_j have been randomly permuted, and where $m_{k_1}, \ldots, m_{k_{M^{-i}}}$ denote the indices of the trees that have been built from bootstrap replicates that do not include $(\mathbf{x}_i, y_i)$. For classification, the permutation importance is derived similarly as in Equation 6.3, except that the out-of-bag average predictions are replaced with the class which is the most likely, as computed from the out-of-bag class probability estimates. Its rationale is that randomly permuting the input variable X_j should break its association with the response Y. Therefore, if X_j is in fact associated to Y, permuting its values should also result in a substantial increase of error, as here measured by the difference between the out-of-bag estimates of the generalization error. That is, the larger the increase of error, the more important the variable, and vice-versa.

Thanks to popular machine learning softwares [Breiman, 2002; Liaw and Wiener, 2002; Pedregosa et al., 2011], both of these variable importance measures have shown their practical utility in an increasing number of experimental studies. Little is known however regarding their inner workings. Strobl et al. [2007b] compare both MDI and MDA and show experimentally that the former is biased towards some predictor variables. As explained by White and Liu [1994] in case of single decision trees, this bias stems from an unfair advantage given by the usual impurity functions $i(t)$ towards predictors with a large number of values (see Section 7.2.2). Strobl et al. [2008] later showed that MDA is biased as well, and that it overestimates the importance of correlated variables – although this effect was not confirmed in a later experimental study by Genuer et al. [2010]. From a theoretical point of view, Ishwaran [2007] provides a detailed theoretical development of a simplified version of MDA, giving key insights for the understanding of the actual MDA.

a importância da impureza MeanDecrease (MDI), independentemente da medida de impureza $i(t)$. Caracterizaremos e derivaremos resultados para esta medida no restante deste texto.

Além do MDI, Breiman [2001, 2002] também propôs avaliar a importância de uma variável X_j medindo a Precisão MeanDecrease (MDA) ou, equivalentemente, medindo o Erro Médio de Aumento, da floresta quando os valores de X_j são permutados aleatoriamente nas amostras fora do saco. Por esse motivo, esta última medida também é conhecida como importância da permutação. Formalmente, na regressão, a importância da permutação de X_j é dada como:

$$\begin{aligned} \operatorname{Imp}(X_j) &= \mathbb{E}_{\pi_j}\left\{\frac{1}{N}\sum_{(\mathbf{x}'_i,y_i)\in\pi_j(\mathcal{L})} L(\frac{1}{M^{-i}}\sum_{l=1}^{M^{-i}}\varphi_{\mathcal{L}^{m_{k_l}}}(\mathbf{x}'_i), y_i)\right\} \\ &\hookrightarrow -\frac{1}{N}\sum_{(\mathbf{x}_i,y_i)\in\mathcal{L}} L(\frac{1}{M^{-i}}\sum_{l=1}^{M^{-i}}\varphi_{\mathcal{L}^{m_{k_l}}}(\mathbf{x}_i), y_i) \end{aligned} \tag{6.3}$$

onde $\pi_j(\mathcal{L})$ denota uma réplica de $\mathcal{L}$ na qual os valores de X_j foram permutados aleatoriamente, e onde $m_{k_1}, \ldots, m_{k_{M^{-i}}}$ denota os índices das árvores que foram construídas a partir de réplicas de bootstrap que não incluem $(\mathbf{x}_i, y_i)$. Para classificação, a importância da permutação é derivada de forma semelhante à Equação 6.3, exceto que as previsões médias fora do saco são substituídas pela classe que é a mais provável, conforme calculada a partir das estimativas de probabilidade da classe fora do saco. Sua lógica é que a permutação aleatória da variável de entrada X_j deveria quebrar sua associação com a resposta Y. Portanto, se X_j está de facto associado a Y, a permutação dos seus valores também deverá resultar num aumento substancial do erro, como aqui medido pela diferença entre as estimativas out-of-bag do erro de generalização. Ou seja, quanto maior o aumento do erro, mais importante é a variável e vice-versa.

Graças aos populares softwares de aprendizado de máquina [Breiman,2002; Liaw e Wiener, 2002; Pedregosa et al., 2011], ambas estas medidas de importância variável têm demonstrado a sua utilidade prática num número crescente de estudos experimentais. Pouco se sabe, entretanto, sobre seu funcionamento interno. Strobl et al. [2007b] compare MDI e MDA e mostre experimentalmente que o primeiro é tendencioso para algumas variáveis preditoras. Conforme explicado por White e Liu [1994] no caso de árvores de decisão únicas, este viés decorre de uma vantagem injusta dada pelas funções de impureza usuais $i(t)$ em relação a preditores com um grande número de valores (ver Seção 7.2.2). Strobl et al. [2008]mostraram mais tarde que o MDA também é te ndencioso e que superestima a importância das variáveis correlacionadas – embora esse efeito não tenha sido confirmado em um estudo experi mental posterior de Genuer et al. [2010]. Do ponto de vista teórico, Ishwaran [2007] fornece um desenvolvimento teórico detalhado de uma versão simplificada do MDA, fornecendo insights importantes para a compreensão do MDA real.

6.2 THEORETICAL STUDY

6.2.1 *Background*

To be self-contained, we first recall several definitions from information theory (see [Cover and Thomas, 2012], for further properties).

We suppose that we are given a probability space $(\Omega, \mathcal{E}, \mathbb{P})$ and consider random variables defined on it taking a finite number of possible values. We use upper case letters to denote such random variables (e.g. $X, Y, Z, W \ldots$) and calligraphic letters (e.g. $\mathcal{X}, \mathcal{Y}, \mathcal{Z}, \mathcal{W} \ldots$) to denote their image sets (of finite cardinality), and lower case letters (e.g. $x, y, z, w \ldots$) to denote one of their possible values. For a (finite) set of (finite) random variables $X = \{X_1, \ldots, X_p\}$, we denote by $P_X(\mathbf{x}) = P_X(x_1, \ldots, x_p)$ the probability $\mathbb{P}(\{\omega \in \Omega \mid \forall j : 1, \ldots, p : X_j(\omega) = x_j\})$, and by $\mathcal{X} = \mathcal{X}_1 \times \cdots \times \mathcal{X}_p$ the set of joint configurations of these random variables. Given two sets of random variables, $X = \{X_1, \ldots, X_p\}$ and $Y = \{Y_1, \ldots, Y_q\}$, we denote by $P_{X|Y}(\mathbf{x} \mid \mathbf{y}) = P_{X,Y}(\mathbf{x}, \mathbf{y})/P_Y(\mathbf{y})$ the conditional density of X with respect to Y.[1]

With these notations, the joint (Shannon) entropy of a set of random variables $X = \{X_1, \ldots, X_p\}$ is thus defined by

$$H(X) = -\sum_{\mathbf{x} \in \mathcal{X}} P_X(\mathbf{x}) \log_2 P_X(\mathbf{x}), \tag{6.4}$$

while the mean conditional entropy of a set of random variables $X = \{X_1, \ldots, X_p\}$, given the values of another set of random variables $Y = \{Y_1, \ldots, Y_q\}$ is defined by

$$H(X \mid Y) = -\sum_{\mathbf{x} \in \mathcal{X}} \sum_{\mathbf{y} \in \mathcal{Y}} P_{X,Y}(\mathbf{x}, \mathbf{y}) \log_2 P_{X|Y}(\mathbf{x} \mid \mathbf{y}). \tag{6.5}$$

The mutual information among the set of random variables $X = \{X_1, \ldots, X_p\}$ and the set of random variables $Y = \{Y_1, \ldots, Y_q\}$ is defined by

$$\begin{aligned} I(X;Y) &= -\sum_{\mathbf{x} \in \mathcal{X}} \sum_{\mathbf{y} \in \mathcal{Y}} P_{X,Y}(\mathbf{x}, \mathbf{y}) \log_2 \frac{P_X(\mathbf{x}) P_Y(\mathbf{y})}{P_{X,Y}(\mathbf{x}, \mathbf{y})} \\ &= H(X) - H(X \mid Y) \\ &= H(Y) - H(Y \mid X) \end{aligned} \tag{6.6}$$

The mean conditional mutual information among the set of random variables $X = \{X_1, \ldots, X_p\}$ and the set of random variables $Y = \{Y_1, \ldots, Y_q\}$, given the values of a third set of random variables $Z = \{Z_1, \ldots, Z_r\}$, is defined by

$$I(X;Y \mid Z) = H(X \mid Z) - H(X \mid Y, Z) \tag{6.7}$$

1 To avoid problems, we suppose that all probabilities are strictly positive, without fundamental limitation.

6.2 estudo teórico

6.2.1 Antecedentes

Para sermos independentes, primeiro recordamos diversas definições da teoria da informação (ver [Cover e Thomas, 2012], para mais propriedades).

Supomos que nos é dado um espaço de probabilidade $(\Omega, \mathcal{E}, \mathbb{P})$ e consideramos variáveis aleatórias definidas nele tomando um número finito de valores possíveis. Usamos letras maiúsculas para denotar tais variáveis aleatórias (por exemplo, $X, Y, Z, W \ldots$) e letras caligráficas (por exemplo, $\mathcal{X}, \mathcal{Y}, \mathcal{Z}, \mathcal{W} \ldots$) para denotar seus conjuntos de imagens (de cardinalidade finita), e letras minúsculas (por exemplo, $x, y, z, w \ldots$) para denotar um de seus valores possíveis. Para um conjunto (finito) de variáveis aleatórias (finitas) $X = \{X_1, \ldots, X_p\}$, denotamos por $P_X(\mathbf{x}) = P_X(x_1, \ldots, x_p)$ a probabilidade $\mathbb{P}(\{\omega \in \Omega \mid \forall j : 1, \ldots, p : X_j(\omega) = x_j\})$, e por $\mathcal{X} = \mathcal{X}_1 \times \cdots \times \mathcal{X}_p$ o conjunto de configurações conjuntas dessas variáveis aleatórias. Dados dois conjuntos de variáveis aleatórias, $X = \{X_1, \ldots, X_p\}$ e $Y = \{Y_1, \ldots, Y_q\}$, denotamos por $P_{X|Y}(\mathbf{x} \mid \mathbf{y}) = P_{X,Y}(\mathbf{x}, \mathbf{y})/P_Y(\mathbf{y})$ a densidade condicional de X em relação a Y.[1]

Com essas notações, a entropia conjunta (Shannon) de um conjunto de variáveis aleatórias $X = \{X_1, \ldots, X_p\}$ é assim definida por

$$H(X) = -\sum_{\mathbf{x} \in \mathcal{X}} P_X(\mathbf{x}) \log_2 P_X(\mathbf{x}), \tag{6.4}$$

enquanto a entropia condicional média de um conjunto de variáveis aleatórias $X = \{X_1, \ldots, X_p\}$, dados os valores de outro conjunto de variáveis aleatórias $Y = \{Y_1, \ldots, Y_q\}$ é definida por

$$H(X \mid Y) = -\sum_{\mathbf{x} \in \mathcal{X}} \sum_{\mathbf{y} \in \mathcal{Y}} P_{X,Y}(\mathbf{x}, \mathbf{y}) \log_2 P_{X|Y}(\mathbf{x} \mid \mathbf{y}). \tag{6.5}$$

A informação mútua entre o conjunto de variáveis aleatórias $X = \{X_1, \ldots, X_p\}$ e o conjunto de variáveis aleatórias $Y = \{Y_1, \ldots, Y_q\}$ é definida por

$$\begin{aligned} I(X;Y) &= -\sum_{\mathbf{x} \in \mathcal{X}} \sum_{\mathbf{y} \in \mathcal{Y}} P_{X,Y}(\mathbf{x}, \mathbf{y}) \log_2 \frac{P_X(\mathbf{x}) P_Y(\mathbf{y})}{P_{X,Y}(\mathbf{x}, \mathbf{y})} \\ &= H(X) - H(X \mid Y) \\ &= H(Y) - H(Y \mid X) \end{aligned} \tag{6.6}$$

A informação mútua condicional média entre o conjunto de variáveis aleatórias $X = \{X_1, \ldots, X_p\}$ e o conjunto de variáveis aleatórias $Y = \{Y_1, \ldots, Y_q\}$, dados os valores de um terceiro conjunto de variáveis aleatórias $Z = \{Z_1, \ldots, Z_r\}$, é definida por

$$I(X;Y \mid Z) = H(X \mid Z) - H(X \mid Y, Z) \tag{6.7}$$

1 Para evitar problemas, supomos que todas as probabilidades são estritamente positivas, sem limitação fundamental.

$$= H(Y \mid Z) - H(Y \mid X, Z)$$
$$= -\sum_{x \in \mathcal{X}} \sum_{y \in \mathcal{Y}} \sum_{z \in \mathcal{Z}} P_{X,Y,Z}(x, y, z) \log_2 \frac{P_{X|Z}(x \mid z) P_{Y|Z}(y \mid z)}{P_{X,Y|Z}(x, y \mid z)}$$

We also recall the chaining rule

$$I(X, Z; Y \mid W) = I(X; Y \mid W) + I(Z; Y \mid W, X), \tag{6.8}$$

and the symmetry of the (conditional) mutual information among sets of random variables

$$I(X; Y \mid Z) = I(Y; X \mid Z). \tag{6.9}$$

6.2.2 *Asymptotic analysis*

Let us now consider the MDI importance as defined by Equation 6.2, and let us assume a set $V = \{X_1, ..., X_p\}$ of *categorical* input variables and a *categorical* output Y. For the sake of simplicity we will use the Shannon entropy as impurity measure, and focus on totally randomized trees; later on we will discuss other impurity measures and tree construction algorithms.

Given a training sample $\mathcal{L}$ of N joint observations of $X_1, ..., X_p, Y$ independently drawn from the joint distribution $P(X_1, ..., X_p, Y)$, let us assume that we infer from it an infinitely large ensemble of *totally randomized and fully developed trees*.

Definition 6.1. *A* totally randomized and fully developed tree *is a decision tree in which each node* t *is partitioned using a variable* X_j *picked uniformly at random among those not yet used at the parent nodes of* t*, where each node* t *is split into* $|\mathcal{X}_j|$ *sub-trees, i.e., one for each possible value of* $\mathcal{X}_j$*, and where the recursive construction process halts only when all* p *variables have been used along the current branch.*

In such a tree, leaves are all at the same depth p, and the set of leaves of a fully developed tree is in bijection with the set $\mathcal{X}$ of all possible joint configurations of the p input variables. For example, if the input variables are all binary, the resulting tree will have exactly 2^p leaves. As a result, totally randomized trees, as considered here, are in fact closer to trees grown with ID3 [Quinlan, 1986] than trees built from the CART procedure (as described throughout Chapters 3 and 4).

Theorem 6.1. *The MDI importance of* $X_j \in V$ *for* Y *as computed with an infinite ensemble of fully developed totally randomized trees and an infinitely large training sample is:*

$$Imp(X_j) = \sum_{k=0}^{p-1} \frac{1}{C_p^k} \frac{1}{p-k} \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B), \tag{6.10}$$

$$= H(Y \mid Z) - H(Y \mid X, Z)$$
$$= -\sum_{x \in \mathcal{X}} \sum_{y \in \mathcal{Y}} \sum_{z \in \mathcal{Z}} P_{X,Y,Z}(x, y, z) \log_2 \frac{P_{X|Z}(x \mid z) P_{Y|Z}(y \mid z)}{P_{X,Y|Z}(x, y \mid z)}$$

Lembramos também a regra do encadeamento

$$I(X, Z; Y \mid W) = I(X; Y \mid W) + I(Z; Y \mid W, X), \tag{6.8}$$

e a simetria da informação mútua (condicional) entre conjuntos de variáveis aleatórias

$$I(X; Y \mid Z) = I(Y; X \mid Z). \tag{6.9}$$

6.2.2 Análise assintótica

Consideremos agora a importância do MDI conforme definido pela Equação 6.2, e assumamos um conjunto $V = \{X_1, ..., X_p\}$ de variáveis de entrada categóricas e uma saída categórica Y. Por uma questão de simplicidade usaremos a entropia de Shannon como medida de impureza e focaremos em árvores totalmente aleatórias; mais tarde discutiremos outras medidas de impureza e algoritmos de construção de árvores.

Dada uma amostra de treinamento $\mathcal{L}$ de N observações conjuntas de $X_1, ..., X_p, Y$ extraídas independentemente da distribuição conjunta $P(X_1, ..., X_p, Y)$, vamos supor que inferimos dela um conjunto infinitamente grande de árvores totalmente aleatórias e totalmente desenvolvidas.

Definição 6.1. Uma árvore totalmente aleatória e totalmente desenvolvida é uma árvore de decisão na qual cada nó t *é particionado usando uma variável* X_j *escolhida uniformemente aleatoriamente entre aquelas ainda não usadas nos nós pais de* t*, onde cada nó* t *é dividido em* $|\mathcal{X}_j|$ *subárvores, ou seja, uma para cada valor possível de* $\mathcal{X}_j$*, e onde o processo de construção recursiva para apenas quando todas as variáveis* p *forem usadas ao longo do ramo atual.*

Em tal árvore, as folhas estão todas na mesma profundidade p, e o conjunto de folhas de uma árvore totalmente desenvolvida está em bijeção com o conjunto $\mathcal{X}$ de todas as configurações conjuntas possíveis das p variáveis de entrada. Por exemplo, se as variáveis de entrada forem todas binárias, a árvore resultante terá exatamente 2^p folhas. Como resultado, as árvores totalmente aleatórias, conforme consideradas aqui, estão de fato mais próximas das árvores cultivadas com ID3 [Quinlan, 1986] do que das árvores construídas a partir do procedimento CART (conforme descrito nos Capítulos 3 e 4).

Teorema 6.1. A importância do MDI de $X_j \in V$ *para* Y *computado com um conjunto infinito de árvores totalmente aleatórias e totalmente desenvolvidas e uma amostra de treinamento infinitamente grande é:*

$$Imp(X_j) = \sum_{k=0}^{p-1} \frac{1}{C_p^k} \frac{1}{p-k} \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B), \tag{6.10}$$

where V^{-j} denotes the subset $V \setminus \{X_j\}$, $\mathcal{P}_k(V^{-j})$ is the set of subsets of V^{-j} of cardinality k, and $I(X_j; Y|B)$ is the conditional mutual information of X_j and Y given the variables in B.

Proof. By expanding $\Delta i(s,t) = i(t) - p_L i(t_L) - p_R i(t_R)$ into Equation 6.2 and using the entropy $H(Y|t) = -\sum_j p(j|t) \log_2(p(j|t))$ as impurity measure $i(t)$, Equation 6.2 can be rewritten in terms of mutual information:

$$\text{Imp}(X_j) = \frac{1}{M} \sum_{m=1}^{M} \sum_{t \in \varphi_m} 1(j_t = j) p(t) I(Y; X_j|t) \tag{6.11}$$

As the size N of the training sample grows to infinity, $p(t)$ becomes the (exact) probability (according to $P(X_1, \ldots, X_p, Y)$) that an object reaches node t, i.e., a probability $P(B(t) = b(t))$ where $B(t) = (X_{i_1}, ..., X_{i_k})$ is the subset of k variables tested in the branch from the root node to the parent of t and $b(t)$ is the vector of values of these variables. As the the number M of totally randomized trees also grows to infinity, the importance of a variable X_j can then be written:

$$\text{Imp}(X_j) = \sum_{B \subseteq V^{-j}} \sum_{b \in \mathcal{X}_{i_1} \times ... \times \mathcal{X}_{i_k}} \alpha(B, b, X_j, p) P(B = b) I(Y; X_j|B = b), \tag{6.12}$$

where b is a set of values for the variables in B and $\alpha(B, b, X_j, p)$ is the probability that a node t (at depth k) in a totally randomized tree tests the variable X_j and is such that $B(t) = B$ and $b(t) = b$.

Let us compute $\alpha(B, b, X_j, p)$. First, let us consider the probability that a node t tests the variable X_j and is such that the branch leading to t follows a path defined, in that particular order, by all k variables $X_{i_1}, ..., X_{i_k} \in B$ and their corresponding values in b. The probability of that branch is the probability of picking (uniformly at random) X_{i_1} at the root node times the probability of testing, in that order, the remaining $X_{i_2}, ..., X_{i_k}$ variables in the sub-tree corresponding to the value x_{i_1} of X_{i_1} defined in b. Note that, by construction, it is certain that this particular sub-tree exists since the root node is split into $|\mathcal{X}_{i_1}|$ sub-trees. Then, the probability of testing X_j at the end of this branch is the probability of picking X_j among the remaining $p - k$ variables. By recursion, we thus have:

$$\frac{1}{p} \frac{1}{p-1} \cdots \frac{1}{p-k+1} \frac{1}{p-k} = \frac{(p-k)!}{p!} \frac{1}{p-k} \tag{6.13}$$

Since the order along which the variables appear in the branch is of no importance, $\alpha(B, b, X_j, p)$ actually includes all $k!$ equiprobable ways of building a branch composed of the variables and values in B and b. Then, since a tree may at most contain a single such branch,

onde V^{-j} denota o subconjunto $V \setminus \{X_j\}$, $\mathcal{P}_k(V^{-j})$ é o conjunto de subconjuntos de V^{-j} de cardinalidade k, e $I(X_j; Y|B)$ é a informação mútua condicional de X_j e Y dadas as variáveis em B.

Prova. Expandindo $\Delta i(s,t) = i(t) - p_L i(t_L) - p_R i(t_R)$ na Equação 6.2 e usando o $H(Y|t) = -\sum_j p(j|t) \log_2(p(j|t))$ de entropia como medida de impureza $i(t)$, a Equação 6.2 pode ser reescrita em termos de informação mútua:

$$\text{Imp}(X_j) = \frac{1}{M} \sum_{m=1}^{M} \sum_{t \in \varphi_m} 1(j_t = j) p(t) I(Y; X_j|t) \tag{6.11}$$

À medida que o tamanho N da amostra de treinamento cresce até o infinito, $p(t)$ torna-se a probabilidade (exata) (de acordo com $P(X_1, \ldots, X_p, Y)$) de que um objeto alcance o nó t, ou seja, uma probabilidade $P(B(t) = b(t))$ onde $B(t) = (X_{i_1}, ..., X_{i_k})$ é o subconjunto de k variáveis testadas na ramificação do nó raiz até o pai de t e $b(t)$ é o vetor de valores dessas variáveis. À medida que o número M de árvores totalmente aleatórias também cresce até o infinito, a importância de uma variável X_j pode então ser escrita:

$$\text{Imp}(X_j) = \sum_{B \subseteq V^{-j}} \sum_{b \in \mathcal{X}_{i_1} \times ... \times \mathcal{X}_{i_k}} \alpha(B, b, X_j, p) P(B = b) I(Y; X_j|B = b), \tag{6.12}$$

onde b é um conjunto de valores para as variáveis em B e $\alpha(B, b, X_j, p)$ é a probabilidade de um nó t (na profundidade k) em uma árvore totalmente aleatória testar a variável X_j e é tal que $B(t) = B$ e $b(t) = b$.

Vamos calcular $\alpha(B, b, X_j, p)$. Primeiro, vamos considerar a probabilidade de um nó t testar a variável X_j e ser tal que o ramo que leva a t segue um caminho definido, nessa ordem específica, por todas as k variáveis$X_{i_1}, ..., X_{i_k} \in B$ e seus valores correspondentes em b. A probabilidade desse ramo é a probabilidade de escolher (uniformemente aleatoriamente) X_{i_1}no nó raiz vezes a probabilidade de testar, nessa ordem, as restantes $X_{i_2}, ..., X_{i_k}$ variáveis na subárvore correspondente ao valor x_{i_1} de X_{i_1} definido em b. Observe que, por construção, é certo que esta subárvore específica existe, uma vez que o nó raiz é dividido em $|\mathcal{X}_{i_1}|$ subárvores. Então, a probabilidade de testar X_j no final deste ramo é a probabilidade de escolher X_j entre as variáveis $p - k$ restantes. Por recursão, temos assim:

$$\frac{1}{p} \frac{1}{p-1} \cdots \frac{1}{p-k+1} \frac{1}{p-k} = \frac{(p-k)!}{p!} \frac{1}{p-k} \tag{6.13}$$

Como a ordem em que as variáveis aparecem no ramo não tem importância, $\alpha(B, b, X_j, p)$ na verdade inclui todos $k!$ maneiras equiprováveis de construir um ramo composto pelas variáveis e valores em Be b. Então, como uma árvore pode conter no máximo um único galho,

whatever the order of the tests, the probabilities may be added up and it comes:

$$\alpha(B, b, X_j, p) = k!\frac{(p-k)!}{p!}\frac{1}{p-k} = \frac{1}{C_p^k}\frac{1}{p-k} \tag{6.14}$$

From the above expression, it appears that $\alpha(B, b, X_j, p)$ depends only on the size k of B and on the number p of variables. As such, by grouping in the previous equation of $\text{Imp}(X_j)$ conditioning variable subsets B according to their sizes and using the definition of conditional mutual information, α can be factored out, hence leading to the form foretold by Theorem 6.1:

$$\text{Imp}(X_j) = \sum_{k=0}^{p-1} \frac{1}{C_p^k}\frac{1}{p-k} \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B). \tag{6.15}$$

□

Theorem 6.2. *For any ensemble of fully developed trees in asymptotic learning sample size conditions (e.g., in the same conditions as those of Theorem 6.1), we have that*

$$\sum_{j=1}^{p} Imp(X_j) = I(X_1, \dots, X_p; Y). \tag{6.16}$$

Proof. For any tree φ, we have that the sum of all importances estimated by using an infinitely large sample $\mathcal{L}$ (or equivalently, by assuming perfect knowledge of the joint distribution $P(X_1, ..., X_p, Y)$) is equal to $H(Y) - \sum_{t \in \widetilde{\varphi}} p(t)H(Y|b(t))$, where $\widetilde{\varphi}$ denotes the set of all leaves of φ, and where $b(t)$ denotes the joint configuration of all input variables leading to leaf t. This is true because the impurities of all test nodes intervening in the computation of the variable importances, except the impurity $H(Y)$ at the root node of the tree, cancel each other when summing up the importances.

Since, when the tree is fully developed, $\sum_{t \in \widetilde{\varphi}} p(t)H(Y|b(t))$ is obviously equal to the mean conditional entropy $H(Y|X_1, \dots, X_p)$ of Y given all input variables, this implies that for any fully developed tree we have that the sum of variable importances is equal to $I(X_1, \dots, X_p; Y)$, and so this relation also holds when averaging over an infinite ensemble of totally randomized trees. Note that, for the same reasons as in the proof of Theorem 6.1, this result holds in asymptotic conditions only, i.e., when $p(t)$ becomes the exact probability $P(B(t) = b(t))$. □

Together, theorems 6.1 and 6.2 show that variable importances derived from totally randomized trees in asymptotic conditions provide a three-level decomposition of the information $I(X_1, \dots, X_p; Y)$ contained in the set of input variables about the output variable. The first level is a decomposition among input variables (see Equation 6.16 of

qualquer que seja a ordem dos testes, as probabilidades podem ser somadas e resulta:

$$\alpha(B, b, X_j, p) = k!\frac{(p-k)!}{p!}\frac{1}{p-k} = \frac{1}{C_p^k}\frac{1}{p-k} \tag{6.14}$$

Da expressão acima, parece que $\alpha(B, b, X_j, p)$ depende apenas do tamanho k de Be do número p de variáveis. Como tal, agrupando na equação anterior de $\text{Imp}(X_j)$ subconjuntos de variáveis condicionantes B de acordo com seus tamanhos e usando a definição de informação mútua condicional, αpode ser fatorado, levando assim à forma predita pelo Teorema6.1:

$$\text{Imp}(X_j) = \sum_{k=0}^{p-1} \frac{1}{C_p^k}\frac{1}{p-k} \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B). \tag{6.15}$$

□

Teorema 6.2. Para qualquer conjunto de árvores totalmente desenvolvidas em condições de tamanho de amostra de aprendizagem assintótica (por exemplo, nas mesmas condições do Teorema 6.1), temos que

$$\sum_{j=1}^{p} Imp(X_j) = I(X_1, \dots, X_p; Y). \tag{6.16}$$

Prova. Para qualquer árvore φ, temos que a soma de todas as importâncias estimadas usando uma amostra infinitamente grande $\mathcal{L}$ (ou equivalentemente, assumindo o conhecimento perfeito da distribuição conjunta $P(X_1, ..., X_p, Y)$) é igual a $H(Y) - \sum_{t \in \widetilde{\varphi}} p(t)H(Y|b(t))$, onde $\widetilde{\varphi}$ denota o conjunto de todas as folhas de φ, e onde $b(t)$ denota a configuração conjunta de todas as variáveis de entrada que levam à folha t. Isto é verdade porque as impurezas de todos os nós de teste que intervêm no cálculo das importâncias das variáveis, exceto a impureza $H(Y)$ no nó raiz da árvore, cancelam-se mutuamente ao somar as importâncias.

Visto que, quando a árvore está totalmente desenvolvida, $\sum_{t \in \widetilde{\varphi}} p(t)H(Y|b(t))$ é obviamente igual à entropia condicional média $H(Y|X_1, \dots, X_p)$ de Ydadas todas as variáveis de entrada, isto implica que para qualquer árvore totalmente desenvolvida temos que a soma das importâncias das variáveis é igual a $I(X_1, \dots, X_p; Y)$, e assim esta relação também se mantém quando se calcula a média de um conjunto infinito de árvores totalmente aleatórias. Observe que, pelas mesmas razões da prova do Teorema 6.1, este resultado é válido apenas em condições assintóticas, ou seja, quando $p(t)$ se torna a probabilidade exata $P(B(t) = b(t))$. □

Juntos, os teoremas 6.1 e 6.2 mostram que as importâncias das variáveis derivadas de árvores totalmente aleatórias em condições assintóticas fornecem uma decomposição em três níveis da informação $I(X_1, \dots, X_p; Y)$ contida no conjunto de variáveis de entrada sobre a variável de saída. O primeiro nível é uma decomposição entre variáveis de entrada (ver Equação 6.16 do

Theorem 6.2), the second level is a decomposition along the degrees k of interaction terms of a variable with the other ones (see the outer sum in Equation 6.10 of Theorem 6.1), and the third level is a decomposition along the combinations B of interaction terms of fixed size k of possible interacting variables (see the inner sum in Equation 6.10).

We observe that the decomposition includes, for each variable, each and every interaction term of each and every degree weighted in a fashion resulting only from the combinatorics of possible interaction terms. In particular, since all $I(X_j; Y|B)$ terms are at most equal to $H(Y)$, the prior entropy of Y, the p terms of the outer sum of Equation 6.10 are each upper bounded by

$$\frac{1}{C_p^k}\frac{1}{p-k}\sum_{B\in\mathcal{P}_k(V^{-j})} H(Y) = \frac{1}{C_p^k}\frac{1}{p-k}C_{p-1}^k H(Y) = \frac{1}{p}H(Y). \quad (6.17)$$

As such, the second level decomposition resulting from totally randomized trees makes the p sub-importance terms

$$\frac{1}{C_p^k}\frac{1}{p-k}\sum_{B\in\mathcal{P}_k(V^{-j})} I(X_j; Y|B) \quad (6.18)$$

to equally contribute (at most) to the total importance, even though they each include a combinatorially different number of terms.

6.3 RELEVANCE OF VARIABLES

Following Kohavi and John [1997], let us define relevant and irrelevant variable as follows:

Definition 6.2. *A variable X_j is* relevant to Y with respect to V *if there exists at least one subset $B \subseteq V$ (possibly empty) such that $I(X_j; Y|B) > 0$.*

Definition 6.3. *A variable X_i is* irrelevant to Y with respect to V *if, for all $B \subseteq V$, $I(X_i; Y|B) = 0$.*

Remark that if X_i is irrelevant to Y with respect to V, then by definition it is also irrelevant to Y with respect to any subset of V. However, if X_j is relevant to Y with respect to V, then it is not necessarily relevant to Y with respect to all subsets of V. Among the relevant variables, we also distinguish the *marginally* relevant ones, for which $I(X_j; Y) > 0$, the *strongly* relevant ones, for which $I(X_j; Y|V^{-j}) > 0$, and the *weakly* relevant variables, which are relevant but not strongly relevant.

Theorem 6.3. *$X_i \in V$ is irrelevant to Y with respect to V if and only if its infinite sample size importance as computed with an infinite ensemble of fully developed totally randomized trees built on V for Y is 0.*

Teorema 6.2), o segundo nível é uma decomposição ao longo dos graus k de termos de interação de uma variável com as outras (veja a soma externa na Equação 6.10 do Teorema 6.1), e o terceiro nível é uma decomposição ao longo das combinações B de termos de interação de tamanho fixo k de possíveis variáveis interagentes (veja a soma interna na Equação 6.10).

Observamos que a decomposição inclui, para cada variável, todo e qualquer termo de interação de cada grau ponderado de forma resultante apenas da combinatória dos possíveis termos de interação. Em particular, como todos os termos $I(X_j; Y|B)$ são no máximo iguais a $H(Y)$, a entropia anterior de Y, os termos p da soma externa da Equação 6.10 são, cada u m, limitados superiormente por

$$\frac{1}{C_p^k}\frac{1}{p-k}\sum_{B\in\mathcal{P}_k(V^{-j})} H(Y) = \frac{1}{C_p^k}\frac{1}{p-k}C_{p-1}^k H(Y) = \frac{1}{p}H(Y). \quad (6.17)$$

Como tal, a decomposição de segundo nível resultante de árvores totalmente aleatórias torna os termos de subimportância p

$$\frac{1}{C_p^k}\frac{1}{p-k}\sum_{B\in\mathcal{P}_k(V^{-j})} I(X_j; Y|B) \quad (6.18)$$

contribuir igualmente (no máximo) para a importância total, embora cada um inclua um número de termos combinatoriamente diferente.

6.3 relevância das variáveis

Seguindo Kohavi e John [1997],vamos definir variáveis relevantes e irrelevantes da seguinte forma:

Definição 6.2. Uma variável X_j é relevante para Y em relação a V se existir pelo menos um subconjunto $B \subseteq V$ (possivelmente vazio) tal que $I(X_j; Y|B) > 0$.

Definição 6.3. Uma variável X_i é irrelevante para Y com relação a V se, para todos $B \subseteq V$, $I(X_i; Y|B) = 0$.

Observe que se X_i é irrelevante para Y com relação a V, então, por definição, também é irrelevante para Y com relação a qualquer subconjunto de V. Contudo, se X_j é relevante para Y com respeito a V, então não é necessariamente relevante para Y com respeito a todos os subconjuntos de V. Entre as variáveis relevantes, distinguimos também as marginalmente relevantes, para as quais $I(X_j; Y) > 0$, as fortemente relevantes, para as quais $I(X_j; Y|V^{-j}) > 0$, e as variáveis fracamente relevantes, que são relevantes, mas não fortemente relevantes.

Teorema 6.3. $X_i \in V$ é irrelevante para Y em relação a V se e somente se a importância do seu tamanho de amostra infinito, conforme calculado com um conjunto infinito de árvores totalmente desenvolvidas e totalmente aleatórias construídas em V para Y é 0.

Proof. The proof directly results from the definition of irrelevance. If X_i is irrelevant with respect to V, then $I(X_i; Y|B)$ is zero for all $B \subseteq V^{-i} \subset V$ and Equation 6.10 reduces to 0. Also, since $I(X_i; Y|B)$ is non-negative for any B, $\text{Imp}(X_i)$ is zero if and only if all its $I(X_i; Y|B)$ terms are zero. Since $\text{Imp}(X_i)$ includes all $I(X_i; Y|B)$ terms for $B \subseteq V^{-i}$, and since all of them are therefore null if $\text{Imp}(X_i) = 0$, X_i is thus, by definition, irrelevant with respect to V^{-i}. X_i is then also trivially irrelevant with respect to $V = V^{-i} \cup \{X_i\}$ since $I(X_i; Y|B \cup \{X_i\}) = 0$ for any B. □

Lemma 6.4. *Let $X_i \notin V$ be an irrelevant variable for Y with respect to V. The infinite sample size importance of $X_j \in V$ as computed with an infinite ensemble of fully developed totally randomized trees built on V for Y is the same as the importance derived when using $V \cup \{X_i\}$ to build the ensemble of trees for Y.*

Proof. Let $X_i \notin V$ be an irrelevant variable with respect to V. For $X_j \in V$, $B \subseteq V^{-j}$, using the chain rules of mutual information, we have:

$$I(X_j, X_i; Y|B) = I(X_j; Y|B) + I(X_i; Y|B \cup \{X_j\}) \tag{6.19}$$
$$= I(X_i; Y|B) + I(X_j; Y|B \cup \{X_i\}) \tag{6.20}$$

If X_i is irrelevant with respect to V, i.e., such that $I(X_i; Y|B) = 0$ for all $B \subseteq V$, then $I(X_i; Y|B \cup \{X_j\})$ and $I(X_i; Y|B)$ both equal 0, leading to

$$I(X_j; Y|B \cup \{X_i\}) = I(X_j; Y|B) \tag{6.21}$$

Then, from Theorem 6.1, the importance of X_j as computed with an infinite ensemble of totally randomized trees built on $V \cup \{X_i\}$ can be simplified to:

$$\begin{aligned}
\text{Imp}(X_j) &= \sum_{k=0}^{p-1+1} \frac{1}{C_{p+1}^k} \frac{1}{p+1-k} \sum_{B \in \mathcal{P}_k(V^{-j} \cup \{X_i\})} I(X_j; Y|B) \\
&= \sum_{k=0}^{p} \frac{1}{C_{p+1}^k} \frac{1}{p+1-k} \left[\sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B) + \sum_{B \in \mathcal{P}_{k-1}(V^{-j})} I(X_j; Y|B \cup \{X_i\}) \right] \\
&= \sum_{k=0}^{p-1} \frac{1}{C_{p+1}^k} \frac{1}{p+1-k} \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B) + \\
&\hookrightarrow \sum_{k=1}^{p} \frac{1}{C_{p+1}^k} \frac{1}{p+1-k} \sum_{B \in \mathcal{P}_{k-1}(V^{-j})} I(X_j; Y|B) \\
&= \sum_{k=0}^{p-1} \frac{1}{C_{p+1}^k} \frac{1}{p+1-k} \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B) + \\
&\hookrightarrow \sum_{k'=0}^{p-1} \frac{1}{C_{p+1}^{k'+1}} \frac{1}{p+1-k'-1} \sum_{B \in \mathcal{P}_{k'}(V^{-j})} I(X_j; Y|B)
\end{aligned}$$

Prova. A prova resulta diretamente da definição de irrelevância. Se X_i é irrelevante em relação a V, então $I(X_i; Y|B)$ é zero para todo $B \subseteq V^{-i} \subset V$ e a Equação 6.10 se reduz a 0. Além disso, como $I(X_i; Y|B)$ não é negativo para qualquer B, $\text{Imp}(X_i)$ é zero se e somente se todos os seus termos $I(X_i; Y|B)$ forem zero. Como $\text{Imp}(X_i)$ inclui todos os termos $I(X_i; Y|B)$ para $B \subseteq V^{-i}$, e como todos eles são, portanto, nulos se $\text{Imp}(X_i) = 0$, X_i é, portanto, por definição, irrelevante em relação a V^{-i}. X_i é então tambem trivialmente irrelevante em relação a $V = V^{-i} \cup \{X_i\}$ desde $I(X_i; Y|B \cup \{X_i\}) = 0$ para qualquer B. □

Lema 6.4. Seja $X_i \notin V$ uma variável irrelevante para Y em relação a V. A importância do tamanho de amostra infinito de $X_j \in V$ conforme calculado com um conjunto infinito de árvores totalmente aleatórias totalmente desenvolvidas construídas em V para Y é a mesma que a importância derivada ao usar $V \cup \{X_i\}$ para construir o conjunto de árvores para Y.

Prova. Seja $X_i \notin V$ uma variável irrelevante em relação a V. Para $X_j \in V$, $B \subseteq V^{-j}$, usando as regras da cadeia de informação mútua, temos:

$$I(X_j, X_i; Y|B) = I(X_j; Y|B) + I(X_i; Y|B \cup \{X_j\}) \tag{6.19}$$
$$= I(X_i; Y|B) + I(X_j; Y|B \cup \{X_i\}) \tag{6.20}$$

Se X_i é irrelevante em relação a V, ou seja, tal que $I(X_i; Y|B) = 0$ para todos $B \subseteq V$, então $I(X_i; Y|B \cup \{X_j\})$ e $I(X_i; Y|B)$ ambos iguais a 0, levando a

$$I(X_j; Y|B \cup \{X_i\}) = I(X_j; Y|B) \tag{6.21}$$

Então, a partir do Teorema 6.1, a importância de X_j computado com um conjunto infinito de árvores totalmente aleatórias construídas em $V \cup \{X_i\}$ pode ser simplificada para:

$$\begin{aligned}
\text{Imp}(X_j) &= \sum_{k=0}^{p-1+1} \frac{1}{C_{p+1}^k} \frac{1}{p+1-k} \sum_{B \in \mathcal{P}_k(V^{-j} \cup \{X_i\})} I(X_j; Y|B) \\
&= \sum_{k=0}^{p} \frac{1}{C_{p+1}^k} \frac{1}{p+1-k} \left[\sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B) + \sum_{B \in \mathcal{P}_{k-1}(V^{-j})} I(X_j; Y|B \cup \{X_i\}) \right] \\
&= \sum_{k=0}^{p-1} \frac{1}{C_{p+1}^k} \frac{1}{p+1-k} \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B) + \\
&\hookrightarrow \sum_{k=1}^{p} \frac{1}{C_{p+1}^k} \frac{1}{p+1-k} \sum_{B \in \mathcal{P}_{k-1}(V^{-j})} I(X_j; Y|B) \\
&= \sum_{k=0}^{p-1} \frac{1}{C_{p+1}^k} \frac{1}{p+1-k} \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B) + \\
&\hookrightarrow \sum_{k'=0}^{p-1} \frac{1}{C_{p+1}^{k'+1}} \frac{1}{p+1-k'-1} \sum_{B \in \mathcal{P}_{k'}(V^{-j})} I(X_j; Y|B)
\end{aligned}$$

$$= \sum_{k=0}^{p-1} \left[\frac{1}{C_{p+1}^{k}} \frac{1}{p+1-k} + \frac{1}{C_{p+1}^{k+1}} \frac{1}{p-k} \right] \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B)$$

$$= \sum_{k=0}^{p-1} \frac{1}{C_p^k} \frac{1}{p-k} \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B) \tag{6.22}$$

The last line above exactly corresponds to the importance of X_j as computed with an infinite ensemble of totally randomized trees built on V, which proves Lemma 6.4. □

Theorem 6.5. *Let $V_R \subseteq V$ be the subset of all variables in V that are relevant to Y with respect to V. The infinite sample size importance of any variable $X_j \in V_R$ as computed with an infinite ensemble of fully developed totally randomized trees built on V_R for Y is the same as its importance computed in the same conditions by using all variables in V. That is:*

$$Imp(X_j) = \sum_{k=0}^{p-1} \frac{1}{C_p^k} \frac{1}{p-k} \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B)$$

$$= \sum_{l=0}^{r-1} \frac{1}{C_r^l} \frac{1}{r-l} \sum_{B \in \mathcal{P}_l(V_R^{-j})} I(X_j; Y|B) \tag{6.23}$$

where r is the number of relevant variables in V_R.

Proof. Let us assume that V_R contains $r \leqslant p$ relevant variables. If an infinite ensemble of totally randomized trees were to be built directly on those r variables then, from Theorem 6.1, the importance of a relevant variable X_j would be:

$$Imp(X_j) = \sum_{l=0}^{r-1} \frac{1}{C_r^l} \frac{1}{r-l} \sum_{B \in \mathcal{P}_l(V_R^{-m})} I(X_j; Y|B) \tag{6.24}$$

Let $X_i \in V \setminus V_R$ be one of the $p-r$ irrelevant variables in V with respect to V. Since X_i is also irrelevant with respect to V_R, using Lemma 6.4, the importance of X_j when the ensemble is built on $V_R \cup \{X_i\}$ is the same as the one computed on V_R only (i.e., as computed by the equation above). Using the same argument, adding a second irrelevant variable $X_{i'}$ with respect to V – and therefore also with respect to $V_R \cup \{X_i\}$ – and building an ensemble of totally randomized trees on $V_R \cup \{X_i\} \cup \{X_{i'}\}$ will yield importances that are the same as those computed on $V_R \cup \{X_i\}$, which are themselves the same as those computed by an ensemble built on V_R. By induction, adding all $p-r$ irrelevant variables has therefore no effect on the importance of X_j, which means that:

$$Imp(X_j) = \sum_{k=0}^{p-1} \frac{1}{C_p^k} \frac{1}{p-k} \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B)$$

$$= \sum_{k=0}^{p-1} \left[\frac{1}{C_{p+1}^{k}} \frac{1}{p+1-k} + \frac{1}{C_{p+1}^{k+1}} \frac{1}{p-k} \right] \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B)$$

$$= \sum_{k=0}^{p-1} \frac{1}{C_p^k} \frac{1}{p-k} \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B) \tag{6.22}$$

A última linha acima corresponde exatamente à importância de X_j computado com um conjunto infinito de árvores totalmente aleatórias construídas em V, o que prova o Lema 6.4. □

Teorema 6.5. Seja $V_R \subseteq V$ o subconjunto de todas as variáveis em V que são relevantes para Y em relação a V. A importância do tamanho de amostra infinito de qualquer variável $X_j \in V_R$ conforme calculada com um conjunto infinito de árvores totalmente aleatórias totalmente desenvolvidas construídas em V_R para Y é a mesma que sua importância calculada nas mesmas condições usando todas as variáveis em V. Aquilo é:

$$Imp(X_j) = \sum_{k=0}^{p-1} \frac{1}{C_p^k} \frac{1}{p-k} \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B)$$

$$= \sum_{l=0}^{r-1} \frac{1}{C_r^l} \frac{1}{r-l} \sum_{B \in \mathcal{P}_l(V_R^{-j})} I(X_j; Y|B) \tag{6.23}$$

onde r é o número de variáveis relevantes em V_R.

Prova. Suponhamos que V_R contenha $r \leqslant p$ variáveis relevantes. Se um conjunto infinito de árvores totalmente aleatórias fosse construído diretamente sobre essas r variáveis então, a partir do Teorema 6.1, a importância de uma variável relevante X_j seria:

$$Imp(X_j) = \sum_{l=0}^{r-1} \frac{1}{C_r^l} \frac{1}{r-l} \sum_{B \in \mathcal{P}_l(V_R^{-m})} I(X_j; Y|B) \tag{6.24}$$

Seja $X_i \in V \setminus V_R$ uma das $p-r$ variáveis irrelevantes em V em relação a V. Como X_i também é irrelevante em relação a V_R, usando o Lema 6.4, a importância de X_j quando o conjunto é construído em $V_R \cup \{X_i\}$ é a mesma que aquela calculada apenas em V_R (ou seja, conforme calculado pela equação acima). Usando o mesmo argumento, adicionar uma segunda variável irrelevante $X_{i'}$ em relação a V – e, portanto, também em relação a $V_R \cup \{X_i\}$ – e construir um conjunto de árvores totalmente aleatórias em $V_R \cup \{X_i\} \cup \{X_{i'}\}$ produzirá importâncias que são as mesmas que aquelas calculadas em $V_R \cup \{X_i\}$, que são elas próprias as mesmas que aquelas calculadas por um conjunto construído em V_R. Por indução, adicionar todas as variáveis $p-r$ irrelevantes não tem, portanto, efeito sobre a importância de X_j, o que significa que:

$$Imp(X_j) = \sum_{k=0}^{p-1} \frac{1}{C_p^k} \frac{1}{p-k} \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B)$$

$$= \sum_{l=0}^{r-1} \frac{1}{C_r^l} \frac{1}{r-l} \sum_{B \in \mathcal{P}_l(V_R^{-m})} I(X_j; Y|B) \tag{6.25}$$

□

Theorems 6.3 and 6.5 show that the importances computed with an ensemble of totally randomized trees depends only on the relevant variables. Irrelevant variables have a zero importance and do not affect the importance of relevant variables. Practically, we believe that such properties are desirable conditions for a sound criterion assessing the importance of a variable. Indeed, noise should not be credited of any importance and should not make any other variable more (or less) important.

Intuitively, the independence with respect to irrelevant variables can be partly attributed to the fact that splitting at t on some irrelevant variable X_i should only dillute the local importance $p(t)\Delta i(t)$ of a relevant variable X_j into the children t_L and t_R, but not affect the total sum. For instance, if X_j was to be used at t, then the local importance would be proportional to $p(t)$. By contrast, if X_i was to be used at t and X_j at t_L and t_R, then the sum of the local importances for X_j would be proportional to $p(t_L) + p(t_R) = p(t)$, which does not change anything. Similarly, one can recursively invoke the same argument if X_j was to be used deeper in t_L or t_R.

A second reason comes from the fact that local importances are collected only in nodes t where X_j is used. By contrast, if local importances were summed over all nodes (e.g., using surrogate splits), then it would necessarily depend on the total number of nodes in a tree, which itself directly depends on p – that is, not on r.

Finally, it is also worth noting that this result is consistent with the work of Biau [2012], who proved that rate of convergence of forests of randomized trees also only depends on the relevant variables.

6.4 VARIABLE IMPORTANCES IN RANDOM FOREST VARIANTS

In this section, we consider and discuss variable importances as computed with other types of ensembles of randomized trees. We first show how our results extend to any other impurity measure, and then analyze importances computed by depth-pruned ensemble of randomized trees and those computed by randomized trees built on random subspaces of fixed size. Finally, we discuss the case of non-totally randomized trees.

6.4.1 *Generalization to other impurity measures*

Although our characterization in sections 6.1 and 6.3 uses Shannon entropy as the impurity measure, theorems 6.1, 6.3 and 6.5 hold for

$$= \sum_{l=0}^{r-1} \frac{1}{C_r^l} \frac{1}{r-l} \sum_{B \in \mathcal{P}_l(V_R^{-m})} I(X_j; Y|B) \tag{6.25}$$

□

Os Teoremas 6.3 e 6.5 mostram que as importâncias calculadas com um conjunto de árvores totalmente aleatórias dependem apenas das variáveis relevantes. Variáveis irrelevantes têm importância zero e não afetam a importância das variáveis relevantes. Na prática, acreditamos que tais propriedades são condições desejáveis para um critério sólido que avalie a importância de uma variável. Na verdade, o ruído não deve ser creditado como tendo qualquer importância e não deve tornar qualquer outra variável mais (ou menos) importante.

Intuitivamente, a independência em relação a variáveis irrelevantes pode ser parcialmente atribuída ao fato de que a divisão em t em alguma variável irrelevante X_i deve apenas diluir a importância local $p(t)\Delta i(t)$de uma variável relevante X_j nos filhos t_L e t_R, mas não afeta a soma total. Por exemplo, se X_j fosse usado em t, então a importância local seria proporcional a $p(t)$. Por outro lado, se X_i fosse usado em t e X_j em t_L e t_R, então a soma das importâncias locais para X_j seria proporcional a $p(t_L) + p(t_R) = p(t)$, o que não muda nada. Da mesma forma, pode-se invocar recursivamente o mesmo argumento se X_j fosse usado mais profundamente em t_L ou t_R.

Uma segunda razão vem do fato de que as importâncias locais são coletadas apenas em nós t onde X_j é usado. Por outro lado, se as importâncias locais fossem somadas em todos os nós (por exemplo, usando divisões substitutas), então isso dependeria necessariamente do número total de nós e m uma árvore, que por sua vez depende diretamente de p – isto é, não de r.

Finalmente, também vale a pena notar que este resultado é consistente com o trabalho de Biau [2012], que provou que a taxa de convergência de florestas de árvores aleatórias também depende apenas das variáveis relevantes.

6.4 importâncias variáveis em variantes florestais aleatórias

Nesta seção, consideramos e discutimos importâncias variáveis calculadas com outros tipos de conjuntos de árvores aleatórias. Primeiro mostramos como nossos resultados se estendem a qualquer outra medida de i mpureza e, em seguida, analisamos as importâncias calculadas por conjuntos de árvores aleatórias podadas em profundidade e aquelas calculadas por árvores aleatórias construídas em subespaços aleatórios de tamanho fixo. Finalmente, discutimos o caso de árvores não totalmente randomizadas.

6.4.1 Generalização para outras medidas de impureza

Embora nossa caracterização nas seções 6.1 e 6.3 use entropias de Shannon como medida de impureza, os teoremas 6.1, 6.3 e 6.5 são válidos para

other impurity measures, simply substituting conditional mutual information for conditional impurity reduction in the different formulas and in the definition of irrelevant variables. In particular, our results thus hold for the Gini index in classification and can be extended to regression problems using variance as the impurity measure.

Let us consider a generic impurity measure $i(Y|t)$ and, by mimicking the notation used for conditional mutual information, let us denote by $G(Y;X_j|t)$ the impurity decrease for a split on X_j at node t:

$$G(Y;X_j|t) = i(Y|t) - \sum_{x\in\mathcal{X}_j} p(t_x)i(Y|t_x), \tag{6.26}$$

where t_x denotes the successor node of t corresponding to value x of X_j. The importance score associated to a variable X_j (see Equation 6.2) is then rewritten:

$$\text{Imp}(X_j) = \frac{1}{M}\sum_{m=1}^{M}\sum_{t\in\varphi_m} 1(j_t = j)p(t)G(Y;X_j|t). \tag{6.27}$$

As explained in the proof of Theorem 6.1, conditioning over a node t is equivalent to conditioning over an event of the form $B(t) = b(t)$, where $B(t)$ and $b(t)$ denote respectively the set of variables tested in the branch from the root to t and their values in this branch. When the learning sample size N grows to infinity, this yields the following population based impurity decrease at node t:

$$\begin{aligned} G(Y;X_j|B(t) = b(t)) &\qquad (6.28)\\ = i(Y|B(t) = b(t)) - \sum_{x\in\mathcal{X}_j} P(X_j = x|B(t) = b(t))i(Y|B(t) = b(t), X_j = x)& \end{aligned}$$

Again by analogy with conditional entropy and mutual information[2], let us define $i(Y|B)$ and $G(Y;X_j|B)$ for some subset of variables $B \subseteq V$ as follows:

$$i(Y|B) = \sum_b P(B = b)i(Y|B = b) \tag{6.29}$$

$$\begin{aligned} G(Y;X_j|B) &= \sum_b P(B = b)G(Y;X_j|B = b) \qquad (6.30)\\ &= i(Y|B) - i(Y|B,X_j)& \end{aligned}$$

where the sums run over all possible combinations b of values for the variables in B.

With these notations, the proof of Theorem 6.1 can be easily adapted to lead to the following generalization of Equation 6.10:

$$\text{Imp}(X_j) = \sum_{k=0}^{p-1}\frac{1}{C_p^k}\frac{1}{p-k}\sum_{B\in\mathcal{P}_k(V^{-j})} G(Y;X_j|B). \tag{6.31}$$

2 Note however that $G(Y;X_j|B)$ does not share all properties of conditional mutual information as for example $G(X_j;Y|B)$ might not be equal to $G(Y;X_j|B)$ or even be defined, depending on the impurity measure and the nature of the output Y.

outras medidas de impureza, simplesmente substituindo a informação mútua condicional pela redução condicional de impureza nas diferentes fórmulas e na definição de variáveis irrelevantes. Em particular, os nossos resultados são válidos para o índice de Gini na classificação e podem ser estendidos a problemas de regressão utilizando a variância como medida de impureza.

Vamos considerar uma medida genérica de impureza $i(Y|t)$ e, imitando a notação usada para informações mútuas condicionais, denotaremos por $G(Y;X_j|t)$ a diminuição de impureza para uma divisão em X_j no nó t:

$$G(Y;X_j|t) = i(Y|t) - \sum_{x\in\mathcal{X}_j} p(t_x)i(Y|t_x), \tag{6.26}$$

onde t_x denota o nó sucessor de t correspondente ao valor $xdeX_j$. A pontuação de importância associada a uma variável X_j (ver Equação 6.2) é então reescrita:

$$\text{Imp}(X_j) = \frac{1}{M}\sum_{m=1}^{M}\sum_{t\in\varphi_m} 1(j_t = j)p(t)G(Y;X_j|t). \tag{6.27}$$

Conforme explicado na prova do Teorema 6.1, o condicionamento sobre o ânodo t é equivalente ao condicionamento sobre um evento da forma $B(t) = b(t)$, onde $B(t)$ e $b(t)$ denotam respectivamente o conjunto de variáveis testadas no ramo da raiz até t e seus valores neste ramo. Quando o tamanho da amostra de aprendizagem N cresce até o infinito, isso produz a seguinte diminuição de impureza baseada na população no nó t:

$$\begin{aligned} G(Y;X_j|B(t) = b(t)) &\qquad (6.28)\\ = i(Y|B(t) = b(t)) - \sum_{x\in\mathcal{X}_j} P(X_j = x|B(t) = b(t))i(Y|B(t) = b(t), X_j = x)& \end{aligned}$$

Novamente, por analogia com entropia condicional e informação mútua[2], vamos definir $i(Y|B)$ e $G(Y;X_j|B)$ para algum subconjunto de variáveis $B \subseteq V$como segue:

$$i(Y|B) = \sum_b P(B = b)i(Y|B = b) \tag{6.29}$$

$$\begin{aligned} G(Y;X_j|B) &= \sum_b P(B = b)G(Y;X_j|B = b) \qquad (6.30)\\ &= i(Y|B) - i(Y|B,X_j)& \end{aligned}$$

onde as somas abrangem todas as combinações possíveis b de valores para as variáveis em B.

Com estas notações, a prova do Teorema 6.1 pode ser facilmente adaptada para levar à seguinte generalização da Equação 6.10:

$$\text{Imp}(X_j) = \sum_{k=0}^{p-1}\frac{1}{C_p^k}\frac{1}{p-k}\sum_{B\in\mathcal{P}_k(V^{-j})} G(Y;X_j|B). \tag{6.31}$$

2 Observe, entretanto, que $G(Y;X_j|B)$ não compartilha todas as propriedades de i nformações mútuas condicionais como, por exemplo, $G(X_j;Y|B)$ pode não ser igual a $G(Y;X_j|B)$ ou mesmo ser definido, dependendo da medida de impureza e da natureza da saída Y.

Note that this generalization is valid without any further specific constraints on the impurity measure $i(Y|t)$.

Let us now define as *irrelevant to* Y *with respect to* V a variable X_i for which, for all $B \subseteq V$, $G(Y; X_i|B) = 0$ (i.e. a variable that neither affects impurity whatever the conditioning). From this definition, one can deduce the following property of an irrelevant variable X_i (for all $B \subseteq V$ and $X_j \in V$):

$$G(Y; X_j|B \cup \{X_i\}) = G(Y; X_j|B).$$

Indeed, by a simple application of previous definitions, we have:

$$\begin{aligned}
&G(Y; X_j|B) - G(Y; X_j|B \cup \{X_i\}) \\
&= i(Y|B) - i(Y|B \cup \{X_j\}) - i(Y|B \cup \{X_i\}) + i(Y|B \cup \{X_i, X_j\}) \\
&= i(Y|B) - i(Y|B \cup \{X_i\}) - i(Y|B \cup \{X_j\}) + i(Y|B \cup \{X_i, X_j\}) \\
&= G(Y; X_i|B) - G(Y; X_i|B \cup \{X_j\}) \\
&= 0, \qquad (6.32)
\end{aligned}$$

where the last step is a consequence of the irrelevance of X_i.

Using this property, the proofs of Lemma 6.4 and Theorem 6.5 can be straightforwardly adapted, showing that, in the general case also, the MDI importance of a variable is invariant with respect to the removal or the addition of irrelevant variables.

Given the general definition of irrelevance, all irrelevant variables also get zero MDI importance but, without further constraints on the impurity measure i, there is no guarantee that all relevant variables (defined as all variables that are not irrelevant) will get a non zero importance. This property, and in consequence theorem 6.3, will be however satisfied as soon as the impurity measure is such that $G(Y; X_j|B) \geqslant 0$ for all $X_j \in V$ and for all $B \subseteq V$.

Previous developments show that all results presented in this chapter remain valid for any impurity measure leading to non negative impurity decreases, provided that the definition of variable irrelevance is adapted to this impurity measure. The choice of a specific impurity measure should thus be guided by the meaning one wants to associate to irrelevance.

Measuring impurity with Shannon entropy, i.e., taking $i(Y|t) = H(Y|t)$ and $i(Y|B = b) = H(Y|B = b)$, one gets back all previous results. Given the properties of conditional mutual information, irrelevance for this impurity measure strictly coincides with conditional independence: a variable X_i is irrelevant to Y with respect to V if and only if $X_i \perp Y|B$ for all $B \subseteq V$.

A common alternative to Shannon entropy for growing classification trees is Gini index, which, in the finite and infinite sample cases, is written:

$$i(Y|t) = -\sum_j p(j|t)(1 - p(j|t)) \qquad (6.33)$$

Note que esta generalização é válida sem quaisquer restrições específicas adicionais na medida de impureza $i(Y|t)$.

Vamos agora definir como irrelevante para Y em relação a V uma variável $X_i$para a qual, para todos $B \subseteq V$, $G(Y; X_i|B) = 0$ (ou seja, uma variável que não afeta a impureza seja qual for o condicionamento). A partir desta definição, pode-se deduzir a seguinte propriedade de uma variável irrelevante X_i (para todos$B \subseteq V$ e $X_j \in V$):

$$G(Y; X_j|B \cup \{X_i\}) = G(Y; X_j|B).$$

Na verdade, por uma simples aplicação das definições anteriores, temos:

$$\begin{aligned}
&G(Y; X_j|B) - G(Y; X_j|B \cup \{X_i\}) \\
&= i(Y|B) - i(Y|B \cup \{X_j\}) - i(Y|B \cup \{X_i\}) + i(Y|B \cup \{X_i, X_j\}) \\
&= i(Y|B) - i(Y|B \cup \{X_i\}) - i(Y|B \cup \{X_j\}) + i(Y|B \cup \{X_i, X_j\}) \\
&= G(Y; X_i|B) - G(Y; X_i|B \cup \{X_j\}) \\
&= 0, \qquad (6.32)
\end{aligned}$$

onde a última etapa é consequência da irrelevância de X_i.

Usando esta propriedade, as provas do Lema 6.4 e do Teorema 6.5 podem ser facilmente adaptadas, mostrando que, também no caso geral, a importância MDI de uma variável é invariante em relação à remoção ou adição de variáveis irrelevantes.

Dada a definição geral de irrelevância, todas as variáveis irrelevantes também têm importância zero no MDI mas, sem restrições adicionais à medida de impureza i, não há garantia de que todas as variáveis relevantes (definidas como todas as variáveis que não são irrelevantes) terão uma importância diferente de zero. Esta propriedade, e consequentemente o teorema 6.3, será contudo satisfeita assim que a medida de impureza for tal que$G(Y; X_j|B) \geqslant 0$ para todo $X_j \in V$ e para todo $B \subseteq V$.

Desenvolvimentos anteriores mostram que todos os resultados apresentados neste capítulo permanecem válidos para qualquer medida de impureza que conduza a diminuições de impureza não negativas, desde que a definição de irrelevância variável seja adaptada a esta medida de impureza. A escolha de uma medida específica de impureza deve, portanto, ser orientada pelo significado que se deseja associar à irrelevância.

Medindo a impureza com a entropia de Shannon, ou seja, tomando $i(Y|t) =H(Y|t)$ e $i(Y|B = b) = H(Y|B = b)$, obtém-se todos os resultados anteriores. Dadas as propriedades da informação mútua condicional, a irrelevância para esta medida de impureza coincide estritamente com a independência condicional: uma variável X_i é irrelevante para Y com relação a V se e somente se $X_i \perp Y|B$ para todos $B \subseteq V$.

Uma alternativa comum à entropia de Shannon para o crescimento de árvores de classificação é o índice de Gini, que, nos casos de amostras finitas e infinitas, é escrito:

$$i(Y|t) = -\sum_j p(j|t)(1 - p(j|t)) \qquad (6.33)$$

$$i(Y|B=b) = -\sum_j P(Y=j|B=b)(1-P(Y=j|B=b)). \quad (6.34)$$

Like the Shannon entropy, this measure leads to non negative impurity decreases and the corresponding notion of irrelevance is also directly related to conditional independence.

The most common impurity measure for regression is variance, which, in the finite and infinite sample cases, is written:

$$i(Y|t) = \frac{1}{N_t}\sum_{i\in t}(y_i - \frac{1}{N_t}\sum_{i\in t} y_i)^2 \quad (6.35)$$

$$i(Y|B=b) = E_{Y|B=b}\{(Y - E_{Y|B=b}\{Y\})^2\}. \quad (6.36)$$

Variance can only decrease as a consequence of a split and therefore, Theorem 6.3 is also valid for this impurity measure, meaning that only irrelevant variables will get a zero variance reduction. Note however that with this impurity measure, irrelevance is not directly related to conditional independence, as some variable X_i can be irrelevant in the sense of our definition and still affects the distribution of output values.

6.4.2 *Pruning and random subspaces*

In sections 6.1 and 6.3, we considered totally randomized trees that were fully developed, i.e. until all p variables were used within each branch. When totally randomized trees are developed only up to some smaller depth $q \leqslant p$, we show in Proposition 6.6 that the variable importances as computed by these trees is limited to the q first terms of Equation 6.10. We then show in Proposition 6.7 that these latter importances are actually the same as when each tree of the ensemble is fully developed over a random subspace [Ho, 1998] of q variables drawn prior to its construction.

Proposition 6.6. *The importance of $X_j \in V$ for Y as computed with an infinite ensemble of pruned totally randomized trees built up to depth $q \leqslant p$ and an infinitely large training sample is:*

$$Imp(X_j) = \sum_{k=0}^{q-1} \frac{1}{C_p^k}\frac{1}{p-k}\sum_{B\in\mathcal{P}_k(V^{-j})} I(X_j; Y|B) \quad (6.37)$$

Proof. The proof of Theorem 6.1 can be directly adapted to prove Proposition 6.6. If the recursive procedure is stopped at depth q, then it means that $B(t)$ may include up to $q-1$ variables, which is strictly equivalent to summing from $k=0$ to $q-1$ in the outer sum of Equation 6.10. □

Proposition 6.7. *The importance of $X_j \in V$ for Y as computed with an infinite ensemble of pruned totally randomized trees built up to depth $q \leqslant p$*

$$i(Y|B=b) = -\sum_j P(Y=j|B=b)(1-P(Y=j|B=b)). \quad (6.34)$$

Tal como a entropia de Shannon, esta medida conduz a diminuições de impureza não negativas e a noção correspondente de irrelevância ta mbém está diretamente relacionada com a independência condicional.

A medida de impureza mais comum para regressão é a variância, que, nos casos de amostras finitas e infinitas, é escrita:

$$i(Y|t) = \frac{1}{N_t}\sum_{i\in t}(y_i - \frac{1}{N_t}\sum_{i\in t} y_i)^2 \quad (6.35)$$

$$i(Y|B=b) = E_{Y|B=b}\{(Y - E_{Y|B=b}\{Y\})^2\}. \quad (6.36)$$

A variância só pode diminuir como consequência de uma divisão e, portanto, o Teorema 6.3 também é válido para esta medida de impureza, o que significa que apenas variáveis irrelevantes obterão uma redução de variância zero. Note-se, contudo, que com esta medida de impureza, a irrelevância não está directamente relacionada com a independência condicional, uma vez que alguma variável X_i pode ser irrelevante no sentido da nossa definição e ainda assim afectar a distribuição dos valores de produção.

6.4.2 Poda e subespaços aleatórios

Nas seções 6.1 e 6.3, consideramos árvores totalmente aleatórias que estavam totalmente desenvolvidas, ou seja, até que todas p as variáveis fossem utilizadas dentro de cada ramo. Quando árvores totalmente aleatórias são desenvolvidas apenas até uma profundidade menor $q \leqslant p$, mostramos na Proposição 6.6 que as importâncias das variáveis calculadas por essas árvores são limitadas aos q primeiros termos da Equação 6.10. Mostramos então na Proposição 6.7 que estas últimas importâncias são na verdade as mesmas que quando cada árvore do conjunto é totalmente desenvolvida sobre um subespaço aleatório [Ho, 1998] de $q$variáveis extraídas antes de sua construção.

Proposição 6.6. A importância de $X_j \in V$ for Y computado com um conjunto infinito de árvores podadas totalmente aleatórias construídas até a profundidade $q \leqslant p$e uma amostra de treinamento infinitamente grande é:

$$Imp(X_j) = \sum_{k=0}^{q-1} \frac{1}{C_p^k}\frac{1}{p-k}\sum_{B\in\mathcal{P}_k(V^{-j})} I(X_j; Y|B) \quad (6.37)$$

Prova. A prova do Teorema 6.1 pode ser adaptada diretamente para provar a Proposição 6.6. Se o procedimento recursivo for interrompido na profundidade q, então significa que $B(t)$ pode incluir até $q-1$ variáveis, o que é estritamente equivalente à soma de $k=0$ a $q-1$ na soma externa da Equação 6.10. □

Proposição 6.7. A importância de $X_j \in V$ for Y calculado com um conjunto infinito de árvores podadas totalmente aleatórias construídas até a profundidade $q \leqslant p$

and an infinitely large training sample is identical to the importance as computed for Y with an infinite ensemble of fully developed totally randomized trees built on random subspaces of q variables drawn from V.

Proof. Let us define a random subspace of size q as a random subset $V_S \subseteq V$ such that $|V_S| = q$. By replacing p with q in Equation 6.10 (since each tree is built on q variables) and adjusting by the probability

$$\frac{C_{p-k-1}^{q-k-1}}{C_p^q}$$

of having selected X_j and the k variables in the branch when drawing V_S prior to the construction of the tree, it comes:

$$\text{Imp}(X_j) = \sum_{k=0}^{q-1} \frac{C_{p-k-1}^{q-k-1}}{C_p^q} \frac{1}{C_q^k} \frac{1}{q-k} \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B) \tag{6.38}$$

The multiplicative factor in the outer sum can then be simplified as follows:

$$\begin{aligned}
\frac{C_{p-k-1}^{q-k-1}}{C_p^q} \frac{1}{C_q^k} \frac{1}{q-k} &= \frac{\frac{(p-k-1)!}{(p-k)!(q-k-1)!}}{\frac{p!}{(p-q)!q!}} \frac{1}{C_q^k} \frac{1}{q-k} \\
&= \frac{(p-k-1)!q!}{(q-k-1)!p!} \frac{1}{C_q^k} \frac{1}{q-k} \\
&= \frac{q(q-1)...(q-k)}{p(p-1)...(p-k)} \frac{1}{C_q^k} \frac{1}{q-k} \\
&= \frac{q(q-1)...(q-k)}{p(p-1)...(p-k)} \frac{k!(q-k)!}{q!} \frac{1}{q-k} \\
&= \frac{1}{p(p-1)...(p-k)} \frac{k!(q-k)!}{(q-k-1)!} \frac{1}{q-k} \\
&= \frac{k!}{p(p-1)...(p-k)} \\
&= \frac{k!(p-k)!}{p!} \frac{1}{p-k} \\
&= \frac{1}{C_p^k} \frac{1}{p-k}
\end{aligned} \tag{6.39}$$

which yields the same importance as in Proposition 6.6 and proves the proposition. □

As long as $q \geqslant r$ (where r denotes the number of relevant variables in V), it can easily be shown that all relevant variables will still obtain a strictly positive importance, which will however differ in general from the importances computed by fully grown totally randomized trees built over all variables. Also, each irrelevant variable of course

e uma amostra de treinamento infinitamente grande é idêntica à importância calculada para Y com um conjunto infinito de árvores totalmente aleatórias totalmente desenvolvidas construídas em subespaços aleatórios de q variáveis extraídas de V.

Prova. Vamos definir um subespaço aleatório de tamanho q como um subconjunto aleatório$V_S \subseteq V$ tal que $|V_S| = q$. Substituindo p por q na Equação 6.10 (uma vez que cada árvore é construída sobre q variáveis) e ajustando pela probabilidade

$$\frac{C_{p-k-1}^{q-k-1}}{C_p^q}$$

de ter selecionado X_j e as k variáveis no ramo ao desenharV_S antes da construção da árvore, vem:

$$\text{Imp}(X_j) = \sum_{k=0}^{q-1} \frac{C_{p-k-1}^{q-k-1}}{C_p^q} \frac{1}{C_q^k} \frac{1}{q-k} \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B) \tag{6.38}$$

O fator multiplicativo na soma externa pode então ser simplificado da seguinte forma:

$$\begin{aligned}
\frac{C_{p-k-1}^{q-k-1}}{C_p^q} \frac{1}{C_q^k} \frac{1}{q-k} &= \frac{\frac{(p-k-1)!}{(p-k)!(q-k-1)!}}{\frac{p!}{(p-q)!q!}} \frac{1}{C_q^k} \frac{1}{q-k} \\
&= \frac{(p-k-1)!q!}{(q-k-1)!p!} \frac{1}{C_q^k} \frac{1}{q-k} \\
&= \frac{q(q-1)...(q-k)}{p(p-1)...(p-k)} \frac{1}{C_q^k} \frac{1}{q-k} \\
&= \frac{q(q-1)...(q-k)}{p(p-1)...(p-k)} \frac{k!(q-k)!}{q!} \frac{1}{q-k} \\
&= \frac{1}{p(p-1)...(p-k)} \frac{k!(q-k)!}{(q-k-1)!} \frac{1}{q-k} \\
&= \frac{k!}{p(p-1)...(p-k)} \\
&= \frac{k!(p-k)!}{p!} \frac{1}{p-k} \\
&= \frac{1}{C_p^k} \frac{1}{p-k}
\end{aligned} \tag{6.39}$$

o que produz a mesma importância que na Proposição 6.6 e prova a proposição. □

Contanto que $q \geqslant r$ (onde r denote o número de variáveis relevantes em V), pode ser facilmente demonstrado que todas as variáveis relevantes ainda obterão uma importância estritamente positiva, que, no entanto, diferirá em geral das importâncias calculadas por árvores totalmente aleatórias construídas sobre todas as variáveis. Além disso, cada variável irrelevante, é claro

keeps an importance equal to zero, which means that, in asymptotic conditions, these pruning and random subspace methods would still allow us identify the relevant variables, as long as we have a good upper bound q on r.

6.4.3 *Non-totally randomized trees*

In our analysis in the previous sections, trees are built totally at random and hence do not directly relate to those built in Random Forest [Breiman, 2001] or in Extremely Randomized Trees [Geurts et al., 2006a]. To better understand the importances as computed by those algorithms, let us consider a close variant of totally randomized trees: at each node t, let us instead draw uniformly at random $1 \leqslant K \leqslant p$ variables and let us choose the one that maximizes $\Delta i(t)$. As previously, t is split into as many subtrees as the cardinality of the chosen variable. Asymptotically, for binary variables, this variant exactly matches Random Forests and Extremely Randomized Trees. For variables with a larger cardinality, the correspondence no longer exactly holds but the trees still closely relate. Notice that, for $K = 1$, this procedure amounts to building ensembles of totally randomized trees as defined before, while, for $K = p$, it amounts to building classical single trees in a deterministic way.

First, the importance of $X_j \in V$ as computed with an infinite ensemble of such randomized trees is not the same as Equation 6.10. For $K > 1$, masking effects indeed appear: at t, some variables are never selected because there always is some other variable for which $\Delta i(t)$ is larger. Such effects tend to pull the best variables at the top of the trees and to push the others at the leaves. As a result, the importance of a variable no longer decomposes into a sum including all $I(X_j; Y|B)$ terms. The importance of the best variables decomposes into a sum of their mutual information alone or conditioned only with the best others – but not conditioned with all variables since they no longer ever appear at the bottom of trees. By contrast, the importance of the least promising variables now decomposes into a sum of their mutual information conditioned only with all variables – but not alone or conditioned with a couple of others since they no longer ever appear at the top of trees. In other words, because of the guided structure of the trees, the importance of X_j now takes into account only some of the conditioning sets B, which may over- or underestimate its overall relevance.

To make things clearer, let us consider a simple example. Let X_1 perfectly explains Y and let X_2 be a slightly noisy copy of X_1 (i.e., $I(X_1; Y) \approx I(X_2; Y)$, $I(X_1; Y|X_2) = \epsilon$ and $I(X_2; Y|X_1) = 0$). Using totally

mantém uma importância igual a zero, o que significa que, em condições assintóticas, esses métodos de poda e subespaço aleatório ainda nos permitiriam identificar as variáveis relevantes, desde que tenhamos um bom limite superior q em r.

6.4.3 Árvores não totalmente randomizadas

Em nossa análise nas seções anteriores, as árvores são construídas totalmente aleatoriamente e, portanto, não se relacionam diretamente c om aquelas construídas em Random Forest [Breiman, 2001] ou em Extremely Randomized Trees [Geurts et al., 2006a]. Para entender melhor as importâncias calculadas por esses algoritmos, vamos considerar uma variante aproximada de árvores totalmente aleatórias: em cada nó t, vamos desenhar uniformemente variáveis $1 \leqslant K \leqslant p$ aleatórias e escolher aquela que maximiza $\Delta i(t)$. Como anteriormente, t é dividido em tantas subárvores quanto a cardinalidade da variável escolhida. Assintoticamente, para variáveis binárias, esta variante corresponde exatamente a Florestas Aleatórias e Árvores Extremamente Randomizadas. Para variáveis com uma cardinalidade maior, a correspondência não é mais válida, mas as árvores ainda se relacionam estreitamente. Observe que, para $K = 1$, este procedimento equivale à construção de conjuntos de árvores totalmente aleatórias conforme definido anteriormente, enquanto, para $K = p$, equivale à construção de árvores únicas clássicas de forma determinística.

Primeiro, a importância de $X_j \in V$ as calculada com um conjunto infinito de tais árvores aleatórias não é a mesma da Equação 6.10. Para$K > 1$, de fato aparecem efeitos de mascaramento: em t, algumas variáveis nunca são selecionadas porque sempre há alguma outra variável para a qual $\Delta i(t)$ é maior. Tais efeitos tendem a puxar as melhores variáveis para o topo das árvores e empurrar as demais para as folhas. Como resultado, a importância de uma variável não se decompõe mais em uma soma incluindo todos os termos $I(X_j; Y|B)$. A importância das melhores variáveis decompõe-se numa soma da sua informação mútua isoladamente ou condicionada apenas às melhores outras – mas não condicionada a todas as variáveis, uma vez que já não aparecem na base das árvores. Em contraste, a importância das variáveis menos promissoras decompõe-se agora numa soma da sua informação mútua condicionada apenas com todas as variáveis – mas não sozinha ou condicionada com algumas o utras, uma vez que já não aparecem no topo das árvores. Em outras palavras, devido à estrutura guiada das árvores, a importância de X_j agora leva em conta apenas alguns dos conjuntos condicionantes B, o que pode superestimar ou subestimar sua relevância geral.

Para tornar as coisas mais claras, consideremos um exemplo simples. Seja X_1 explica perfeitamente Y e seja X_2 uma cópia ligeiramente barulhenta de X_1 (isto é,$I(X_1; Y) \approx I(X_2; Y)$, $I(X_1; Y|X_2) = \epsilon$ e $I(X_2; Y|X_1) = 0$). Usando totalmente

randomized trees, the importances of X_1 and X_2 are nearly equal – the importance of X_1 being slightly higher than the importance of X_2:

$$\text{Imp}(X_1) = \frac{1}{2}I(X_1;Y) + \frac{1}{2}I(X_1;Y|X_2) = \frac{1}{2}I(X_1;Y) + \frac{\epsilon}{2} \tag{6.40}$$

$$\text{Imp}(X_2) = \frac{1}{2}I(X_2;Y) + \frac{1}{2}I(X_2;Y|X_1) = \frac{1}{2}I(X_2;Y) + 0 \tag{6.41}$$

In non-totally randomized trees, for $K = 2$, X_1 is always selected at the root node and X_2 is always used in its children. Also, since X_1 perfectly explains Y, all its children are pure and the reduction of entropy when splitting on X_2 is null. As a result, $\text{Imp}_{K=2}(X_1) = I(X_1;Y)$ and $\text{Imp}_{K=2}(X_2) = I(X_2;Y|X_1) = 0$. Masking effects are here clearly visible: the true importance of X_2 is masked by X_1 as if X_2 were irrelevant, while it is only a bit less informative than X_1.

As a direct consequence of the example above, for $K > 1$, it is no longer true that a variable is irrelevant if and only if its importance is zero. In the same way, it can also be shown that the importances become dependent on the number of irrelevant variables. Let us indeed consider the following counter-example: let us add in the previous example an irrelevant variable X_i with respect to $\{X_1, X_2\}$ and let us keep $K = 2$. The probability of selecting X_2 at the root node now becomes positive, which means that $\text{Imp}_{K=2}(X_2)$ now includes $I(X_2;Y) > 0$ and is therefore strictly larger than the importance computed before. For K fixed, adding irrelevant variables dampens masking effects, which thereby makes importances indirectly dependent on the number of irrelevant variables.

In conclusion, the importances as computed with totally randomized trees exhibit properties that do not possess, by extension, neither Random Forests nor Extremely Randomized Trees. With totally randomized trees, the importance of X_j only depends on the relevant variables and is 0 if and only if X_j is irrelevant. As we have shown, it may no longer be the case for $K > 1$. Asymptotically, the use of totally randomized trees for assessing the importance of a variable may therefore be more appropriate. In a finite setting (i.e., a limited number of samples and a limited number of trees), guiding the choice of the splitting variables remains however a sound strategy. In such a case, $I(X_j;Y|B)$ cannot be measured neither for all X_j nor for all B. It is therefore pragmatic to promote those that look the most promising, even if the resulting importances may be biased.

6.5 ILLUSTRATION

In this section, we consider the digit recognition problem of [Breiman et al., 1984] for illustrating variable importances as computed with totally randomized trees. We verify that they match with our theoretical developments and that they decompose as foretold. We also

árvores aleatórias, as importâncias de X_1 e X_2 são quase iguais – a importância de X_1 é ligeiramente maior que a importância de X_2:

$$\text{Imp}(X_1) = \frac{1}{2}I(X_1;Y) + \frac{1}{2}I(X_1;Y|X_2) = \frac{1}{2}I(X_1;Y) + \frac{\epsilon}{2} \tag{6.40}$$

$$\text{Imp}(X_2) = \frac{1}{2}I(X_2;Y) + \frac{1}{2}I(X_2;Y|X_1) = \frac{1}{2}I(X_2;Y) + 0 \tag{6.41}$$

Em árvores não totalmente aleatórias, para $K = 2$, X_1 é sempre selecionado no nó raiz e X_2 é sempre usado em seus filhos. Além disso, comoX_1 explica perfeitamente Y,todos os seus filhos são puros e a redução da entropia ao dividir em X_2 é nula. Como resultado, Imp $_{K=2}(X_1) =$I$(X_1;Y)$ e $\text{Imp}_{K=2}(X_2) = I(X_2;Y|X_1) = 0$. Os efeitos de mascaramento são claramente visíveis aqui: a verdadeira importância de X_2 é mascarada por X_1 como se X_2fosse irrelevante, embora seja apenas um pouco menos informativo que X_1.

Como consequência direta do exemplo acima, para $K > 1$, não é mais verdade que uma variável é irrelevante se e somente se a sua importância for zero. Da mesma forma, também pode ser demonstrado que as importâncias passam a depender do número de variáveis irrelevantes. Consideremos de facto o seguinte contra-exemplo: adicionemos no exemplo anterior uma variável irrelevante X_i em relação a $\{X_1, X_2\}$ e mantenhamos $K = 2$. A probabilidade de selecionar X_2 no nó raiz agora torna-se positiva, o que significa que $\text{Imp}_{K=2}(X_2)$ agora incluiI$(X_2;Y) > 0$ e é, portanto, estritamente maior que a importância calculada anteriormente. Para K fixo, adicionar variáveis irrelevantes amortece os efeitos de mascaramento, o que torna as importâncias indiretamente dependentes do número de variáveis irrelevantes.

Concluindo, as importâncias calculadas com árvores totalmente aleatórias exibem propriedades que não possuem, por extensão, nem Florestas Aleatórias nem Árvores Extremamente Randomizadas. Com árvores totalmente aleatórias, a importância de X_j depende apenas das variáveis relevantes e é 0 se e somente se X_j é irrelevante. Como mostramos, pode não ser mais o caso de $K > 1$. Assintoticamente, o uso de árvores totalmente aleatórias para avaliar a importância de uma variável pode, portanto, ser mais apropriado. Num cenário finito (ou seja, um número limitado de amostras e um número limitado de árvores), orientar a escolha das variáveis de divisão continua, no entanto, a ser uma estratégia sólida. Nesse caso, $I(X_j;Y|B)$ não pode ser medido nem para todos X_j nem para todos B. É, portanto, pragmático promover aqueles que parecem mais promissores, mesmo que as importâncias resultantes possam ser tendenciosas.

6.5 ilustração

Nesta seção, consideramos o problema de reconhecimento de dígitos de [Breiman et al., 1984] para ilustrar importâncias de variáveis calculadas com árvores totalmente aleatórias. Verificamos que coincidem com os nossos desenvolvimentos teóricos e que se decompõem conforme previsto. Nós também

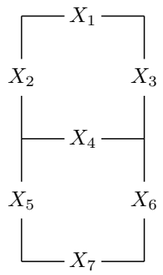


Figure 6.1: 7-segment display

compare these importances with those computed by an ensemble of non-totally randomized trees, as discussed in section 6.4.3.

Let us consider a seven-segment indicator displaying numerals using horizontal and vertical lights in on-off combinations, as illustrated in Figure 6.1. Let Y be a random variable taking its value in $\{0, 1, ..., 9\}$ with equal probability and let $X_1, ..., X_7$ be binary variables whose values are each determined univocally given the corresponding value of Y in Table 6.1.

Since Table 6.1 perfectly defines the data distribution, and given the small dimensionality of the problem, it is practicable to directly apply Equation 6.10 to compute variable importances. To verify our theoretical developments, we then compare in Table 6.2 variable importances as computed by Equation 6.10 and those yielded by an ensemble of 10000 totally randomized trees ($K = 1$). Note that given the known structure of the problem, building trees on a sample of finite size that perfectly follows the data distribution amounts to building them on a sample of infinite size. At best, trees can thus be built on a 10-sample dataset, containing exactly one sample for each of the equiprobable outcomes of Y. As the table illustrates, the importances yielded by totally randomized trees match those computed by Equation 6.10, which confirms Theorem 6.1. Small differences stem from the fact that a finite number of trees were built in our simulations, but those discrepancies should disappear as the size of the ensemble grows towards infinity. It also shows that importances indeed add up to $I(X_1, ...X_7; Y)$, which confirms Theorem 6.2. Regarding the actual importances, they indicate that X_5 is stronger than all others, followed – in that order – by X_2, X_4 and X_3 which also show large importances. X_1, X_7 and X_6 appear to be the less informative. The table also reports importances for increasing values of K. As discussed before, we see that a large value of K yields importances that can be either overestimated (e.g., at $K = 7$, the importances of X_2 and X_5 are larger than at $K = 1$) or underestimated due to masking effects (e.g., at $K = 7$, the importances of X_1, X_3, X_4 and X_6 are smaller than at $K = 1$, as if they were less important). As Figure 6.2 illustrates, it can also be observed that masking effects may even induce changes in the

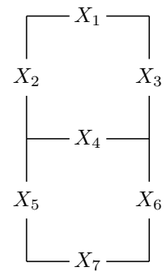


Figura 6.1: Display de 7 segmentos

compare essas importâncias com aquelas calculadas por um conjunto de árvores não totalmente aleatórias, conforme discutido na seção 6.4.3.

Consideremos um indicador de sete segmentos exibindo numerais usando luzes horizontais e verticais em combinações liga-desliga, conforme ilustrado na Figura 6.1. Seja Y uma variável aleatória que assume seu valor em $\{0, 1, ..., 9\}$ com igual probabilidade e sejam $X_1, ..., X_7$ variáveis binárias cujos valores são determinados univocamente, dado o valor correspondente de Y na Tabela 6.1.

Como a Tabela 6.1 define perfeitamente a distribuição dos dados, e dada a pequena dimensionalidade do problema, é praticável aplicar diretamente a Equação 6.10 para calcular as importâncias das variáveis. Para verificar nossos desenvolvimentos teóricos, comparamos então na Tabela 6.2 as importâncias das variáveis calculadas pela Equação 6.10 e aquelas produzidas por um conjunto de 10.000 árvores totalmente aleatórias ($K = 1$). Observe que, dada a estrutura conhecida do problema, construir árvores em uma amostra de tamanho finito que segue perfeitamente a distribuição dos dados equivale a construí-las em uma amostra de tamanho infinito. Na melhor das hipóteses, as árvores podem ser construídas sobre um conjunto de dados de 10 amostras, contendo exatamente uma amostra para cada um dos resultados equiprováveis de Y. Como ilustra a tabela, as importâncias obtidas por árvores totalmente aleatórias correspondem às calculadas pela Equação 6.10, o que confirma o Teorema 6.1. Pequenas diferenças decorrem do fato de que um número finito de árvores foi construído em nossas simulações, mas essas discrepâncias devem desaparecer à medida que o tamanho do conjunto cresce em direção ao infinito. Também mostra que as importâncias realmente somam $I(X_1, ...X_7; Y)$, o que confirma o Teorema 6.2. Quanto às importâncias reais, indicam que X_5 é mais forte que todos os outros, seguido – nesta ordem – por X_2, X_4 e X_3 que também apresentam grandes importâncias. X_1, X_7 e X_6 parecem ser menos informativos. A tabela também relata importâncias para valores crescentes de K. Conforme discutido anteriormente, vemos que um grande valor de K produz importâncias que podem ser superestimadas (por exemplo, em $K = 7$, as importâncias de X_2 e X_5 são maiores do que em $K = 1$) ou subestimadas devido a efeitos de mascaramento (por exemplo, em $K = 7$, as importâncias de X_1, X_3, X_4 e X_6 são menores que em $K = 1$, como se fossem menos importantes). Como ilustra a Figura 6.2, também pode ser observado que os efeitos de mascaramento podem até induzir alterações na

y	X_1	X_2	X_3	X_4	X_5	X_6	X_7
0	1	1	1	0	1	1	1
1	0	0	1	0	0	1	0
2	1	0	1	1	1	0	1
3	1	0	1	1	0	1	1
4	0	1	1	1	0	1	0
5	1	1	0	1	0	1	1
6	1	1	0	1	1	1	1
7	1	0	1	0	0	1	0
8	1	1	1	1	1	1	1
9	1	1	1	1	0	1	1

Table 6.1: Values of $Y, X_1, ..., X_7$

variable rankings (e.g., compare the rankings at $K = 1$ and at $K = 7$), which thus confirms that importances are differently affected.

To better understand why a variable is important, it is also insightful to look at its decomposition into its p sub-importances terms, as shown in Figure 6.3. Each row in the plots of the figure corresponds to one the p variables $X_1, \dots, X_7$ and each column to a size k of conditioning sets. As such, the value at row j and column k corresponds the importance of X_j when conditioned with k other variables (e.g., to the term $\frac{1}{C_p^k}\frac{1}{p-k}\sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B)$ in Equation 6.10 in the case of totally randomized trees). In the left plot, for $K = 1$, the figure first illustrates how importances yielded by totally randomized trees decomposes along the degrees k of interactions terms. We can observe that they each equally contribute (at most) the total importance of a variable. The plot also illustrates why X_5 is important: it is informative either alone or conditioned with any combination of the other variables (all of its terms are significantly larger than 0). By contrast, it also clearly shows why X_6 is not important: neither alone nor combined with others X_6 seems to be very informative (all of its terms are close to 0). More interestingly, this figure also highlights redundancies: X_7 is informative alone or conditioned with a couple of others (the first terms are significantly larger than 0), but becomes uninformative when conditioned with many others (the last terms are closer to 0). The right plot, for $K = 7$, illustrates the decomposition of importances when variables are chosen in a deterministic way. The first thing to notice is masking effects. Some of the $I(X_j; Y|B)$ terms are indeed clearly never encountered and their contribution is therefore reduced to 0 in the total importance. For instance, for $k = 0$, the subimportances of X_2 and X_5 are positive, while all others are null, which means that only those two variables are ever selected at the root node, hence masking the others. As a consequence, this also means that the

y	X_1	X_2	X_3	X_4	X_5	X_6	X_7
0	1	1	1	0	1	1	1
1	0	0	1	0	0	1	0
2	1	0	1	1	1	0	1
3	1	0	1	1	0	1	1
4	0	1	1	1	0	1	0
5	1	1	0	1	0	1	1
6	1	1	0	1	1	1	1
7	1	0	1	0	0	1	0
8	1	1	1	1	1	1	1
9	1	1	1	1	0	1	1

Tabela 6.1: Valores de $Y, X_1, ..., X_7$

classificações variáveis (por exemplo, compare as classificações em $K = 1$ e em $K = 7$), o que confirma que as importâncias são afetadas de forma diferente.

Para entender melhor por que uma variável é importante, também é interessante observar sua decomposição em termos de subimportâncias, conforme mostrado na Figura 6.3. Cada linha nos gráficos da figura corresponde a uma das p variáveis $X_1, \dots, X_7$ e cada coluna com um tamanho k de conjuntos de condicionamento. Assim, o valor na linha j e na coluna k corresponde à importância de X_j quando condicionado com k outras variáveis (por exemplo, ao termo $\frac{1}{C_p^k}\frac{1}{p-k}\sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B)$ na Equação 6.10 no caso de árvores totalmente aleatórias). No gráfico da esquerda, para $K = 1$, a figura primeiro ilustra como as importâncias produzidas por árvores totalmente aleatórias se decompõem ao longo dos graus k dos termos de interação. Podemos observar que cada um deles contribui igualmente (no máximo) para a importância total de uma variável. O gráfico também ilustra por que X_5 é importante: é informativo sozinho ou condicionado a qualquer combinação de outras variáveis (todos os seus termos são significativamente maiores que 0). Por outro lado, também mostra claramente porque X_6 não é importante: nem sozinho nem combinado com outros X_6 parece ser muito informativo (todos os seus termos estão próximos de 0). O mais interessante é que esta figura também destaca redundâncias: X_7 é informativo sozinho ou condicionado com alguns outros (os primeiros termos são significativamente maiores que 0), mas torna-se não informativo quando condicionado com muitos outros (os últimos termos estão mais próximos de 0). O gráfico da direita, para $K = 7$, ilustra a decomposição de importâncias quando as variáveis são escolhidas de forma determinística. A primeira coisa a notar são os efeitos de mascaramento. Alguns dos termos $I(X_j; Y|B)$ de fato nunca são encontrados e sua contribuição é, portanto, reduzida a 0 na importância total. Por exemplo, para $k = 0$, as subimportâncias de X_2 e X_5 são positivas, enquanto todas as outras são nulas, o que significa que apenas essas duas variáveis são selecionadas no nó raiz, mascarando assim as outras. Como consequência, isto também significa que o

	Eqn. 6.10	$K = 1$	$K = 2$	$K = 3$	$K = 4$	$K = 5$	$K = 6$	$K = 7$
X_1	0.412	0.414	0.362	0.327	0.309	0.304	0.305	0.306
X_2	0.581	0.583	0.663	0.715	0.757	0.787	0.801	0.799
X_3	0.531	0.532	0.512	0.496	0.489	0.483	0.475	0.475
X_4	0.542	0.543	0.525	0.484	0.445	0.414	0.409	0.412
X_5	0.656	0.658	0.731	0.778	0.810	0.827	0.831	0.835
X_6	0.225	0.221	0.140	0.126	0.122	0.122	0.121	0.120
X_7	0.372	0.368	0.385	0.392	0.387	0.382	0.375	0.372
$\sum$	3.321	3.321	3.321	3.321	3.321	3.321	3.321	3.321

Table 6.2: Variable importances as computed with an ensemble of randomized trees, for increasing values of K. Importances at $K = 1$ follow their theoretical values, as predicted by Equation 6.10 in Theorem 6.1. However, as K increases, importances diverge due to masking effects. In accordance with Theorem 6.2, their sum is also always equal to $I(X_1, \dots, X_7; Y) = H(Y) = \log_2(10) = 3.321$ since inputs allow to perfectly predict the output.

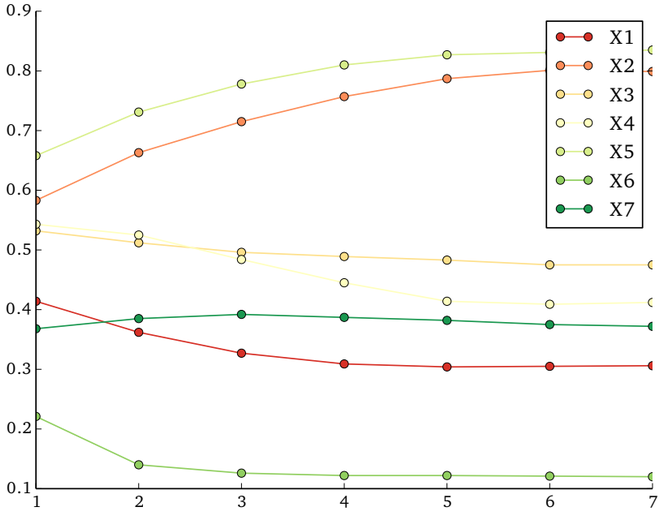


Figure 6.2: Variable importances as computed with an ensemble of randomized trees, for increasing values of K. Importances at $K = 1$ follow their theoretical values, as predicted by Equation 6.10 in Theorem 6.1. However, as K increases, importances diverge due to masking effects.

	Eq. 6.10	$K = 1$	$K = 2$	$K = 3$	$K = 4$	$K = 5$	$K = 6$	$K = 7$
X_1	0.412	0.414	0.362	0.327	0.309	0.304	0.305	0.306
X_2	0.581	0.583	0.663	0.715	0.757	0.787	0.801	0.799
X_3	0.531	0.532	0.512	0.496	0.489	0.483	0.475	0.475
X_4	0.542	0.543	0.525	0.484	0.445	0.414	0.409	0.412
X_5	0.656	0.658	0.731	0.778	0.810	0.827	0.831	0.835
X_6	0.225	0.221	0.140	0.126	0.122	0.122	0.121	0.120
X_7	0.372	0.368	0.385	0.392	0.387	0.382	0.375	0.372
$\sum$	3.321	3.321	3.321	3.321	3.321	3.321	3.321	3.321

Tabela 6.2: Importâncias das variáveis calculadas com um conjunto de árvores aleatórias, para valores crescentes de K. As importâncias em $K = 1$ seguem seus valores teóricos, conforme previsto pela Equação 6.10 no Teorema 6.1. No entanto, à medida que K aumenta, as importâncias divergem devido aos efeitos de mascaramento. De acordo com o Teorema 6.2, a sua soma também é sempre igual a $I(X_1, \dots, X_7; Y) = H(Y) = \log_2(10) = 3.321$ uma vez que as entradas permitem prever perfeitamente a saída.

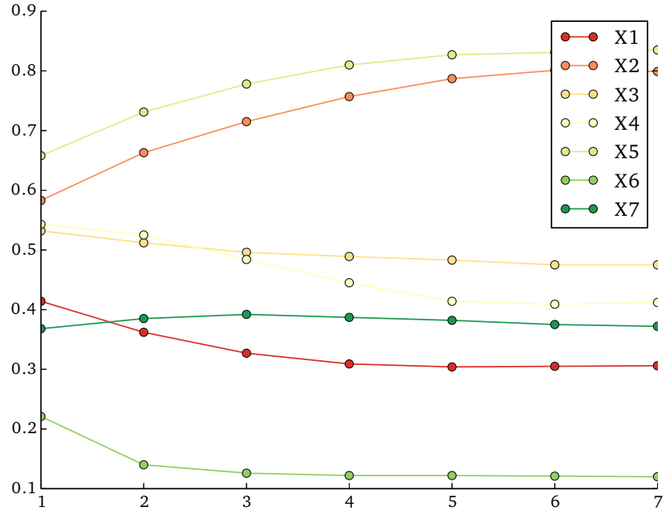


Figura 6.2: Importâncias de variáveis calculadas com um conjunto de árvores aleatórias, para valores crescentes de K. As importâncias em $K = 1$ seguem seus valores teóricos, conforme previsto pela Equação 6.10 no Teorema 6.1. No entanto, à medida que K aumenta, as importâncias divergem devido aos efeitos de mascaramento.

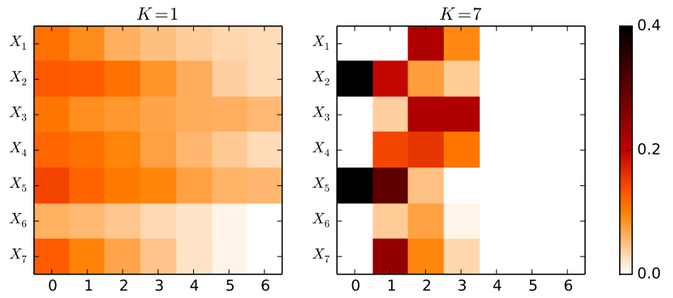


Figure 6.3: Decomposition of variable importances along the degrees k of interactions of one variable with the other ones. At $K = 1$, all $I(X_j; Y|B)$ are accounted for in the total importance, while at $K = 7$ only some of them are taken into account due to masking effects.

importances of the remaining variables are biased and that they actually only account of their relevance when conditioned to X_2 or X_5, but not of their relevance in other contexts. At $k = 0$, masking effects also amplify the contribution of $I(X_2; Y)$ (resp. $I(X_5; Y)$) since X_2 (resp. X_5) appears more frequently at the root node than in totally randomized trees. In addition, because nodes become pure before reaching depth p, conditioning sets of size $k \geqslant 4$ are never actually encountered, which means that we can no longer know whether variables are still informative when conditioned to many others. All in all, this figure thus indeed confirms that importances as computed with non-totally randomized trees take into account only some of the conditioning sets B, hence biasing the measured importances.

6.6 CONCLUSIONS

In this chapter, we made a first step towards understanding variable importances as computed with a forest of randomized trees. In particular, we derived a theoretical characterization of the Mean Decrease Impurity importances as computed by totally randomized trees in asymptotic conditions. We showed that they offer a three-level decomposition of the information jointly provided by all input variables about the output (Section 6.2). We then demonstrated (Section 6.3) that MDI importances as computed by totally randomized trees exhibit desirable properties for assessing the relevance of a variable: it is equal to zero if and only if the variable is irrelevant and it depends only on the relevant variables. We discussed the case of Random Forests and Extremely Randomized Trees (Section 6.4) and finally illustrated our developments on an artificial but insightful example (Section 6.5).

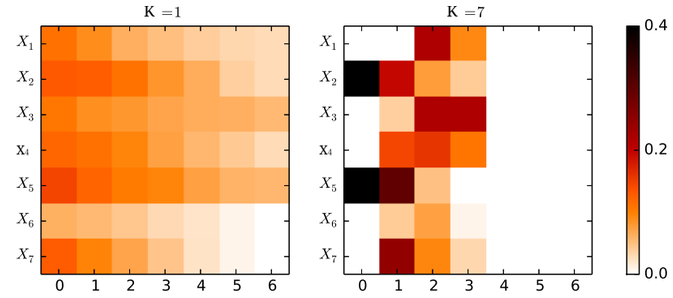


Figura 6.3: Decomposição das importâncias das variáveis ao longo dos graus k de interações de uma variável com as outras. Em $K = 1$, todos $I(X_j; Y|B)$ são contabilizados na importância total, enquanto em $K = 7$ apenas alguns deles são levados em consideração devido a efeitos de mascaramento.

As importâncias das restantes variáveis são tendenciosas e que na verdade só consideram a sua relevância quando condicionadas a X_2 ou X_5, mas não a sua relevância noutros contextos. Em $k = 0$, os efeitos de mascaramento também amplificam a contribuição de $I(X_2; Y)$ (resp. $I(X_5; Y)$) já que X_2 (resp. X_5) aparece com mais frequência no nó raiz do que em árvores totalmente aleatórias. Além disso, como os nós se tornam puros antes de atingirem a profundidade p, conjuntos de condicionamento de tamanho $k \geqslant 4$ nunca são realmente encontrados, o que significa que não podemos mais saber se as variáveis ainda são informativas quando condicionadas a muitas outras. Em suma, esta figura confirma de fato que as importâncias calculadas com árvores não totalmente aleatórias levam em consideração apenas alguns dos conjuntos de condicionamento B, distorcendo assim as importâncias medidas.

6.6 conclusões

Neste capítulo, demos um primeiro passo para compreender as importâncias das variáveis calculadas com uma floresta de árvores aleatórias. Em particular, derivamos uma caracterização teórica das importâncias da diminuição média da impureza conforme calculada por árvores totalmente aleatórias em condições assintóticas. Mostrámos que oferecem uma decomposição em três níveis da informação fornecida conjuntamente por todas as variáveis de entrada sobre o produto (Secção 6.2). Demonstramos então (Seção 6.3) que as importâncias do MDI calculadas por árvores totalmente aleatórias exibem propriedades desejáveis para avaliar a relevância de uma variável: é igual a zero se e somente se a variável for irrelevante e depende apenas das variáveis relevantes. Discutimos o caso das Florestas Aleatórias e das Árvores Extremamente Aleatórias (Seção 6.4) e finalmente ilustramos nossos desenvolvimentos com um exemplo artificial, mas esclarecedor (Seção 6.5).

There remain several limitations to our framework that we would like address in the future. First, our results should be adapted to binary splits as used within an actual Random Forest-like algorithm. In this setting, any node t is split in only two subsets, which means that any variable may then appear one or several times within a branch, and thus should make variable importances now dependent on the cardinalities of the input variables. In the same direction, our framework should also be extended to the case of continuous variables. Finally, results presented in this work are valid in an asymptotic setting only. An important direction of future work includes the characterization of the distribution of variable importances in a finite setting.

Restam várias limitações à nossa estrutura que gostaríamos de abordar no futuro. Primeiro, nossos resultados devem ser adaptados às divisões binárias usadas em um algoritmo real do tipo Random Forest. Nesta configuração, qualquer nó t é dividido em apenas dois subconjuntos, o que significa que qualquer variável pode então aparecer uma ou várias vezes dentro de uma ramificação e, portanto, deve tornar as importâncias das variáveis agora dependentes das cardinalidades das variáveis de entrada. Na mesma direção, a nossa estrutura também deve ser estendida ao caso de variáveis contínuas. Finalmente, os resultados apresentados neste trabalho são válidos apenas em um cenário assi ntótico. Uma direção importante de trabalhos futuros inclui a caracterização da distribuição de importâncias variáveis em um cenário finito.

7 FURTHER INSIGHTS FROM IMPORTANCES

Outline

In this chapter, we build upon results from Chapter 6 to further study variable importances as computed from random forests. In Section 7.1, we first examine importances for variables that are redundant. In Section 7.2, we revisit variable importances in the context of binary decision trees and ordered variables. In this framework, we highlight various sources of bias that may concurrently happen when importances are computed from actual random forests. Finally, we present in Section 7.3 some successful applications of variable importance measures.

Caution. *The work presented in this chapter is exploratory. Conclusions should be considered with a grain of salt, until further empirical verifications.*

7.1 REDUNDANT VARIABLES

In most machine learning problems, it is typical for input variables to be correlated, at least to some extent, and to share common bits of information. In image classification for instance, pixels are usually highly correlated and individually represent nearly the same information as their neighbors. In that sense, variables are often *partially redundant*, i.e., some of the variables may share some of the same information about the output variable Y. In the extreme case, redundancy is *total* or *complete*, with some of the variables redundantly conveying exactly the same information with respect to the output variable Y. In this section, we study redundancy in random forests and show that it may have a significant effect on both the accuracy of the ensemble and variable importance measures.

As a guiding example for our discussion, let us consider a set of input variables and let us discuss the effect of adding redundant variables on the structure of randomized trees. Intuitively, two variables X_i and X_j are redundant if one can perfectly explains the other and vice-versa. Formally, we define redundancy as follows:

Definition 7.1. *Two variables X_i, X_j are* totally redundant *if no additional information is required for describing X_i given X_j and vice-versa. I.e., if*

$$H(X_i|X_j) = H(X_j|X_i) = 0. \tag{7.1}$$

In particular, a variable X_j and its copy, denoted X'_j, are totally redundant. With respect to random forests, adding copies of variables

7 FURHERINSIGHTSFROMIMPORTANCES

Contorno

Neste capítulo, baseamo-nos nos resultados do Capítulo 6 para estudar melhor as importâncias das variáveis calculadas a partir de florestas aleatórias. Na Seção 7.1, primeiro examinamos as importâncias das variáveis que são redundantes. Na Seção 7.2, revisitamos as importâncias das variáveis no contexto de árvores de decisão binárias e variáveis ordenadas. Nesta estrutura, destacamos várias fontes de viés que podem ocorrer simultaneamente quando as importâncias são calculadas a partir de florestas aleatórias reais. Finalmente, apresentamos na Secção 7.3 algumas aplicações bem sucedidas de medidas de importância variável.

Cuidado. O trabalho apresentado neste capítulo é exploratório. As conclusões devem ser consideradas com cautela, até novas verificações empíricas.

7.1 variáveis redundantes

Na maioria dos problemas de aprendizado de máquina, é típico que as variáveis de entrada sejam correlacionadas, pelo menos até certo ponto, e compartilhem informações comuns. Na classificação de imagens, por exemplo, os pixels são geralmente altamente correlacionados e representam individualmente quase a mesma informação que os seus vizinhos. Nesse sentido, as variáveis são muitas vezes parcialmente redundantes, ou seja, algumas das variáveis podem compartilhar algumas das mesmas informações sobre a variável de saída Y. No caso extremo, a redundância é total ou completa, com algumas das variáveis transmitindo de forma redundante exatamente a mesma informação em relação à variável de saída Y. Nesta seção, estudamos a redundância em florestas aleatórias e mostramos que ela pode ter um efeito significativo tanto na precisão do conjunto quanto nas medidas de importância variável.

Como exemplo orientador para a nossa discussão, consideremos um conjunto de variáveis de entrada e discutamos o efeito da adição de variáveis redundantes na estrutura de árvores aleatórias. Intuitivamente, duas variáveisX_i e X_j são redundantes se uma puder explicar perfeitamente a outra e vice-versa. Formalmente, definimos redundância da seguinte forma:

Definição 7.1. Duas variáveis X_i, X_j são totalmente redundantes se nenhuma informação adicional for necessária para descrever X_i dado X_j e vice-versa. Ou seja, se

$$H(X_i|X_j) = H(X_j|X_i) = 0. \tag{7.1}$$

Em particular, uma variável X_j e sua cópia, denotada X'_j, são totalmente redundantes. Com relação às florestas aleatórias, adicionando cópias de variáveis

(e.g., duplicating X_j, hence resulting in a new set of $p+1$ input variables) has no effect when the selection of the split is deterministic (e.g., in RF for K set to the maximum value). No matter the number of totally redundant variables, the best split that is selected is always the same, even if the same splits need to be recomputed multiple times due to redundancy. When the choice of the best split is stochastic however (e.g., for K strictly smaller than the total number of variables), adding multiple copies of a variable X_j results in splits that may be biased towards this variable (or one of its copies), which in turn may have a significant effect on the resulting accuracy of the ensemble. For a fixed value of K, it is indeed not difficult to see that adding copies of X_j increases the probability of X_j, or of one of its copies, to be in the random subset of K input variables on which to look for splits. As a corollary, it therefore also simultaneously decreases the probability of any of the others to be selected, hence biasing the structure of the generated decision trees. Note that the resulting net effect on accuracy depends on the nature of duplicated variable. If X_j is very informative with respect to the input, then favoring splits on X_j by adding copies may result in an increase of accuracy. By contrast, if X_j is irrelevant, then adding copies increases the risk of overfitting.

With respect to variable importances, the effect of adding redundant variables can be derived both qualitatively and quantitatively using results from Chapter 6. From Theorem 6.5, we already know that adding irrelevant variables does not change the resulting variable importances. Adding copies of a relevant variable however, has an effect on both the importance of the duplicated variable and on the importance of the remaining variables. As in the previous chapter, let us assume a set $V = \{X_1, ..., X_p\}$ of categorical input variables and a categorical output Y, for which we derive MDI importances, as computed from totally randomized and fully developed trees built on an infinitely large dataset.

Lemma 7.1. *Let X_i and X_j be totally redundant variables. For any conditioning set B,*

$$I(X_i; Y|B, X_j) = I(X_j; Y|B, X_i) = 0 \tag{7.2}$$

$$I(X_i; Y|B) = I(X_j; Y|B). \tag{7.3}$$

Proof. By symmetry of the mutual information, it comes

$$\begin{aligned} I(X_i; X_j) &= H(X_i) - H(X_i|X_j) \\ &= H(X_j) - H(X_j|X_i), \end{aligned} \tag{7.4}$$

which implies that $H(X_i) = H(X_j)$ since $H(X_i|X_j) = H(X_j|X_i) = 0$ if X_i and X_j are totally redundant. Since $0 \leqslant H(X_i|X_j, B) \leqslant H(X_i|X_j)$ and $H(X_i|X_j) = 0$, we also have $H(X_i|X_j, B) = 0$ for any conditioning set B. Likewise, $H(X_j|X_i, B) = 0$. By reusing the same argument for $I(X_i; X_j|B)$ (instead of $I(X_i; X_j)$), equality therefore extends to any

(por exemplo, duplicar X_j, resultando em um novo conjunto de variáveis de entrada $p+1$) não tem efeito quando a seleção da divisão é determinística (por exemplo, em RF para K definido para o valor máximo). Não importa o número de variáveis totalmente redundantes, a melhor divisão selecionada é sempre a mesma, mesmo que as mesmas divisões precisem ser recalculadas diversas vezes devido à redundância. Entretanto, quando a escolha da melhor divisão é estocástica (por exemplo, para K estritamente menor que o número total de variáveis), adicionar múltiplas cópias de uma variável X_j resulta em divisões que podem ser tendenciosas para esta variável (ou uma de suas cópias), o que por sua vez pode ter um efeito significativo na precisão resultante do conjunto. Para um valor fixo de K, não é realmente difícil ver que adicionar cópias de X_j aumenta a probabilidade de X_j, ou de uma de suas cópias, estar no subconjunto aleatório de K variáveis de entrada nas quais procurar divisões. Como corolário, portanto, também diminui simultaneamente a probabilidade de qualquer um dos outros ser seleccionado, distorcendo assim a estrutura das árvores de decisão geradas. Observe que o efeito líquido resultante na precisão depende da natureza da variável duplicada. Se X_j for muito informativo em relação à entrada, favorecer divisões em X_j adicionando cópias pode resultar em um aumento de precisão. Por outro lado, se X_j for irrelevante, adicionar cópias aumenta o risco de overfitting.

No que diz respeito às importâncias das variáveis, o efeito da adição de variáveis redundantes pode ser derivado tanto qualitativa quanto quantitativamente usando os resultados do Capítulo 6. Do Teorema 6.5, já sabemos que adicionar variáveis irrelevantes não altera as importâncias das variáveis resultantes. Adicionar cópias de uma variável relevante, entretanto, tem efeito tanto na importância da variável duplicada quanto na importância das variáveis restantes. Como no capítulo anterior, vamos assumir um conjunto $V = \{X_1, ..., X_p\}$ de variáveis de entrada categóricas e uma saída categórica Y, para a qual derivamos importâncias do MDI, calculadas a partir de árvores totalmente aleatórias e totalmente desenvolvidas construídas em um conjunto de dados infinitamente grande.

Lema 7.1. Sejam X_i e X_j variáveis totalmente redundantes. Para qualquer conjunto de condicionamento B,

$$I(X_i; Y|B, X_j) = I(X_j; Y|B, X_i) = 0 \tag{7.2}$$

$$I(X_i; Y|B) = I(X_j; Y|B). \tag{7.3}$$

Prova. Pela simetria da informação mútua, chega-se

$$\begin{aligned} I(X_i; X_j) &= H(X_i) - H(X_i|X_j) \\ &= H(X_j) - H(X_j|X_i), \end{aligned} \tag{7.4}$$

o que implica que $H(X_i) = H(X_j)$ já que $H(X_i|X_j) = H(X_j|X_i) = 0$ seX_i e X_j são totalmente redundantes. Desde $0 \leqslant H(X_i|X_j, B) \leqslant H(X_i|X_j)$ e $H(X_i|X_j) = 0$, também temos $H(X_i|X_j, B) = 0$ para qualquer conjunto de condicionamento B. Da mesma forma, $H(X_j|X_i, B) = 0$. Ao reutilizar o mesmo argumento para $I(X_i; X_j|B)$ (em vez de $I(X_i; X_j)$), a igualdade, portanto, se estende a qualquer

conditioning set B, giving $H(X_i|B) = H(X_j|B)$. From these, it follows that,

$$I(X_i; Y|B, X_j) = H(X_i|B, X_j) - H(X_i|B, X_j, Y) = 0 - 0, \tag{7.5}$$
$$I(X_j; Y|B, X_i) = H(X_j|B, X_i) - H(X_j|B, X_i, Y) = 0 - 0, \tag{7.6}$$

which proves Equation 7.2. We also have

$$\begin{aligned} I(X_i; Y|B) &= H(X_i|B) - H(X_i|B, Y) && (7.7)\\ &= H(X_j|B) - H(X_j|B, Y) && (7.8)\\ &= I(X_j; Y|B), && (7.9) \end{aligned}$$

which proves Equation 7.3. □

Proposition 7.2. *Let $X_j \in V$ be a relevant variable with respect to Y and V and let $X'_j \notin V$ be a totally redundant variable with respect to X_j. The infinite sample size importance of X_j as computed with an infinite ensemble of fully developed totally randomized trees built on $V \cup \{X'_j\}$ is*

$$Imp(X_j) = \sum_{k=0}^{p-1} \frac{p-k}{p+1} \frac{1}{C_p^k} \frac{1}{p-k} \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B) \tag{7.10}$$

Proof. From Theorem 6.1, the variable importance of X_j is

$$\begin{aligned} \text{Imp}(X_j) &= \sum_{k=0}^{p-1+1} \frac{1}{C_{p+1}^k} \frac{1}{p+1-k} \sum_{B \in \mathcal{P}_k(V^{-j} \cup \{X'_j\})} I(X_j; Y|B)\\ &= \sum_{k=0}^{p-1} \frac{1}{C_{p+1}^k} \frac{1}{p+1-k} \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B)\\ &= \sum_{k=0}^{p-1} \frac{p-k}{p+1} \frac{1}{C_p^k} \frac{1}{p-k} \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B), \end{aligned} \tag{7.11}$$

since from Lemma 7.1, $I(X_j; Y|B \cup X'_j) = 0$ for all $B \in \mathcal{P}(V^{-j})$. □

Lemma 7.3. *Let X_i and X_j be totally redundant variables. For any conditioning set B and for any variable X_l,*

$$I(X_l; Y|B, X_i) = I(X_l; Y|B, X_j) = I(X_l; Y|B, X_i, X_j) \tag{7.12}$$

Proof. From the chaining rule of the mutual information, we have

$$\begin{aligned} I(X_i, X_j, X_l; Y|B) &= I(X_l; Y|B) + I(X_i, X_j; Y|B, X_l)\\ &= I(X_l; Y|B) + I(X_i; Y|B, X_l) + I(X_i; Y|B, X_j, X_l)\\ &= I(X_l; Y|B) + I(X_i; Y|B, X_l) \quad \text{(Lemma 7.1)}\\ &= I(X_i, X_l; Y|B)\\ &= I(X_i; Y|B) + I(X_l; Y|B, X_i). \end{aligned} \tag{7.13}$$

conjunto de condicionamento B, dando $H(X_i|B) = H(X_j|B)$. Destes, segue-se que,

$$I(X_i; Y|B, X_j) = H(X_i|B, X_j) - H(X_i|B, X_j, Y) = 0 - 0, \tag{7.5}$$
$$I(X_j; Y|B, X_i) = H(X_j|B, X_i) - H(X_j|B, X_i, Y) = 0 - 0, \tag{7.6}$$

o que prova a Equação 7.2. Nós também temos

$$\begin{aligned} I(X_i; Y|B) &= H(X_i|B) - H(X_i|B, Y) && (7.7)\\ &= H(X_j|B) - H(X_j|B, Y) && (7.8)\\ &= I(X_j; Y|B), && (7.9) \end{aligned}$$

o que prova a Equação 7.3. □

Proposição 7.2. Seja $X_j \in V$ uma variável relevante em relação a Y e V e sej a $X'_j \notin V$ uma variável totalmente redundante em relação a X_j. A importância do tamanho de amostra infinito de X_j conforme calculado c om um conjunto infinito de árvores totalmente aleatórias totalmente desenvolvidas construídas em $V \cup \{X'_j\}$ é

$$Imp(X_j) = \sum_{k=0}^{p-1} \frac{p-k}{p+1} \frac{1}{C_p^k} \frac{1}{p-k} \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B) \tag{7.10}$$

Prova. Do Teorema 6.1, a importância variável de X_j é

$$\begin{aligned} \text{Imp}(X_j) &= \sum_{k=0}^{p-1+1} \frac{1}{C_{p+1}^k} \frac{1}{p+1-k} \sum_{B \in \mathcal{P}_k(V^{-j} \cup \{X'_j\})} I(X_j; Y|B)\\ &= \sum_{k=0}^{p-1} \frac{1}{C_{p+1}^k} \frac{1}{p+1-k} \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B)\\ &= \sum_{k=0}^{p-1} \frac{p-k}{p+1} \frac{1}{C_p^k} \frac{1}{p-k} \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B), \end{aligned} \tag{7.11}$$

desde o Lema 7.1, $I(X_j; Y|B \cup X'_j) = 0$ para todos $B \in \mathcal{P}(V^{-j})$. □

Lema 7.3. Sejam X_i e X_j variáveis totalmente redundantes. Para qualquer conjunto de condicionamento B e para qualquer variável X_l,

$$I(X_l; Y|B, X_i) = I(X_l; Y|B, X_j) = I(X_l; Y|B, X_i, X_j) \tag{7.12}$$

Prova. Da regra de encadeamento da informação mútua, temos

$$\begin{aligned} I(X_i, X_j, X_l; Y|B) &= I(X_l; Y|B) + I(X_i, X_j; Y|B, X_l)\\ &= I(X_l; Y|B) + I(X_i; Y|B, X_l) + I(X_i; Y|B, X_j, X_l)\\ &= I(X_l; Y|B) + I(X_i; Y|B, X_l) \quad \text{(Lemma 7.1)}\\ &= I(X_i, X_l; Y|B)\\ &= I(X_i; Y|B) + I(X_l; Y|B, X_i). \end{aligned} \tag{7.13}$$

By symmetry,

$$I(X_i, X_j, X_l; Y|B) = I(X_j; Y|B) + I(X_l; Y|B, X_j), \tag{7.14}$$

which proves that $I(X_l; Y|B, X_i) = I(X_l; Y|B, X_j)$, by combining both equations and using the fact that $I(X_i; Y|B) = I(X_j; Y|B)$ (Lemma 7.1).

From the chaining rule, we also have

$$\begin{aligned} I(X_i, X_j, X_l; Y|B) &= I(X_i, X_j; Y|B) + I(X_l; Y|B, X_i, X_j) \\ &= I(X_i; Y|B) + I(X_j; Y|B, X_i) + I(X_l; Y|B, X_i, X_j) \\ &= I(X_i; Y|B) + I(X_l; Y|B, X_i, X_j). \end{aligned} \tag{7.15}$$

By combining this last equation with Equation 7.13, we finally have $I(X_l; Y|B, X_i) = I(X_l; Y|B, X_i, X_j)$, which proves Lemma 7.3. □

Proposition 7.4. *Let $X_j \in V$ be a relevant variable with respect to Y and V and let $X'_j \notin V$ be a totally redundant variable with respect to X_j. The infinite sample size importance of $X_l \in V^{-j}$ as computed with an infinite ensemble of fully developed totally randomized trees built on $V \cup \{X'_j\}$ is*

$$\begin{aligned} Imp(X_l) = &\sum_{k=0}^{p-2} \frac{p-k}{p+1} \frac{1}{C_p^k} \frac{1}{p-k} \sum_{B \in \mathcal{P}_k(V^{-l} \setminus X_j)} I(X_l; Y|B) + \\ &\hookrightarrow \sum_{k=0}^{p-2} \left[\sum_{k'=1}^{2} \frac{C_2^{k'}}{C_{p+1}^{k+k'}} \frac{1}{p+1-(k+k')} \right] \sum_{B \in \mathcal{P}_k(V^{-l} \setminus X_j)} I(X_l; Y|B \cup X_j). \end{aligned} \tag{7.16}$$

Proof. From Lemma 7.3, conditioning by either X_j, X'_j or by both variables yield terms $I(X_l; Y|B, X_j)$, $I(X_l; Y|B, X'_j)$ and $I(X_l; Y|B, X_j, X'_j)$ that are all equal. From Theorem 6.1, the variable importance of X_l can therefore be rewritten as follows:

$$\begin{aligned} Imp(X_l) &= \sum_{k=0}^{p-1+1} \frac{1}{C_{p+1}^k} \frac{1}{p+1-k} \sum_{B \in \mathcal{P}_k(V^{-l} \cup X'_j)} I(X_l; Y|B) \\ &= \sum_{k=0}^{p-2} \sum_{k'=0}^{2} \frac{1}{C_{p+1}^{k+k'}} \frac{1}{p+1-(k+k')} \sum_{\substack{B \in \mathcal{P}_k(V^{-l} \setminus X_j) \\ B' \in \mathcal{P}_{k'}(\{X_j, X'_j\})}} I(X_l; Y|B \cup B') \\ &= \sum_{k=0}^{p-2} \frac{1}{C_{p+1}^k} \frac{1}{p+1-k} \sum_{B \in \mathcal{P}_k(V^{-l} \setminus X_j)} I(X_l; Y|B) + \\ &\hookrightarrow \sum_{k=0}^{p-2} \left[\sum_{k'=1}^{2} \frac{C_2^{k'}}{C_{p+1}^{k+k'}} \frac{1}{p+1-(k+k')} \right] \sum_{B \in \mathcal{P}_k(V^{-l} \setminus X_j)} I(X_l; Y|B \cup X_j) \\ &= \sum_{k=0}^{p-2} \frac{p-k}{p+1} \frac{1}{C_p^k} \frac{1}{p-k} \sum_{B \in \mathcal{P}_k(V^{-l} \setminus X_j)} I(X_l; Y|B) + \end{aligned}$$

Por simetria,

$$I(X_i, X_j, X_l; Y|B) = I(X_j; Y|B) + I(X_l; Y|B, X_j), \tag{7.14}$$

o que prova que $I(X_l; Y|B, X_i) = I(X_l; Y|B, X_j)$, combinando ambas as equações e usando o fato de que $I(X_i; Y|B) = I(X_j; Y|B)$ (Lema 7.1). Da regra do encadeamento, também temos

$$\begin{aligned} I(X_i, X_j, X_l; Y|B) &= I(X_i, X_j; Y|B) + I(X_l; Y|B, X_i, X_j) \\ &= I(X_i; Y|B) + I(X_j; Y|B, X_i) + I(X_l; Y|B, X_i, X_j) \\ &= I(X_i; Y|B) + I(X_l; Y|B, X_i, X_j). \end{aligned} \tag{7.15}$$

Combinando esta última equação com a Equação 7.13, finalmente temos $I(X_l; Y|B, X_i) = I(X_l; Y|B, X_i, X_j)$, o que prova o Lema 7.3. □

Proposição 7.4. Seja $X_j \in V$ uma variável relevante em relação a Y e V e sej a $X'_j \notin V$ uma variável totalmente redundante em relação a X_j. A importância do tamanho de amostra infinito de $X_l \in V^{-j}$ conforme calculado com um conjunto infinito de árvores totalmente aleatórias totalmente desenvolvidas construídas em $V \cup \{X'_j\}$ é

$$\begin{aligned} Imp(X_l) = &\sum_{k=0}^{p-2} \frac{p-k}{p+1} \frac{1}{C_p^k} \frac{1}{p-k} \sum_{B \in \mathcal{P}_k(V^{-l} \setminus X_j)} I(X_l; Y|B) + \\ &\hookrightarrow \sum_{k=0}^{p-2} \left[\sum_{k'=1}^{2} \frac{C_2^{k'}}{C_{p+1}^{k+k'}} \frac{1}{p+1-(k+k')} \right] \sum_{B \in \mathcal{P}_k(V^{-l} \setminus X_j)} I(X_l; Y|B \cup X_j). \end{aligned} \tag{7.16}$$

Prova. Do Lema 7.3, o condicionamento por X_j, X'_j ou por ambas as variáveis produz termos $I(X_l; Y|B, X_j)$, $I(X_l; Y|B, X'_j)$ e $I(X_l; Y|B, X_j, X'_j)$ que s ão todos iguais. A partir do Teorema 6.1, a importância variável de X_l pode, portanto, ser reescrita da seguinte forma:

$$\begin{aligned} Imp(X_l) &= \sum_{k=0}^{p-1+1} \frac{1}{C_{p+1}^k} \frac{1}{p+1-k} \sum_{B \in \mathcal{P}_k(V^{-l} \cup X'_j)} I(X_l; Y|B) \\ &= \sum_{k=0}^{p-2} \sum_{k'=0}^{2} \frac{1}{C_{p+1}^{k+k'}} \frac{1}{p+1-(k+k')} \sum_{\substack{B \in \mathcal{P}_k(V^{-l} \setminus X_j) \\ B' \in \mathcal{P}_{k'}(\{X_j, X'_j\})}} I(X_l; Y|B \cup B') \\ &= \sum_{k=0}^{p-2} \frac{1}{C_{p+1}^k} \frac{1}{p+1-k} \sum_{B \in \mathcal{P}_k(V^{-l} \setminus X_j)} I(X_l; Y|B) + \\ &\hookrightarrow \sum_{k=0}^{p-2} \left[\sum_{k'=1}^{2} \frac{C_2^{k'}}{C_{p+1}^{k+k'}} \frac{1}{p+1-(k+k')} \right] \sum_{B \in \mathcal{P}_k(V^{-l} \setminus X_j)} I(X_l; Y|B \cup X_j) \\ &= \sum_{k=0}^{p-2} \frac{p-k}{p+1} \frac{1}{C_p^k} \frac{1}{p-k} \sum_{B \in \mathcal{P}_k(V^{-l} \setminus X_j)} I(X_l; Y|B) + \end{aligned}$$

$$\hookrightarrow \sum_{k=0}^{p-2} \left[\sum_{k'=1}^{2} \frac{C_2^{k'}}{C_{p+1}^{k+k'}} \frac{1}{p+1-(k+k')} \right] \sum_{B \in \mathcal{P}_k(V^{-l} \setminus X_j)} I(X_l; Y|B \cup X_j).$$

□

Proposition 7.2 shows that the importance of X_j decreases when a redundant variable X'_j is added to the set of input variables, since all mutual information terms are multiplied by a factor $\frac{p-k}{p+1} < 1$. Intuitively, this result is in fact expected since the same information is then conveyed within two variables (i.e., in X_j and its copy X'_j). It also shows that the terms in the total importance are not all modified in the same way. The weight of the terms corresponding to small conditioning sets remains nearly unchanged (i.e., for a large number p of variables and small values of k, $\frac{p-k}{p+1}$ is close to 1), while the weight of the terms of large conditioning sets is greatly impacted (i.e., for large values of k, $\frac{p-k}{p+1}$ tends to 0).

As shown by Proposition 7.4, the effect of adding a redundant variable X'_j on the importance of the other variables X_l (for $l \neq j$) is twofold. The first part of Equation 7.16 shows that the weight of all terms that do not include X_j (or its copy) strictly decreases by a factor $\frac{p-k}{p+1}$. The second part of Equation 7.16 shows that the the weight of all terms that include X_j (or its copy) increases, since several equivalent conditioning sets (i.e., $B \cup \{X_j\}$, $B \cup \{X_j\}'$ and $B \cup \{X_j, X'_j\}$) can now appear within the branches of a tree. Like for Proposition 7.2, impurity terms are not all modified in the same way and changes depend on the size k of the conditioning set B. Overall, the net effect on the total importance of X_l is therefore a compromise between these opposing changes. If the weighted sum of the $I(X_l; Y|B, X_j)$ terms is small with respect to the sum of the terms that do not include X_j (or its copy), then the decrease effect dominates and the importance of X_l should be smaller. By contrast, if the $I(X_l; Y|B, X_j)$ terms are larger, then the increase effect dominates and the resulting importance is larger. As shown later in Figure 7.2, redundant variables therefore increase the importance of all variables that *interact* with the duplicated variable.

Propositions 7.2 and 7.4 can be extended to the case where N_c redundant variables X_j^c (for $c = 1, \ldots, N_c$) are added to the input variables instead of one, leading to the general Proposition 7.5. From this result, the same qualitative conclusions can be drawn, except that the decrease or increase effects discussed above are even stronger as more redundant variables are added.

Proposition 7.5. *Let $X_j \in V$ be a relevant variable with respect to Y and V and let $X_j^c \notin V$ (for $c = 1, \ldots, N_c$) be N_c totally redundant variables with respect to X_j. The infinite sample size importances of X_j and $X_l \in V$*

$$\hookrightarrow \sum_{k=0}^{p-2} \left[\sum_{k'=1}^{2} \frac{C_2^{k'}}{C_{p+1}^{k+k'}} \frac{1}{p+1-(k+k')} \right] \sum_{B \in \mathcal{P}_k(V^{-l} \setminus X_j)} I(X_l; Y|B \cup X_j).$$

□

A proposição 7.2 mostra que a importância de X_j diminui quando uma variável redundante X'_j é adicionada ao conjunto de variáveis de entrada, uma vez que todos os termos de informação mútua são multiplicados por um fator $\frac{p-k}{p+1} < 1$. Intuitivamente, este resultado é de facto esperado, uma vez que a mesma informação é então transmitida dentro de duas variáveis (isto é, em X_j e na sua cópia X'_j). Mostra também que os termos de importância total não são todos modificados da mesma forma. O peso dos termos correspondentes a pequenos conjuntos de condicionamento permanece quase inalterado (ou seja, para um grande número p de variáveis e pequenos valores de k, $\frac{p-k}{p+1}$ é próximo de 1), enquanto o peso dos termos de grandes conjuntos de condicionamento é grandemente impactado (ou seja, para grandes valores de k, $\frac{p-k}{p+1}$ tende a 0).

Como mostra a Proposição 7.4, o efeito da adição de uma variável redundante X'_j sobre a importância das outras variáveis X_l (para $l \neq j$) é duplo. A primeira parte da Equação 7.16 mostra que o peso de todos os termos que não incluem X_j (ou sua cópia) diminui estritamente por um fator $\frac{p-k}{p+1}$. A segunda parte da Equação 7.16 mostra que o peso de todos os termos que incluem X_j (ou sua cópia) aumenta, uma vez que vários conjuntos de condicionamento equivalentes (isto é, $B \cup \{X_j\}$, $B \cup \{X_j\}'$ e $B \cup \{X_j, X'_j\}$) podem agora aparecer dentro dos ramos de uma árvore. Como na Proposição 7.2, os termos de impureza não são todos modificados da mesma maneira e as mudanças dependem do tamanho k do conjunto de condicionamento B. No geral, o efeito líquido sobre a importância total de X_l é, portanto, um compromisso entre estas mudanças opostas. Se a soma ponderada dos termos $I(X_l; Y|B, X_j)$ for pequena em relação à soma dos termos que não incluem X_j (ou sua cópia), então o efeito de diminuição domina e a importância de X_l deve ser menor. Por outro lado, se os t ermos $I(X_l; Y|B, X_j)$ forem maiores, então o efeito de aumento domina e a importância resultante é maior. Como mostrado mais adiante na Figura 7.2, as variáveis redundantes aumentam, portanto, a importância de todas as variáveis que interagem com a variável duplicada.

As proposições 7.2 e 7.4 podem ser estendidas para o caso em que N_c variáveis redundantes X_j^c (para $c = 1, \ldots, N_c$) são adicionadas às variáveis de entrada em vez de uma, levando à Proposição geral 7.5. Deste resultado, podem ser tiradas as mesmas conclusões qualitativas, excepto que os efeitos de diminuição ou aumento discutidos acima são ainda mais fortes à medida que mais variáveis redundantes são adicionadas.

Proposição 7.5. Seja $X_j \in V$ uma variável relevante em relação a Y e V e seja $X_j^c \notin V$ (para $c = 1, \ldots, N_c$) N_c variáveis totalmente redundantes em relação a X_j. As importâncias infinitas do tamanho da amostra de X_j e $X_l \in V$

as computed with an infinite ensemble of fully developed totally randomized trees built on $V \cup \{X_j^1, \dots, X_j^{N_c}\}$ *are*

$$Imp(X_j) = \sum_{k=0}^{p-1} \left[\frac{C_p^k (p-k)}{C_{p+N_c}^k (p+N_c-k)} \right] \frac{1}{C_p^k} \frac{1}{p-k} \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B),$$

$$Imp(X_l) = \sum_{k=0}^{p-2} \left[\frac{C_p^k (p-k)}{C_{p+N_c}^k (p+N_c-k)} \right] \frac{1}{C_p^k} \frac{1}{p-k} \sum_{B \in \mathcal{P}_k(V^{-l} \setminus X_j)} I(X_l; Y|B) +$$

$$\hookrightarrow \sum_{k=0}^{p-2} \left[\sum_{k'=1}^{N_c+1} \frac{C_{N_c+1}^{k'}}{C_{p+N_c}^{k+k'}} \frac{1}{p+N_c-(k+k')} \right] \sum_{B \in \mathcal{P}_k(V^{-l} \setminus X_j)} I(X_l; Y|B \cup X_j).$$

Proof. Omitted here, but Proposition 7.5 can be proved by generalizing for N_c the proofs of Propositions 7.2 and 7.4. □

Finally, let us note that Propositions 7.2, 7.4 and 7.5 are in fact valid as soon as variables X_i and X_j satisfy conditions of Lemma 7.1, even if they are not totally redundant. Accordingly, we call two variables satisfying these conditions *totally redundant with respect to the output* Y.

As an illustrative example, let us reconsider the LED classification problem from Section 6.5, for which X_5 was found to be the most important variable. As shown in Figure 7.1, adding variables that are redundant with X_5 makes its importance decrease, as predicted by our theoretical result from Propositions 7.2 and 7.5. When 5 or more copies of X_5 are added, the importance of X_5 is the smallest of all, as if X_5 had become less informative. Similarly, we observe that the importance of the other variables remains about the same or slightly decreases, as if they all had become a bit less informative. With regards to our previous results in Propositions 7.4 and 7.5, this indicates that the importance due to the $I(X_l; Y|B)$ terms prevails from the importance due to the $I(X_l; Y|B, X_5^c)$ terms. As a matter of fact, this example highlights a fundamental property of variable importances as computed in a random forest: *importance measures are computed not only with respect to the output* Y*, but also with respect to all the other input variables that define the problem*. In particular, a variable which is not important is not necessarily uninformative, as the example illustrates. A variable may be considered as less important because the information it conveys is also redundantly conveyed and diluted in other variables, and not necessarily because it has no information about the output.

As a second example, Figure 7.2 illustrates redundancy effects for a XOR classification problem defined on two variables X_1 and X_2. Again, the importance of the duplicated variable X_1 decreases as redundant variables are added, which confirms our results from Propositions 7.2 and 7.5. More interestingly, we now observe that the importance of the other variable, X_2, increases as copies of X_1 are added. For this problem, the $I(X_2; Y|B, X_1^c)$ terms are prevalent with respect to

conforme calculado com um conjunto infinito de árvores totalmente desenvolvidas e aleatórias construídas em $V \cup \{X_j^1, \dots, X_j^{N_c}\}$ *são*

$$Imp(X_j) = \sum_{k=0}^{p-1} \left[\frac{C_p^k (p-k)}{C_{p+N_c}^k (p+N_c-k)} \right] \frac{1}{C_p^k} \frac{1}{p-k} \sum_{B \in \mathcal{P}_k(V^{-j})} I(X_j; Y|B),$$

$$Imp(X_l) = \sum_{k=0}^{p-2} \left[\frac{C_p^k (p-k)}{C_{p+N_c}^k (p+N_c-k)} \right] \frac{1}{C_p^k} \frac{1}{p-k} \sum_{B \in \mathcal{P}_k(V^{-l} \setminus X_j)} I(X_l; Y|B) +$$

$$\hookrightarrow \sum_{k=0}^{p-2} \left[\sum_{k'=1}^{N_c+1} \frac{C_{N_c+1}^{k'}}{C_{p+N_c}^{k+k'}} \frac{1}{p+N_c-(k+k')} \right] \sum_{B \in \mathcal{P}_k(V^{-l} \setminus X_j)} I(X_l; Y|B \cup X_j).$$

Prova. Omitida aqui, mas a Proposição 7.5 pode ser provada ge neralizando para N_c as provas das Proposições 7.2 e 7.4. □

Finalmente, observemos que as Proposições 7.2, 7.4 e 7.5 são de fato válidas assim que as variáveis X_i e X_j satisfaçam as condições do Lema 7.1, mesmo que não sejam totalmente redundantes. Conseqüentemente, chamamos duas variáveis que satisfazem essas condições de totalmente redundantes em relação à saída Y.

Como exemplo ilustrativo, vamos reconsiderar o problema de classificação do DEL da Seção 6.5, para o qual X_5 foi considerado a variável mais importante. Conforme mostrado na Figura 7.1, adicionar variáveis redundantes com X_5 faz com que sua importância diminua, conforme previsto pelo nosso resultado teórico das Proposições 7.2 e 7.5. Quando 5 ou mais cópias de X_5 são adicionadas, a importância de X_5 é a menor de todas, como se X_5 tivesse se tornado menos informativo. Da mesma forma, observamos que a importância das outras variáveis permanece praticamente a mesma ou diminui ligeiramente, como se todas elas tivessem se tornado um pouco menos informativas. Com relação aos nossos resultados anteriores nas Proposições 7.4 e 7.5, isso indica que a importância devida aos termos $I(X_l; Y|B)$ prevalece da importância devida aos termos $I(X_l; Y|B, X_5^c)$. Na verdade, este exemplo destaca uma propriedade fundamental das importâncias das variáveis calculadas numa floresta aleatória: as medidas de importância são calculadas não apenas em relação à saída Y, mas também em relação a todas as outras variáveis de entrada que definem o problema. Em particular, uma variável que não é importante não é necessariamente pouco informativa, como ilustra o exemplo. Uma variável pode ser considerada menos importante porque a informação que ela transmite também é transmitida de forma redundante e diluída em outras variáveis, e não necessariamente porque não contém informações sobre o resultado.

Como segundo exemplo, a Figura 7.2 ilustra os efeitos de redundância para um problema de classificação XOR definido em duas variáveis X_1 e X_2. Novamente, a importância da variável duplicada X_1 diminui à medida que variáveis redundantes são adicionadas, o que confirma nossos resultados das Proposições 7.2 e 7.5. O mais interessante é que observa mos agora que a importância da outra variável, X_2, aumenta à medida que cópias de X_1 são adicionadas. Para este problema, os termos $I(X_2; Y|B, X_1^c)$ são predominantes em relação a

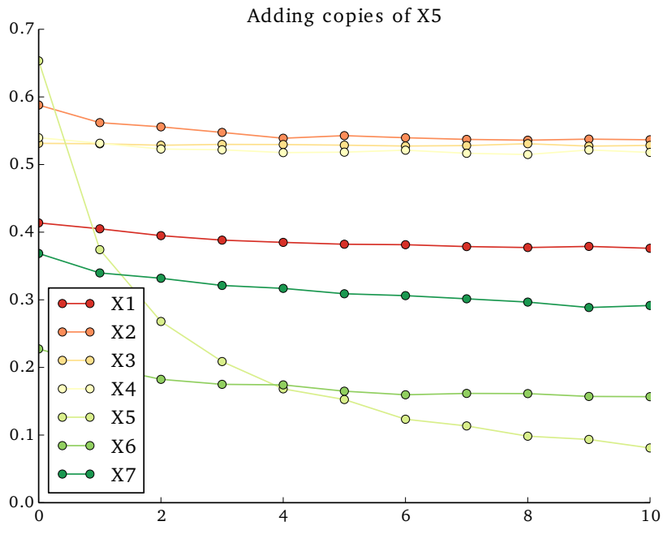


Figure 7.1: Adding copies of X_5 on the LED classification task. The more redundant variables are added, the lesser the importance of X_5.

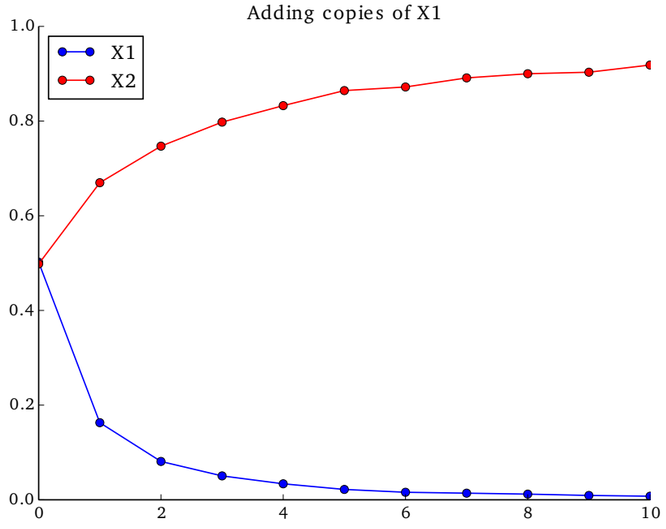


Figure 7.2: Adding copies of X_1 on a XOR classification task. The more redundant variables are added, the lesser the importance of X_1, but the larger the importance of X_2.

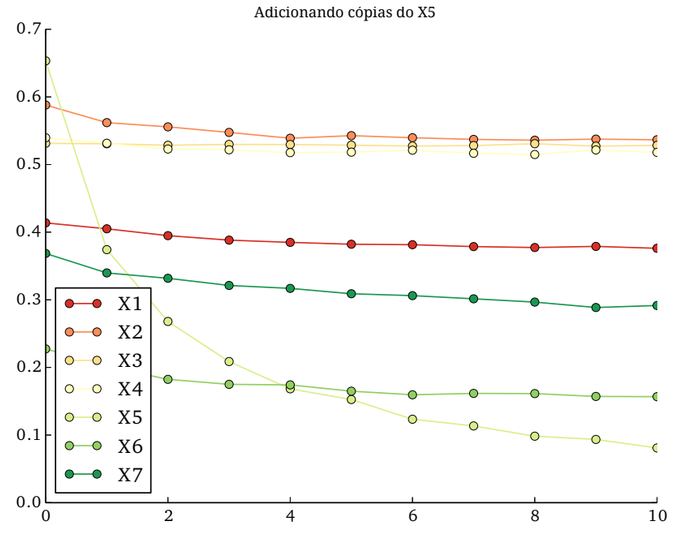


Figura 7.1: Adicionando cópias de X_5 na tarefa de classificação de LED. Quanto mais variáveis redundantes forem adicionadas, menor será a importância de X_5.

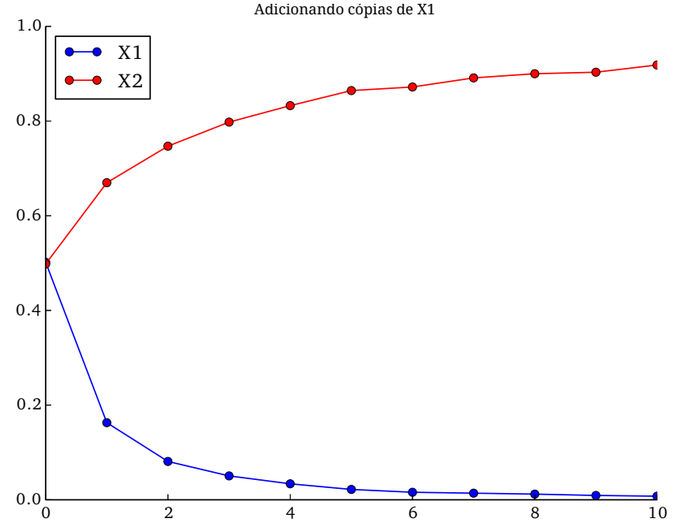


Figura 7.2:Adicionando cópias de X_1 em uma tarefa de classificação XOR. Quanto mais variáveis redundantes forem adicionadas, menor será a importância de X_1, mas maior será a importância de X_2.

the $I(X_2; Y|B)$ terms (which is in fact unique and equal to 0), thereby artificially increasing the overall importance of X_2 as redundancy augments, as expected from Propositions 7.4 and 7.5.

Overall, results presented in this section call for caution when interpreting variable importance scores. Due to redundancy effects – either total, as studied here, or partial as it would often arise in practice – it may happen that the total importance of a given variable is either misleadingly low or deceptively high because the same information is spread within several redundant variables and therefore taken into account several times within the total importances. As such, we advise to complement the interpretation with a systematic decomposition of variable importance scores, e.g., as previously done in Figure 6.3, in order to better understand why a variable is in fact important and to possibly detect redundancy.

7.2 BIAS IN VARIABLE IMPORTANCES

In this section, we study sources of bias in variable importances and show that variable selection (as previously discussed in Section 6.4.3) is not the only cause of bias. In practice, complementary forces due to masking effects, impurity misestimations and the structure of the trees make variable importances deviate from the theoretical results found in asymptotic conditions for totally randomized trees.

7.2.1 *Bias due to masking effects*

As shown in the previous chapters, the guided selection of the split variable (i.e., for $K > 1$) is necessary for balancing bias and variance in randomized decision trees and to produce accurate ensembles. In particular, we studied that decision trees built with too much randomization usually lead to an increase of bias (with respect to the generalization error) which cannot be compensated by a reciprocal decrease of variance, making it necessary to adjust the value of K to find the appropriate trade-off. By contrast, we also showed in Section 6.4.3 that when variable selection is not totally random (i.e., as soon as $K > 1$), masking effects induce a bias with respect to variable importances, since it forces some of the branches to never be built, and therefore some of the conditioning sets $B \in \mathcal{P}(V)$ to never be taken into account. As a consequence, random forests whose parameter K has been tuned to maximize accuracy may yield variable importances that are biased (either over- or underestimated). More specifically, it can be shown (see Section 6.4.3) that a relevant variable may be null with regards to its importance, thereby making it indistinguishable from irrelevant variables, and that the importance of relevant variables becomes dependent on the number of irrelevant variables.

os termos $I(X_2; Y|B)$ (que é de fato único e igual a 0), aumentando assim artificialmente a importância geral de X_2 como aumento de redundância, como esperado das Proposições 7.4 e 7.5.

No geral, os resultados apresentados nesta secção exigem cautela na interpretação das pontuações de importância das variáveis. Devido a efeitos de redundância – sejam totais, como aqui estudados, ou parciais, como muitas vezes surgiriam na prática – pode acontecer que a importância total de uma determinada variável seja enganosamente baixa ou enganosamente alta porque a mesma informação está espalhada por diversas variáveis redundantes e, portanto, levada em conta várias vezes dentro das importâncias totais. Como tal, aconselhamos complementar a interpretação com uma decomposição sistemática das pontuações de importância das variáveis, por exemplo, como feito anteriormente na Figura 6.3, para melhor compreender porque é que uma variável é de f acto importante e para possivelmente detectar redundância.

7.2 viés em importâncias variáveis

Nesta seção, estudamos fontes de viés nas importâncias das variáveis e mostramos que a seleção de variáveis (conforme discutido anteriormente na Seção 6.4.3) não é a única causa do viés. Na prática, forças complementares devido a efeitos de mascaramento, estimativas i ncorretas de impurezas e estrutura das árvores fazem com que as importâncias das variáveis se desviem dos resultados teóricos encontrados em condições assintóticas para árvores totalmente aleatórias.

7.2.1 Viés devido a efeitos de mascaramento

Conforme mostrado nos capítulos anteriores, a seleção guiada da variável dividida (ou seja, para $K > 1$) é necessária para equilibrar viés e variância em árvores de decisão aleatórias e para produzir conjuntos precisos. Em particular, estudamos que árvores de decisão construídas com muita randomização geralmente levam a um aumento de viés (com relação ao erro de generalização) que não pode ser compensado por uma di minuição recíproca de variância, tornando necessário ajustar o valor de K para encontrar o trade-off apropriado. Por outro lado, também mostramos na Seção 6.4.3 que quando a seleção de variáveis não é totalmente aleatória (ou seja, assim que $K > 1$), os efeitos de mascaramento induzem um viés em relação às importâncias das variáveis, uma vez que força alguns dos ramos a nunca serem construídos e, portanto, alguns dos conjuntos de condicionamento $B \in \mathcal{P}(V)$ a nunca serem levados em conta. Como consequência, florestas aleatórias cujo parâmetro K foi ajustado para maximizar a precisão podem produzir importâncias variáveis que são tendenciosas (sobre ou subestimadas). Mais especificamente, pode ser mostrado (ver Secção 6.4.3) que uma variável relevante pode ser nula no que diz respeito à sua importância, tornando-a assim indistinguível de variáveis irrelevantes, e que a importância das variáveis relevantes torna-se dependente do número de variáveis irrelevantes.

	X_1	$\sim \mathcal{N}(0,1)$
	X_2	$\sim \mathcal{M}(2)$
	X_3	$\sim \mathcal{M}(4)$
	X_4	$\sim \mathcal{M}(10)$
	X_5	$\sim \mathcal{M}(20)$
null case	Y	$\sim \mathcal{B}(0.5)$
power case	$Y\|X_2 = 0$	$\sim \mathcal{B}(0.5 - \text{relevance})$
	$Y\|X_2 = 1$	$\sim \mathcal{B}(0.5 + \text{relevance})$

Table 7.1: Input variables are independent random variables as defined from the table. $\mathcal{N}(0,1)$ is the standard normal distribution. $\mathcal{M}(k)$ is the multinomial distribution with values in $\{0,\dots,k-1\}$ and equal probabilities. $\mathcal{B}(p)$ is the binomial distribution. In the null case, Y is independent from $X_1,\dots,X_5$. In the power case, Y depends on the value X_2 while other input variables remain irrelevant.

7.2.2 *Bias due to empirical impurity estimations*

The analysis of variable importances carried out so far has considered asymptotic conditions for which the true node impurity $i(t)$ is assumed to be known. In practice however, due to the finite size of the learning set, impurity measurements suffer from an empirical misestimation bias. In this section, we study this effect in the context of heterogeneous variables[1], with respect to their scale or their number of categories, and show that the misestimation of node impurity is directly proportional to the cardinality of the split variable and inversely proportional to the number N_t of samples used for the evaluation. As a result, impurity reductions become overestimated as we go deeper in the tree and/or as the number of values of the variable is large. In consequence, variable importances also suffer from bias, making variables of higher cardinality wrongly appear as more important.

To guide our discussion, let us revisit the simulation studies from [Strobl et al., 2007b] which consider a binary output variable Y and five input variables $X_1,\dots,X_5$, as defined in Table 7.1.

Let us first analyze the case where Y is independent from $X_1,\dots,X_5$, i.e., such that none of the input variables are informative with respect to the output. For a randomly sampled dataset $\mathcal{L}$ of $N = 120$ samples, Figure 7.3 plots variable importances for different kinds of random forests. TRT corresponds to totally randomized trees, as defined in Section 6.2, RF corresponds to standard Random Forest with bootstrap sampling and ETs corresponds to Extremely Randomized Trees.

1 As an example, in the case of meteorological problems, variables often comprise mixed environmental measurements of different nature and scale, like speed of wind, temperature, humidity, pressure, rainfall or solar radiation.

	X_1	$\sim \mathcal{N}(0,1)$
	X_2	$\sim \mathcal{M}(2)$
	X_3	$\sim \mathcal{M}(4)$
	X_4	$\sim \mathcal{M}(10)$
	X_5	$\sim \mathcal{M}(20)$
caso nulo	Y	$\sim \mathcal{B}(0.5)$
caixa de energia	$Y\|X_2 = 0$	$\sim \mathcal{B}(0.5 - \text{relevância})$
	$Y\|X_2 = 1$	$\sim \mathcal{B}(0.5 + \text{relevância})$

Tabela 7.1: Variáveis de entrada são variáveis aleatórias independentes confor me definido na tabela. $\mathcal{N}(0,1)$ é a distribuição normal padrão. $\mathcal{M}(k)$ é a distribuição multinomial com valores em $\{0,\dots,k-1\}$ e probabilidades iguais. $\mathcal{B}(p)$ é a distribuição binomial. No caso nulo, Y é independente de $X_1,\dots,X_5$. No caso de potência, Y depende do valor X_2 enquanto outras variáveis de entrada permanecem irrelevantes.

7.2.2 Viés devido a estimativas empíricas de impurezas

A análise de importâncias variáveis realizada até agora considerou condições assintóticas para as quais a verdadeira impureza do nó $i(t)$ é assumida como conhecida. Na prática, porém, devido ao tamanho finito do conjunto de aprendizagem, as medições de impurezas sofrem de um viés de estimativa errada empírica. Nesta seção, estudamos esse efeito no contexto de variáveis heterogêneas[1], com relação à sua escala ou número de categorias, e mostramos que a estimativa incorreta da impureza do nó é diretamente proporcional à cardinalidade da variável de divisão e inversamente proporcional ao número N_t de amostras usadas para a avaliação. Como resultado, as reduções de impurezas tornam-se superestimadas à medida que nos aprofundamos na árvore e/ou à medida que o número de valores da variável é grande. Consequentemente, as importâncias das variáveis também sofrem vieses, fazendo com que variáveis de maior cardinalidade pareçam erroneamente como mais importantes.

Para orientar nossa discussão, vamos revisitar os estudos de simulação de [Strobl et al., 2007b] que consideram uma variável de saída binária Y e cinco variáveis de entrada $X_1,\dots,X_5$, conforme definido na Tabela 7.1.

Analisemos primeiro o caso em que Y é independente de $X_1,\dots,X_5$, ou seja, tal que nenhuma das variáveis de entrada seja informativa em relação à saída. Para um conjunto de dados amostrados aleatoriamente $\mathcal{L}$ de $N = 120$ amostras, a Figura 7.3 representa graficamente as i mportâncias das variáveis para diferentes tipos de florestas aleatórias. TRT corresponde a árvores totalmente aleatórias, conforme definido na Seção 6.2, RF corresponde a Floresta Aleatória padrão com amostragem boot-strap e ETs corresponde a Árvores Extremamente Randomizadas.

1 Por exemplo, no caso de problemas meteorológicos, as variáveis muitas vezes compreendem medições ambientais mistas de natureza e escala diferentes, como velocidade do vento, temperatura, humidade, pressão, precipitação ou radiação solar.

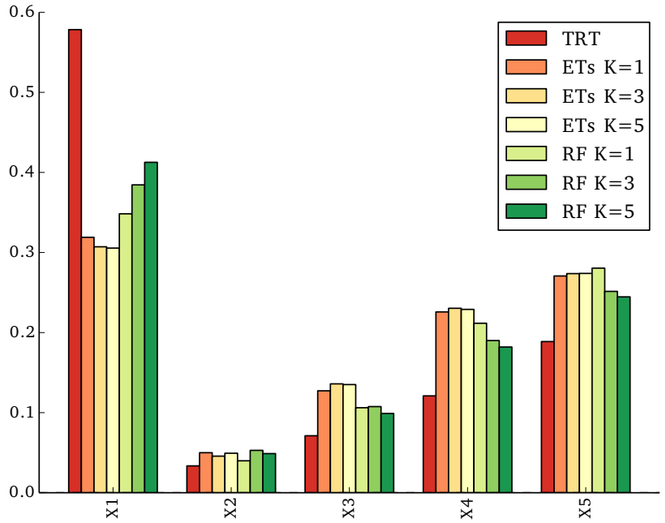


Figure 7.3: Variable importances for Y independent of $X_1, \dots, X_5$. ($N = 120$, $M = 500$)

Both RF and ETs use binary splits while TRT rely on multiway exhaustive splits[2]. In asymptotic conditions, we proved in Theorem 6.3 that the importance of irrelevant variables is strictly equal to 0. For a finite value of N however, this result does not hold, as Figure 7.3 indeed confirms. In contrast with what would be expected, we observe that the importance of none of the variables is in fact nowhere close to 0. In light of Theorem 6.2 however, this result is not that surprising: as long as decision trees can be fully developed, the sum of variable importances is equal to the (empirically estimated) mutual information between $X_1, \dots, X_p$ and the output Y, which is itself upper bounded by the (empirically estimated) entropy $H(Y)$ of the output variable. In this case, $H(Y) = \log_2(2) = 1$, which indeed corresponds to the sum of variable importances for all methods compared in the figure. More importantly, we also observe that the larger the cardinality of the variable (i.e., the larger its number of unique values), the larger its importance. For example, the importance of X_1 (for which samples all have unique values) or of X_5 (which counts up to 40 unique values) appears nearly 3 times larger as the importance of X_2 (which is binary).

In their study, Strobl et al. [2007b] argue that this bias is due to variable selection: *variables with more potential cut-points are more likely to produce a good criterion value by chance, as in a multiple testing situation*. As a result, variables of higher cardinality are more likely to be chosen for splitting than those of lower cardinality. While this bias has

2 Thereby creating as many branches as the number of values of the split variable, even if this variable is continuous and count unique values only.

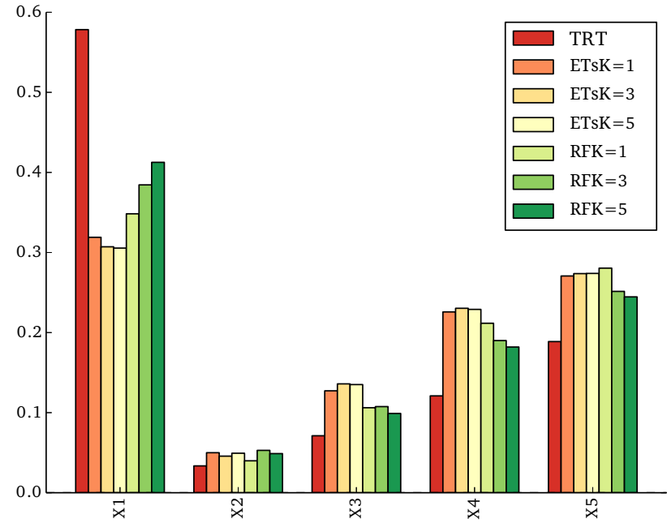


Figura 7.3: Importâncias das variáveis para Y independente de $X_1, \dots, X_5$. ($N = 120$, $M = 500$)

Tanto RF quanto ETs usam divisões binárias, enquanto o TRT depende de divisões exaustivas multivias[2]. Em condições assintóticas, provamos no Teorema 6.3 que a importância das variáveis irrelevantes é estritamente igual a 0. Para um valor finito de N no entanto, este resultado não é válido, como a Figura 7.3 de facto confirma. Em contraste com o que seria esperado, observamos que a importância de nenhuma das variáveis não está, de facto, nem perto de 0. À luz do Teorema 6.2, no entanto, este resultado não é tão surpreendente: desde que as árvores de decisão possam ser totalmente desenvolvidas, a soma das importâncias das variáveis é igual à informação mútua (estimada empiricamente) entre $X_1, \dots, X_p$ e a saída Y, que é limitada superiormente pela entropia (estimada empiricamente) $H(Y)$ da variável de saída. Neste caso, $H(Y) = \log_2(2) = 1$, que de fato corresponde à soma das importâncias das variáveis para todos os métodos comparados na figura. Mais importante ainda, observamos também que quanto maior for a cardinalidade da variável (ou seja, quanto maior for o seu número de valores únicos), maior será a sua importância. Por exemplo, a importância de X_1 (para o qual todas as amostras têm valores únicos) ou de X_5 (que conta até 40 valores únicos) parece quase 3 vezes maior que a importância de X_2 (que é binário).

Em seu estudo, Stroblet al. [2007b] argumentam que esse viés se deve à seleção de variáveis: variáveis com mais pontos de corte potenciais têm maior probabilidade de produzir um bom valor de critério por acaso, como em uma situação de testes múltiplos. Como resultado, variáveis de maior cardinalidade têm maior probabilidade de serem escolhidas para divisão do que aquelas de menor cardinalidade. Embora esse preconceito tenha

2 Criando assim tantas ramificações quanto o número de valores da variável de divisão, mesmo que esta variável seja contínua e conte apenas valores únicos.

been known for long in decision trees [Kononenko, 1995; Kim and Loh, 2001; Hothorn et al., 2006], we argue that it is not the primary cause for the bias observed here. Indeed, this argument does not directly explain why similar trends are also observed when no variable selection is performed (e.g., for TRT or for $K = 1$) nor why it similarly happens when cut-points are chosen at random, independently of the cardinality of the variable (e.g., in ETs).

For multiway splits like in TRT, bias in variable importances can be traced to the misestimations of the mutual information terms

$$\Delta i(s,t) \approx I(X_j; Y|t) \tag{7.17}$$

due to the finite size N_t of the node samples. As shown in [Goebel et al., 2005], when X_j and Y are independent random variables (i.e., when X_j is irrelevant), the distribution of finite sample size estimates of their mutual information follows approximately a gamma distribution

$$\widehat{I}(X_j; Y) \sim \Gamma\Big(\frac{1}{2}(|\mathcal{X}_j|-1)(|\mathcal{Y}|-1), \frac{1}{N_t \log 2}\Big) \tag{7.18}$$

whose mean is linearly proportional to the cardinalities $|\mathcal{X}_j|$ and $|\mathcal{Y}|$ of X_j and Y and inversely proportional to N_t, that is

$$\mathbb{E}\{\widehat{I}(X_j; Y)\} = \frac{(|\mathcal{X}_j|-1)(|\mathcal{Y}|-1)}{2N_t \log 2}. \tag{7.19}$$

As a result, estimates get larger as X_j counts many unique values, and become even larger as nodes are deep in the tree (since N_t gets smaller). Consequently, the weighted mean of all such estimated impurity terms $I(X_j; Y|t)$, for all nodes t where X_j is the split variable, and resulting in the total importance $\text{Imp}(X_j)$, is also linearly dependent on the cardinality of X_j. For TRT, this result explains why variables of high cardinality in Figure 7.3 appear as more important than those of lower cardinality. Intuitively, the closer the number of unique values with respect to the total number of samples, the larger the impurity decrease when splitting exhaustively on this variable. In the extreme case, when values for X_j are all unique, splitting on the variable perfectly memorizes the values of Y, resulting in child nodes that are all pure, therefore maximizing the estimated mutual information. As such, this explains why X_1, whose values are all unique, appears as the most important variable.

For binary splits (i.e., for RF and ETs) the mutual information $I(X_j; Y|t)$ is not directly estimated at each node. Rather, in the case of ordered variables, $\Delta i(s,t)$ corresponds to an estimate of the mutual information $I(X_j \leqslant v; Y|t)$ of the binary split $X_j \leqslant v$. Under the simplifying assumption that binary splits on the same variable are all directly consecutive in the decision tree, it is easy to see that binary trees are equivalent to multiway trees [Knuth, 1968], as illustrated in Figure 7.4.

é conhecido há muito tempo em árvores de decisão [Kononenko, 1995; Kim e Loh, 2001; Hothorn et al., 2006], argumentamos que essa não é a principal causa do viés observado aqui. Na verdade, este argumento não explica diretamente por que tendências semelhantes também são observadas quando nenhuma seleção de variável é realizada (por exemplo, para TRT ou para $K = 1$) nem por que isso acontece de forma semelhante quando os pontos de corte são escolhidos aleatoriamente, independentemente da cardinalidade da variável (por exemplo, em ETs).

Para divisões multivias como no TRT, o viés nas importâncias das variáveis pode ser atribuído às estimativas incorretas dos termos de informação mútua

$$\Delta i(s,t) \approx I(X_j; Y|t) \tag{7.17}$$

devido ao tamanho finito N_t das amostras de nós. Conforme mostrado em [Goebel et al., 2005], quando X_j e Y são variáveis aleatórias independentes (ou seja, quando X_j é irrelevante), a distribuição de estimativas de tamanho de amostra finito de suas informações mútuas segue aproximadamente uma distribuição gama.

$$\widehat{I}(X_j; Y) \sim \Gamma\Big(\frac{1}{2}(|\mathcal{X}_j|-1)(|\mathcal{Y}|-1), \frac{1}{N_t \log 2}\Big) \tag{7.18}$$

cuja média é linearmente proporcional às cardinalidades $|\mathcal{X}_j|$ e $|\mathcal{Y}|$ deX_j e Y e inversamente proporcional a N_t, ou seja

$$\mathbb{E}\{\widehat{I}(X_j; Y)\} = \frac{(|\mathcal{X}_j|-1)(|\mathcal{Y}|-1)}{2N_t \log 2}. \tag{7.19}$$

Como resultado, as estimativas ficam maiores à medida que X_j contam muitos valores exclusivos e ficam ainda maiores à medida que os nós estã o mais profundos na árvore (já que N_t fica menor). Consequentemente, a média ponderada de todos esses termos de impureza estimados $I(X_j; Y|t)$, para todos os nós t onde X_j é a variável de divisão, e resultando na importância total $\text{Imp}(X_j)$, também é linearmente dependente da cardinalidade de X_j. Para o TRT, este resultado explica porque as variáveis de alta cardinalidade na Figura 7.3 aparecem como mais importantes do que aquelas de menor cardinalidade. Intuitivamente, quanto mais próximo o número de valores únicos em relação ao número total de amostras, maior será a diminuição da impureza ao dividir exaustiva mente nesta variável. No caso extremo, quando os valores de X_j são todos únicos, a divisão na variável memoriza perfeitamente os valores de Y, resultando em nós filhos que são todos puros, maximizando assim a informação mútua estimada. Como tal, isto explica porque X_1, cujos valores são todos únicos, aparece como a variável mais importante.

Para divisões binárias (isto é, para RF e ETs) a informação mútua $I(X_j; Y|t)$ não é estimada diretamente em cada nó. Pelo contrário, no caso de variáveis ordenadas, $\Delta i(s,t)$ corresponde a uma estimativa da informação mútua $I(X_j \leqslant v; Y|t)$ da divisão binária $X_j \leqslant v$. Sob a suposição simplificadora de que as divisões binárias na mesma variável são todas diretamente consecutivas na árvore de decisão, é fácil ver que as árvores binárias são equivalentes às árvores multivias [Knuth, 1968], conforme ilustrado na Figura 7.4.

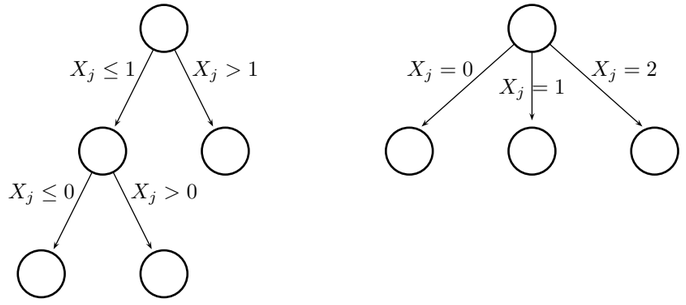


Figure 7.4: Consecutive binary splits on the same variable are equivalent to direct multiway splits.

Using an argument similar to the proof of Theorem 6.2, intermediate impurity terms between the first split and the last of those splits cancel each other when summing up the importances, which finally amounts to collect an actual estimate of $I(X_j; Y|t)$ from the sequence of binary splits. For the same reasons as before, variables of high cardinality are therefore biased in the same way. (As we will study in Section 7.2.3, this consecutiveness assumption does not hold in practice, making the importances from binary decision trees strictly different from those of multiway decision trees. Yet, the qualitative conclusions are still valid since variables of high cardinality can be reused far more often than variables of low cardinality, before all possible splits are exhausted.)

In both situations, the origin of the problem stems from the fact that node impurity is misestimated when the size N_t of the node samples is too small. To a large extent, the issue is aggravated by the fact that trees are fully developed by default, making impurity terms collected near the leaves usually unreliable. As a precaution, a safe and effective solution for the bias problem therefore simply consists in collecting impurity terms only for those nodes where the impurity estimates can be considered as reliable. Equivalently, the construction of the tree can also be stopped early, when impurity estimates become unreliable, e.g., by limiting the depth of the tree, controlling for the minimum number of samples in internal nodes or using any of the other stopping criteria defined in Section 3.5. Among all alternatives, conditional inference trees [Hothorn et al., 2006] and earlier methods [Quinlan, 1986; Wehenkel, 1998] make use of statistical tests for assessing the independence of X_j and Y at a pre-specified level of confidence α. If the null hypothesis cannot be rejected, then recursive partitioning halts. In particular, variable importances collected from conditional inference trees were shown experimentally by Strobl et al. [2007b] not to suffer from bias. The authors argue that it is due to the unbiased variable selection mechanisms also implemented in conditional inference trees. By contrast, we argue that the absence of bias

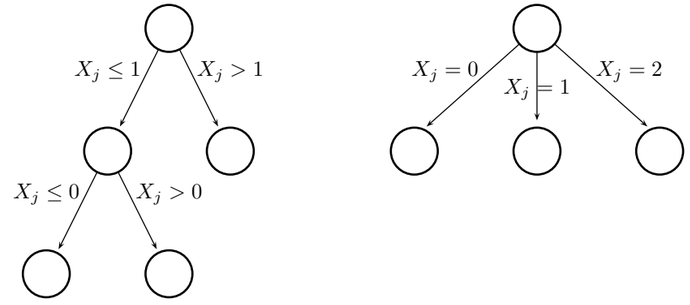


Figura 7.4:Divisões binárias consecutivas na mesma variável são equivalentes a divisões multivias diretas.

Usando um argumento semelhante à prova do Teorema 6.2, os termos de impureza intermediários entre a primeira divisão e a última dessas divisões cancelam-se mutuamente ao somar as importâncias, o que finalmente equivale a coletar uma estimativa real de $I(X_j; Y|t)$ da sequência de divisões binárias. Pelas mesmas razões de antes, variáveis de alta cardinalidade são, portanto, tendenciosas da mesma forma. (Como estudaremos na Seção 7.2.3, esta suposição de consecutividade não é válida na prática, tornando as importâncias das árvores de decisão binárias estritamente diferentes daquelas das árvores de decisão multivias. No entanto, as conclusões qualitativas ainda são válidas, uma vez que variáveis de alta cardinalidade podem ser reutilizadas com muito mais frequência do que variáveis de baixa cardinalidade, antes que todas as divisões possíveis sejam esgotadas.)

Em ambas as situações, a origem do problema decorre do fato de que a impureza do nó é mal estimada quando o tamanho N_t das amostras do nó é muito pequeno. Em grande medida, o problema é agravado pelo facto de as árvores estarem totalmente desenvolvidas por defeito, fazendo com que os termos de impureza recolhidos perto das folhas normalmente não sejam fiáveis. Como precaução, uma solução segura e eficaz para o problema do viés consiste simplesmente em coletar termos de impureza apenas para os nós onde as estimativas de impureza podem ser consideradas confiáveis. De forma equivalente, a construção da árvore também pode ser interrompida antecipadamente, quando as estimativas de impurezas se tornam não confiáveis, por exemplo, limitando a profundidade da árvore, controlando o número mínimo de amostras em nós internos ou usando qualquer outro critério de parada definido na Seção 3.5. Entre todas as alternativas, árvores de inferência condicional [Hothorn et al., 2006] e métodos anteriores [Quinlan, 1986; Wehenkel, 1998] faz uso de testes estatísticos para avaliar a independência de X_j e Y em um nível de confiança pré-especificado α. Se a hipótese nula não puder ser rejeitada, o particionamento recursivo será interrompido. Em particular, as importâncias variáveis coletadas de árvores de inferência condicional foram demonstradas experimentalmente por Strobl et al.[2007b] para não sofrerem de viés. Os autores argumentam que isso se deve aos mecanismos imparciais de seleção de variáveis também implementados em árvores de inferência condicional. Por outro lado, argumentamos que a ausência de preconceito

in importances from such trees is mostly due to the early stopping criterion, and not to variable selection. Although variable selection plays an important and exacerbating role, it is not the true cause of the observed bias. Indeed, since the impurity reduction of variables of high cardinality is overestimated, searching for a split among $K > 1$ randomly drawn variables increases their probability of being selected when compared to others of lower cardinality, therefore masking and reducing the importances of these latter variables. Yet, bias stems from the fact that impurity reductions are misestimated in the first place.

As an illustrative example, Figure 7.5 reconsiders the simulation study of Table 7.1, when varying both the maximum depth of the trees (from `max_depth` $= 1$ to 9) and the relevance of X_2 with respect to Y. Let us first consider ETs built with no variable selection ($K = 1$), as shown in four top plots of the figure. For the null case, when relevance $= 0$, limiting the depth of the trees correctly fixes the bias that was observed before. For shallow trees, the importances before normalization of all five input variables are close to 0, as expected from irrelevant variables. The normalized importances, as shown in the figure, are also all close to $\frac{1}{p} = 0.2$, confirming that no variable is detected as more relevant than the others. However, when the depth of the decision trees increases, importances deviate and bias proportional to the cardinality of the variables appears as discussed before. When X_2 is a relevant variable (i.e., for relevance > 0), its importance is expected to be strictly positive and at least as large as the importances of the irrelevant variables. For relevance $= 0.1$ and `max_depth=1`, the importance of X_2 appears nearly 6 times larger than the importances of the other variables, confirming that X_2 is correctly identified as a relevant variable. For deeper trees however, noise dominates and the importance of the irrelevant variables is larger due to misestimations of the impurity terms. As relevance increases, X_2 can more clearly be identified as a relevant variable. In particular, the more relevant X_2, that is the stronger the signal, the deeper trees can be built until X_2 is made unrecognizable from the irrelevant variables.

By comparison, the four bottom plots of Figure 7.5 illustrate variable importances for RF built with variable selection ($K = 5$). Again, we observe that limiting the depth of the trees helps reduce the misestimation bias. However, the supplementary effect due variable selection is also clearly visible: variables of larger cardinality appear as significantly more important than the other variables. Consequently, this makes the detection of X_2 as a relevant variable more difficult when trees are grown deeper. When comparing the relative importance of X_2 for a given depth and relevance, we indeed observe that X_2 consistently appears as less important in RF than in ETs. It is only for very shallow trees (`max_depth=1` or 2) and high relevance that X_2 is identified with higher confidence than in ETs.

nas importâncias de tais árvores se deve principalmente ao critério de parada antecipada, e não à seleção de variáveis. Embora a seleção de variáveis desempenhe um papel importante e exacerbador, não é a verdadeira causa do viés observado. Na verdade, como a redução de impureza de variáveis de alta cardinalidade é superestimada, a busca por uma divisão entre$K > 1$ variáveis sorteadas aleatoriamente aumenta sua probabilidade de serem selecionadas quando comparadas a outras de menor cardinalidade, mascarando e reduzindo a importância destas últimas variáveis. No entanto, o preconceito decorre do facto de as reduções de impurezas serem mal estimadas, em primeiro lugar.

A título de exemplo ilustrativo, a Figura 7.5 reconsidera o estudo de simulação da Tabela 7.1, ao variar tanto a profundidade máxima das árvores (de max_profundidade $= 1$ a 9) quanto a relevância de X_2 em relação a Y. Consideremos primeiro ETs construídos sem seleção de variáveis ($K = 1$), conforme mostrado nos quatro gráficos superiores da figura. Para o caso nulo, quando relevante $= 0$, limitar a profundidade das árvores corrige corretamente o viés observado anteriormente. Para árvores rasas, as importâncias antes da normalização de todas as cinco variáveis de entrada são próximas de 0, como esperado de variáveis irrelevantes. As importâncias normalizadas, conforme mostrado na figura, também estão todas próximas de $\frac{1}{p} = 0.2$, confirmando que nenhuma variável é detectada como mais relevante que as outras. Porém, quando a profundidade das árvores de decisão aumenta, as importâncias se desvia m e o viés proporcional à cardinalidade das variáveis aparece conforme discutido anteriormente. Quando X_2 é uma variável relevante (ou seja, para relevância > 0), espera-se que sua importância seja estritamente positiva e pelo menos tão grande quanto as importâncias das variáveis irrelevantes. Para relevância $= 0.1$ e profundidade máxima=1, a importância de X_2 aparece quase 6 vezes maior que as importâncias das outras variáveis, confirmando que X_2 está corretamente identificado co mo uma variável relevante. Contudo, para árvores mais profundas, o ruído domina e a importância das variáveis irrelevantes é maior devido a estimativas erradas dos termos de impureza. À medida que a relevância aumenta, X_2 pode ser identificado mais claramente como uma variável relevante. Em particular, quanto mais relevante X_2, ou seja, quanto mais forte o sinal, mais profundas as árvores podem ser construídas até que X_2 se torne irreconhecível a partir das variáveis irrelevantes.

Em comparação, os quatro gráficos inferiores da Figura 7.5 ilustram as importâncias das variáveis para RF construído com seleção de variáveis ($K = 5$). Novamente, observamos que limitar a profundidade das árvores ajuda a reduzir o viés de estimativa incorreta. No entanto, o efeito suplementar devido à seleção de variáveis também é claramente visível: variáveis de maior cardinalidade aparecem como significativamente mais importantes do que as outras variáveis. Conseqüentemente, isso torna a detecção de X_2 como uma variável relevante mais difícil quando as árvores crescem mais profundamente. Ao comparar a importância relativa de X_2 para uma dada profundidade e relevância, observamos de fato que X_2 parece consistentemente como menos importante emRF do que em inETs. É apenas para árvores muito rasas (profundidade máxima=1 ou 2) e de alta relevância que X_2é identificado com maior confiança do que em ETs.

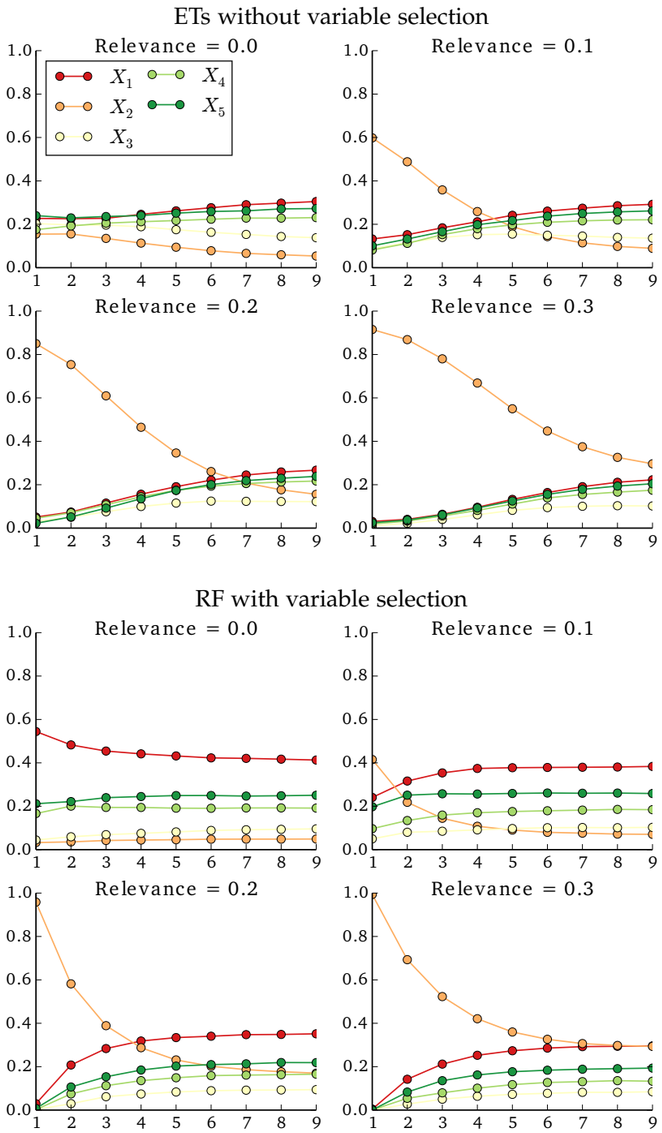


Figure 7.5: Variable importances of $X_1, \ldots, X_5$ when varying both the maximum depth of the trees and the degree of relevance of X_2. Importance scores are normalized by the sum of importances. (Top) ETs with no variable selection, K = 1, N = 120, M = 500. (Bottom) Random Forest with variable selection, K = 5, N = 120, M = 500.

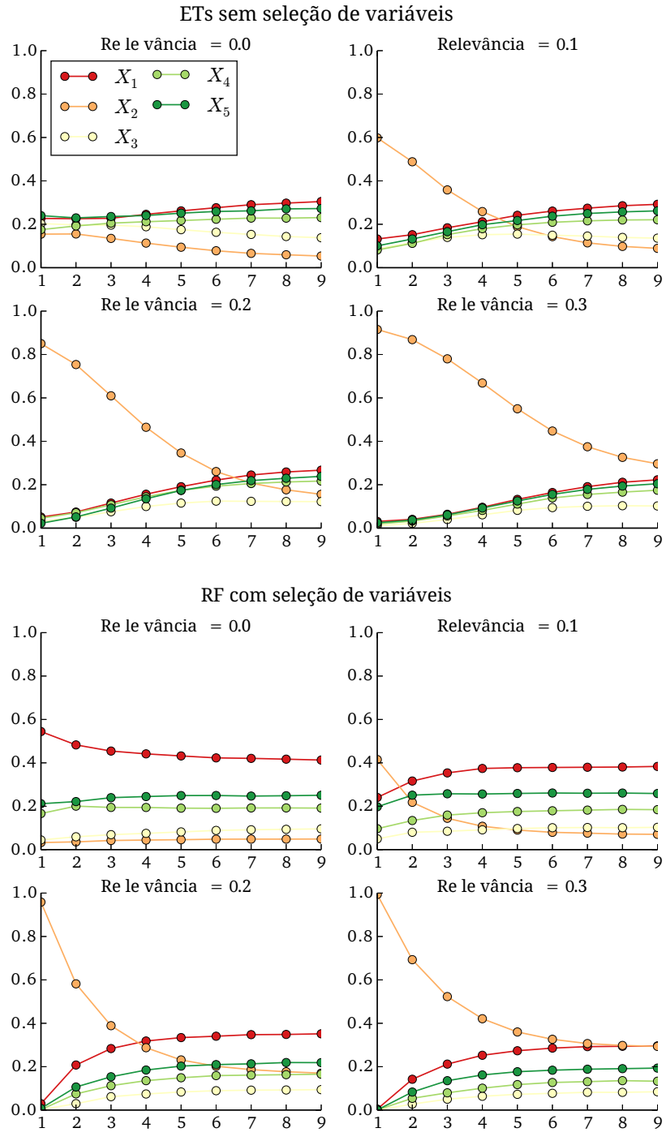


Figura 7.5: Importâncias das variáveis de $X_1, \ldots, X_5$ ao variar tanto a profundidade máxima das árvores quanto o grau de relevância de X_2. As pontuações de importância são normalizadas pela soma das importâncias. (Topo) ETs sem seleção de variáveis, K = 1, N = 120, M = 500. (Inferior) Floresta Aleatória com seleção de variáveis, K = 5, N = 120,M = 500.

In conclusion, evaluating impurity on small samples lead to over-estimations of the mutual information, resulting in biased variable importances. In particular, the higher the cardinality of the variable, the larger the misestimations. To minimize this effect, caution should be taken by only considering impurity terms that were computed from a large enough sample. This can be guaranteed, e.g., by stopping the construction of the tree early or making leaves grow more slowly than the size N of the learning set. Additionally, we have also shown that variable selection may increase the bias due to over-estimations. In this case, a simple remedy consists in not using variable selection when assessing the relevance of variables. Finally, let us also note the connection with Propositions 6.6 and 6.7: as long as trees are built to a maximum depth which is larger than the number r of relevant variables, early stopping the construction of the trees does not prevent us from detecting the relevant variables.

7.2.3 *Bias due to binary trees and threshold selection*

Previous developments from Chapter 6 studied variable importances for fully developed totally randomized trees and multiway exhaustive splits. In practice however, random forests usually rely on binary splits rather than on multiway splits. In terms of impurity, this results in additional and distinct information terms that were not previously accounted for, because i) a same variable can be reused several times along the same branch and ii) binary splits discretize the information contained in a variable, making variable importances dependent on the split threshold selection strategy. In lack of a rigorous theoretical framework, we give in this section preliminary insights on variable importances for binary decision trees, hopefully shedding some light on their interpretation.

As a simplistic but illustrative example, let us consider a toy classification problem composed of a ternary input variable X_1 and of a binary input variable X_2, both of them being ordered. Let us further assume that input samples are uniformly drawn from Table 7.2, which defines the output as $Y = X_1 < 1$ and X_2 as a copy of Y. With respect to Y, both variables are as informative and one would expect their importances to be the same. With totally randomized trees and exhaustive splits, two equiprobable decision trees can be built, as represented in Figure 7.6. In both of them, splitting either on X_1 or on X_2 at the root node results in child nodes that are pure, hence halting the construction process. As expected, the measured variable importances are the same:

$$\text{Imp}(X_1) = \frac{1}{2} I(X_1; Y) = \frac{1}{2} H(Y) = 0.459,$$
$$\text{Imp}(X_2) = \frac{1}{2} I(X_2; Y) = \frac{1}{2} H(Y) = 0.459.$$

Concluindo, a avaliação da impureza em amostras pequenas leva a superestimações da informação mútua, resultando em importâncias tendenciosas das variáveis. Em particular, quanto maior a cardinalidade da variável, maiores serão as estimativas erradas. Para minimizar esse efeito, deve-se tomar cuidado ao considerar apenas os termos de i mpureza que foram calculados a partir de uma amostra grande o suficiente. Isso pode ser garantido, por exemplo, interrompendo a construção da árvore mais cedo ou fazendo as folhas crescerem mais lentamente do que o tamanho N do conjunto de aprendizagem. Além disso, também mostramos que a seleção de variáveis pode aumentar o viés devido a superestimações. Neste caso, uma solução simples consiste em não utilizar a seleção de variáveis ao avaliar a relevância das variáveis. Finalmente, notemos também a ligação com as Proposições 6.6 e 6.7: desde que as árvores sejam construídas a uma profundidade máxima que seja maior que o número r de variáveis relevantes, parar precocemente a construção das árvores não nos impede de detectar as variáveis r elevantes.

7.2.3 Bias devido a árvores binárias e seleção de limite

Desenvolvimentos anteriores do Capítulo 6 estudaram importâncias variáveis para árvores totalmente randomizadas e divisões exaustivas multivias. Na prática, entretanto, florestas aleatórias geralmente dependem de divisões binárias em vez de divisões multivias. Em termos de impureza, isto resulta em termos de informação adicionais e distintos que não foram previamente contabilizados, porque i) uma mesma variável pode ser reutilizada várias vezes ao longo do mesmo ramo e ii) as divisões binárias discretizam a informação contida numa variável, tornando as importâncias das variáveis dependentes da estratégia de selecção do limite de divisão. Na falta de uma estrutura teórica rigorosa, fornecemos nesta seção insights preliminares sobre importâncias variáveis para árvores de decisão binárias, esperançosamente lançando alguma luz sobre sua interpretação.

Como exemplo simplista, mas ilustrativo, consideremos um problema de classificação de brinquedos composto por uma variável de entrada ternária X_1 e por uma variável de entrada binária X_2, ambas ordenadas. Suponhamos ainda que as amostras de entrada sejam extraídas uniformemente da Tabela 7.2, que define a saída como $Y = X_1 < 1$ e X_2 como uma cópia de Y. Com relação a Y, ambas as variáveis são i gualmente informativas e seria de esperar que suas importâncias fossem as mesmas. Com árvores totalmente aleatórias e divisões exaustivas, duas árvores de decisão equiprováveis podem ser construídas, conforme representado na Figura 7.6. Em ambos, a divisão em X_1 ou emX_2 no nó raiz resulta em nós filhos que são puros, interrompendo assim o processo de construção. Como esperado, as importâncias das variáveis medidas são as mesmas:

$$\text{Imp}(X_1) = \frac{1}{2} I(X_1; Y) = \frac{1}{2} H(Y) = 0.459,$$
$$\text{Imp}(X_2) = \frac{1}{2} I(X_2; Y) = \frac{1}{2} H(Y) = 0.459.$$

y	X_1	X_2
0	0	0
1	1	1
1	2	1

Table 7.2: Toy problem. All 3 possible samples are equiprobable.

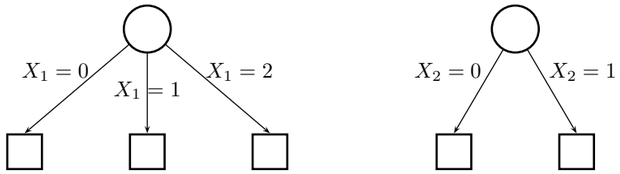


Figure 7.6: Totally randomized trees built from Table 7.2. Both decision trees are equiprobable. The resulting variable importances indicate that both X_1 and X_2 are as important.

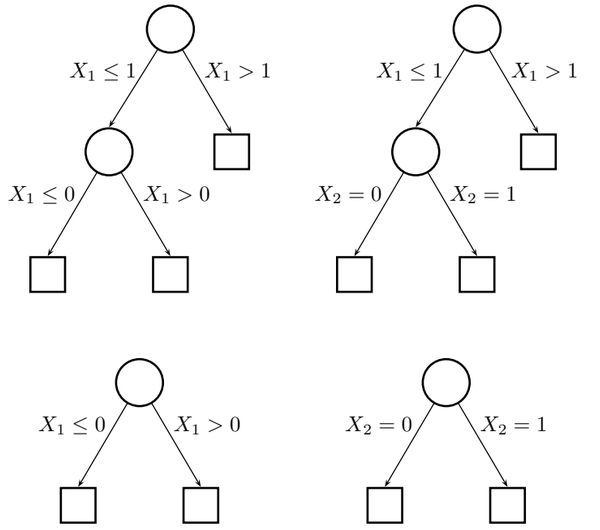


Figure 7.7: Extremely randomized trees ($K = 1$) built from Table 7.2. From left to right, top to bottom, decision trees are respectively generated with probability $\frac{1}{8}$, $\frac{1}{8}$, $\frac{1}{4}$ and $\frac{1}{2}$. The resulting variable importances indicate that X_2 is now more important than X_1.

y	X_1	X_2
0	0	0
1	1	1
1	2	1

Tabela 7.2: Problema do brinquedo. Todas as 3 amostras possíveis são equiprováveis.

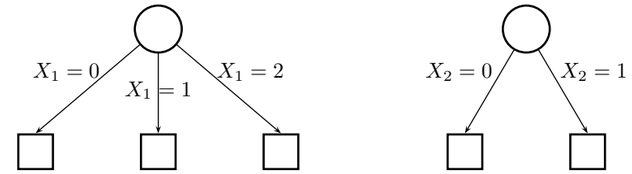


Figura 7.6: Árvores totalmente randomizadas construídas a partir da Tabela 7.2. Ambas as árvores de decisão são equiprováveis. As importâncias das variáveis resultantes indicam que X_1 e X_2 são igualmente importantes.

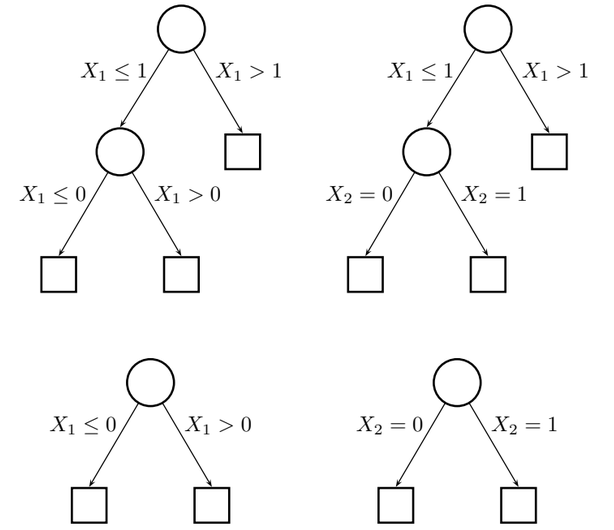


Figura 7.7: Árvores extremamente aleatórias ($K = 1$) construídas a partir da Tabela 7.2. Da esquerda para a direita, de cima para baixo, são geradas árvores de decisão respectivamente com probabilidade 18, 18, 14 e 12. As importâncias das variáveis resultantes indicam que X_2 agora é mais importante que X_1.

By contrast, using binary splits and ETs (for $K = 1$) results in 4 different decision trees, as represented in Figure 7.7. When splitting on X_1 at the root node, two binary splits are now possible with equal probability: $t_L = X_1 \leqslant 0, t_R = X_1 > 0$ or $t_L = X_1 \leqslant 1, t_R = X_1 > 1$. In the former case, the resulting child nodes are pure, hence halting the construction process. In the latter case, the right child (corresponding to $X_1 > 1$) is pure, while the left child is not. For this node, recursive partitioning proceeds and a second binary split can be made either on X_1 or on X_2. Overall, the measured variable importances in these binary trees, in asymptotic conditions, are

$$\begin{aligned}
\text{Imp}(X_1) &= \frac{2}{8} I(X_1 \leqslant 1; Y) + \frac{1}{8} P(X_1 \leqslant 1) I(X_1 \leqslant 0; Y | X_1 \leqslant 1) + \frac{1}{4} I(X_1 \leqslant 0; Y) \\
&= 0.375 \\
\text{Imp}(X_2) &= \frac{1}{2} I(X_2; Y) + \frac{1}{8} P(X_1 \leqslant 1) I(X_2; Y | X_1 \leqslant 1) \\
&= 0.541,
\end{aligned}$$

which makes them strictly different from the importances collected from multiway totally randomized trees. In particular, due to the binarization of the split variables, importances now account for conditioning sets that may include several values of a same variable. For instance, the importance of X_2 includes $I(X_2; Y | X_1 \leqslant 1)$, which measures the mutual information between X_2 and Y when X_1 is either equal to 0 or 1. With multiway exhaustive splits, these conditioning sets are not considered because branches correspond to single values only. Similarly, importances also account for binarized mutual information terms such as $I(X_1 \leqslant 1; Y)$. Again, these are not evaluated in totally randomized trees because of the multiway splits.

Accordingly, threshold selection in binary splits has a dominant effect on variable importances since it controls how the original variables are binarized. In ETs, all intermediate thresholds v are possible, resulting in a combinatorial number of additional impurity terms $I(X_j \leqslant v; Y | \cdot)$ and $I(\cdot; Y | X_j \leqslant v, \cdot)$. In RF, only the best threshold is selected locally at each node, resulting in fewer additional impurity terms $I(X_j < v^*; Y | \cdot)$ and $I(\cdot; Y | X_j < v^*, \cdot)$ (and masking all others).

As a last example, Figure 7.8 illustrates variable importances for X_1 and X_2 when increasing the cardinality $L = |\mathcal{X}_1|$ of X_1 on the previous toy problem. That is, let us redefine the output as $Y = X_1 < \frac{L}{2}$, for $\mathcal{X}_1 = \{0, \dots, L-1\}$, while keeping X_2 as a binary variable defined as a copy of Y. Assuming that all input samples are equiprobable, the importances yielded by totally randomized trees with multiway splits remain the same as before. For ETs however, increasing L induces even more new impurity terms that are now accounted for in the importances. As the figure shows, increasing the cardinality of X_1 makes its importance decrease, but also simultaneously makes the importance of X_2 increase. Indeed, splitting on X_2 always yield

Por outro lado, o uso de divisões binárias e ETs (para $K = 1$) resulta em 4 árvores de decisão diferentes, conforme representado na Figura 7.7. Ao dividir em X_1 no nó raiz, duas divisões binárias agora são possíveis com igual probabilidade: $t_L = X_1 \leqslant 0, t_R = X_1 > 0$ ou $t_L = X_1 \leqslant 1$, $t_R = X_1 > 1$. No primeiro caso, os nós filhos resultantes são puros, interrompendo assim o processo de construção. Neste último caso, o filho direito (correspondente a $X_1 > 1$) é puro, enquanto o filho esquerdo não o é. Para este nó, o particionamento recursivo prossegue e uma segunda divisão binária pode ser feita em X_1 ou em X_2. No geral, as importâncias das variáveis medidas nestas árvores binárias, em condições assintóticas, são

$$\begin{aligned}
\text{Imp}(X_1) &= \frac{2}{8} I(X_1 \leqslant 1; Y) + \frac{1}{8} P(X_1 \leqslant 1) I(X_1 \leqslant 0; Y | X_1 \leqslant 1) + \frac{1}{4} I(X_1 \leqslant 0; Y) \\
&= 0.375 \\
\text{Imp}(X_2) &= \frac{1}{2} I(X_2; Y) + \frac{1}{8} P(X_1 \leqslant 1) I(X_2; Y | X_1 \leqslant 1) \\
&= 0.541,
\end{aligned}$$

o que os torna estritamente diferentes das importâncias coletadas de árvores multiway totalmente randomizadas. Em particular, devido à binarização das variáveis divididas, as importâncias agora levam em co nta conjuntos de condicionamento que podem incluir vários valores de uma mesma variável. Por exemplo, a importância de X_2 inclui $I(X_2; Y | X_1 \leqslant 1)$, que mede a informação mútua entre X_2 e Y quando X_1 é ig ual a 0 ou 1. Com divisões exaustivas multivias, esses conjuntos de condicionamento não são considerados porque os ramos correspondem apenas a valores únicos. Da mesma forma, as importâncias também leva m em conta termos binarizados de informação mútua, como $I(X_1 \leqslant 1; Y)$. Novamente, estes não são avaliados em árvores totalmente aleatórias devido às divisões multiway.

Conseqüentemente, a seleção de limiares em divisões binárias tem u m efeito dominante nas importâncias das variáveis, uma vez que controla como as variáveis originais são binarizadas. Nos ETs, todos os limites intermediários v são possíveis, resultando em um número combinatório de termos de impureza adicionais$I(X_j \leqslant v; Y | \cdot)$ e $I(\cdot; Y | X_j \leqslant v, \cdot)$. Na RF, apenas o melhor limite é selecionado localmente em cada nó, resultando em menos termos de impureza adicionais $I(X_j < v^*; Y | \cdot)$ e $I(\cdot; Y | X_j < v^*, \cdot)$ (e mascarando todos os outros).

Como último exemplo, a Figura 7.8 ilustra as importâncias das variáveis para X_1e X_2 ao aumentar a cardinalidade $L = |\mathcal{X}_1|$ de X_1 no problema do brinquedo anterior. Ou seja, vamos redefinir a saída como $Y = X_1 < \frac{L}{2}$, para $\mathcal{X}_1 = \{0, \dots, L-1\}$, mantendo X_2 como uma variável binária definida como uma cópia de Y. Assumindo que todas as amostras de entrada são equiprováveis, as importâncias produzidas por árvores totalmente aleatórias com divisões multiway permanecem as mesmas de antes. Para os ETs, no entanto, o aumento de L induz ainda mais novos termos de impureza que agora são contabilizados nas importâncias. Co mo mostra a figura, aumentar a cardinalidade deX_1 faz com que sua importância diminua, mas também faz simultaneamente com que a importância de X_2 aumente. Na verdade, dividir X_2 sempre rende

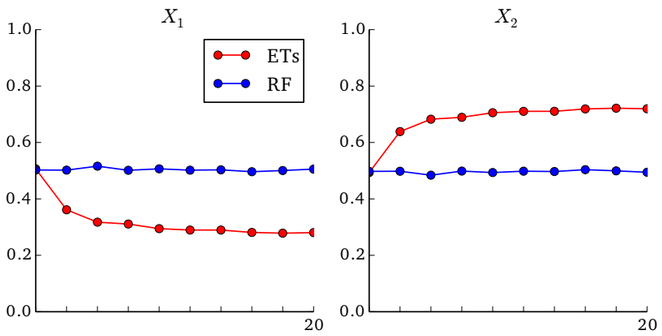


Figure 7.8: Variable importances of X_1 and X_2 when increasing the cardinality of X_1.

child nodes that are pure, which is unlikely to be the case when splitting randomly on X_1. For RF, only the best thresholds are selected, yielding in this case child nodes that are always pure, as for totally randomized trees. As such, their importances respectively appear as equal as the figure confirms.

Overall, these additional effects due to binary splits make variable importances computed from classical random forests very difficult to interpret and understand, as soon as data include many variables of different number of categories. While they can still be used to identify the most relevant variables, caution should be taken when interpreting the amplitude of importances. As our last example illustrates, they may be misleadingly low or high because of combinatorial effects, solely due to the possible ways variables are binarized through the implemented threshold selection mechanisms.

Encoding L-ary variables into binary variables

Partitioning node samples with binary splits on L-ary input variables is equivalent to individually transforming each input variable X_j into a set of $L_j - 1$ binary input variables $\{X_j^l | l = 1, \dots, L_j - 1\}$, each encoding for one the possible splits, and then partitioning node samples using of one these new variables. If we consider the binarized learning set $\mathcal{L}'$, for which all p the input variables have been transformed into $\sum_j (L_j - 1)$ binary variables, then our theoretical framework for totally randomized trees and exhaustive splits can be applied and Theorem 6.1 could possibly be adapted. The only difference lies in the way binary variables are drawn at random: instead of splitting randomly on one of the $\sum_j (L_j - 1)$ variables, first, one of the p original variables is drawn uniformly at random; second, one of its $L_j - 1$ binary variables is selected for splitting the current node. In this setting, the importance of X_j therefore amounts to the sum of importances $\sum_l \text{Imp}(X_j^l)$ of its binary variables.

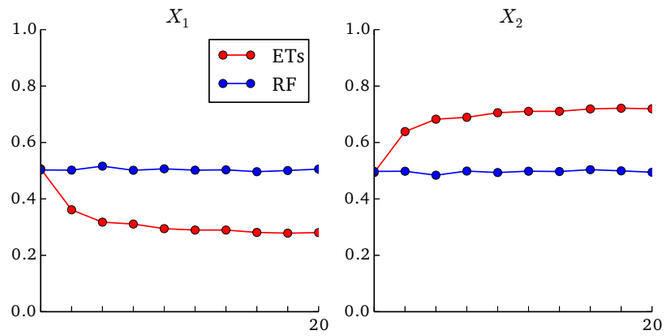


Figura 7.8: Importâncias das variáveis X_1 e X_2 ao aumentar a cardinalidade de X_1.

nós filhos que são puros, o que é improvável que seja o caso quando dividido aleatoriamente em X_1. Para RF, apenas os melhores limites são selecionados, produzindo neste caso nós filhos que são sempre puros, como para árvores totalmente aleatórias. Como tal, as suas importâncias aparecem respectivamente tão iguais como a figura confirma.

No geral, esses efeitos adicionais devido às divisões binárias tornam as importâncias das variáveis calculadas a partir de florestas aleatórias clássicas muito difíceis de interpretar e compreender, assim que os dados incluem muitas variáveis de diferentes números de categorias. Embora ainda possam ser usados para identificar as variáveis mais relevantes, deve-se ter cautela na interpretação da amplitude das importâncias. Co mo ilustra o nosso último exemplo, eles podem ser enganosamente baixos ou altos devido a efeitos combinatórios, unicamente devido às possíveis formas como as variáveis são binarizadas através dos mecanismos de seleção de limiares implementados.

Codificando variáveis L-árias em variáveis binárias

Particionar amostras de nós com divisões binárias em variáveis de e ntrada L-árias é equivalente a transformar individualmente cada variável de entrada X_j em um conjunto de $L_j - 1$ variáveis de entrada binárias $\{X_j^l | l = 1, \dots, L_j - 1\}$, cada uma codificando para uma das divisões possíveis, e então particionar amostras de nós usando uma dessas novas variáveis. Se considerarmos o conjunto de aprendizagem binarizado $\mathcal{L}'$, para o qual todas p as variáveis de entrada foram transformadas em $\sum_j (L_j - 1)$ variáveis binárias, então nossa estrutura teórica para árvores totalmente aleatórias e divisões exaustivas pode ser aplicada e o Teorema 6.1 poderia possivelmente ser adaptado. A única diferença está na forma como as variáveis binárias são desenhadas aleatoriamente: em vez de dividir aleatoriamente uma das variáveis $\sum_j (L_j - 1)$, primeiro, uma das variáveis p originais é desenhada uniformemente de forma aleatória; segundo, uma de suas variáveis binárias $L_j - 1$é selecionada para dividir o nó atual. Nesta configuração, a importância de X_j equivale, portanto, à soma das importâncias $\sum_l \text{Imp}(X_j^l)$ de suas variáveis binárias.

7.3 APPLICATIONS

In spite of the various concurrent effects discussed earlier, variable importance measures have been used with success in a wide range of scientific applications. While they have proven to be a good proxy for assessing the relevance of input variables, providing helpful insights, too often variable importances are used as a black-box metric, hence under-exploiting the information they offer. As such, examples presented in this section all demonstrate that any progress towards a better theoretical understanding of variable importances may help to further advance a wide range of research domains.

7.3.1 *Feature selection*

Within the past 10 or 20 years, typical machine learning problems have grown from a few tens of input variables to domains exploring hundreds of thousands of variables, often comprising noisy, irrelevant or redundant predictors, mixing both numerical and categorical variables and involving complex interaction effects. In this context, the feature selection problem consists in identifying a subset of the original input variables that are useful for building a good model [Guyon and Elisseeff, 2003; Liu and Yu, 2005]. The advantages and benefits of reducing the dimensionality of the problem include: speeding up machine learning algorithms, reducing measurement and storage requirements, improving the accuracy of the models or facilitating data visualization and understanding.

Because of the properties of random forests (good prediction performance, robustness to noisy variables, support of numerical and categorical variables and ability to model complex interactions), variable importances often provide an effective solution for the feature selection problem. The most straightforward solution consists in ranking variables according to their importance scores and to only keep the most important ones. Depending on the objective, the best subset of variables can be identified in several ways:

- When the goal is simply to reduce dimensionality because of speed and storage requirements, the simplest solution is to keep only those variables whose importances $\text{Imp}(X_j)$ is greater than some manually defined threshold α.

- If the goal is to improve accuracy, a good subset of variables can typically be found by tuning the threshold α so as to minimize some user-defined criterion (e.g., the zero-one loss in classification or the squared error loss in regression) for the model built on the subset $\{X_j | \text{Imp}(X_j) > \alpha\}$.

  At the price of more computational efforts, even better performance can usually be reached by embedding variable im-

7.3 aplicações

Apesar dos vários efeitos simultâneos discutidos anteriormente, medidas de importância variável têm sido utilizadas com sucesso numa ampla gama de aplicações científicas. Embora tenham provado ser um bom substituto para avaliar a relevância das variáveis de entrada, fornecendo informações úteis, muitas vezes as importâncias das variáveis são utilizadas como uma métrica de caixa preta, subexplorando assim a informação que oferecem. Como tal, todos os exemplos apresentados nesta secção demonstram que qualquer progresso no sentido de uma melhor compreensão teórica das importâncias variáveis pode ajudar a avançar ainda mais numa vasta gama de domínios de investigação.

7.3.1 Seleção de recursos

Nos últimos 10 ou 20 anos, os problemas típicos de aprendizagem automática cresceram de algumas dezenas de variáveis de entrada para domínios que exploram centenas de milhares de variáveis, muitas vezes compreendendo preditores ruidosos, irrelevantes ou redundantes, misturando variáveis numéricas e categóricas e envolvendo efeitos de interação complexos. Neste contexto, o problema de seleção de características consiste em identificar um subconjunto das variáveis de entrada originais que são úteis para a construção de um bom modelo [Guyon e Elisseeff, 2003; Liu e Yu, 2005]. As vantagens e benefícios de reduzir a dimensionalidade do problema incluem: acelerar algoritmos de aprendizado de máquina, reduzir requisitos de medição e armazenamento, melhorar a precisão dos modelos ou facilitar a visualização e compreensão dos dados.

Devido às propriedades das florestas aleatórias (bom desempenho de previsão, robustez a variáveis ruidosas, suporte de variáveis numéricas e categóricas e capacidade de modelar interações complexas), as importâncias das variáveis geralmente fornecem uma solução eficaz para o problema de seleção de recursos. A solução mais simples consiste em ordenar as variáveis de acordo com as suas pontuações de importância e manter apenas as mais importantes. Dependendo do objetivo, o melhor subconjunto de variáveis pode ser identificado de diversas maneiras:

- Quando o objetivo é simplesmente reduzir a dimensionalidade devido aos requisitos de velocidade e armazenamento, a solução mais simples é manter apenas as variáveis cuja importância $\text{Imp}(X_j)$ é maior do que algum limite definido manualmente α.

- Se o objetivo é melhorar a precisão, um bom subconjunto de variáveis normalmente pode ser encontrado ajustando o limite α de modo a minimizar algum critério definido pelo usuário (por exemplo, a perda de zero-um na classificação ou a perda de erro quadrático na regressão) para o modelo construído no subconjunto $\{X_j | \text{Imp}(X_j) > \alpha\}$.

  Ao preço de mais esforços computacionais, um desempenho ainda melhor pode geralmente ser alcançado incorporando-se variáveis.

portances into a dedicated iterative feature selection procedure, such as those described in [Guyon et al., 2002; Tuv et al., 2009].

- In some other applications, the objective is to identify variables that are relevant, in order to better understand the underlying relations with the output Y. In asymptotic conditions, this could be done by discarding all variables whose importances is null, as shown by Theorem 6.5. In a finite setting however, bias due to masking effects or impurity misestimations (as previously discussed in Section 7.2) makes it more difficult to identify variables that are truly relevant since their importances might appear to be lower than those of irrelevant variables. Yet, several options are available for controlling and limiting false positives, such as stopping the construction process when impurity estimations become statistically unreliable (Section 7.2.2), comparing the importances of the original input variables to artificial contrasts [Tuv et al., 2006] or robustly controlling the conditional error rate through permutation tests [Huynh-Thu et al., 2012].

7.3.2 *Biomarker identification*

With the rise of *-omics* data, random forests have become one of the most popular tools in life sciences, providing practitioners with both high-prediction accuracy and helpful insights about the importances of variables. In many cases, variable importance measures (either MDI or the permutation importance) is exploited to better understand complex interaction effects between the inputs and the output. Examples of successful applications include the identification of disease-associated SNPs in genome-wide association studies [Lunetta et al., 2004; Meng et al., 2009; Botta et al., 2014], the discovery of important genes and pathways from micro-array gene-expression data [Pang et al., 2006; Chang et al., 2008] or the identification of factors for predicting protein-protein interactions [Qi et al., 2006]. These examples are not isolated and tens of further studies based on random forests could in fact be cited from the fields of genomics, metabolomics, proteomics or transcriptomics. Some of them are reviewed in [Geurts et al., 2009; Touw et al., 2013; Boulesteix et al., 2012].

In light of our study and discussion of variable importances, recommendations for biomarker identification depend on the exact objective of the application. If the goal is to identify all relevant variables, then totally randomized trees with multiway splits should be preferred. They indeed constitute the only method for which variable importances are unbiased, taking into account all possible interaction terms in a fair and exhaustive way. The only caveat is that a (very) large number of trees may be required before variable importances converge, making them computationally intensive to compute, even

importâncias para um procedimento de seleção de recursos iterativo dedicado, como aqueles descritos em [Guyon et al., 2002; Tuv et al., 2009].

- Em algumas outras aplicações, o objetivo é identificar variáveis que sejam relevantes, a fim de compreender melhor as relações subjacentes com o resultado Y. Em condições assintóticas, isso poderia ser feito descartando todas as variáveis cuja importância é nula, conforme mostrado pelo Teorema 6.5. Num cenário finito, contudo, o enviesamento devido a efeitos de mascaramento ou a estimativas erradas de impurezas (conforme discutido anteriormente na Secção 7.2) torna mais difícil identificar variáveis que são verdadeiramente relevantes, uma vez que as suas importâncias podem parecer inferiores às das variáveis irrelevantes. No entanto, várias opções estão disponíveis para controlar e limitar falsos positivos, como interromper o processo de construção quando as estimativas de impureza se tornam estatisticamente não confiáveis (Seção 7.2.2), comparar as importâncias das variáveis de entrada originais com contrastes artificiais [Tuv et al., 2006] ou controlar de forma robusta a taxa de erro condicional por meio de testes de permutação [Huynh-Thu et al.,2012].

7.3.2 Identificação de biomarcadores

Com o surgimento dos dados -ômicos, as florestas aleatórias tornaram-se uma das ferramentas mais populares nas ciências da vida, fornecendo aos profissionais alta precisão de previsão e insights úteis sobre a importância das variáveis. Em muitos casos, as medidas de importância variável (seja o MDI ou a importância da permutação) são exploradas para melhor compreender os efeitos complexos de interacção entre os factores de produção e os produtos. Exemplos de aplicações bem-sucedidas incluem a identificação de SNPs associados a doenças em estudos de associação genômica ampla [Lunetta et al., 2004; Meng et al., 2009; Botta et al., 2014], a descoberta de genes e caminhos importantes a partir de dados de expressão gênica de microarranjos[Pang et al., 2006; Chang et al., 2008] ou a identificação de fatores para prever interações proteína-proteína [Qi et al., 2006]. Esses exemplos não são isolados e dezenas de outros estudos baseados em florestas aleatórias poderiam, de fato, ser citados nas áreas de genômica, metabolômica.

proteômica ou transcriptômica. Alguns deles são revisados em [Geurts et al., 2009; Touw et al., 2013; Boulesteix et al., 2012].

À luz do nosso estudo e discussão sobre importâncias variáveis, as recomendações para identificação de biomarcadores dependem do objetivo exato da aplicação. Se o objetivo é identificar todas as variáveis relevantes, então árvores totalmente aleatórias com divisões multiway devem ser preferidas. Na verdade, constituem o único método para o qual as importâncias variáveis são imparciais, tendo em conta todos os termos de interação possíveis de uma forma justa e exaustiva. A única ressalva é que um número (muito) grande de árvores pode ser necessário antes que as importâncias das variáveis convirjam, tornando-as computacionalmente intensivas para serem computadas, mesmo

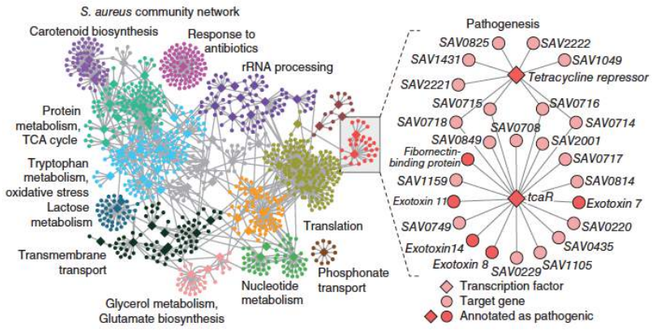


Figure 7.9: Gene regulatory network in Staphylococcus aureus. Image from [Marbach et al., 2012].

if the full randomization of the induction procedure actually make individual decision trees to be quite fast to generate. By contrast, if the objective is only to identify a subset of good predictors for predicting the output (hence possibly omitting relevant but redundant variables), then non-totally randomized trees (e.g., standard RF or ETs) with K chosen to maximize accuracy should be preferred. Even if some informative variables may be masked because of variable selection, a good subset of input variables should still be identifiable from importance scores. From a computational point of view, convergence should also be faster, making this approach more appealing when resources are limited. In all cases, impurity misestimations should be controlled by stopping the construction process early or collecting importances only for those nodes where impurity reductions can be considered as reliable. Finally, whenever practical, variable importances should also be decomposed in order to better understand why some variables appear as more important than others. In particular, studying the decomposition might help identify redundancy effects.

7.3.3 *Network inference*

Given a set of input variables $V = \{X_1, \dots, X_p\}$, the *network inference* problem consists in the identification of conditional dependencies between variables. In genomics for example, regulatory network inference consists in the identification of interactions between genes or transcription factors on the basis of their expression level, in order to reconstruct a global network of interactions (as illustrated in Figure 7.9 for Staphylococcus aureus).

As proposed in the GENIE3 method [Huynh-Thu et al., 2010], the network inference problem can be solved generically by remarking that it can decomposed into p independent supervised learning problems. Formally, for each input variable X_j (for $j = 1, \dots, p$), GENIE3 considers the supervised sub-problem consisting in the prediction of

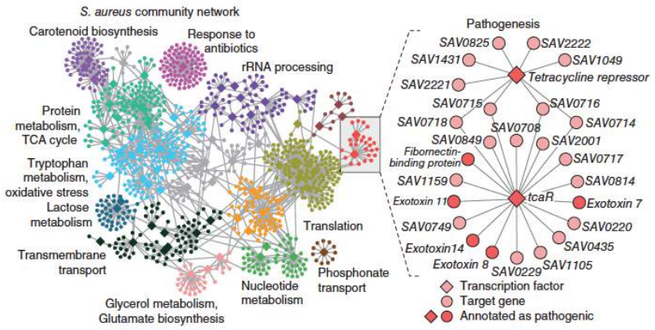


Figura 7.9: Rede reguladora genética em Staphylococcus aureus. Imagem de[Marbach et al., 2012].

se a randomização completa do procedimento de indução realmente fizer com que as árvores de decisão individuais sejam bastante rápidas de gerar. Por outro lado, se o objetivo é apenas identificar um subconjunto de bons preditores para prever o resultado (portanto, possivelmente omitindo variáveis relevantes, mas redundantes), então árvores não totalmente aleatórias (por exemplo, FR padrão ou ETs) com K escolhidas para maximizar a precisão devem ser preferidas. Mesmo que algumas variáveis informativas possam ser mascaradas devido à selecção de variáveis, um bom subconjunto de variáveis de entrada deverá ainda assim ser identificável a partir das pontuações de importância. Do ponto de vista computacional, a convergência também deverá ser mais rápida, tornando esta abordagem mais apelativa quando os recursos são li mitados. Em todos os casos, as estimativas erradas de impurezas devem ser controladas interrompendo o processo de construção antecipada mente ou coletando importâncias apenas para os nós onde as reduções de impurezas podem ser consideradas confiáveis. Finalmente, sempre que for prático, as importâncias das variáveis também devem ser decompostas, a fim de compreender melhor porque é que algumas variáveis parecem ser mais importantes do que outras. Em particular, estudar a decomposição pode ajudar a identificar efeitos de redundância.

7.3.3 Inferência de rede

Dado um conjunto de variáveis de entrada $V = \{X_1, \dots, X_p\}$, o problema de inferência em redes consiste na identificação de dependências condicionais entre variáveis. Na genómica, por exemplo, a inferência de redes reguladoras consiste na identificação de interacções entre genes ou factores de transcrição com base no seu nível de expressão, a fim de reconstruir uma rede global de interacções (como ilustrado na Figura 7.9 para Staphylococcus aureus).

Conforme proposto no método GENIE3 [Huynh-Thu et al., 2010], o problema de inferência de rede pode ser resolvido genericamente observando que ele pode ser decomposto em p problemas de aprendizagem supervisionada independentes. Formalmente, para cada variável de entrada X_j (para $j = 1, \dots, p$), o GENIE3 considera o subproblema supervisionado que consiste na previsão de

the target variable X_j from the remaining $p-1$ variables V^{-j}. Using random forests for solving each of these p problems, variable importances can be derived and used as an indication of the (directed) putative link between the predictor variables X_i (for $i \neq j$) and the target X_j. Intuitively, the larger the importance, the more likely the conditional dependency with X_j. Once all p problems are solved, putative links are aggregated over all p variables to provide a ranking of interactions from which the network can finally be reconstructed.

Again, our previous theoretical analysis calls for caution when plugging variable importances into network inference procedures. Indeed, since the goal is to retrieve all *direct* interactions, importances might in fact not be as appropriate as desired since they also account for indirect or combined effects. In this case, a good heuristic is to intentionally induce masking effects (i.e., by setting $K > 1$) in order to recover only the strongest (and assumingly direct) interactions. Alternatively, a promising strategy might be to directly look for *strongly* relevant variables, that is for variables X_i such that $I(X_i; X_j|B) > 0$ for *all* B. In both cases, impurity misestimations effects should be mitigated by using an adequate stopping criterion. When input variables are of different scale of measurements or vary in the number of categories, care should finally be taken when aggregating variable importances into a single ranking of interactions. As shown by Theorem 6.2, variable importances are in this case upper bounded by the entropy $H(X_j)$ of the target variable, which may greatly varies from one target to another. As such, variable importances are not directly comparable and should preferably be normalized (e.g., by $H(X_j)$) before their aggregation.

a variável de destino X_j das $p-1$ variáveis V^{-j} restantes. Usando fl orestas aleatórias para resolver cada um desses problemas p, as importâncias das variáveis podem ser derivadas e usadas como uma indicação do link putativo (dirigido) entre as variáveis preditoras X_i (para $i \neq j$) e o alvo X_j. Intuitivamente, quanto maior a importância, maior a probabilidade de dependência condicional com X_j. Uma vez resolvidos todos os p problemas, os links putativos são agregados sobre todas as p variáveis para fornecer uma classificação de interações a partir da qual a rede pode finalmente ser reconstruída.

Novamente, nossa análise teórica anterior exige cautela ao inserir importâncias variáveis em procedimentos de inferência de rede. Na verdade, uma vez que o objectivo é recuperar todas as interacções directas, as importâncias podem, de facto, não ser tão apropriadas quanto desejado, uma vez que também contabilizam efeitos indirectos ou combinados. Neste caso, uma boa heurística é induzir intencionalmente efeitos de mascaramento (isto é, configurando $K > 1$) para recuperar apenas as interações mais fortes (e presumivelmente diretas). Alternativamente, uma estratégia promissora poderia ser procurar diretamente variáveis fortemente relevantes, isto é, variáveis X_i tais que $I(X_i; X_j|B) > 0$para todos B. Em ambos os casos, os efeitos das esti mativas erradas de impurezas devem ser mitigados através da utilização de um critério de parada adequado. Quando as variáveis de entrada têm diferentes escalas de medição ou variam no número de categorias, deve-se finalmente tomar cuidado ao agregar as importâncias das variáveis em uma única classificação de interações. Como mostra o Teorema 6.2, as importâncias das variáveis são, neste caso, limitadas superiormente pela entropia $H(X_j)$ da variável alvo, que pode variar muito de um alvo para outro. Como tal, as importâncias das variáveis não são diretamente comparáveis e devem preferencialmente ser normalizadas (por exemplo, por $H(X_j)$) antes da sua agregação.

Part III

SUBSAMPLING DATA

Parte III

SUBSAMPLINGDATA

8 ENSEMBLES ON RANDOM PATCHES

Outline

In this chapter, we consider supervised learning under the assumption that the available memory is small compared to the size of the dataset. This general framework is relevant in the context of big data, distributed databases and embedded systems. In Section 8.1, we propose a very simple, yet effective, ensemble framework that builds each individual model of the ensemble from a random patch of data obtained by drawing random subsets of *both* samples and input variables from the whole dataset. In sections 8.2 and 8.3, we carry out an extensive and systematic evaluation of this method on 29 datasets, using decision trees as base models. With respect to popular ensemble methods, these experiments show that the proposed method provides on par performance in terms of accuracy while simultaneously lowering the memory needs, and attains significantly better performance when memory is severely constrained. We conclude and discuss future work directions in Section 8.4. *This chapter is based on previous work published in [Louppe and Geurts, 2012].*

Within the past few years, big data has become a popular trend among many scientific fields. In life sciences, computer vision, Internet search or finance, to cite a few, quantities of data have grown so large that it is increasingly difficult to process, analyze or visualize. In many cases, single computers are no longer fit for big data and distributed environments need to be considered to handle it. Although research is very active in this area, machine learning is no exception to this new paradigm. Much still needs to be done and methods and algorithms have to be reinvented to take this constraint into account.

In this context, we consider supervised learning problems for which the dataset is so large that it cannot be loaded into memory. Breiman [1999] proposed the Pasting method to tackle this problem by learning an ensemble of models individually built on random subsets of the training examples, hence alleviating the memory requirements since the base models would be built on only small parts of the whole dataset. Earlier, Ho [1998] proposed to learn an ensemble of models individually built on random subspaces, i.e., on random subsets of the input variables (or *features*). While the first motivation of the Random Subspace method was to increase the diversity within the models of the ensemble, it can actually also be seen as way to reduce the memory requirements of building individual models. In

8 ENSEMBLES ON RANDOM PATCHES

Contorno

Neste capítulo, consideramos a aprendizagem supervisionada sob a suposição de que a memória disponível é pequena em comparação com o tamanho do conjunto de dados. Este quadro geral é relevante no contexto de big data, bases de dados distribuídas e sistemas incorporados. Na Seção 8.1, propomos uma estrutura de conjunto muito simples, mas eficaz, que constrói cada modelo individual do conjunto a partir de um fragmento aleatório de dados obtidos pelo desenho de subconjuntos aleatórios de amostras e variáveis de entrada de todo o conjunto de dados. Nas seções 8.2 e 8.3, realizamos uma avaliação extensa e sistemática deste método e m 29 conjuntos de dados, utilizando árvores de decisão como modelos base. Com relação aos métodos de conjunto populares, esses experi mentos mostram que o método proposto fornece desempenho equivalente em termos de precisão, ao mesmo tempo em que reduz as necessidades de memória, e atinge desempenho significativamente melhor quando a memória é severamente restrita. Concluímos e discutimos futuras direções de trabalho na Seção 8.4. Este capítulo é baseado em trabalhos anteriores publicados em [Louppe and Geurts, 2012].

Nos últimos anos, o big data se tornou uma tendência popular entre muitos campos científicos. Nas ciências da vida, na visão computacional, na pesquisa na Internet ou nas finanças, para citar alguns, as quantidades de dados cresceram de tal forma que é cada vez mais difícil processar, analisar ou visualizar. Em muitos casos, computadores individuais já não são adequados para big data e é necessário considerar ambientes distribuídos para lidar com isso. Embora a investigação seja muito ativa nesta área, a aprendizagem automática não é exceção a este novo paradigma. Muito ainda precisa ser feito e métodos e algoritmos têm que ser reinventados para levar em conta esta restrição.

Neste contexto, consideramos problemas de aprendizagem supervisionada para os quais o conjunto de dados é tão grande que não pode ser carregado na memória. Breiman[1999] propôs o método Pasting para resolver esse problema, aprendendo um conjunto de modelos construídos individualmente em subconjuntos aleatórios dos exemplos de treinamento, aliviando assim os requisitos de memória, uma vez que os modelos básicos seriam construídos apenas em pequenas partes de todo o conjunto de dados. Anteriormente, Ho [1998] propôs aprender um conjunto de modelos construídos individualmente em subespaços aleatórios, ou seja, em subconjuntos aleatórios de variáveis (ou características) de entrada. Embora a primeira motivação do método Random Subspace tenha sido aumentar a diversidade dentro dos modelos do conjunto, ele também pode ser visto como uma forma de reduzir os requisitos de memória para a construção de modelos individuais. Em

this work, we propose to combine and leverage both approaches at the same time: learn an ensemble of models on *random patches*, i.e., on random subsets of the samples *and* of the input variables. Through an extensive empirical study, we show that this approach (1) improves or preserves comparable accuracy with respect to other ensemble approaches which build base models on the whole dataset while (2) drastically lowering the memory requirements and hence allowing an equivalent reduction of the global computing time. In other words, this analysis shows that there is usually no need to build individual models on the whole dataset. For the same accuracy, models can be learned independently on small portions of the data, within significantly lower computational requirements.

8.1 RANDOM PATCHES

The Random Patches algorithm proposed in this work (further referred to as RP) is a wrapper ensemble method that can be described in the following terms. Let $\mathcal{R}(\alpha_s, \alpha_f, \mathcal{L})$ be the set of all random patches of size $\alpha_s N \times \alpha_f p$ than can be drawn from the dataset $\mathcal{L}$, where N (resp., p) is the number of samples in $\mathcal{L}$ (resp., the number of input variables in $\mathcal{L}$) and where $\alpha_s \in [0, 1]$ (resp. α_f) is an hyperparameter that controls the number of samples in a patch (resp., the number of variables). That is, $\mathcal{R}(\alpha_s, \alpha_f, \mathcal{L})$ is the set of all possible subsets containing $\alpha_s N$ samples (among N) with $\alpha_f p$ variables (among p). The method then works as follows:

Algorithm 8.1. *Random Patches algorithm.*

1: **for** $m = 1, \dots, M$ **do**
2: Draw a patch $r \sim \mathcal{U}(\mathcal{R}(\alpha_s, \alpha_f, \mathcal{L}))$ uniformly at random
3: Build a model on the selected patch r
4: **end for**
5: Aggregate the predictions of the M models in an ensemble

While the RP algorithm can exploit any kind of base estimators, we consider in this work only tree-based estimators. In particular, we evaluate the RP algorithm using standard classification trees (as described in Chapter 3) and (single) extremely randomized trees [Geurts et al., 2006a]. Unless stated otherwise, trees are unpruned and grown using Gini index as impurity criterion for node splitting. The parameter K of extremely randomized trees within RP is set to its maximum value $K = \alpha_f p$ (i.e., corresponding to no further random selection of variables).

The first benefit of RP is that it generalizes both the Pasting Rvotes (P) method [Breiman, 1999] (and its extensions [Chawla et al., 2004; Basilico et al., 2011]) and the Random Subspace (RS) algorithm [Ho, 1998]. Both are indeed merely particular cases of RP: setting $\alpha_s = 1.0$ yields RS while setting $\alpha_f = 1.0$ yields P. As such, it is expected that

Neste trabalho, propomos combinar e alavancar ambas as abordagens ao mesmo tempo: aprender um conjunto de modelos em patches aleatórios, ou seja, em subconjuntos aleatórios das amostras e das variáveis de entrada. Através de um extenso estudo empírico, mostramos que esta abordagem (1) melhora ou preserva uma precisão comparável em relação a outras abordagens de conjunto que constroem modelos básicos em todo o conjunto de dados, enquanto (2) reduz drasticamente os requisitos de memória e, portanto, permite uma redução equivalente do tempo de computação global. Por outras palavras, esta análise mostra que normalmente não há necessidade de construir modelos individuais sobre todo o conjunto de dados. Para a mesma precisão, os modelos podem ser aprendidos independentemente em pequenas porções dos dados, dentro de requisitos computacionais significativamente mais baixos.

8.1 patches aleatórios

O algoritmo Random Patches proposto neste trabalho (mais conhecido como RP) é um método de conjunto wrapper que pode ser descrito nos seguintes termos. Seja $\mathcal{R}(\alpha_s, \alpha_f, \mathcal{L})$ o conjunto de todos os patches aleatórios de tamanho $\alpha_s N \times \alpha_f p$ que podem ser extraídos do conjunto de dados $\mathcal{L}$, onde N (resp., p) é o número de amostras em $\mathcal{L}$(resp., o número de variáveis de entrada em $\mathcal{L}$) e onde $\alpha_s \in [0, 1]$ (resp. α_f) é um hiperparâmetro que controla o número de amostras em um patch (resp., o número de variáveis). Ou seja, $\mathcal{R}(\alpha_s, \alpha_f, \mathcal{L})$ é o conjunto de todos os subconjuntos possíveis contendo $\alpha_s N$ amostras (entre N) com $\alpha_f p$ variáveis (entre$p$). O método então funciona da seguinte maneira:

Algoritmo 8.1. Algoritmo de patches aleatórios.

1: para $m = 1, \dots, M$ fazer 2: Desenhar um patch $r \sim \mathcal{U}(\mathcal{R}(\alpha_s, \alpha_f, \mathcal{L}))$ de maneira uniforme e aleatória 3: Construir um modelo no patch selecionado r4: terminar para 5: Agregar as previsões dos M modelos em um conjunto

Embora o algoritmo RP possa explorar qualquer tipo de estimadores de base, consideramos neste trabalho apenas estimadores baseados em árvores. Em particular, avaliamos o algoritmo RP usando árvores de classificação padrão (conforme descrito no Capítulo 3) e árvores (únicas) extremamente aleatórias [Geurts et al., 2006a]. Salvo indicação em contrário, as árvores não são podadas e cultivadas usando o índice de Gini como critério de impureza para a divisão dos nós. O parâmetro K de árvores extremamente aleatórias dentro de RP é definido para seu valor máximo $K = \alpha_f p$ (ou seja, correspondendo a nenhuma seleção aleatória adicional de variáveis).

O primeiro benefício do RP é que ele generaliza o método Pasting Rvotes (P) [Breiman, 1999] (e suas extensões [Chawla et al., 2004; Basilico et al., 2011]) e o algoritmo Random Subspace (RS) [Ho,1998]. Ambos são, na verdade, apenas casos particulares de RP: definir $\alpha_s = 1.0$ produz RS enquanto definir $\alpha_f = 1.0$ produz P. Como tal, espera-se que

when both hyper-parameters α_s and α_f are tuned simultaneously, RP should be at least as good as the best of the two methods, provided there is no overfitting associated with this tuning.

When the base estimators are standard decision trees (resp., extremely randomized trees with $K = \alpha_f p$), interesting parallels can also be drawn between RP and the RF algorithm (resp., ET). For $\alpha_s = 1.0$, the value of $\alpha_f p$ is indeed nearly equivalent to the number K of features randomly considered when splitting a node. A major difference remains though. In RP, the subset of features is selected globally once and for all, prior to the construction of each tree. By contrast, in RF (resp., in ET) subsets of features are drawn locally at each node. Clearly, the former approach already appears to be more attractive when dealing with large databases. Non-selected features indeed do not need to be considered at all, hence lowering the memory requirements for building a single tree. Another interesting parallel can be made when bootstrap samples are used like in RF: it nearly amounts to set $\alpha_s = 0.632$, i.e. the average proportion of unique samples in a bootstrap sample. Differences are that in a bootstrap sample, the number of unique training samples varies from one to another (while it would be fixed to $0.632N$ in RP), and that samples are not all equally weighted.

In addition, RP also closely relates to the SubBag algorithm which combines Bagging and RS for constructing ensembles. Using N bootstrapped samples (i.e., nearly equivalent to $\alpha_s = 0.632$) and setting $\alpha_f = 0.75$, Panov and Džeroski [2007] showed that SubBag has comparable performance to that of RF. An added advantage of SubBag, and hence of RP, is that it is applicable to any base estimator without the need to randomize the latter.

8.2 ON ACCURACY

Our validation of the RP algorithm is carried out in two steps. In this section, we first investigate how RP compares with other popular tree-based ensemble methods in terms of accuracy. In the next section, we then focus on its memory requirements for achieving optimal accuracy and its capability to handle strong memory constraints, again in comparison with other ensemble methods.

Considering accuracy only, our main objective is to investigate whether the additional degrees of freedom brought by α_s and α_f significantly improve, or degrade, the performance of RP. At first glance, one might indeed think that since the base estimators are (intentionally) built on parts of the data, the accuracy of the ensemble will be lower than if they were all built on the whole set. Additionally, our goal is also to see whether sampling features once globally, instead of locally at each node, impairs performance, as this is the main difference between RP and state-of-the-art methods such as RF or ET.

quando ambos os hiperparâmetros α_s e α_f são ajustados simultaneamente, o RP deve ser pelo menos tão bom quanto o melhor dos dois métodos, desde que não haja overfitting associado a esse ajuste.

Quando os estimadores de base são árvores de decisão padrão (resp., árvores extremamente aleatórias com $K = \alpha_f p$), paralelos interessantes também podem ser traçados entre RP e o algoritmo RF (resp., ET). Para $\alpha_s = 1.0$, o valor de $\alpha_f p$ é de fato quase equivalente ao número$K$ de recursos considerados aleatoriamente ao dividir um nó. Porém, uma grande diferença permanece. No RP, o subconjunto de características é selecionado globalmente de uma vez por todas, antes da construção de cada árvore. Por outro lado, em RF (resp., em ET) subconjuntos de rec ursos são desenhados localmente em cada nó. Claramente, a primeira abordagem já parece ser mais atraente quando se trata de grandes bases de dados. Na verdade, os recursos não selecionados não precisam ser considerados, reduzindo assim os requisitos de memória para a construção de uma única árvore. Outro paralelo interessante pode ser f eito quando amostras de bootstrap são usadas como em RF: quase e quivale ao conjunto $\alpha_s = 0.632$, ou seja, a proporção média de amostras únicas em uma amostra de bootstrap. As diferenças são que, em uma amostra de bootstrap, o número de amostras de treinamento exclusivas varia de uma para outra (embora seja fixado em $0.632N$ em RP) e que as amostras não são todas igualmente ponderadas.

Além disso, RP também está intimamente relacionado ao algoritmo SubBag que combina Bagging e RS para a construção de conjuntos. Usando N amostras boot-strap (ou seja, quase equivalentes a $\alpha_s = 0.632$) e configurando$\alpha_f = 0.75$, Panov e Džeroski [2007] mostraram que SubBag tem desempenho comparável ao de RF. Uma vantagem adicional do SubBag e, portanto, do RP, é que ele é aplicável a qualquer estimador base sem a necessidade de randomizá-lo.

8.2 sobre precisão

Nossa validação do algoritmo RP é realizada em duas etapas. Nesta seção, primeiro investigamos como o RP se compara com outros métodos populares de conjuntos baseados em árvores em termos de precisão. Na próxima seção, focaremos nos requisitos de memória para alcançar a precisão ideal e na capacidade de lidar com fortes restrições de memória, novamente em comparação com outros métodos de conjunto.

Considerando apenas a precisão, nosso principal objetivo é investigar se os graus adicionais de liberdade trazidos por α_s e α_f melhoram ou degradam significativamente o desempenho do RP. À primeira vista, pode-se de facto pensar que, uma vez que os estimadores de base são (intencionalmente) construídos sobre partes dos dados, a precisão do conjunto será menor do que se todos fossem construídos sobre o conjunto completo. Além disso, nosso objetivo também é verificar se a amostragem realizada uma vez globalmente, em vez de localmente em cada nó, prejudica o desempenho, já que esta é a principal diferença entre RP e métodos de última geração, como RF ou ET.

8.2.1 *Protocol*

We compare our method with P and RS, as well as with RF and ET. For RP, P and RS, two variants have been considered, one using standard decision trees (suffixed below with '-DT') as base estimators, and the other using extremely randomized trees (suffixed below with '-ET') as base estimators. Overall, 8 methods are compared: RP-DT, RP-ET, P-DT, P-ET, RS-DT, RS-ET, RF and ET.

We evaluate the accuracy of the methods on an extensive list of both artificial and real classification problems. For each dataset, three random partitions were drawn: the first and larger (50% of the original dataset) to be used as the training set, the second (25%) as validation set and the third (25%) as test set. For all methods, the hyper-parameters α_s and α_f were tuned on the validation set with a grid-search procedure, using the grid {0.01, 0.1, ..., 0.9, 1.0} for both α_s and α_f. All other hyper-parameters were set to default values. In RF and ET, the number K of features randomly selected at each node was tuned using the grid $\alpha_f p$. For all ensembles, 250 fully developed trees were generated and the generalization accuracy was estimated on the test set. Unless otherwise mentioned, for all methods and for all datasets, that procedure was repeated 50 times, using the same 50 random partitions between all methods, and all scores reported below are averages over those 50 runs. All algorithms and experiments have been implemented in Python, using Scikit-Learn [Pedregosa et al., 2011] as base framework.

8.2.2 *Small datasets*

Before diving into heavily computational experiments, we first wanted to validate our approach on small to medium datasets. To that end, experiments were carried out on a sample of 16 well-known and publicly available datasets (see Table 8.1) from the UCI machine learning repository [Frank and Asuncion, 2010], all chosen a priori and independently of the results obtained. Overall, these datasets cover a wide range of conditions, with the sample sizes ranging from 208 to 20000 and the number of features varying from 6 to 168. Detailed average performances of the 8 methods for all 16 datasets using the protocol described above are reported in Table 8.2. Below, we analyze general trends by performing various statistical tests.

Following recommendations in [Demsar, 2006], we first performed a Friedman test that rejected the hypothesis that all algorithms are equivalent at a significance level $\alpha = 0.05$. We then proceeded with a post-hoc Nemenyi test for a pairwise comparison of the average ranks of all 8 methods. According to this test, the performance of two classifiers is significantly different (at $\alpha = 0.05$) if their average ranks differ by at least the critical difference $CD = 2.6249$ (See [Demsar,

8.2.1 Protocolo

Comparamos nosso método com P e RS, bem como com RF e ET. Para RP, P e RS, duas variantes foram consideradas, uma usando árvores de decisão padrão (sufixadas abaixo com '-DT') como estimadores base, e a outra usando árvores extremamente aleatórias (sufixadas abaixo com '-ET') como estimadores base. No geral, são comparados 8 métodos: RP-DT, RP-ET, P-DT, P-ET, RS-DT, RS-ET, RF e ET.

Avaliamos a precisão dos métodos em uma extensa lista de proble mas de classificação artificiais e reais. Para cada conjunto de dados foram desenhadas três partições aleatórias: a primeira e maior (50% do conjunto de dados original) para ser usada como conjunto de treinamento, a segunda (25%) como conjunto de validação e a terceira (25%) como conjunto de teste. Para todos os métodos, os hiperparâmetros α_s e α_f foram ajustados no conjunto de validação com um procedimento de busca em grade, usando a grade {0.01, 0.1, ..., 0.9, 1.0} para ambos α_se α_f. Todos os outros hiperparâmetros foram definidos com valores padrão. Em RF e ET, o número K de características selecionadas aleatoriamente em cada nó foi ajustado usando a grade $\alpha_f p$. Para todos os conjuntos, foram geradas 250 árvores totalmente desenvolvidas e a precisão da generalização foi estimada no conjunto de teste. Salvo indicação em contrário, para todos os métodos e para todos os conjuntos de dados, esse procedimento foi repetido 50 vezes, usando as mesmas 50 partições aleatórias entre todos os métodos, e todas as pontuações relatadas abaixo são médias dessas 50 execuções. Todos os algoritmos e experimentos f oram implementados em Python, usando Scikit-Learn [Pedregosa et al., 2011] como framework base.

8.2.2 Pequenos conjuntos de dados

Antes de mergulhar em experimentos fortemente computacionais, primeiro queríamos validar nossa abordagem em conjuntos de dados pequenos e médios. Para tanto, foram realizados experimentos em uma amostra de 16 conjuntos de dados bem conhecidos e publicamente disponíveis (ver Tabela 8.1) do repositório de aprendizado de máquina da UCI [Frank e Assunção, 2010], todos escolhidos a priori e independentemente dos resultados obtidos. No geral, esses conjuntos de dados cobrem uma ampla gama de condições, com t amanhos de amostra variando de 208 a 20.000 e o número de características variando de 6 a 168. O desempenho médio detalhado dos 8 métodos para todos os 16 conjuntos de dados usando o protocolo descrito acima é relatado na Tabela 8.2. Abaixo, analisamos tendências gerais realizando vários testes estatísticos.

Seguindo as recomendações do [Demsar, 2006], realizamos primeiro um teste de Friedman que rejeitou a hipótese de que todos os algoritmos são equivalentes em um nível de significância $\alpha = 0.05$. Em seguida, procedemos com um teste post-hoc de Nemenyi para uma comparação aos pares das classificações médias de todos os 8 métodos. De acordo com este teste, o desempenho de dois classificadores é significativamente diferente (em $\alpha = 0.05$) se suas classificações médias diferirem em pelo menos a diferença crítica $CD = 2.6249$ (Ver [Demsar,

Dataset	N	p
DIABETES	768	8
DIG44	18000	16
IONOSPHERE	351	34
PENDIGITS	10992	16
LETTER	20000	16
LIVER	345	6
MUSK2	6598	168
RING-NORM	10000	20
SATELLITE	6435	36
SEGMENT	2310	19
SONAR	208	60
SPAMBASE	4601	57
TWO-NORM	9999	20
VEHICLE	1692	18
VOWEL	990	10
WAVEFORM	5000	21

Table 8.1: Small datasets.

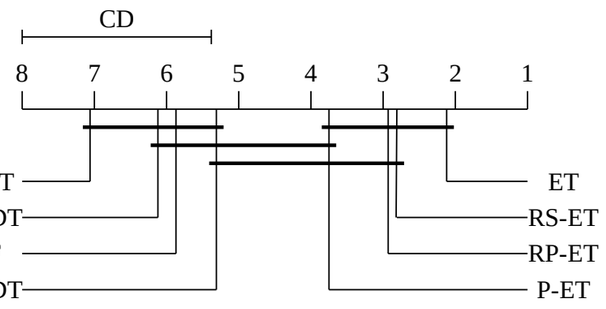


Figure 8.1: Average ranks of all methods on small datasets.

2006] for further details). The diagram of Figure 8.1 summarizes these comparisons. The top line in the diagram is the axis along which the average rank R_m of each method m is plotted, from the highest ranks (worst methods) on the left to the lowest ranks (best methods) on the right. Groups of methods that are not statistically different from each other are connected. The critical difference CD is shown above the graph. To further support these rank comparisons, we also compare the 50 accuracy values obtained over each dataset split for each pair of methods by using a paired t-test (with $\alpha = 0.01$). The results of these comparisons are summarized in Table 8.3 in terms of "Win-Draw-Loss" statuses of all pairs of methods; the three values at the intersection of row i and column j of this table respectively indicate on how many datasets method i is significantly better/not significantly different/significantly worse than method j.

Conjunto de dados	N	p
diabetes	768	8
dig44	18000	16
ionosfera	351	34
pendígios	10992	16
carta	20000	16
fígado	345	6
almíscar2	6598	168
norma do anel	10000	20
satélite	6435	36
segmento	2310	19
sonar	208	60
base de spa m	4601	57
duas normas	9999	20
veículo	1692	18
vogal	990	10
forma de onda	5000	21

Tabela 8.1: Pequenos conjuntos de dados.

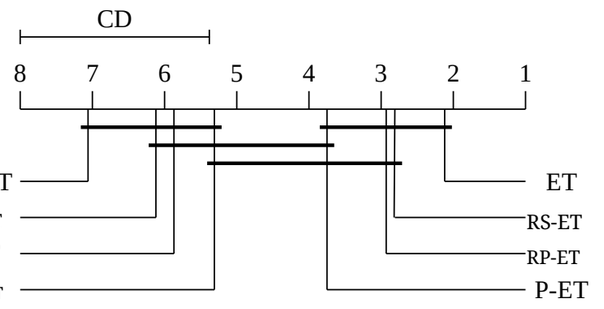


Figura 8.1: Classificações médias de todos os métodos em pequenos conjuntos de dados.

2006] para mais detalhes). O diagrama da Figura 8.1 resume essas comparações. A linha superior do diagrama é o eixo ao longo do qual a classificação média R_m de cada método m é traçada, desde as classificações mais altas (piores métodos) à esquerda até as classificações mais baixas (melhores métodos) à direita. Grupos de métodos que não são estatisticamente diferentes entre si são conectados. A diferença crítica CD é mostrada acima do gráfico. Para apoiar ainda mais essas comparações de classificação, também comparamos os 50 valores de precisão obtidos em cada divisão do conjunto de dados para cada par de métodos usando um teste t pareado (com $\alpha = 0.01$). Os resultados destas comparações estão resumidos na Tabela 8.3 em termos de status "Vitória-Empate-Perda" de todos os pares de métodos; os três valores na interseção da linha i e da coluna j desta tabela indicam, respectivamente, quantos conjuntos de dados o método i é significativamente melhor/não significativamente diferente/significativamente pior que o método j.

Validation	RF	ET	P-DT	P-ET	RS-DT	RS-ET	RP-DT	RP-ET
DIABETES	77.12 (6)	77.25 (5)	77.67 (4)	78.01 (3)	75.11 (8)	76.77 (7)	78.82 (2)	**79.07** (1)
DIG44	94.99 (7)	**95.78** (1)	91.86 (8)	95.46 (4)	95.07 (6)	95.69 (3)	95.13 (5)	95.72 (2)
IONOSPHERE	94.40 (6)	95.15 (3)	93.86 (8)	94.75 (5)	94.11 (7)	94.90 (4)	95.20 (2)	**95.36** (1)
PENDIGITS	98.94 (7)	**99.33** (1)	98.09 (8)	99.28 (4)	99.02 (6)	99.31 (3)	99.07 (5)	99.32 (2)
LETTER	95.36 (7)	**96.38** (1)	92.72 (8)	95.87 (4)	95.68 (6)	96.08 (3)	95.74 (5)	96.10 (2)
LIVER	72.37 (5)	71.90 (6)	72.55 (4)	72.88 (3)	68.06 (8)	70.88 (7)	**74.53** (1)	74.37 (2)
MUSK2	97.18 (7)	**97.73** (1)	96.89 (8)	97.60 (4)	97.58 (6)	97.72 (3)	97.60 (5)	97.73 (2)
RING-NORM	97.44 (6)	98.10 (5)	96.41 (8)	97.28 (7)	98.25 (4)	98.41 (3)	98.50 (2)	**98.54** (1)
SATELLITE	90.97 (7)	**91.56** (1)	90.01 (8)	91.40 (5)	91.31 (6)	91.50 (3)	91.41 (4)	91.54 (2)
SEGMENT	97.46 (6)	98.17 (2)	96.78 (8)	98.10 (4)	97.33 (7)	98.14 (3)	97.52 (5)	**98.21** (1)
SONAR	82.92 (7)	86.92 (3)	80.03 (8)	84.73 (5)	83.07 (6)	87.07 (2)	85.42 (4)	**88.15** (1)
SPAMBASE	94.80 (7)	95.36 (3)	93.69 (8)	95.01 (6)	95.01 (5)	95.50 (2)	95.11 (4)	**95.57** (1)
TWO-NORM	97.54 (6)	97.77 (2)	97.52 (7)	97.59 (5)	97.46 (8)	97.63 (4)	97.76 (3)	**97.82** (1)
VEHICLE	88.67 (5)	88.68 (4)	88.26 (8)	88.74 (3)	88.41 (7)	88.60 (6)	**89.22** (1)	89.21 (2)
VOWEL	92.04 (5)	**95.12** (1)	85.19 (8)	93.49 (4)	89.76 (7)	94.34 (3)	91.10 (6)	94.48 (2)
WAVEFORM	85.45 (6)	85.96 (2)	84.89 (8)	85.68 (5)	84.91 (7)	85.69 (4)	85.85 (3)	**86.21** (1)
Average rank	6.25	2.5625	7.4375	4.4375	6.5	3.75	3.5626	1.5
Test	RF	ET	P-DT	P-ET	RS-DT	RS-ET	RP-DT	RP-ET
DIABETES	75.62 (4)	75.38 (5)	75.67 (3)	**76.34** (1)	73.03 (8)	74.63 (7)	75.32 (6)	75.82 (2)
DIG44	94.96 (6)	**95.67** (1)	91.79 (8)	95.39 (4)	94.98 (5)	95.58 (2)	94.95 (7)	95.55 (3)
IONOSPHERE	92.20 (6)	**93.22** (1)	92.09 (7)	92.40 (4)	92.02 (8)	93.22 (2)	92.34 (5)	92.68 (3)
PENDIGITS	98.84 (7)	**99.23** (1)	97.97 (8)	99.21 (3)	98.95 (5)	99.21 (2)	98.93 (6)	99.20 (4)
LETTER	95.27 (7)	**96.29** (1)	92.57 (8)	95.89 (4)	95.61 (5)	96.03 (2)	95.61 (6)	95.99 (3)
LIVER	69.95 (3)	68.22 (6)	**70.43** (1)	69.58 (5)	63.17 (8)	67.35 (7)	70.20 (2)	69.67 (4)
MUSK2	97.08 (7)	**97.61** (1)	96.69 (8)	97.54 (4)	97.47 (5)	97.58 (2)	97.42 (6)	97.56 (3)
RING-NORM	97.48 (6)	98.07 (5)	96.42 (8)	97.25 (7)	98.16 (4)	**98.31** (1)	98.22 (3)	98.30 (2)
SATELLITE	90.67 (7)	91.22 (2)	89.66 (8)	91.20 (3)	91.15 (5)	**91.28** (1)	91.04 (6)	91.20 (4)
SEGMENT	97.02 (5)	**97.93** (1)	96.44 (8)	97.86 (3)	96.86 (7)	97.90 (2)	96.88 (6)	97.84 (4)
SONAR	79.53 (5)	**82.76** (1)	75.15 (8)	80.07 (4)	78.50 (6)	82.19 (2)	78.26 (7)	81.92 (3)
SPAMBASE	94.76 (7)	95.17 (3)	93.51 (8)	94.84 (5)	94.88 (4)	95.22 (2)	94.80 (6)	**95.22** (1)
TWO-NORM	97.22 (7)	**97.50** (1)	97.26 (6)	97.29 (4)	97.20 (8)	97.33 (2)	97.33 (3)	97.28 (5)
VEHICLE	87.47 (7)	87.85 (3)	87.01 (8)	87.98 (2)	87.50 (6)	87.68 (5)	87.73 (4)	**88.08** (1)
VOWEL	91.51 (5)	**93.95** (1)	84.17 (8)	92.93 (4)	89.51 (7)	93.60 (2)	89.89 (6)	93.09 (3)
WAVEFORM	85.23 (5)	**85.77** (1)	84.68 (8)	85.40 (3)	84.74 (7)	85.38 (4)	85.16 (6)	85.56 (2)
Average rank	5.875	2.125	7.0625	3.75	6.125	2.8125	5.3125	2.9375

Table 8.2: Accuracy on small datasets (in %).

Validação	RF	ET	P-DT	P-ET	RS-DT	RS-ET	RP-DT	RP-ET
diabetes	77,12 (6)	77,25 (5)	77,67 (4)	78,01 (3)	75,11 (8)	76,77 (7)	78,82 (2)	**79,07 (1)**
dig44	94,99 (7)	**95,78 (1)**	91,86 (8)	95,46 (4)	95,07 (6)	95,69 (3)	95,13 (5)	95,72 (2)
ionosfera	94,40 (6)	95,15 (3)	93,86 (8)	94,75 (5)	94,11 (7)	94,90 (4)	95,20 (2)	**95,36 (1)**
pendígios	98,94 (7)	**99,33 (1)**	98,09 (8)	99,28 (4)	99,02 (6)	99,31 (3)	99,07 (5)	99,32 (2)
carta	95,36 (7)	**96,38 (1)**	92,72 (8)	95,87 (4)	95,68 (6)	96,08 (3)	95,74 (5)	96,10 (2)
fígado	72,37 (5)	71,90 (6)	72,55 (4)	72,88 (3)	68,06 (8)	70,88 (7)	**74,53 (1)**	74,37 (2)
almíscar2	97,18 (7)	**97,73 (1)**	96,89 (8)	97,60 (4)	97,58 (6)	97,72 (3)	97,60 (5)	97,73 (2)
norma do anel	97,44 (6)	98,10 (5)	96,41 (8)	97,28 (7)	98,25 (4)	98,41 (3)	98,50 (2)	**98,54 (1)**
satélite	90,97 (7)	**91,56 (1)**	90,01 (8)	91,40 (5)	91,31 (6)	91,50 (3)	91,41 (4)	91,54 (2)
segmento	97,46 (6)	98,17 (2)	96,78 (8)	98,10 (4)	97,33 (7)	98,14 (3)	97,52 (5)	**98,21 (1)**
sonar	82,92 (7)	86,92 (3)	80,03 (8)	84,73 (5)	83,07 (6)	87,07 (2)	85,42 (4)	**88,15 (1)**
base de spa m	94,80 (7)	95,36 (3)	93,69 (8)	95,01 (6)	95,01 (5)	95,50 (2)	95,11 (4)	**95,57 (1)**
duas normas	97,54 (6)	97,77 (2)	97,52 (7)	97,59 (5)	97,46 (8)	97,63 (4)	97,76 (3)	**97,82 (1)**
veículo	88,67 (5)	88,68 (4)	88,26 (8)	88,74 (3)	88,41 (7)	88,60 (6)	**89,22 (1)**	89,21 (2)
vogal	92,04 (5)	**95,12 (1)**	85,19 (8)	93,49 (4)	89,76 (7)	94,34 (3)	91,10 (6)	94,48 (2)
forma de onda	85,45 (6)	85,96 (2)	84,89 (8)	85,68 (5)	84,91 (7)	85,69 (4)	85,85 (3)	**86,21 (1)**
Classificação média	6.25	2.5625	7.4375	4.4375	6.5	3.75	3.5626	1.5
Test	RF	ET	P-DT	P-ET	RS-DT	RS-ET	RP-DT	RP-ET
diabetes	75,62 (4)	75,38 (5)	75,67 (3)	**76,34 (1)**	73,03 (8)	74,63 (7)	75,32 (6)	75,82 (2)
dig44	94,96 (6)	**95,67 (1)**	91,79 (8)	95,39 (4)	94,98 (5)	95,58 (2)	94,95 (7)	95,55 (3)
ionosfera	92,20 (6)	**93,22 (1)**	92,09 (7)	92,40 (4)	92,02 (8)	93,22 (2)	92,34 (5)	92,68 (3)
pendígios	98,84 (7)	**99,23 (1)**	97,97 (8)	99,21 (3)	98,95 (5)	99,21 (2)	98,93 (6)	99,20 (4)
carta	95,27 (7)	**96,29 (1)**	92,57 (8)	95,89 (4)	95,61 (5)	96,03 (2)	95,61 (6)	95,99 (3)
fígado	69,95 (3)	68,22 (6)	**70,43 (1)**	69,58 (5)	63,17 (8)	67,35 (7)	70,20 (2)	69,67 (4)
almíscar2	97,08 (7)	**97,61 (1)**	96,69 (8)	97,54 (4)	97,47 (5)	97,58 (2)	97,42 (6)	97,56 (3)
norma do anel	97,48 (6)	98,07 (5)	96,42 (8)	97,25 (7)	98,16 (4)	**98,31 (1)**	98,22 (3)	98,30 (2)
satélite	90,67 (7)	91,22 (2)	89,66 (8)	91,20 (3)	91,15 (5)	**91,28 (1)**	91,04 (6)	91,20 (4)
segmento	97,02 (5)	**97,93 (1)**	96,44 (8)	97,86 (3)	96,86 (7)	97,90 (2)	96,88 (6)	97,84 (4)
sonar	79,53 (5)	**82,76 (1)**	75,15 (8)	80,07 (4)	78,50 (6)	82,19 (2)	78,26 (7)	81,92 (3)
base de spa m	94,76 (7)	95,17 (3)	93,51 (8)	94,84 (5)	94,88 (4)	95,22 (2)	94,80 (6)	**95,22 (1)**
duas normas	97,22 (7)	**97,50 (1)**	97,26 (6)	97,29 (4)	97,20 (8)	97,33 (2)	97,33 (3)	97,28 (5)
veículo	87,47 (7)	87,85 (3)	87,01 (8)	87,98 (2)	87,50 (6)	87,68 (5)	87,73 (4)	**88,08 (1)**
vogal	91,51 (5)	**93,95 (1)**	84,17 (8)	92,93 (4)	89,51 (7)	93,60 (2)	89,89 (6)	93,09 (3)
forma de onda	85,23 (5)	**85,77 (1)**	84,68 (8)	85,40 (3)	84,74 (7)	85,38 (4)	85,16 (6)	85,56 (2)
Classificação média	5.875	2.125	7.0625	3.75	6.125	2.8125	5.3125	2.9375

Tabela 8.2: Precisão em pequenos conjuntos de dados (em %).

	RF	ET	P-DT	P-ET	RS-DT	RS-ET	RP-DT	RP-ET
RF	—	1/2/13	12/4/0	1/7/8	4/7/5	2/2/12	1/10/5	0/4/12
ET	13/2/1	—	14/1/1	10/5/1	13/3/0	4/11/1	12/2/2	5/10/1
P-DT	0/4/12	1/1/14	—	0/4/12	2/3/11	2/1/13	0/4/12	0/4/12
P-ET	8/7/1	1/5/10	12/4/0	—	9/6/1	2/6/8	9/6/1	0/11/5
RS-DT	5/7/4	0/3/13	11/3/2	1/6/9	—	0/2/14	1/11/4	0/4/12
RS-ET	12/2/2	1/11/4	13/1/2	8/6/2	14/2/0	—	11/4/1	1/13/2
RP-DT	5/10/1	2/2/12	12/4/0	1/6/9	4/11/1	1/4/11	—	0/6/10
RP-ET	12/4/0	1/10/5	12/4/0	5/11/0	12/4/0	2/13/1	10/6/0	—

Table 8.3: Pairwise t-test comparisons on small datasets.

Since all methods are variants of ensembles of decision trees, average accuracies are not strikingly different from one method to another (see Table 1 of the supplementary materials). Yet, significant trends appear when looking at Figure 8.1 and Table 8.3. First, all ET-based methods are ranked before DT-based methods, including the popular Random Forest algorithm. Overall, the original ET algorithm is ranked first ($R_{ET} = 2.125$), then come RS-ET and RP-ET at close positions ($R_{RS-ET} = 2.8125$ and $R_{RP-ET} = 2.9375$) while P-ET is a bit behind ($R_{P-ET} = 3.75$). According to Figure 8.1, only ET is ranked significantly higher than all DT-based method but looking at Table 8.3, the worse ET-based variant (P-ET) is still 9 times significantly better (w.r.t. the 50 runs over each set) and only 1 time significantly worse than the best DT-based variant (RP-DT). The separation between these two families of algorithm thus appears quite significant. This observation clearly suggests that using random split thresholds, instead of optimized ones like in decision trees, pays off in terms of generalization.

Among ET-based methods, RP-ET is better than P-ET but it is superseded by ET and RS-ET in terms of average rank. Since RS-ET is a particular case of RP-ET, this suggests that we are slightly overfitting when tuning the additional parameter α_s. And indeed RP-ET is better ranked than RS-ET in average on the validation set (results not shown). Table 8.3 however indicates otherwise and makes RP-ET appear as slightly better than RS-ET (2/13/1). Regarding ET over RP-ET, the better performance of the former (5/10/1) is probably due to the fact that in ET subsets of features are redrawn locally at each node when building trees and not once and for all prior to their construction. This gives less chances to generate improper trees because of a bad initial choice of features and thus leads to a lower bias and a better accuracy.

Among DT-based methods, RP-DT now comes first (mean rank of 5.3125), then RF ($R_{RF} = 5.875$), RS-DT ($R_{RS-DT} = 6.125$) and then P-DT in last ($R_{P-DT} = 7.0625$). RP is only significantly worse than another DT-based variant on 1 dataset. The extra-randomization

	RF	ET	P-DT	P-ET	RS-DT	RS-ET	RP-DT	RP-ET
RF	—	02/01/13	04/12/0	1/7/8	07/04/05	02/02/12	1/10/5	04/04/12
ET	13/2/1	—	14/1/1	05/10/1	13/3/0	11/04/1	2/12/2	10/05/1
P-DT	04/04/12	01/01/14	—	04/04/12	03/02/11	01/02/13	04/04/12	04/04/12
P-ET	07/08/1	1/5/10	04/12/0	—	06/09/1	6/02/8	06/09/1	0/11/5
RS-DT	5/7/4	03/013	3/11/2	1/6/9	—	02/02/14	11/01/4	04/04/12
RS-ET	2/12/2	11/01/4	13/1/2	06/08/2	14/2/0	—	04/11/1	13/01/2
RP-DT	10/05/1	02/02/12	04/12/0	1/6/9	11/04/1	04/01/11	—	06/06/10
RP-ET	04/12/0	1/10/5	04/12/0	11/05/0	04/12/0	13/02/1	06/10/0	—

Tabela 8.3: Comparações do teste t pareado em pequenos conjuntos de dados.

Como todos os métodos são variantes de conjuntos de árvores de decisão, as precisões médias não são muito diferentes de um método para outro (ver Tabela 1 dos materiais suplementares). No entanto, surgem tendências significativas quando se olha para a Figura 8.1 e a Tabela 8.3. Primeiro, todos os métodos baseados em ET são classificados antes dos métodos baseados em DT, incluindo o popular algoritmo Random Forest. No geral, o algoritmo ET original é classificado em primeiro lugar ($R_{ET} = 2$ 0,125), depois vem RS-ET e RP-ET em posições fechadas ($R_{RS-ET} = 2$ 0,8125 e $R_{RP-ET} = 2$ 0,9375), enquanto P-ET está um pouco atrás ($R_{P-ET} = 3$ 0,75). De acordo com a Figura 8.1, apenas o ET é classificado significativamente mais alto do que todos os métodos baseados em DT, mas olhando para a Tabela 8.3, a pior variante baseada em ET (P-ET) ainda é 9 vezes significativamente melhor (em relação às 50 execuções em cada conjunto) e apenas 1 vez significativamente pior do que a melhor variante baseada em DT (RP-DT). A separação entre estas duas famílias de algoritmos parece, portanto, bastante significativa. Esta observação sugere claramente que o uso de limiares de divisão aleatória, em vez de limiares otimizados como nas árvores de decisão, compensa em termos de generalização.

Entre os métodos baseados em ET, o RP-ET é melhor que o P-ET, mas é substituído pelo ET e pelo RS-ET em termos de classificação média. Como RS-ET é um caso particular de RP-ET, isso sugere que estamos sobreajustando ligeiramente ao ajustar o parâmetro adicional α_s. E, de fato, o RP-ET é melhor classificado do que o RS-ET, em média, no conjunto de validação (resultados não mostrados). A Tabela 8.3, entretanto, indica o contrário e faz com que o RP-ET pareça um pouco melhor que o RS-ET (13/02/1). Em relação ao ET sobre RP-ET, o melhor desempenho do primeiro (5/10/1) provavelmente se deve ao fato de que no ET os subconjuntos de características são redesenhados localmente em cada nó durante a construção das árvores e não de uma vez por todas antes de sua construção. Isto dá menos chances de gerar árvores impróprias devido a uma má escolha inicial de características e, portanto, leva a um viés menor e a uma melhor precisão.

Entre os métodos baseados em DT, RP-DT agora vem em primeiro lugar (classificação média de 5,3125), depois RF ($R_{RF} = 5.875$), RS-DT ($R_{RS-DT} = 6.125$) e então P-DT em último ($R_{P-DT} = 7.0625$). RP é apenas significativamente pior do que outra variante baseada em DT em 1 conjunto de dados.

brought by the random choices of both samples and features seems to be beneficial with decision trees that do not benefit from the randomization of discretization thresholds. The fact that RF samples features locally does not appear here anymore as an advantage over RP (RF is significantly worse on 5 problems and better on only one), probably because the decrease of bias that it provides does not exceed the increase of variance with respect to global feature selection.

8.2.3 *Larger datasets*

While the former experiments revealed promising results, it is fair to ask whether the conclusions that have been drawn would hold on and generalize to larger problems, for example when dealing with a few relevant features buried into hundreds or thousands of not important features (e.g., in genomic data), or when dealing with many correlated features (e.g., in images). To investigate this question, a second bench of experiments was carried out on 13 larger datasets (see Table 8.4). All but MADELON are real data. In terms of dimensions, these datasets are far bigger, ranging from a few hundreds of samples and thousands of features, to thousands of samples but hundreds of features. As such, the complexity of the problems is expected to be greater. We adopted the exact same protocol as for smaller datasets. However, to lower computing times, for datasets marked with $*$, the methods were run using 100 trees instead of 250 and the minimum number of samples required in an internal node was set to 10 in order to control complexity. Detailed results are provided in Table 8.5 and are summarized in Figure 8.2 and Table 8.6, respectively in terms of average rank (the critical difference at $\alpha = 0.05$ is now 2.9120) and Win/Draw/Loss statuses obtained with paired t-tests. A Friedman test (at $\alpha = 0.05$) still indicates that some methods are significantly different from the others.

As it may be observed from Figure 8.2, the average ranks of the methods are closer to each other than in the previous experiments, now ranging from 2.38 to 6.61, while they were previously ranging from 2.12 to 7. Methods are more connected by critical difference bars. This suggests that overall they behave more similarly to each other than before. General trends are nevertheless comparable to what we observed earlier. ET-based methods still seem to be the front-runners. From Figure 8.2, RS-ET, ET and RP-ET are in the top 4, while P-DT, RF and RP-DT remain in the second half of the ranking. Surprisingly however, RS-DT now comes right after RS-ET and ET and just before RP-ET whereas it ranked penultimate on the smaller datasets. Table 8.6 however suggests that RS-DT performs actually a little worse against RP-ET (1/10/2). All in all, it thus still seems beneficial to randomize split thresholds on the larger datasets.

trazida pelas escolhas aleatórias de amostras e características parece ser benéfica com árvores de decisão que não se beneficiam da randomização dos limites de discretização. O fato de as amostras de RF apresentarem características localmente não aparece mais aqui como uma vantagem sobre o RP (a RF é significativamente pior em 5 problemas e melhor em apenas um), provavelmente porque a diminuição do viés que ela proporciona não excede o aumento da variância em relação à seleção global de características.

8.2.3 Conjuntos de dados maiores

Embora as experiências anteriores tenham revelado resultados promissores, é justo perguntar se as conclusões que foram tiradas se manteriam e seriam generalizadas para problemas maiores, por exemplo, quando se trata de algumas características relevantes enterradas em centenas ou milhares de características não importantes (por exemplo, e m dados genómicos), ou quando se trata de muitas características correlacionadas (por exemplo, em imagens). Para investigar esta questão, foi realizada uma segunda bancada de experiências em 13 conjuntos de dados maiores (ver Tabela 8.4). Todos, exceto madelon, são dados reais. Em termos de dimensões, esses conjuntos de dados são muito maiores, variando de algumas centenas de amostras e milhares de recursos a milhares de amostras, mas centenas de recursos. Como tal, espera-se que a complexidade dos problemas seja maior. Adotamos exatamente o mesmo protocolo para conjuntos de dados menores. Porém, para diminuir o t empo de computação, para conjuntos de dados marcados com $*$, os métodos foram executados usando 100 árvores em vez de 250 e o número mínimo de amostras necessárias em um nó interno foi definido como 10 para controlar a complexidade. Os resultados detalhados são fornecidos na Tabela 8.5 e estão resumidos na Figura 8.2 e Tabela 8.6, respectivamente em termos de classificação média (a diferença crítica em $\alpha = 0.05$ é agora 2,9120) e status de Vitória/Empate/Perda obtidos com testes t pareados. Um teste de Friedman (em $\alpha = 0.05$) ainda indica que alguns métodos são significativamente diferentes de outros.

Como pode ser observado na Figura 8.2, os postos médios dos métodos estão mais próximos entre si do que nos experimentos anteriores, variando agora de 2,38 a 6,61, enquanto anteriormente variavam de 2,12 a 7. Os métodos estão mais conectados por barras de diferença crítica. Isto sugere que, em geral, eles se comportam de maneira mais semelhante entre si do que antes. As tendências gerais são, no entanto, comparáveis ao que observámos anteriormente. Os métodos baseados em ET ainda parecem ser os pioneiros. Na Figura 8.2, RS-ET, ET e RP-ET estão entre os 4 primeiros, enquanto P-DT, RF e RP-DT permanecem na segunda metade do ranking. Surpreendentemente, no entanto, o RS-DT vem agora logo depois do RS-ET e ET e logo antes do RP-ET, embora tenha ficado em penúltimo lugar nos conjuntos de dados menores. A Tabela 8.6, entretanto, sugere que o desempenho do RS-DT é, na verdade, um pouco pior em relação ao RP-ET (1/10/2). Em suma, ainda parece benéfico randomizar os limites de divisão nos conjuntos de dados maiores.

Dataset	N	p
CIFAR10*	60000	3072
MNIST3VS8	13966	784
MNIST4VS9	13782	784
MNIST*	70000	784
ISOLET	7797	617
ARCENE	900	10000
BREAST2	295	24496
MADELON	4400	500
MARTI0	500	1024
REGED0	500	999
SECOM	1567	591
TIS	13375	927
SIDO0*	12678	4932

Table 8.4: Large datasets.

Comparing ET-based variants, ET is no longer the best method on average, but RS-ET is (with 4/9/0 for RS-ET versus ET). This suggests than on larger datasets, picking features globally at random prior to the construction of the trees is as good, or even beat picking them locally at each node. Due to the quantitatively larger number of samples in a patch, and also to the larger number of redundant features expected in some large datasets (e.g., in CIFAR10 or MNIST), it is indeed less likely to build improper trees with strong biases. As a result, variance can be further decreased by sampling globally. In support of this claim, on a few problems such as ARCENE, BREAST2, or MADELON that contain many irrelevant features, ET remains the best method. In that case, it is indeed more likely to sample globally improper random patches, and hence to build improper trees. The average rank of RP-ET suggests that it performs worse than RS-ET and thus that there is some potential overfitting when tuning α_s in addition to α_f. This difference is however not confirmed in Table 8.6 where the accuracies of these two methods are shown to be never significantly different (0/13/0). RP-ET is also on a perfect par with ET (1/11/1). Among DT-based variants, RP-DT, which was the best performer on small datasets, is still ranked above RF and P-DT, but it is now ranked below RS-DT with a win/draw/loss of 0/11/2. This is again due to some overfitting.

While less conclusive than before, the results on larger datasets are consistent with what we observed earlier. In particular, they indicate that the Random Patches method (with ET) remains competitive with the best performers.

Conjunto de dados	N	p
cifar10*	60000	3072
mnist3vs8	13966	784
mnist4vs9	13782	784
mnista*	70000	784
isoleta	7797	617
arceno	900	10000
peito2	295	24496
madelão	4400	500
marti0	500	1024
reged0	500	999
secom	1567	591
TIS	13375	927
sido0*	12678	4932

Tabela 8.4: Grandes conjuntos de dados.

Comparando variantes baseadas em ET, ET não é mais o melhor método em média, mas RS-ET é (com 4/9/0 para RS-ET versus ET). Isso sugere que em conjuntos de dados maiores, escolher recursos global mente aleatoriamente antes da construção das árvores é tão bom, ou até mesmo melhor, selecioná-los localmente em cada nó. Devido ao número quantitativamente maior de amostras em um patch, e também ao maior número de características redundantes esperadas em alguns grandes conjuntos de dados (por exemplo, em cifar10 ou mnist), é de fato menos provável construir árvores impróprias com fortes vieses. Como resultado, a variância pode ser ainda mais diminuída através da amostragem global. Em apoio a esta afirmação, em alguns problemas como arcene, breast2 ou madelon que contêm muitas características irrelevantes, a TE continua a ser o melhor método. Nesse caso, é de fato mais provável coletar amostras aleatórias globalmente impróprias e, portanto, construir árvores impróprias. A classificação média do RP-ET sugere que ele tem um desempenho pior que o RS-ET e, portanto, há algum overfitting potencial ao ajustar α_s além de α_f. Esta diferença, no entanto, não é confirmada na Tabela 8.6, onde as precisões destes dois métodos nunca são significativamente diferentes (0/13/0). RP-ET também está em perfeita sintonia com ET (1/11/1). Entre as variantes baseadas em DT, RP-DT, que teve o melhor desempenho em pequenos conjuntos de dados, ainda está classificado acima de RF e P-DT, mas agora está classificado abaixo de RS-DT com uma vitória/empate/derrota de 0/11/2. Isso se deve novamente a algum overfitting.

Embora menos conclusivos do que antes, os resultados em conjuntos de dados maiores são consistentes com o que observamos anteriormente. Em particular, indicam que o método Random Patches (com ET) permanece competitivo com os melhores desempenhos.

Validation	RF	ET	P-DT	P-ET	RS-DT	RS-ET	RP-DT	RP-ET
CIFAR10	45.16 (6)	**46.17** (1)	44.92 (7)	44.88 (8)	45.83 (5)	46.09 (3)	45.89 (4)	46.13 (2)
MNIST3V8	98.42 (7)	98.77 (5)	97.63 (8)	98.57 (6)	98.79 (4)	98.86 (2)	98.81 (3)	**98.87** (1)
MNIST4V9	98.47 (7)	98.82 (3)	97.39 (8)	98.47 (6)	98.77 (5)	98.89 (2)	98.80 (4)	**98.91** (1)
MNIST	96.14 (7)	96.56 (3)	94.35 (8)	96.19 (6)	96.52 (5)	96.57 (2)	96.53 (4)	**96.57** (1)
ISOLET	93.96 (7)	95.07 (3)	90.90 (8)	94.23 (6)	94.38 (5)	95.15 (2)	94.47 (4)	**95.20** (1)
ARCENE	73.84 (7)	77.76 (3)	73.76 (8)	75.12 (6)	76.40 (5)	77.60 (4)	79.44 (2)	**80.00** (1)
BREAST2	69.64 (6)	69.91 (4)	70.54 (3)	69.83 (5)	69.59 (7)	69.54 (8)	**72.59** (1)	71.62 (2)
MADELON	76.12 (8)	**81.41** (1)	78.48 (7)	81.06 (5)	80.68 (6)	81.29 (3)	81.18 (4)	81.36 (2)
MARTI	87.66 (8)	87.92 (6)	88.08 (4)	88.09 (3)	87.77 (7)	88.00 (5)	88.19 (2)	**88.20** (1)
REGED	98.00 (6)	98.46 (2)	97.08 (8)	98.24 (5)	98.00 (7)	98.41 (3)	98.40 (4)	**98.57** (1)
SECOM	93.37 (5)	93.36 (6)	93.34 (8)	93.42 (2)	93.35 (7)	93.40 (4)	93.41 (3)	**93.45** (1)
TIS	91.81 (5)	91.53 (7)	91.59 (6)	91.50 (8)	92.04 (3)	91.97 (4)	**92.26** (1)	92.06 (2)
SIDO	97.46 (5)	97.44 (6)	97.36 (8)	97.36 (7)	97.47 (3)	97.46 (4)	**97.52** (1)	97.52 (2)
Avg. rank	6.4615	3.8461	7.0	5.6153	5.3076	3.5384	2.8461	1.3846
Test	RF	ET	P-DT	P-ET	RS-DT	RS-ET	RP-DT	RP-ET
CIFAR10	44.87 (7)	45.95 (2)	44.75 (8)	44.95 (6)	45.86 (4)	**46.02** (1)	45.74 (5)	45.93 (3)
MNIST3V8	98.31 (7)	98.64 (5)	97.47 (8)	98.48 (6)	98.70 (2)	**98.71** (1)	98.68 (4)	98.68 (3)
MNIST4V9	98.33 (6)	98.68 (2)	97.16 (8)	98.32 (7)	98.59 (4)	**98.69** (1)	98.59 (5)	98.67 (3)
MNIST	96.10 (7)	96.47 (3)	94.30 (8)	96.18 (6)	96.47 (4)	**96.49** (1)	96.46 (5)	96.48 (2)
ISOLET	93.87 (7)	**94.95** (1)	90.71 (8)	94.15 (6)	94.33 (4)	94.94 (2)	94.29 (5)	94.88 (3)
ARCENE	71.04 (4)	**72.56** (1)	66.16 (8)	68.56 (7)	71.52 (3)	72.24 (2)	69.44 (6)	70.00 (5)
BREAST2	65.86 (6)	**66.16** (1)	65.54 (8)	65.59 (7)	66.13 (2)	65.91 (5)	65.94 (4)	66.08 (3)
MADELON	75.33 (8)	**80.83** (1)	77.86 (7)	80.78 (2)	80.06 (6)	80.59 (3)	80.06 (5)	80.42 (4)
MARTI	87.74 (8)	87.85 (6)	**88.24** (1)	88.17 (2)	87.82 (7)	88.06 (4)	88.08 (3)	88.01 (5)
REGED	97.60 (5)	**98.20** (1)	96.40 (8)	97.92 (3)	97.39 (6)	98.00 (2)	97.32 (7)	97.82 (4)
SECOM	93.30 (7)	93.22 (8)	**93.38** (1)	93.30 (6)	93.37 (2)	93.31 (4)	93.33 (3)	93.31 (5)
TIS	91.68 (5)	91.42 (7)	91.49 (6)	91.40 (8)	**92.05** (1)	91.91 (3)	92.05 (2)	91.90 (4)
SIDO	97.33 (4)	97.33 (3)	97.26 (7)	97.26 (8)	**97.35** (1)	97.34 (2)	97.33 (5)	97.32 (6)
Avg. rank	6.2307	3.1538	6.6153	5.6923	3.5384	2.3846	4.5384	3.8461

Table 8.5: Accuracy on larger datasets (in %).

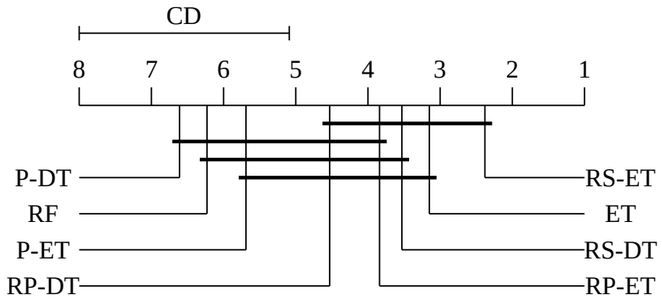


Figure 8.2: Average ranks of all methods on larger datasets.

Validação	RF	ET	P-DT	P-ET	RS-DT	RS-ET	RP-DT	RP-ET
cifar10	45,16 (6)	**46,17 (1)**	44,92 (7)	44,88 (8)	45,83 (5)	46,09 (3)	45,89 (4)	46,13 (2)
mnist3v8	98,42 (7)	98,77 (5)	97,63 (8)	98,57 (6)	98,79 (4)	98,86 (2)	98,81 (3)	**98,87 (1)**
mnist4v9	98,47 (7)	98,82 (3)	97,39 (8)	98,47 (6)	98,77 (5)	98,89 (2)	98,80 (4)	**98,91 (1)**
mnista	96,14 (7)	96,56 (3)	94,35 (8)	96,19 (6)	96,52 (5)	96,57 (2)	96,53 (4)	**96,57 (1)**
isoleta	93,96 (7)	95,07 (3)	90,90 (8)	94,23 (6)	94,38 (5)	95,15 (2)	94,47 (4)	**95,20 (1)**
arceno	73,84 (7)	77,76 (3)	73,76 (8)	75,12 (6)	76,40 (5)	77,60 (4)	79,44 (2)	**80,00 (1)**
peito2	69,64 (6)	69,91 (4)	70,54 (3)	69,83 (5)	69,59 (7)	69,54 (8)	**72,59 (1)**	71,62 (2)
madelão	76,12 (8)	**81,41 (1)**	78,48 (7)	81,06 (5)	80,68 (6)	81,29 (3)	81,18 (4)	81,36 (2)
martí	87,66 (8)	87,92 (6)	88,08 (4)	88,09 (3)	87,77 (7)	88,00 (5)	88,19 (2)	**88,20 (1)**
regado	98,00 (6)	98,46 (2)	97,08 (8)	98,24 (5)	98,00 (7)	98,41 (3)	98,40 (4)	**98,57 (1)**
secom	93,37 (5)	93,36 (6)	93,34 (8)	93,42 (2)	93,35 (7)	93,40 (4)	93,41 (3)	**93,45 (1)**
TIS	91,81 (5)	91,53 (7)	91,59 (6)	91,50 (8)	92,04 (3)	91,97 (4)	**92,26 (1)**	92,06 (2)
SIDO	97,46 (5)	97,44 (6)	97,36 (8)	97,36 (7)	97,47 (3)	97,46 (4)	**97,52 (1)**	97,52 (2)
Média classificação	6.4615	3.8461	7.0	5.6153	5.3076	3.5384	2.8461	1.3846
Test	RF	ET	P-DT	P-ET	RS-DT	RS-ET	RP-DT	RP-ET
cifar10	44,87 (7)	45,95 (2)	44,75 (8)	44,95 (6)	45,86 (4)	**46,02 (1)**	45,74 (5)	45,93 (3)
mnist3v8	98,31 (7)	98,64 (5)	97,47 (8)	98,48 (6)	98,70 (2)	**98,71 (1)**	98,68 (4)	98,68 (3)
mnist4v9	98,33 (6)	98,68 (2)	97,16 (8)	98,32 (7)	98,59 (4)	**98,69 (1)**	98,59 (5)	98,67 (3)
mnista	96,10 (7)	96,47 (3)	94,30 (8)	96,18 (6)	96,47 (4)	**96,49 (1)**	96,46 (5)	96,48 (2)
isoleta	93,87 (7)	**94,95 (1)**	90,71 (8)	94,15 (6)	94,33 (4)	94,94 (2)	94,29 (5)	94,88 (3)
arceno	71,04 (4)	**72,56 (1)**	66,16 (8)	68,56 (7)	71,52 (3)	72,24 (2)	69,44 (6)	70,00 (5)
peito2	65,86 (6)	**66,16 (1)**	65,54 (8)	65,59 (7)	66,13 (2)	65,91 (5)	65,94 (4)	66,08 (3)
madelão	75,33 (8)	**80,83 (1)**	77,86 (7)	80,78 (2)	80,06 (6)	80,59 (3)	80,06 (5)	80,42 (4)
martí	87,74 (8)	87,85 (6)	**88,24 (1)**	88,17 (2)	87,82 (7)	88,06 (4)	88,08 (3)	88,01 (5)
regado	97,60 (5)	**98,20 (1)**	96,40 (8)	97,92 (3)	97,39 (6)	98,00 (2)	97,32 (7)	97,82 (4)
secom	93,30 (7)	93,22 (8)	**93,38 (1)**	93,30 (6)	93,37 (2)	93,31 (4)	93,33 (3)	93,31 (5)
TIS	91,68 (5)	91,42 (7)	91,49 (6)	91,40 (8)	**92,05 (1)**	91,91 (3)	92,05 (2)	91,90 (4)
SIDO	97,33 (4)	97,33 (3)	97,26 (7)	97,26 (8)	**97,35 (1)**	97,34 (2)	97,33 (5)	97,32 (6)
Média classificação	6.2307	3.1538	6.6153	5.6923	3.5384	2.3846	4.5384	3.8461

Tabela 8.5: Precisão em conjuntos de dados maiores (em %).

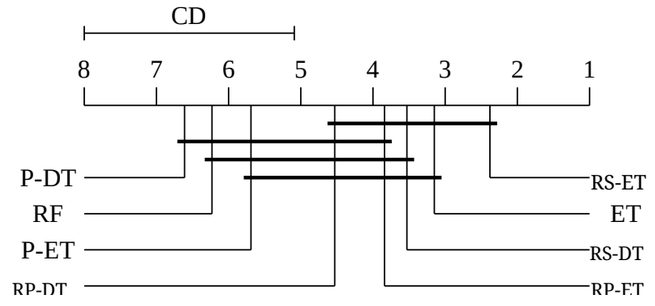


Figura 8.2: Classificações médias de todos os métodos em conjuntos de dados maiores.

	RF	ET	P-DT	P-ET	RS-DT	RS-ET	RP-DT	RP-ET
RF	—	1/5/7	8/3/2	2/6/5	0/6/7	0/5/8	0/6/7	0/6/7
ET	7/5/1	—	9/2/2	7/6/0	3/7/3	0/9/4	5/6/2	1/11/1
P-DT	2/3/8	2/2/9	—	1/5/7	0/3/10	0/3/10	1/3/9	0/4/9
P-ET	5/6/2	0/6/7	7/5/1	—	0/6/7	0/5/8	2/5/6	1/5/7
RS-DT	7/6/0	3/7/3	10/3/0	7/6/0	—	1/8/4	2/11/0	1/10/2
RS-ET	8/5/0	4/9/0	10/3/0	8/5/0	4/8/1	—	4/8/1	0/13/0
RP-DT	7/6/0	2/6/5	9/3/1	6/5/2	0/11/2	1/8/4	—	1/9/3
RP-ET	7/6/0	1/11/1	9/4/0	7/5/1	2/10/1	0/13/0	3/9/1	—

Table 8.6: Pairwise t-test comparisons on larger datasets.

8.2.4 *Conclusions*

Overall, this extensive experimental study reveals many interesting results. The first and foremost result is that ensembles of randomized trees nearly always beat ensembles of standard decision trees. As off-the-shelf methods, we advocate that ensembles of such trees should be preferred to ensembles of decision trees. In particular, these results show that the well-known Random Forest algorithm does not compete with the best performers. Far more important to our concern though, this study validates our RP approach. Building ensembles (of ET) on random patches of data is competitive in terms of accuracy. Overall, there is no strong statistical evidence that the method performs less well, but there is also no conclusive evidence that it significantly improves performance. Yet, results show that RP is often as good as the very best methods. Regarding the shape of the random patches, the strategy behind Pasting (i.e., α_s free and $\alpha_f = 1.0$) proved to be (very) ineffective on many datasets while the Random Subspace algorithm (i.e., $\alpha_s = 1.0$ and α_f free) always ranked among the very best performers. On average, RS indeed came in second on the small datasets and in first on the larger datasets, which tends to indicate that sampling features is crucial in terms of accuracy. As for patches of freely adjustable size (i.e., using RP), they showed to be slightly sensitive to overfitting but proved to remain closely competitive with the very best methods. In addition, these results also suggest that sampling features globally, once and for all, prior to the construction of a (randomized) decision tree, does not actually impair performance. For instance, RS-ET or RP-ET are indeed not strongly worse, nor better, than ET, in which candidates features are re-sampled locally at each node.

8.3 ON MEMORY

Section 8.2 reveals that building an ensemble of base estimators on random patches, instead of the whole data, is a competitive strategy.

	RF	ET	P-DT	P-ET	RS-DT	RS-ET	RP-DT	RP-ET
RF	—	1/5/7	8/3/2	2/6/5	0/6/7	0/5/8	0/6/7	0/6/7
ET	05/07/1	—	02/09/2	7/6/0	3/7/3	0/9/4	5/6/2	11/01/1
P-DT	2/3/8	2/2/9	—	1/5/7	03/010	03/010	1/3/9	0/4/9
P-ET	5/6/2	0/6/7	05/07/1	—	0/6/7	0/5/8	2/5/6	1/5/7
RS-DT	7/6/0	3/7/3	3/10/0	7/6/0	—	1/8/4	11/02/0	1/10/2
RS-ET	05/08/0	4/9/0	3/10/0	05/08/0	08/04/1	—	08/04/1	0/13/0
RP-DT	7/6/0	2/6/5	03/09/01	6/5/2	0/11/2	1/8/4	—	1/9/3
RP-ET	7/6/0	11/01/1	04/09/0	05/07/1	10/02/1	0/13/0	09/03/1	—

Tabela 8.6: Comparações do teste t pareado em conjuntos de dados maiores.

8.2.4 Conclusões

No geral, este extenso estudo experimental revela muitos resultados interessantes. O primeiro e mais importante resultado é que os conjuntos de árvores aleatórias quase sempre superam os conjuntos de árvores de decisão padrão. Como métodos disponíveis no mercado, defendemos que conjuntos de tais árvores devem ser preferidos a conjuntos de árvores de decisão. Em particular, estes resultados mostram que o conhecido algoritmo Random Forest não compete com os melhores desempenhos. Muito mais importante para a nossa preocupação, porém, este estudo valida nossa abordagem de RP. Construir conjuntos (de ET) em frag mentos aleatórios de dados é competitivo em termos de precisão. No geral, não há provas estatísticas fortes de que o método tenha um desempenho pior, mas também não há provas conclusivas de que melhore significativamente o desempenho. No entanto, os resultados mostram que a PR é frequentemente tão boa quanto os melhores métodos. Em relação ao formato dos patches aleatórios, a estratégia por trás do Pasting (ou seja, α_s free e $\alpha_f = 1.0$) provou ser (muito) ineficaz em muitos conjuntos de dados, enquanto o algoritmo Random Subspace (ou seja, $\alpha_s = 1.0$ e α_f free) sempre foi classificado entre os de melhor desempenho. Em média, o RS ficou de facto em segundo lugar nos conjuntos de dados pequenos e e m primeiro lugar nos conjuntos de dados maiores, o que tende a indicar que as características de amostragem são cruciais em termos de precisão. Quanto aos patches de tamanho livremente ajustável (isto é, usando RP), eles mostraram ser ligeiramente sensíveis ao overfitting, mas provaram permanecer estreitamente competitivos com os melhores métodos. Além disso, esses resultados também sugerem que a amostragem global de recursos, de uma vez por todas, antes da construção de uma árvore de decisão (randomizada), na verdade não prejudica o desempenho. Por exemplo, RS-ET ou RP-ET não são de fato nem piores nem melhores que ET, nos quais as características candidatas são reamostradas localmente em cada nó.

8.3 na memória

A Seção 8.2 revela que construir um conjunto de estimadores básicos em manchas aleatórias, em vez de todos os dados, é uma estratégia competitiva.

In the context of big data, that is when the size of the dataset is far bigger than the available memory, this suggests that using random parts of the data of the appropriate size to build each base estimator would likely result in an ensemble which is actually as good as if the whole data could have been loaded and used.

Formally, we assume a general framework where the number of data units that can be loaded at once into memory is constrained to be lower than a given threshold μ_{max}. Not considering on-line algorithms within the scope of this study, μ_{max} can hence be viewed as the total units of data allowed to be used to build a single base estimator. In the context of our sampling methods, the amount of memory required for a patch is given by $(\alpha_s N)(\alpha_f p)$ and thus constraining memory by μ_{max} is equivalent to constraining the relative patch size $\alpha_s\alpha_f$ to be lower than $\mu'_{max} = \mu_{max}/(Np)$. While simplistic[1], this framework has the advantage of clearly addressing one of the main difficulties behind big data, that is the lack of fast memory. Yet, it is also relevant in other contexts, for example when data is costly to access (e.g., on remote locations) or when algorithms are run on embedded systems with strong memory constraints.

In Section 8.3.1, we first study the effects of α_s and α_f on the accuracy of the resulting ensemble and show that it is problem and base estimator dependent. Second, we show that the memory requirements, i.e., the relative size $\alpha_s\alpha_f$ of the random patches, can often be drastically reduced without significantly degrading the performance of the ensemble (Section 8.3.2). Third, because the sensitivity of the ensemble to α_s and α_f is problem and base estimator specific, we show that under very strong memory constraints adjusting both parameters at the same time, as RP does, is no longer merely as good but actually significantly better than other ensemble methods (Section 8.3.3).

8.3.1 *Sensitivity to α_s and α_f*

Let us first consider and analyze the triplets

$$\{(\alpha_s, \alpha_f, \mathrm{Acc}_{\mathcal{L}}(\alpha_s, \alpha_f)) | \forall \alpha_s, \alpha_f\} \tag{8.1}$$

for various problems, where $\mathrm{Acc}_{\mathcal{L}}(\alpha_s, \alpha_f)$ is the average test accuracy of an ensemble built on random patches of size $\alpha_s\alpha_f$ (using the same protocol as previously) on the dataset $\mathcal{L}$.

As Figure 8.3 illustrates for six datasets, the surfaces defined by these points vary significantly from one problem to another. We observed four main trends. In Figures 8.3a, and 8.3b (resp., 8.3c), accuracy increases with α_s (resp., α_f) while adjusting α_f (resp., α_s) has no or limited impact. In Figure 8.3d, the best strategy is to increase both α_s and α_f. Finally, in Figures 8.3e and 8.3f, the surface features

1 e.g., the quantity of memory used by the estimator itself is not taken into account.

No contexto de big data, isto é, quando o tamanho do conjunto de dados é muito maior do que a memória disponível, isto sugere que a utilização de partes aleatórias dos dados do tamanho apropriado para construir cada estimador de base provavelmente resultaria num conjunto que é na verdade tão bom como se todos os dados pudessem ter sido carregados e utilizados.

Formalmente, assumimos uma estrutura geral onde o número de unidades de dados que podem ser carregadas de uma só vez na memória é limitado a ser inferior a um determinado limite μ_{max}. Não considerando algoritmos on-line no escopo deste estudo, μ_{max} pode, portanto, ser visto como o total de unidades de dados que podem ser usadas para construir um estimador de base única. No contexto dos nossos métodos de amostragem, a quantidade de memória necessária para um patch é dada por $(\alpha_s N)(\alpha_f p)$ e, portanto, restringir a memória por μ_{max} é equivalente a restringir o tamanho relativo do patch $\alpha_s\alpha_f$a um valor inferior a $\mu'_{max} = \mu_{max}/(Np)$. Embora simplista,[1], essa estrutura tem a vantagem de abordar claramente uma das principais dificuldades por trás do big data: a falta de memória rápida. No entanto, também é relevante em outros contextos, por exemplo, quando o acesso aos dados é caro (por exemplo, em locais remotos) ou quando os algoritmos são executados em sistemas embarcados com fortes restrições de memória.

Na Seção 8.3.1, primeiro estudamos os efeitos de α_s e α_f na precisão do conjunto resultante e mostramos que ele depende do problema e do estimador base. Em segundo lugar, mostramos que os requisitos de memória, ou seja, o tamanho relativo $\alpha_s\alpha_f$ dos patches aleatórios, muitas vezes podem ser drasticamente reduzidos sem degradar significativamente o desempenho do conjunto (Seção 8.3.2). Terceiro, como a sensibilidade do conjunto a α_s e α_f é específica do problema e do estimador de base, mostramos que, sob restrições de memória muito fortes, ajustar ambos os parâmetros ao mesmo tempo, como faz RP, não é mais apenas tão bom, mas na verdade significativamente melhor que outros métodos de conjunto (Seção 8.3.3).

8.3.1 Sensibilidade a α_s e α_f

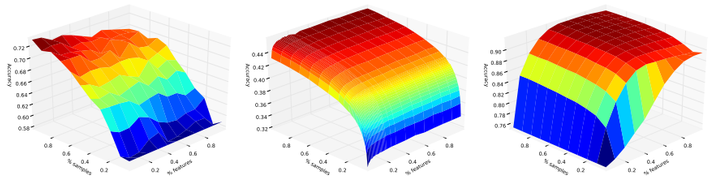
Vamos primeiro considerar e analisar os trigêmeos

$$\{(\alpha_s, \alpha_f, \mathrm{Acc}_{\mathcal{L}}(\alpha_s, \alpha_f)) | \forall \alpha_s, \alpha_f\} \tag{8.1}$$

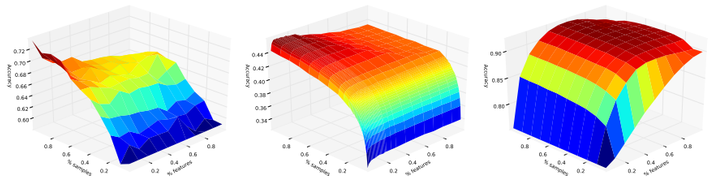
para vários problemas, onde $\mathrm{Acc}_{\mathcal{L}}(\alpha_s, \alpha_f)$ é a precisão média do teste de um conjunto construído em patches aleatórios de tamanho $\alpha_s\alpha_f$ (usando o mesmo protocolo anterior) no conjunto de dados $\mathcal{L}$.

Como a Figura 8.3 ilustra para seis conjuntos de dados, as superfícies definidas por estes pontos variam significativamente de um problema para outro. Observamos quatro tendências principais. Nas Figuras 8.3a e 8.3b (resp., 8.3c), a precisão aumenta com α_s (resp., α_f) enquanto o ajuste α_f (resp., α_s) tem impacto nulo ou limitado. Na Figura 8.3d, a melhor estratégia é aumentar α_s e α_f. Finalmente, nas Figuras 8.3e e 8.3f, as características da superfície

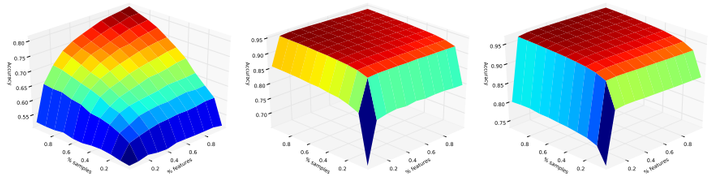
1, por exemplo, a quantidade de memória utilizada pelo próprio estimador não é levada em consideração.



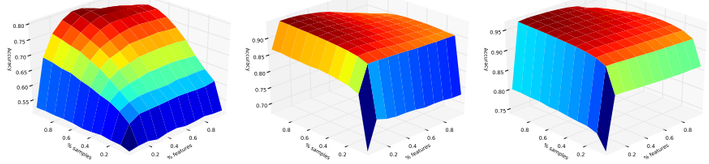
(a) ARCENE (RP-ET) (b) CIFAR10 (RP-ET) (c) TIS (RP-ET)



(a) ARCENE (RP-DT) (b) CIFAR10 (RP-DT) (c) TIS (RP-DT)



(d) MADELON (RP-ET) (e) ISOLET (RP-ET) (f) MNIST3VS8 (RP-ET)

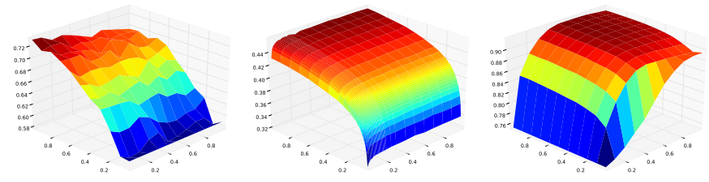


(d) MADELON (RP-DT) (e) ISOLET (RP-DT) (f) MNIST3VS8 (RP-DT)

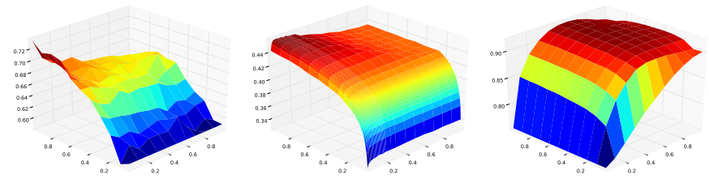
Figure 8.3: Learning surfaces.

plateaus, which means that beyond some threshold, increasing α_s or α_f does not yield any significant improvement. Interestingly, in most of the cases, the optimum corresponds to a value $\alpha_s\alpha_f$ much smaller than 1.

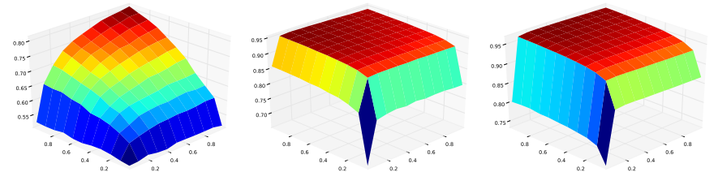
The choice of the base estimators does not have a strong impact on the aspect of the curves (compare the 1st and 3rd rows of subfigures in Figure 8.3 with those in the 2nd and 4th rows). The only difference is the decrease of the accuracy of RP-DT when α_s and α_f grow towards 1.0. Indeed, since the only source of randomization in RP-DT is patch selection, it yields in this case ensembles of identical trees and therefore amounts to building a single tree on the whole dataset. By contrast, because of the extra-randomization of the split



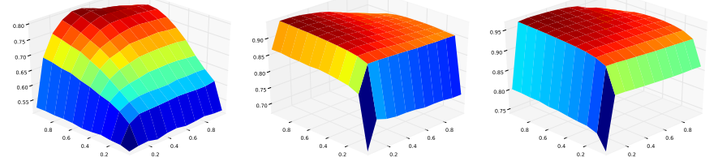
(a) arceno (RP-ET) (b) cifar10 (RP-ET) (c) isto (RP-ET)



(a) arceno (RP-DT) (b) cifar10 (RP-DT) (c) isto (RP-DT)



(d) madelão (RP-ET) (e) isoleta (RP-ET) (f) mnist3vs8 (RP-ET)



(d) madelão (RP-DT) (e) isoleta (RP-DT) (f) mnist3vs8 (RP-DT)

Figura 8.3: Superfícies de aprendizage
m.

platôs, o que significa que além de algum limite, aumentar α_s ouα_f não produz nenhuma melhoria significativa. Curiosamente, na maioria dos casos, o ótimo corresponde a um valor $\alpha_s\alpha_f$ muito menor que 1.

A escolha dos estimadores base não tem um forte impacto no aspecto das curvas (compare as linhas 1st e 3rd das subfiguras da Figura 8.3 co
m as das linhas 2nd e 4th). A única diferença é a diminuição da precisão do RP-DT quando α_s e $\alpha_f$crescem para 1,0. Na verdade, uma vez que a única fonte de randomização no RP-DT é a seleção de patches, neste caso produz-se conjuntos de árvores idênticas e, portanto, equivale à construção de uma única árvore em todo o conjunto de dados. Por outro lado, devido à extra-randomização da divisão

thresholds in ET, there is typically no drop of accuracy for RP-ET when α_s and α_f grow to 1.0.

Overall, this analysis suggests that not only the best pair $\alpha_s\alpha_f$ depends on the problem, but also that the sensitivity of the ensemble to changes to the size of a random patch is both problem and base estimator specific. As a result, these observations advocate for a method that could favor α_s, α_f or both, and do so appropriately given the base estimator.

8.3.2 *Memory reduction, without significant loss*

We proceed to study in this section the actual size of the random patches when the values of α_s and α_f are tuned using an independent validation set. Our results are summarized in Figure 8.4a. Each ellipse corresponds to one of the 29 datasets of our benchmark, whose center is located at $(\overline{\alpha_s}, \overline{\alpha_f})$ (i.e., the average parameter values over the 50 runs) and whose semi-axes correspond to the standard deviations of α_s and α_f. Any point in the plot corresponds to a pair (α_s, α_f) and thus to a relative consumption $\mu' = \alpha_s\alpha_f$ of memory. To ease readability, level curves are plotted for $\mu' = 0.01, 0.1, ..., 0.9$. In the right part of the figure, the histogram counts the number of datasets such that $\overline{\alpha_s} \cdot \overline{\alpha_f}$ falls in the corresponding level set.

Figure 8.4a corroborates our previous discussion. On some datasets, it is better to favor α_s while on some other increasing α_f is a better strategy. The various sizes of the ellipses also confirm that the sensitivity to variations of α_s and α_f is indeed problem-specific. The figure also clearly highlights the fact that, even under no memory constraint, the optimal patches rarely consume the whole memory. A majority of ellipses indeed lie below the level set $\mu' = 0.5$ and only a couple of them are above $\mu' = 0.75$. With respect to ET or RF for which the base estimators are all built on the whole dataset, this means that ensembles of patches are not only as competitive but also less memory greedy. In addition, the figure also points out the difference between RP-ET and RP-DT as discussed in the previous section. To ensure diversity, RP-DT is constrained to use smaller patches than RP-ET, hence explaining why the ellipses in red are on average below those in blue. While RP-DT proved to be a bit less competitive in terms of accuracy, this indicates on the other hand that RP-DT may actually be more interesting from a memory consumption point of view.

In Section 8.3.1, we observed plateaus or very gentle slopes around the optimal pair (α_s, α_f). From a memory point of view, this suggests that the random patches are likely to be reducible without actually degrading the accuracy of the resulting ensemble. Put otherwise, our interest is to find the smallest size $\alpha_s\alpha_f$ such that the accuracy of the resulting ensemble is not significantly worse than an ensemble built without such constraint. To that end, we study the extent at which the

limites em ET, normalmente não há queda na precisão do RP-ET quando α_s e α_f crescem para 1,0.

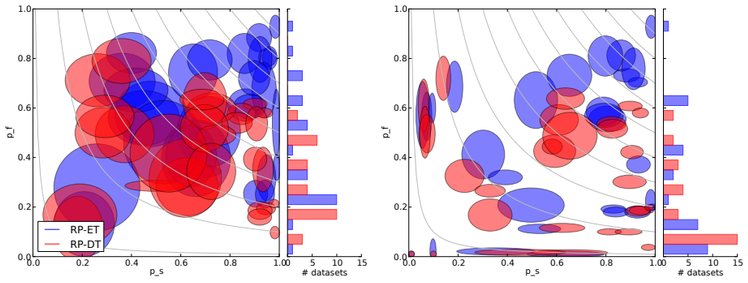
No geral, esta análise sugere que não apenas o melhor par $\alpha_s\alpha_f$ depende do problema, mas também que a sensibilidade do conjunto às mudanças no tamanho de um patch aleatório é específica tanto do problema quanto do estimador base. Como resultado, essas observações defendem um método que poderia favorecer α_s, α_f ou ambos, e fazê-lo de forma adequada, dado o estimador base.

8.3.2 Redução de memória, sem perda significativa

Prosseguimos estudando nesta seção o tamanho real dos patches aleatórios quando os valores de α_s e α_f são ajustados usando um c onjunto de validação independente. Nossos resultados estão resumidos na Figura 8.4a. Cada elipse corresponde a um dos 29 conjuntos de dados do nosso benchmark, cujo centro está localizado em (α_s, α_f) (ou seja, os valores médios dos parâmetros ao longo das 50 execuções) e cujos semieixos correspondem aos desvios padrão de α_s e α_f. Qualquer ponto no gráfico corresponde a um par (α_s, α_f)e, portanto, a um consumo relativo $\mu' = \alpha_s\alpha_f$ de memória. Para facilitar a legibilidade, as curvas de nível são plotadas para $\mu' = 0.01$, 0,1, ...,0,9. Na parte direita da figura, o histograma conta o número de conjuntos de dados de modo que $\alpha_s \cdot \alpha_f$ caia no conjunto de níveis correspondente.

A Figura 8.4a corrobora nossa discussão anterior. Em alguns c onjuntos de dados, é melhor favorecer α_s enquanto em outros, aumentar α_f é uma estratégia melhor. Os vários tamanhos das elipses também confirmam que a sensibilidade às variações de α_s e α_f é de fato específica do problema. A figura também destaca claramente o fato de que, mesmo sem restrição de memória, os patches ideais raramente consomem toda a memória. A maioria das elipses realmente está abaixo do nível definido $\mu' = 0.5$ e apenas algumas delas estão acima de $\mu' = 0.75$. No que diz respeito a ET ou RF para os quais os estimadores de base são todos construídos em todo o conjunto de dados, isto significa que os conjuntos de patches não são apenas tão competitivos, mas também menos ávidos de memória. Além disso, a figura também aponta a diferença entre RP-ET e RP-DT conforme discutido na seção anterior. Para garantir a diversidade, o RP-DT é obrigado a usar manchas menores do que o RP-ET, explicando assim porque as elipses em vermelho estão, em média, abaixo daquelas e m azul. Embora o RP-DT tenha provado ser um pouco menos competitivo em termos de precisão, isso indica, por outro lado, que o RP-DT pode na verdade ser mais interessante do ponto de vista do consumo de memória.

Na Seção 8.3.1, observamos platôs ou declives muito suaves em torno do par ideal (α_s, α_f). Do ponto de vista da memória, isso sugere que as manchas aleatórias provavelmente serão redutíveis sem realmente degradar a precisão do conjunto resultante. Dito de outra forma, nosso interesse é encontrar o menor tamanho $\alpha_s\alpha_f$ tal que a precisão do conjunto resultante não seja significativamente pior do que um conjunto construído sem tal restrição. Para tanto, estudaremos até que ponto o

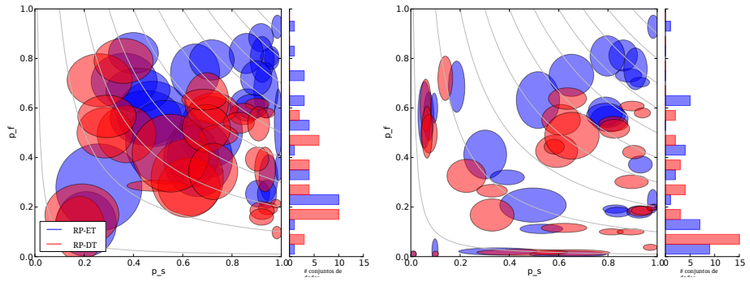


(a) Original patches (b) Reduced patches

Figure 8.4: Optimal sizes of the random patches on our benchmark.

constraint $\alpha_s \alpha_f < \mu'_{max}$ can be strengthened without any significant drop in accuracy. If μ'_{max} can be reduced significantly then it would indeed mean that even when only small parts of the data are actually used to build single base estimators, competitive performance can still be achieved.

Figure 8.4b summarizes our results. For all datasets, μ'_{max} was set to the lowest value such that it cannot be statistically detected that the average accuracy of the resulting ensemble is different from the average accuracy of an ensemble built with no memory constraint (at $\alpha = 0.05$). With regard to Figure 8.4a, the shift of most ellipses to lower memory level sets confirm our first intuition. In many cases, the size of the random patches can indeed be reduced, often drastically, without significant decrease of accuracy. For more than half of the datasets, memory can indeed be decreased to $\mu' = 0.1$ or $\mu' = 0.2$. In other words, building trees on small parts of the data (i.e., 10% or 20% of the original dataset) is, for more than half of the datasets, enough to reach competitive accuracy. Also, the sensitivity to α_s and α_f is now even more patent. Some ensembles use very few samples ($\alpha_s < 0.1$) but with many features, while other uses many samples with few features ($\alpha_f < 0.1$). Again, from a memory point of view, RP-DT appears to be more interesting than RP-ET. The memory reduction is larger, as the histogram indicates. Optimized splits in the decision trees may indeed lead to a better exploitation of the data, hence to a potentially larger reduction of memory. In conclusion, while not detecting significant differences in accuracy does not allow to conclude that the performances are truly similar, these figures at least illustrate that memory requirements can be drastically reduced without apparent loss in accuracy.



(a) Patches originais (b) Manchas reduzidas

Figura 8.4: Tamanhos ideais dos patches aleatórios em nosso benchmark.

a restrição $\alpha_s \alpha_f < \mu'_{max}$ pode ser reforçada sem qualquer queda significativa na precisão. Se μ'_{max} puder ser reduzido significativamente, isso significaria de fato que mesmo quando apenas pequenas partes dos dados são realmente usadas para construir estimadores de base única, o desempenho competitivo ainda pode ser alcançado.

A Figura 8.4b resume nossos resultados. Para todos os conjuntos de dados, μ'_{max} foi definido com o valor mais baixo, de modo que não pode ser detectado estatisticamente que a precisão média do conjunto r esultante é diferente da precisão média de um conjunto construído sem restrição de memória (em $\alpha = 0{,}05$). No que diz respeito à Figura 8.4a, a mudança da maioria das elipses para conjuntos de níveis de memória mais baixos confirma a nossa primeira intuição. Em muitos casos, o tamanho dos patches aleatórios pode de fato ser reduzido, muitas vezes drasticamente, sem diminuição significativa da precisão. Para mais da metade dos conjuntos de dados, a memória pode realmente ser reduzida para $\mu' = 0.1$ ou $\mu' = 0.2$. Em outras palavras, construir árvores em pequenas partes dos dados (ou seja, 10% ou 20% do conjunto de dados original) é, para mais da metade dos conjuntos de dados, suficiente para alcançar precisão competitiva. Além disso, a sensibilidade para α_s e α_f é agora ainda mais evidente. Alguns conjuntos usam muito poucas a mostras ($\alpha_s < 0.1$), mas com muitos recursos, enquanto outros usam muitas amostras com poucos recursos ($\alpha_f < 0.1$). Novamente, do ponto de vista da memória, o RP-DT parece ser mais interessante que o RP-ET. A redução de memória é maior, conforme indica o histograma. Divisões otimizadas nas árvores de decisão podem de fato levar a uma melhor exploração dos dados e, portanto, a uma redução potencialmente maior de memória. Em conclusão, embora a não detecção de diferenças significativas na precisão não permita concluir que os desempenhos seja m verdadeiramente semelhantes, estes números pelo menos ilustram que os requisitos de memória podem ser drasticamente reduzidos sem perda aparente de precisão.

8.3.3 *Memory reduction, with loss*

The previous section has shown that the memory consumption can be reduced up to some threshold μ'_{max} with no significant loss in accuracy. In this section we now look at the accuracy of the resulting ensemble when μ'_{max} is further decreased. We argue that with severe constraints, and because datasets have all a different sensitivity, it is even more crucial to better exploit data and thus to find the right trade-off between both α_s and α_f, as only RP can.

To illustrate our point, Figure 8.5 compares for 6 representative datasets the accuracy of the methods with respect to the memory constraint $\alpha_s\alpha_f < \mu'_{max}$. A plain line indicates that the generalization error of the best resulting ensemble under memory constraint μ'_{max} is significantly (at $\alpha = 0.05$) worse on the test sets than when there is no constraint (i.e., $\mu'_{max} = 1$). A dotted line indicates that on average, on the test set, the ensemble is not significantly less accurate.

As the figure shows, when μ'_{max} is low, RP-based ensembles often achieve the best accuracy. Only on ARCENE (Figure 8.5a), RS seems to be a better strategy, suggesting some overfitting in setting α_s in RP. On all 5 other example datasets, RP is equivalent or better than RS and P for low values of μ'_{max}, with the largest gaps appearing on ISOLET (Figure 8.5e) and MNIST3VS8 (Figure 8.5f). As already observed in the previous section, although RP-DT is not the best strategy when memory is unconstrained, its curve dominates the curve of RP-ET for small values of μ'_{max} in Figures 8.5b, 8.5c, and 8.5d. Because split thresholds are not randomized in RP-DT, this method is more resistant than RP-ET to the strong randomization induced by a very low μ'_{max} threshold.

For comparison, Figure 8.5 also features the learning curves of both ET and RF (with K optimized on the validation set), in which the trees have all been built on the *same* training sample of $\mu'_{max}N$ instances, with all features. These results are representative of the use of a straightforward sub-sampling of the instances to handle the memory constraint. On all datasets, this setting yields very poor performance when μ'_{max} is low. Building base estimators on re-sampled random patches thus brings a clear advantage to RP, RS and P and hence confirms the conclusions of Basilico et al. [2011] who showed that using more data indeed produces more accurate models than learning from a single sub-sample. This latter experiment furthermore shows that the good performances of RP cannot be trivially attributed to the fact that our datasets contain so many instances that only processing a sub-sample of them would be enough. On most problems, the slopes of the learning curves of RF and ET indeed suggest that convergence has not yet been reached on these datasets. Yet, important improvement are gained by sub-sampling random patches. Contrary to intuition, it also does not appear that the bigger the datasets, the

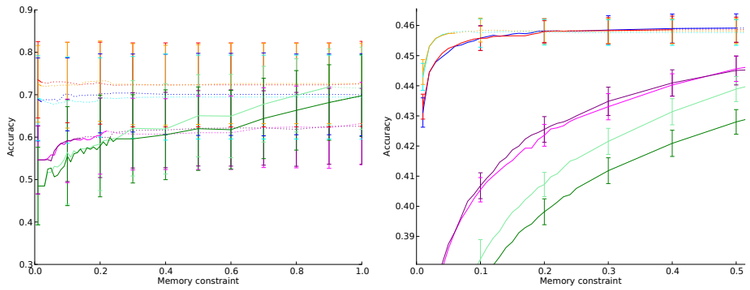
8.3.3 Redução de memória, com perda

A seção anterior mostrou que o consumo de memória pode ser reduzido até certo limite μ'_{max} sem perda significativa de precisão. Nesta seção, veremos agora a precisão do conjunto resultante quando μ'_{max} é ainda mais diminuído. Argumentamos que, com restrições severas, e porque os conjuntos de dados têm uma sensibilidade diferente, é ainda mais crucial explorar melhor os dados e, assim, encontrar o compromisso certo entre α_s e α_f, como só o RP pode.

Para ilustrar nosso ponto de vista, a Figura 8.5 compara, para 6 conjuntos de dados representativos, a precisão dos métodos em relação à restrição de memória $\alpha_s\alpha_f < \mu'_{max}$. Uma linha simples indica que o erro de generalização do conjunto de melhor resultado sob a restrição de memória μ'_{max} é significativamente (em $\alpha = 0.05$) pior nos conjuntos de teste do que quando não há restrição (ou seja, $\mu'_{max} = 1$). Uma linha pontilhada indica que, em média, no conjunto de teste, o conjunto não é significativamente menos preciso.

Como mostra a figura, quando μ'_{max} é baixo, os conjuntos baseados e m RP geralmente alcançam a melhor precisão. Apenas no arceno (Figura 8.5a), RS parece ser uma estratégia melhor, sugerindo algum overfitting na configuração α_s em RP. Em todos os outros 5 conjuntos de dados de exemplo, RP é equivalente ou melhor que RS e P para valores baixos de μ'_{max}, com as maiores lacunas aparecendo em isolet (Figura 8.5e) e mnist3vs8 (Figura 8.5f). Como já observado na seção anterior, embora o RP-DT não seja a melhor estratégia quando a memória é irrestrita, sua curva domina a curva do RP-ET para pequenos valores de μ'_{max} nas Figuras 8.5b, 8.5c e 8.5d. Como os limiares divididos não são randomizados no RP-DT, este método é mais resistente que o RP-ET à forte randomização induzida por um limianμ'_{max} muito baixo.

Para comparação, a Figura 8.5 também apresenta as curvas de aprendizado de ET e RF (com K otimizado no conjunto de validação), nas quais as árvores foram todas construídas na mesma amostra de treinamento de $\mu'_{max}N$ instâncias, com todos os recursos. Esses resultados são representativos do uso de uma subamostragem direta das instâncias para lidar com a restrição de memória. Em todos os conjuntos de dados, esta configuração produz um desempenho muito ruim quando μ'_{max} é baixo. Construir estimadores de base em manchas aleatórias rea mostradas traz assim uma clara vantagem para RP, RS e P e, portanto, confirma as conclusões de Basilico et al. [2011] que mostrou que usar mais dados realmente produz modelos mais precisos do que aprender co m uma única subamostra. Além disso, este último experimento mostra que o bom desempenho do RP não pode ser atribuído trivialmente ao fato de que nossos conjuntos de dados contêm tantas instâncias que apenas processar uma subamostra delas seria suficiente. Na maioria dos problemas, as inclinações das curvas de aprendizagem de RF e ET sugere m de facto que a convergência ainda não foi alcançada nestes conjuntos de dados. No entanto, melhorias importantes são obtidas através da subamostragem de amostras aleatórias. Contrariamente à intuição, também não parece que quanto maiores forem os conjuntos de dados, maior será a

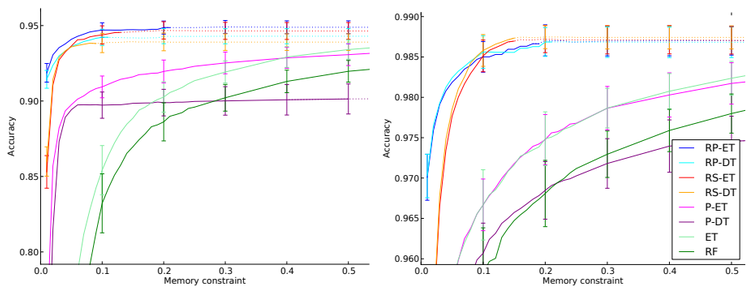


(a) ARCENE

(b) CIFAR10

(c) TIS

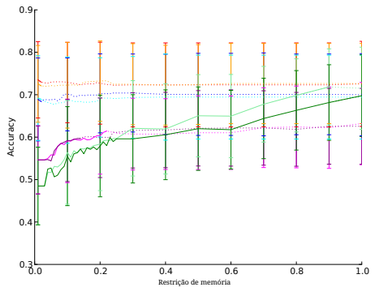
(d) MADELON



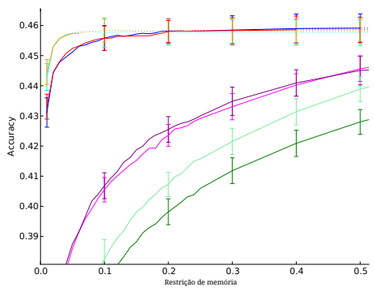
(e) ISOLET

(f) MNIST3VS8

Figure 8.5: Accuracy under memory constraint.



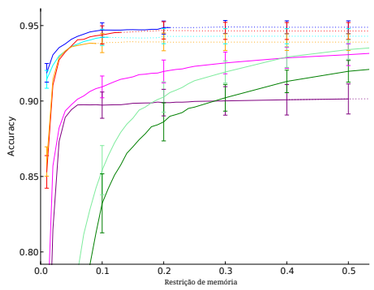
(a) arceno



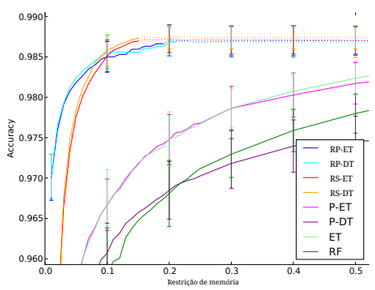
(b)cifar10

(c) isso

(d) madelão



(e) isoleta



(f) mnist3vs8

Figura 8.5: Precisão sob restrição de memória nt.

lower μ'_{max} can be reduced without loss of accuracy. Indeed, we found no conclusive correlation between the size Np of the dataset and the minimum value of μ'_{max} to reach good performance[2]. Overall, these results thus indicate that building an ensemble on random patches is not only a good strategy when data is abundant and redundant but also that it works even for scarce datasets with limited information regarding the problem.

8.3.4 *Conclusions*

We have shown in this section that the memory requirements of sampling based ensembles are intrinsically low. Better, we have shown that they can often be drastically decreased without significant loss in accuracy. When the size of the dataset is far bigger than the available memory, we have also demonstrated that sampling data along both samples and features, as RP does, not only competes with other ensemble algorithms but also significantly improves the accuracy of the resulting ensemble. It also brings a significant improvement over a straightforward sub-sampling of the instances.

8.4 OVERALL CONCLUSIONS

The main contribution of this work is to explore a new framework for supervised learning in the context of very strong memory constraints or, equivalently, very large datasets. To address such problems, we proposed the Random Patches ensemble method that builds each individual model of the ensemble from a random patch of the dataset obtained by drawing random subsets of both samples and features from the whole dataset. Through extensive experiments with tree-based estimators, we have shown that this strategy works as well as other popular randomization schemes in terms of accuracy (Section 8.2), at the same time reduces very significantly the memory requirements to build each individual model (Section 8.3.2), and, given its flexibility, attains significantly better accuracy than other methods when memory is severely constrained (Section 8.3.3). Since all models are built independently of each other, the approach is furthermore trivial to parallelize. All in all, we believe that the paradigm of our method highlights a very promising direction of research to address supervised learning on big data.

There remain several open questions and limitations to our approach that we would like to address in the future. First, this study motivates our interest in experimenting with truly large-scale problems (of giga-scale and higher). Since RP already appears advantageous for small to medium datasets, the potential benefits on very large-scale data indeed look very promising.

2 Spearman's rank correlation coefficient is -0.0354 ($p - value = 0.8550$).

menor μ'_{max} pode ser reduzido sem perda de precisão. Na verdade, não encontramos nenhuma correlação conclusiva entre o tamanho Np do conjunto de dados e o valor mínimo de μ'_{max} para alcançar um bom desempenho[2]. No geral, estes resultados indicam que construir um conjunto em fragmentos aleatórios não é apenas uma boa estratégia quando os dados são abundantes e redundantes, mas também funciona mesmo para conjuntos de dados escassos com informações limitadas sobre o problema.

8.3.4 Conclusões

Mostramos nesta seção que os requisitos de memória de conjuntos baseados em amostragem são intrinsecamente baixos. Melhor ainda, mostramos que muitas vezes eles podem ser diminuídos drasticamente sem perda significativa de precisão. Quando o tamanho do conjunto de dados é muito maior do que a memória disponível, também demonstramos que a amostragem de dados ao longo de amostras e recursos, como faz o RP, não apenas compete com outros algoritmos de conjunto, mas também melhora significativamente a precisão do conjunto resultante. Também traz uma melhoria significativa em relação a uma subamostragem simples das instâncias.

8.4 conclusões gerais

A principal contribuição deste trabalho é explorar uma nova estrutura para aprendizagem supervisionada no contexto de restrições de memória muito fortes ou, equivalentemente, de conjuntos de dados muito grandes. Para resolver tais problemas, propusemos o método de conjunto Random Patches que constrói cada modelo individual do conjunto a partir de um patch aleatório do conjunto de dados obtido pelo desenho de subconjuntos aleatórios de amostras e características de todo o conjunto de dados. Através de extensos experimentos com estimadores baseados em árvores, mostramos que esta estratégia funciona tão bem quanto outros esquemas de randomização populares em termos de precisão (Seção 8.2), ao mesmo tempo reduz significativamente os requisitos de memória para construir cada modelo individual (Seção 8.3.2) e, dada a sua flexibilidade, atinge uma precisão significativamente melhor do que outros métodos quando a memória é severamente restrita (Seção 8.3.3). Como todos os modelos são construídos independentemente uns dos outros, a abordagem é, além disso, trivial para paralelizar. Em suma, acreditamos que o paradigma do nosso método destaca uma direção de investigação muito promissora para abordar a aprendizagem supervisionada em big data.

Permanecem várias questões em aberto e limitações à nossa abordagem que gostaríamos de abordar no futuro. Primeiro, este estudo motiva nosso interesse em experimentar problemas verdadeiramente de grande escala (de giga-escala e superiores). Como a PR já parece vantajosa para conjuntos de dados pequenos e médios, os benefícios potenciais em dados de grande escala parecem realmente muito promissores.

2 O coeficiente de correlação de classificação de Spearman é -0,0354 ($p - valor = 0,8550$).

Second, the conclusions drawn in sections 8.2 and 8.3 are all based on the optimal values of the parameters α_s and α_f tuned through an exhaustive grid search on the validation set. Our analysis did not account for the memory and time required for tuning these two parameters. In practice, hyper-parameter tuning can not be avoided as we have shown that the optimal trade-off between α_f and α_s was problem dependent. It would therefore be interesting to design an efficient strategy to automatically find and adjust the values of α_s and α_f, taking into account the global memory constraint. Our simplistic framework also only accounts for the memory required to store the training set in memory and not for the total memory required to actually build the ensemble.

We have only explored uniform sampling of patches of fixed size in our experiments. In the context of the Pasting approach, Breiman proposed an iterative instance weighting scheme that proved to be more efficient than uniform sampling [Breiman, 1999]. It would be interesting to extend this approach when sampling both instances and features. Yet, parallelization would not be trivial anymore, although probably still possible in the line of the work in [Chawla et al., 2004].

Finally, our analysis of RP is mostly empirical. In the future, we would like to strengthen these results with a more theoretical analysis. A starting point could be the work in [Zinkevich et al., 2010] that studies a scheme similar to the Pasting method applied to linear models trained through parallel stochastic gradient descent. The extension of this work to non parametric tree-based estimators does not appear trivial however, since these latter are not well characterized theoretically. In retrospect, latest results [Scornet et al., 2014] regarding the consistency of random forests could certainly also be used to motivate the use of random subsets of samples. Similarly, theoretical results from Chapter 6 (Section 6.4.2) also indicate that the use of random subsets of features does not systematically degrade performance.

Em segundo lugar, as conclusões tiradas nas seções 8.2 e 8.3 são todas baseadas nos valores ótimos dos parâmetros α_s e α_f ajustados através de uma pesquisa exaustiva em grade no conjunto de validação. Nossa análise não levou em conta a memória e o tempo necessários para ajustar esses dois parâmetros. Na prática, o ajuste de hiperparâmetros não pode ser evitado, pois mostramos que o compromisso ideal entre α_f e α_s dependia do problema. Seria, portanto, interessante projetar uma estratégia eficiente para encontrar e ajustar automaticamente os valores de α_s eα_f, levando em consideração a restrição de memória global. Nossa estrutura simplista também leva em conta apenas a memória necessária para armazenar o conjunto de treinamento na memória e não a memória total necessária para realmente construir o conjunto.

Exploramos apenas amostragem uniforme de manchas de tamanho fixo em nossos experimentos. No contexto da abordagem Pasting, Brei man propôs um esquema iterativo de ponderação de instâncias que provou ser mais eficiente do que uniformsampling [Breiman, 1999]. Seria interessante estender esta abordagem ao amostrar instâncias e recursos. No entanto, a paralelização não seria mais trivial, embora provavelmente ainda seja possível na linha do trabalho em [Chawla et al., 2004].

Finalmente, nossa análise de RP é principalmente empírica. No futuro, gostaríamos de reforçar estes resultados com uma análise mais teórica. Um ponto de partida poderia ser o trabalho de [Zinkevich et al., 2010]que estuda um esquema semelhante ao método Pasting aplicado a modelos lineares treinados através de descida gradiente estocástica paralela. A extensão deste trabalho a estimadores não paramétricos baseados em árvores não parece trivial, no entanto, uma vez que estes últimos não estão bem caracterizados teoricamente. Em retrospecto, os resultados mais recentes [Scornet et al., 2014] relativos à consistência de florestas aleatórias certamente também poderiam ser usados para motivar o uso de subconjuntos aleatórios de amostras. Da mesma forma, os resultados teóricos do Capítulo 6 (Seção 6.4.2) também indicam que o uso de subconjuntos aleatórios de recursos não degrada sistematicamente o desempenho.

9 CONCLUSIONS

By and large, machine learning remains an open field of research for which many questions are still left unanswered, even regarding well-established methods. In this dissertation, we have revisited decision trees and random forests, consistently calling into question each and every part of these algorithms, in order to shed new light on their learning capabilities, inner workings and interpretability.

In Part I of this work, we laid out the decision trees and random forests methodology in the context of classification and regression tasks. Our treatment first considered the induction of individual decision trees and put them into a unified and composable framework. In particular, our analysis reviewed assignment rules, stopping criteria and splitting rules, theoretically motivating their design and purpose whenever possible. We then proceeded with a systematic study of randomized ensemble methods within the bias-variance framework. We established that variance depends on the correlation between individual tree predictions, thereby showing why randomization acts as a mechanism for reducing the generalization error of an ensemble. Random forest and its variants were then presented within the framework previously introduced, and their properties and features discussed and reviewed. Our contributions followed with an original time and space complexity analysis of random forests, hence showing their good computational performance and scalability to larger problems. Finally, the first part of this work concluded with an in-depth discussion of implementation details of random forests, highlighting and discussing considerations that are critical, yet easily overlooked, for guaranteeing good computational performance. While not directly apparent within this manuscript, this discussion also underlined our contributions in terms of software, within the open source Sckit-Learn library. As open science and reproducibility concerns are gaining momentum, we indeed believe that good quality software should be an integrative part, acknowledged for its own value and impact, of any modern scientific research activity.

Part II of this dissertation analyzed and discussed the interpretability of random forests in the eyes of variable importance measures. The core of our contributions rests in the theoretical characterization of the Mean Decrease of Impurity variable importance measure, from which we have then proved and derived some of its properties in the case of multiway totally randomized trees and in asymptotic conditions. In particular, we have shown that variable importances offer a three-level decomposition of the information jointly provided by the

9 CONCLUSIONS

Em geral, a aprendizagem automática continua a ser um campo aberto de investigação para o qual muitas questões ainda permanecem sem r esposta, mesmo no que diz respeito a métodos bem estabelecidos. Nesta dissertação, revisitámos árvores de decisão e florestas aleatórias, questionando consistentemente cada parte destes algoritmos, a fim de lançar nova luz sobre as suas capacidades de aprendizagem, funcionamento interno e interpretabilidade.

Na Parte I deste trabalho, apresentamos a metodologia de árvores de decisão e florestas aleatórias no contexto de tarefas de classificação e regressão. Nosso tratamento considerou primeiro a indução de árvores de decisão individuais e as colocou em uma estrutura unificada e co mbinável. Em particular, a nossa análise revisou regras de atribuição, critérios de paragem e regras de divisão, motivando teoricamente a sua concepção e finalidade sempre que possível. Em seguida, procedemos co m um estudo sistemático de métodos de conjuntos randomizados dentro da estrutura de viés-variância. Estabelecemos que a variância depende da correlação entre as previsões das árvores individuais, mostrando assim porque a randomização atua como um mecanismo para reduzir o erro de generalização de um conjunto. A floresta aleatória e suas variantes foram então apresentadas dentro da estrutura previamente apresentada, e suas propriedades e características discutidas e revisadas. Nossas contribuições seguiram com uma análise original da complexidade de tempo e espaço de florestas aleatórias, mostrando assim seu bom desempenho co mputacional e escalabilidade para problemas maiores. Finalmente, a primeira parte deste trabalho concluiu com uma discussão aprofundada dos detalhes de implementação de florestas aleatórias, destacando e discutindo considerações que são críticas, mas facilmente negligenciadas, para garantir um bom desempenho computacional. Embora não seja diretamente aparente neste manuscrito, esta discussão também sublinhou nossas contribuições em termos de software, dentro da biblioteca de código aberto Sckit-Learn. À medida que as preocupações com a ciência aberta e a reprodutibilidade ganham força, acreditamos de facto que software de boa qualidade deve ser uma parte integrante, reconhecida pelo seu próprio valor e impacto, de qualquer actividade de investigação científica moderna.

A parte II desta dissertação analisou e discutiu a interpretabilidade de florestas aleatórias aos olhos de medidas de importância variável. O núcleo de nossas contribuições repousa na caracterização teórica da medida de importância variável de Decréscimo Médio de Impureza, a partir da qual provamos e derivamos algumas de suas propriedades no caso de árvores multiway totalmente randomizadas e em condições assintóticas. Em particular, mostramos que as importâncias variáveis oferecem uma decomposição em três níveis da informação fornecida conjuntamente pelo

input variables about the output, accounting for all possible interaction terms in a fair and exhaustive way. More interestingly, we have also shown that variable importances only depend on relevant variables and that the importance of irrelevant variables is strictly equal to zero, thereby making importances a sound and appropriate criterion for assessing the usefulness of variables. In consequence of this work, our analysis then demonstrated that variable importances as computed from non-totally randomized trees (e.g., standard Random Forest or Extremely Randomized Trees) suffer from a combination of defects, due to masking effects, misestimations of node impurity or due to the binary structure of decision trees. Overall, we believe that our analysis should bring helpful insights in a wide range of applications, by shedding new light on variable importances. In particular, we advise to complement their interpretation and analysis with a systematic decomposition of their terms, in order to better understand why variables are (or are not) important.

This preliminary work unveils various directions of future work, both from a theoretical and practical point of view. To our belief, the most interesting theoretical open question would be the characterization of the distribution of variable importances in the finite setting. Such a characterization would indeed allow to more reliably distinguish irrelevant variables (whose importances are positive in the finite case) from relevant variables. Another interesting direction of future work would be to derive a proper characterization of variable importances in the case of binary trees – even if we believe, as pointed out earlier, that variable importances derived from such ensembles may in fact not be as appropriate as desired. From a more practical point of view, this study also calls for a re-analysis of previous empirical studies. We indeed believe that variable importances along with their decomposition should yield new insights in many cases, providing a better understanding of the interactions between the input variables and the output, but also between the input variables themselves. Again, we recommend multiway totally randomized trees to mitigate sources of bias as much as possible.

Finally, Part III addressed limitations of random forests in the context of large datasets. Through extensive experiments, we have shown that subsampling either samples, features or both simultaneously provides on par performance while lowering at the same time the memory requirements. Overall this paradigm highlights an intriguing practical fact: there is often no need to build single models over immensely large datasets. Good performance can often more simply be achieved by building models on small random parts of the data and then combining them all in an ensemble, thereby avoiding all practical and computational burdens of making large data fit into memory. Again, this work raises interesting questions of further work. From a theoretical point of view, one would be to identify the statistical

variáveis de entrada sobre a saída, contabilizando todos os termos de interação possíveis de forma justa e exaustiva. O mais interessante é que também mostrámos que as importâncias das variáveis dependem apenas de variáveis relevantes e que a importância das variáveis irrelevantes é estritamente igual a zero, tornando assim as importâncias um critério sólido e apropriado para avaliar a utilidade das variáveis. Como consequência deste trabalho, nossa análise demonstrou então que as importâncias das variáveis calculadas a partir de árvores não totalmente aleatórias (por exemplo, Floresta Aleatória padrão ou Árvores Extremamente Randomizadas) sofrem de uma combinação de defeitos, devido a efeitos de mascaramento, estimativas incorretas de impureza dos nós ou devido à estrutura binária das árvores de decisão. No geral, acreditamos que a nossa análise deverá trazer informações úteis numa ampla gama de aplicações, lançando uma nova luz sobre as importâncias variáveis. Em particular, aconselhamos complementar a sua interpretação e análise com uma decomposição sistemática dos seus termos, para melhor compreender porque é que as variáveis são (ou não) importantes.

Este trabalho preliminar revela várias direções de trabalhos futuros, tanto do ponto de vista teórico como prático. Em nossa opinião, a questão teórica aberta mais interessante seria a caracterização da distribuição de importâncias variáveis no cenário finito. Tal caracterização permitiria, de facto, distinguir de forma mais fiável variáveis irrelevantes (cujas importâncias são positivas no caso finito) de variáveis relevantes. Outra direção interessante de trabalho futuro seria derivar uma caracterização adequada de importâncias variáveis no caso de árvores binárias – mesmo se acreditarmos, como apontado anteriormente, que importâncias variáveis derivadas de tais conjuntos podem de fato não ser tão apropriadas quanto desejado. De um ponto de vista mais prático, este estudo também exige uma reanálise de estudos empíricos anteriores. Na verdade, acreditamos que as importâncias das variáveis, juntamente com a sua decomposição, deverão produzir novos conhecimentos em muitos casos, proporcionando uma melhor compreensão das interacções entre as variáveis de entrada e os resultados, mas também entre as próprias variáveis de entrada. Novamente, recomendamos árvores totalmente aleatórias para mitigar ao máximo as fontes de viés.

Finalmente, a Parte III abordou as limitações das florestas aleatórias no contexto de grandes conjuntos de dados. Através de extensos experimentos, mostramos que a subamostragem de amostras, recursos ou ambos simultaneamente fornece desempenho equivalente e, ao mesmo tempo, reduz os requisitos de memória. No geral, esse paradigma destaca um fato prático intrigante: muitas vezes não há necessidade de construir modelos únicos em conjuntos de dados imensamente grandes. Muitas vezes, um bom desempenho pode ser alcançado de forma mais simples através da construção de modelos em pequenas partes aleatórias dos dados e, em seguida, combinando-os todos em um conjunto, evitando assim todos os encargos práticos e computacionais de fazer grandes dados caberem na memória. Novamente, este trabalho levanta questões interessantes para trabalhos futuros. Do ponto de vista teórico, seria identificar a estatística

properties in the learning problem that are necessary for guaranteeing subsampling strategies to work. In particular, in which cases is it better to subsample examples rather than features? From a more practical point of view, other directions of research also include the study of smarter sampling strategies or the empirical verification that conclusions extend to non tree-based methods.

Overall, this thesis calls for a permanent re-assessment of machine learning methods and algorithms. It is only through a better understanding of their mechanisms that algorithms will advance in a consistent and reliable way. Always seek for the what and why. In conclusion, machine learning should not be considered as a black-box tool, but as a methodology, with a rational thought process that is entirely dependent on the problem we are trying to solve.

propriedades no problema de aprendizagem que são necessárias para garantir que as estratégias de subamostragem funcionem. Em particular, em quais casos é melhor subamostrar exemplos em vez de recursos? De um ponto de vista mais prático, outras direções de pesquisa também incluem o estudo de estratégias de amostragem mais inteligentes ou a verificação empírica de que as conclusões se estendem a métodos não baseados em árvores.

No geral, esta tese exige uma reavaliação permanente dos métodos e algoritmos de aprendizado de máquina. Só através de uma melhor compreensão dos seus mecanismos é que os algoritmos avançarão de forma consistente e fiável. Sempre busque o quê e por quê. Concluindo, o aprendizado de máquina não deve ser considerado uma ferramenta de caixa preta, mas sim uma metodologia, com um processo de pensamento racional que depende inteiramente do problema que estamos tentando resolver.

Part IV

APPENDIX

Parte IV

APENDIX

NOTATIONS

$\mathcal{A}$	A supervised learning algorithm 12
$\mathcal{A}(\theta, \mathcal{L})$	The model $\varphi_{\mathcal{L}}$ produced by algorithm $\mathcal{A}$ over $\mathcal{L}$ and hyper-parameters θ 17
α_s	The proportion of samples in a random patch ... 170
α_f	The proportion of features in a random patch ... 170
b_l	The l-th value of a categorical variable 39
B	A subset $B \subseteq V$ of variables 128
c_k	The k-th class 11
C_p^k	The number of k-combinations from a set of p elements 129
$C(N)$	The time complexity for splitting N samples 88
$\mathbb{E}$	Expectation .....
$\overline{\mathrm{E}}(\varphi_{\mathcal{L}}, \mathcal{L}')$	The average prediction error of $\varphi_{\mathcal{L}}$ over $\mathcal{L}'$ 13
$\mathrm{Err}(\varphi_{\mathcal{L}})$	The generalization error of $\varphi_{\mathcal{L}}$ 12, 55
$H(X)$	The Shannon entropy of X 126
$H(X\|Y)$	The Shannon entropy of X conditional to Y 126
$\mathcal{H}$	The space of candidate models 16
$i(t)$	The impurity of node t 30, 40
$i_R(t)$	The impurity of node t based on the local resubstitution estimate 41, 46
$i_H(t)$	The entropy impurity of node t 45
$i_G(t)$	The Gini impurity of node t 45
$\Delta i(s, t)$	The impurity decrease of the split s at node t 30
$I(X; Y)$	The mutual information between X and Y 126
$\mathrm{Imp}(X_j)$	The variable importance of X_j 124, 125
J	The number of classes 11
K	The number of folds in cross-validation ..... 14 The number of input variables drawn at each node for finding a split 71
$K(\mathbf{x}_i, \mathbf{x}_j)$	The kernel of $\mathbf{x}_i$ and $\mathbf{x}_j$ 22, 79
L	A loss function ..... 12 The number of values of a categorical variable ... 39
$\mathcal{L}$	A learning set $(\mathbf{X}, \mathbf{y})$ 10
$\mathcal{L}^m$	The m-th bootstrap replicate of $\mathcal{L}$ 70

NOTATIONS

$\mathcal{A}$ Um algoritmo de aprendizagem supervisionada.12 $\mathcal{A}(\theta, \mathcal{L})$ O modelo $\varphi_{\mathcal{L}}$ produzido pelo algoritmo $\mathcal{A}$ sobre $\mathcal{L}$ e hiperparâmetros θ .17 α_s A proporção de amostras em um patch aleatório . . . 170 α_f A proporção de recursos em um patch aleatório . . . 170 b_l O l-ésimo valor de uma variável categórica.39 B Um subconjunto $B \subseteq V$ de variáveis.128 c_k A k-ésima classe.11 C_p^k O número de combinações k de um conjunto de p elementos.129 $C(N)$ A complexidade do tempo para dividir N amostras . . . . . 88 $\mathbb{E}$ Expectativa. $E(\varphi_{\mathcal{L}}, \mathcal{L}')$ O erro médio de previsão de $\varphi_{\mathcal{L}}$ sobre $\mathcal{L}'$ . . . . . 13 $\mathrm{Err}(\varphi_{\mathcal{L}})$ O erro de generalização de $\varphi_{\mathcal{L}}$ .12, 55 $H(X)$ A entropia de Shannon de X .126 $H(X|Y)$ A entropia de Shannon de X condicional a Y . . . . . 126 $\mathcal{H}$ O espaço dos modelos candidatos.16 $i(t)$ A impureza do nó t . 30, 40 $i_R(t)$ A impureza do nó t com base na estimativa de substituição local.41, 46 $i_H(t)$ A impureza de entropia do nó t .45 $i_G(t)$ A impureza Gini do nó t .45 $\Delta i(s, t)$ A diminuição da impureza da divisão s no nó t . . . 30 $I(X; Y)$ As informações mútuas entre X e Y . . . . . 126 $\mathrm{Imp}(X_j)$ A importância variável de X_j .124, 125 J O número de aulas.11 K O número de dobras na validação cruzada.14 O número de variáveis de entrada desenhadas em cada nó para encontrar uma divisão.71 $K(x_i, x_j)$ O kernel de x_i e x_j .22, 79 L Uma função de perda.12 O número de valores de uma variável categórica. . . 39 $\mathcal{L}$ Um conjunto de aprendizagem (X, y) .10 $\mathcal{L}^m$ A m-ésima réplica de bootstrap de $\mathcal{L}$ .70

$\mathcal{L}_t$	The subset of node samples falling into node t ... 30
M	The number of base models in an ensemble 62
$\mu_{\mathcal{L},\theta_m}(\mathbf{x})$	The mean prediction at $X = \mathbf{x}$ of $\varphi_{\mathcal{L},\theta_m}$ 64
N	The number of input samples 10
N_t	The number of node samples in node t 30
N_{ct}	The number of node samples of class c in node t . 32
Ω	The universe, or population, from which cases are sampled 9
p	The number of input variables 9
p_L	The proportion of node samples going to t_L 30
p_R	The proportion of node samples going to t_R 30
$p(t)$	The estimated probability $p(X \in \mathcal{X}_t) = \frac{N_t}{N}$ 33
$p(c\|t)$	The empirical probability estimate $p(Y = c\|X \in \mathcal{X}_t) = \frac{N_{ct}}{N_t}$ of class c at node t 33
$\widehat{p}_{\mathcal{L}}$	An empirical probability estimate computed from the learning set $\mathcal{L}$ 59
$P(X, Y)$	The joint probability distribution of the input variables $X = (X_1, \ldots, X_p)$ and the output variable Y . 11
$\mathcal{P}_k(V)$	The set of subsets of V of size k 128
φ	A model or function $\mathcal{X} \mapsto \mathcal{Y}$ ..... 10 A single decision tree 27
$\widetilde{\varphi}$	The set of terminal nodes in φ 32
$\varphi(\mathbf{x})$	The prediction of φ for the sample $\mathbf{x}$ 10
$\varphi_{\mathcal{L}}$	A model built from $\mathcal{L}$ 12
$\varphi_{\mathcal{L},\theta}$	A model built from $\mathcal{L}$ with random seed θ 61
φ_B	A Bayes model 15
$\psi_{\mathcal{L},\theta_1,\ldots,\theta_M}$	An ensemble of M models built from $\mathcal{L}$ and random seeds $\theta_1, \ldots, \theta_M$ 62
$\mathcal{Q}$	A set $\mathcal{Q} \subseteq \mathcal{S}$ of splits of restricted structure 27, 38
$\mathcal{Q}(X_j)$	The set $\mathcal{Q}(X_j) \subseteq \mathcal{Q}$ of univariate binary splits that can be defined on variable X_j 38, 39
$\rho(\mathbf{x})$	The correlation coefficient between the predictions at $X = \mathbf{x}$ of two randomized models 65
s	A split 27, 37
s^*	The best split 30, 39
s_j^*	The best binary split defined on variable X_j ... 30, 39
s_j^v	The binary split $(\{\mathbf{x}\|x_j \leqslant v\}, \{\mathbf{x} > v\})$ defined on variable X_j with discretization threshold v 38
s_t	The split labeling node t 27
$\tilde{s}_t^j$	The best surrogate split for s_t defined from X_j .. 124

$\mathcal{L}_t$ O subconjunto de amostras de nós que caem no nó t . . . 30M O número de modelos básicos em um conjunto . . . . . 62$\mu_{\mathcal{L},\theta_m}(\mathbf{x})$ A previsão média em $X = \mathbf{x}$ de $\varphi_{\mathcal{L},\theta_m}$ .64N O número de amostras de entrada.10N_t O número de amostras de nós no nó t .30N_{ct} O número de amostras de nós da classe c no nó t . 32Ω O universo, ou população, da qual os casos são amostrados.9p O número de variáveis de entrada.9p_L A proporção de amostras de nós que vão para t_L . . . . . 30p_R A proporção de amostras de nós que vão para t_R . . . . . 30$p(t)$ A probabilidade estimada $p(X \in \mathcal{X}_t) = \frac{N_t}{N}$ . . . . . 33$p(c|t)$ A estimativa de probabilidade empírica $p(Y = c|X \in \mathcal{X}_t) = \frac{N_{ct}}{N_t}$da classe c no nó t .33$\widehat{p}_{\mathcal{L}}$ Uma estimativa de probabilidade empírica calculada a partir do conjunto de aprendizagem $\mathcal{L}$.59$P(X, Y)$ A distribuição de probabilidade conjunta das variáveis de entrada $X = (X_1, \ldots, X_p)$ e da variável de s aída Y. 11$\mathcal{P}_k(V)$ O conjunto de subconjuntos de Vde tamanho k .128φ Um modelo ou função $\mathcal{X} \mapsto \mathcal{Y}$ .10 Uma única árvore de decisão.27$\widetilde{\varphi}$ O conjunto de nós terminais em φ .32$\varphi(\mathbf{x})$ A previsão de φ para a a mostra x.10$\varphi_{\mathcal{L}}$ Um modelo construído a partir de $\mathcal{L}$ .12$\varphi_{\mathcal{L},\theta}$ Um modelo construído a partir de $\mathcal{L}$ com semente aleatória θ . . . . . 61φ_B Um modelo Bayesi~~ano~~.15$\psi_{\mathcal{L},\theta_1,\ldots,\theta_M}$ Um conjunto de M modelos co nstruídos a partir de $\mathcal{L}$e sementes aleatórias $\theta_1, \ldots, \theta_M$ .62$\mathcal{Q}$ Um conjunto $\mathcal{Q} \subseteq \mathcal{S}$ de divisões de estrutura restrita. . . . 27, 38$\mathcal{Q}(X_j)$ O conjunto $\mathcal{Q}(X_j) \subseteq \mathcal{Q}$ de divisões binárias univariadas que podem ser definidas na variável X_j .38, 39$\rho(\mathbf{x})$ O coeficiente de correlação entre as previsões em$X = \mathbf{x}$ de dois modelos aleatórios.65s Uma divisão.27, 37s^* A melhor divisão.30, 39s_j^* A melhor divisão binária definida na variável X_j . . . 30, 39s_j^v A divisão binária $(\{\mathbf{x}|x_j \leqslant v\}, \{\mathbf{x} > v\})$ definida na variável X_j com limite de discretização v .38s_t O nó de rotulagem dividido t .27$\tilde{s}_t^j$ A melhor divisão substituta para s_t definida em X_j . . 124

$\mathcal{S}$	The set of all possible splits s 38
$\sigma^2_{\mathcal{L},\theta_m}(\boldsymbol{x})$	The prediction variance at $X = \boldsymbol{x}$ of $\varphi_{\mathcal{L},\theta_m}$ 64
t	A node in a decision tree 27
t_L	The left child of node t 30, 37
t_R	The right child of node t 30, 37
θ	A vector of hyper-parameter values ..... 17 A random seed 62
θ^*	The optimal hyper-parameters 17
$\widehat{\theta}^*$	The approximately optimal hyper-parameters 17
θ_m	The seed of the m-th model in an ensemble 62
v	A discretization threshold in a binary split 38
v_k	The k-th value of an ordered variable, when node samples are in sorted order 48
v'_k	The mid-cut point between v_k and v_{k+1} 48
V	The set $\{X_1, \ldots, X_p\}$ of input variables 127
V^{-j}	$V \setminus \{X_j\}$ 128
$\mathbb{V}$	Variance .....
$\boldsymbol{x}$	A case, sample or input vector $(x_1, \ldots, x_p)$ 9
$\boldsymbol{x}_i$	The i-th input sample in $\mathcal{L}$ 10
x_j	The value of variable X_j for the sample $\boldsymbol{x}$ 9
$\boldsymbol{X}$	The $N \times p$ matrix representing the values of all N samples for all p input variables 10
X_j	The j-th input variable or feature 9, 10
X	The random vector $(X_1, \ldots, X_p)$ 11
$\mathcal{X}_j$	The domain or space of variable X_j 9
$\mathcal{X}$	The input space $\mathcal{X}_1 \times \cdots \times \mathcal{X}_p$ 9
$\mathcal{X}_t$	The subspace $\mathcal{X}_t \subseteq \mathcal{X}$ represented by node t 27
y	A value of the output variable Y 9
$\widehat{y}_t$	The value labelling node t 28
$\widehat{y}^*_t$	The optimal value labelling node t 32
$\boldsymbol{y}$	The output values $(y_1, \ldots, y_N)$ 10
Y	The output or response variable Y 9
$\mathcal{Y}$	The domain or space of variable Y 9

$\mathcal{S}$ O conjunto de todas as divisões possíveis s .38$\sigma^2_{\mathcal{L},\theta_m}(x)$ A variação da previsão em $X = x$ de $\varphi_{\mathcal{L},\theta_m}$ ..... 64t Um nó em uma árvore de decisão.27t_L O filho esquerdo do nó t .30, 37t_R O filho certo do nó t .30, 37θ Um vetor de valores de hiperparâmetros.17 Uma semente aleatória.62θ^* Os hiperparâmetros ideais.17$\widehat{\sim}\theta^*$ Os hiperparâmetros aproximadamente ideais. . . . 17θ_m A semente do m-ésimo modelo em um conjunto. . . . . 62v Um limite de discretização em uma divisão binária . . . . . 38v_k O k-ésimo valor de uma variável ordenada, quando as amostras de nós estão em ordem de classificação.48v'_k O ponto intermediário entre v_k e v_{k+1} .48V O conjunto $\{X_1, \ldots, X_p\}$ de variáveis de entrada.127V^{-j} $V \setminus \{X_j\}$ .128$\mathbb{V}$ Variação.x Um caso, amostra ou vetor de entrada $(x_1, \ldots, x_p)$ ..... 9 x_i A i-ésima amostra de entrada em $\mathcal{L}$ .10x_j O valor da variável X_j para a amostra x.9 X A matriz $N \times p$ que representa os valores de todas as amostras N para todas as variáveis de entrada p .10X_j A j-ésima variável ou r ecurso de entrada.9, 10X O vetor aleatório $(X_1, \ldots, X_p)$ .11$\mathcal{X}_j$ O domínio ou espaço da variável X_j .9$\mathcal{X}$ O espaço de entrada $\mathcal{X}_1 \times \cdots \times \mathcal{X}_p$ .9$\mathcal{X}_t$ O subespaço $\mathcal{X}_t \subseteq \mathcal{X}$ representado pelo nó t . . . . . 27y Um valor da variável de saída Y .9$\widehat{y}_t$ O nó de rotulagem de valor t .28$\widehat{y}^*_t$ O nó de rotulagem de valor ideal t .32 y Os valores de saída $(y_1, \ldots, y_N)$ .10Y A variável de saída ou resposta Y .9$\mathcal{Y}$ O domínio ou espaço da variável Y. 9

REFERENCES

J. Adamo. Fuzzy decision trees. *Fuzzy sets and systems*, 4(3):207–219, 1980.

Y. Amit, D. Geman, and K. Wilder. Joint induction of shape features and tree classifiers. *Pattern Analysis and Machine Intelligence, IEEE Transactions on*, 19(11):1300–1305, 1997.

I. Arel, D. Rose, and T. Karnowski. Deep machine learning-a new frontier in artificial intelligence research [research frontier]. *Computational Intelligence Magazine, IEEE*, 5(4):13–18, 2010.

S. Arlot and A. Celisse. A survey of cross-validation procedures for model selection. *Statistics Surveys*, 4:40–79, 2010.

J. Basilico, M. Munson, T. Kolda, K. Dixon, and W. Kegelmeyer. Comet: A recipe for learning and using large ensembles on massive data. In *Data Mining (ICDM), 2011 IEEE 11th International Conference on*, pages 41–50. IEEE, 2011.

S. Behnel, R. Bradshaw, C. Citro, L. Dalcin, D. S. Seljebotn, and K. Smith. Cython: the best of both worlds. *CiSE*, 13(2):31–39, 2011.

Y. Bengio, A. Courville, and P. Vincent. Representation learning: A review and new perspectives. 2013.

J. L. Bentley and M. D. McIlroy. Engineering a sort function. *Software: Practice and Experience*, 23(11):1249–1265, 1993.

J. L. Bentley, D. Haken, and J. B. Saxe. A general method for solving divide-and-conquer recurrences. *ACM SIGACT News*, 12(3):36–44, 1980.

J. Bergstra and Y. Bengio. Random search for hyper-parameter optimization. *The Journal of Machine Learning Research*, 13:281–305, 2012.

G. Biau. Analysis of a random forests model. *The Journal of Machine Learning Research*, 98888:1063–1095, 2012.

G. Biau, L. Devroye, and G. Lugosi. Consistency of random forests and other averaging classifiers. *The Journal of Machine Learning Research*, 9:2015–2033, 2008.

C. Bishop and N. Nasrabadi. *Pattern recognition and machine learning*, volume 1. springer New York, 2006.

REFERENCES

J. Adamo. Árvores de decisão difusas. Conjuntos e sistemas difusos, 4(3):207–219, 1980. Y. Amit, D. Geman e K. Wilder. Indução conjunta de características de forma e classificadores de árvore. Análise de padrões e inteligência de máquina, Transações IEEE em, 19(11):1300–1305, 1997. I. Arel, D. Rose e T. Karnowski. Aprendizado de máquina profundo: uma nova fronteira na pesquisa em inteligência artificial [fronteira de pesquisa]. Computational Intelligence Magazine, IEEE, 5(4):13–18, 2010. S. Arlot e A. Celisse. Um levantamento de procedimentos de validação cruzada para seleção de modelos. Pesquisas Estatísticas, 4:40–79, 2010. J. Basilico, M. Munson, T. Kolda, K. Dixon e W. Kegelmeyer. Cometa: uma receita para aprender e usar grandes conjuntos em dados massivos. Em Data Mining (ICDM), 11ª Conferência Internacional IEEE de 2011, páginas 41–50. IEEE, 2011. S. Behnel, R. Bradshaw, C. Citro, L. Dalcin, DS Seljebotn e K. Smith. Cython: o melhor dos dois mundos. CiSE, 13(2):31–39, 2011. Y. Bengio, A. Courville e P. Vincent. Aprendizagem de representação: uma revisão e novas perspectivas. 2013. JL Bentley e MD McIlroy. Engenharia de uma função de classificação. Software: Prática e Experiência, 23(11):1249–1265, 1993. JL Bentley, D. Haken e JB Saxe. Um método geral para resolver recorrências de dividir e conquistar. ACM SIGACT News, 12(3):36–44, 1980. J. Bergstra e Y. Bengio. Pesquisa aleatória para otimização de hiperparâmetros. The Journal of Machine Learning Research, 13:281–305, 2012. G. Biau. Análise de um modelo de florestas aleatórias. The Journal of Machine Learning Research, 98888:1063–1095, 2012. G. Biau, L. Devroye e G. Lugosi. Consistência de florestas aleatórias e outros classificadores de média. The Journal of Machine Learning Research, 9:2015–2033, 2008. C. Bishop e N. Nasrabadi. Reconhecimento de padrões e aprendizado de máquina, volume 1. Springer Nova York, 2006.

H. Blockeel, L. De Raedt, and J. Ramon. Top-down induction of clustering trees. *arXiv preprint cs/0011032*, 2000.

A. Blumer, A. Ehrenfeucht, D. Haussler, and M. K. Warmuth. Occam's razor. *Information processing letters*, 24(6):377–380, 1987.

B. E. Boser, I. M. Guyon, and V. N. Vapnik. A training algorithm for optimal margin classifiers. In *Proceedings of the fifth annual workshop on Computational learning theory*, pages 144–152. ACM, 1992.

V. Botta. A walk into random forests: adaptation and application to genome-wide association studies. 2013.

V. Botta, G. Louppe, P. Geurts, and L. Wehenkel. Exploiting snp correlations within random forest for genome-wide association studies. *PloS one*, 9(4):e93379, 2014.

L. Bottou and O. Bousquet. The tradeoffs of large-scale learning. *Optimization for Machine Learning*, page 351, 2011.

A.-L. Boulesteix, S. Janitza, J. Kruppa, and I. R. König. Overview of random forest methodology and practical guidance with emphasis on computational biology and bioinformatics. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 2(6):493–507, 2012.

G. Bradski and A. Kaehler. *Learning OpenCV: Computer vision with the OpenCV library*. O'Reilly Media, Inc., 2008.

L. Breiman. Parsimonious binary classification trees. *Preliminary report. Santa Monica, Calif.: Technology Service Corporation*, 1978a.

L. Breiman. Description of chlorine tree development and use. Technical report, Technical Report, Technology Service Corporation, Santa Monica, CA, 1978b.

L. Breiman. Bagging predictors. 1994.

L. Breiman. Bias, variance, and arcing classifiers. 1996.

L. Breiman. Pasting small votes for classification in large databases and on-line. *Machine Learning*, 36(1-2):85–103, 1999.

L. Breiman. Some infinity theory for predictor ensembles. Technical report, Technical Report 579, Statistics Dept. UCB, 2000.

L. Breiman. Random Forests. *Machine learning*, 45(1):5–32, 2001.

L. Breiman. Manual on setting up, using, and understanding random forests v3. 1. *Statistics Department University of California Berkeley, CA, USA*, 2002.

200

H. Blockeel, L. De Raedt e J. Ramon. Indução de cima para baixo de árvores de agrupamento. Pré-impressão arXiv cs/0011032, 2000. A. Blumer, A. Ehrenfeucht, D. Haussler e MK Warmuth. Navalha de Occam. Cartas de processamento de informações, 24(6):377–380, 1987. BE Boser, IM Guyon e VN Vapnik. Um algoritmo de treinamento para classificadores de margem ideais. Nos Anais do quinto workshop anual sobre teoria da aprendizage m computacional, páginas 144–152. ACM, 1992. V. Botta. Um passeio pelas florestas aleatórias: adaptação e aplicação a estudos de associação genômica ampla. 2013. V. Botta, G. Louppe, P. Geurts e L. Wehenkel. Explorando correlações snp dentro de florestas aleatórias para estudos de associação genômica ampla. PloS um, 9(4):e93379, 2014. L. Bottou e O. Bousquet. As vantagens e desvantagens da aprendizagem em larga escala. Otimização para aprendizado de máquina, página 351, 2011. A.-L. Boulesteix, S. Janitza, J. Kruppa e IR König. Visão geral da metodologia florestal aleatória e orientação prática com ênfase em biologia computacional e bioinformática. Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery, 2(6):493–507, 2012. G. Bradski e A. Kaehler. Aprendendo OpenCV: Visão computacional com a biblioteca OpenCV. O'Reilly Media, Inc., 2008. L. Breiman. Árvores de classificação binária parcimoniosas. Relatório preliminar. Santa Monica, Califórnia: Technology Service Corporation, 1978a. L. Breiman. Descrição do desenvolvimento e uso da árvore de cloro. Relatório técnico, Relatório Técnico, Technology Service Corporation, Santa Monica, CA, 1978b. L. Breiman. Preditores de ensacamento. 1994. L. Breiman. Classificadores de polarização, variância e arco. 1996. L. Breiman. Colagem de pequenos votos para classificação em grandes bases de dados e on-line. Aprendizado de Máquina, 36(1-2):85–103, 1999. L. Breiman. Alguma teoria do infinito para conjuntos de preditores. Relatório técnico, Relatório Técnico 579, Departamento de Estatística UCB, 2000. L. Breiman. Florestas Aleatórias. Aprendizado de máquina, 45(1):5–32, 2001. L. Breiman. Manual sobre como configurar, usar e compreender florestas aleatórias v3. 1. Departamento de Estatística da Universidade da Califórnia em Berkeley, CA

, *USA*, 2002.

L. Breiman. Consistency for a simple model of random forests. Technical report, UC Berkeley, 2004.

L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone. *Classification and regression trees*. 1984.

A. E. Bryson. *Applied optimal control: optimization, estimation and control*. CRC Press, 1975.

L. Buitinck, G. Louppe, M. Blondel, F. Pedregosa, A. Mueller, O. Grisel, V. Niculae, P. Prettenhofer, A. Gramfort, J. Grobler, et al. API design for machine learning software: experiences from the scikit-learn project. *arXiv preprint arXiv:1309.0238*, 2013.

J. S. Chang, R.-F. Yeh, J. K. Wiencke, J. L. Wiemels, I. Smirnov, A. R. Pico, T. Tihan, J. Patoka, R. Miike, J. D. Sison, et al. Pathway analysis of single-nucleotide polymorphisms potentially associated with glioblastoma multiforme susceptibility using random forests. *Cancer Epidemiology Biomarkers & Prevention*, 17(6):1368–1373, 2008.

S. Chatrchyan, V. Khachatryan, A. Sirunyan, A. Tumasyan, W. Adam, E. Aguilo, T. Bergauer, M. Dragicevic, J. Erö, C. Fabjan, et al. Observation of a new boson at a mass of 125 GeV with the CMS experiment at the LHC. *Physics Letters B*, 2012.

N. V. Chawla, L. O. Hall, K. W. Bowyer, and W. P. Kegelmeyer. Learning ensembles from bites: A scalable and accurate approach. *J. Mach. Learn. Res.*, 5:421–451, Dec. 2004. ISSN 1532-4435.

T. H. Cormen, C. E. Leiserson, et al. *Introduction to algorithms*, volume 2. 2001.

C. Cortes and V. Vapnik. Support-vector networks. *Machine learning*, 20(3):273–297, 1995.

T. Cover and P. Hart. Nearest neighbor pattern classification. *Information Theory, IEEE Transactions on*, 13(1):21–27, 1967.

T. M. Cover and J. A. Thomas. *Elements of information theory*. Wiley-interscience, 2012.

A. Criminisi and J. Shotton. *Decision Forests for Computer Vision and Medical Image Analysis*. Springer, 2013.

A. Cutler and G. Zhao. Pert-perfect random tree ensembles. *Computing Science and Statistics*, 33:490–497, 2001.

R. L. De Mántaras. A distance-based attribute selection measure for decision tree induction. *Machine learning*, 6(1):81–92, 1991.

J. Demsar. Statistical comparisons of classifiers over multiple data sets. *The Journal of Machine Learning Research*, 7:1–30, 2006.

201

L. Breiman. Consistência para um modelo simples de florestas aleatórias. Relatório técnico, UC Berkeley, 2004. L. Breiman, J. H. Friedman, R. A. Olshen e C. J. Stone. Árvores de classificação e regressão. 1984. AE Bryson. Controle ótimo aplicado: otimização, estimativa e controle. CRC Press, 1975. L. Buitinck, G. Louppe, M. Blondel, F. Pedregosa, A. Mueller, O. Grisel, V. Niculae, P. Prettenhofer, A. Gramfort, J. Grobler, et al. Design de API para software de aprendizado de máquina: experiências do projeto scikit-learn. pré-impressão arXiv arXiv:1309.0238, 2013. JS Chang, R.-F. Si m, JK Wiencke, JL Wiemels, I. Smirnov, AR Pico, T. Tihan, J. Patoka, R. Miike, JD Sison, et al. Análise da via de polimorfismos de nucleotídeo único potencialmente associados à suscetibilidade ao glioblastoma multiforme usando florestas aleatórias. Cancer Epidemiology Biomarkers & Prevention, 17(6):1368–1373, 2008. S. Chatrchyan, V. Khachatryan, A. Sirunyan, A. Tumasyan, W. Adam, E. Aguilo, T. Bergauer, M. Dragicevic, J. Erö, C. Fabjan, et al. Observação de um novo bóson com massa de 125 GeV com o experimento CMS no LHC. Letras de Física B, 2012. NV Chawla, LO Hall, KW Bowyer e WP Kegelmeyer. Aprendendo conjuntos a partir de mordidas: uma abordagem escalável e precisa. J. Mach. Aprender. Res., 5:421–451, dezembro de 2004. ISSN 1532-4435. TH Cormen, CE Leiserson, et al. Introdução aos algoritmos, volume 2. 2001. C. Cortes e V. Vapnik. Redes de vetores de suporte. Aprendizado de máquina, 20(3):273–297, 1995. T. Cover e P. Hart. Classificação do padrão do vizinho mais próximo. Teoria da Informação, IEEE Transactions on, 13(1):21–27, 1967. T. M. Cover e J. A. Thomas. Elementos da teoria da informação. Wiley- interciência, 2012. A. Criminisi e J. Shotton. Florestas de decisão para visão computacional e análise de imagens médicas. Springer, 2013. A. Cutler e G. Zhao. Conjuntos de árvores aleatórias perfeitamente perfeitos. Ciência da Computação e Estatística, 33:490–497, 2001. R. L. De Mántaras. Uma medida de seleção de atributos baseada em distância para indução de árvores de decisão. Aprendizado de máquina, 6(1):81–92, 1991. J. Demsar. Comparações estatísticas de classificadores em vários conjuntos de dados. O Journal of Machine Learning Research, 7:1–30, 2006.

J. Demšar, T. Curk, A. Erjavec, Črt Gorup, T. Hočevar, M. Milutinovič, M. Možina, M. Polajnar, M. Toplak, A. Starič, M. Štajdohar, L. Umek, L. Žagar, J. Žbontar, M. Žitnik, and B. Zupan. Orange: Data mining toolbox in python. *Journal of Machine Learning Research*, 14:2349–2353, 2013.

M. Denil, D. Matheson, and N. De Freitas. Narrowing the gap: Random forests in theory and in practice. *arXiv preprint arXiv:1310.1415*, 2013a.

M. Denil, D. Matheson, and N. de Freitas. Consistency of online random forests. *arXiv preprint arXiv:1302.4853*, 2013b.

L. Devroye, L. Gyorfi, and G. Lugosi. A probabilistic theory of pattern recognition, 1996.

T. G. Dietterich. An experimental comparison of three methods for constructing ensembles of decision trees: Bagging, boosting, and randomization. *Machine learning*, 40(2):139–157, 2000a.

T. G. Dietterich. Ensemble methods in machine learning. In *Multiple classifier systems*, pages 1–15. Springer, 2000b.

T. G. Dietterich and E. B. Kong. Machine learning bias, statistical bias, and statistical variance of decision tree algorithms. *ML-95*, 255, 1995.

P. Domingos. A unified bias-variance decomposition for zero-one and squared loss. *AAAI/IAAI*, 2000:564–569, 2000.

R. Duda, P. Hart, and D. Stork. *Pattern classification*. John Wiley & Sons, 2012.

B. Efron. Bootstrap methods: another look at the jackknife. *The annals of Statistics*, pages 1–26, 1979.

D. Ferrucci, E. Brown, J. Chu-Carroll, J. Fan, D. Gondek, A. Kalyanpur, A. Lally, J. W. Murdock, E. Nyberg, J. Prager, et al. Building Watson: An overview of the DeepQA project. *AI magazine*, 31(3):59–79, 2010.

W. D. Fisher. On grouping for maximum homogeneity. *Journal of the American Statistical Association*, 53(284):789–798, 1958.

E. Fix and J. L. Hodges. Disciminatory analysis—nonparametric discrimination; consistency properties. Technical Report Project 21-49-004, Report No. 4, USAF School of Aviation Medicine, 1951.

A. Frank and A. Asuncion. UCI machine learning repository, 2010. URL http://archive.ics.uci.edu/ml.

Y. Freund and R. E. Schapire. A desicion-theoretic generalization of on-line learning and an application to boosting. In *Computational learning theory*, pages 23–37. Springer, 1995.

202

J. Demšar, T. Curk, A. Erjavec, ˇCrt Gorup, T. Ho ˇcevar, M. Milutinovi ˇc, M. Možina, M. Polajnar, M. Toplak, A. Stari ˇc, M. Štajdohar, L. Umek, L. Žagar, J. Žbontar, M. Žitnik e B. Zupan. Laranja: caixa de ferramentas de mineração de dados em python. Journal of Machine Learning Research, 14:2349–2353, 2013. M. Denil, D. Matheson e N. De Freitas. Reduzindo a lacuna: Florestas aleatórias na teoria e na prática. Pré-impressão do arXiv arXiv:1310.1415, 2013a. M. Denil, D. Matheson e N. de Freitas. Consistência de florestas aleatórias online. Pré-impressão do arXiv arXiv:1302.4853, 2013b. L. Devroye, L. Gyorfi e G. Lugosi. Uma teoria probabilística de reco nhecimento de padrões, 1996. T. G. Dietterich. Uma comparação experimental de três métodos para construção de conjuntos de árvores de decisão: Bagging, boosting e randomização. Aprendizado de máquina, 40(2):139–157, 2000a. T. G. Dietterich. Métodos ensemble em aprendizado de máquina. Em Sistemas classificadores múltiplos, páginas 1–15. Springer, 2000b. TG Dietterich e EB Kong. Viés de aprendizado de máquina, viés estatístico e variância estatística de algoritmos de árvore de decisão. ML-95, 255, 1995. P. Domingos. Uma decomposição unificada de polarização-variância para zero-um e perda quadrada. AAAI/IAAI, 2000:564–569, 2000. R. Duda, P. Hart e D. Stork. Classificação de padrões. John Wiley & Filhos, 2012. B. Efron. Métodos Bootstrap: outra olhada no canivete. The Annals of Statistics, páginas 1–26, 1979. D. Ferrucci, E. Brown, J. Chu-Carroll, J. Fan, D. Gondek, A. Kalyanpur, A. Lally, J. W. Murdock, E. Nyberg, J. Prager, et al. Construindo Watson: uma visão geral do projeto DeepQA. Revista AI, 31(3):59–79, 2010. WD Fisher. No agrupamento para máxima homogeneidade. Journal of the American Statistical Association, 53(284):789–798, 1958. E. Fix e JL Hodges. Análise discriminatória – discriminação não paramétrica; propriedades de consistência. Projeto de Relatório Técnico 21-49-004, Relatório No. 4, Escola de Medicina de Aviação da USAF, 1951. A. Frank e A. Assunção. Repositório de aprendizado de máquina UCI, 2010. URL http://archive.ics.uci.edu/ml. Y. Freund e RE Schapire. Uma generalização teórica de decisão da aprendizagem on-line e uma aplicação para impulsionar. Na teoria da aprendizagem computacional, páginas 23–37. Springer, 1995.

J. H. Friedman. A recursive partitioning decision rule for nonparametric classification. *Computers, IEEE Transactions on*, 100(4):404–408, 1977.

J. H. Friedman. A tree-structured approach to nonparametric multiple regression. In *Smoothing techniques for curve estimation*, pages 5–22. Springer, 1979.

J. H. Friedman. Multivariate adaptive regression splines. *The annals of statistics*, pages 1–67, 1991.

J. H. Friedman. On bias, variance, 0/1 loss, and the curse-of-dimensionality. *Data mining and knowledge discovery*, 1(1):55–77, 1997.

J. H. Friedman. Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, pages 1189–1232, 2001.

J. Gama. Functional trees. *Machine Learning*, 55(3):219–250, 2004.

S. Geman, E. Bienenstock, and R. Doursat. Neural networks and the bias/variance dilemma. *Neural computation*, 4(1):1–58, 1992.

R. Genuer, J.-M. Poggi, and C. Tuleau-Malot. Variable selection using random forests. *Pattern Recognition Letters*, 31(14):2225–2236, 2010.

P. Geurts. Contributions to decision tree induction: bias/variance tradeoff and time series classification. 2002.

P. Geurts. Bias vs variance decomposition for regression and classification. In *Data Mining and Knowledge Discovery Handbook*, pages 749–763. Springer, 2005.

P. Geurts and G. Louppe. Learning to rank with extremely randomized trees. In *JMLR: Workshop and Conference Proceedings*, volume 14, 2011.

P. Geurts and L. Wehenkel. Investigation and reduction of discretization variance in decision tree induction. In *Machine Learning: ECML 2000*, pages 162–170. Springer, 2000.

P. Geurts, D. Ernst, and L. Wehenkel. Extremely randomized trees. *Machine Learning*, 63(1):3–42, 2006a.

P. Geurts, L. Wehenkel, and F. d'Alché Buc. Kernelizing the output of tree-based methods. In *Proceedings of the 23rd international conference on Machine learning*, pages 345–352. Acm, 2006b.

P. Geurts, A. Irrthum, and L. Wehenkel. Supervised learning with decision tree-based methods in computational and systems biology. *Molecular Biosystems*, 5(12):1593–1605, 2009.

203

JH Friedman. Uma regra de decisão de particionamento recursivo para classificação não paramétrica. Computadores, Transações IEEE em, 100(4):404–408, 1977. JH Friedman. Uma abordagem estruturada em árvore para regressão múltipla não paramétrica. Em Técnicas de suavização para estimativa de curva, páginas 5–22. Springer, 1979. JH Friedman. Splines de regressão adaptativa multivariada. Os anais das estatísticas, páginas 1–67, 1991. J. H. Friedman. Sobre viés, variância, perda de 0/1 e a maldição da dimensionalidade. Mineração de dados e descoberta de conhecimento, 1(1):55–77, 1997. JH Friedman. Aproximação de função gananciosa: uma máquina de aumento de gradiente. Anais de Estatística, páginas 1189–1232, 2001. J. Gama. Árvores funcionais. Aprendizado de Máquina, 55(3):219–250, 2004. S. Geman, E. Bienenstock e R. Doursat. Redes neurais e o dilema viés/variância. Computação neural, 4(1):1–58, 1992. R. Genuer, J.-M. Poggi e C. Tuleau-Malot. Seleção de variáveis usando florestas aleatórias. Cartas de reconhecimento de padrões, 31(14):2225–2236, 2010. P. Geurts. Contribuições para a indução de árvores de decisão: compensação de viés/variância e classificação de séries temporais. 2002. P. Geurts. Decomposição de viés versus variância para regressão e classificação. No Manual de mineração de dados e descoberta de conhecimento, páginas 749–763. Springer, 2005. P. Geurts e G. Louppe. Aprendendo a classificar árvores extremamente aleatórias. E m JMLR: Anais de Workshop e Conferência, volume 14, 2011. P. Geurts e L. Wehenkel. Investigação e redução da variância da discretização na indução de árvores de decisão. Em Machine Learning: ECML 2000, páginas 162–170. Springer, 2000. P. Geurts, D. Ernst e L. Wehenkel. Árvores extremamente aleatórias. Aprendizado de Máquina, 63(1):3–42, 2006a. P. Geurts, L. Wehenkel e F. d'Alché Buc. Kernelizando a saída de métodos baseados em árvore. Em Anais da 23ª conferência internacional sobre aprendizado de máquina, páginas 345–352. Acm, 2006b. P. Geurts, A. Irrthum e L. Wehenkel. Aprendizagem supervisionada com métodos baseados em árvores de decisão em biologia computacional e de sistemas. Biossistemas Moleculares, 5(12):1593–1605, 2009.

M. Gillo. Maid: A honeywell 600 program for an automatised survey analysis. *Behavioral Science*, 17:251–252, 1972.

C. Gini. Variabilità e mutabilità. *Reprinted in Memorie di metodologica statistica (Ed. Pizetti E, Salvemini, T). Rome: Libreria Eredi Virgilio Veschi*, 1, 1912.

B. Goebel, Z. Dawy, J. Hagenauer, and J. C. Mueller. An approximation to the distribution of finite sample size mutual information estimates. In *Communications, 2005. ICC 2005. 2005 IEEE International Conference on*, volume 2, pages 1102–1106. IEEE, 2005.

M. T. Goodrich and R. Tamassia. *Algorithm Design: Foundation, Analysis and Internet Examples*. John Wiley & Sons, 2006.

I. Guyon. A scaling law for the validation-set training-set size ratio. *AT&T Bell Laboratories*, 1997.

I. Guyon and A. Elisseeff. An introduction to variable and feature selection. *The Journal of Machine Learning Research*, 3:1157–1182, 2003.

I. Guyon, J. Weston, S. Barnhill, and V. Vapnik. Gene selection for cancer classification using support vector machines. *Machine learning*, 46(1-3):389–422, 2002.

M. Hall, E. Frank, G. Holmes, B. Pfahringer, P. Reutemann, and I. H. Witten. The WEKA data mining software: an update. *ACM SIGKDD Explorations Newsletter*, 11(1):10–18, 2009.

T. Hastie, R. Tibshirani, J. Friedman, and J. Franklin. The elements of statistical learning: data mining, inference and prediction. *The Mathematical Intelligencer*, 27(2):83–85, 2005.

D. Heath, S. Kasif, and S. Salzberg. Induction of oblique decision trees. In *IJCAI*, pages 1002–1007. Citeseer, 1993.

G. Hinton. Learning multiple layers of representation. *Trends in cognitive sciences*, 11(10):428–434, 2007.

T. K. Ho. The random subspace method for constructing decision forests. *Pattern Analysis and Machine Intelligence, IEEE Transactions on*, 20(8):832–844, 1998.

A. Hoecker, P. Speckmayer, J. Stelzer, J. Therhaag, E. von Toerne, H. Voss, M. Backes, T. Carli, O. Cohen, A. Christov, et al. Tmva-toolkit for multivariate data analysis. *arXiv preprint physics/0703039*, 2007.

T. Hothorn, K. Hornik, and A. Zeileis. Unbiased recursive partitioning: A conditional inference framework. *Journal of Computational and Graphical Statistics*, 15(3):651–674, 2006.

204

M. Gillo. Maid: Um programa honeywell 600 para análise automatizada de pesquisas. Ciência Comportamental, 17:251–252, 1972. C. Gini. Variabilidade e mutabilidade. Reimpresso em Memorie di metodologica estatística (Ed. Pizetti E, Salvemini, T). Roma: Libreria Eredi Virgilio Veschi, 1, 1912. B. Goebel, Z. Dawy, J. Hagenauer e J. C. Mueller. Uma aproximação para a distribuição de estimativas de informações mútuas de tamanho de amostra finita. Em Comunicações, 2005. ICC 2005. 2005 IEEE International Conference on, volume 2, páginas 1102–1106. IEEE, 2005. MT Goodrich e R. Tamassia. Projeto de Algoritmos: Fundamentos, Análise e Exemplos de Internet. John Wiley & Filhos, 2006. I. Guyon. Uma lei de escalonamento para a proporção do tamanho do conjunto de treinamento do conjunto de validação. Laboratórios AT&T Bell, 1997. I. Guyon e A. Elisseeff. Uma introdução à seleção de variáveis e recursos. The Journal of Machine Learning Research, 3:1157–1182, 2003. I. Guyon, J. Weston, S. Barnhill e V. Vapnik. Seleção de genes para classificação de câncer usando máquinas de vetores de suporte. Aprendizado de máquina, 46(1-3):389–422, 2002. M. Hall, E. Frank, G. Holmes, B. Pfahringer, P. Reutemann e IH Witten. O software de mineração de dados WEKA: uma atualização. Boletim Informativo de Explorações ACM SIGKDD, 11(1):10–18, 2009. T. Hastie, R. Tibshirani, J. Friedman e J. Franklin. Os elementos da aprendizagem estatística: mineração de dados, inferência e previsão. O Inteligência Matemática, 27(2):83–85, 2005. D. Heath, S. Kasif e S. Salzberg. Indução de árvores de decisão oblíquas. No IJCAI, páginas 1002–1007. Citeseer, 1993. G. Hinton. Aprendendo múltiplas camadas de representação. Tendências em ciências cognitivas, 11(10):428–434, 2007. T. K. Ho. O método de subespaço aleatório para construção de florestas de decisão. Análise de padrões e inteligência de máquina, IEEE Transactions on, 20(8):832–844, 1998. A. Hoecker, P. Speckmayer, J. Stelzer, J. Therhaag, E. von Toerne, H. Voss, M. Backes, T. Carli, O. Cohen, A. Christov, et al. Tmva- kit de ferramentas para análise multivariada de dados. física de pré-impressão arXiv/0703039, 2007. T. Hothorn, K. Hornik e A. Zeileis. Particionamento recursivo imparcial: Uma estrutura de inferência condicional. Jornal de Estatística Computacional e Gráfica, 15(3):651–674, 2006.

J. D. Hunter. Matplotlib: A 2d graphics environment. *CiSE*, pages 90–95, 2007.

V. A. Huynh-Thu, A. Irrthum, L. Wehenkel, P. Geurts, et al. Inferring regulatory networks from expression data using tree-based methods. *PloS one*, 5(9):e12776, 2010.

V. A. Huynh-Thu, Y. Saeys, L. Wehenkel, P. Geurts, et al. Statistical interpretation of machine learning-based feature importance scores for biomarker discovery. *Bioinformatics*, 28(13):1766–1774, 2012.

L. Hyafil and R. L. Rivest. Constructing optimal binary decision trees is np-complete. *Information Processing Letters*, 5(1):15–17, 1976.

H. Ishwaran. Variable importance in binary regression trees and forests. *Electronic Journal of Statistics*, 1:519–537, 2007.

H. Ishwaran and U. B. Kogalur. Consistency of random survival forests. *Statistics & probability letters*, 80(13):1056–1064, 2010.

G. M. James. Variance and bias for general loss functions. *Machine Learning*, 51(2):115–135, 2003.

H. Kim and W.-Y. Loh. Classification trees with unbiased multiway splits. *Journal of the American Statistical Association*, 96(454), 2001.

K. Kira and L. A. Rendell. The feature selection problem: Traditional methods and a new algorithm. In *AAAI*, pages 129–134, 1992.

D. Knuth. *The Art of Computer Programming: Fundamental algorithms*. The Art of Computer Programming. Addison-Wesley, 1968.

D. E. Knuth. Two notes on notation. *The American Mathematical Monthly*, 99(5):403–422, 1992.

D. Kocev, C. Vens, J. Struyf, and S. Džeroski. *Ensembles of multi-objective decision trees*. Springer, 2007.

R. Kohavi and G. H. John. Wrappers for feature subset selection. *Artificial intelligence*, 97(1):273–324, 1997.

R. Kohavi, D. H. Wolpert, et al. Bias plus variance decomposition for zero-one loss functions. In *ICML*, pages 275–283, 1996.

R. Kohavi et al. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *IJCAI*, volume 14, pages 1137–1145, 1995.

I. Kononenko. On biases in estimating multi-valued attributes. In *IJCAI*, volume 95, pages 1034–1040, 1995.

A. Krogh, J. Vedelsby, et al. Neural network ensembles, cross validation, and active learning. *Advances in neural information processing systems*, pages 231–238, 1995.

205

JD Hunter. Matplotlib: Um ambiente gráfico 2D. CiSE, páginas 90–95, 2007. VA Huynh-Thu, A. Irrthum, L. Wehenkel, P. Geurts, et al. Inferir redes regulatórias a partir de dados de expressão usando métodos baseados em árvore. PloS um, 5(9):e12776, 2010. VA Huynh-Thu, Y. Saeys, L. Wehenkel, P. Geurts, et al. Interpretação estatística de pontuações de importância de recursos baseadas em aprendizado de máquina para descoberta de biomarcadores. Bioinformática, 28(13):1766–1774, 2012. L. Hyafil e RL Rivest. A construção de árvores de decisão binárias ideais é np-completa. Cartas de Processamento de Informações, 5(1):15–17, 1976. H. Ishwaran. Importância variável em árvores e florestas de regressão binária. Electronic Journal of Statistics, 1:519–537, 2007. H. Ishwaran e UB Kogalur. Consistência de florestas de sobrevivência aleatórias. Estatísticas e cartas de probabilidade, 80(13):1056–1064, 2010. GM James. Variância e viés para funções de perda geral. Aprendizado de Máquina, 51(2):115–135, 2003. H. Kim e W.-Y. Ah. Árvores de classificação com divisões multivias i mparciais. Jornal da American Statistical Association, 96(454), 2001. K. Kira e LA Rendell. O problema de seleção de recursos: métodos t radicionais e um novo algoritmo. Em AAAI, páginas 129–134, 1992. D. Knuth. A Arte da Programação de Computadores: Algoritmos Fundamentais. A Arte da Programação de Computadores. Addison-Wesley, 1968. DE Knuth. Duas notas sobre notação. The American Mathematical Monthly, 99(5):403–422, 1992. D. Kocev, C. Vens, J. Struyf e S. Džeroski. Conjuntos de árvores de decisão multiobjetivos. Springer, 2007. R. Kohavi e GH John. Wrappers para seleção de subconjunto de recursos. Inteligência artificial, 97(1):273–324, 1997. R. Kohavi, DH Wolpert, et al. Decomposição de polarização mais variância para funções de perda zero-um. No ICML, páginas 275–283, 1996. R. Kohavi et al. Um estudo de validação cruzada e bootstrap para estimativa de precisão e seleção de modelos. Em IJCAI, volume 14, páginas 1137–1145, 1995. I. Kononenko. Sobre vieses na estimativa de atributos com múltiplos valores. Em IJCAI, volume 95, páginas 1034–1040, 1995. A. Krogh, J. Vedelsby, et al. Conjuntos de redes neurais, validação cruzada e aprendizagem ativa. Avanços em sistemas de processamento de informação neural, páginas 231–238, 1995.

J. B. Kruskal. Multidimensional scaling by optimizing goodness of fit to a nonmetric hypothesis. *Psychometrika*, 29(1):1–27, 1964.

R. Kufrin. Decision trees on parallel processors. *Machine Intelligence and Pattern Recognition*, 20:279–306, 1997.

L. I. Kuncheva and J. J. Rodríguez. An experimental study on rotation forest ensembles. In *Multiple Classifier Systems*, pages 459–468. Springer, 2007.

S. W. Kwok and C. Carter. Multiple decision trees. In *Proceedings of the Fourth Annual Conference on Uncertainty in Artificial Intelligence*, pages 327–338. North-Holland Publishing Co., 1990.

Y. Liao, A. Rubinsteyn, R. Power, and J. Li. Learning Random Forests on the GPU. 2013.

A. Liaw and M. Wiener. Classification and regression by randomforest. *R News*, 2(3):18–22, 2002. URL http://CRAN.R-project.org/doc/Rnews/.

Y. Lin and Y. Jeon. Random forests and adaptive nearest neighbors. 2002.

H. Liu and L. Yu. Toward integrating feature selection algorithms for classification and clustering. *Knowledge and Data Engineering, IEEE Transactions on*, 17(4):491–502, 2005.

G. Louppe and P. Geurts. A zealous parallel gradient descent algorithm. 2010.

G. Louppe and P. Geurts. Ensembles on random patches. In *Machine Learning and Knowledge Discovery in Databases*, pages 346–361. Springer, 2012.

G. Louppe, L. Wehenkel, A. Sutera, and P. Geurts. Understanding variable importances in forests of randomized trees. In *Advances in Neural Information Processing Systems*, pages 431–439, 2013.

K. L. Lunetta, L. B. Hayward, J. Segal, and P. Van Eerdewegh. Screening large-scale association study data: exploiting interactions using random forests. *BMC genetics*, 5(1):32, 2004.

P. MacCullagh and J. Nelder. *Generalized linear models*, volume 37. CRC press, 1989.

O. Z. Maimon and L. Rokach. *Data mining and knowledge discovery handbook*, volume 1. Springer, 2005.

D. Marbach, J. C. Costello, R. Küffner, N. M. Vega, R. J. Prill, D. M. Camacho, K. R. Allison, M. Kellis, J. J. Collins, G. Stolovitzky, et al. Wisdom of crowds for robust gene network inference. *Nature methods*, 9(8):796–804, 2012.

206

JB Kruskal. Dimensionamento multidimensional otimizando a qualidade do ajuste a uma hipótese não métrica. Psicometria, 29(1):1–27, 1964. R. Kufrin. Árvores de decisão em processadores paralelos. Inteligência de Máquina e Reconhecimento de Padrões, 20:279–306, 1997. L. I. Kuncheva e J. J. Rodríguez. Um estudo experimental sobre conjuntos florestais de rotação. Em Sistemas Classificadores Múltiplos, páginas 459–468. Springer, 2007. S W Kwok e C. Carter. Múltiplas árvores de decisão. Nos Anais da Quarta Conferência Anual sobre Incerteza em Inteligência Artificial, páginas 327–338. North-Holland Publishing Co., 1990. Y. Liao, A. Rubinsteyn, R. Power e J. Li. Aprendendo Florestas Aleatórias na GPU. 2013. A. Liaw e M. Wiener. Classificação e regressão por randomforest. R News, 2(3):18–22, 2002. URL http://CRAN.R-project.org/doc/Rnews/. Y. Lin e Y. Jeon. Florestas aleatórias e vizinhos mais próximos adaptativos. 2002. H. Liu e L. Yu. Para integrar algoritmos de seleção de recursos para classificação e clustering. Conhecimento e Engenharia de Dados, Transações IEEE em, 17(4):491–502, 2005. G. Louppe e P. Geurts. Um zeloso algoritmo de descida de gradiente paralelo. 2010. G. Louppe e P. Geurts. Conjuntos em patches aleatórios. Em Machine Learning and Knowledge Discovery in Databases, páginas 346–361. Springer, 2012. G. Louppe, L. Wehenkel, A. Sutera e P. Geurts. Compreensão das importâncias variáveis em florestas de árvores aleatórias. Em Avanços em Sistemas de Processamento de Informação Neural, páginas 431–439, 2013. KL Lunetta, LB Hayward, J. Segal e P. Van Eerdewegh. Triagem de dados de estudos de associação em larga escala: explorando interações usando florestas aleatórias. Genética BMC, 5(1):32, 2004. P. MacCullagh e J. Nelder. Modelos lineares generalizados, volume 37. CRC press, 1989. O. Z. Maimon e L. Rokach. Manual de mineração de dados e descoberta de conhecimento, volume 1. Springer, 2005. D. Marbach, J. C. Costello, R. Küffner, N. M. Vega, R. J. Prill, D. M. Camacho, K. R. Allison, M. Kellis, J. J. Collins, G. Stolovitzky, et al. Sabedoria das multidões para inferência robusta de redes genéticas. Métodos da natureza, 9(8):796–804, 2012.

R. Marée, L. Wehenkel, and P. Geurts. Extremely randomized trees and random subwindows for image classification, annotation, and retrieval. In *Decision Forests for Computer Vision and Medical Image Analysis*, pages 125–141. Springer, 2013.

R. Marée, L. Rollus, B. Stevens, G. Louppe, O. Caubo, N. Rocks, S. Bekaert, D. Cataldo, and L. Wehenkel. A hybrid human-computer approach for large-scale image-based measurements using web services and machine learning. *Proceedings IEEE International Symposium on Biomedical Imaging*, 2014.

A. McGovern, D. John Gagne II, L. Eustaquio, G. Titericz, B. Lazorthes, O. Zhang, G. Louppe, P. Prettenhofer, J. Basara, T. Hamill, and D. Margolin. Solar energy prediction: An international contest to initiate interdisciplinary research on compelling meteorological problems. 2014.

N. Meinshausen. Quantile regression forests. *The Journal of Machine Learning Research*, 7:983–999, 2006.

Y. A. Meng, Y. Yu, L. A. Cupples, L. A. Farrer, and K. L. Lunetta. Performance of random forest when snps are in linkage disequilibrium. *BMC bioinformatics*, 10(1):78, 2009.

R. Messenger and L. Mandell. A modal search technique for predictive nominal scale multivariate analysis. *Journal of the American statistical association*, 67(340):768–772, 1972.

J. Mingers. An empirical comparison of pruning methods for decision tree induction. *Machine learning*, 4(2):227–243, 1989a.

J. Mingers. An empirical comparison of selection measures for decision-tree induction. *Machine learning*, 3(4):319–342, 1989b.

L. Mitchell, T. M. Sloan, M. Mewissen, P. Ghazal, T. Forster, M. Piotrowski, and A. S. Trew. A parallel random forest classifier for r. In *Proceedings of the second international workshop on Emerging computational methods for the life sciences*, pages 1–6. ACM, 2011.

T. Mitchell. *Machine learning*. McGraw-Hill, New York, 1997.

M. Miyakawa. Criteria for selecting a variable in the construction of efficient decision trees. *Computers, IEEE Transactions on*, 38(1): 130–141, 1989.

F. Moosmann, B. Triggs, F. Jurie, et al. Fast discriminative visual codebooks using randomized clustering forests. In *NIPS*, volume 2, page 4, 2006.

J. Morgan and R. Messenger. Thaid: a sequential search program for the analysis of nominal scale dependent variables. *Survey Research Center, Institute for Social Research, University of Michigan.[251]*, 1973.

207

R. Marée, L. Wehenkel e P. Geurts. Árvores extremamente aleatórias e subjanelas aleatórias para classificação, anotação e recuperação de imagens. Em Florestas de Decisão para Visão Computacional e Análise de Imagens Médicas, páginas 125–141. Springer, 2013. R. Marée, L. Rollus, B. Stevens, G. Louppe, O. Caubo, N. Rocks, S. Bekaert, D. Cataldo e L. Wehenkel. Uma abordagem híbrida humano-computador para medições baseadas em imagens em grande escala usando serviços web e aprendizado de máquina. Proceedings IEEE International Symposium on Biomedical Imaging, 2014. A. McGovern, D. John Gagne II, L. Eustaquio, G. Titericz, B. Lazorthes, O. Zhang, G. Louppe, P. Prettenhofer, J. Basara, T. Hamill e D. Margolin. Previsão de energia solar: Um concurso i nternacional para iniciar pesquisas interdisciplinares sobre problemas meteorológicos convincentes. 2014. N. Meinshausen. Florestas de regress ão quantílica. The Journal of Machine Learning Research, 7: 983–999, 2006. Y. A. Meng, Y. Yu, L. A. Cupples, L. A. Farrer e K. L. Lunetta. Desempenho de floresta aleatória quando snps estão em desequilíbrio de ligação. Bioinformática BMC, 10(1):78, 2009. R. Messenger e L. Mandell. Uma t écnica de busca modal para análise multivariada preditiva em escala nominal. Jornal da associação estatística americana, 67(340):768–772, 1972. J. Mingers. Uma comparação empírica de métodos de poda para indução de árvores de decisão. Aprendizado de máquina, 4(2):227–243, 1989a. J. Mingers. Uma comparação empírica de medidas de seleção para indução de árvores de decisão. Aprendizado de máquina, 3(4):319–342, 1989b. L. Mitchell, T. M. Sloan, M. Mewissen, P. Ghazal, T. Forster, M. Piotrowski e A. S. Trew. Um classificador de floresta aleatório paralelo para r. Em Anais do segundo workshop internacional sobre métodos computacionais emergentes para as ciências da vida, páginas 1–6. ACM, 2011. T. Mitchell. Aprendizado de máquina. McGraw-Hill, Nova York, 1997. M. Miyakawa. Critérios para seleção de uma variável na construção de árvores de decisão eficientes. Computadores, Transações IEEE em, 38(1): 130–141, 1989. F. Moosmann, B. Triggs, F. Jurie, et al. Livros de códigos visuais discriminativos rápidos usando florestas de agrupamento aleatório. Em NIPS, volume 2, página 4, 2006. J. Morgan e R. Messenger. Thaid: progra ma de busca sequencial para análise de variáveis dependentes de escala nominal. Centro de Pesquisa de Pesquisa, Instituto de Pesquisa Social, Universidade de Michigan.*[251]*, 1973.

J. N. Morgan and J. A. Sonquist. Problems in the analysis of survey data, and a proposal. *Journal of the American Statistical Association*, 58(302):415–434, 1963.

K. V. S. Murthy and S. L. Salzberg. *On growing better decision trees from data*. PhD thesis, Citeseer, 1995.

D. R. Musser. Introspective sorting and selection algorithms. *Softw., Pract. Exper.*, 27(8):983–993, 1997.

C. Nadeau and Y. Bengio. Inference for the generalization error. *Machine Learning*, 52(3):239–281, 2003.

R. Narayanan, D. Honbo, G. Memik, A. Choudhary, and J. Zambreno. Interactive presentation: An fpga implementation of decision tree classification. In *Proceedings of the conference on Design, automation and test in Europe*, pages 189–194. EDA Consortium, 2007.

C. Olaru and L. Wehenkel. A complete fuzzy decision tree technique. *Fuzzy sets and systems*, 138(2):221–254, 2003.

T. E. Oliphant. Python for scientific computing. *CiSE*, 9(3):10–20, 2007.

H. Pang, A. Lin, M. Holford, B. E. Enerson, B. Lu, M. P. Lawton, E. Floyd, and H. Zhao. Pathway analysis using random forests classification and regression. *Bioinformatics*, 22(16):2028–2036, 2006.

P. Panov and S. Džeroski. Combining bagging and random subspaces to create better ensembles. *Advances in intelligent data analysis VII*, pages 118–129, 2007.

F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al. Scikit-learn: Machine learning in python. *The Journal of Machine Learning Research*, 12:2825–2830, 2011. URL http://scikit-learn.org.

F. Perez and B. E. Granger. IPython: a system for interactive scientific computing. *CiSE*, 9(3):21–29, 2007.

Y. Qi, Z. Bar-Joseph, and J. Klein-Seetharaman. Evaluation of different biological data and computational classification methods for use in protein interaction prediction. *Proteins: Structure, Function, and Bioinformatics*, 63(3):490–500, 2006.

J. R. Quinlan. *Discovering rules by induction from large collections of examples*. Expert systems in the micro electronic age. Edinburgh University Press, 1979.

J. R. Quinlan. Induction of decision trees. *Machine learning*, 1(1):81–106, 1986.

208

JN Morgan e JA Sonquist. Problemas na análise de dados de pesquisas e u ma proposta. Journal of the American Statistical Association, 58(302):415–434, 1963. KVS Murthy e SL Salzberg. Sobre o crescimento de melhores árvores de decisão a partir de dados. Tese de doutorado, Citeseer, 1995. DR Musser. Algoritmos introspectivos de ordenação e seleção. Softw., Prac. Exper., 27(8):983–993, 1997. C. Nadeau e Y. Bengio. Inferência para o erro de generalização. Aprendizado de Máquina, 52(3):239–281, 2003. R. Narayanan, D. Honbo, G. Memik, A. Choudhary e J. Zambreno. Apresentação interativa: Uma implementação FPGA de classificação de árvore de decisão. Em Anais da conferência sobre Design, automação e teste na Europa, páginas 189–194. Consórcio EDA, 2007. C. Olaru e L. Wehenkel. Uma técnica completa de árvore de decisão fuzzy. Conjuntos e sistemas difusos, 138(2):221–254, 2003. TE Oliphant. Python para computação científica. CiSE, 9(3):10–20, 2007. H. Pang, A. Lin, M. Holford, BE Enerson, B. Lu, MP Lawton, E. Floyd e H. Zhao. Análise de caminhos usando classificação e regressão de florestas aleatórias. Bioinformática, 22(16):2028–2036, 2006. P. Panov e S. Džeroski. Combinando ensacamento e subespaços aleatórios para criar conjuntos melhores. Avanços na análise inteligente de dados VII, páginas 118–129, 2007. F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al. Scikit-learn: Aprendizado de máquina em python. The Journal of Machine Learning Research, 12:2825–2830, 2011. URL http://scikit-learn. organização. F. Perez e BE Granger. IPython: um sistema para computação científica interativa. CiSE, 9(3):21–29, 2007. Y. Qi, Z. Bar-Joseph e J. Klein-Seetharaman. Avaliação de diferentes dados biológicos e métodos de classificação computacional para uso na predição de interações proteicas. Proteínas: Estrutura, Função e Bioinformática, 63(3):490–500, 2006. JR Quinlan. Descobrir regras por indução a partir de grandes coleções de exemplos. Sistemas especialistas na era microeletrônica. Imprensa da Universidade de Edimburgo, 1979. JR Quinlan. Indução de árvores de decisão. Aprendizado de máquina, 1(1):81–106, 1986.

J. R. Quinlan. *C4.5: programs for machine learning*, volume 1. Morgan kaufmann, 1993.

J. J. Rodriguez, L. I. Kuncheva, and C. J. Alonso. Rotation forest: A new classifier ensemble method. *Pattern Analysis and Machine Intelligence, IEEE Transactions on*, 28(10):1619–1630, 2006.

B. Scholkopf and A. J. Smola. *Learning with kernels: support vector machines, regularization, optimization, and beyond*. MIT press, 2001.

D. F. Schwarz, I. R. König, and A. Ziegler. On safari to random jungle: a fast implementation of random forests for high-dimensional data. *Bioinformatics*, 26(14):1752–1758, 2010.

E. Scornet, G. Biau, and J.-P. Vert. Consistency of random forests. *arXiv preprint arXiv:1405.2881*, 2014.

R. Sedgewick and P. Flajolet. *An Introduction to the Analysis of Algorithms*. Pearson Education, 2013.

R. Sedgewick and K. Wayne. *Algorithms*. Pearson Education, 2011.

C. E. Shannon and W. Weaver. *The mathematical theory of communication*. The University of Illinois Press, 1949.

T. Sharp. Implementing decision trees and forests on a gpu. In *Computer Vision–ECCV 2008*, pages 595–608. Springer, 2008.

J. A. Sonquist. *Multivariate model building: The validation of a search strategy*. Survey Research Center, University of Michigan, 1970.

J. A. Sonquist, E. L. Baker, and J. N. Morgan. *Searching for structure: An approach to analysis of substantial bodies of micro-data and documentation for a computer program*. Survey Research Center, University of Michigan Ann Arbor, MI, 1974.

A. Srivastava, E.-H. Han, V. Kumar, and V. Singh. *Parallel formulations of decision-tree classification algorithms*. Springer, 2002.

D. J. Stekhoven and P. Bühlmann. Missforest - non-parametric missing value imputation for mixed-type data. *Bioinformatics*, 28(1):112–118, 2012.

M. Stone. Cross-validation: a review. *Statistics: A Journal of Theoretical and Applied Statistics*, 9(1):127–139, 1978.

C. Strobl, A.-L. Boulesteix, and T. Augustin. Unbiased split selection for classification trees based on the gini index. *Computational Statistics & Data Analysis*, 52(1):483–501, 2007a.

C. Strobl, A.-L. Boulesteix, A. Zeileis, and T. Hothorn. Bias in random forest variable importance measures: Illustrations, sources and a solution. *BMC bioinformatics*, 8(1):25, 2007b.

209

JR Quinlan. C4.5: programas para aprendizado de máquina, volume 1. Morgan kaufmann, 1993. J. J. Rodriguez, L. I. Kuncheva e C. J. Alonso. Floresta de rotação: um novo método de conjunto de classificadores. Análise de padrões e inteligência de máquina, Transações IEEE em, 28(10):1619–1630, 2006. B. Scholkopf e AJ Smola. Aprendendo com kernels: suporte a máquinas de vetores, regularização, otimização e muito mais. Imprensa do MIT, 2001. DF Schwarz, IR König e A. Ziegler. No safari para a selva aleatória: uma implementação rápida de florestas aleatórias para dados de alta dimensão. Bioinformática, 26(14):1752–1758, 2010. E. Scornet, G. Biau e J.-P. Verde. Consistência de florestas aleatórias. Pré-impressão do arXiv arXiv:1405.2881, 2014. R. Sedgewick e P. Flajolet. Uma introdução à análise de algoritmos. Pearson Education, 2013. R. Sedgewick e K. Wayne. Algoritmos. Pearson Education, 2011. CE Shannon e W. Weaver. A teoria matemática da comunicação. Imprensa da Universidade de Illinois, 1949. T. Sharp. Implementando árvores de decisão e florestas em uma GPU. In Computer Vision–ECCV 2008, páginas 595–608. Springer, 2008. JA Sonquist. Construção de modelo multivariado: A validação de uma estratégia de busca. Centro de Pesquisa de Pesquisa, Universidade de Michigan, 1970. JA Sonquist, EL Baker e JN Morgan. Procurando estrutura: Uma abordagem para análise de conjuntos substanciais de microdados e documentação para um programa de computador. Centro de Pesquisa de Pesquisa, Universidade de Michigan Ann Arbor, MI, 1974. A. Srivastava, E.-H. Han, V. Kumar e V. Singh. Formulações paralelas de algoritmos de classificação de árvores de decisão. Springer, 2002. DJ Stekhoven e P. Bühlmann. Missforest - imputação não paramétrica de valores faltantes para dados de tipo misto. Bioinformática, 28(1):112–118, 2012. M. Stone. Validação cruzada: uma revisão. Estatística: Um Jornal de Estatística Teórica e Aplicada, 9(1):127–139, 1978. C. Strobl, A.-L. Boulesteix e T. Augustin. Seleção dividida imparcial para árvores de classificação com base no índice de gini. Estatística Computacional e Análise de Dados, 52(1):483–501, 2007a. C. Strobl, A.-L. Boulesteix, A. Zeileis e T. Hothorn. Viés em medidas aleatórias de importância de variáveis florestais: ilustrações, fontes e uma solução. Bioinformática BMC, 8(1):25, 2007b.

C. Strobl, A.-L. Boulesteix, T. Kneib, T. Augustin, and A. Zeileis. Conditional variable importance for random forests. *BMC bioinformatics*, 9(1):307, 2008.

A. Sutera, A. Joly, F.-L. Vincent, A. Qiu, G. Louppe, D. Ernest, and P. Geurts. Simple connectome inference from partial correlation statistics in calcium imaging. 2014.

R. Tibshirani. *Bias, variance and prediction error for classification rules*. Citeseer, 1996.

G. Toussaint. Bibliography on estimation of misclassification. *Information Theory, IEEE Transactions on*, 20(4):472–479, 1974.

W. G. Touw, J. R. Bayjanov, L. Overmars, L. Backus, J. Boekhorst, M. Wels, and S. A. van Hijum. Data mining in the life sciences with random forest: a walk in the park or lost in the jungle? *Briefings in bioinformatics*, 14(3):315–326, 2013.

G. Tsoumakas and I. Katakis. Multi-label classification: An overview. *International Journal of Data Warehousing and Mining (IJDWM)*, 3(3): 1–13, 2007.

E. Tuv, A. Borisov, and K. Torkkola. Feature selection using ensemble based ranking against artificial contrasts. In *Neural Networks, 2006. IJCNN'06. International Joint Conference on*, pages 2181–2186. IEEE, 2006.

E. Tuv, A. Borisov, G. Runger, and K. Torkkola. Feature selection with ensembles, artificial variables, and redundancy elimination. *The Journal of Machine Learning Research*, 10:1341–1366, 2009.

S. Van der Walt, S. C. Colbert, and G. Varoquaux. The NumPy array: a structure for efficient numerical computation. *CiSE*, 13(2):22–30, 2011.

J. Vitek. How to grow Distributed Random Forests. 2013.

L. Wehenkel. On uncertainty measures used for decision tree induction. In *Information Processing and Management of Uncertainty in Knowledge-Based Systems*, 1996.

L. Wehenkel. Discretization of continuous attributes for supervised learning. variance evaluation and variance reduction. 1997.

L. A. Wehenkel. *Automatic learning techniques in power systems*. Number 429. Springer, 1998.

A. P. White and W. Z. Liu. Technical note: Bias in information-based measures in decision tree induction. *Machine Learning*, 15(3):321–329, 1994.

210

C. Strobl, A.-L. Boulesteix, T. Kneib, T. Augustin e A. Zeileis. Importância da variável condicional para florestas aleatórias. Bioinformática BMC, 9(1):307, 2008. A. Sutera, A. Joly, F.-L. Vincent, A. Qiu, G. Louppe, D. Ernest e P. Geurts. Inferência simples do conectoma a partir de estatísticas de correlação parcial em imagens de cálcio. 2014. R. Tibshirani. Viés, variância e erro de predição para regras de classificação. Citeseer, 1996. G. Toussaint. Bibliografia sobre estimativa de classificação incorreta. Teoria da Informação, IEEE Transactions on, 20(4):472–479, 1974. W. G. Touw, J. R. Bayjanov, L. Overmars, L. Backus, J. Boekhorst, M. Wels e S. A. van Hijum. Mineração de dados nas ciências biológicas com floresta aleatória: um passeio no parque ou perdido na selva? Briefings in bioinformatics, 14(3):315–326, 2013. G. Tsoumakas e I. Katakis. Classificação multi-rótulo: uma visão geral. Jornal Internacional de Armazenamento e Mineração de Dados (IJDWM), 3(3): 1–13, 2007. E. Tuv, A. Borisov e K. Torkkola. Seleção de recursos usando classificação baseada em conjunto contra contrastes artificiais. Em Redes Neurais, 2006. IJCNN'06. Conferência Conjunta Internacional sobre, páginas 2181–2186. IEEE, 2006. E. Tuv, A. Borisov, G. Runger e K. Torkkola. Seleção de recursos com conjuntos, variáveis artificiais e eliminação de redundância. The Journal of Machine Learning Research, 10:1341–1366, 2009. S. Van der Walt, SC Colbert e G. Varoquaux. O array NumPy: uma estrutura para computação numérica eficiente. CiSE, 13(2):22–30, 2011. J. Vitek. Como cultivar florestas aleatórias distribuídas. 2013. L. Wehenkel. Sobre medidas de incerteza utilizadas para indução de árvores de decisão. Em Processamento de Informação e Gestão de Incerteza em Sistemas Baseados em Conhecimento, 1996. L. Wehenkel. Discretização de atributos contínuos para aprendizagem supervisionada. avaliação de variância e redução de variância. 1997. LA Wehenkel. Técnicas de aprendizagem automática em sistemas de potência. Número 429. Springer, 1998. A. P. White e W. Z. Liu. Nota técnica: Viés nas medidas baseadas em informação na indução de árvores de decisão. Aprendizado de Máquina, 15(3):321–329, 1994.

D. H. Wolpert and W. G. Macready. An efficient method to estimate bagging's generalization error. *Machine Learning*, 35(1):41–55, 1999.

Y. Yuan and M. J. Shaw. Induction of fuzzy decision trees. *Fuzzy Sets and systems*, 69(2):125–139, 1995.

D. Zhang, M. M. Islam, and G. Lu. A review on automatic image annotation techniques. *Pattern Recognition*, 45(1):346–362, 2012.

G. Zhao. *A new perspective on classification*. PhD thesis, Utah State University, Department of Mathematics and Statistics, 2000.

Z.-H. Zhou. *Ensemble methods: foundations and algorithms*. CRC Press, 2012.

D. A. Zighed and R. Rakotomalala. *Graphes d'induction: apprentissage et data mining*. Hermes Paris, 2000.

M. Zinkevich, M. Weimer, A. Smola, and L. Li. Parallelized stochastic gradient descent. In J. Lafferty, C. K. I. Williams, J. Shawe-Taylor, R. Zemel, and A. Culotta, editors, *Advances in Neural Information Processing Systems 23*, pages 2595–2603, 2010.

211

DH Wolpert e WG Macready. Um método eficiente para estimar o erro de generalização do ensacamento. Aprendizado de Máquina, 35(1):41–55, 1999. Y. Yuan e MJ Shaw. Indução de árvores de decisão difusas. Conjuntos Fuzzy e sistemas, 69(2):125–139, 1995. D. Zhang, M. M. Islam e G. Lu. Uma revisão sobre técnicas automáticas de anotação de imagens. Reconhecimento de padrões, 45(1):346–362, 2012. G. Zhao. Uma nova perspectiva sobre classificação. Tese de doutorado, Utah State University, Departamento de Matemática e Estatística, 2000. Z.-H. Zhou. Métodos ensemble: fundamentos e algoritmos. CRC Press, 2012. DA Zighed e R. Rakotomalala. Gráficos de indução: aprendizagem e mineração de dados. Hermes Paris, 2000. M. Zinkevich, M. Weimer, A. Smola e L. Li. Descida gradiente estocástica paralelizada. Em J. Lafferty, CKI Williams, J. Shawe-Taylor, R. Zemel e A. Culotta, editores, Advances in Neural Information Processing Systems 23, páginas 2595–2603, 2010.